

TECHNISCHE UNIVERSITÄT DRESDEN

FAKULTÄT INFORMATIK
INSTITUT FÜR SYSTEMARCHITEKTUR
PROFESSUR FÜR DATENBANKEN
PROF. DR.-ING. WOLFGANG LEHNER
PROF. APL. DR.-ING. DIRK HABICH

Diplomarbeit

zur Erlangung des akademischen Grades
Diplom-Informatiker

Aktive cache-bewusste Hardware-Adaption: Eine Cache-Engine für Trie-basierte Indexstrukturen

Benjamin-Elias Probst
(Geboren am 11. April 1996 in Potsdam)

Hochschullehrer: apl. Prof. Dr.-Ing. Dirk Habich
Betreuer: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, 26. September 2026

Aufgabenstellung

Aktive cache-bewusste Hardware-Adaption:

Eine Cache-Engine für Trie-basierte Indexstrukturen

Kontext und Motivation

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanksysteme. Die Professur für Datenbanken an der TU Dresden adressiert die hardware-nahe Optimierung solcher Systeme. Seit der Einführung des Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Techniken und verwandter Algorithmenfamilien vorgeschlagen — von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation —, doch nutzt fast jede Veröffentlichung eigene Datenstrukturen, verschiedene Lastprofile und inkonsistente Messmethodik. Ergebnisse sind dadurch über Veröffentlichungen hinweg schwer vergleichbar. Zuletzt sind die Gewichtungen über Verbesserungen einzelner Algorithmus-Bestandteile derzeit nicht trennbar.

Problemstellung

Suchverfahren — von ausgewogenen Suchbäumen (B-/B⁺-Bäume) über Tries und Radix-Bäume bis zu spezialisierten, eigenentwickelten Konstrukten — spannen einen großen, aber konzeptionell einheitlichen, *Entwurfsraum* (*design space*) auf: Bereits eine zwei- bzw. dreikomponentige Datenstruktur erlaubt astronomisch viele Entwürfe, und ein konkreter Algorithmus ist ein einzelner Punkt in diesem Raum, beschrieben durch viele einzelne Entwurfsentscheidungen — wie Knoten, Verweise und Daten repräsentiert, durchlaufen, im Speicher angeordnet, und alloziert werden [IZA⁺18, IZH⁺18]. Cache-line-bewusstes Verhalten entsteht dabei nicht an einer einzelnen dieser Stellen, sondern an *vielen verschiedenen strukturellen Stellen* eines Algorithmus bzw. dessen Speicherverwaltung und aus dem Zusammenspiel multipler konzeptioneller Schichten eines Algorithmus. Der Stand der Technik umfasst zahlreiche ausgereifte Verfahren, doch optimiert jedes nur einzelne dieser Stellen und belegt seinen Nutzen in einer eigenen Messumgebung, was den Fortschritt über die Verfahren hinweg schwer vergleichbar macht. Es fehlt ein gemeinsames, generisches Konzept und **Prinzip**, das cache-line-bewusstes Verhalten, neben anderen Messschwerpunkten, an den verschiedenen Stellen eines Suchalgorithmus — austauschbar, systematisch und fair — messen und auswerten kann.

Zielsetzung

Ziel der Arbeit ist es, ein solches Prinzip (ein Rahmenwerk) zu finden, zu entwerfen und prototypisch als Experimentier-Framework umzusetzen, das mit Konzentrationspunkt auf eine dynamische Zusammensetzung der verteilten Algorithmus-Konzepte: (i) cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegt, (ii) Algorithmen des Standes der Technik als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert, (iii) cache-line-bewusstes Verhalten sowohl an den einzelnen direkt und indirekt betroffenen Stellen als auch für den gesamten Algorithmus auf gemeinsamer CPU-only-Basis misst und auswertet und (iv) auf Basis der feingliedrigen Messergebnisse den besten Gesamtalgorithmus als eigenständige Binary hinter einem einheitlichen Suchalgorithmus-Interface ausliefert. Als Prüfling für die testweise Eingliederung einer neuen Technologie in den Stand der Technik trägt die Arbeit PRT-ART bei (einen Probst-Redirect-Tree-/ART-Hybriden), der als konzeptionell neuartiger Suchalgorithmus untersucht wird, allerdings nur einige neue Aspekte innerhalb der Klassifizierung von Suchalgorithmen liefert und keinen vollständigen Algorithmus.

Die hier geforderte Trennbarkeit der einzelnen Algorithmus-Bestandteile ist in der vorliegenden Arbeit konstruktiv herzustellen und messbar zu machen.

Teilaufgaben

1. Systematische Erfassung der allgemeinen Entwurfsbestandteile von Suchalgorithmen — insbesondere der cache-bewussten — und Identifikation der Stellen, an denen cache-line-bewusste und suchalgorithmus-spezifische Entscheidungen wirksam werden. Dazu zählt die systematische Untergliederung der Entwurfsbestandteile in Entwurfsebenen einer verallgemeinerten Algorithmus-Kategorisierung.

-
2. Rekonstruktion von Entwürfen des Stands der Technik — möglichst aus dem Originalcode der jeweiligen Veröffentlichung und Original Compiler Binary linking — (ART, HOT, Masstree, CoCotrie, START, B²-Tree, Wormhole, SuRF sowie weitere Verfahren) als explizite, vergleichbare Konfigurationen.
 3. Entwurf und Implementierung von PRT-ART als Prüfling mit mehreren eigenen Entwurfsbestandteilen, der gegen den erfassten Stand der Technik antritt.
 4. Erweiterung des Gesamtablaufs zu einer durchgehend deklarativ gesteuerten Kette: Diese beginnt mit einer XML-Beschreibung des Experiments, aus der für das betreffende System ein automatisierter, plattformbezogener System-Bau abgeleitet wird, der seinerseits alle erdenklichen Binary-Permutationen der Entwurfsbestandteile erzeugt. Deren Messergebnisse werden auf die beste gemessene System-Binary und System-Binary-Rekombination hin ausgewertet; eine finale Messung der einen Binary bzw. hybrider Binary-Rekombinationen erzeugt eine Auswertung über CSV-/xlsx-Dateien, die abschließend in LaTeX und Diagramme übersetzt wird. Dass deklarative Konfigurationsbeschreibungen den automatisierten Aufbau modularer Systemvarianten steuern, ist ein etabliertes Konstrukt [CE00, ABKS13].
 5. Aufbau einer reproduzierbaren Mess-Pipeline (Binary → CSV → LaTeX → Diagramm) mit profilbewusstem Workload-Routing aller im Stand der Technik gefundenen Workload-Frameworks über *alle* verfügbaren Workloads und Hardware-Counter-Erfassung.
 6. Systematische, möglichst vollständige Messung über die Konfigurationen in drei verpflichtenden Messreihen — (A) Prüfling gegen den Stand der Technik, (B) systematische Variation der Entwurfsentscheidungen, (C) Merge/Regression alter gegen neue Varianten —, wobei jede Reihe in drei Granularitäten erhoben wird: *Micro-Benchmarking* (jeder einzelne Entwurfsbestandteil wird — innerhalb der Suchalgorithmus-Hülle wie der Algorithmus-Kategorie-Hülle — in jeder gemeinsamen Interface-Funktion seiner Kategorie isoliert gegen Lastprofile gemessen; die Algorithmen der Kategorie werden je Interface-Funktion untereinander vergleichend ausgewertet, unter Einhaltung latenzbewusster Best Practices der Datenbankforschung), *Makro-Benchmarking* (die einzelnen Interface-Operationen des Suchalgorithmus als Ganzes — z. B. Einfügen, Punktsuche, Löschen — werden über die Interface-Funktionen der Algorithmus-Kategorie gegen Beispiellasten bzw. Test-Teilbereiche der Algorithmus-Bestandteile gemessen) und *Gesamt-Benchmarking* (etablierte YCSB-ähnliche Lastprofile und vergleichbare Frameworks über jede Rekombination der Entwurfsbestandteile).
 7. Auswertung, an welchen Stellen und in welchen Kombinationen cache-line-bewusste Entscheidungen für welche datentyp-spezifischen Lastmuster vorteilhaft oder nachteilig sind — ermittelt aus der Rangbildung über die drei Benchmarking-Granularitäten und aufbereitet als auswertbare Diagramme und Tabellen.
 8. Automatisierung der Systemerkennung und Binary-Auswahl des optimalsten cache-line aware Algorithmus, anhand von empirisch belegten Messwerten, zusammen mit einer anhand der Messwerte dynamisch gesteuerten cache-line aware Anpassung des Algorithmus-Verhaltens als statische heuristische System-Richtlinie für das Algorithmusverhalten unter verschiedenen Lasten und Operationen. Diese sogenannte Extraktion soll als XML-Lastprofil-Ergebnis für die testende Umgebung für einen Gesamtalgorithmus unter Referenz verschiedener angewendeter Lastprofile abgelegt werden, sie ist für jeden Architekturfokus wiederverwendbar, um automatisch Heuristiken zu erstellen.

Methodik und Evaluation

Die Evaluation umfasst neben dem CPU-only-Kern auch SIMD- und nebenläufige Multi-Threading-Pfade und vergleicht die Konfigurationen gegen etablierte Baselines, darunter eine nicht-baumartige

Hash-Tabelle (`flat_hash_map`) als Gegenprobe innerhalb derselben Gattung. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik — etwa `url`, `dna`, `protein`, `xml`, `tpcds-id` und `trec-terms` — unter variierten Datenvolumina, Keylängen, Präfixüberlappungen, lokaler Dichte, Skew, Treffer-/Fehlerquoten und Value-Größen. Gemessen werden Exact-/Prefix-Lookup, Prefix-Enumeration, Einfügen/Aktualisieren/Löschen unter allen Varianten der C++ standard-Interfaces bekannter `std::map` und `std::vector` Container-Hüllen, p50/p95/p99-Latenz, Durchsatz, Bytes pro Key sowie Hardwarezähler (Cycles, Instructions, L1/L2/LLC- und dTLB-Misses, Branch-Mispredictions). Zunächst wird das Single-Thread-Verhalten untersucht, danach folgen leseparallele Multi-Thread-Szenarien.

Faire Vergleichbarkeit wird durch feste Regeln gesichert: ein gemeinsamer Minimalmodus (Common-Denominator) wird vom PRT-ART-Native-Modus getrennt, alle Verfahren erhalten identische Schlüsselräume und Seeds, fehlende Fähigkeiten werden als N/A ausgewiesen statt verdeckt emuliert, und Compiler, Flags, ISA-Pfad, Allokator sowie Commit-Hash jeder Baseline werden protokolliert. Für die Reproduzierbarkeit wird jede Gesamt-Algorithmus-Konfiguration in mehreren getrennten Läufen vermessen, deren Streuung Teil der Aussage bleibt (Perzentile über HDR-Histogramme statt Mittelung); jeder Datensatz trägt eine Akte aus Quelle, Prüfsumme und Seed-Regel, um den Originalcode zweifelsfrei nachzuweisen.

Umfangsabgrenzung

Der Kernbeitrag ist CPU-fokussiert mit SIMD-Erweiterungen und Multi-Threading, beschränkt sich dabei aber auf leseparallele Verfahren mit serialisiertem Schreiben (multi-read, single-write). Die GPU liegt vorerst außerhalb des Umfangs (zukünftig erweiterbar); volle Schreibparallelität und eine Engine-/Optimierer-Integration sind optionale Erweiterungen. Alle Erweiterungen der Architekturen sind ihrerseits dynamische Erweiterungen gegen den CPU-Standard-Lauf und führen zu tieferen Mess-Permutationen mit multipler Laufzeit. Die systemoptimierte Binary lässt sich dabei wahlweise mit oder ohne Mess-Observer übersetzen — als Debug-, Mess- oder Release-Variante.

Selbstständigkeitserklärung

Hiermit erkläre ich, Benjamin-Elias Probst (Mat.-Nr. 4510512, geboren am 11. April 1996 in Potsdam), dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

*Aktive cache-bewusste Hardware-Adaption:
Eine Cache-Engine für Trie-basierte Indexstrukturen*

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 26. September 2026

Benjamin-Elias Probst
Mat.-Nr. 4510512

Kurzfassung

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanken. Seit dem Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Optimierungen vorgeschlagen—von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation—, doch wurden sie nie in einem einzigen, fairen Messrahmen kombiniert und verglichen: Jede Veröffentlichung nutzt eigene Datenstrukturen, Lastprofile und Messmethodik, wodurch Ergebnisse über Veröffentlichungen hinweg schwer vergleichbar sind. Diese Arbeit stellt ein Cache-Engine-Framework vor, dessen Module auf Binärebene durch eine einzige, ABI-stabile C-Schnittstelle getrennt sind (je Modul genau ein exportiertes Lebewesen); dahinter liegt eine dreischichtige Abstraktion innerhalb der erzeugten Tier-Binary (CacheEngine → ExecutionEngine → SearchEngine), die die Such-API bereitstellt und vom Anatomie-Modell (Gattung, Lebewesen, Organe) abgegrenzt bleibt. Das Framework zerlegt cache-bewusste Suchalgorithmen mithilfe eines Planers und eines CacheEngineBuilder in austauschbare, verteilte (über den Code gestreute) Entwurfsentscheidungen eines großen Entwurfsraums (*design space*) [IZA⁺18, IZH⁺18] und setzt sie durch dynamische Zusammensetzung mittels Compile-Zeit-Permutation (keine Laufzeit-Adaption) wieder zusammen. Strukturell verschiedene Verfahren—von baumartigen Indizes (Tries, B⁺-Bäume) bis zu nicht-baumartigen Hash-Tabellen—werden als Punkte desselben Entwurfsraums hinter einem einheitlichen Interface ausgedrückt; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Entwurfsentscheidungen. Algorithmen des Standes der Technik werden als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert und auf drei Granularitäten gemessen—einzelne Entwurfsentscheidungen, einzelne Interface-Operationen und ganze Algorithmen—unter bekannten Workload-Framework-Lastprofilen (YCSB und weitere, von diesem ein Framework integrierte Frameworks), wobei jedes Modul wahlweise mit oder ohne Mess-Observer für Debug, Messung oder Release gebaut werden kann. Als Prüfling trägt zu dieser Arbeit der PRT-ART bei, ein Probst-Redirect-Tree-/ART-Hybrid mit acht eigenen Bausteinschichten; ein Mess-Treiber mit profilbewusstem Workload-Routing bewertet ihn unter drei verpflichtenden Messreihen gegen dreißig rekonstruierte Stand-der-Technik-Verfahren (acht etablierte Referenzverfahren mit Bausteinen und weitere); Fernziel ist die Auslieferung der besten Rekombination als eigenständige Binary.

Inhaltsverzeichnis

1	Einleitung	5
1.1	Motivation	5
1.2	Problemstellung	6
1.3	Forschungsfragen	6
1.4	Zielsetzung und Beiträge	9
1.5	Aufbau der Arbeit	12
2	Suchbäume und Grundlagen	13
2.1	Konzept und Überblick über den Stand der Technik	13
2.2	Hardware: historische Anpassung an die Suchleistung	16
2.2.1	Cache-Organisation: Assoziativität, Fehlzugriffs-Klassen und Parallelität	16
2.2.2	Cache-line-orientierte Layout-Forschung	17
2.2.3	Prefetching	18
2.2.4	Hauptspeicher und Adressübersetzung: DRAM, NUMA und TLB	18
2.2.5	Schreibpfad, Ersetzung und Kohärenz	19
2.2.6	Cache-bewusste und cache-oblivious Datenstrukturen	20
2.3	Workloads und Benchmark-Konventionen	21
2.4	Software-Mittel und Zusammenführung zur Abstraktion	22
2.5	Heuristiken als Entscheidungsregeln	23
3	Konzepte eines cache-bewussten Mess-Systems	25
3.1	Problemstellung und Anforderungen an ein Mess-System	25
3.2	Stand der Technik als Achsen-Instanzen	27
3.3	Lösungskonzept: das Anatomie-Modell	38
3.3.1	Achsen-Komposition und die Lebewesen-Metapher	38
3.3.2	Das Drei-Ebenen-Modell und die vier Konzepte	41
3.3.3	Entwurfsmuster und Übersetzungszeit-Muster	44
3.3.4	Fünf Interface-Schichten: E4 bis E0	47
3.4	Das Mess-System	49
3.4.1	Die Dreiteilung: Mess-, System- und Organ-Achsen	51
3.4.2	Komplex-Achsen: die Ziel-ISA als feste Rekombination	54
3.4.3	Hardware-Erkennung als zweigeteilte Mess-Achse	55
3.4.4	Identitäts-Stempel: eine Zeile je Realm	56
3.4.5	Das Lager: Bestände unter Stempel-Schlüsseln	57
3.4.6	Subsysteme und die Kette von der XML zur Tier-Binary	59
3.5	PRT-ART als Demonstrator	62

3.6	Schärfung durch Heuristiken	63
3.6.1	Heuristik-Schleife und gelernte Lastprofile	64
3.6.2	Die vier Betriebsmodi des Builders	65
3.6.3	Messkurven-Typsystem, Filterkette und Messfehler-Erkennung	67
4	Implementierung und konkrete Algorithmusbestandteile	71
4.1	Vier-Repository-Architektur	71
4.2	PRT-ART im Detail	74
4.3	Achsen-Implementierung und Mess-System-Ebenen	76
4.3.1	Die eine Architektur im Code	80
4.3.2	Vom Profil zur Tier-Binary: Planer, Builder und Mess-Pipeline	82
4.4	Qualitätssicherung: Contract-Tests je Schicht	87
4.5	Realm-Wurzeln im Code: System- und Mess-Achsen	89
4.5.1	Muster-Register, Hybrid-Stufe und Lager	97
5	Evaluation	101
5.1	Methodik	102
5.1.1	Hypothesen und Messgrößen	102
5.1.2	Vollautomatische Experiment-Evaluation aus der XML	103
5.1.3	Workloads und Datensätze	104
5.1.4	Plattformen, Hersteller-Pfade und Regimes	104
5.1.5	Mess-Ebenen, Kollektoren und Zähler	105
5.1.6	Bau-Umfang und Optimierungsstufen	107
5.1.7	Fairness und Reproduzierbarkeit	108
5.1.8	Der heuristische Gegenbeweis als vierte Dimension	110
5.2	Ergebnisse und Analyse	112
5.2.1	Identität der Ergebnisse: Stempel, Lager, Mappe	112
5.2.2	Datenfluss von der Messung zum Manuskript	113
5.2.3	Messreihen und Verbund-Strategien	114
5.2.4	Vorliegende Messungen	115
5.2.5	Achsen-Sensitivitätsanalyse	115
5.2.6	Gültigkeit und Grenzen	116
5.2.7	Zusammenfassung der Evaluation	116
6	Fazit und Ausblick	119
6.1	Beantwortung der Forschungsfragen	119
6.2	Limitierungen	128
6.3	Ausblick	130
	Literaturverzeichnis	133
	Abbildungsverzeichnis	155
	Tabellenverzeichnis	163

A	Vollständige Messreihen	167
A.1	Kartesische Smoke-Messreihe (43 Permutationen, DLL-Messung)	169
A.2	Bias-Bruch-Matrix	170
A.3	Latenz-Surfaces je Schnittstellen-Funktion	173
A.4	Dreidimensionale Rohdaten-Ansicht	173
A.5	Arbeitsmengen-Sweep	175
A.6	Per-Achsen-Latenz-Attribution	175
A.6.1	Per-Achsen-Observer-Detail	176
A.7	Latenz-Verteilung	178
A.8	Referenzbezogene Auswertung	179
A.8.1	Verhältnis-Matrizen	180
A.8.2	Normalisierte Balken	182
A.9	Achsen-Austauschbarkeit	183
A.10	Limitierungen	184
B	Code-Struktur	187
C	Glossar	201
D	Bausteine-Matrix	215
D.1	T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)	216
D.2	T1 Cache-Speicher-Traversierung (Fanout-Lookup)	219
D.3	T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)	220
D.4	T3 Pfadkompression	221
D.5	T4 Knotentypen (Node-Type-Achse)	222
D.6	T5 Speicher-Layout (Memory Layout)	224
D.7	T6 Allokator-Achse (T6)	225
D.8	T7 Prefetch-Strategie	228
D.9	T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)	229
D.10	T9 Serialisierung (Kodierung und Kompression)	230
D.11	T10 Wert-Handle (Value-Handle)	231
D.12	T11 Index-Organisation	232
D.13	T12 I/O-Dispatch-Achse	234
D.14	T13 Migrations-Strategie-Achse	235
D.15	T14 Filter-Achse	237
D.16	T15 Pufferung Q1 (Operations-Puffer)	238
D.17	T16 Pufferung Q2 (Flush-Policy)	241
D.18	T17 Persistenz-Ziel (Persistence-Target)	242
D.19	Organ-Meta-Meta-Achse: Festplatten-E/A (disk_io)	242
D.20	System-Sphäre: Ziel-ISA (target_isa)	243
D.21	System-Sphäre: Betriebssystem (operating_system)	246
D.22	System-Sphäre: Erweiterungs-Hardware (external_utils)	247

D.23 System-Sphäre: Äußere Komplex-Klammer (<code>build_target_complex</code>) und Bau-Werkzeugkette (<code>build_toolchain</code>)	249
D.24 Mess-Sphäre: Mess-Kategorien (<code>measurement_category</code>)	251
D.25 Mess-Sphäre: Mess-Tooling/Collector (<code>collector</code>)	252
D.26 Mess-Sphäre: Telemetrie-Bausteine (konservierter Katalog)	252
D.27 Mess-Sphäre: Last-Rahmenwerk (<code>load_framework</code>) — Meta-Meta-Achse	254
D.28 A-Korpus Allokator-Korpus (A01–A23 sowie A24)	255
D.29 Achsen-Inventar aus den generierten Registries	258
D.30 Konservierte Entwürfe	258
D.30.1 Build-SIMD: Vektor-/Accelerator-Erweiterungs-Achse	258
D.30.2 Build-PG Seitentyp-Achse (PAGE-TYPE)	260
D.30.3 Build-HW — Plattform-Hardware-Strategie	261
E Architekturentscheidungen	263
F Vergleichsinterfaces: <code>std::map</code> und <code>std::vector</code>	273
F.1 <code>std::map</code> (geordnet-assoziativ)	274
F.2 <code>std::vector</code> (Sequenz)	277
F.3 Zusammengesetzte Funktionen und Konformitäts-Teilmenge	279

1 Einleitung

1.1 Motivation

Suchbäume und Indexstrukturen bilden das Fundament moderner datenintensiver Systeme — von In-Memory-Datenbanken über Dateisysteme bis zur Sprachverarbeitung. Zugleich werden Prozessoren von der Speicher-Hierarchie dominiert: nicht die Zahl der Operationen, sondern ihr Cache-Verhalten bestimmt die Laufzeit [HP19, Dre07, WM95]. Aus diesem Spannungsfeld erwächst die Leitfrage der Arbeit. Im Zentrum stehen dabei trie-basierte Indizes und cache-line-optimierte Verarbeitung. Seit dem Adaptive Radix Tree (ART) [LKN13] von 2013 hat sich die Forschung ausdifferenziert: Trie-Höhen-Optimierung (HOT [BZP⁺18]), hybride B⁺-Strukturen (Masstree [MKM12], B²-Baum [SSL⁺21]), hash-/trie-/B⁺-hybride geordnete Indizes (Wormhole [WNJ19]), succinct-Repräsentationen (CoCo-Trie [BFTV24], SuRF [ZLL⁺18]) und selbstoptimierende Ansätze (START [FJKN20]).

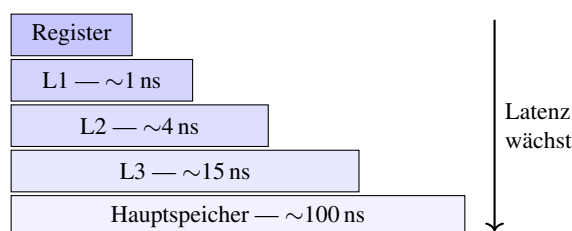


Abbildung 1.1: Die „Cache-Wand“: typische Größenordnungen der Zugriffslatenz je Ebene der Speicher-Hierarchie (Literaturwerte nach [HP19, Dre07]); über die Hierarchie wächst die Latenz um rund zwei Größenordnungen.

Parallel dazu ist ein eigener Forschungskorpus zu Cache-Bewusstheit, Prefetching und Allokationsstrategien entstanden. Heuristiken wie Cache-Sensitive Search Trees [RR99, RR00], cache-oblivious Layouts [FLPR99], Prefetching-B⁺-Bäume [CGM01] und hierarchisches Prefetching [ZGH⁺25] adressieren jeweils einen separat betrachteten Aspekt der Suchalgorithmus-Architektur, wobei der Fokus meist auf empirisch gefundenen Verbesserungen und Messungen liegt. Weil so vieles auf cache-line-optimiertem Verhalten beruht, liegt eine Segmentierung der Suchalgorithmus-Bestandteile in *Achsen* nahe — eine rückwärtsgerichtete Abstraktion bekannter Bausteine in wiederverwendbare Module über die wichtigsten Arbeiten des Standes der Technik hinweg.

Diese Arbeit vollzieht diese Segmentierung mit einem Rahmenwerk, dessen Bausteine sich als austauschbare Entwurfsdimensionen eines großen Entwurfsraums (*design space*) auffassen lassen [IZA⁺18, IZH⁺18]. Strukturell verschiedene Suchverfahren — von baumartigen Indizes bis zu nicht-baumartigen Hash-Tabellen, samt Spezialeigenschaften wie Cache-Line-Bewusstheit — werden so als Punkte desselben Entwurfsraums vergleichbar; nicht-baumartige Verfahren belegen dabei eine eingeschränkte Teilmenge (*subset*) der Dimensionen über demselben Suchalgorithmus-Interface, als bildeten sie die einzige Verzweigung unterhalb der Wurzel. Als Prüfling für die Stand-der-Technik-Zugehörigkeit dient PRT-ART, ein Probst-Redirect-Tree-/ART-Hybrid und eine abstrakte, unvollständige Sammlung neuartiger

Suchalgorithmus-Bestandteile.

1.2 Problemstellung

Cache-Bewusstheit ist in heutigen Suchstrukturen entweder fest verdrahtet oder ganz ignoriert; vor allem aber lässt sich nicht *trennen*, welche einzelne Entwurfsentscheidung welchem Lastmuster nützt — das *Trennbarkeits-Problem*, der rote Faden dieser Arbeit.

Bestehende cache-bewusste Index-Entwürfe optimieren ein oder wenige Aspekte isoliert und berichten Ergebnisse, die sich über Veröffentlichungen hinweg schwer vergleichen lassen, da jede Arbeit eigene Datenstrukturen, Workloads und Mess-Methodiken verwendet. Die Gewichtungen der Verbesserungen einzelner Algorithmus-Bestandteile bleiben dabei untrennbar: Ein gemeldeter Leistungsgewinn lässt sich nicht eindeutig auf einen einzelnen Bestandteil (Seitentyp, Layout, Value-Ablage, Prefetching) zurückführen, weil die Bestandteile in den Original-Entwürfen verflochten bleiben. Es fehlt ein gemeinsamer Rahmen, der (i) solche Entwürfe in austauschbare Achsen zerlegt, (ii) publizierte Algorithmen als explizite Konfigurationen rekonstruiert und (iii) sowohl einzelne Achsen als auch ganze Algorithmen auf einer einheitlichen, fairen Basis vermessen kann.

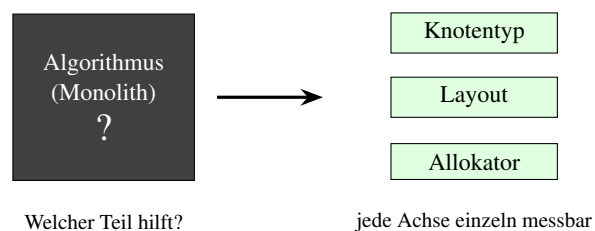


Abbildung 1.2: Das Trennbarkeits-Problem: als Monolith bleibt unklar, welche Entwurfsentscheidung wirkt; erst die Zerlegung in orthogonale Achsen macht jeden Beitrag isoliert messbar.

1.3 Forschungsfragen

Aus der Aufgabenstellung und dem Ziel der Cache-Line-Optimierung ergeben sich — aufgrund der Allgegenwärtigkeit des Einflusses der Cache-Line-Einstellungen — eine Hauptfrage (FF0) und vier messbare Forschungsfragen (FF1–FF4); ergänzende Aspekte sind ihnen als vertiefende *Teilfragen* zugeordnet. Quer zu allen fünf Fragen liegt eine Kategorisierungs-Frage, die diese Arbeit als Begriffsprägung mitführt: Schon die Aufgabenstellung unterscheidet das Gemessene (die Entwurfsbestandteile des Algorithmus), die Plattform, auf der gebaut und gemessen wird, und das Messen selbst. Ob sich die Achsen des Entwurfsraums entlang dieser drei Belange sauber in *Organ-Achsen* (die austauschbaren Entwurfsbestandteile des Algorithmus), *System-Achsen* (die Belange der Bau- und Ziel-Plattform) und *Mess-Achsen* (die Belange des Messens selbst) trennen lassen — und wie eine solche Trennung zu bewerkstelligen ist, falls sie in einem Entwurf noch nicht vorhanden ist — wird in dieser Arbeit als Frage ausgelegt; die Teilfragen sind darum entlang dieser drei *Achsen-Kategorien* benannt. Der Ist-Stand dazu ist eindeutig: die Arbeit legt die Dreiteilung als drei getrennte Achsen-Register¹ an — ein Organ-Achsen-Register, ein System-Achsen-Register und ein Mess-Achsen-Register — und prüft ihre Tragfähigkeit; die Antworten entwickeln die

¹Ein Achsen-Register ist das Verzeichnis der Achsen einer Kategorie samt der je Achse angebotenen Bausteine (im Code eine Registry).

Kapitel 3, 4 und 5. Jede Frage wird dabei an genau einer Stelle beantwortet: FF1 in Abschnitt 3.4.1 und Kapitel 4, FF2 in Abschnitt 3.4 und Kapitel 5 (FF2.d in Abschnitt 2.3 und Abschnitt 5.1.3), FF3 in Abschnitt 3.5 und Kapitel 5, FF4 in Abschnitt 4.5 und Abschnitt 4.3.2 (FF4.a); Kapitel 6 zieht die Bilanz je Haupt- und Teilfrage. Die Teilfragen tragen dazu die Kennungen FF0.E, FF1.a, FF1.b, FF2.a, FF2.b, FF2.c, FF2.d, FF3.a, FF3.b, FF4.a und FF4.E.

FF0 (Hauptfrage) Wie stark verbessern *aktive, messgetriebene*² cache-bewusste Entscheidungen — Seitentyp, Speicher-Layout, Value-Ablage und Prefetching — die Leistung trie-basierter Suchstrukturen gegenüber *passiven, statisch ausgelegten* Varianten auf unterschiedlichen CPU-Plattformen (Hybrid-CPU's, Sapphire Rapids)?³ *Erweiterung FF0.E (Achsen-Kategorien)*: Lässt sich ein gemessener Gewinn überhaupt eindeutig zuordnen — der Entwurfsentscheidung selbst (Organ), der Bau- und Ziel-Plattform (System) oder dem Mess-Aufbau (Mess) —, und welche Trennung der Achsen-Kategorien ist dafür nötig?

FF1 (Komposition) Wie lässt sich ein Suchalgorithmus-Entwurf kanonisch in orthogonale Achsen zerlegen und in einer Permutationsmatrix⁴ systematisieren, sodass die Stand-der-Technik-Verfahren möglichst aus ihrem Originalcode bias-frei als Profile rekonstruierbar und beliebig permutierbar werden? *Teilfrage FF1.a (Kategorien-Schnitt)*: Zerfällt die kanonische Zerlegung selbst entlang der drei Achsen-Kategorien — müssen Plattform- und Mess-Belange aus der Permutationsmatrix herausgehalten werden, damit die Zerlegung der Entwurfsbestandteile orthogonal bleibt —, und wie ist ein solcher Schnitt zu bewerkstelligen, falls er im Entwurf noch nicht angelegt ist? *Teilfrage FF1.b (Organ-Kopplungen und System-Variation)*: Welche wechselseitigen Abhängigkeiten zwischen Achsen (Knotentyp $\times$ SIMD $\times$ Pfadkompression)⁵ begrenzen die Isolation einzelner Achsen, und wie verhält sich dieselbe Organ-Belegung im Cache-Update- und Cache-Line-Füllverhalten (wie der Cache aktualisiert wird und wie vollständig die geladenen Cache-Lines genutzt werden), wenn allein die ISA-/SIMD-Achse (SSE2/AVX2/AVX-512, NEON/SVE2, RVV) variiert wird — soweit die jeweilige Zielplattform zum Bau freigegeben ist (Abschnitt 3.4.1)?

FF2 (Mess-Methodik) Wie muss eine reproduzierbare Mess-Pipeline mit Hardware-Zähler-Erhebung⁶ gestaltet sein, damit jede Achsen-Permutation fair (unter sonst gleichen Bedingungen, also gleicher Compiler- und Flag-Basis) in drei Messreihen — A (Prüfling vs. Stand der Technik), B (systematische Variation der Entwurfsentscheidungen) und C (Merge/Regression alt gegen neu) — über

²Messgetrieben heißt eine Entscheidung, die auf erhobenen Messwerten gründet und nicht auf festen Modellannahmen; statisch ausgelegt ist eine Variante, deren Entscheidung beim Entwurf fest vorgegeben wird.

³Vermessene Flotte dieser Arbeit sind die drei registrierten Maschinen-Signaturen: prod1 (AMD Zen 5, Ryzen 9 9950X3D), prod2 (Intel Core i9-12900K, Alder Lake — eine Hybrid-CPU mit P- und E-Kernen) und ein ODROID-Board mit Intel-Gracemont-Kernen. Die Server-Plattform Sapphire Rapids (ZIH-Knoten Barnard, Xeon 8470) ist in der Plattform-Erkennung angelegt, aber nicht Teil der Messreihen (n/a).

⁴Kanonisch heißt die Zerlegung, wenn jede Achse einen festen Platz in einer verbindlichen Reihenfolge hat. Die Permutationsmatrix ist die Tabelle dieser Achsen mit ihren zulässigen Belegungen; jede Wahl genau einer Belegung je Achse ist eine Achsen-Permutation, und ein Profil ist die deklarative Beschreibung (als XML) eines Entwurfs durch seine Belegung dieser Achsen. Bias-frei heißt: ohne Verzerrung durch eine Nachimplementierung oder durch ungleiche Messbedingungen.

⁵SIMD (*single instruction, multiple data*) bezeichnet Befehle, die eine Operation auf mehrere Datenelemente zugleich anwenden; die Pfadkompression fasst in einem Trie Knotenfolgen ohne Verzweigung zu einem Knoten zusammen; die ISA (*instruction set architecture*) ist der Befehlssatz der Ziel-CPU, und SSE2/AVX2/AVX-512, NEON/SVE2 und RVV sind die SIMD-Erweiterungen der Befehlssätze x86, ARM und RISC-V.

⁶Die Mess-Pipeline ist die Kette aus Bau der Binaries, Messlauf und Auswertung; Hardware-Zähler sind die Leistungszähler der CPU (*Performance Monitoring Counter*, PMC), die Ereignisse wie Cache-Fehlzugriffe je Lauf zählen.

drei Granularitäten (Mikro-, Makro- und Gesamt-Benchmarking: einzelne Bestandteile, Interface-Operationen, ganze Algorithmen) gemessen wird, und unter welchen plattform-kalibrierten Kriterien (Schwellen, die je Maschinen-Signatur aus Messläufen auf der jeweiligen Plattform bestimmt werden) gilt eine Plattform als erfolgreich verifiziert? *Teilfrage FF2.a (Mess-Achsen)*: Müssen die Mess-Instrumente selbst — Zeitnahme, Zähler-Erhebung, Last-Erzeugung — als eigene Achsen geführt werden, die stets präsent sind, ohne je zu verändern, was gebaut und gemessen wird, und wie ist diese Trennung zu bewerkstelligen, falls sie noch nicht vorhanden ist? *Teilfrage FF2.b (Organ-Achsen)*: Wie lassen sich widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine Cache-Line bei CSS/CSB⁺ [RR99, RR00] gegenüber sechzehn und mehr bei Hankins/Patel [HP03], dort Knoten von 512 Byte und größer bei 32-Byte-Cache-Lines, sowie mehrere, per Prefetch gemeinsam geladene Cache-Lines je Knoten bei den Prefetching-B⁺-Bäumen von Chen, Gibbons und Mowry [CGM01]) durch achsen-isolierte, unter sonst gleichen Bedingungen permutierte Messung bias-frei nachmessen, statt sie aus inkompatiblen Original-Setups zu übernehmen? *Teilfrage FF2.c (Erweiterbarkeit)*: Wie erweiterbar ist eine solche Mess-Pipeline — wie gliedern sich neue Prüflinge und ihre Entwurfsbestandteile ein, wie werden sie abstrahiert, und ordnen sie sich bestehenden Konzepten⁷ zu, begründen sie neue Konzepte oder eine Mischform; auf welcher konzeptionellen Ebene sollte dieser Schnitt verlaufen? Zentral ist dabei: Was ist überhaupt eine Erweiterung des Systems, und wie wirkt sich das Hinzufügen oder Wegschneiden einzelner Entwurfsbestandteile — etwa derjenigen, die aus einer einzelnen Veröffentlichung stammen — auf das erreichbare Maß an Cache-Line-Bewusstheit des Gesamtsystems aus? Braucht es dafür einen Betriebsmodus, der die Entwurfsbestandteile eines wählbaren Referenzentwurfs aus der Literatur als geschlossene Achsenmenge isoliert anbietet, um genau diese eine Veröffentlichung — bei weiterhin freistehenden Mess- und System-Achsen — originalgetreu durchzumessen? *Teilfrage FF2.d (Workload-Stand der Technik)*: Welche Workloads gehören gerade für Suchalgorithmen zum Stand der Technik?

FF3 (Prüfling-Vergleich) Gewinnt der Prüfling PRT-ART gegenüber den acht Rang-1-Entwürfen im Stand der Technik (*State of the Art*, kurz SOTA; ART, HOT, Masstree, CoCo-Trie, START, B²-Baum, Wormhole, SuRF)⁸ und der Rang-2/3-SOTA *nicht* durch asymptotische Neuheit, sondern durch bessere Mikroarchitektur-Eigenschaften⁹ — höhere Cache-Line-Auslastung, weniger LLC- und dTLB-Misses, geringere Branch-Kosten, kleinere heiße Suchpfade (die häufig durchlaufenen Knotenfolgen) — bei Punktsuche, Prefix-Lookup (Suche nach einem Schlüssel-Präfix) und Prefix-Enumeration (Aufzählung aller Schlüssel zu einem Präfix), gemessen unter den drei Pflicht-Messreihen A (Prüfling vs. Stand der Technik), B (systematische Variation der Entwurfsentscheidungen) und C (Merge/Regression alt gegen neu)? *Teilfrage FF3.a*

⁷Konzept heißt in dieser Arbeit die oberste Einteilung der Algorithmen nach ihrem nach außen sichtbaren Interface (etwa Map oder Container); jedes Konzept trägt seine eigenen Gattungen als Tier-Interfaces, deren Genera die Implementierungen sind.

⁸Die *Ränge* ordnen die Stand-der-Technik-Veröffentlichungen nach Zentralität für diese Arbeit: Rang-1 umfasst die zehn wichtigsten Kern-Arbeiten — darunter die acht eigenständigen Suchentwürfe sowie ARTSync (P08) [LABN16] und LOUDS (P09) [Jac89], die als Organe eingehen —, Rang-2 die cache-bewussten Layout- und B⁺-Vorarbeiten und Rang-3 die Prefetching-, Synchronisations- und TU-Dresden-Hardware-Arbeiten.

⁹Die Cache-Line-Auslastung ist der Anteil der genutzten Bytes einer geladenen Cache-Line; LLC ist der Last-Level-Cache, die letzte Cache-Stufe vor dem Hauptspeicher; der dTLB (*data translation lookaside buffer*) puffert Adressübersetzungen für Datenzugriffe; ein Miss ist ein Fehlzugriff, der die nächste, langsamere Stufe bemüht; Branch-Kosten sind die Kosten von Sprungbefehlen, vor allem falsch vorhergesagter Verzweigungen.

(*Organ-Achsen*): Welche lokale Seitendarstellung (8-/16-Bit-Punktseite — die Direktindex-Seiten Array_{256} und Array_{65535} — gegenüber der Kompaktseite — den Vektor-Seiten $\text{Vector}\langle u8, u8 \rangle$ und $\text{Vector}\langle u16, u16 \rangle$) ist je Verzweigungspunkt (je Knoten, an dem sich der Suchpfad verzweigt) am günstigsten, und nach welchen plattform-kalibrierten Umschaltregeln — einschließlich der Reihenfolge $\text{Array}_{256} \rightarrow \text{Array}_{65535} \rightarrow \text{Vector}\langle u8, u8 \rangle \rightarrow \text{Vector}\langle u16, u16 \rangle$ je Belegungsichte und Zugriffsart — soll adaptiv gewechselt werden? Der Prüfling schaltet derzeit nach festen Dichte-Schwellen von 25, 50 und 75 Prozent¹⁰ um (Abschnitt 3.5). *Teilfrage FF3.b (Compile-Time-Zwang)*: Erzwingen latenz-kritische Messungen, dass an sich zur Laufzeit wählbare Entscheidungen — etwa solche adaptiven Umschaltregeln — bereits zur Übersetzungszeit als Anforderungen fixiert werden, sodass sich jede Achse in eine übersetzungszeit-fixierte Haupt-Achse und laufzeit-variable Unter-Achsen gliedert — eine Zuspitzung, die FF4 aufnimmt?

FF4 (Compile-Time vs. Laufzeit) Welche Achsen-Eigenschaften müssen zur Übersetzungszeit fixiert werden, um plattform-optimierte, provenienz-nachweisbare¹¹ Binaries je Permutation zu erzeugen, und welche können bzw. müssen zur Laufzeit gewählt werden (bit-codierte Binary-Identität je Permutation, laufzeit-adaptive Layout-Wahl), ohne die wissenschaftliche Vergleichbarkeit (gleiche Compiler- und Flag-Basis) zu verletzen? Wird es durch die Messung möglich, ISA+OS-optimierte Binaries für eine Suchalgorithmus-Rekombination zu erzeugen, deren heuristische Optimierung perfekt auf ihre Laufzeitumgebung zugeschnitten ist? *Teilfrage FF4.a (deklarative Kette)*: Lässt sich der gesamte Ablauf — vom plattformbezogenen Bau aller Binary-Permutationen der Entwurfsbestandteile über den Messlauf und die Auswahl der besten gemessenen Binary oder Binary-Rekombination bis zur Auswertung in CSV-/xlsx-Dateien, LaTeX und Diagrammen — durchgehend aus einer XML-Beschreibung des Experiments erzeugen, sodass ein Experiment vollständig und reproduzierbar durch seine Deklaration bestimmt ist? *Erweiterung FF4.E (System-Achsen)*: Verläuft die Grenze zwischen Übersetzungszeit und Laufzeit entlang der Achsen-Kategorien — gehören insbesondere die Plattform-Eigenschaften (ISA, Betriebssystem, Erweiterungs-Hardware wie GPU-, FPGA- oder NPU-Beschleuniger) als System-Belange in die Übersetzungszeit-Identität der Binary — also in ihren Fingerprint, ohne die Permutations-Identität zu verändern —, während Mess-Wahlen (die Belegungen der Mess-Achsen) laufzeit-variabel bleiben dürfen?

1.4 Zielsetzung und Beiträge

Die Zielsetzung folgt der Aufgabenstellung in vier Schritten: cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegen, Stand-der-Technik-Entwürfe als explizite Konfigurationen rekonstruieren, cache-line-bewusstes Verhalten sowohl je Bestandteil als auch für den ganzen Algorithmus

¹⁰Punktseite und Kompaktseite sind die zwei Seitenformen des PRT-ART-B⁺-Knotens: Die Punktseite adressiert Einträge direkt über ein Feld mit 8- oder 16-Bit-Index, die Kompaktseite hält sortierte Schlüssel-Wert-Vektoren. Die Dichte-Schwellen sind die Umschaltregeln des Prüflings zwischen den Seitendarstellungen; die im Glossar genannte Grenze bei 50 Prozent ist die Seitentyp-Grenze zwischen Punkt- und Kompaktseite, keine weitere Umschaltregel.

¹¹Provenienz-nachweisbar heißt: Die Herkunft einer Binary — welche Achsen-Belegung, welche Plattform und welche Werkzeugkette sie hervorgebracht haben — ist an der Binary selbst nachprüfbar. Dazu trägt jede Binary zwei Identitäten: die Binary-Identität je Permutation, also ihre Belegung der Organ-Achsen (im Code die `binary_id`, ein Pfad aus einem Segment je Achse mit umkehrbar eindeutiger Zahlencodierung), und die Übersetzungszeit-Identität, die zusätzlich Bau-Plattform, Werkzeugkette, Bau-Steuergrößen und Quellstand umfasst und als Fingerprint — ein Hash über alle bau-bestimmenden Glieder — in die Binary einkompiliert wird; die erste heißt auch Permutations-Identität.

auf gemeinsamer CPU-only-Basis messen und auswerten sowie den besten Gesamtalgorithmus nach einer autonomen Heuristik-Profil-Auswertung und Heuristik-Funktions-Modellierung als eigenständige Binary ausliefern. Der Code der Diplomarbeit ist dabei der *formale Konsument und Treiber* der Cache-Engine: Er lädt den Prüfling PRT-ART, kompiliert ihn per Metaprogrammierung gegen die Kernbestandteile der Cache-Engine, liefert ABI-stabile *Lebewesen* (Suchalgorithmus-Instanzen) aus und treibt sie über das Prüf-Dock — das Mess-Interface des Konzepts (Kapitel 3) — zur Messung. Das zugrunde liegende Architekturmodell — die drei Interface-Ebenen (*Konzept* als äußere Mess-Surface, *Gattung* als Tier-Interface, im Code *AnatomyGenus*, und *Genus* als dessen Implementierung), die Achsen-Organen als eigene Klassifikation, die vier Subsysteme und die ABI-stabile Auslieferung der Suchalgorithmus-*Lebewesen* — entwickeln Kapitel 3 und Kapitel 4. Gemeint sind dabei die vier Träger-Subsysteme Mess-Treiber, *CacheEngineBuilder*, *CacheEngine* und Prüfling; davon zu unterscheiden sind die zwölf Stufen der Empfehlungs-Pipeline im Code. Das Konzept der Achsen-Architektur selbst ist aus einer Vorarbeit des Autors übernommen — dem Achsen-Konzept dieser Vorläufer-Implementierung *UltiHash* (zur Herkunft siehe Abschnitt 3.3.3) —, der Beitrag dieser Arbeit ist seine Übertragung auf cache-bewusste Suchstrukturen und seine mess-orientierte Realisierung. Die zentralen Beiträge sind:

- Eine ABI-stabile Modul-Schnittstelle: eine einheitliche C-Modulgrenze (`comdare_create_anatomy()` $\rightarrow$ `IAnatomyBase`) exportiert je Permutation genau ein Lebewesen; dahinter liegt eine dreischichtige Abstraktionskette — eine Vermittlungs-Fassade (`ICacheEngine`) über der Laufzeit-ABI-Schicht (`IAnatomyBase` $\rightarrow$ `IExecutionEngine`) über den Achsen-Organen. Die Abgrenzung des Suchmaschinen-Moduls ist konzeptionell festgelegt, als eigenes Modul aber nicht implementiert; sein Provider ist ausdrücklich zurückgestellt, und das Suchmaschinen-Modul ist als Topic-Namensraum innerhalb der Cache-Engine realisiert. Der Prüfling-Suchkern spezialisiert seine Hybrid-API variadisch nach Parameterzahl (1 Parameter $\Rightarrow$ `std::vector`-API; 2 Parameter $\Rightarrow$ `std::map`-API; $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>>`), wie es die Aufgabenstellung über alle Varianten der bekannten `std::map`- und `std::vector`-Container-Hüllen fordert; die Cache-Engine stellt dazu die Gattungs-Aritäten ihrer Gattungen bereit.
- Eine Achsen-Permutationsmatrix in dreigeteilter Ordnung: achtzehn Organ-Haupt-Achsen in fester Slot-Reihenfolge (T00–T17) bilden die Binary-Permutation; drei System-Haupt-Achsen — Ziel-ISA, Betriebssystem und Erweiterungs-Hardware — beschreiben die Bau- und Ziel-Plattform, wobei ihre Komplex-Klammer `build_target_complex` ausdrücklich keine vierte Haupt-Achse ist; und ein eigenständiger Mess-Achsen-Realm hält die Mess-Belange von beidem getrennt, denn weder System- noch Mess-Achsen gehen je in die Permutations-Identität der Binary ein. Eine optionale neunzehnte Organ-Achse für Festplatten-Ein-/Ausgabe, die nicht eine Belegung beschreibt, sondern beschreibt, wie andere Achsen beschrieben werden, tritt hinzu, sobald Festplatten-Ein-/Ausgabe in den Algorithmus-Entwurf eingebunden wird; sie stempelt im Klammer-Anhang der Organ-Zeile und bleibt außerhalb der Permutations-Identität. Umgesetzt ist die Matrix mit Boost.MP11-Compile-Time-Typlisten-Bausteinen — Laufzeit-Tags und `std::variant` sind in den Tier-Binaries als Grundsatz dieser Arbeit ausgeschlossen (`[[no-runtime-switch]]`) — samt Compile-Time-Fallback zwischen Prüfling-Stack und Stand-der-Technik-Stack, der sechzehn der achtzehn Achsen aus dem Cache-Engine-Rahmen übernimmt und zwei durch Prüfling-Slots ersetzt.

- Dreiunddreißig Stand-der-Technik-Profile als persistierte XML-Konfigurationen (P01–P33): 30 Vollprofile eigenständiger Suchentwürfe — 8 Rang-1 und 22 Rang-2/3 — sowie 3 abstrakte Profile (OLC, LOUDS, VAMPIR), die nur seziert als Organe bzw. Mess-Technologie eingehen. Der `ExperimentDriver` überführt sie automatisch in vorkompilierte C++-Module; der Profilsatz deckt damit alle 33 analysierten Veröffentlichungen ab. Der XML-Entwurf bildet mithilfe der Cache-Engine die einzelnen Algorithmus-Bestandteile zur Kompilierzeit tatsächlich ab und ist damit eine Compiler-Compiler-Bauanleitung in einem modellgetriebenen Optimierungs-Stack (*model-driven*) [Sch06]; dass deklarative Konfigurationsbeschreibungen den automatisierten Aufbau modularer Systemvarianten steuern, ist ein etabliertes Konstrukt [CE00, ABKS13].
- PRT-ART als Prüfling mit acht eigenen Bausteinschichten (4+1+2-Pool-Allokator, optimistisches Lock-Coupling mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, DensityTracker, Hypothesen-Metriken H1/H2/H3, Inline/External/ChainRef-ValueHandles, Signaling-Bit-Serialisierung). Im Mess-Apparat meldet der Prüfling vier Prüfling-Slots an — Seitentyp (`page_type`, in der Cache-Engine die Achse `node_type`), Prefetch, Telemetrie und den ChainRef-Value-Handle — sowie ein Merge-Organ für die Pfadkompression, das die Cache-Engine-eigene Patricia-Strategie enthält; im codegen-fähigen Mess-Pfad wirken davon zwei (Prefetch und Value-Handle), während MultiLevel-Layout, Pool-Allokator und Serialisierung über den Cache-Engine-Standard laufen (Abschnitt 3.5). Die Telemetrie ist in der Cache-Engine eine System-Achse und kein Kompositions-Slot; der gleichnamige Prüfling-Slot ist ein verbleibender Bestand. Die OLC-Nebenläufigkeit ist eine Bausteinschicht des Prüflings, kein angemeldeter Slot. Zu beachten ist, dass die Slot-Nummern des Prüflings nicht der Slot-Reihenfolge T00–T17 der Cache-Engine folgen. Der Einsatz des Prüflings zeigt die Erweiterbarkeit des Systems am Beispiel — zur effizienten Weiterentwicklung neuartiger Ansätze im Umgang mit cache-aware Algorithmen und beschleunigter Suche.
- Ein Mess-Treiber mit Defined-/Full-Modus-Unterscheidung (ergänzt um eine deterministisch gestichproben Full-Variante), profilbewusstem Workload-Routing — im Standard die Voll-Permutation über alle registrierten Workloads, deren Umsetzungsstand Kapitel 5 ausweist — und drei Granularitäten: Mikro-, Makro- und Gesamt-Benchmarking (einzelne Bestandteile, Interface-Operationen, ganze Algorithmen).
- Eine durchgehend deklarativ gesteuerte Bau- und Mess-Kette hinter diesem Treiber: ein *Experiment-Planer* löst die XML-Beschreibung des Experiments gegen die Achsen-Angebote der drei Achsen-Register auf, und der `CacheEngineBuilder` baut und vermischt daraus plattformbezogen die Binary-Permutationen — bis zur Auswertung über die `xlsx`-Ausgabe (die Mappe ist Ausgabe und Standard; die CSV-Form ist kein einzelnes Kind-Dokument, sondern ein geordneter Baum aus CSV-Dateien — ein Gesamtordner je Mappe, darunter ein Ordner je Blatt, darin die CSV-Dateien der einzelnen Unter-Achsen-Fahrten —, der immer aus der Mappe erzeugt wird, als Strategie derselben Factory, und per XML statt der Mappe oder zusätzlich zu ihr am selben Ziel liegt) und die daraus erzeugten LaTeX-Tabellen und TikZ-Diagramme.
- Eine bias-freie Vergleichs-Matrix, die *alle* registrierten Lastprofile gegen *alle* rekonstruierten Lebewesen fährt — die aus den Profil-XML gebauten Original-Entwurfs-Binaries — und so

der Selbstbevorzugung durch autor-eigene Mess-Aufbauten in Einzelveröffentlichungen begegnet (englischer Musterbegriff: *home-field advantage*) [KHA⁺19].

- Eine generische Plattform-Probe, die die Cache-Eigenschaften je Zielplattform ermittelt — Line-Größe und Größen je Hierarchie-Ebene aus sysfs und CPUID — und als feste Größen in eine systemoptimierte Binary einfließen lässt. Die Kern-Topologie (P-/E-Kern-Trennung, Pinning) ist als Plattform-Belang derselben Probe vorgesehen und für die Cache-Line-Messung auf Hybrid-CPU's Pflicht, ihre Erfassung durch die Probe ist jedoch nicht implementiert — der Umsetzungsstand steht in Kapitel 5. Der Messplan sieht für die beiden Produktionsmaschinen zwei Betriebssystem-Regimes vor (immutable's Talos und root-Linux mit vollem Hardware-Zähler-Zugriff); gemessen ist das root-Linux-Regime, das Talos-Regime nicht.
- *Als Zielsetzung und Ausblick:* die Auslieferung und begründete Auswertung des besten Gesamtalgorithmus — automatisch ausgewählt aus den feingliedrigen Messergebnissen unter dem Messaspekt des besten cache-bewussten Verhaltens und der Beschleunigung durch Cache-Line-Anpassung, im Vergleich aller messbaren Parameter gegen die Binary-Wettbewerber — als eigenständige, ABI-stabile Binary hinter dem einheitlichen Suchalgorithmus-Interface. Die Mess-Pipeline liefert die dafür nötige Rangbildung bereits, und ein erstes Inkrement der automatischen Auswahl und des Binary-Versands ist als eigenständiges Werkzeug (`best_binary_selector`) implementiert; die heuristik-gesteuerte Vollautomatisierung bleibt Ausblick.

1.5 Aufbau der Arbeit

Der Aufbau folgt dem Weg vom Allgemeinen zum Speziellen: Kapitel 2 legt — konzeptionell und bildhaft — die Grundlagen aus bekanntem Wissen: die Landschaft der Suchstrukturen, die Hardware-Eigenschaften der Speicherhierarchie, die Workload-Konventionen der Vergleichsliteratur, die Software-Mittel der Abstraktion und den Heuristik-Begriff. Kapitel 3 entwickelt daraus die Konzepte eines cache-bewussten Mess-Systems: die Problemstellung und ihre Anforderungen, den Stand der Technik als Achsen-Instanzen, das Anatomie-Modell mit seinen Schichten, das Mess-System selbst, den Prüfling PRT-ART als Demonstrator und die heuristische Auswertung samt ihrer Betriebsmodi. Kapitel 4 setzt dieses Konzept als Software-Architektur um — Repositorien, der Prüfling im Code, Achsen-Architektur und Übersetzungs-Pipeline, Schichten-Verträge mit Contract-Tests sowie die System-Achsen im Code. Kapitel 5 wertet entlang der Forschungsfragen aus, einschließlich des heuristischen Gegenbeweises; Kapitel 6 schließt mit Fazit und Ausblick. Die Anhänge vertiefen begleitend: die Messreihen-Dokumentation (Anhang A), die Code-Struktur (Anhang B), das Glossar (Anhang C), die vollständige Bausteine-Matrix (Anhang D), die Architekturentscheidungen (Anhang E) und die Vergleichs-Interfaces (Anhang F).

2 Suchbäume und Grundlagen

Dieses Kapitel führt vom vertrauten Gegenstand — den Suchbäumen — über die Hardware, die ihre Leistung prägt, und die Workloads, unter denen sie verglichen werden, zu den Software-Mitteln und zum Heuristik-Begriff, mit denen sich ihre Entwurfsentscheidungen abstrahieren und auswählen lassen. Alles bleibt bei *bekannten* Begriffen, Konzepten und Veröffentlichungen; die systematischen Kataloge und das eigene Begriffsgerüst folgen in Kapitel 3.

2.1 Konzept und Überblick über den Stand der Technik

Zunächst wird die Landschaft der Suchstrukturen kartiert — vergleichsbasierte Bäume, digitale und präfixbasierte Verfahren, Hashing, räumliche und flache Strukturen — und die bekannten Vertreter werden mit ihren Kernideen und Belegstellen eingeordnet. Die Gesamtheit der jeweils besten veröffentlichten Verfahren bezeichnet die Literatur als *Stand der Technik* (englisch *State of the Art*, kurz SOTA); gemeint sind damit in dieser Arbeit stets die publizierten Suchstruktur-Entwürfe, an denen sich ein neuer Entwurf messen lassen muss. Hier deutet sich bereits an, dass sich jeder Algorithmus in einem gemeinsamen Entwurfsraum verorten lässt; wie sich dieser Entwurfsraum systematisch aufspannt, entwickelt Kapitel 3. Suchstrukturen lassen sich danach einordnen, mit welchem Mechanismus sie einen

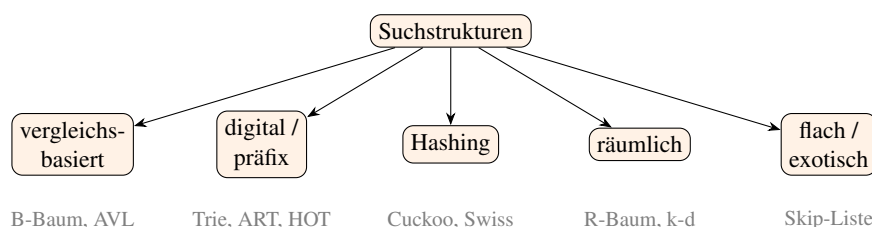


Abbildung 2.1: Die Landschaft der Suchstrukturen: fünf Klassen mit typischen Vertretern.

Schlüssel auf einen Datensatz abbilden [Knu98]. *Vergleichsbasierte* Strukturen lokalisieren den Schlüssel über Ordnungsvergleiche und setzen eine totale Ordnung voraus — von den selbstbalancierenden AVL-Bäumen [AVL62], deren Geschwister-Teilbäume sich in der Höhe um höchstens eins unterscheiden, und den Rot-Schwarz-Bäumen [Bay72, GS78], die dieselbe logarithmische Höhe über eine Knotenfärbung mit lockereren, günstiger zu wartenden Invarianten sichern, über die block- und externspeicherorientierten B-/B⁺-Bäume [BM72, Com79], deren Knoten typischerweise einer Cache- oder Speicherblock-Größe entsprechen [GL01, HP03] und die in der B⁺-Variante alle Nutzdaten in den Blättern halten und innere Knoten auf Wegweiser reduzieren, bis zum hauptspeicher-nahen T-Baum [LC86], der knotenlokale Wertebereiche mit der AVL-Balance verbindet und so die Zahl der Zeiger-Indirektionen je Suchschritt senkt. *Digitale* Strukturen verzweigen dagegen nach den Bytes des Schlüssels statt durch Vergleich, sodass die Suchpfadlänge von der Schlüssellänge und nicht von der Schlüsselanzahl abhängt: der *Trie* [Bri59, Fre60] als Grundform, in dem jeder Wurzelpfad einen Schlüssel buchstabiert und gemeinsame Präfixe geteilt werden,

die pfadkomprimierten Radix-/PATRICIA-Bäume [Mor68]¹, die dünn belegte Ein-Kind-Ketten kollabieren, und der *Adaptive Radix Tree* (ART) [LKN13], der die Knotenrepräsentation an den tatsächlichen Verzweigungsgrad anpasst. ART dient dieser Arbeit als Referenzentwurf: Seine Entwurfsentscheidungen — die nach Verzweigungsgrad gestaffelten Knotenklassen Node4, Node16, Node48 und Node256 sowie die lineare Suche innerhalb kleiner Knoten — kehren in Kapitel 3 als einzeln vermessbare Teilentscheidungen wieder, während der ART-Gesamtbaum als Prüfling und nicht als durchgängig mitgemessene Struktur auftritt.

Jenseits der baumartigen Verfahren bildet *Hashing* Schlüssel direkt über eine Hashfunktion ab und kennt keine inhärente Ordnung, weshalb Bereichsabfragen teuer sind [Knu98]; Cuckoo-Hashing [PR04], das im Gegensatz zu offener und verketteter Adressierung konstante Worst-Case-Lookups garantiert, und Tabellen wie SwissTable [Goo18], die dank SIMD-Gruppen-Probing hohe praktische Dichte erzielen, bilden die nicht-baumartige Vergleichsklasse der Literatur. *Räumliche* Strukturen (k-d-Baum [Ben75], Quadtree [FB74], R-Baum [Gut84]) partitionieren einen mehrdimensionalen Schlüsselraum und liegen außerhalb des Umfangs dieser Arbeit, ebenso wie die theoretischen „Exoten“ — der van-Emde-Boas-Baum [EB75] und der y-fast Trie [Wil83] mit $O(\log \log u)$ über einem Universum der Größe u oder der die Vergleichsschranke unterschreitende Fusion Tree [FW93]. Zur flachen Klasse zählen die randomisierte Skip-Liste [Pug90], das flache, sortierte Array mit Binär- oder Interpolationssuche [Knu98] — streng genommen das einzelne Blatt eines Baumes — und das Suffix-Array, das dieses Layout auf String-Schlüsseln zu einem platzsparenden Volltext-Index verallgemeinert und dessen leichtgewichtige, speicherbudgetierte Konstruktion als Musterbeispiel des Algorithm-Engineerings gilt [MF02].

Der Umfang dieser Arbeit gliedert sich entlang dieser Landschaft in drei Stufen. Untersuchungsgegenstand sind erstens die baumartigen Strukturen der vergleichs- und der digitalbasierten Klasse (Abbildung 2.1) und mit ihnen die elementaren Suchmethoden auf sortierten Feldern, aus denen Baumknoten bestehen: die Binär- und die k-äre Suche [Knu98, SGL09], die Interpolationssuche [PIA78], das Eytzinger-Layout [KM17]² und die lineare Suche innerhalb kleiner Knoten [LKN13]; das flache, sortierte Feld liegt damit nicht außerhalb des Umfangs, sondern ist als Knoteninneres sein kleinster Baustein. Zweitens gehören approximative Mitgliedschaftsfilter als vorgeschaltete Bausteine zum Umfang — der Bloom-Filter [Blo70], der Cuckoo-Filter [FAKM14], der Xor-Filter [GL20] und SuRF [ZLL⁺18] als bereichsfähiger Filter; Cuckoo-Hashing tritt in dieser Arbeit also nicht als Hash-Tabelle, sondern in seiner Filter-Form auf. Drittens liegen weitere Vertreter — die Skip-Liste, die Hash-Suche, der binäre Suchbaum, der B-Baum, die Gesamtbäume ART [LKN13], HOT [BZP⁺18], START [FJKN20], Wormhole [WNJ19] und SuRF [ZLL⁺18] sowie eine skalare Nachbildung der SwissTable — als zuschaltbare, im Referenzlauf nicht aktivierte Bausteine vor; die SwissTable-Nachbildung erlaubt dabei keine Messaussage über die SIMD-basierte Vorlage. Außerhalb des Umfangs bleiben die räumlichen Strukturen, die theoretischen Exoten und das Suffix-Array. So verschieden die Klassen äußerlich wirken, teilen sie doch wiederkehrende, voneinander trennbare Teilentscheidungen; welche Bausteine im Referenzlauf tatsächlich vermessen werden, nennt Kapitel 3 mit Implementierungsstand und Quelle.

Quer zu dieser Suchalgorithmus-Sicht steht die etablierte Untergliederung der *Container* der

¹PATRICIA (*Practical Algorithm To Retrieve Information Coded in Alphanumeric*): Morrisons pfadkomprimierter binärer Trie, dessen innere Knoten nur die Bitposition festhalten, an der sich die Schlüssel des jeweiligen Teilbaums erstmals unterscheiden [Mor68].

²Eytzinger-Layout: die implizite, zeigerfreie Anordnung eines sortierten Feldes in der Ebenenreihenfolge eines vollständigen Binärbaums — die Kinder des Elements an Position i liegen an den Positionen $2i$ und $2i + 1$ —, sodass der Suchpfad ohne Zeiger berechenbar ist und die Speicherzugriffe der Suche für Prefetching vorhersagbar werden [KM17].

C++-Standardbibliothek [ISO24]: Sequenz-Container (`std::vector`) ordnen nach Einfügeposition, geordnet-assoziative (`std::map`, `std::set`) nach Schlüsselordnung, ungeordnete (`std::unordered_map`) bilden über eine Hashfunktion ab. Diese genormten Schnittstellen-Verträge bilden die gemeinsame Basis, gegen die sich strukturell verschiedene Verfahren einheitlich vermessen lassen. Diese Arbeit setzt auf zwei von ihnen als Vertrags-Hüllen auf: auf der geordneten Map als gemeinsamem Nenner vergleichs- wie digitalbasierter Suchstrukturen und auf dem `std::vector` als Sequenz-Sicht; die Hash-Tabelle bleibt als ungeordnete Gegenprobe der begriffliche Gegenpol der Literatur, ist aber keine eigene Vertrags-Hülle dieser Arbeit — ihre Bausteine sind, soweit vorhanden, der geordneten Map-Hülle untergeordnet und nur zuschaltbar (vorstehender Absatz). Die Einordnung beider Hüllen in das Begriffsgerüst dieser Arbeit entwickelt Kapitel 3; ihre vollständige Ausführung in der Implementierung zeigt Kapitel 4.

Alle bisher genannten Verfahren sind *fest entworfene* Strukturen: ihr innerer Aufbau steht fest, bevor die erste Anfrage eintrifft. Eine etablierte Forschungslinie verlagert den Entwurf dagegen in die Laufzeit. *Database Cracking* [IKM07] baut den Index als Nebenprodukt der Anfrageverarbeitung adaptiv auf — jede Anfrage partitioniert die Daten ein Stück weiter —, und *Stochastic Cracking* [HIKY12] macht dieses Vorgehen gegen ungünstige Anfragefolgen robust; *gelernte Indizes* [KBC⁺18] ersetzen die Suchstruktur durch ein Modell der Schlüsselverteilung, das die Position eines Schlüssels näherungsweise vorhersagt. Jüngere Arbeiten führen diese Linie aktiv-tunend fort: VEGA justiert einen gelernten Index zur Laufzeit mit gruppenweiser Lern-Granularität nach [LCL⁺25], und AirIndex entwirft die Hierarchie eines Index automatisch aus dem Daten- und Speicher-I/O-Profil des Zielsystems [CLP23]. Diese Ansätze zeigen einen gemeinsamen Trend: weg von der fest verdrahteten Einzelstruktur, hin zu konfigurierbaren, sich anpassenden Entwürfen. Für die vorliegende Arbeit sind sie die nächstverwandten bekannten Ansätze, denn sie belegen, dass die Entwurfsentscheidungen einer Suchstruktur veränderlich gehalten werden können; wie sich solche Entscheidungen systematisch zerlegen und einzeln vermessen lassen, entwickelt Kapitel 3.

Für die cache-bezogenen Eigenschaften dieser Entwürfe hat die Literatur eine gestufte Begriffs-Taxonomie herausgebildet, die diese Arbeit durchgängig verwendet. Ihr Ausgangspunkt ist das *I/O-Modell* (External-Memory-Modell) von Aggarwal und Vitter: ein zweistufiger Speicher, dessen Ebenen Daten nur in Blöcken fester Größe austauschen; Kostenmaß eines Algorithmus ist allein die Zahl der Blocktransfers [AV88]. Auf die Cache-Hierarchie übertragen heißt ein Entwurf *cache-aware*, wenn er die konkreten Parameter der Zielplattform — Blockgröße, Kapazität — kennt und explizit nutzt; *cache-conscious* bezeichnet seit Chilimbi, Hill und Larus dieselbe Haltung aus Sicht der Datenstruktur: ihr Layout wird durch Clustering, Färbung und Kompression gezielt an Cache-Blöcke angepasst [CHL99]. *Cache-oblivious* ist die Gegenposition von Frigo, Leiserson, Prokop und Ramachandran: Der Algorithmus kennt keinerlei Parameter; analysiert wird im *Ideal-Cache-Modell* — zwei Ebenen, Kapazität Z , Line-Länge L , volle Assoziativität, optimale Ersetzung —, und ein optimaler cache-oblivious Entwurf ist dann auf *jeder* Ebene einer mehrstufigen Hierarchie zugleich nahezu optimal [FLPR99]. *Cache-sensitive* schließlich ist die Wortwahl von Rao und Ross für ihre auf die Cache-Line zugeschnittenen Suchbäume (Abschnitt 2.2) und wird weitgehend synonym zu cache-aware geführt [RR99, RR00]; als deutschen Oberbegriff für die parameter-kennenden Spielarten verwendet diese Arbeit *cache-bewusst*.

Quer zu dieser Parameter-Frage liegt die Zeitpunkt-Frage: *Wann* fällt die cache-bezogene Entwurfsentscheidung? Ein *statischer* Entwurf fixiert sie vor der ersten Anfrage — sei es cache-bewusst auf ermittelte

Parameter oder cache-oblivious parameterfrei —, während die *aktiven beziehungsweise adaptiven* Ansätze des vorigen Absatzes die Entscheidung in die Laufzeit verlagern und aus beobachtetem Verhalten nachführen. Beide Unterscheidungen — Parameter-Kenntnis und Entscheidungs-Zeitpunkt — sind voneinander unabhängig; der Heuristik-Begriff, der sie später verbindet, folgt in Abschnitt 2.5.

2.2 Hardware: historische Anpassung an die Suchleistung

Die Leistung von Suchstrukturen wird seit jeher von der Hardware getrieben. Nachgezeichnet wird die historische Einführung cache-naher Hardware — Cache-Hierarchie, Cache-Line, Adressübersetzung, Kohärenz — mit Fokus auf die *statischen und übersetzungszeit-relevanten* Eigenschaften, die später die Abstraktion in Abschnitt 2.4 stützen. Moderne Prozessoren schalten mehrere Cache-Ebenen (L1, L2 und

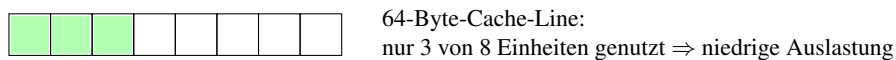


Abbildung 2.2: Cache-Line-Auslastung: eine Cache-Line wird stets vollständig geladen; werden nur wenige ihrer Bytes genutzt, sinkt die effektive Bandbreite — cache-bewusstes Layout hebt die Auslastung.

— nicht auf jeder Plattform — L3) zwischen die Register und den Hauptspeicher [HP19]; ein Zugriff, der sein Datum nahe findet, ist günstig, während ein Fehlzugriff bis zum Hauptspeicher einige hundert Zyklen kostet [Dre07], auf Mehr-Sockel- und Chiplet-Systemen zusätzlich abhängig vom besitzenden NUMA-Knoten. Die Übertragungseinheit zwischen den Ebenen ist die *Cache-Line* (Abbildung 2.2); ihre Größe ist keine universelle Konstante, sondern je Mikroarchitektur fest verdrahtet — 64 Byte auf x86-64 und gängigen ARM-Kernen (etwa Cortex-A76 oder Neoverse V2), 128 Byte etwa auf Apple-Silicon-L2 und System-Level-Cache [App24], schmalere 32 Byte auf älteren oder eingebetteten Kernen [HP19, Int24a, Adv23]. Programme können sie zur Laufzeit *auslesen* — über `CPUID` (x86), `CTR_EL0` (ARM) oder `riscv_hwprobe` (RISC-V; eingeschränkt, u. a. CBO-Blockgrößen) [Arm24, Int24b, Lin26a]³ —, aber nicht verändern; auch die Tiefe der Hierarchie variiert (einfache RISC-V-Kerne wie der SiFive U74 enden bei L2, manche AMD-Prozessoren bringen einen gestapelten 3D-V-Cache-L3 mit). Für ein cache-line-bewusstes Layout ist die je Zielplattform ermittelte Line-Größe daher ein *Eingabeparameter der Übersetzung*, keine feste Annahme; wie dieser Eingabeparameter in dieser Arbeit gebunden wird, zeigt Kapitel 3. Der Transfer einer Line erfolgt zudem nicht streng blockweise am Stück: *critical-word-first* liefert zuerst das angefragte Wort, und DDR5 unterteilt den Speicherkanal in zwei unabhängige 32-Bit-Sub-Kanäle [KAS21]; nicht-temporale bzw. Write-Combining-Schreibzugriffe umgehen überdies den normalen Write-Allocate-Pfad, statt die betroffene Cache-Line in die Cache-Hierarchie zu holen [Int24b].

2.2.1 Cache-Organisation: Assoziativität, Fehlzugriffs-Klassen und Parallelität

Wo eine Line im Cache liegen darf, bestimmt die *Assoziativität*: Der Cache ist in *Sets* gegliedert, von denen jedes n Ways (Line-Plätze) hält; die Speicheradresse zerfällt in Tag, Set-Index und Line-Offset,

³`CTR_EL0` ist das *Cache Type Register* der ARM-Architektur, ein Systemregister, das u. a. die kleinste Line-Größe der Instruktions- und der Daten-Caches kodiert [Arm24]; `riscv_hwprobe` ist der Linux-Systemaufruf, über den ein RISC-V-Programm Eigenschaften des laufenden Kerns abfragt, darunter die Blockgrößen der Cache-Block-Operationen (CBO) zum Nullsetzen, zur Verwaltung und zum Vorabholen von Cache-Blöcken [Lin26a].

und jede Adresse kann nur in den n Plätzen *ihres* Sets liegen — direkt abgebildet ($n = 1$) und voll-assoziativ (ein einziges Set) sind die Grenzfälle, gängige Daten-Caches liegen mit 4- bis 16-fach dazwischen [HP19, Int24a, Adv23]. Nach ihrer Ursache klassifizieren Hill und Smith Fehlzugriffe in drei Klassen (*3-C-Modell*): *compulsory* (Erstzugriff — die Line war nie im Cache), *capacity* (die Arbeitsmenge übersteigt die Kapazität) und *conflict* (mehr heiße Lines fallen auf ein Set, als es Ways hat) [HS89]; die Lehrbuch-Darstellung ergänzt in Mehrkern-Systemen *coherence*-Fehlzugriffe als vierte Klasse [HP19]. Die Klassifikation ist handlungsleitend, denn sie trennt die Gegenmittel: Compulsory-Misses adressiert Prefetching (Abschnitt 2.2.3), Capacity-Misses adressieren kompaktere Layouts, und Conflict-Misses entstehen gerade bei den regelmäßigen Adressmustern großer Knoten-Arrays — liegen heiße Elemente um Zweierpotenz-Abstände auseinander, konkurrieren sie um dieselben Sets [HP19, Dre07].

Ein Fehlzugriff blockiert einen modernen Cache zudem nicht: Seit Krofts *lockup-free*-Organisation führt der Cache je ausstehendem Miss ein Statusregister (*Miss-Status-Holding-Register*, MSHR) und bedient währenddessen weitere Zugriffe [Kro81]. Die kleine, feste Zahl dieser Register — bei Intel als *Line-Fill-Buffer* zwischen L1 und L2 realisiert — begrenzt, wie viele Speicherzugriffe gleichzeitig unterwegs sein können: die *Memory-Level-Parallelität* (MLP) [HP19, Int24a]. Ob eine Suchstruktur diese Parallelität überhaupt anbieten kann, ist eine Entwurfsfrage — der Suchpfad eines Baumes ist eine Kette voneinander abhängiger Zugriffe und nutzt sie ohne Weiteres nicht (Abschnitt 2.2.3).

Zwei weitere Eigenschaften wirken an den Line-Grenzen: *Ausrichtung* (Alignment) — ein Objekt, das eine Line-Grenze überspannt, kostet zwei Transfers statt einem, weshalb cache-bewusste Layouts Knoten an Line-Grenzen ausrichten — und *False Sharing*: Schreiben zwei Kerne auf *disjunkte* Felder derselben Line, wandert die Line trotz logischer Unabhängigkeit zwischen den Kernen hin und her, weil die Kohärenz (Abschnitt 2.2.5) stets ganze Lines verwaltet [Dre07, Int24a].

2.2.2 Cache-line-orientierte Layout-Forschung

Eben dieses cache-line-orientierte Layout ist der historische Leitgedanke der einschlägigen Forschung: Der CSS-Baum [RR99] führt den Cache-Line-Knoten ein, der CSB⁺-Baum [RR00] das zusammenhängende Geschwister-Cluster, Hankins und Patel [HP03] vermessen den Effekt der Knotengröße und Samuel et al. [SPB05] machen den Knoten prozessor-bewusst; der Adaptive Radix Tree [LKN13] staffelt mit Node4/16/48/256 die Knotengröße cache-line-orientiert, und Schmidt et al. [SKK⁺25] vermessen Stride-gegen Sequenz-Zugriffe auf High-Bandwidth-Memory. Als konkrete Layout-Optionen kennt die Literatur das dichte und das cache-line-ausgerichtete Array-of-Structs, das spaltenorientierte Struct-of-Arrays, das bit-gepackte, succinct Layout, wie es LOUDS-basierte Tries [Jac89] und SuRF [ZLL⁺18] nutzen, und das SIMD-gekachelte AoSoA⁴. Die mehrstufige Hierarchie selbst ist überdies Gegenstand software-/hardware-kooperativer Prefetch-Verfahren wie dem hierarchischen Instruktions-Prefetcher für Server-Anwendungen von Zhang et al. [ZGH⁺25], der in diese Arbeit als Prefetching-Instanz und nicht als datenstruktur-spezifischer Baum-Datenprefetch eingeht.

⁴AoSoA (*Array of Structures of Arrays*): ein hybrides Layout, das die Datensätze in Kacheln fester Breite gruppiert und innerhalb jeder Kachel feldweise ablegt, sodass ein SIMD-Register genau eine Kachelspalte lädt; es verbindet die Zugriffslokalität des Array-of-Structs mit der Vektorisierbarkeit des Struct-of-Arrays.

2.2.3 Prefetching

Prefetching holt Daten *vor* ihrer Verwendung in den Cache und ist das wichtigste Gegenmittel gegen Compulsory-Misses und unverdeckte Speicherlatenz (Abschnitt 2.2.1). Die Hardware-Prefetcher moderner Kerne bilden wenige, dokumentierte Klassen: Next-/Adjacent-Line-Prefetcher holen die Nachbar-Line jeder verfehlten Line, Stream-Prefetcher erkennen auf- oder absteigende Zugriffsfolgen und laufen ihnen voraus, Stride-Prefetcher lernen je Lade-Instruktion (am Instruction-Pointer erkannt) konstante Abstände, und komplexere raum- oder zeitkorrelierende Verfahren sind Gegenstand der Forschung; die Übersicht gibt Mittal [Mit16], die konkreten Prefetcher der L1-/L2-Stufen dokumentieren die Hersteller-Handbücher [Int24a, Adv23]. Bewertet wird ein Prefetcher an drei Größen: *Genauigkeit* (wie viele geholte Lines wurden genutzt), *Abdeckung* (wie viele Misses wurden vermieden) und *Rechtzeitigkeit* — zu späte Prefetches verdecken die Latenz nur teilweise, zu frühe verdrängen noch benötigte Lines [Mit16].

Ergänzend stellen die Architekturen *Software*-Prefetch-Instruktionen bereit (x86: `PREFETCHh`-Familie samt nicht-temporaler Variante; ARM: `PRFM`), mit denen das Programm eine Adresse samt Ziel-Ebene selbst ankündigt [Int24b, Arm24]. Ihre zentrale Stellgröße ist die *Prefetch-Distanz* — wie weit voraus angekündigt wird: groß genug, um die Speicherlatenz zu verdecken, klein genug, um die geholte Line nicht vor Gebrauch wieder zu verlieren; sie hängt von Latenz und Verarbeitungstempo ab und ist damit eine je Plattform zu kalibrierende Größe [Mit16]. In der Index-Forschung begründen Chen et al. genau darüber breitere B^+ -Knoten: Werden alle Lines eines mehrzeiligen Knotens parallel vorangekündigt, sinkt die effektive Knoten-Ladezeit, und die optimale Knotenbreite wächst über die eine Cache-Line hinaus [CGM01]; Fractal Prefetching überträgt dasselbe Prinzip ebenen-übergreifend auf Platten-Seiten [CGM02].

Seine Grenze findet Prefetching am *Pointer-Chasing*, dem Kern jeder Baumsuche: Die Adresse des nächsten Knotens steht erst fest, wenn die Line des aktuellen Knotens eingetroffen ist. Eine solche abhängige Ladekette bietet weder einem Stride- noch einem Stream-Prefetcher ein Muster, und sie serialisiert die Speicherzugriffe — die vorhandene Memory-Level-Parallelität (Abschnitt 2.2.1) bleibt ungenutzt [Mit16]. Die bekannten Gegenmittel setzen deshalb nicht am Vorhersagen, sondern am Entwurf an: mehr Nutzbytes je abhängigem Schritt (breitere, line-ausgerichtete Knoten [CGM01]) oder mehrere *unabhängige* Suchen, deren Zugriffe so verschränkt werden, dass die Ladekette der einen die Wartezeit der anderen füllt — Entwurfsantworten, die in Abschnitt 2.2.6 als Parameter wiederkehren.

2.2.4 Hauptspeicher und Adressübersetzung: DRAM, NUMA und TLB

Hinter dem letzten Cache liegt der DRAM-Hauptspeicher, dessen Kenngrößen für die Generation DDR5 die JEDEC-Norm JESD79-5 festlegt [JED20]. Zwei Größen werden dabei häufig vermengt: Die *Datenrate* in Megatransfers je Sekunde (MT/s; zwei Transfers je Takt) misst die Bandbreite des Interfaces, die *CAS-Latenz* (CL) zählt Taktzyklen bis zur ersten Antwort einer geöffneten Zeile — da die CL-Zyklenzahl mit der Nennrate mitwächst, senken steigende Datenraten die absolute Zugriffslatenz kaum [HP19, KAS21]. Parallelität liefert der Aufbau: mehrere Speicherkanäle je System, die bereits erwähnte DDR5-Zweiteilung jedes Kanals in unabhängige 32-Bit-Sub-Kanäle sowie Bank-Groups innerhalb jedes Chips, deren verschränkte Ansteuerung Zeilen-Aktivierungszeiten überdeckt [JED20, KAS21]. Auf dem Modul hinterlegt sind die *JEDEC-Nennprofile* der Norm; daneben existieren herstellerspezifische Übertaktungsprofile (Intel XMP, AMD EXPO) jenseits der Norm-Raten — welche Rate eine Maschine tatsächlich fährt, ist darum

eine dokumentationspflichtige Eigenschaft der Zielpattform, keine aus dem Modul ablesbare Konstante. In Mehr-Sockel- und Chiplet-Systemen kommt die Topologie hinzu: Bei *NUMA* (Non-Uniform Memory Access) besitzt jeder Knoten eigenen lokalen Speicher, und Zugriffe auf entfernte Knoten kosten spürbar mehr als lokale. Das Betriebssystem platziert Seiten standardmäßig nach der *First-Touch*-Politik — eine Seite wird auf dem Knoten desjenigen Threads abgelegt, der sie zuerst beschreibt [Lam13]. Für Messungen an Suchstrukturen ist das eine stille Vorbedingung: Wer die Struktur auf einem anderen Knoten aufbaut, als er sie vermisst, misst die Interconnect-Latenz mit.

Neben der Daten-Cache-Hierarchie existiert eine zweite Lokaltätsgrenze: der *Translation Lookaside Buffer* (TLB) cacht Virtuell-zu-Physisch-Abbildungen, und ein TLB-Miss löst einen teuren Page-Walk aus [HP19]. Sein *Reach* — Eintragszahl mal Seitengröße — begrenzt den ohne Page-Walk adressierbaren Speicherbereich. Prozessoren trennen aus Lokaltätsgründen den Instruktions- vom Daten-TLB (dTLB) — da Instruktionszugriffe deutlich lokaler sind als Datenzugriffe — und legen ihn mehrstufig aus (L1-dTLB plus gemeinsamer L2-STLB) [Int24b, Dre07]; für Suchstrukturen ist der dTLB die zweite, eigenständige Grenze neben der Cache-Line, denn große oder weit gestreute Knoten erhöhen den dTLB-Druck unabhängig von der Cache-Auslastung. Im Stand der Technik tritt der dTLB vorwiegend als *Mess-Ziel* auf; algorithmenseitig adressieren ihn große Seiten (Huge-Pages, 2 MiB/1 GiB), die den TLB-Reach vergrößern, von mehreren Allokatoren — *mimalloc* [LZM19], *jemalloc* [Eva06], *TCMalloc* [HKT⁺21] und *snmalloc* [LDPB19] — unterstützt werden und die Schmidt et al. [SKK⁺25] mit 2-MiB-Seiten empirisch nutzen; diese vier Allokatoren liegen in dieser Arbeit als austauschbare Allokator-Bausteine vor, und welche Allokator-Bausteine im Referenzlauf aktiviert sind, nennt Kapitel 3. Eine eigenständige dTLB-Optimierung bleibt in den Suchalgorithmus-Veröffentlichungen jedoch unterbelegt, eine Beobachtung, auf die die Forschungslücke in Kapitel 3 (Abschnitt 3.1) zurückkommt.

2.2.5 Schreibpfad, Ersetzung und Kohärenz

Auch das Schreib- und Update-Verhalten der Caches ist überwiegend mikroarchitektonisch fixiert. Schreibzugriffe folgen *Write-Through*, das unmittelbar bis in die nächste Ebene durchschreibt, oder *Write-Back*, das die Änderung zunächst nur in der Cache-Line vermerkt (*dirty*) und erst bei deren Verdrängung zurückschreibt — was Bus-Verkehr spart und die Schreiblatenz senkt; orthogonal dazu legt die Allokations-Politik (Write-Allocate, typisch bei Write-Back, bzw. No-Write-Allocate, typisch bei Write-Through) die Behandlung eines Schreib-Fehlzugriffs fest; ARMv8 macht diese Wahl über Seiten-Attribute — Write-Through, Write-Back oder Non-Cacheable samt Read- bzw. Read-Write-Allocate-Hinweisen — explizit [Arm24], während x86 im Normalfall Write-Back mit Write-Allocate verwendet [Int24b]. Welche Line verdrängt wird, entscheidet eine heuristische Ersetzungsstrategie von LRU/Pseudo-LRU bis zur zwei-Bit-sparsamen, scan- und thrash-resistenten *Re-Reference Interval Prediction* (dynamische Variante DRRIP) [JTJE10], die auf der adaptiven Insertions-Politik [QJP⁺07] aufbaut; wie viele Ebenen eine Line zugleich führen, regelt die Inklusions-Politik — ein *inklusive* Cache hält jede Line der höheren Ebenen redundant, ein *exklusiver* (Victim-Cache) schließt sie aus, ein *nicht-inklusive* lässt beides zu —, und in Mehrkern-Systemen sichert ein invalidierungsbasiertes Kohärenzprotokoll die Konsistenz: das verbreitete MESI führt jede Line in einem der Zustände *Modified*, *Exclusive*, *Shared* oder *Invalid*, herstellerseitig um Varianten wie MOESI (AMD [Adv23]) oder MESIF (Intel [Int24a]) erweitert [HP19]. Als konkrete Instanz dieser fixierten Eigenschaften variiert etwa die L3-Inklusions-Politik zwischen den Generationen — inklusiv bei früheren Intel-Client-Generationen (etwa Nehalem), nicht-inklusive bei Intel-Servern ab Skylake-SP/Xeon Scalable

und überwiegend victim-artig bei AMD Zen [Int24a, Adv23]. Da all dies zur Laufzeit überwiegend nicht frei wählbar ist, sind diese Eigenschaften je Zielplattform gegeben und nur durch Messung zugänglich; ihre Wirkung — insbesondere das *Cache-Line-Ping-Pong* bei konkurrierenden Schreibzugriffen auf dieselbe Line — muss ein faires Messverfahren eigens sichtbar machen (Kapitel 3).

2.2.6 Cache-bewusste und cache-oblivious Datenstrukturen

Mit der Begriffs-Taxonomie aus Abschnitt 2.1 und den nun eingeführten Parametern lässt sich der Stand der Technik der cache-nahen Datenstrukturen ordnen. Den empirischen Anstoß der cache-bewussten Linie gaben Ailamaki et al., deren Zeit-Zerlegung kommerzieller Datenbanksysteme zeigte, dass rund die Hälfte der Ausführungszeit in Speicher-Wartezyklen liegt — vor allem in L2-Daten- und L1-Instruktions-Fehlzugriffen [ADHW99]. Die Antworten der Folgejahre binden je einen Teil der Parameter aus den vorigen Abschnitten explizit in den Entwurf ein: die Cache-Line-Größe (CSS-/CSB⁺-Baum [RR99, RR00]), die Prefetch-Distanz samt mehrzeiliger Knoten (pB⁺-Baum [CGM01], ebenen-übergreifend Fractal Prefetching [CGM02]), Seitengröße und SIMD-Breite zugleich (FAST, ein architektur-sensitives Baum-Layout mit verzahnter Seiten-, Line- und SIMD-Blockung [KCS⁺10]) oder den Verzweigungsgrad je Cache-Line (ART [LKN13]); die Layout-Techniken des Clustering und der Kompression gehen auf Chilimbi et al. zurück [CHL99]. Die cache-oblivious Gegenposition ist ebenso instanziiert: Bender et al. begründen den cache-oblivious B-Baum über das van-Emde-Boas-Layout [BDFC05], aufbauend auf dem Order-Maintenance-/Packed-Memory-Primitiv [BCD⁺02], der cache-oblivious Traversierung geordneter Mengen mit lokalitätserhaltender $O(k/B)$ -Schranke [BCDFC02] und dem Baum-Layout fester Topologie, das die erwartete Zahl der Blocktransfers in einem mehrstufigen Speicher-Modell auch bei unbekannter Block-Größe nahezu optimal minimiert [BDFC02, BDFC00]; Saikkonen und Soisalon-Soininen halten eine cache-sensitive Layout-Invariante auch unter Updates [SSS08, SSS16], und Graefe/Larson [GL01] vergleichen beide Ansätze über die CPU-Cache-Ebenen, während sämtliche praktischen Trie- und B⁺-Indizes cache-bewusst ausgelegt sind. Tabelle 2.1 ordnet die genannten Entwürfe danach, welche Hardware-Parameter sie explizit in den Entwurf binden.

Tabelle 2.1: Cache-nahe Struktur-Entwürfe und die Hardware-Parameter, die sie explizit in den Entwurf binden.

Entwurf	Cache-Line	Seite/TLB	SIMD-Breite	Prefetch-Distanz	parameterfrei
CSS-Baum [RR99]	•	—	—	—	—
CSB ⁺ -Baum [RR00]	•	—	—	—	—
pB ⁺ -Baum [CGM01]	•	—	—	•	—
Fractal Prefetching [CGM02]	•	•	—	•	—
FAST [KCS ⁺ 10]	•	•	•	—	—
ART [LKN13]	◦	—	•	—	—
Cache-oblivious B-Baum [BDFC05]	—	—	—	—	•
Knotengrößen-Studie Hankins/Patel [HP03]	•	—	—	—	—

Legende: • = expliziter Entwurfsparameter der Originalarbeit; ◦ = indirekt berücksichtigt (ART staffelt die Knotengrößen cache-line-orientiert, ohne die Line-Größe zu parametrisieren); — = nicht adressiert. Einordnung dieser Arbeit auf Basis der zitierten Originalarbeiten.

Zwei Beobachtungen an der Tabelle führen weiter. Erstens bindet jeder Entwurf nur *wenige* Parameter fest ein — kein bekannter Entwurf behandelt die Parameter-Menge der vorigen Abschnitte als offenen Eingaberaum. Zweitens lassen beide Philosophien dieselbe Frage offen: Wie lässt sich *gemessenes*

Lokalitätswissen der konkreten Zielplattform — deren Cache-Eigenschaften überwiegend fest verdrahtet und nur zum kleineren Teil konfigurierbar sind (Abschnitt 2.2.5) — systematisch mit den tatsächlich wählbaren cache-nahen Einstellungen verbinden? Diese Frage nimmt Kapitel 3 (Abschnitt 3.1) wieder auf.

2.3 Workloads und Benchmark-Konventionen

Ob eine Suchstruktur die Hardware-Eigenschaften aus Abschnitt 2.2 gut nutzt, zeigt sich erst unter Last; dafür haben sich feste Begriffe und Benchmark-Konventionen etabliert. Eine *Last* (Operation) ist ein einzelner Aufruf der Such-Schnittstelle — eine Punktsuche (auch Punktanfrage: die Suche nach genau einem Schlüssel), ein Einfügen, ein Löschen oder ein Bereichs-Scan. Ein *Workload* (in dieser Arbeit synonym: *Lastprofil*) ist ein strukturiertes Profil solcher Operationen: ein Operations-Mix (etwa 50 % Lesen / 50 % Aktualisieren) über einer Schlüssel-Zugriffsverteilung (gleichverteilt, Zipf-verteilt, jüngste Werte) auf einem Datensatz definierter Größe und Schlüssel-Charakteristik. Realistische Workloads sind dabei Folgen vieler aufeinanderfolgender Interface-Aufrufe — nicht isolierte, unrealistische Einzelzugriffe.

Die verbreitetste Workload-Konvention ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]. Er definiert eine Familie von Schlüssel-Wert-Workloads: YCSB-A (50/50 Lesen/Aktualisieren, Zipf-verteilt), YCSB-B (95/5 Lesen/Aktualisieren), YCSB-C (nur Lesen), YCSB-D (jüngste Werte), YCSB-E (kurze Bereichs-Scans) und YCSB-F (Read-Modify-Write). Daneben stützt sich die Literatur auf weitere etablierte Familien: den Index-Benchmark *SOSD* [KMR⁺19], die Benchmarks der *TPC*-Familie (transaktional TPC-C, analytisch TPC-DS) [Tra24], die Integration in reale Systeme (etwa SuRF [ZLL⁺18] als Bloom-Filter-Ersatz in RocksDB), reale String-Korpora (URLs, DNA, Wörterbuch-Terme) sowie — bei den hardware-nahen Arbeiten — *SPEC CPU* [Sta17] und *CloudSuite* [FAK⁺12]. Diese Frameworks erzeugen unterschiedliche, untereinander nicht unmittelbar vergleichbare Lasten.

Das *Ziel der Workload-Variation* ist der Bruch eines methodischen Bias: jede Veröffentlichung wählt typischerweise das Lastprofil, unter dem ihr eigener Algorithmus am besten abschneidet. Erst wenn alle Lastprofile gegen alle Vergleichskandidaten laufen, lässt sich bias-reduzierte Vergleichbarkeit gewinnen. Für diese Arbeit liefern die genannten Konventionen das Rohmaterial eines gemeinsamen Lastprofil-Satzes; wie dieser Satz aufgebaut ist und über alle Kandidaten gefahren wird, entwickelt Kapitel 3, die zugehörige Mess-Matrix gehört zur Evaluationsmethodik (Kapitel 5).

Beobachtet wird unter Last mit einem etablierten Satz von Kenngrößen. Auf Cache-Ebene zählen die Fehlzugriffe je Ebene, ausgedrückt als *Miss-Rate* (Fehlzugriffe je Zugriff) oder — robuster beim Vergleich verschieden zugriffsintensiver Programme — als *MPKI* (Misses je tausend Instruktionen); auf Kern-Ebene messen *IPC* beziehungsweise *CPI* (Instruktionen je Takt und dessen Kehrwert) die erreichte Ausführungsdichte, und Stall-Zyklen-Zerlegungen weisen aus, welcher Anteil der Taktzyklen auf wartende Speicherzugriffe entfällt [HP19]; dass dieser Anteil bei Datenbank-Lasten dominieren kann, war gerade der Befund von Ailamaki et al. [ADHW99] (Abschnitt 2.2.6). Für cache-bewusste Layouts kommt die *Cache-Line-Auslastung* hinzu — der Anteil tatsächlich genutzter Bytes je geladener Line (Abbildung 2.2) —, die unmittelbar misst, was ein Layout an der Übertragungseinheit gewinnt.

Erhoben werden diese Größen über die *Performance-Monitoring-Counter* (PMC) der Prozessoren: wenige programmierbare Zählregister je Kern, denen per Event-Auswahl ein Hardware-Ereignis (etwa L2-Miss oder dTLB-Miss) zugewiesen wird; zugänglich machen sie Werkzeuge wie *perf* [Mel10] oder

VTune [Rei05]. Ihre Grenzen sind ebenso dokumentiert: Zählerstände sind über Wiederholungen nicht streng deterministisch und zählen je nach Ereignis systematisch über [WM08], und sollen mehr Ereignisse beobachtet werden, als Register existieren, multiplext das Werkzeug die Zähler — das Ergebnis ist dann eine Hochrechnung, keine Zählung. Ein belastbarer Vergleich braucht darum Wiederholungen und eine ausgewiesene Streuungs-Angabe [HB15, WM08]. Welche dieser Kenngrößen diese Arbeit erhebt und wie sie zu Mess-Kategorien geordnet werden, entwickelt Kapitel 3.

2.4 Software-Mittel und Zusammenführung zur Abstraktion

Mit welchen Software-System-Mitteln — Übersetzungszeit-Metaprogrammierung, statische Nutzung der Hardware-Eigenschaften aus Abschnitt 2.2 — lässt sich die dort offen gebliebene Abstraktion *umsetzen*? Zur Verfügung stehen die etablierten Mittel des software-technischen Entwurfs: die benannten Entwurfsmuster [GHJV94], der statische Polymorphismus des *Curiously Recurring Template Pattern* (CRTP) [Cop95, VJG17]⁵ samt *Policy-based Design* [Ale01] sowie die Komposition aus austauschbaren Bausteinen, wie sie die invasive Software-Komposition aus Gray-Box-Komponenten [Aßm03], die generative Programmierung [CE00] und die feature-orientierten Software-Produktlinien [ABKS13, BO92] beschreiben; C++23-Module strukturieren die Übersetzungseinheiten solcher Baustein-Kompositionen, garantieren für sich genommen aber noch keine portable ABI-Stabilität [Xu24] — relevant, sobald vorübersetzte Bausteine ausgetauscht werden sollen; wie diese Arbeit die Binärschnittstelle ihrer vorübersetzten Bausteine stattdessen stabil hält, zeigt Kapitel 4.

Damit aus austauschbaren Bausteinen ein konkretes System wird, braucht es neben den Kompositionstechniken eine *Beschreibungsform* für die jeweils gewählte Konfiguration. Die etablierte Antwort sind *deklarative Konfigurationsmodelle*: Feature-Modelle beschreiben seit der FODA-Studie⁶ konfigurierbare Variantenräume als Merkmal-Hierarchien mit Auswahl-Constraints [KCH⁺90], die generative Programmierung erzeugt aus solchem Konfigurationswissen im Problemraum die Systemvariante im Lösungsraum [CE00], und die Produktlinien-Literatur systematisiert die zugehörigen Bindungszeitpunkte von der Übersetzungszeit bis zur Laufzeit [ABKS13]. Der industrielle Werkzeugbau folgt demselben Prinzip: Mavens *Project Object Model* ist die deklarative — als XML-Datei gehaltene — Manifestation dessen, *was* ein modulares Java-System ausmacht, während der Build-Lebenszyklus das *Wann* und *Wie* übernimmt [Apa26]; CMake erzeugt aus einer deklarativen Beschreibung über austauschbare *Generatoren* je Plattform ein natives Buildsystem [Kit26]; und Mokhov, Mitchell und Peyton Jones zeigen, dass sich Build-Systeme insgesamt als Kombination weniger Scheduler- und Rebuilder-Bausteine über deklarativen Abhängigkeitsbeschreibungen wissenschaftlich fassen und neu zusammensetzen lassen [MMP18]. Dass deklarative Modelle ein System über die Bauzeit hinaus auch im Betrieb begleiten können, entwickelt die *models@run.time*-Linie [BBF09]; Götz zeigt in der TU-Dresdner MQuAT-Dissertation, wie deklarative (Konfigurations-)Modelle die Selbstoptimierung komponentenbasierter Systeme durch Kontrakt-Verhandlung steuern [Göt13].

⁵CRTP: Eine Klasse leitet von einer Klassenschablone ab, der sie sich selbst als Schablonenargument übergibt (`class Derived: Base<Derived>`); die Basis kann dadurch bereits zur Übersetzungszeit auf Elemente der abgeleiteten Klasse zugreifen und ersetzt virtuelle Aufrufe durch statisch aufgelöste — ohne Laufzeit-Indirektion und ohne Tabelle virtueller Funktionen. Benannt und beschrieben hat das Muster Coplien 1995 im *C++ Report* [Cop95].

⁶FODA (*Feature-Oriented Domain Analysis*): die Domänenanalyse-Methode des Software Engineering Institute, die die gemeinsamen und die variablen Merkmale (*Features*) einer Produktfamilie in einem Merkmal-Baum mit Pflicht-, Optional- und Alternativ-Knoten samt Auswahl-Constraints erfasst [KCH⁺90].

Für diese Arbeit bilden beide Stränge zusammen den Werkzeugkasten: die Kompositions-Techniken, mit denen sich die je Zielplattform gemessenen Hardware-Eigenschaften zur Übersetzungszeit in ein konkretes Layout umsetzen lassen, und die deklarativen Konfigurationsmodelle, mit denen sich die gewählte Konfiguration beschreiben und reproduzieren lässt; wie beide dafür kombiniert werden, entwickelt Kapitel 3.

2.5 Heuristiken als Entscheidungsregeln

Sowohl die adaptiven und gelernten Indizes (Abschnitt 2.1) als auch die Ersetzungsstrategien der Caches (Abschnitt 2.2) treffen ihre Entscheidungen *heuristisch*. Eine *Heuristik* ist eine Entscheidungsregel ohne Optimalitätsgarantie: Sie bildet beobachtbare Merkmale — etwa die je Zielplattform ermittelten Cache-Eigenschaften (Abschnitt 2.2) oder das gemessene Lastprofil (Abschnitt 2.3) — auf eine konkrete Konfiguration ab, ohne deren Bestmöglichkeit zu beweisen. Sie tritt überall dort an die Stelle einer exakten Lösung, wo diese unbekannt oder zu teuer zu bestimmen ist; die heuristischen Ersetzungsstrategien des Caches selbst (LRU, RRIP; Abschnitt 2.2) sind das hardwareseitige Vorbild dieser Denkweise [JTJE10]. Heuristiken lassen sich danach unterscheiden, *wann* ihre Entscheidung fällt. Eine *statische Heuristik* ist zur Übersetzungszeit fixiert: Die gewählte Konfiguration wird fest in das übersetzte Programm einkompiliert und ändert sich zur Laufzeit nicht mehr. Eine *dynamische Heuristik* wählt die Konfiguration hingegen zur Laufzeit, abhängig vom tatsächlich beobachteten Verhalten, und kann sie im Betrieb anpassen. Beide Spielarten setzen dieselbe *Heuristik-Funktion* voraus, verschieben aber den Zeitpunkt ihrer Auswertung. Formal ist diese Heuristik-Funktion ein *Mapping* von einem Messergebnis beziehungsweise Lastprofil auf eine Konfiguration: $h : \mathcal{M} \rightarrow \mathcal{K}$, wobei $\mathcal{M}$ den Raum der erhobenen Merkmale (Messwerte, Lastprofil-Kennzahlen) und $\mathcal{K}$ den Raum der zulässigen Konfigurationen bezeichnet. Die vorliegende Arbeit misst diesen Zusammenhang zunächst systematisch aus (Kapitel 3); die *automatisierte* Bestimmung von h selbst — der Mechanismus, der aus den Messergebnissen die jeweils beste Konfiguration ableitet — ist Zielsetzung und Ausblick dieser Arbeit (Abschnitt 6.3).

Schließlich sind die in der Literatur teils synonym gebrauchten Begriffe abzugrenzen. *Adaptiv* (*adaptive*) heißt eine Heuristik, die ihre Entscheidung an veränderliche Bedingungen anpasst; *dynamisch* (*dynamic*) bezieht sich enger auf den Zeitpunkt — die Auswertung zur Laufzeit statt zur Übersetzungszeit; und *messgetrieben* (*measurement-driven*; gleichbedeutend *messwertgetrieben*) betont die Datenquelle — die Entscheidung gründet auf erhobenen Messwerten und nicht auf festen Modellannahmen. Eine adaptive Heuristik ist somit stets dynamisch, eine dynamische nicht notwendig adaptiv, und beide können, müssen aber nicht messgetrieben sein.

Damit sind die Grundlagen beisammen: die Landschaft der Suchstrukturen, die Hardware-Eigenschaften, die ihre Leistung prägen, die Workload-Konventionen, unter denen sie verglichen werden, die Software-Mittel der Abstraktion und der Heuristik-Begriff, der Messwissen mit Konfigurationswahl verbindet. Wie sich mit genau diesen Mitteln ein Suchalgorithmus in orthogonale, einzeln vermessbare Entwurfsentscheidungen zerlegen lässt, entwickelt Kapitel 3; die konkrete Umsetzung der dort eingeführten Bausteine zeigt Kapitel 4.

3 Konzepte eines cache-bewussten Mess-Systems

Auf den Grundlagen aus Kapitel 2 aufbauend entwickelt dieses Kapitel zunächst die Problemstellung und die Anforderungen an ein Mess-System und als deren *Lösung* das Anatomie-Modell und das darauf aufbauende Mess-System — demonstriert am Prüfling PRT-ART. Drei Ordnungen führt das Kapitel dabei neu ein, die die Kapitel 4 bis 6 voraussetzen: die Dreiteilung der Achsen in Mess-, System- und Organ-Achsen, die vier Träger-Stufen des Apparats — Planer, CacheEngineBuilder, Tier-Binary und Hybrid-Tier-Binary — mit ihrem Identitäts-Stempel sowie das Lager, in dem Gebautes und Gemessenes unter Stempel-Schlüsseln liegt.

3.1 Problemstellung und Anforderungen an ein Mess-System

Den Ausgangspunkt liefert der Befund der Grundlagen (Kapitel 2): Über die Leistung cache-bewusster Suchstrukturen entscheidet nicht die asymptotische Komplexität, sondern das Cache-Line-Verhalten unter realer Last. Dieses Verhalten ist jedoch das Ergebnis vieler ineinandergreifender Entwurfsentscheidungen — wie Knoten verzweigen, wie Seiten im Speicher angeordnet werden, wie alloziert, vorgeladen und synchronisiert wird —, und in den publizierten Verfahren sind diese Entscheidungen monolithisch verwachsen: Wer ein Verfahren vermisst, vermisst stets das ganze Bündel, nie die einzelne Entscheidung. Genau das ist die Problemstellung dieser Arbeit, das *Trennbarkeits-Problem*: Der Beitrag jeder einzelnen Entwurfsentscheidung zum Cache-Line-Verhalten soll isoliert sichtbar werden — je Entscheidung wie für den ganzen Algorithmus.

Der Stand der Technik verschärft dieses Problem in zwei Richtungen. Inhaltlich optimiert jede der untersuchten Arbeiten nur wenige Entwurfsentscheidungen isoliert, und im untersuchten Korpus belegt *kein einziges Verfahren alle Entwurfsdimensionen gemeinsam*: Manche Dimensionen sind ausschließlich mit Baseline- oder Lehrbuch-Varianten besetzt (das Mapping zwischen Algorithmus- und Cache-Traversierung), andere treten nur als *Mess-Ziel* auf (der dTLB), die räumliche Such-Klasse fehlt ganz, und Entscheidungen wie Ein-/Ausgabe, Migration, Filter und Pufferung sind zwar je einzeln, aber nie gemeinsam mit den Such-Entscheidungen in einem Entwurf belegt. Methodisch kommt eine zweite Lücke hinzu: Ergebnisse werden in nicht vergleichbaren Settings berichtet, und die häufige Wahl szenariospezifischer, oft günstig zugeschnittener Lastprofile erschwert einen fairen Quervergleich; es fehlt eine bias-reduzierte Vermessung *aller* Lastprofile gegen *alle* Verfahren auf einer gemeinsamen Basis.

Aus dieser doppelten Lücke folgen vier Anforderungen an eine Lösung. Sie muss cache-bewusste Suchalgorithmen erstens in austauschbare Entwurfsbestandteile *zerlegen*, sodass jede Entscheidung einzeln fassbar wird; zweitens die Stand-der-Technik-Entwürfe als explizite Konfigurationen dieser Bestandteile *rekonstruieren*, damit Bekanntes und Neues im selben Raum vergleichbar werden; drittens das cache-line-bewusste Verhalten je Bestandteil wie für den ganzen Algorithmus auf gemeinsamer Basis *messen* — alle Lastprofile gegen alle Verfahren, bias-reduziert; und viertens aus den Messungen den besten Gesamtalgorithmus ableiten und systemoptimiert *ausliefern* — realisiert als Ranking je Interface-

Funktion und Metrik, als Pareto-Front nicht-dominierten Kandidaten und als Versand-Artefakt aus Binary, Versions-Sidecar und Manifest (Abschnitt 3.6.1). Der cache-aware-Fokus ist dabei keine Randbedingung, sondern die Leitgröße: Genau das Cache-Line-Verhalten, das über die Leistung entscheidet, soll je Entwurfsentscheidung isoliert messbar werden.

Die Mess-Anforderung präzisiert sich an der Gegenüberstellung cache-bewusster und cache-oblivious Entwürfe (Abschnitt 2.2) zu einem dritten, dazwischenliegenden Weg: Die tatsächlichen Cache-Eigenschaften (Line-Größe, Hierarchie-Tiefe, Topologie) werden je Zielplattform *ermittelt* und gehen als feste Größen in eine systemoptimierte Binary ein, während die *konfigurierbaren* cache-nahen Einstellungen als Unter-Achsen-Parameter gegen eine bereits geladene Binary permutiert werden — im Mess-Realm sind das die registrierten dynamischen Dimensionen Prefetch-Distanz, Pool-Budget, Stapelgröße, Inline-Schwelle, Thread-Zahl und Workload sowie, wo Architektur und Betriebssystem es erlauben, der Hardware-Prefetcher, abgestuft nach dem, was Hardware und Betriebssystem erlauben, von voll laufzeit-konfigurierbar (mutables Linux mit MSR-Zugriff) über boot- bzw. übersetzungsstatisch (immutable Betriebssysteme) bis fixiert (read-only-Cache-Register auf ARM und RISC-V); welche Abstufung eine Plattform bietet, ermittelt die Plattform-Probe des Planers. So wird cache-oblivious-Verhalten um *gemessenes* Lokaliätswissen ergänzt.

Die vier Anforderungen verdichten sich zu einem Zielbild, dem *Entwurfsraum*: Jede Teilentscheidung — etwa Knotentyp, Speicher-Layout oder Allokator — spannt eine eigene Achse auf, und jeder Stand-der-Technik-Algorithmus ist ein Punkt im kartesischen Produkt dieser Achsen (Abbildung 3.1).

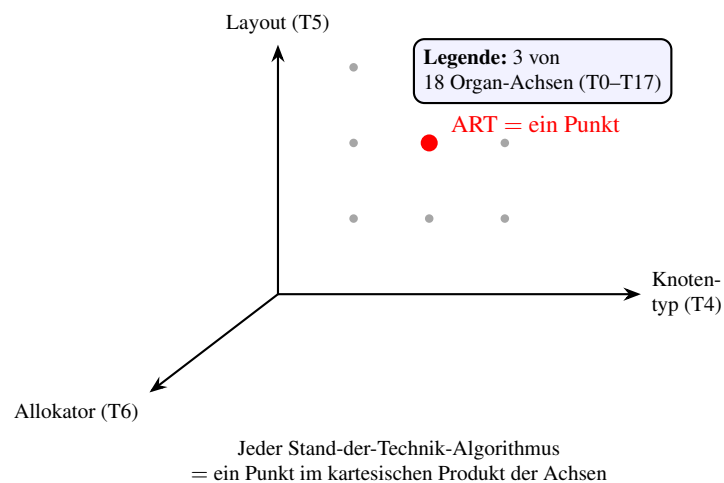


Abbildung 3.1: Der Entwurfsraum: die Achsen spannen ein multidimensionales kartesisches Produkt auf; ein konkreter Algorithmus ist ein Punkt darin. Zur Veranschaulichung sind drei der achtzehn Organ-Achsen gewählt — das reale Speicherorgan aus Knotentyp (T4), Speicher-Layout (T5) und Allokator (T6); der volle Entwurfsraum hat achtzehn Organ-Dimensionen; dazu tritt eine optionale neunzehnte Organ-Achse für die Festplatten-Ein-/Ausgabe, die nicht eine Belegung beschreibt, sondern beschreibt, wie andere Achsen beschrieben werden (Abschnitt 3.4.1).

Ist der Entwurfsraum so aufgespannt, wird das Trennbarkeits-Problem lösbar: Eine Entwurfsentscheidung zu variieren heißt, sich entlang genau einer Achse zu bewegen, während alle übrigen fixiert bleiben — ihr Beitrag wird isoliert messbar. Was diese Arbeit beiträgt, ist folglich die systematische, orthogonale Achsen-Zerlegung der bekannten Entwürfe samt einer *permutierten Mess-Architektur*, die sie als explizite

Konfigurationen rekonstruiert und auf gemeinsamer Basis vermisst; denn cache-aware Algorithmen sind ohne eine Aufteilung der das Caching maßgeblich bestimmenden Eigenschaften nicht differenzierbar. Die vier Anforderungen beschreiben dabei zugleich eine Verarbeitungs-Kette — von der Definition einer Achsen-Belegung über ihre Erzeugung und Vermessung bis zur Auswertung —, und ein System dieser Größe bleibt nur beherrschbar, wenn diese Stationen als getrennte Schichten mit festen Verträgen entwickelt werden; das Schichtenmodell dazu entwickelt Abschnitt 3.3.4. Welche der katalogisierten Instanzen das Framework übernimmt und welche es als überholt ersetzt, entwickelt der folgende Abschnitt 3.2 dialektisch; das Lösungskonzept (Abschnitt 3.3) und das darauf aufbauende Mess-System (Abschnitt 3.4) sind die Lösung, die die doppelte Lücke schließt.

3.2 Stand der Technik als Achsen-Instanzen

Die aus Kapitel 2 bekannten Verfahren erscheinen nun systematisch als Achsen-Kompositionen; die vollständigen Kataloge dieser Sezierung versammelt der vorliegende Abschnitt. Diese Aneignung verläuft *dialektisch* (These–Antithese–Synthese): je Achse wird ein fremdes Organ *übernommen* (These) oder eine überholte Annahme *ersetzt* (Antithese), woraus der eigene Beitrag als Synthese hervorgeht. Leitend ist der „dritte Weg“ aus Abschnitt 3.1 — gemessenes Lokaliätswissen statt fest verdrahteter oder ignorierte Cache-Bewusstheit —, der jede Achse als orthogonales, permutierbares und einzeln messbares Organ explizit macht. Bei vielen Achsen ist die Antithese keine überholte Fremd-Instanz, sondern die implizite Annahme, die Entscheidung gar nicht als wählbar zu betrachten; die Synthese besteht dann darin, sie überhaupt als Achse explizit zu machen (in der Ersetzen-Spalte als „—“ markiert). Zwei Cluster-Zuordnungen sind dabei bewusst Näherungen: Die Cache-Traversal-Achse (T1) bezieht ihre Anregungen sowohl aus der Layout-Theorie als auch aus dem Prefetching, und die Reklamations-Unterachse des Allokators (T6) teilt ihre Quellen mit der Synchronisation (T8) — eine Inter-Organ-Verdrahtung, keine Cluster-Verletzung. Getragen wird der dialektische Vergleich von der Lebewesen-Metapher: Jedes katalogisierte Verfahren ist ein Lebewesen, dessen Organ — die Belegung einer Achse — übernommen oder ersetzt wird, und das Framework ist das Lebewesen, das aus den übernommenen Organen entsteht. Die Metapher selbst (Organ, Blut, Lebensraum, Labor) definiert Abschnitt 3.3.1; hier wird sie nur angeschnitten. Tabelle 3.1 fasst diese Aneignung je Kompositions-Achse zusammen.

Tabelle 3.1: Dialektische Aneignung der Stand-der-Technik-Instanzen je Kompositions-Achse (in kanonischer Slot-Reihenfolge T0–T17; Telemetrie und Ziel-ISA gehören zur System-Sphäre bzw. zum Mess-Tooling, Abschnitt 3.4.1. Cluster-Kürzel: A Trie/Knoten, B Hybrid/Layout, C Allokation, D Prefetching, E Synchronisation, F Hardware/Messung/Engineering)

Achse		übernommen (These)			ersetzt (Antithese)	Synthese-Kern
T0	Search-Algorithm (A)	ART, SuRF, äre/Interpolations-Suche	HOT, Wormhole;	START, k-	Monolith „Struktur ist Verfahren“	Such-Paradigma als permutierbare Achse
T1	Cache-Traversal (B)	Cache-Line-Stride (Chen), cache-oblivious (Bender), HBM (Schmidt)			implizite Lehrbuch-Linearsuche	Algorithmus- und Cache-Sicht getrennt
T2	Mapping (B)	lineare 1:1-Abbildung; Hash-Redirect, Permutations-Index			—	Direct- vs. Pool-Relative-Mapping explizit

Achse	übernommen	ersetzt	Synthese-Kern
T3 Path-Compression (A)	byte-weise (ART), Patricia (HOT), Macro-Node (CoCo), Multi-Byte (START)	unkomprimierter Rohpfad	Kompressions-Granularität wählbar
T4 Node-Type (A)	ART Node4/16/48/256	feste Knotengröße	adaptive Knotenklasse + Storage-Delegation
T5 Memory-Layout (B)	AoS-aligned, SoA, LOUDS-bitgepackt, CSS/CSB ⁺ , Ao-SoA	—	vier Layout-Dimensionen permutierbar
T6 Allocator (C)	A01–A23 (Hoard, mimalloc, jemalloc, TCMalloc, ...) + RCU/Hazard	algorithmus-fest verdrahteter Allokator	Allokation als gemeinsames Interface-Organ (sieben Unter-Achsen)
T7 Prefetch (D)	Distance-Estimator (ART), Hardware (Wormhole), Stride (Chen), adaptiv	Prefetch nicht als Achse isoliert	Path-Oriented Bundle-Prefetch (eigen)
T8 Concurrency (E)	OLC (ART), RCU, Hazard-Pointer; Lock-/Wait-Free	grobes Sperren als einzige Annahme	Pattern $\perp$ Reclamation als zwei Unter-Achsen
T9 Serialization (F)	Raw-Binary, Varint (ART), succinct (LOUDS/FST), LZ77	—	Kodierung/Kompression wählbar
T10 Value-Handle (F)	Inline, External-Pool, Immutable-Shared (RCU), Versioned	implizit fest eingebetteter Wert	Platzierung, Ownership und Versionierung (+ Chain-Ref)
T11 Index-Organization (B)	Clustered, IOT, Non-Clustered, Heap	—	Storage-Order als eigene Achse
T12 I/O-Dispatch (F)	Buffered, mmap, Direct-IO	mmap als Anti-Pattern	Persistenz von der Suchstruktur entkoppelt
T13 Migration (F)	Hot/Cold, Multi-Tier NVM, LeCaR-ML	Tiering nie mit Such-Achsen kombiniert	Migrationslogik als messbare Achse
T14 Filter (F)	Bloom, Cuckoo, SuRF-Range, Xor	Filter nie mit Such-Achsen integriert	Filter als orthogonales Organ
T15 Queuing-q1 (F)	Disruptor-Ring, Delta-Chain, CoW, Epoch-RCU, SPSC/MPMC	Operations-Puffer nie als Achse	Puffer mit Zugriffsmuster-Klassen
T16 Queuing-q2 (F)	Eager/Lazy, Watermark (LSM), Timed	Flush-Policy implizit	adaptive EWMA-Flush-Policy (eigen)
T17 Persistence-Target (F)	In-Memory-Ablage als Standard-Ziel	implizite In-Memory-Annahme	Persistenz-Ziel als eigene Achse (im Vermessungskatalog auf In-Memory festgelegt)
<i>Größen der System- und Mess-Sphäre (Abschnitt 3.4.1):</i>			
target_isa (System)	Vendor-Spec x86-64/AArch64/Power/RISC-V (+ SIMD)	—	Ziel-ISA als System-Komplex-Achse
Telemetrie (System/Mess)	Density-Tracker, Insert-Zähler, HdrHistogram, Pfad-Zähler	Per-Node-Zähler (Cache-Line-Ping-Pong)	kohärenz-schonend Leaf-Only; zweigeteilt: Mess-Unter-Achse + Bau-Größe

Der systematische Kern ist die achsenweise *Sezierung*: Jeder Gesamt-Algorithmus wird in seine Or-

gane zerlegt, und je Achse werden die beitragenden Organ-Algorithmen aufgeführt, mit kurzem Verweis auf das Herkunfts-Lebewesen. Das Framework führt *achtzehn Organ-Kompositions-Achsen* in fester Slot-Reihenfolge (T0–T17); daneben stehen die Größen der System-Sphäre — Ziel-ISA, SIMD- und Erweiterungs-Hardware sowie die zweigeteilte Telemetrie (die Dreiteilung der Achsen ordnet Abschnitt 3.4.1) — und die Prüfling-Build-Achse Seitentyp. Neben den achtzehn kanonischen Organ-Haupt-Achsen — *Haupt-Achse* heißt eine Achse, deren gewählte Belegung übersetzungsstatisch in die Binary einkompiliert wird — führt der Organ-Katalog eine optionale *Organ-Meta-Meta-Achse* für die Festplatten-Ein-/Ausgabe (*Meta-Meta* heißt: Die Achse beschreibt nicht eine Belegung, sondern beschreibt, wie andere Achsen beschrieben werden; die Kurzdefinition samt Ebenen-Ordnung gibt Abschnitt 3.4.1), die nur bei einem Platten-Rückschreibe-Ziel wirksam wird und übersetzungsstatisch verbaut ist. Viele Achsen sind in *Unter-Achsen* (sub-axes) verfeinert — Achsen unterhalb einer Haupt-Achse: als statische Unter-Achsen eine übersetzungszeitliche Auswahl unter Teil-Algorithmen (etwa SA1–SA4 oder AA1–AA7), als dynamische Unter-Achsen Laufzeit-Parameter, die ohne Neuübersetzung variiert werden — und liefern anderen Achsen verteilte Interfaces für deren Detail-Operationen — genau diese *Achsen-Interfaces* erfasst das Mikro-Benchmarking (Abschnitt 3.4.6), das je Interface-Aufruf Laufzeit und Erfolgs-Parameter erhebt (Abschnitt 3.4). Tabelle 3.2 gibt den Überblick, welche Veröffentlichungen je Achse ein Organ beitragen; die thematischen Herkunfts-Cluster der Vorarbeiten erscheinen dabei *aufgelöst* in ihren Organ-Beiträgen.

Die *Such-Achse* (T0) umfasst die meisten Organe — als Original-Bindungen ART [LKN13], HOT [BZP⁺18], START [FJKN20], SuRF [ZLL⁺18] und Wormhole [WNJ19], dazu generische Such-Methoden wie k-äre Suche [SGL09], Interpolationssuche [PIA78], Eytzinger-Layout-Suche [KM17], Skip-Liste [Pug90], B-Baum [BM72] und Open-Addressing-Hashing [Knu98]. Die Traversierungs-Achsen trennen Algorithmus- und Cache-Sicht: T1 enthält im Code die drei Traversal-Bausteine `LinearFanout`, `HashLookup` und `BinarySearchFanout`; die verwandte Literatur spannt die Achse von der Cache-Line-Wanderung über Stride-Prefetch [CGM01] und HBM-Nähe [SKK⁺25] bis zur cache-oblivious-Rekursion [FLPR99] (Anhang D), während T2 (Mapping) mit der linearen 1:1-Abbildung (`DirectPlacement`) und dem pool-relativen Offset-Mapping (`PoolRelative`) eine dünn besetzte Achse bleibt — Hash-Redirect [WNJ19] (MP04) und Permutations-Index [MKM12] (MP03) führt der Code als Roadmap-Werte ohne Registry-Eintrag. Die *Pfadkompression* (T3) instanziiert die byte-weise Variante [LKN13], den Patricia-Split [Mor68, BZP⁺18] und die Durchreich-Variante `None`; die Macro-Node [BFTV24] und das Multi-Byte-Rewiring [FJKN20] sind verwandte Literatur der Achse ohne eigenen Baustein (die Unter-Achse PC1 führt die Granularität `multi-byte` als Tag), und die *Knotentyp-Achse* (T4) bündelt im Code die vier adaptiven ART-Knotenformate Node4/16/48/256 [LKN13]; HOT-Compound [BZP⁺18], Internal/Border [MKM12], Decision/Span [SSL⁺21], LOUDS-Dense/Sparse [ZLL⁺18] und die CSS-/CSB⁺-/Hankins-Knoten sind Paper-Knotenformate (Rekonstruktions-Ziele dieser Arbeit, keine CE-Baseline), die PRT-ART-Familie bringt der Prüfling über seine Build-Achse Seitentyp ein.

Die *Speicher-Layout-Achse* (T5) reicht im Code vom cache-line-ausgerichteten AoS (`AoSStrictMemoryLayout`, `CacheLineAlignedMemoryLayout`) über SoA und AoSoA bis zum bitgepackten LOUDS-Layout (`PackedBitmapMemoryLayout`) [Jac89]; zeigerfreie Anordnung [RR99], Geschwister-Cluster [RR00] und eingebetteter Baum [SSL⁺21] sind verwandte Literatur der Achse (Anhang D). Die *Allokator-Achse* (T6) ist mit dem Korpus A01–A23 am dichtesten besetzt und in sieben Unter-Achsen verfeinert — von der Free-List-Topologie über Thread-Lokalität/NUMA-

Tabelle 3.2: Achsen-Sezierung der Suchverfahren: achtzehn Organ-Achsen in kanonischer Slot-Reihenfolge (T0–T17), die Größen der System- und Mess-Sphäre, die Prüfling-Build-Achse und die optionale Organ-Meta-Meta-Achse; je Achse die ein Organ beitragenden Quellen.

Achse	Unter-Achsen	Beitragende Quellen
T0 search_algo	SA1–SA4	P01,P02,P05,P07,P10; k-ary (TUD 2009), Interpol., Eytzinger, Skip-List, B-Baum, Hash
T1 cache_traversal	CT1–CT2	Cache-Line-Default; P17,P21,P32
T2 mapping	MP1–MP2	P03,P07; CE-Baseline
T3 path_compression	PC1–PC3	P01,P02,P04,P05,P09
T4 node_type	NT1–NT3	P01,P02,P03,P04,P05,P06,P11,P12,P13
T5 memory_layout	HM1–HM3 ^b	P06,P09,P11,P12,P32; Default
T6 allocator ^a	AA1–AA7	A01–A23; P29,P30
T7 prefetch	PF1–PF3	P07,P21,P23,P25,P26,P27
T8 concurrency	CC1–CC2	P08,P29,P30; Dijkstra,Courtois,Herlihy,Michael/Scott
T9 serialization	SR1–SR3	P01,P09,P10; LZ77
T10 value_handle	VH1–VH3	P01,P03,P07,P29
T11 index_organization	io1–io3	Bayer/McCreight,Comer,Oracle
T12 io_dispatch	IO1–IO3	Stonebraker,Crotty; Baselines
T13 migration_policy	MG1–MG3	Anti-Caching,NVM-Tiering,LeCaR; None
T14 filter	FT1–FT3	Bloom,Cuckoo,P10,Xor
T15 queuing_q1	QS1–QS6	Disruptor, Bw-Tree, Driscoll, P29, Pugh, LSM, Lamport, Michael/Scott
T16 queuing_q2	FS1–FS4	Eager/Lazy/Watermark/Kafka/RocksDB
T17 persistence_target	—	CE-Baseline (In-Memory); Disk-Writeback als abgeschaltete Option
Sys. target_isa	IS1–IS3	x86-64/AArch64/Power/RISC-V (Vendor-Spec)
Sys. external_utils: SIMD	SE1–SE3	SSE2/AVX2/AVX-512/NEON/SVE2/RVV; GPU: CUDA
Sys. external_utils: Erw.-HW	HW1–HW4	HW-Strategie je Veröffentlichung (Tab. 3.4)
Sys./Mess telemetry	TM1–TM3	P01,P02,P16,P17,P26,P28; HdrHistogram (Gil Tene)
Build (Prüfling) page_type	PG1–PG3	P01,P02,P04,P05,P06,P10,P11,P12,P13,P20; PRT-ART (2026)
Organ-Meta-Meta disk_io	staging (Laufzeit)	CE-Baseline; optional, abgeschaltet (Abschnitt 3.4.1)

^a Korpus 23 Arbeiten (A01–A23); der Code stellt 26 Vendor-Implementierungen bereit.

^b HM4 (Stride-Muster) ist eine geplante vierte Unter-Achse ohne tragenden Layout-Baustein; der Code führt keinen Tag für sie (Anhang D, Abschnitt „T5 Speicher-Layout“).

Affinität [UHL⁺17] und die Allokationsstrategie bis zur Reklamation (Epoch, RCU [McK01], Hazard-Pointer [Mic04a], QSBR); der Code stellt 26 Allokator-Implementierungen zum 23-Arbeiten-Korpus bereit (einzelne Arbeiten steuern mehrere Varianten bei); die 23 Allokator-Profilen mit Lizenz und Workload-Affinität listet Tabelle 3.3, zehn davon sind mit Adapter und Original-Code angebunden. Die *Wert-Achse* (T10) unterscheidet Inline-Wert [LKN13], externen Pool [WNJ19], versionierten Zeiger [MKM12], immutable Shared-Ref [McK01] und die verkettete externe Referenz (ChainRef, lokale Re-Implementierung ohne Paper-Vorlage) — fünf von fünf Bausteinen der Registry —, die *Serialisierungs-Achse* (T9) Raw-Binary, succinct LOUDS/FST und Varint [LKN13].

Auf der hardware-nahen Seite enthält die *Prefetch-Achse* (T7) im Code vier Bausteine — keinen Prefetch (NonePrefetch, Mess-Baseline), den distanzgeschätzten Software-Prefetch nach ART [LKN13] (DistanceEstimatorPrefetch), die explizite Hardware-Prefetch-Instruktion aus Wormhole [WNJ19] (HardwarePrefetch) und den pfadorientierten Bündel-Prefetch des Prüflings (PathOrientedPrefetch, diese Arbeit); die verwandte Literatur spannt die Achse von software-festem [CGM01] über adaptiv-distanziertes [Kha11] und fill-buffer-bewusstes [MWR25]

Tabelle 3.3: Allokator-Profile (23, alle mit `expected_workload`-Tag) mit Lizenz, Workload-Affinität und Anbindungsstand

Profil	Familie	Lizenz	expected_workload	Code
hoard	A01	Apache-2.0	YCSB_A	ja
slab	A02	none	YCSB_A	—
michael_lockfree	A03	LGPL-2.1-or-later	YCSB_B	ja
mimalloc	A04	MIT	YCSB_A	ja
jemalloc	A05	BSD-2	YCSB_B	ja
tcmmalloc	A06	Apache-2.0	YCSB_C	ja
snmalloc	A07	MIT	YCSB_A	ja
scalloc	A08	BSD-3	YCSB_A	ja
numalloc	A09	unknown	YCSB_B	—
rpmalloc	A10	Public Domain	YCSB_C	ja
lrmalloc	A11	MIT	YCSB_B	ja
cama	A12	none	YCSB_A	—
starmalloc	A13	Apache-2.0	YCSB_B	—
tcmmalloc_warehouse	A14	Apache-2.0	YCSB_C	—
hmalloc	A15	none	YCSB_B	—
pim_malloc	A16	MIT	YCSB_E	—
crystalline	A17	unknown	YCSB_B	—
exgen_malloc	A18	unknown	YCSB_C	—
buddy	A19	none	YCSB_C	—
dlmalloc	A20	Public Domain (CC0)	YCSB_C	ja
ptmalloc2	A21	LGPL-2.1+	YCSB_B	—
pmr_resource	A22	Standard-Library	YCSB_B	—
vmem_magazines	A23	none	YCSB_A	—

`expected_workload` = YCSB-Workload-Klasse (A/B/C/E); Lizenz wortgleich aus dem Profil (none = kein OSS-Code, unknown = im Profil nicht belegt); Code = Adapter und Original-Code im Repository vorhanden (ja) oder Dokumentations-Akte ohne Adapter (—). Zwei Bezugsgrößen: 23 Profile A01–A23 (Profil-Zählung dieser Tabelle) gegenüber 26 Allokator-Wrappern des Codes (Wrapper-Zählung, Anhang D): A22 zerfällt dort in die drei Standardbibliotheks-Baselines A22a–A22c (`std_malloc`, `pmr_resource`, `pool_resource`), und die aus dem SOTA-Profil P33 rekonstruierte Allokator-Front A24 tritt ohne eigenes Allokator-Profil hinzu.

bis zu pfadorientiertem [ZSZ⁺24] und hierarchischem Bündel-Prefetch [ZGH⁺25] (Anhang D). Die System-Achse *Ziel-ISA* und die SIMD-Erweiterung des Erweiterungs-Hardware-Hubs (beide System-Sphäre, Abschnitt 3.4.1) führen die Vendor-Spezifikationen (x86-64, AArch64, Power, RISC-V mit SSE2/AVX2/AVX-512, NEON/SVE2, RVV). Zwei übergreifende Strategie-Achsen halten fest, was ein Algorithmus aktiv *nutzt* bzw. wie er Arbeit *verteilt* — die *Hardware-Strategie* (SIMD-Familie, Cache-Level-Targeting, NUMA, Prefetch-Hardware, Atomic-Familie) und die *Scheduling-Strategie* (Worker-Pool-Layout, SIMD-Worker-Limit — hardwarebedingt typisch zwei von N Kernen —, heterogener P-/E-Kern-Dispatch, Co-Routinen und Batch-Granularität) —, die Tabelle 3.4 je Veröffentlichung zuordnet. Die *Telemetrie* — in der Dreiteilung dieser Arbeit keine Organ-Achse, sondern zweigeteilt in Mess-Tooling-Unter-Achse und Bau-Größe der System-Sphäre (Abschnitt 3.4.1) — instanziiert Density-Tracker [LKN13], Insert-Zähler [BZP⁺18], HdrHistogram, Pfad-Lese-Zähler [ZSZ⁺24] und die Probability-Hints (P16/P17 Bender [BDFC00, BDFC05]) — im Code in zwei Familien: der CRTP-Organ-Registry `AllTelemetries` mit vier Strategien (Leaf-Only-Zähler, Density-Tracker, Insert-Zähler, Latenz-Histogramm) und der vtable-Familie `ITelemetryStrategy` mit sechs Arten, darunter Pfad-Lese-Zähler und Probability-Hints (Anhang D) —; die kohärenz-schonenden Leaf-Only-Varianten gehen auf Kuehn zurück (P28 [Kue23]; persönliche Kommunikation 2023, über den

publizierten DaMoN-2023-Stand zum cache- und verteilungsbewussten B⁺-Layout hinaus), während der Zähler in allen (auch inneren) Knoten als *Anti-Pattern* (Cache-Line-Ping-Pong) gilt.

Tabelle 3.4: In dieser Arbeit zugewiesene Ziel-Hardware- und Scheduling-Profile je rekonstruiertem SOTA-Verfahren (Auswahl; die Profile sind Rekonstruktions-Ziele dieser Arbeit, nicht zwingend Original-Paper-Eigenschaften; die Spalte Hardware-Strategie folgt dem Vokabular des im Code konservierten, konsumentenlosen Achsen-Entwurfs `hardware_strategy.hpp`, die Ziel-ISA jeder Rekonstruktion steht im Feld `<isa>` ihres SOTA-Profiles (System-Sphäre, im Code eine Build-Konstante; P03 und P31: `X86_AVX2`, P27 und P32: `X86_AVX512`); der Achsen-Katalog HW1–HW4 mit den Plattform-Profilen folgt in Anhang D, Abschnitt „Build-HW“, eine Paper-für-Paper-Liste gibt es nicht).

Veröffentlichung	Hardware-Strategie	Scheduling-Strategie
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (Bit-Vektor), L1-aware, CAS	thread-per-core
P03 Masstree	scalar (Profil-ISA <code>X86_AVX2</code>), L1-aware, CAS	thread-per-core + work-stealing
P05 START	AVX2 (Multi-Byte-Span), L1-aware, CAS	thread-per-core
P20 B-Trees Are Back	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (Instr.-Bundle), all-cache, CAS	thread-per-core + Co-Routine
P28 Kuehn	PREFETCH (scalar Counter), L1-aware, CAS	thread-per-core + Sampling
P31 Ungethuem	AVX-512 (HBM, Profil-ISA <code>X86_AVX2</code>), HBM-aware, NUMA-aware, CAS	hybrid (P-/E-Kerne)
P32 Schmidt	AVX-512 (SIMD-Stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)	thread-per-core + 2-SIMD-Worker-Limit + hybrid + Co-Routine + micro-batch

Die *Nebenläufigkeits-Achse* (T8) gliedert sich in die zwei Unter-Achsen Muster und Reklamation (Pattern ⊥ Reclamation, Tabelle 3.1): Die Muster reichen vom seriellen Nullpunkt über grobes Sperren [Dij65], Leser-/Schreiber-Sperren [CHP71] und optimistisches Lock-Coupling [LABN16] zu lock-freien [MS96], wait-freien [Her91] sowie RCU- [McK01] und Hazard-Pointer-Verfahren [Mic04a]; ein Sperr-Modus innerhalb eines Musters (read-only, read-write, upgradeable, optimistische Validierung) ist ein im Code konservierter, konsumentenloser Achsen-Entwurf (`locking_mode.hpp`) und kein Baustein: implementiert sind die Muster als Unter-Achse CC1 und die Reklamation als CC2 (Anhang D). Die *I/O-Achse* (T12) instanziiert den RAM-only-Nullpunkt der Achse (InMemoryOnly, Mess-Baseline ohne I/O-Latenz) sowie gepufferte [Sto81], direkte und memory-mapped Ein-/Ausgabe [CLP22] — vier von vier Bausteinen der Registry. Die Achse setzt dabei auf dem Blocktransfer-Denken der External-Memory-Tradition auf (Abschnitt 2.2): Für das I/O-effiziente Datenstruktur-Engineering stellt TPIE eine Referenz-Bibliothek der External-Memory-Linie bereit (ESA 2002) [APV02]; die Lehrbuch-Grundlage liefern Mehlhorn und Sanders [MS08]. Die *Migrations-Achse* (T13) instanziiert Hot/Cold-Trennung und mehrstufige Tier-Migration über die Stufen RAM→SSD→HDD (`kTiers = 3` im Code; die NVM-Stufe der Paper-Vorlage [RLK⁺18] ist im Code nicht implementiert), und die *Pufferungs-Achsen* (T15/T16)

reichen vom Disruptor-Ringpuffer [TFB⁺11] über die Bw-Tree-Delta-Kette [LLS13], den Copy-on-Write-Puffer [DSST89] und den Tombstone-Puffer [OCGO96] bis zu lock-freien SPSC- [Lam77, Lam83] und MPMC-Queues [MS96]. Die *Index-Organisations-Achse* (T11) unterscheidet geclusterte Anordnung [BM72], Heap-Ablage, nicht-geclusterten Sekundärindex [Com79] und die indexorganisierte Tabelle (IOT, Daten in den Blatt-Seiten des Index eingebettet) — vier von vier Bausteinen der Registry —, die *Filter-Achse* (T14) den Bloom- [Blo70], Cuckoo- [FAKM14], SuRF- [ZLL⁺18] und Xor-Filter [GL20]. Die *Persistenz-Ziel-Achse* (T17) schließlich macht das Ablage-Ziel explizit — In-Memory als Standard, ein Platten-Rückschreibe-Ziel als abgeschaltete Option in Kombination mit der optionalen, abgeschalteten Organ-Meta-Meta-Achse `disk_io` (Abschnitt 3.4.1) —; im golden-Referenzkatalog ist sie auf ihren In-Memory-Vertreter festgelegt, sodass dieser feste Regressions-Detektor $2^{17} = 131072$ Identitäten zählt (unter der überholten Annahme genau zweier Auslegungen je Achse), während die Obergrenze der Binary-Identitäten je System-Permutation das Produkt der je Achse im XML-Experiment gewählten Auslegungen ist, das der Planer aus den Eigenschaften des XML-Experiments bestimmt und das erst zur Ausführung bekannt ist (Abschnitt 3.4.6). Diese Rekombination wird in der Realität nicht erreicht, da nicht alle Achsen-Organe miteinander kompatibel sind.

Auch die *Lasten* erscheinen als Instanzen. Die in der Literatur verwendeten Workloads entstammen *mehreren*, untereinander nicht vergleichbaren Benchmark-Frameworks; Tabelle 3.5 ordnet sie den nutzenden Veröffentlichungen zu. Am verbreitetsten ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]; daneben treten der Index-Benchmark SOSD [KMR⁺19], die TPC-Familie [Tra24], die CPU-Suite SPEC CPU [Sta17], die Scale-out-Suite CloudSuite [FAK⁺12] sowie — für den Allokator-Korpus — mimalloc-bench [Lc21] und die Klassiker Larson, threadtest und shbench auf. Da diese Frameworks unvergleichbare Lasten erzeugen, abstrahiert die Arbeit sie zu einem gemeinsamen Satz von Lastprofilen; im Mess-Realm ist das Last-Rahmenwerk als Meta-Meta-Achse registriert und führt YCSB als einzigen aktivierten Baustein, während TPC, SOSD, SPEC CPU, CloudSuite und die Allokator-Suiten über einen *Datensatz-Lader* angebunden werden (Abschnitt 3.4.1). Vorgesehen ist, alle verfügbaren Benchmark-Systeme für das Messsystem unter dem Dach dieser Achse zu vereinen; im Code wie im Plan gelten die Benchmark-Systeme dabei über ihre Lastprofile als Unter-Achse des Messsystems (der Laufzeit-Parameter `workload` des Last-Rahmenwerks), weil sie laufzeitspezifische Änderungen der Kette von der XML zur Tier-Binary (Abschnitt 3.4.6) verwalten: Lastprofile und Datensätze wechseln, ohne dass die Tier-Binary neu übersetzt wird.

Aus dieser Analyse destilliert die Arbeit *vierzehn* distinkte Lastprofile (LP01–LP14, Tabelle 3.6): Sie geben den heterogenen Lasten ein gemeinsames, uint64-basiertes Operations-Modell — der Op-Mix nennt die Anteile von Einfügen, Lookup, Löschen, Scan und Read-Modify-Write über definierten Schlüssel-Verteilungen — und sind je aus konkreten Veröffentlichungen abgeleitet.

Der hier sezierte Stand der Technik beruht auf einem festen Korpus: Die Analyse stützt sich auf 33 tiefen-analyisierte Suchalgorithmus-Veröffentlichungen (P01–P33) und einen parallelen Allokator-Korpus aus 23 Arbeiten (A01–A23), deren Achsen-Belegungen als XML-Profile in der `comdare-cache-engine` persistiert sind (davon drei als abstrakte Profile, siehe unten); die vollständigen Bausteine- und Allokator-Matrizen führt der Anhang. Die achsenweise Sezierung dieses Korpus folgt einer Provenienz-Tabelle (Quellen gegen die Originalveröffentlichungen geprüft) über rund 110 Organ-Algorithmen; jeder Stand-der-Technik-Algorithmus erscheint so als ein Punkt im kartesischen Produkt der Achsen (Abbildung 3.1). Dass das Achsen-Gerüst auch nicht-baumartige Verfahren aufnimmt, belegt eine reine Hash-Tabelle als

Tabelle 3.5: Benchmark-Frameworks und nutzende Veröffentlichungen

Framework	Typ	Nutzende Veröffentlichungen
YCSB [CST ⁺ 10]	KV-Workload	P02, P03, P07, P20, P26
SOSD [KMR ⁺ 19]	Index-Benchmark	P05
TPC-C / TPC-DS [Tra24]	OLTP / analytisch	P19
Real-String-Korpora	String-Datensätze	P04, P06, P10
RocksDB-Integration	Real-System	P10 (SuRF)
SPEC CPU [Sta17]	CPU-Suite	P23, P24, P27, A18
CloudSuite [FAK ⁺ 12]	Scale-out-Server	P24, P27
IBM DB2	Produktions-DBMS	P22
mimalloc-bench [Lc21]	Allokator-Suite	A04, A07, A09, A10, A18
Larson / threadtest / sh-bench	Allokator-Klassiker	A01, A03, P08, P30
LADDIS	Kernel-Allokator	A02
AggSum / Memory-Stream	Bandbreite	P32
Custom-Microbenchmarks	je Veröffentlichung (Zipf/Uniform)	P01, P04, P06, P11–P14, P21, P28

Tabelle 3.6: Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)

LP	Kennung	Op-Mix	Verteilung	neg%	Abgeleitet aus
LP01	bulk-insert-dense	100/0/0/0/0	uniform-seq	0	P01, P02, P03, P05, P12
LP02	bulk-insert-sparse	100/0/0/0/0	uniform-rand	0	P01,P02,P08
LP03	insert-value-length-sweep	100/0/0/0/0	uniform-seq	0	P01,P08
LP04	static-read-positive	0/100/0/0/0	uniform	0	P02, P05, P11, P12, P16, P25
LP05	static-read-zipfian	0/100/0/0/0	zipfian(.99)	0	P10,P20,P25,P28
LP06	static-read-negsweep	0/100/0/0/0	uniform	0–100	P04,P06,P22
LP07	static-read-string-corpus	0/100/0/0/0	real-corpus	50	P04,P06,P10
LP08	range-scan-heavy	0/0/0/100/0	uniform-seq	0	P01,P10,P12,P32
LP09	insert-lookup-5050	50/50/0/0/0	uniform	0	P03,P29
LP10	delete-heavy-post-build	50/0/50/0/0	uniform-rand	0	P01,P08
LP11	mixed-ycsb-oltp	25/50/0/0/25	zipfian(.99)	0	P10,P19,P20
LP12	concurrent-rmw-stress	25/9/36/9/9	uniform	0	P07,P29
LP13	concurrent-read-scaling	0/100/0/0/0	uniform	0	P07,P08,P25,P28
LP14	dynamic-realistic-trace	9/91/0/0/0	tpcc-nonuniform	0	P19,P20

Op-Mix = Anteile von Einfügen/Lookup/Löschen/Scan/RMW; neg% = Anteil negativer Lookups.

Gegenprobe innerhalb des Map-Konzepts — als zwei wählbare T0-Bausteine, von denen je Permutation genau einer gewählt wird und die beide permutiert werden: ein Open-Addressing-Hash nach Knuth (S14, implementiert) und die SwissTable [Goo18] (S22, als skalare Re-Implementierung des Swiss-Table-Prinzips vorhanden, standardmäßig abgeschaltet). Die Gegenprobe erfüllt das Punktsuche-Subset des `std::map`-Interface (`find/insert/erase`) als gewählte Hülle über die kartesisch konstruierbaren Gesamt-Suchalgorithmen; die ordnungsbasierten Operationen entfallen prinzipbedingt, da eine Hash-Tabelle ungeordnet ist. Über dieses Subset belegt sie ein limitiertes Achsen-Subset: Allokation, Prefetch, Nebenläufigkeit und Speicher-Layout bleiben geteilt — und auf der System-Seite ist das SIMD-Gruppen-Lookup der Vorlage [Goo18] an der SIMD-Erweiterung vorgesehen, die Nachbildung prüft die Control-Byte-Gruppen skalar —, während die baum- und trie-spezifischen Achsen nicht wegfallen: die Pfadkompression (T3) führt dafür die Durchreich-Variante (`None`), der Knotentyp (T4) besitzt im Code keine `None`-Variante und wird regulär mit einem der vier ART-Knotenformate belegt, das der

Hash-Baustein für seine Suche nicht nutzt, und die geordnete Bereichsiteration ist keine Organ-Achse, sondern die additive Scan-Fähigkeit der Modulgrenze (Anhang F), die der Hash-Baustein nicht anbietet; die Gegenprobe erfordert so kein eigenes Konzept.

Die Veröffentlichungen entstammen sechs thematischen *Herkunfts-Clustern*, die hier nur der ganzheitlichen Einordnung jedes Lebewesens dienen, bevor es in seine Organe zerlegt wird. Die *Trie-Familie* umfasst ART (P01) [LKN13], HOT (P02) [BZP⁺18], Masstree (P03) [MKM12], CoCo-Trie (P04) [BFTV24], START (P05) [FJKN20] sowie die succinct-Tries LOUDS (P09) [Jac89] und SuRF (P10) [ZLL⁺18]. Die *Hybrid- und B⁺-Familie* vereint den B²-Baum (P06) [SSL⁺21], Wormhole (P07) [WNJ19] und die cache-sensitiven CSS-, CSB⁺- und Hankins-Bäume (P11–P13) [RR99, RR00, HP03]. Die *Layout-Theorie* reicht von Samuel (P14) [SPB05] und Graefe/Larson (P15) [GL01] über die cache-oblivious Entwürfe von Bender et al. (P16/P17) [BDFC00, BDFC05, BDFC02, BCDFC02, BCD⁺02] bis zu den layout-invarianten Bäumen von Saikkonen/Soisalon-Soininen (P18/P19) [SSS08, SSS16]. Das *Prefetching* umfasst *B-Trees Are Back* (P20) [MBL25], die Prefetching-B⁺-Bäume von Chen et al. (P21/P22) [CGM01, CGM02], Khan (P23) [Kha11], Naderan-Tahan (P24) [NTSA16], Mahling (P25) [MWR25] sowie Zhang (P26/P27) [ZSZ⁺24, ZGH⁺25] und Kuehn (P28) [Kue23]. Die *Synchronisation* umfasst das optimistische Lock-Coupling von ART (P08) [LABN16], RCU (P29) [McK01] und Hazard-Pointer (P30) [Mic04a]; die hardware-nahe *TU-Dresden-Linie* schließlich Ungethüm (P31) [UHL⁺17], Schmidt (P32) [SKK⁺25] und VAMPIR (P33) [BH23]. Der parallele *Allokator-Korpus* (A01–A23) spannt von den Klassikern dmalloc [Lea96], Hoard [BMBW00], Slab samt Magazines-/Vmem-Erweiterung [Bon94, BA01] und Michaels lock-freier Allokation [Mic04b] über die mehrkern-skalierbaren mimalloc [LZM19], jemalloc [Eva06], TCMalloc [GM05], sncmalloc [LDPB19], scallocc [AKLS15], rpmalloc [Jan17] und lrmalloc [LR18] bis zu den cache-aware/vorhersagbaren CAMA [HBHR11], NUMAllocc [YZZ⁺23], StarMalloc [RFP24] und PIM-malloc [VIA26]. Abbildung 3.2 verdichtet die wichtigsten rekonstruierten Suchstrukturen zu einer Galerie — je Verfahren seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet; alle erscheinen so als Punkte im gemeinsamen Entwurfsraum.

ART (P01) Trie adaptive Knoten 4/16/48/256, byte- weise Verzweigung, Pfadkompression Achsen T4, T3	HOT (P02) Trie höhenoptimierter dynamischer Span, kompakte SIMD-Knoten Achsen T4, T0	Masstree (P03) Trie + B ⁺ Trie aus B ⁺ -Bäumen, 8- Byte-Slices, B ⁺ -Blattknoten Achsen T4, T10	CoCo-Trie (P04) Succinct kompakt-kollabierter Trie, succinct kodierte Pfade Achsen T3, T4
START (P05) Trie selbstoptimierender ad- aptiver Radix-Span Achsen T0, T4	B²-Tree (P06) Hybrid B ⁺ mit binären In- Knoten, cache-bewusst Achsen T4, T5	Wormhole (P07) Hybrid Trie + Hash + B ⁺ -Leaf, O(log L) Punktsuche Achsen T0, T2	LOUDS (P09) Succinct unäre Grad-Sequenz, Bitmap-kodierter Trie Achsen T9, T5
SuRF (P10) Succinct succinct Range-Filter über LOUDS Achsen T14, T9	CSS-Tree (P11) cache-sensitiv zeigerloser impliziter Baum, arithmetische Navigation Achsen T4, T5	CSB⁺-Tree (P12) cache-sensitiv B ⁺ mit einem Kind-Gruppen-Zeiger Achsen T4, T5	SwissTable Hash (Gegenprobe) SIMD-Control-Bytes; nicht- baumartig, im selben Achsen-System Achse T0 + SIMD (System)

Abbildung 3.2: Galerie der wichtigsten rekonstruierten Suchstrukturen: je Verfahren (mit Paper-ID) seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet. Trie-, Hybrid-, succinct- und cache-sensitive Verfahren stehen neben einer nicht-baumartigen Hash-Gegenprobe (SwissTable) — alle als Punkte im selben Achsen-Entwurfsraum. Die vollständige Achsen-Sezierung führt Tabelle 3.2.

Für jeden SOTA-Algorithmus (*State of the Art* — der Stand der Technik aus Abschnitt 2.1) und jeden Allokator hält ein Profil-XML die Achsen-Belegung fest: die Organ-Belegung (`node`, `traversal`, `value_handle`, `concurrency`, `allocator`, `prefetch`, `layout`, `reclamation`), die Prüfling-Build-Achse `page` und — getrennt davon, weil keine Organ-Belegung — die Ziel-ISA (`isa`, System-Sphäre) und die Telemetrie (`telemetry`, Mess-Tooling); dazu eine Schlüssel-Wert-Signatur und einen optionalen `<expected_workload>`-Tag, der die Mess-Heuristik überschreibt (Tabelle 3.7). Für jede der 33 Korpus-Arbeiten und jeden der 23 Allokatoren liegt ein Profil-XML mit `<expected_workload>`-Tag vor. Die Profile sind zweierlei Typs — der Typ ist im Profil-XML markiert und in der Prüflings-Registry der Cache-Engine übersetzungsstatisch verankert: 30 *Vollprofile* beschreiben eigenständige Suchentwürfe vollständig; drei *abstrakte Profile* gehen nur seziert ein — das optimistische Lock-Coupling von ART [LABN16] als Nebenläufigkeits-Organ, die succinct LOUDS-Repräsentation [Jac89] als Speicher-Layout-Organ (u. a. in SuRF [ZLL⁺18]) und das Profiling-Werkzeug VAMPIR [BH23] als Mess-Technologie. Ein abstraktes Profil stellt einige Organe selbst und überlässt die übrigen dem Standard-Baukasten; es ist derselbe Typ, dem der Prüfling PRT-ART entspricht (Abschnitt 3.5), und Vollprofile fremder Prüflinge sind auf demselben Weg eingliederbar. Der Entwurf sieht je Achse eine getrennte Permutation über *Profil-Ranges* vor — ein Wertebereich je Achse statt eines Einzelwerts; die persistierten Profile enthalten je Achse einen Einzelwert, die Bereichs-Permutation ist nicht implementiert.

Über alle 33 Profile hinweg verteilen sich diese Affinitäten auf $13 \times$ YCSB_C (leselastig), $10 \times$ YCSB_A (Zipf-verteiltes Lesen und Aktualisieren), $6 \times$ YCSB_E (Bereichs-Scans) und $4 \times$ YCSB_B (95/5 Lesen/Aktualisieren); PRT-ART selbst ist in seinem Prüflings-Profil mit YCSB_F (Read-Modify-Write, für die DensityTracker-Aktualisierungen) getaggt.

Der Kern-Befund dieser Lastprofil-Analyse ist ein systematischer *Bias*: Jede Veröffentlichung wählt das Lastprofil, unter dem ihr eigener Entwurf am besten abschneidet. Statische, read-only-orientierte Tries und Filter (HOT [BZP⁺18], CoCo [BFTV24], B²-Baum [SSL⁺21], SuRF [ZLL⁺18], CSS [RR99]) rahmen „build-once, 100 % positive Lookups“ und meiden Aktualisierungen; komprimierte Tries und Filter (CoCo, B²-Baum, SuRF) machen den Negativ-Anfrage-Anteil zur Primärachse, weil sie bei Misses früh abbrechen; zipfian-zentrierte Arbeiten (B-Trees Are Back [MBL25], Mahling [MWR25], Kuehn [Kue23]) begünstigen cache-bewusste Reordering-Layouts; schreib- und nebenläufigkeits-lastige Arbeiten (ART [LKN13], Masstree [MKM12], Wormhole [WNJ19], OLC-ART [LABN16], RCU [McK01]) treten umgekehrt nur dort an, wo statische Strukturen versagen; und prefetch- bzw. hardware-nahe Arbeiten (CSB⁺ [RR00], Chen [CGM01], Schmidt [SKK⁺25]) vermessen über Knotengröße und Prefetch eher Cache und TLB als die Algorithmus-Logik. Vierzehn der Veröffentlichungen steuern kein distinktes Lastprofil zum Katalog bei — teils Survey-, Theorie- und Hardware-Arbeiten ohne eigenes Index-Lastprofil, teils Arbeiten, deren Custom-Microbenchmarks in bereits katalogisierten Profilen aufgehen. Erst wenn *alle* Lastprofile gegen *alle* Lebewesen-Binaries (und die Container-Hüllen, Abschnitt 3.3.2) laufen — der Workload als dynamische Achse (Kapitel 5) —, lässt sich ein Vorteil, der nur unter dem selbstgewählten Lastprofil besteht, sichtbar machen und bias-reduzierte Vergleichbarkeit gewinnen.

Schließlich erscheinen auch die Güte-Kriterien wissenschaftlichen Messens (vgl. die Definitions-Klassen in Abschnitt 3.4) als konkrete Technologien. *Hardware-Performance-Zähler* (PMC, über `perf`) [Me110] sind nahezu universell und liefern Cache-Miss-Raten, dTLB-Misses, Branch-Misses und IPC/CPI — die Mehrzahl der Arbeiten P01–P28 berichtet solche Zähler —; *Profiler* wie OTF2/VAMPIR [BH23] und

Tabelle 3.7: SOTA-Profil (33: 30 Vollprofile, 3 abstrakte) mit Workload-Affinität

Profil	Paper-Ref.	Typ	expected_workload
art	P01 (Leis 2013)	voll	YCSB_C
hot	P02 (Binna 2018)	voll	YCSB_C
masstree	P03 (Mao 2012)	voll	YCSB_A
coco_trie	P04 (Boffa 2024)	voll	YCSB_C
start	P05 (Fent 2020)	voll	YCSB_C
b2tree	P06 (Schmeisser 2022)	voll	YCSB_E
wormhole	P07 (Wu 2019)	voll	YCSB_A
olc	P08 (Leis 2016)	abstrakt	YCSB_B
louds	P09 (Jacobson 1989)	abstrakt	YCSB_A
surf	P10 (Zhang 2018)	voll	YCSB_A
css_tree	P11 (Rao/Ross 1999)	voll	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	voll	YCSB_E
hankins	P13 (Hankins 2003)	voll	YCSB_C
samuel	P14 (Samuel 2005)	voll	YCSB_C
graefe	P15 (Graefe 2001)	voll	YCSB_C
bender_treelayout	P16 (Bender 2002)	voll	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	voll	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	voll	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	voll	YCSB_E
btreesareback	P20 (Mueller 2025)	voll	YCSB_E
chen_prefetch	P21 (Chen 2001)	voll	YCSB_A
chen_fractal	P22 (Chen 2002)	voll	YCSB_A
khan_adaptive	P23 (Khan 2010)	voll	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	voll	YCSB_A
mahling	P25 (Mahling 2025)	voll	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	voll	YCSB_A
zhang_asplos	P27 (Zhang 2025)	voll	YCSB_A
kuehn	P28 (Kuehn 2023)	voll	YCSB_C
rcu	P29 (McKenney 2001)	voll	YCSB_B
hazard_pointers	P30 (Michael 2004)	voll	YCSB_B
ungethuem	P31 (Ungethuem 2017)	voll	YCSB_C
to_stride	P32 (Schmidt 2025)	voll	YCSB_C
vampir	P33 (Berthold 2023)	abstrakt	YCSB_B

expected_workload = YCSB-Workload-Klasse (A/B/C/E/F); Typ: voll = Vollprofil eines eigenständigen Suchentwurfs,
abstrakt = abstraktes Profil, das nur seziert eingeht.

VTune [Rei05] erheben trace-basierte Laufzeitprofile, *Simulatoren* wie gem5 [BBB⁺11] liefern hardwaregenaue Schätzungen, wo reale Zähler fehlen (so P25 Mahling [MWR25], P27 Zhang [ZGH⁺25]), und das *HdrHistogram* [Ten12] erhebt die Latenz-Perzentile (p50/p95/p99). Auf der Erhebung baut die Auswertung auf: Statt arithmetischer Mittel werden Median und Perzentile berichtet, Unterschiede mit dem verteilungsfreien *Mann-Whitney-U-Test* [MW47] (mit Holm-Korrektur der Familienfehlerrate [Hol79]) auf Signifikanz geprüft und über das Effektmaß *Cliff's δ* [Cli93] quantifiziert, den Empfehlungen für wissenschaftliches Benchmarking [HB15] folgend. Eine eigene methodische Linie bildet die hardwarenahe Messmethodik der TU Dresden (Habich-Gruppe): Ungethüm et al. [UHL⁺17] systematisieren die Hardware-Optimierungen für Datenbank-Maschinen, Schmidt et al. [SKK⁺25] vermessen Stride-gegen Sequenz-Zugriffe auf HBM mit dTLB- und L2/L3-Zählern, und VAMPIR [BH23] profiliert nicht-funktionale Eigenschaften heterogener Speicher. Aus all diesen Technologien ergibt sich der Satz der Mess-Kategorien — Cache-Line-Auslastung, Cache-Misses (L1/L2/L3), dTLB-Misses, Speicher-Fußabdruck, Branch-Misses, IPC/CPI, Latenz (Mittel und Perzentile p50–p999), Durchsatz, Energie und

Fill-Buffer-Auslastung —, deren zeit- und observer-basierte Teilmenge der eigene Apparat bereits auflöst (Wall-Clock, HdrHistogram in der nachgelagerten Auswertung, Per-Achsen-Observer, die Statistik-Triade aus Median/Perzentilen, Signifikanztest und Effektmaß, die Zwei-Phasen-Operationsschleife und das Konformitäts-Gate gegen `std::map`), während die zählerbasierten PMC-Kategorien im Mess-Pfad verbindlich eingebaut sind und ihre Belegung je Mikroarchitektur an einen Ereignis-Katalog gebunden ist (Abschnitt 3.4.1); die vollständige Mess-Architektur entwickeln das Mess-System (Abschnitt 3.4) und Kapitel 5.

3.3 Lösungskonzept: das Anatomie-Modell

Aus der dialektischen Aneignung des Standes der Technik ergibt sich das Lösungskonzept dieser Arbeit; dieser Abschnitt entwickelt es als *Anatomie-Modell* und definiert dabei die Begrifflichkeiten und konzeptionellen Spezialkonstrukte, auf denen die folgenden Abschnitte aufbauen.

3.3.1 Achsen-Komposition und die Lebewesen-Metapher

Die zentrale Idee ist, einen Algorithmus nicht als Monolith zu behandeln, sondern als *Komposition orthogonaler Achsen* — jede Achse eine Teilentscheidung eines Cache-bewussten bzw. System-bewussten Index (wie Seiten angeordnet werden, wie Knoten verzweigen, wie Speicher alloziert wird). Ein konkreter Algorithmus ist dann ein Punkt im kartesischen Produkt aller Achsen, ein Stand-der-Technik-Entwurf eine explizite Konfiguration. Entscheidend ist die *modulare Austauschbarkeit*: Eine neue Achsen-Variante lässt sich isoliert untersuchen — ihr Effekt je Achse *und* für den ganzen Algorithmus messen —, ohne die übrigen neu zu implementieren; genau das löst das Trennbarkeits-Problem, da der Beitrag jedes Bestandteils bei sonst fixierten Achsen isoliert messbar wird. Dass diese Austauschbarkeit modular ist, zeigt sich im Code: Die Referenz-Kompositionen von ART und PRT-ART unterscheiden sich in genau einer Achse, der Pfadkompression (Abschnitt 3.5). Diese Zerlegung beruht auf der Beobachtung gemeinsamer Belange unterschiedlicher Suchalgorithmen und Veröffentlichungen, sowie der Vergleichbarkeit der Ziele unterschiedlicher Suchalgorithmen und kann daher durch eine Metapher getragen werden. Erstens teilen Suchalgorithmen im großen Bild dieselbe *Baum-Anatomie* in unterschiedlicher Ausprägung — von ausgewogenen Suchbäumen über Tries und Radix-Bäume bis zu eigenentwickelten Konstrukten; selbst eine sortierte Schlüssel-Wert-Liste ist streng genommen ein einzelnes Blatt eines Baumes, dessen Werte über Verweise weiter auf Daten-Blätter zeigen. Die Haupt-Differenzen zwischen den Algorithmus-Ausprägungen belaufen sich dadurch auf die Auffächerung der Knoten, die Baum-Tiefe, die Baum-Filter und Regelwerke, und die Implementierungs-Details der Serialisierung von Speicherseiten, Such-Seiten und Baum-Knoten. Die Baum-Struktur ist damit universell über alle Suchalgorithmus-Architekturen hinweg und bildet die Zeichenfläche, auf der die Achsen platziert werden.

Nicht-baumartige Suchverfahren — etwa die Hash-Gegenprobe aus Abschnitt 3.2 — bleiben dasselbe Konzept mit eingeschränktem Profil, denn ein eigenes Konzept entstünde erst, wenn sich der Basis-Achsen-Satz grundlegend unterschiede: Auch eine Hash-Tabelle ist ein Baum, der von der Wurzel seines Interfaces über eine Auffächerung an Hashes sucht — nur der Kern-Algorithmus des Knotens ist ein anderer. Deshalb gehört jedes Verfahren, das einen Schlüssel in einen Wert übersetzt, gleich wie, zum Konzept Map und seiner Gattung *SearchAlgorithm* (Abschnitt 3.3.2). Zweitens fasst eine anschauliche Metapher einen konkreten Algorithmus als *Lebewesen* auf, dessen *Organe* die Achsen sind. Die Metapher ist

kanonisch besetzt: Das *Blut* ist das flüssige Organ, das alle Organ-Achsen verbindet und im Planer zu einer Übersetzungszeit-Konfiguration verdichtet wird; die System-Achsen sind der *Lebensraum* (der Planet) des Tiers mit den Möglichkeiten und Ressourcen, die sich dort finden; die Mess-Achsen sind die Ausstattung des *Labors*, in dem das Tier untersucht wird (Abbildung 3.3). Ein Tier kann ohne Messung sehr gut leben, aber nur das gesündeste Tier soll in den produktiven Einsatz gegeben werden — und jedes Tier soll bestmöglich an seinen Lebensraum angepasst sein: Besitzt es eine Eigenschaft nicht, kann es diesen Teil seines Lebensraums (die Hardware) nicht zu seinem Vorteil nutzen; oder die Messung ergründet, dass seine Spezialisierung gar nicht auf diesen Lebensraum ausgelegt sein sollte, weil sie die Qualität des Tiers dort negativ beeinflusst — ein Affe soll klettern und ein Fisch schwimmen, nicht umgekehrt, weil unterschiedliche Zielstellungen zu unterschiedlichen Entwürfen geführt haben. Diese Metapher ist dem Hintergrund des Autors in der Biomedizinischen Technik geschuldet und soll die Analyse durch Bildhaftigkeit vereinfachen; ab hier arbeitet die Arbeit durchgängig mit diesem Modell.

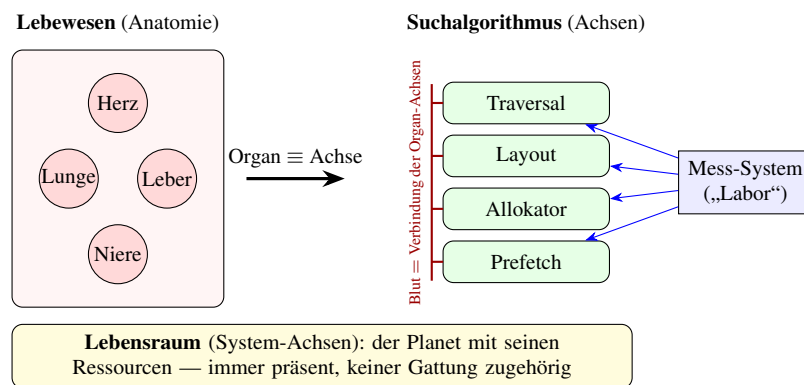


Abbildung 3.3: Die Anatomie-Brücke: wie ein Lebewesen aus zusammenwirkenden Organen besteht, ist ein Suchalgorithmus eine Komposition von Achsen-Organen; das Blut verbindet die Organ-Achsen, der Lebensraum (die System-Achsen) trägt das Tier, und das Mess-System übernimmt die Rolle des Labors, das jedes Organ einzeln vermisst.

Die Permutation der Achsen ist in dieser Lesart ein Organ-Austausch: Der Austausch von Organen zwischen Lebewesen erzeugt neue Komposit-Algorithmen mit bisher unbekanntem Cache-Line-Verhalten. Die Organe selbst werden aus dem Stand der Technik gewonnen — die Entwurfsbausteine der bekannten Verfahren (Abschnitt 2.1) werden in orthogonale Achsen zerlegt und in einer gemeinsamen Achsen-Matrix gesammelt, sodass jede Achse die Beiträge des gesamten Korpus erntet (Abbildung 3.4).

Die so gewonnene *Anatomie* ist mehr als die Aufzählung der Organe: Sie ist ihre *Verdrahtung*. Jedes Organ exportiert ein generalisiertes Interface, über das die übrigen seine Dienste nutzen, ohne dessen konkrete Belegung zu kennen — so stellt das Allokations-Organ ein gemeinsames Allokations-Interface bereit, das Knoten-Layout, Wert-Ablage und Serialisierung für ihre Speicheranforderungen verwenden (Abbildung 3.5); der Hub dieser Verdrahtung ist im Code das Speicher-Organ `LayoutAwareChunkedStore`, das Knotentyp, Layout und Allokator komponiert („Speicher-Organ-Interface“ ist der Konzeptname, kein Code-Typ). Die Nutzungs-Regeln sind weit gefasst: Eine Achse darf fremde Achsen-Interfaces nutzen, sich selbst rekursiv aufrufen und andere Gattungen verwenden — die Such-Gattung greift in ihren Interface-Funktionen auf Achsen zu und verwendet eigene Container-Gattungen, und Teile des Achsen-Pools werden für die Container wiederverwendet (Abschnitt 3.4). Software-technisch ist dies die *Feature-Interaktion* feature-orientierter Software-Produktlinien [ABKS13, BO92].

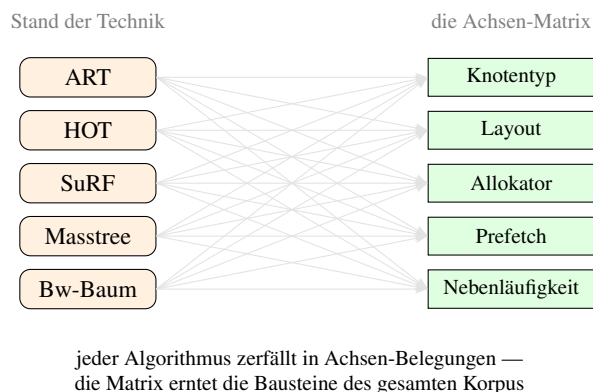


Abbildung 3.4: Synthese Stand der Technik → Achsen: die Entwurfsbausteine der bekannten Verfahren werden in orthogonale Achsen zerlegt; jede Achse sammelt die Beiträge des gesamten Korpus — die Umkehrung von Abbildung 3.1 (dort ist ein Algorithmus ein Punkt, hier eine Achse ein Sammelbecken).

Eine Achse entspricht einem Feature, ein Lebewesen einer Feature-Komposition. Das *Konzept* ist die Feature-*Kern*-Komposition: der gemeinsame Interface-Kern, den alle Gattungen eines Konzepts teilen und der in der Metaprogrammierung über alle Variationen hinweg gleich verwendet wird — so wie verschiedene Map-Kategorien (Hash-Map, flache Map) einen gemeinsamen Kern an Interface-Funktionen teilen; für das Container-Konzept besteht dieser Kern aus Identität, Achsen-Beobachtung und Größe. Die *Gattung* (Abschnitt 3.3.2) ist die Feature-*Spezial*-Komposition: das reale Tier-Interface über den Kern hinaus — SearchAlgorithm, Set, Sequence, Adapter, View und FunctionInterfaceReroute — und damit eine statische Feature-Set-Definition; das *Genus* ist ein konkretes Produkt dieses Feature-Sets, die Implementierung des Gattungs-Interfaces mit ihrer Achsen-Belegung. Die Anatomie ist die Verdrahtung zwischen den Features. Welche Achse welche nutzt, zeigt der Inter-Organ-Nutzungs-Graph (Abbildung 3.6).

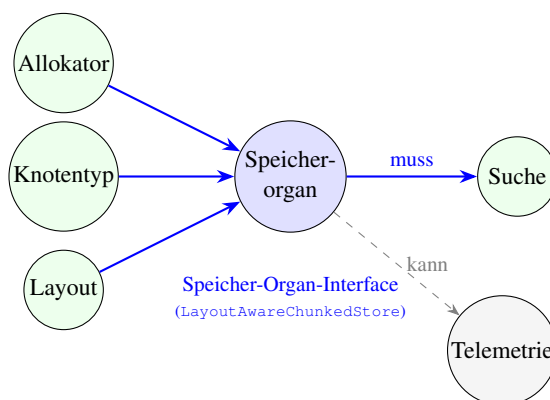


Abbildung 3.5: Achse = Organ, Anatomie = Verdrahtung: das Speicher-Organ (Komposition aus Knotentyp, Layout und Allokator) exportiert ein generalisiertes Speicher-Organ-Interface (im Code `LayoutAwareChunkedStore`), über das die Suche läuft (durchgezogen, verpflichtend); weitere Organe wie die Telemetrie können es über optionale Beobachter-Hooks nutzen (gestrichelt, optional), müssen aber nicht.

Setzt man diese paarweisen *Organ-Kopplungen* — eine Organ-Kopplung ist die wechselseitige Abhängigkeit zweier Achsen, bei der die eine das Interface der anderen nutzt (verpflichtend oder optional) oder deren

zulässige Belegungen einschränkt; Teilfrage FF1.b (Abschnitt 1.3) spricht sie für Knotentyp, SIMD und Pfadkompression an — über alle Organe zusammen, entsteht ein gerichteter Inter-Organ-Nutzungs-Graph.

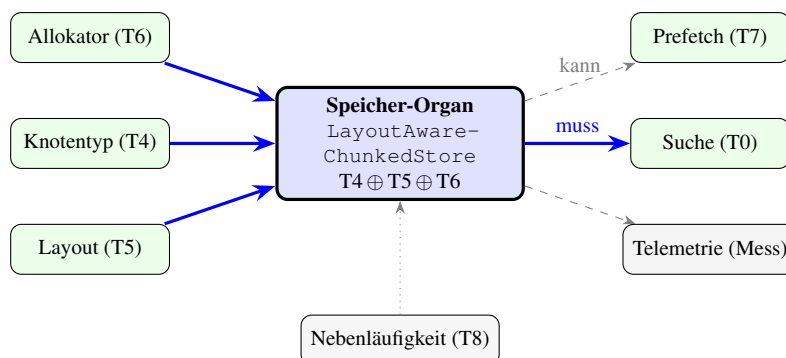


Abbildung 3.6: Inter-Organ-Nutzungs-Graph („welche Achse nutzt welche“): Es gibt genau *einen* kanonischen Hub — das Speicher-Organ `LayoutAwareChunkedStore`, das Knotentyp (T4), Layout (T5) und Allokator (T6) komponiert. *Verpflichtend* (durchgezogen) konsumiert der Store diese drei Achsen, und die Suche (T0) läuft über ihn. Alle übrigen Kopplungen sind *optional* (gestrichelt): Prefetch (T7) und die Telemetrie (Mess-Instrumentierung, Abschnitt 3.4.1) lesen den Store nur über `requires`-gegate Beobachter-Hooks — eine Achse *kann* fremde Interfaces nutzen, *muss* aber nicht. Die Kopplung der Nebenläufigkeit (T8) an den Store ist nicht angebunden (gepunktet). Der Graph ist eine Beispiel-Verdrahtung des Speicher-Organs, keine im Code ausgewertete Struktur.

3.3.2 Das Drei-Ebenen-Modell und die vier Konzepte

Diese Organe sind in *eine* Architektur eingebettet — eine einzige, in sich geschlossene Hierarchie ohne Parallel-Bäume. An ihrer Wurzel steht die `ExecutionEngine` als gemeinsame, ausmessbare Abstraktion über allem Messbaren — sie bleibt bestehen, weil `IExecutionEngine` die gemeinsame messbare Wurzel bildet und `IAnatomyBase` die anatomie-tragende Sicht unmittelbar darunter; darunter lassen sich drei Interface-Ebenen unterscheiden: das *Konzept* als nach außen sichtbare Mess-Surface (die Tierklasse mit ihrem *Prüf-Dock*), darunter die *Gattung* als Tier-Interface, darunter das *Genus* als Implementierung dieses Interfaces; die *Organe* bilden unterhalb des Genus eine eigene Klassifikation (Abbildung 3.7). Die Ebenen-Trennung ist an der ABI-Grenze ablesbar: Jede Tier-Binary exportiert zwei buchstabengleiche C-Symbole, die Konzept und Gattung ohne Instanz beantworten, wobei das Konzept nicht als zweites Datum gepflegt, sondern zur Laufzeit aus der Gattung ermittelt wird — der Lader ermittelt das Konzept so über ein konzeptunabhängiges Symbol, ohne es vorab zu kennen. „Lebewesen“ und `SearchAlgorithm` bezeichnen denselben Gegenstand — Metapher und technischer Begriff —; die Anatomie ist kein zweites Modell, sondern der *Körper* dieses Lebewesens: die feste Komposition seiner achtzehn Achsen-Organe.

Auf der obersten Ebene stehen vier Konzepte. Das Map-Konzept — das schlüssel-wert-orientierte Prüf-Dock und der Fokus dieser Arbeit — besitzt als einzige Gattung `SearchAlgorithm` den vollen Achtzehn-Achsen-„Elefanten“; das `Container`-Konzept (schlüsselloser Speicher) fächert in die Gattungen `Set`, `Sequence`, `Adapter` und `View` auf, die ebenfalls aus Achsen des gemeinsamen Pools gebildet werden und deren Slot-Zahl je Gattung übersetzungsstatisch festliegt; das `Graph`-Konzept ist als Inter-

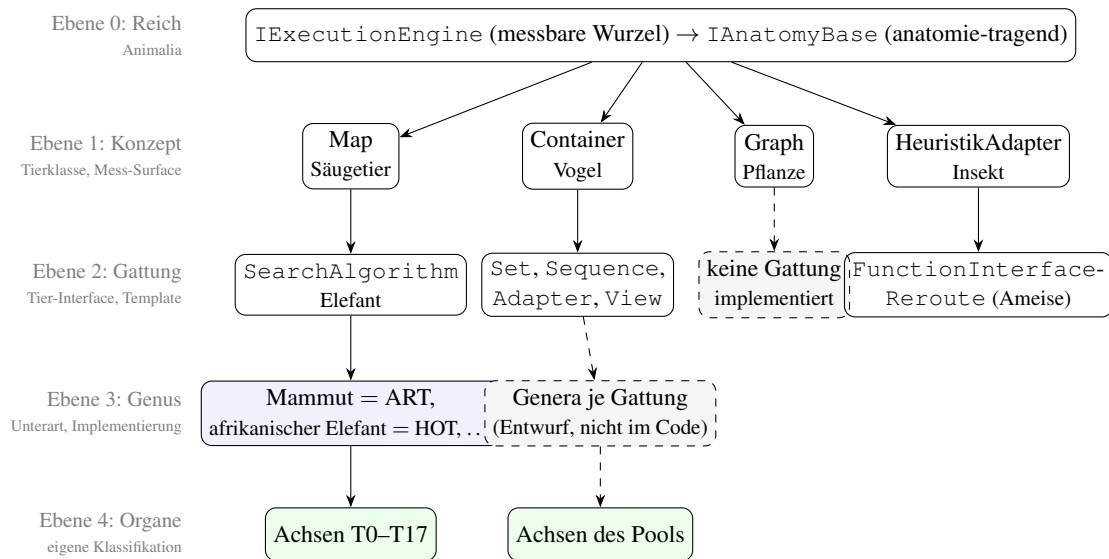


Abbildung 3.7: Das Drei-Ebenen-Modell: die drei Interface-Ebenen Konzept (Ebene 1, Tierklasse, Mess-Surface: Prüf-Dock) → Gattung (Ebene 2, Tier-Interface, Template nach außen) → Genus (Ebene 3, Unterart: Implementierung des Gattungs-Interfaces, ggf. mit erweiterten Spezial-Interfaces); darunter die Organe/Achsen (Ebene 4, eigene Klassifikation), darüber das Reich (Ebene 0). Vier Konzepte: Map (Säugetier) besitzt als einzige Gattung SearchAlgorithm (Elefant), deren Genera die konkreten Implementierungen sind (Mammut = ART, afrikanischer Elefant = HOT, indischer Elefant = START); Container (Vogel) die Gattungen Set/Sequence/Adapter/View (Specht/Storch/Kakadu/Falke), die aus demselben Achsen-Pool gebildet werden; Graph (Pflanze) ist als Interface angelegt, eine Gattung ist nicht implementiert; HeuristikAdapter (Insekt) besitzt die Reroute-Gattung FunctionInterfaceReroute (Ameise) des Hybrid-Tiers. Heute trägt der Code das Genus nur als Achsen-Belegung; die Genus-Implementierungsklasse ist ein erkannter Architekturfehler (Behebung nach der Verteidigung). Tier-Binary (Ebene 5a) und Hybrid-Tier (Ebene 5b) sind die Nachphase der Kompilation (Abbildung 3.8).

face angelegt, eine Gattung ist nicht implementiert; und das HeuristikAdapter-Konzept besitzt die Reroute-Gattung FunctionInterfaceReroute — die Gattung des Hybrid-Tiers (Abschnitt 3.6.2), das per Metaprogrammierung die Interfaces eines Konzepts samt Gattung erbt und Anfragen an die eigentlichen Tier-Binaries durchstellt; Klassifikation (welcher Art das Reroute ist) und Durchstellung (die geerbte Ziel-Gattung, die die Binary nach außen meldet) liegen dabei auf getrennten Ebenen (Abbildung 3.8). Merkregel: „Adapter“ ohne Präfix meint immer die Container-Gattung; das Hybrid-Konzept heißt stets HeuristikAdapter. Das dort als Wurzel gezeigte Reich „Animalia“ ($\equiv$ IAnatomyBase) ist die anatomie-tragende Sicht unmittelbar unterhalb der messbaren Wurzel IExecutionEngine aus Abbildung 3.7; die Code-Trennung der beiden Interfaces vertieft Kapitel 4. Nicht-baumartige Verfahren erfordern so kein eigenes Konzept, sondern belegen ein eingeschränktes Achsen-Subset derselben Gattung.

Die Anatomie wird von vier *Träger-Stufen* (carrier stages) getragen: dem Planer, dem *CacheEngineBuilder* (CEB) — dem Messwerkzeug, das der Planer je Ziel-Plattform und Mess-Aufgabe erzeugt —, der Tier-Binary und der optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobenen Hybrid-Tier-Binary. Der Planer löst die Anwender-XML gegen die drei Angebots-Kataloge auf und wählt zu seiner

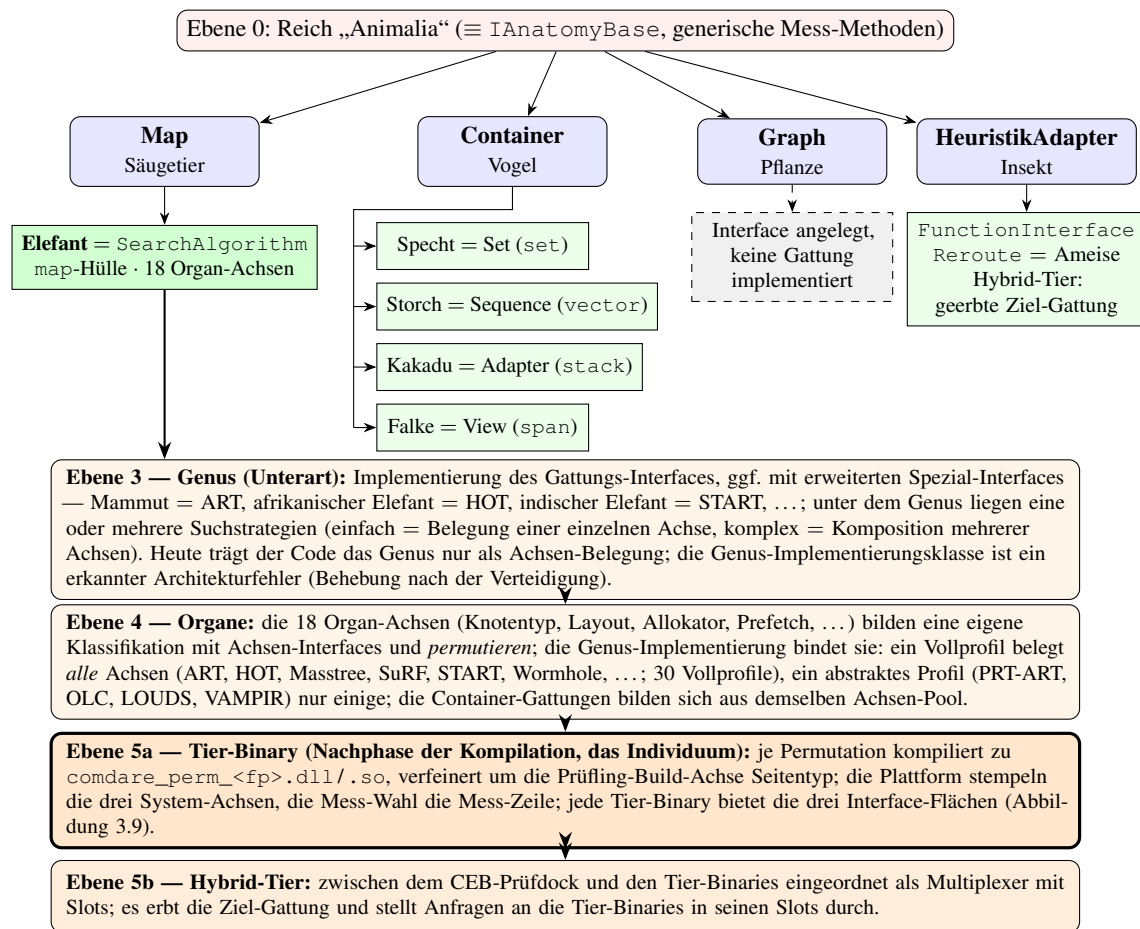


Abbildung 3.8: Die Konzept-/Tier-Hierarchie über die Ebenen 0 bis 5. Ebene 0 ist das Reich „Animalia“ (IAnatomyBase, die generischen Mess-Methoden). Map, Container, Graph und HeuristikAdapter sind die vier *Konzepte* (Ebene 1, die Tierklassen Säugetier/Vogel/Pflanze/Insekt: Mess-Surface und Prüf-Dock). Jedes Konzept beherbergt seine *eigenen* Gattungen (Ebene 2, die Tier-Interfaces): Map ausschließlich den „Elefanten“ (Gattung SearchAlgorithm, volle Achtzehn-Achsen-Anatomie), Container die Vogel-Gattungen Specht/Storch/Kakadu/Falke = Set/Sequence/Adapter/View, Graph ein angelegtes Interface ohne implementierte Gattung, HeuristikAdapter die Reroute-Gattung „Ameise“ des Hybrid-Tiers. Der Elefanten-Ast ist bis zum Individuum ausgezogen: die Genera (Ebene 3, Unterarten = Implementierungen des Gattungs-Interfaces, z. B. Mammot = ART, afrikanischer Elefant = HOT, indischer Elefant = START) binden die achtzehn Achsen-Organen (Ebene 4, eine eigene Klassifikation), die zu einer konkreten Achsen-Komposition permutieren; jede Permutation wird als eigenständige Tier-Binary kompiliert (Ebene 5a, die Nachphase der Compilation und das Individuum, zusätzlich verfeinert um die Prüfling-Build-Achse Seitentyp und plattform-gestempelt durch die drei System-Achsen, Abschnitt 3.4.1); das Hybrid-Tier (Ebene 5b) ist zwischen dem CEB-Prüfdock und den Tier-Binaries als Multiplexer mit Slots eingeordnet. *Jede Tier-Binary ist damit ein eigenes Tier mit einem tatsächlichen Binary-Gattungs-Interface und drei Interface-Flächen*; die Hierarchie reicht vom Reich bis zur einzelnen kompilierten Permutation. Vgl. das verdichtete Modell in Abbildung 3.7.

Laufzeit die Mess-Achsen; der CEB enthält diese Mess-Wahl einkompiliert, kennt die Plattform und permutiert System- und Organ-Achsen zu Tier-Binaries; die Tier-Binary enthält die Organ-Permutation; die Hybrid-Tier-Binary führt zusätzlich ihre angeschlossenen Tiere. Die Steuerung sitzt im Planer, und die Kette der Freigaben ist gerichtet: Die Mess-Achsen bestimmen den CEB-Typ, der CEB definiert die System-Achsen und gibt den Bau-Raum frei, und dieser Raum permutiert System- und Organ-Achsen zur Tier-Binary. Keine Stufe durchläuft dabei Schichten — jede enthält ihre Schicht fest einkompiliert; nur die Konfiguration wandert von der XML bis zur fertigen Tier-Binary. Die Interfaces der Träger folgen einem Drei-Flächen-Modell: Fläche 1 ist das Gattungs-Interface einer Tier-Binary (Laufzeit, Abstract Factory), Fläche 2 der Stempel (Übersetzungszeit, ABI-stabil, Abschnitt 3.4.4), Fläche 3 der Mess-Durchstich (IMessVisitor), der die Messung als eigene Naht quert, damit Konzept- und Gattungs-Interfaces unverändert bleiben. An Tier- und Hybrid-Tier-Binary sind alle drei Flächen ausgeprägt; Planer und CEB besitzen die Stempel-Fläche, und der CEB führt zusätzlich seine eigene Mess-Gate-Belegung, deren Deckung gegen die Tier-Gates (CEB aus, Tier an) durch einen Test belegt ist. Die Verträge zwischen Planer, CEB, Tier und Hybrid sind je Stufe lokal auf sieben Stempel-Interfaces verteilt — eine Gesamt-Selbst-Version führt nur der Planer, Mess- und System-Zeile führen CEB, Tier und Hybrid, die Organ-Zeile führen nur Tier und Hybrid (die Absenz beim CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“), Fingerprint und Gesamt-Stempel führen alle vier, und die angeschlossenen Tiere führt nur der Hybrid (Abbildung 3.9).

Über die Träger-Stufen läuft eine *Ablauf-Kette* aus vier Arbeitsmodi (work modes), die im Mess-Realm als Unter-Achse registriert ist und in fester Reihenfolge kumulativ durchlaufen wird: *build* baut die Binaries und misst nicht; *measure* ist der deterministische Ein-Thread-Mess-Lauf, dessen Ergebnis das Messwertlager füllt; *compare* liest das Messwertlager mit eigenen Optionen und errechnet, welche Tier-Binary oder Hybrid-Rekombination laut Vermessung die beste sein muss; *release* erzeugt daraus die voll optimierte Binary ohne Mess-Instrumentierung und misst ihre Wall-Clock nach. Auf der Hybrid-Ebene wiederholt sich dieselbe Kette synchron (Abbildung 3.10). Einen Debug-Arbeitsmodus gibt es nicht; die Bau-Varianten Debug, Messung und Release gelten fort, und die Debug-Variante wird weiter ausgebaut: *-debug* ist ein Schalter der Planer-Shell für den Verdrahtungs-Check — ein Flag und kein eigener Arbeitsmodus der Kette build/measure/compare/release; es schaltet den indirekten Debug-Modus der Debug-Variante ein (den nicht offiziellen fünften Wartungsmodus): unter *-debug* fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces (wie geplant: Tests nur mit den Debug-Binaries, Debug- und Release-Kompilat parallel), und dieser indirekte Debug-Modus fällt mit dem Release-Bau zusammen und wird nicht separat geführt; für Anwender bleibt er gesperrt (rc 8). Umsetzungsstand: Die vier Modi sind wählbar und validierbar und wirken auf den Bau der Binaries; der modus-spezifische Ablauf von compare (Replay-Vergleich statt Messen) und release (Erzeugung der optimalen Binary aus den Messwerten) ist für spätere Haupt-Wellen vorgesehen, aber nicht implementiert — compare und release sind wählbare, validierbare Etiketten mit release-gleicher Bausemantik.

3.3.3 Entwurfsmuster und Übersetzungszeit-Muster

Der eigentliche Beitrag liegt damit weniger in einer einzelnen Struktur als in einem *Werkzeug*: Statt das Vergleichsproblem direkt zu lösen, baut diese Arbeit zuerst dieses Werkzeug — den Permutations- und Mess-Apparat —, mit dem sich beliebige Achsen-Kombinationen erzeugen und fair vermessen lassen. Wissenschaftlich ordnet sich dieser Apparat in das Software-Engineering der *Komposition* ein:

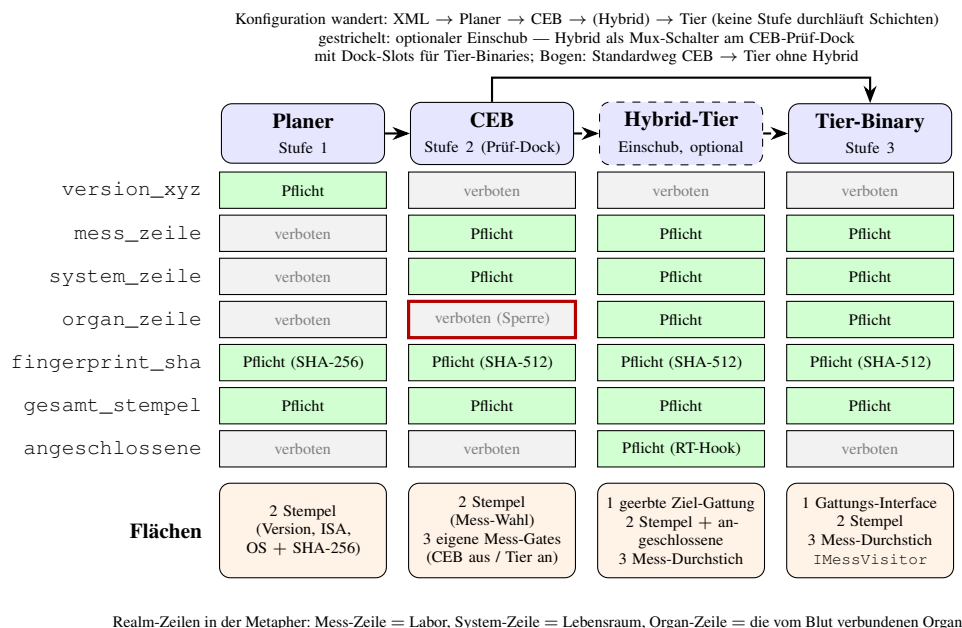


Abbildung 3.9: Träger-Stufen und Interface-Flächen: die dreiwertige Zulassungsmatrix der sieben Stempel-Interfaces über die vier Träger-Stufen Planer, CacheEngineBuilder (CEB), Hybrid-Tier-Binary (optionaler Einschub) und Tier-Binary (28 Zellen; grün = Pflicht, grau = verboten; die Zelle Organ-Zeile/CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“) und darunter die Interface-Flächen je Stufe — Fläche 1 Gattungs-Interface (Laufzeit), Fläche 2 Stempel (Übersetzungszeit, ABI-stabil), Fläche 3 Mess-Durchstich. Die Konfiguration wandert von der XML über Planer und CEB bis zur Tier-Binary; die Hybrid-Tier-Binary ist eine optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Stufe (gestrichelt: Mux-Schalter am Prüf-Dock mit Dock-Slots für Tier-Binaries; die Spaltenordnung der Matrix folgt der Verarbeitungs-Topologie mit dem Hybrid an seiner Einschub-Position, der Bogen ist der Standardweg ohne Hybrid); keine Stufe durchläuft Schichten, jede enthält ihre Schicht einkompiliert.

die invasive Software-Komposition aus Gray-Box-Komponenten [Aßm03], die generative Programmierung [CE00] und die feature-orientierten Software-Produktlinien [ABKS13, BO92]. Das Achsen-Konzept selbst stammt aus einer Vorarbeit des Autors unter der Marke Comdare, ursprünglich zur Deduplikations-Optimierung der UltiHash-Datenbank; ComdareDB ist deren Nachfolge-Produkt als Konzept ohne Code, keine weitere Entität. Der Beitrag dieser Arbeit ist die Übertragung des Prinzips auf cache-bewusste Suchstrukturen als kompilierte, maschinenoptimierte Binaries — seine design-pattern-getriebene, mess-orientierte Realisierung. Software-technisch ist der Apparat eine Komposition benannter Entwurfsmuster und zweier neuer Übersetzungszeit-Muster (Abbildung 3.11).

Die klassischen Muster sind dabei keine Zierde, sondern die Verkabelung des Apparats: *Strategy* kapselt jede der achtzehn Organ-Achsen als austauschbaren Algorithmus, *Director* und *Builder* emittieren den Plan (ein Walk, je Ausgabe-Syntax ein Builder), eine *Abstract Factory* konstruiert die Docks der Hybrid-Stufe und die Ergebnis-Mappen an genau einem Ort, ein *Iterator* durchläuft den lazy materialisierten Experimentbaum, *Adapter* bindet Fremd-Code hinter der stabilen Engine-Schnittstelle an, *Observer* erhebt die Mess-Snapshots je Achse, ein *Visitor* ist die Mess-Naht am dlopen-Übergang, *Chain of Responsibility* bildet die Hardware-Erhebungskette und die Auswahl-Filterkette, und ein *Memento* (Copy-on-Write) sichert den Zustand zwischen Warmlauf und Messung [GHJV94].

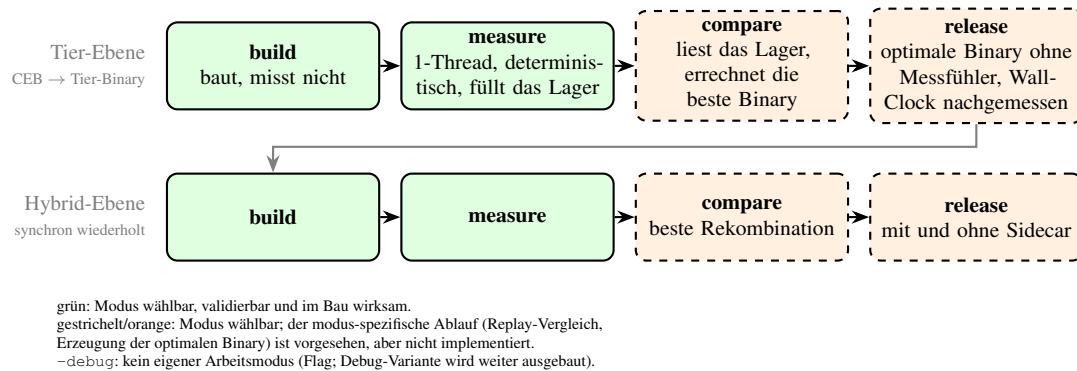


Abbildung 3.10: Die Ablauf-Kette der vier Arbeitsmodi build → measure → compare → release als registrierte Unter-Achse des Mess-Realms, kumulativ in dieser Reihenfolge und auf der Hybrid-Ebene synchron wiederholt; compare liest das Messwertlager und errechnet die beste Binary bzw. Rekombination, release erzeugt die voll optimierte Binary ohne Messfühler und misst ihre Wall-Clock nach. Ein Debug-Zustand existiert nicht; unter -debug fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces, ein indirekter Debug-Modus, der mit dem Release-Bau zusammenfällt. Die Zuordnung zu den Betriebs-Zielbildern Mess-, Auswertungs-, Arbeits- und Hybrid-Modus gibt Abschnitt 3.6.2.

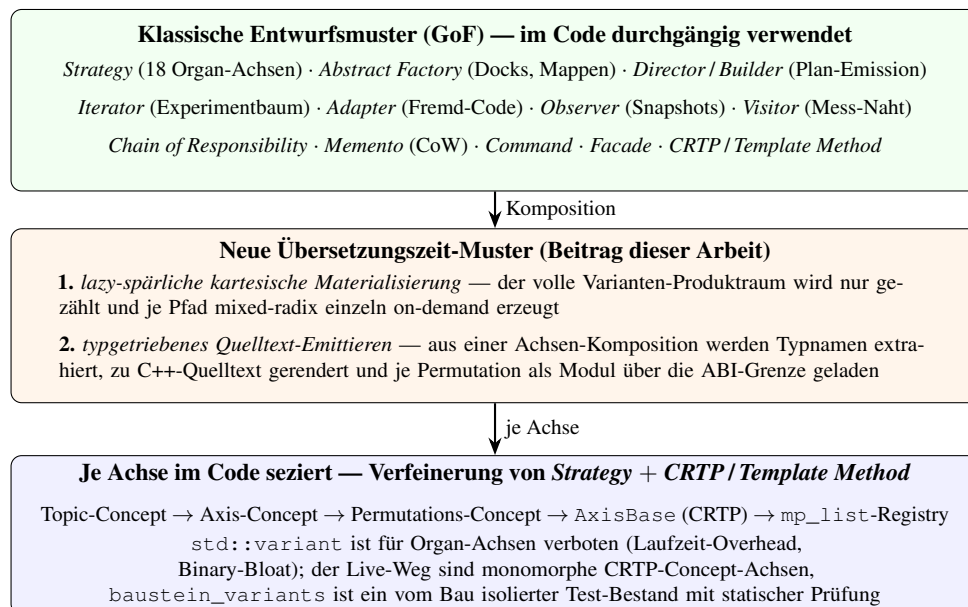


Abbildung 3.11: Entwurfsmuster als Bausteine der Architektur: oben die im Code durchgängig verwendeten klassischen GoF-Muster mit ihrer jeweiligen Rolle, in der Mitte die zwei in dieser Arbeit benannten neuen Übersetzungszeit-Muster, unten die code-seziert *Drei-Stufen-Concept-Kette* je Achse (Topic-Concept → Axis-Concept → CacheEngine-Permutations-Concept → CRTP-Basis → mp_list-Registry) als Verfeinerung von Strategy/CRTP; ein literaler std::variant tritt in keiner Organ-Achse auf (Verbot), sondern nur in einem vom Bau isolierten Test-Bestand. Ihre Komposition bildet die design-pattern-getriebene, mess-orientierte Architektur (Kapitel 4).

Die Mess-Hüllen der Achsen halten keine Komponenten-Instanz und kein Voll-Interface und sind ausdrücklich kein *Decorator*, und ein *Composite* im Sinne des Katalogs hat keine Entsprechung im Code. Nicht verwendet werden im Hot-Path und in den Organ-Achsen *Singleton*, *Flyweight* und *Bridge*: host-seitige Registries nutzen Meyers-Singletons (die eine Instanz als funktionslokales statisches Objekt) [Mey05], die Katalog-Erzeugung Flyweight-Tabellen je Achse, und „Bridge“ tritt im Code nur als umgangssprachliche Bezeichnung für Verbindungsstücke auf — in Kommentaren zu ABI-Adaptern der Gattungen, Plattform-Sonde, Code-Erzeugung, Permutations-Befehlen, CMake-Listen und Vergleichs-Pipeline, für die CI-Bridge-Jobs des Planers und die Mikroarchitektur Sandy Bridge sowie in einer Versionskonstante der Cache-Line-Naht —, nicht als Muster im Sinne des Katalogs. Das *Command*-Muster erfüllt mehrere Rollen — die übersetzungszeitliche Command-Basis der Achsen, die System-Komplex-Achse als Command-Wrapper und den typisierten Steuer-Befehlskanal —; der Befehls-Kanal des Hybrid-Modus, des Betriebs, in dem ein Heuristik-Tier zwischen vorgeladenen Tier-Binaries an der ABI-Grenze wechselt (Abschnitt 3.6.2), ist eine dieser Rollen, und eine *Facade* bündelt die Anhang-Erzeugung und die lesende Plan-Emission (Muster-Register mit Code-Orten: Abschnitt 4.5.1, Anhang E, ADR-11). Den statischen Polymorphismus liefert das *Curiously Recurring Template Pattern* (CRTP) samt *Template-Method*-Concept-Prüfungen; die Wahl zwischen Prüfling- und Cache-Engine-Standard je Achse trifft die Verbund-Strategie des Merge-Plans (Verbund1_CeOnly, Verbund2_Replace, Verbund2_Hybrid, Verbund3_Union; Abschnitt 3.4.6), die Achsen selbst sind monomorphe CRTP-Concept-Typen, und *Policy-based Design* fasst die Komposition als Tupel von Achsen-Policies [Ale01]. C++23-Module strukturieren dabei die Übersetzungseinheiten, garantieren für sich genommen aber noch keine portable ABI-Stabilität [Xu24] — diese stellt erst die C-stabile Modulgrenze her. Über die etablierten Muster hinaus benennt die Arbeit zwei Konstruktionen, für die es keine bekannte wissenschaftliche Bezeichnung gibt: die *lazy-spärliche kartesische Materialisierung*, die den kombinatorisch explodierenden Varianten-Produktraum aller Achsen-Kataloge nur arithmetisch zählt und je Pfad mixed-radix einzeln on-demand erzeugt, statt den Compiler mit dem vollen Typ-Baum zu sprengen; und das *typgetriebene Quelltext-Emittieren*, das aus einer compile-zeitlichen Achsen-Komposition voll-qualifizierte Typnamen extrahiert, zu C++-Quelltext rendert und je Permutation als eigenständiges Modul über die ABI-Grenze lädt — so wird der Prüfling PRT-ART per Metaprogrammierung in Matrix-Schichten gegen die Cache-Engine kompiliert.

Wie diese *eine* Architektur über eine ABI-stabile Hülle im Code sichtbar wird, vertieft Kapitel 4 (Abschnitt 4.3.1); zuvor ordnet der folgende Unterabschnitt den Apparat als Ganzes: in welchen Schichten er von der Experiment-Definition bis zur Auswertung organisiert ist und wie er fertiggestellt wird.

3.3.4 Fünf Interface-Schichten: E4 bis E0

Das Anatomie-Modell ordnet das *Gemessene*: was ein Lebewesen ist und aus welchen Organen es besteht. Quer dazu braucht der Apparat, der solche Lebewesen erzeugt und vermisst, ein zweites Ordnungsprinzip — eine Sicht auf die *Experiment-Pipeline*, die eine deklarierte Achsen-Belegung in gemessene Ergebnisse überführt. Diese Pipeline gliedert sich in fünf Schichten E4 bis E0, ergänzt um den Querschnitt M (Abbildung 3.12). Die Zählung läuft von der abstraktesten zur konkretesten Ebene. *E4* ist die Definitions- und Auswertungs-Schicht — die Ebene, die ein Anwender des Frameworks in der Regel verwendet: Experimente werden deklarativ als XML-Profile beschrieben, und am Ende der Pipeline nimmt dieselbe Schicht die Messergebnisse als Rückkanal entgegen und bereitet sie zu Tabellen und Diagrammen auf. *E3* ist der

Permutations- B^+ -Baum: Er entfaltet die deklarierte Belegung zum Experimentbaum, wobei die übersetzungsstatischen Achsen die Identität der zu bauenden Binary bestimmen und die dynamischen Achsen die Laufzeit-Dimension aufspannen. *E2* sind die Tier-Binaries selbst: je Permutation eine eigenständige, über die ABI-stabile Schnittstelle geladene Binary (Ebene 5a der Konzept-Hierarchie, Abbildung 3.8). *E1* ist die Laufzeit-Konfiguration — die präziseste Detail-Schicht, in der die dynamischen Unter-Achsen-Parameter an der bereits geladenen Binary variiert werden. *E0* schließlich ist die Infrastruktur (Bau-, Werkzeug- und Plattform-Unterbau); ihre Fertigstellung bildet den letzten Schritt der Reihenfolge. Die E-Kennungen sind eine Konzept-Ordnung der Pipeline, keine Code-Konstante; je Schicht ist ihr Code-Ort benannt: für *E4* die XML-Parse-Naht des Experiment-Eintritts und die Auswertungs-Generatoren des übergeordneten Projekts, für *E3* der Experiment-Plan-Direktor, für *E2* die ABI-Deklaration der Anatomie-Module, für *E1* die registrierten dynamischen Dimensionen des Mess-Realms und für *E0* die CMake-Konfiguration. Das Mess-System durchdringt als Querschnitt *M* alle Schichten, ohne selbst eine zu sein — diese Sonderrolle vertieft Abschnitt 3.4.

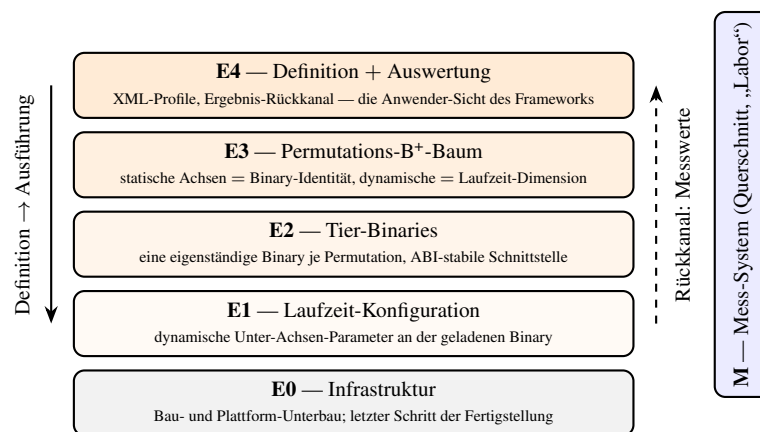


Abbildung 3.12: Die fünf Interface-Schichten der Experiment-Pipeline: von der deklarativen Definitions- und Auswertungs-Schicht *E4* (Anwender-Sicht) über den Permutations- B^+ -Baum (*E3*) und die Tier-Binaries (*E2*) zur Laufzeit-Konfiguration (*E1*); die Infrastruktur (*E0*) stützt alles; ihre Fertigstellung bildet den letzten Schritt. Der Rückkanal führt die Messwerte zur Auswertung nach *E4* zurück; das Mess-System durchdringt als Querschnitt *M* alle Schichten (Labor-Ausstattung, Abschnitt 3.4). Jede Schicht wird einzeln fertiggestellt, mit festgeschriebenem Interface-Vertrag zur Nachbar-Ebene.

Grundlegend ist das Fertigstellungs-Prinzip dieser Schichtung: Jede Schicht wird *einzeln* fertiggestellt, mit einem explizit *festgeschriebenen Interface-Vertrag* zur jeweiligen Nachbar-Ebene — die Grenze *E4*→*E3* etwa schreibt fest, wie eine XML-Definition in Achsen-Ebenen übersetzt wird, die Grenze *E3*→*E2*, wie aus einem Baum-Pfad ein Bau-Auftrag wird, und die Grenze *E2*→*E1*, welche Laufzeit-Stellgrößen eine Binary über die ABI entgegennimmt. Die Prüffrage je Grenze lautet stets: Reicht die Schicht ihre Definition *verlustfrei* an die nächste weiter? Ein System, dessen Produktraum jede vollständige Materialisierung ausschließt (Abschnitt 3.4.6), ist für gleichzeitige Änderungen über mehrere Ebenen zu groß; erst die festgeschriebenen Verträge machen jede Ebene einzeln abnehmbar, bevor die nächste beginnt. Dieses Prinzip korrespondiert mit der Matrix-Programmiersweise des Apparats (Abschnitt 3.3.3): So wie der Prüfling in Matrix-Schichten gegen die Cache-Engine kompiliert wird, wird der Apparat selbst in Interface-Schichten gegen festgeschriebene Verträge entwickelt; wie jede Schicht dabei einzeln — gegen einen Ersatz ihrer Nachbar-Ebene — geprüft wird, zeigt Abschnitt 4.4.

Die E-Schichten sind ausdrücklich vom Drei-Ebenen-Modell (Abschnitt 3.3.2) zu unterscheiden, so ähnlich beide auf den ersten Blick wirken: Das Drei-Ebenen-Modell ist die *Anatomie-Sicht* — es ordnet *ein* Lebewesen vom Konzept über Gattung und Genus zu den Organen und beschreibt damit die Struktur des Gemessenen. Die E-Schichten sind die *Pipeline-Sicht* — sie ordnen den Weg eines Experiments von der Definition über Baum, Binary und Laufzeit zurück zur Auswertung und beschreiben damit den Prozess des Messens. Beide Sichten stehen orthogonal zueinander: Dasselbe Lebewesen, das anatomisch in Konzept, Gattung, Genus und Organe zerfällt, wird durch die Schichten hindurch *konfiguriert* — seine Achsen-Belegung wandert von der XML über den Baum in den Bau-Auftrag, während jede Träger-Stufe ihre Schicht fest einkompiliert enthält (Abschnitt 3.3.2). Nachdem damit geklärt ist, was komponiert wird und in welchen Schichten der Apparat es verarbeitet, wendet sich der folgende Abschnitt der Frage zu, wie sich die so komponierten Lebewesen fair vermessen lassen.

3.4 Das Mess-System

Damit strukturell verschiedene Suchalgorithmen *fair* verglichen werden, bildet die Arbeit sie alle auf eine einzige, gut verstandene Schnittstelle ab: den C++-Vertrag der geordneten Map-Hülle `std::map<Key, Value>` mit Punktsuche (`find`), Einfügen/Aktualisieren (`insert`, `emplace`), Löschen (`erase`) und ordnungsbasierter Navigation (`lower_bound/upper_bound`, Bereichs-Iteration) [ISO24]. Die Achsen beschreiben das *innere* Verhalten hinter dieser festen Schnittstelle, sodass der Vergleich zweier konformer `std::map`-Implementierungen der Gegenstand der Messung wird; parallel dient die Sequenz-Hülle `std::vector` (indizierter Zugriff, Größenänderung, zusammenhängender Speicher) als Container-Vergleichsfall, um zu prüfen, ob ein Achsen-Komposit auch die Container- und nicht nur die Suchsemantik erfüllt. Hinter der festen Außen-Schnittstelle unterscheidet das System mehrere *Ebenen der Dynamik*, die festlegen, *wann* eine Achsen-Entscheidung fällt: die oberste Trennung Übersetzungszeit gegen Laufzeit; die Gattung, die nur eine Teilmenge der Achsen belegt; die zur Übersetzungszeit in eine Binary einkompilierten Achsenalgorithmen (statische Rekombination); die dynamischen Unter-Achsen, deren Laufzeit-Parameter — die registrierten dynamischen Dimensionen Prefetch-Distanz, Pool-Budget, Stapelgröße, Inline-Schwelle, Thread-Zahl und Workload — ohne Neuübersetzung variiert werden; und die statischen Unter-Achsen als übersetzungszeitliche Auswahl unter Teil-Algorithmen unterschiedlicher Komplexität. Darin liegt die Gliederung jeder Achse, die Teilfrage FF3.b (Abschnitt 1.3) anspricht: Die *Haupt-Achse* ist die übersetzungszeit-fixierte Wahl — bei Organ-Achsen ein Segment der `binary_id`, bei System-Achsen ein Stempel-Eintrag samt Bau-Versionskennung —, die *Unter-Achsen* sind ihre Verfeinerung, dynamisch als laufzeit-variable Parameter, die die Mess-Registry als dynamische Dimensionen der Stufe `runtime` führt, oder statisch als übersetzungszeitliche Tupel ihrer Haupt-Achse. Die Idee, Datenstrukturen nicht einzeln, sondern als Entwurfsraum zu betrachten, prägt Idreos' *Periodic Table of Data Structures* [IZA⁺18] und den *Data Calculator* [IZH⁺18]: Strukturen entstehen aus der Kombination weniger Entwurfs-Primitive, deren Kosten sich synthetisieren lassen — das deckt sich unmittelbar mit der Achsen-Sezierung dieser Arbeit (Abschnitt 3.2), in der die klassischen Familien bloße Lese-Gruppierungen orthogonaler Achsen-Belegungen sind. Gegenüber der statischen Entwurfsraum-Aufzählung des Data Calculator und dem klassischen *Policy-based Design* [Ale01] führt diese Taxonomie compile- und laufzeitvariable Achsen-Ebenen jedoch ausdrücklich gemeinsam; eine etablierte Benennung dieser Staffellung ist in der

gesichteten Literatur nicht belegt (vgl. Abschnitt 3.3.3).

Die Container-Gattungen werden aus demselben Achsen-Pool gebildet wie die Suchalgorithmen und sind keine Standard-Container — aus Notwendigkeit, nicht aus Bequemlichkeit: Nur so bleiben die im Kompilat einer Tier-Binary erlaubten Achsen kontrollierbar, nur so reproduziert die Mikro-Messung bei jedem Durchlauf dieselbe Laufzeit, und nur so bleiben die Eigenschaften von `SearchAlgorithm` und `Container`, die sich ähneln, über ein einheitliches Interface lückenlos vergleichbar; mit den Standard-Containern der C++-Bibliothek ginge diese Kontrolle verloren. Der gemeinsame Kern des Container-Konzepts besteht aus Identität, Achsen-Beobachtung und Größe (Abschnitt 3.3.1); die Gattungen `Set`, `Sequence`, `Adapter` und `View` sind implementiert. Die Hüllen `std::map(SearchAlgorithm)` und `std::vector(Container)` sind in der Implementierung vollständig aufzuführen; die Messung konzentriert sich auf die benötigten Interface-Funktionen, die Implementierung aller Funktionen ist vorgesehen. Die ordnungsbasierten und iterativen Operationen — `find` ist eine Elementfunktion des `std::map`-Vertrags [ISO24], Bereichs-Suchen sind es nicht — bündelt eine eigene Bibliothek über die Iteratoren der Gattungs-Interfaces, die alle kartesischen Produkte der Achsen zur Übersetzungszeit gleichermaßen abdeckt; diese Gattungs-Iterator-Bibliothek ist nicht implementiert.

Diese Bausteine sind nur so aussagekräftig wie ihre Vermessung. Wissenschaftliches Messen unterliegt etablierten Gütekriterien [HB15]: *Reproduzierbarkeit* (feste Seeds, deterministische Reihenfolge, dokumentierte Konfiguration, sodass Läufe methodisch wiederholbar und statistisch vergleichbar sind), *Validität* (intern: kausal isolierte Effekte; Konstrukt: die Metrik misst tatsächlich das gemeinte Konzept; extern: Übertragbarkeit auf andere Plattformen und Lasten), *Fairness* (identische Schlüssel, Workloads und Warmup; fehlende Fähigkeiten als *N/A* statt verdeckt emuliert) und die *Streuung* als Teil der Aussage (Perzentile statt arithmetischer Mittel, mit Signifikanzprüfung; die Verteilungssicht liefern HDR-Histogramme in der nachgelagerten Auswertung). Berichtete Perzentile folgen dem Nearest-Rank-Verfahren: der Rang $R(q, n) = \lceil q \cdot n \rceil$ ist einsbasiert, der zugehörige Feldindex $k = R - 1$ nullbasiert; der Median ist der Sonderfall $q = 0,5$ desselben Verfahrens und liefert bei geradem n die untere Mitte, nicht das Mittel der beiden mittleren Werte; p95, p99 und p999 sind dieselben Ränge mit $q = 0,95$, $q = 0,99$ und $q = 0,999$. Auf diesen Kriterien bauen die verwendeten Mess-Muster auf: eine *Zwei-Phasen-Operationsschleife* trennt einen verworfenen Erstlauf von der eigentlichen Messung — der Erstlauf ist der Warmlauf eines potentiell kalten Caches, sodass die Messung in einem definitiv warmen Cache startet, und der Parameter-Unterschied zwischen kaltem Erstlauf und warmer Messung ist selbst eine Erkenntnisquelle: Er weist einen tatsächlich zuvor kalten Cache nach und schätzt die Beschleunigung, die das Verfahren gegen einen kalten Cache erzielt; nur Zeilen mit gültiger zweiter Phase gehen in die Auswertung ein —; die Erhebung erfolgt *hybrid* — isolierte Achsen-Algorithmen direkt gegeneinander, ein zusammengesetztes Lebewesen zentral über eine ABI-stabile Per-Achsen-Beobachter-Schnittstelle —; ein *Konformitäts-Gate* prüft vor jeder Messung den `std::map`-Vertrag, damit nur Vergleichbares verglichen wird, und fährt unter `-debug` zusätzlich den Standard-Testsatz des jeweiligen Tier-Interfaces: den generischen Gattungs-Teil (gültig für alle Genera der Gattung) und den Genus-speziellen Teil (die gegebenenfalls erweiterten Spezial-Interfaces des Genus); und ein Plausibilitäts-Kriterium verwirft jeden Lauf, in dem eine Messung mit Messfühlern nicht langsamer ist als dieselbe ohne. Wo ein solches Kriterium eine Schwelle braucht, ist sie *plattform-kalibriert* — je Maschinen-Signatur aus Messläufen auf der jeweiligen Plattform bestimmt, nicht als feste Modellannahme gesetzt —; Umsetzungsstand: Die plattform-kalibrierten Schwellwerte liegen nicht vor; ihre Festlegung setzt die Messläufe voraus (Abschnitt 6.1). Diese Konzepte bilden die

Definitions-Klassen des Messens; ihre technischen *Instanzen* — Hardware-Performance-Zähler, OTF2-/VAMPIR-Profilung, die TU-Dresden-Hardware-Messmethodik — hat Abschnitt 3.2 bereits als konkrete Technologien eingeführt.

Die Mess-Tooling-Haupt-Achse spannt drei *Mess-Ebenen* (measurement levels) auf, in genau dieser und keiner anderen Reihenfolge: *wallclock* (w) — die Wall-Clock der Gesamtläufe im CacheEngineBuilder über die Last-Rahmenwerke; *macro* (ma) — die Zeit der Gattungs-Interface-Funktionen der Tier-Binary; *micro* (mi) — die Zeit und die Erfolgs-Parameter je Achsen-Interface. Teilmengen der drei Ebenen sind je Stufe erlaubt, jede andere Reihenfolge ist syntaktisch falsch; w/ma/mi ist der registrierte Alias. Jeder Interface-Aufruf wird von Anfang bis Ende auf seine *Erfolgs-Parameter* und seine Gesamt-Laufzeit gemessen; das Messgerät ist der Mess-Checkpoint (`checkpoint_measure`), der in den mi-Schichten die achsen-spezifischen Erfolgs-Parameter erhebt, und ab der ma-Ebene werden alle Parameter einschließlich der PMC-Zähler zusammengeführt (Full Join der mi-Erfolgs-Parameter). Ein Mess-Kanal ist stets über seinen Pfad Achse → Gattung → Kategorie adressiert, nie über einen freien Namen. Drei Punkte sind an dieser Stelle nicht umgesetzt: der Katalog der Erfolgs-Parameter je Achse (Gegenstand künftiger Arbeit), die Anbindung des Checkpoints an die Mess-Schleife und die Kollektor-Mechanik, die die Parameter einsammelt.

3.4.1 Die Dreiteilung: Mess-, System- und Organ-Achsen

Innerhalb dieser Definitions-Klassen beantwortet das Mess-System die in der Einleitung als Frage ausgelegte Kategorisierungs-Frage (Abschnitt 1.3): Die Achsen des Entwurfsraums trennen sich entlang der drei dort benannten Belange — das Messen selbst, die Bau- und Ziel-Plattform, das Gemessene — in *drei* Achsen-Kategorien, die im Folgenden als eigenständige *Realms* mit je eigener Klassen-Wurzel, je eigenem Angebots-Katalog und je eigener Stempel-Zeile (Abschnitt 3.4.4) entwickelt werden. Verbindlich ist die kanonische Gesamt-Ordnung aller Haupt-Achsen, und in ihr werden die Realms hier vorgestellt: erst das Mess-Tooling, dann die drei System-Achsen in ihrer festen Reihenfolge, dann die Organ-Slots T0–T17. Zwei Begriffe greifen dabei vor: Jede Tier-Binary besitzt einen Identitäts-*Stempel* — je Realm eine Zeile plus einen Fingerprint — und wird unter diesem Fingerprint im *Lager* abgelegt, dem Bestand aus Binaries und Messwerten, aus dem der Apparat Vorhandenes wiedererkennt, statt es neu zu bauen oder zu messen (Abschnitte 3.4.4 und 3.4.5). Diese Dreiteilung ist die zweite Gestalt des Trennbarkeits-Problems: Nicht nur die Entwurfsentscheidungen untereinander, auch das Gemessene, die Plattform und das Mess-Instrument müssen getrennt bleiben, damit ein gemessener Effekt eindeutig zuzuordnen ist. Die Mess-Achsen klären die Frage, ob überhaupt ein Messapparat in die Cache Engine und die Tier-Binaries eingebaut werden soll.

Die *Mess-Achsen* sind die Belange des Messens selbst — in der Metapher die Ausstattung des Labors. Der Mess-Realm umfasst (a) das Last-Rahmenwerk (`load_framework`) als Meta-Meta-Achse, das die Workload-Konventionen aus Abschnitt 2.3 in den Apparat einbringt — mit YCSB als einzigem aktivierten Baustein, die übrigen Rahmenwerke über den Datensatz-Lader (Abschnitt 3.2); (b) die Mess-Tooling-Haupt-Achse mit den drei Mess-Ebenen wallclock, macro und micro (Abschnitt 3.4), die allein die Mess-Stempel-Zeile belegt; (c) die Mess-Kategorien (Abschnitt 3.2) als *Unter-Achse* dieses Toolings, die sich als Spalten der Ergebnis-Tabellen manifestieren, nicht als eigene Haupt-Achsen — die Registry führt sechzehn Kategorien, doch mit der Erhebung je Achsen-Interface und dessen Parametersatz wächst ihre Zahl dynamisch (sechzehn Kategorien mal Parametersätze) und ist nie eine feste Anzahl; und (d)

die Ablauf-Methodik als Unter-Achse `work_mode` mit den vier Modi `build`, `measure`, `compare` und `release` (Abschnitt 3.3.2), wobei `-debug` ein Flag der Planer-Shell und kein eigener Arbeitsmodus der Kette `build/measure/compare/release` ist; es schaltet den indirekten Debug-Modus der Debug-Variante ein (den nicht offiziellen fünften Wartungsmodus), die weiter ausgebaut wird: unter `-debug` fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces, und dieser indirekte Debug-Modus fällt mit dem Release-Bau zusammen; die Bau-Varianten `Debug`, `Messung` und `Release` gelten fort. Keine dieser Unter-Achsen wird je Teil einer Binary-Identität: Mess-Achsen sind strikt `binary-id-neutral`. Innerhalb der Kategorien besteht eine *Regime-Zweiteilung*: das zeit- und observer-basierte Regime (Laufzeit-Uhren und Beobachter-Hüllen; neun Kategorien, darunter Latenz, Durchsatz, Speicher-Fußabdruck und Fill-Buffer-Auslastung) und das zählerbasierte PMC-Regime (Hardware-Performance-Zähler; sieben Kategorien, darunter Cache-, dTLB- und Branch-Misses, IPC/CPI und Energie). Das Regime ist ein Merkmal der Mess-Achse, nicht des gemessenen Organs: Dieselbe Organ-Achse kann von beiden Regimen beleuchtet werden. Die observer-basierte Teilmenge löst der Apparat auf, die PMC-Kategorien sind im Mess-Pfad verbindlich eingebaut, und die getrennte Messung von P- und E-Kernen auf Intel und AMD ist als Scheduling-Dimension der Ziel-ISA angelegt. Nicht implementiert sind an dieser Stelle die Erhebungsquelle je Achsen-Interface-Parametersatz und die Anbindung der P-/E-Kern-Messung an das Lauf-Profil.

Die *System-Achsen* sind die Belange der Bau- und Ziel-Plattform. Kanonisch sind es *genau drei* Haupt-Achsen in fester Ordnung: die Ziel-ISA (`target_isa`), das Betriebssystem (`operating_system`) und die Erweiterungs-Hardware (`external_utils`); letztere ist ausschließlich der Sammel-Hub der Hardware-Meta-Achsen (SIMD-/Vektor-Erweiterungen, GPU/FPGA/NPU, externe Beschleuniger), und gebündelt werden die drei durch den Command-Wrapper `build_target_complex` (Abschnitt 3.4.2). In der Anatomie-Metapher sind die System-Achsen kein weiteres Organ, sondern der *Lebensraum* des Tiers: immer präsent, sobald gemessen wird, keiner Gattung zugehörig — und ein Tier ist umso besser, je genauer es an ihn angepasst ist. Daraus folgen drei Invarianten. Erstens berühren System-Achsen niemals die Binary-Identität (in der Registry: `binary_id = never`) — kein Mess-Instrument und keine Plattform-Größe verändert, *was* an Organen gebaut wird; umgekehrt dürfen die Effekte der Organ-Achsen nie mit den Effekten der Plattform und der Mess-Instrumente vermengt werden. Zweitens sind sie gleichwohl permutierbar: Der Planer gibt frei, welche System-Achsen-Belegungen der CacheEngineBuilder baut — das ist die Freigabe einer Zielformat-Plattform zum Bau, die Teilfrage FF1.b (Abschnitt 1.3) voraussetzt —, und der Bau-Raum des CEB permutiert System- und Organ-Achsen gemeinsam zu Tier-Binaries; auch die System-Unter-Achsen permutieren — das Scheduling mit seinen fünf Dimensionen am Wrapper der Ziel-ISA (Abschnitt 3.2). Drittens sind verschiedene Bau-Welten derselben Organ-Permutation verschiedene Artefakte und müssen im Fingerprint verschieden sein: Bau-Steuergrößen wie Optimierungsstufe und SIMD-Erweiterung multiplizieren deshalb nur die *Bau-Matrix* — sie erscheinen in der Bau-Versionskennung der Binary und im Werkzeugketten-Glied des Fingerprints (Abschnitt 3.4.4) —, nie den Organ-Permutationsraum.

Die *Organ-Achsen* sind das *Gemessene*: Organe des Lebewesens, Teil der Gattungen, permutierbar — jede Belegung verändert die Identität der Tier-Binary. Kanonisch sind es *achtzehn* Organ-Haupt-Achsen in fester Slot-Reihenfolge T0–T17 (Tabelle 3.2); die T-Nummern bezeichnen die Slots der Komposition, keine Implementierungs-Nummerierung. Das Persistenz-Ziel ist die achtzehnte Organ-Achse; Telemetrie und Ziel-ISA gehören nicht zu den Organen, sondern zum Mess-Tooling bzw. zur System-Sphäre. Dazu

tritt eine optionale *Organ-Meta-Meta-Achse* — *Meta-Meta* im Sinn der Indirektion einer Beschreibung: eine Achse, die nicht eine Belegung beschreibt, sondern beschreibt, wie andere Achsen beschrieben werden. In der vierstufigen Ebenen-Ordnung dieser Arbeit (M0 der Binärcode, M1 Konzeption und Laufzeit-Implementierung der Haupt- und Unter-Achsen, M2 die literale Klassifizierung der unter ihrem Begriff geführten Haupt- und Unter-Achsen — im Code die Registry-Zeile —, M3 die Meta-Meta-Achsen; die Zuordnung von M3 ist gesetzt, der Schnitt zwischen M2 und M1 ist Lesart) ist Meta-Meta die oberste Stufe M3; im Stempel ist die Meta-Meta-Ebene eines Eintrags seine Klammer-Tiefe — eine Haupt-Achse ist klammerlos, eine Meta-Meta hängt als Klammer-Anhang an ihrer Realm-Zeile (Abschnitt 3.4.4) —, und die Bit-Codierung dieser Ebene im Eintrag beschreibt Abschnitt 4.5 —, die Festplatten-Ein-/Ausgabe `disk_io`: Sie stempelt im Klammer-Anhang der Organ-Zeile, berührt die `binary_id` nicht, wird nur bei einem Platten-Rückschreibe-Ziel wirksam und ist abgeschaltet; im golden-Referenzkatalog bleibt ihr Anhang leer.

Konzeptionell besitzt die Dreiteilung zwei Realm-Wurzeln: Der System-Realm wurzelt am `CacheEngineBuilder`, der die Plattform kennt und die Tier-Binaries baut; der Mess-Realm wurzelt am *Experiment-Planer*, der die Experimente aus der XML-Beschreibung auflöst (Abschnitt 3.4.6). Jede der drei Kategorien besitzt damit eine eigene abstrakte Klassen-Wurzel und einen eigenen Angebots-Katalog (drei Registry-XML); die Code-Gestalt dieser Wurzeln entwickelt Abschnitt 4.5.

Die Zuordnung der weiteren Größen ist eindeutig: Der Compiler ist eine Unter-Achsen-Gruppe der Bau-Werkzeugkette in der äußeren Komplex-Klammer — `build_toolchain` mit Compiler, Optimierungsstufe und Atomik-Fähigkeit (Abschnitt 3.4.2); das Scheduling ist eine Unter-Achse am Wrapper der Ziel-ISA mit fünf Dimensionen; das Last-Rahmenwerk gehört dem Mess-Realm; die Ziel-ISA ist die Komplex-Achse `target_isa`; und die Telemetrie ist *zweigeteilt*, denn Zähler-Instrumentierung muss zum Messen eingebaut sein: Als Sweep-Unter-Achse des Mess-Toolings im Planer erzeugt sie eine Übersetzungszeit-Haupt-Größe im Builder.

Dieses letzte Muster ist generisch und trägt einen eigenen Namen: die *zweigeteilte Haupt-Achse* — eine Achse, die über die Laufzeit der Vorstufe *und* die Übersetzungszeit der Folgestufe läuft, entlang der Kette Planer → Builder → Tier-Binary; die Unter-Achsen-Freigabe der Vorstufe wird zur Haupt-Achsen-Annahme der Folgestufe. Das Last-Rahmenwerk ist die erste Instanz (der Planer wählt es zur Laufzeit, der Builder enthält es einkompiliert), die Telemetrie die zweite, und die Hardware-Erkennung (Abschnitt 3.4.3) die dritte.

System- und Organ-Achsen sind *zweiphasig*: Der Planer gibt ihre Belegung zu seiner Laufzeit frei, die Folgestufe enthält sie einkompiliert. Mess-Achsen sind *dreiphasig*: Der Planer wählt sie zur Laufzeit; der `CacheEngineBuilder` enthält die Mess-Wahl als seine Provenienz einkompiliert — N Tooling-Konfigurationen ergeben N CEB-Pipelines —; und die Tier- bzw. Hybrid-Binary führt den Mess-Gate-Zustand ihrer eigenen Übersetzungseinheit als Fingerprint-Glied, sodass zwei Kompilate desselben Quelltexts mit und ohne Messfühler verschiedene Identitäten besitzen. Die Mess-Achsen definieren damit die Verträge über CEB und Tier/Hybrid hinweg und durchdringen sie zur Übersetzungszeit, weil der Planer ihre Eigenschaften zur Laufzeit freigibt; die Steuerung sitzt im Planer, der Vertrag ist je Träger-Stufe lokal (Abbildung 3.9). Dass die Gate-Kombination CEB aus, Tier an gedeckt ist, ist durch einen eigenen Test belegt.

Alle drei Achsen-Realms bindet ein methodisches Prinzip, der *Grundsatz der ausgewiesenen Nichterhebung*: Ein nicht erhobener Wert ist ein fehlender Wert (NA-Semantik im Sinn der Statistik fehlender

Daten [Rub76]), nie eine Null und nie ein Phantomwert. Wo ein Wert nicht wirklich erhoben werden kann — eine Organ-Achse ohne tatsächlichen Laufzeit-Parameter, ein Zähler ohne Hardware-Zugriff, eine Mess-Quelle, die auf der Plattform nicht existiert —, meldet das System *explizit ungültig oder leer* statt eines plausiblen Phantomwerts. Lieber eine als ungültig markierte Messung als ein synthetischer Effekt: Ein Phantomwert würde die Heuristik-Ableitung (Abschnitt 3.6) systematisch vergiften, während eine ausgewiesene Lücke sichtbar bleibt und gezielt geschlossen werden kann. Das Prinzip gilt durchgängig — von der Achsen-Verdrahtung (nur Achsen mit tatsächlich messbarem Laufzeit-Knopf melden Laufzeit-Effekte; die übrigen gelten als übersetzungsstatisch) über die Verfügbarkeits-Meldung der Zähler-Quellen bis zur Auswertung, die aus einer leeren Eingabe keinen leeren Plot erzeugt.

Was diese Mess-Muster konkret erheben, sind drei gegenläufige Komponenten — *Latenz*, *Durchsatz* und *Speicherverbrauch* —, gegen die sich jede Achsen-Permutation als Trade-off-Profil aufspannen lässt: dieselbe Permutation, die in Abbildung 3.1 ein Punkt im Entwurfsraum ist, erscheint hier als Punkt im Inneren des Trade-off-Dreiecks (Abbildung 3.13); bei mehreren konkurrierenden Zielgrößen ist die „beste“ Permutation kein Einzelsieger, sondern ein Element der Pareto-Front der nicht-dominierten Kandidaten [Ehr05] (Abschnitt 3.6.1). Die vierte, für cache-bewusste Suchstrukturen zentrale Mess-Komponente — das cache-bewusste Verhalten auf Knoten-, Daten- und Index-Ebene — wird über Hardware-Performance-Zähler erhoben: Die PMC-Erfassung ist in jedem offiziellen Mess-Lauf verbindlich eingebaut und wird von den emittierten Mess-Jobs an allen Treiber-Bau-Stellen gesetzt; nur die hardwarefreie Test-Zelle baut ohne sie. Die Belegung ist mikroarchitektur-gebunden: Für AMD Zen 5 (prod1) ist der Ereignis-Katalog für L2-Demand-Misses und Fremd-CCX-Fills gegengeprüft, für Intel liegt kein Katalog-Eintrag vor, und der Last-Level-Zähler scheitert auf Zen 5 beim Öffnen; jede dieser Lücken erscheint als fehlender Wert, nie als Null. Die getrennte Messung der P- und E-Kerne auf Intel und AMD ist als Scheduling-Dimension `hetero_core_dispatch` der Ziel-ISA (Werte `None`, `HybridAware`, `PCoresOnly`, `ECoresOnly`) und über die Kern-Pinning-Bausteine angelegt; das Mess-Modell führt dafür sechs Kern-Modi: vier Pinning-Modi je Kern-Sorte (P- oder E-Kerne, gepinnt auf einen einzelnen Kern oder auf alle Kerne der Sorte), den ungepinnten Referenzlauf als fünften und *HybridAware* als sechsten Modus¹. Die Cache-Line-Awareness-Messung setzt den HybridAware-Modus voraus, weil sich Cache-Line-Effekte sonst nicht gezielt an bestimmten hybriden Kernen vermessen lassen; der Modus ist deshalb Pflicht-Bestandteil des Mess-Modells, keine Option. Umsetzungsstand: Der Pinning-Aktuator ist implementiert; die Anbindung der Kern-Modi an das Lauf-Profil — für prod2, den Intel Core i9-12900K (Alder Lake) mit P- und E-Kernen — fehlt.

3.4.2 Komplex-Achsen: die Ziel-ISA als feste Rekombination

Nach außen verhalten sich die drei System-Haupt-Achsen wie *eine*: Eine *Komplex-Achse* ist eine Klammer über einer festen Rekombination mehrerer Haupt-Achsen — alle Glieder bleiben Haupt-Achsen, treten aber gemeinsam als eine auf (als Befehls-Klammer im Sinne des *Command*-Musters gebündelt). Die *äußere* Komplex-Klammer spannt `target_isa × operating_system × external_utils` und enthält als Unter-Achsen-Gruppe die Bau-Werkzeugkette `build_toolchain` (Compiler, Optimierungsstufe, Atomik-Fähigkeit); als Klammer ist sie selbst *kein* Glied der kanonischen Ordnung.

¹HybridAware: Der Lauf kennt die heterogene Kern-Topologie der Maschine, pinnt seine Threads zur Laufzeit über die Affinitäts-Schnittstelle des Betriebssystems (unter Linux `sched_setaffinity`) auf ausgewählte Kerne beider Sorten und misst dort mit dem zur Kern-Sorte gehörenden Zähler-Satz; anders als `PCoresOnly` und `ECoresOnly` legt er die Kern-Sorte nicht für den ganzen Lauf fest, anders als `None` läuft er nicht ungepinnt.

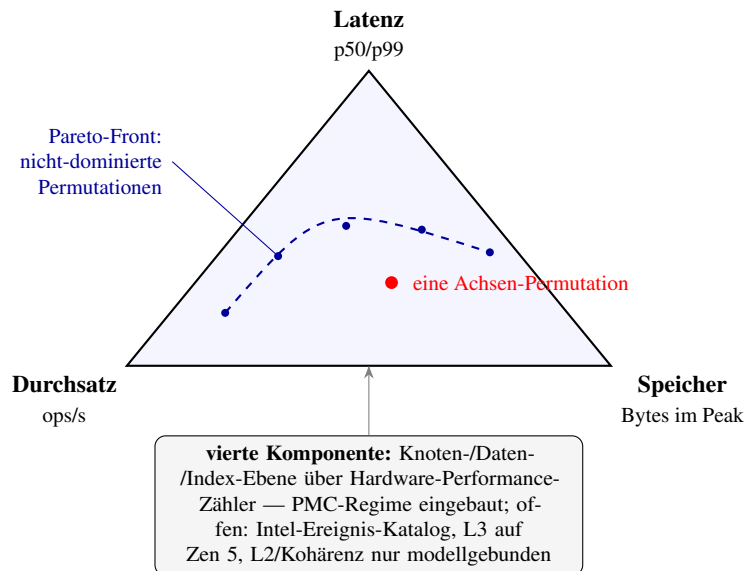


Abbildung 3.13: Das Mess-Trade-off-Dreieck: jede Achsen-Permutation spannt sich gegen die drei gegenläufigen Mess-Komponenten Latenz, Durchsatz und Speicherverbrauch auf und ist ein Punkt in seinem Inneren (vgl. den Entwurfsraum in Abbildung 3.1, wo dieselbe Permutation ein Punkt ist); die Pareto-Front der nicht-dominierten Permutationen ist die Brücke von den drei Größen zur „besten Binary“. Die für cache-bewusste Suchstrukturen zentrale vierte Komponente — das Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im Mess-Pfad verbindlich eingebaut; offen sind der Intel-Ereignis-Katalog, der Last-Level-Zähler auf Zen 5 und die nur modellgebundene L2-/Kohärenz-Belegung.

Die Ziel-ISA ist *in sich* eine zweite, innere Komplex-Achse: eine feste Rekombination der Identitäts-Glieder RAM-Frequenz, CAS-Latenz und CPU-Fabrikation, rekursiv von der äußeren Klammer gewrapp; an ihrem Wrapper hängen die vereinbarten Unter-System-Achsen (Scheduling, NUMA-Knoten, Speicherseiten-Größe). Die Glieder werden je Maschine *deklariert* — in derselben Anwender-XML, die auch das Experiment anzeigt (Abschnitt 3.4.6) — und je Betriebssystem und ISA ermittelt; bei exaktem Eigenschafts-Match ist dieselbe Achse über formal verschiedene Maschinentypen wiederverwendbar, denn der Rechnername dient nur dem Instanz-Lookup — die Klassen-Identität ist das Eigenschafts-Tupel. Im Identitäts-Stempel (Abschnitt 3.4.4) belegt eine Komplex-Achse genau *ein* Feld ihrer Typ-Zeile, keine Klammer je Glied; die Betriebssystem-Distribution ist dort eine Stempel-Variable, keine vierte Ausprägung. Die CAS-Latenz ist ein rein *deklariertes* Glied (Provenienz-Stufe „deklariert, nicht gemessen“, Abschnitt 3.4.3); die Arbeit weist das aus, statt einen Messwert vorzutauschen.

3.4.3 Hardware-Erkennung als zweigeteilte Mess-Achse

Damit die deklarierten Plattform-Glieder keine still veraltenden Konstanten werden, führt das Konzept die Hardware-Erkennung als weitere zweigeteilte Haupt-Achse ein — und zwar als *Mess-Achse*: Sie misst Eigenschaften der Hardware aus, verändert aber weder System- noch Organ-Achsen. *Ebene 1* liegt beim Planer: Zu *seiner* Laufzeit erkennt er grob die Zelle aus ISA-Vendor-Familie und Betriebssystem und wählt per Metaprogrammierung aus, welche feingranulare Erkennungs-Software samt Betriebssystem-Zugriffen in den `CacheEngineBuilder` einkompiliert wird — eine Übersetzungszeit-Zellwahl, kein Laufzeit-

Umschalter über Betriebssysteme. Die Auswahl-Matrix ist *total* über alle deklarierten ISA-Komplexe und alle dokumentierten Betriebssysteme: Nicht implementierte Zellen existieren als ausgewiesene Nur-Deklarations-Spezialisierung, und ein neuer Katalog-Eintrag ohne Zelle bricht die Übersetzung, statt eine stille Lücke zu lassen. *Ebene 2* liegt beim Builder: Mit der einkompilierten Erkennung erhebt er zu seiner Laufzeit die Ist-Eigenschaften der Maschine und setzt damit die Übersetzungszeit-Stempel der Tier-Binaries korrekt. Der Stempel dieser Mess-Achse selbst enthält die Vendor-Erkennungs-Familie und das Betriebssystem — als Geräte-Segment der Mess-Stempel-Zeile (Abschnitt 3.4.4) —, niemals einen Messwert. Die Kern-Topologie hybrider CPUs (P- und E-Kerne) wird als Unter-Achse der Ziel-ISA geführt (Scheduling-Dimension `hetero_core_dispatch`), damit Cache-Line-Effekte gezielt je Kern-Typ vermessen werden können (Sechs-Modus-Modell mit HybridAware als Pflicht-Modus, Abschnitt 3.4.1). Jedes Erhebungs-Ergebnis enthält eine *Provenienz*-Stufe samt Erhebungszeitpunkt, in absteigender Vertrauens-Ordnung: *konfiguriert-gemessen* (privilegierte Systemabfrage), *SPD-JEDEC-Basiswert* (unprivilegiert aus dem Modul-Speicher gelesen, bewusst nur die JEDEC-Nennrate; Übertaktungs-Profile erscheinen als separates Angebots-Feld, nie als Ist-Wert) und *deklariert-nicht-gemessen* (Anwender-XML). Eine niedrigere Stufe darf eine höhere nur mit mitwandernder Kennzeichnung verdrängen — sonst stünde eine deklarierte Nennrate in der Ergebnis-Tabelle, während die Maschine womöglich schneller fährt: schlimmer als eine als deklariert ausgewiesene Nennrate. Nicht Erhobenes erhält ein ausdrückliches n/a-Zeichen, nie eine Null in der Wert-Spalte (Grundsatz der ausgewiesenen Nichterhebung, Abschnitt 3.4.1); vier Fehlerklassen trennen fehlende, unlesbare, korrupte und unbekannt formatierte Quellen, und eine korrupte Quelle degradiert nie still zur nächsten Stufe. Der Gewinn dieser Achse ist deshalb nicht ein besserer Einzelwert, sondern die *automatisierte Wiedervorlage*: Ein einmalig erhobenes und dann festgeschriebenes Werkzeug-Ergebnis — so ist die für prod2 hinterlegte RAM-Nennrate von 4800 MT/s in der System-Registry als deklariert markiert, mit `dmidecode` als Deklarationsquelle — wird durch eine Erhebung ersetzt, die bei jedem Lauf erneut stattfindet und ihre Vertrauensstufe mitführt. Provenienz-Vokabular, SPD-Parser, Fehler-Domäne, Erhebungs-Kette und Plattform-Auswahl-Matrix dieser Achse sind implementiert (Kapitel 4).

3.4.4 Identitäts-Stempel: eine Zeile je Realm

Die Dreiteilung wird an der Tier-Binary selbst ablesbar. Ihre *Permutations*-Identität ist die `binary_id`: der Pfad aus achtzehn `achse=wert`-Segmenten in kanonischer Slot-Reihenfolge — ein Segment je Organ-Achse, und *nur* Organ-Achsen. Zusätzlich enthält jede Tier-Binary einkompilierte *Stempel-Zeilen*, genau eine je Realm. Die *Organ-Zeile* hält für jede der achtzehn Haupt-Achsen den gewählten Algorithmus samt Algorithmus-Version fest. Die *System-Zeile* hält die drei System-Haupt-Achsen; als Version führen sie ihre *Bau*-Version — die geordnete Suffix-Kennung aus Werkzeugkette, Optimierungsstufe, Erweiterung und weiteren Bau-Steuergrößen, in der ein leeres Glied schlicht kein Segment erzeugt —, denn eine Plattform-Achse hat keine Algorithmus-Version. Die *Mess-Zeile* führt die Menge der einkompilierten Mess-Toolings als klammerlose Segmente und hängt das Last-Rahmenwerk — die Meta-Meta-Haupt-Achse des Mess-Realms — als Klammer-Anhang an ihr *Ende* (Muster `measurement_tooling=wallclock@1.0.0.c;[load_framework=yicsb@1.0.0.c]`); eine *leere* Mess-Zeile ist dabei eine explizit ausgewiesene Aussage — kein Mess-Tooling einkompiliert — und bleibt auch ohne diesen Anhang leer. Eine Komplex-Achse belegt in ihrer Zeile genau ein Feld (Abschnitt 3.4.2). Die Eintrags-Grammatik ist einmal für alle Zeilen definiert:

`achse=algo@X.Y.Z[.flag]*` mit rekursiven Komposit-Klammern — `c{p.e}` ist die Basis `c` mit den Untergliedern `p` und `e` und wird nie flach als `cpe` gelesen; die Klammer-Tiefe eines Eintrags ist seine Meta-Meta-Ebene, null die Haupt-Achse. Auf Tiefe 0 stehen die Hardware-Basis-Flags `c` = CPU, `g` = GPU, `f` = FPGA und `n` = NPU, die die Ziel-Hardware-Klasse nennen, neben den SIMD-Breiten-Basen `x128`, `x256` und `x512`, der Medien-Basis `m64` und Companion-Flags wie `gfni`. Die Basen `g`, `f` und `n` sind im Katalog reserviert und werden heute nicht produziert, weil die Cache-Engine ausschließlich CPU-Code erzeugt; jede von ihr erzeugte Version trägt die CPU-Basis `c`: Ein Eintrag mit Flags, aber ohne `c` (etwa `1.0.0.g`) ist nicht wohlgeformt, die flaglose Form `1.0.0` ist wohlgeformt und fällt allein an der separaten CPU-Pflicht-Wache. Unter `c` stehen als Unterglieder die Kern-Klassen `p` = Performance-Core und `e` = Efficiency-Core; `c` ohne Klammer bedeutet semantisch `c{p}`, als Stempel-Zeichenketten werden `c` und `c{p}` aber nicht zusammengeführt, weil der Stempel Identität ist und der Parser nicht normalisiert; `c{e}` bezeichnet einen Efficiency-Core-Algorithmus, `c{p.e}` beide Kern-Klassen. Die Identität eines Flags ist das Paar aus Token und Elternteil: `f` auf Tiefe 0 ist FPGA, `f` in der Klammer von `x512` das AVX-512-Foundation-Subset. Eine vierte *Realm*-Zeile für die Misch-Strategie eines Prüflings kann es dagegen nicht geben: Ein Prüfling mischt gegen die Organ-Achsen und ist damit bereits in der Organ-Zeile beschrieben. Kenntlich wird die Mischung dort am Achsen-Namen, den der Prüfling unter eigenem Namensraum-Präfix stempelt (Muster `prt-art.memory.abc=prt_patricia@2.3.4.c{p.e}`), sodass unterschiedliche Misch-Konstellationen am Namen unterscheidbar bleiben — nicht an der Versions-Beifferung. Genau auf dieser Namens-Ebene bleiben die Hybrid- und Full-Join-Stände am Kompilat unterscheidbar; byte-verschiedene Mischbauten besitzen deshalb nie namensgleiche Organ-Segmente.

Den Stempel schließt je Träger-Stufe ein *Fingerprint*; der Stempel übernimmt damit neben der Identität die Rollen Cache-, Lager- und Wiedererkennungs-Schlüssel sowie Einordnung. Die Tier-Binary führt einen SHA-512 über ein Preimage aus elf Gliedern in fester Reihenfolge: die Format-Kennung (`fingerprint_format=6`), die Mess-, System- und Organ-Zeile in dieser Ordnung, das Unter-Achsen-Werteset, das Werkzeugketten-Glied, die Enabled-Mengen der Bau-Achsen, den Overlay-Quell-Hash, den tatsächlich wirksamen Mess-Gate-Zustand der Übersetzungseinheit, die Hybrid-Komposit-Zeile und die Basis der Bau-Version; an diesem einen vergleichbaren Wert hängt die Wiedererkennung einer bereits gebauten Permutation im Lager (Abschnitt 3.4.5), und die Format-Kennung als erstes Glied führt bei jeder Layout-Evolution deterministisch zur Nichtübereinstimmung statt zur stillen Kollision. Der Planer führt einen SHA-256 über seine Version, die Ziel-ISA und das Betriebssystem; der CacheEngine-Builder führt einen SHA-512 als Provenienz seiner einkompilierten Mess-Wahl, der ausdrücklich kein Wiederverwendungs-Kriterium ist, weil er Werkzeugkette und Zelle absichtlich nicht enthält. Gemessene Werte — etwa eine erhobene RAM-Rate — erscheinen niemals in Stempel oder `binary_id`: Gestempelt wird die Übersetzungszeit-Plattform-Klasse, nie ein Messwert (Abschnitt 3.4.3).

3.4.5 Das Lager: Bestände unter Stempel-Schlüsseln

Das *Lager* (repository) ist der persistente Bestand des Apparats, in dem Gebautes und Gemessenes unter Stempel-Schlüsseln liegt (Abbildung 3.14). Der Lager-Baum ist nach Gattung, Gattung und Realm gegliedert — `gattung=` → `genus=` → `realm=(binaries oder messdaten)` —, und das Hybrid-Konzept wird darin wie jede andere Gattung einsortiert. Vier Bestände sind im Schlüssel-Schema übersetzungsstatistisch festgeschrieben: Bestand 1 sind die Tier-Binaries, geschlüsselt über ihren SHA-512-

Fingerprint, die Mess-Zelle daneben geklammert; Bestand 2 sind die Messwerte, geschlüsselt über den Fingerprint der vermessenen Binary und die Hardware-Identität der messenden Maschine (eine Klassen-Eigenschaft aus CPU-Fabrikation und RAM-Paar, nie der Rechnername), die Plattform-Zelle daneben geklammert — ein Messwert geht nie in seinen eigenen Schlüssel ein; Bestand 3 sind die Synthese-Batches, die synthetisierten Mess-Kurven als Dateien, geschlüsselt über Maschine, Voll-Stempel und die Referenz auf Mess-Ebene und Kanal; Bestand 4 sind die Lösungen, geschlüsselt über den kanonisierten Hash der Anwender-XML, die Maschine und die Referenzen auf die beteiligten Bestände. Die Messung ist immer eingeschaltet; im Lager wird nur nachgemessen, was dort fehlt, und eine Permutation wird über ihren Fingerprint wiedererkannt statt neu gebaut. Weil der Mess-Gate-Zustand ein Fingerprint-Glied ist, liegen je Permutation die Binary mit und die Binary ohne Messfühler als zwei Einträge des Bestands 1 nebeneinander — die Vorbedingung dafür, dass ein Release ohne Messfühler gegen die vermessene Fassung nachgemessen werden kann (Abschnitt 3.6.2).

Zwei Konsequenzen sind Eigenschaften des Lagers, nicht seiner Implementierung: Ein Algorithmus-Versionswechsel ändert die Organ-Zeile und damit den Fingerprint, löst also den Neubau des betroffenen Lager-Stapels samt Neumessung aus; und weil die Hardware-Identität Teil des Messwert-Schlüssels ist, hat jede Maschinen-Konfiguration ihren eigenen Messwert-Bestand. Umsetzungsstand: Bestand 1 und 2 sowie das Schlüssel-Schema aller vier Bestände sind implementiert; die Mechanik von Bestand 3 und 4, die Kurven-Facette, der Ablage-Pfad des Hybrid-Tiers und die Zielwahl über die XML bilden den Lager-Vollausbau und sind nicht implementiert.

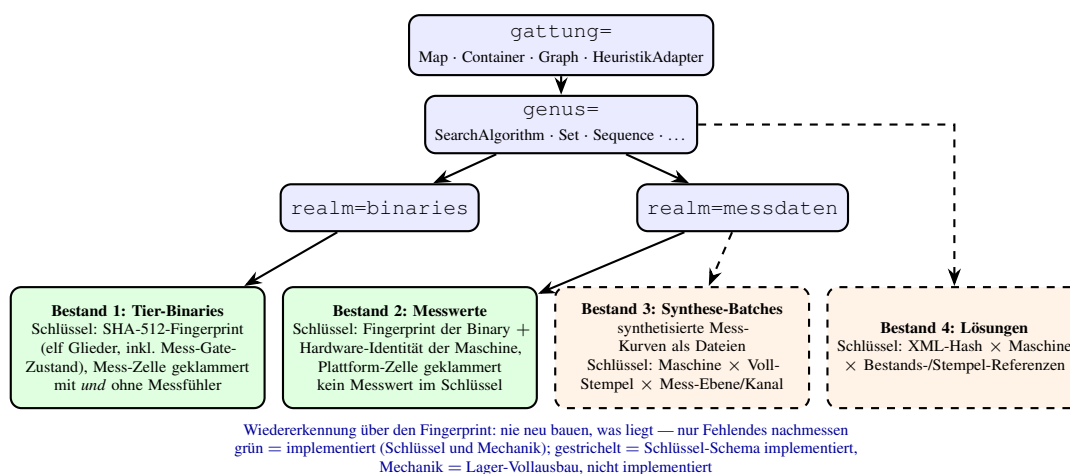


Abbildung 3.14: Das Lager: der Lager-Baum ist nach Konzept, Gattung und Realm gegliedert; darunter liegen vier Bestände unter Stempel-Schlüsseln — Bestand 1 die Tier-Binaries (SHA-512-Fingerprint; die Binary mit und die ohne Messfühler sind zwei Einträge, weil der Mess-Gate-Zustand ein Fingerprint-Glied ist), Bestand 2 die Messwerte (Fingerprint der Binary plus Hardware-Identität der Maschine), Bestand 3 die Synthese-Batches mit den synthetisierten Mess-Kurven als Dateien und Bestand 4 die Lösungen je Anwender-XML. Der Apparat erkennt Vorhandenes über den Fingerprint wieder und misst nur Fehlendes nach.

3.4.6 Subsysteme und die Kette von der XML zur Tier-Binary

Organisiert ist der Apparat als *ein* Hauptkanal: Eine Anwender-XML ist die einzige Quelle des gesamten Prozesses. Sie *zeigt an*, welche Experimente, Achsen, Einstellungen und Abläufe durchzuführen sind, während je Realm ein *Registry*-Katalog das *Angebot* des Existierenden hält — das deklarative Konfigurationsmodell aus Abschnitt 2.4, nach dem Vorbild des Java-POM: ein deklaratives Manifest, das gegen ein Repository aufgelöst wird; eine unregistrierte Referenz ist ein harter Validierungsfehler, keine stille Auslassung. Der Experiment-*Planer* löst die XML gegen die drei Angebots-Kataloge auf und baut je Ziel-Plattform und Mess-Aufgabe *einen* `CacheEngineBuilder` (CEB), in den die gewählten Mess-Achsen einkompiliert werden; dieser Builder orchestriert mit ihnen den Bau der Tier-Binaries und stempelt sie mit seiner Plattform (Abschnitt 3.4.4); die Tier-Binary schließlich enthält die Organ-Permutation. Jede Stufe der Kette speist sich so aus ihrem art-eigenen Angebot — Mess-Katalog → Planer, System-Katalog → Builder, Organ-Katalog → Tier-Binary —, und die Tier-Binary bleibt in zwei strikt getrennten Schichten lesbar: dem Hardware- und Umgebungs-Layer der System-Achsen und dem Anwendungs-Logik-Layer der Organ-Achsen; so wenig, wie man ein Betriebssystem mit dem Anwendungs-Frontend gleichsetzt, so wenig verschmelzen diese beiden Layer in der Binary. Gesteuert wird die Anlage auf zwei Wegen: als Planer per Shell direkt oder per API-Kommandozeile mit einem headless Server, der den Planer als Zelle auf Kubernetes oder bare metal bedient; ein optionaler, kommandozeilen-geführter Erstellungsmodus deklariert die verfügbaren XML-Einstellungen aus der erkannten Hardware und bietet nur Wählbares an. Von diesen Wegen ist die Planer-Shell implementiert; der headless Dienst und der Erstellungsmodus sind nicht implementiert.

Auf diesen Mess-Mustern und dieser Kette aufbauend trennt das Mess-System vier Subsysteme und prüft in drei Stufen, die die drei Pflicht-Messreihen speisen; den Permutationsraum reduziert es gezielt. Vier formal getrennte Subsysteme mit unterschiedlichen Verantwortlichkeiten bilden das Modell: der `messung_driver` (der Auswertungs-Orchestrator, der je Pflicht-Messreihe A/B/C die Pipeline durchläuft; er liegt im übergeordneten Projekt-Repository), der `CacheEngineBuilder` (ein autonomes Plattform-Ausmess-System, das die Sieben-Phasen-Pipeline aus Abschnitt 4.3.2 über seinen `ExperimentDriver` ausführt), die `CacheEngine` (die Achsen-Bausteine-Bibliothek mit den Standard-Implementierungen je Achse) und der Prüfling PRT-ART (der einige Achsen-Organen selbst stellt und für die übrigen Cache-Engine-Dienste konsumiert); Cache-Engine, Prüfling und diese Arbeit liegen in je eigenen Repositories — vier Repositories insgesamt —, und die Planer-Rolle der Kette ist als eigene Binary realisiert (`comdare-experiment-planner`), die Builder-Rolle als Mess-Treiber (`comdare-messung-driver`); der Selbst-Stempel des Mess-Treibers (`-version`) gibt die Zeilen beider Rollen — die Planer-Zeile und die Zeile der CEB-Vertragsversion — aus einem Treiber-Binary aus (Kapitel 4). Die `ExecutionEngine` ist dabei kein fünftes Subsystem, sondern die gemeinsame messbare Wurzel (das `IExecutionEngine`-Interface), über die diese vier verbunden sind: Unter ihr registrieren sich der Prüfling und jede Engine als `IExecutionEngine`, und der Builder enumeriert den Permutationsraum je registrierter Engine. Definierend ist die *bidirektionale* Cache-Engine ↔ Prüfling-Beziehung: Die Cache-Engine instantiiert den Prüfling je Permutations-Konfiguration, und der Prüfling ruft zur Laufzeit Cache-Engine-Dienste (Telemetrie, Prefetch, Heuristik) während seiner eigenen Lookups auf. Diese Symmetrie begründet die *Multi-Prüfling*-Fähigkeit, mit der Messreihe A den Prüfling gegen viele SOTA-Adapter in einer einzigen Pipeline vergleicht (Abbildung 3.15).

Der Builder orchestriert eine *Drei-Stufen-Prüfung*: **Stufe 1** (nur Cache-Engine) nutzt die Standard-Variante

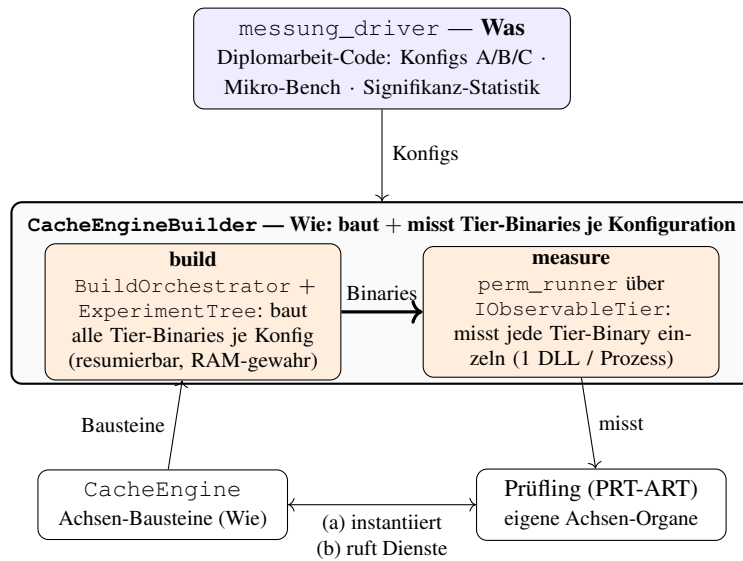


Abbildung 3.15: Das M-Modell mit Fokus auf die Build- und Mess-Phasen. Die äußere `messung_driver`-Schleife (**Was**, Diplomarbeit-Code) speist die Konfigurationen der drei Messreihen A/B/C ein; der `CacheEngineBuilder` (**Wie**, Bibliothek) zerfällt in die beiden eigentlich interessanten Phasen: **build** (`BuildOrchestrator/ExperimentTree` baut je Konfiguration eine eigenständige Tier-Binary, resumierbar und RAM-gewahr) und **measure** (`perm_runner` treibt und misst jede Tier-Binary einzeln über die Beobachter-Schnittstelle `IObservableTier`). Die `CacheEngine` liefert die Standard-Achsen-Bausteine, der Prüfling PRT-ART seine eigenen Organe; ihre bidirektionale Beziehung ((a) instantiiert, (b) ruft Dienste) begründet die Multi-Prüfling-Fähigkeit. Die `ExecutionEngine` bleibt die gemeinsame messbare Wurzel.

je Achse, **Stufe 2** (nur Prüfling) ersetzt die überschriebenen Achsen durch die Prüfling-Varianten und behält sonst die Standards, und **Stufe 3** (Full Join, nicht-redundant) kombiniert Standard- und alle Prüfling-Varianten; im Code sind es die Verbund-Strategien des Merge-Plans — Stufe 1 ist `Verbund1_CeOnly`, Stufe 2 zerfällt in Ersetzen (`Verbund2_Replace`) und Hybrid-Mischung (`Verbund2_Hybrid`), Stufe 3 ist `Verbund3_Union` —, die die Drei-Stufen-Sicht zusammenfasst. Jede Stufe kann bis zu tausende nicht-redundante Permutations-Binaries erzeugen, die zur Laufzeit über die C-stabile Modulgrenze als C++23-Module geladen werden; materialisiert wird stets eine spärliche, gedeckelte Teilmenge, der volle Produktraum wird nur gezählt (Abbildung 3.16).

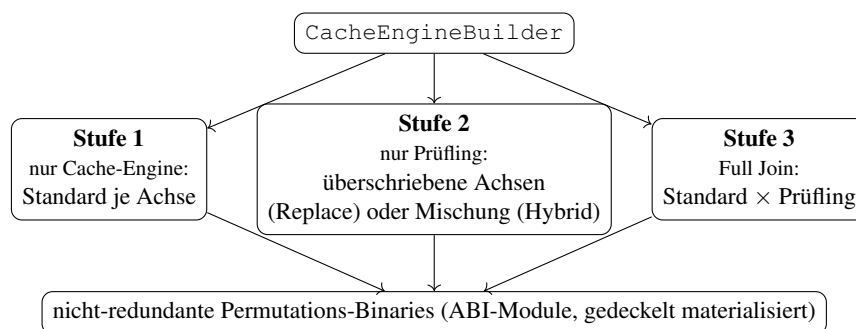


Abbildung 3.16: Die Drei-Stufen-Prüfung des Builders: jede Stufe erzeugt nicht-redundante Permutations-Binaries; ihre Zuordnung zu den Messreihen A/B/C zeigt Tabelle 3.8.

Diese Stufen bilden *nicht* eins-zu-eins auf die drei Pflicht-Messreihen ab, sondern speisen sie: Die Standard-Binaries der Stufe 1 (samt der rekonstruierten SOTA-Profile) und die Prüfling-Binaries der Stufe 2 bilden *gemeinsam* die Vergleichs-Messreihe A; der nicht-redundante Full Join der Stufe 3 ist die Grundlage der Variations-Messreihe B; und die Merge-/Regressions-Messreihe C (alt gegen neu) ist an keine einzelne Stufe gebunden, sondern läuft build-übergreifend als alt-gegen-neu-Vergleich — über aufeinanderfolgende Stände derselben Konfiguration ebenso wie über konkrete Merge-Punkte (Tabelle 3.8).

Tabelle 3.8: Zuordnung der Drei-Stufen-Prüfung zu den Pflicht-Messreihen

Prüf-Stufe	Inhalt	speist Messreihe
Stufe 1 (nur Cache-Engine)	Standard-Bausteine + SOTA-Profile	A (Baseline auch für C)
Stufe 2 (nur Prüfling)	überschriebene Achsen = Prüfling-Varianten	A
Stufe 3 (Full Join)	Standard $\times$ Prüfling, nicht-redundant	B
build-übergreifend	alt gegen neu (Regression und Merge-Punkte)	C

Die drei Prüf-Stufen bilden nicht eins-zu-eins, sondern in dieser Zuordnung auf die drei Pflicht-Messreihen A/B/C ab; Reihe C ist build-/versionsübergreifend (nicht an eine einzelne Prüfstufe gebunden).

Die drei Messreihen — alle vom `messung_driver` über die `ExperimentDriver`-Bibliothek ausgeführt, konfiguriert entweder als drei feste Reihen oder über eine externe `messreihen.xml` — sind damit: *Reihe A* (PRT-ART vs. Stand der Technik) mit den Modi `A_defined` (schnelle Verifikation gegen die 8 Rang-1-Vollprofile) und `A_full` (alle 33 Profile, 30 Vollprofile und 3 abstrakte; der Standard, da erst der Voll-Lauf die bias-reduzierte Vermessung aller Lastprofile gegen alle Verfahren einlöst, Abschnitt 3.1); *Reihe B* (systematische Variation der Achsen-Entscheidungen unter sonst gleichen Bedingungen, um den Beitrag jeder Achse zu isolieren); und *Reihe C* (Merge/Regression alt gegen neu: konkrete Merge-Punkte zwischen PRT-ART-Bausteinen und Cache-Engine-Permutationen — etwa das PRT-ART-OLC mit `tcmalloc` und ART-Page, der 4+1+2-Pool mit der HOT-Page, das Path-Prefetch mit der B^2 -Baum-Page oder das MultiLevel-Layout mit der Wormhole-Page —, build-übergreifend als alt-gegen-neu-Vergleich gefahren). Jede Reihe wird in drei Granularitäten erhoben, die den drei Mess-Ebenen entsprechen (Abschnitt 3.4). *Mikro-Benchmarking* (Ebene micro) vermisst jede Achse über ihr *Achsen-Interface* — die Funktionsmenge, die *alle* Algorithmen dieser Achse liefern müssen (jeder Allokator stellt Speicher bereit, gleich welcher): die achsen-spezifischen Haupteigenschaften, intern gemessen. Die Achse ist dabei *keine* Interface-Funktion des Suchalgorithmus; dessen Gattungs-Interface-Funktionen *verwenden* die Achsen-Interfaces — wie auch Achsen-Algorithmen fremde Achsen-Interfaces verwenden dürfen, denn Container etwa werden überall gebraucht. *Makro-Benchmarking* (Ebene macro) misst die einzelnen `std::map`-Interface-Operationen (Einfügen, Punkt- und Bereichssuche, Aktualisieren, Löschen) gegen Beispiel-Lasten, und *Gesamt-Benchmarking* (Ebene wallclock, im `CacheEngineBuilder` über die Last-Rahmenwerke) fährt die YCSB-Lastprofile über jede Rekombination — `w/ma/mi`, in dieser Reihenfolge. Als Gegenprobe zur internen Mikro-Messung misst der Apparat zusätzlich drei *Wallclock-Ebenen*: die Wallclock der Achsen-Interfaces unter jedem Algorithmus, die Wallclock der Gattungs-Interface-Funktionen und die Wallclock einer Test-Last aus den Last-Rahmenwerken. Architektur-Auflage dieses Schnitts ist, dass alle Achsen-Algorithmen ausschließlich Achsen-Interfaces statt generischer Betriebssystem-Aufrufe verwenden — inline einkompiliert per Metaprogrammierung, ohne `std::variant` —, damit die Mikro-Messung einer Achse tatsächlich die Achse misst. Genau diese Methodik ist die Antwort auf das Trennbarkeits-Problem: Indem jeder Bestandteil zunächst isoliert

über sein Achsen-Interface vermessen wird, bevor er im Operations- und Gesamt-Kontext erscheint, werden die Beiträge getrennt messbar, statt im Gesamtdurchsatz vermischt zu bleiben.

Den so aufgespannten Raum reduziert das System gezielt. Der golden-Referenzkatalog — je Organ-Achse zwei aktivierte Vertreter, die Persistenz-Achse auf ihren In-Memory-Vertreter festgelegt — zählt $2^{17} = 131072$ Binary-Identitäten je System-Permutation (je Kombination der Bau-Steuergrößen, Abschnitt 3.4.1), jede mit vollem Achtzehn-Segment-Pfad (Abschnitt 3.4.4), unter der vereinfachenden Annahme genau zweier Auslegungen je Achse (an/aus) über siebzehn Achsen; diese Annahme ist überholt, die Achsen tragen mehrere Auslegungen, und der Katalog bleibt Regressions-Detektor des Ganzsystems mit einer festgeschriebenen CRC-64-Prüfsumme über alle Identitäten, nicht Obergrenze (auch der kleine 320er-Katalog ist als Mess-Kanon überholt; Kanon ist das XML-Profil über die Planer- und Director-Kette). Die Obergrenze der Binary-Identitäten je System-Permutation ist keine Konstante, sondern das Produkt über die Organ-Achsen der Zahl der im XML-Experiment gewählten Auslegungen (je Achse mehrere, nicht an/aus); der Planer bestimmt sie aus den Eigenschaften des XML-Experiments, sie ist erst zur Ausführung bekannt. Die Mächtigkeit des realen Permutationsraums über alle Achsen-Kataloge samt ihrer Unter-Achsen rechnet der Planer dagegen dynamisch aus seinen Eingängen (Subkommando `simulate`) als Obergrenze, die erst zur Ausführung bekannt ist — sie liegt Größenordnungen darüber und ist nie eine feste Anzahl. Die Planungs-Simulation rechnet heute das Produkt der XML-Freigaben je Organ-Achse mal System-Permutationen arithmetisch; die Planer-Erweiterung, die diese Obergrenze und die Batch-Größe eines Experiments als Grundlage der Last-Abschätzung bei der Ausführung korrekt erfasst, ist geplant und steht aus. Drei Mechanismen dämpfen die Explosion: Der `<expected_workload>`-Profil-Filter routet nur kompatible Workloads; der `Defined`-Modus führt nur die deklarierten Tupel aus, aus denen die Messreihen des Manuskripts gespeist werden; und der `Full`- bzw. `Full-Sampled`-Modus fährt den Vollraum oder eine deterministische, seed-stabile $1:n$ -Teilmenge davon (Vorgabe $n = 1000$). Der Default-Workload ist die Voll-Permutation über alle verfügbaren Workloads; das Routing dieser Vorgabe ist vorgesehen, aber nicht implementiert (Abschnitt 5.1.3). Voll- und Sampled-Läufe fahren auf den registrierten Maschinen-Signaturen — `prod1` (AMD Zen 5), `prod2` (Intel Core i9-12900K, Alder Lake) und ein ODROID-Board mit Intel-Gracemont-Kernen — unter Beachtung der vom Plattform-Probe gemeldeten Achsen-Kompatibilität; die Zugehörigkeit zum ZIH-Cluster der TU Dresden ist als Unter-Achse der Ziel-ISA deklariert, ein Lauf dort ist nicht gemessen (n/a).

3.5 PRT-ART als Demonstrator

PRT-ART ist der *Prüfling*, an dem sich die Tragfähigkeit des Systems zeigt: ein abstraktes Lebewesen, das einige Achsen-Organe selbst stellt und die übrigen über die Verbund-Strategie des Merge-Plans aus dem Standard-Baukasten der Cache-Engine bezieht (Abschnitt 3.4.6) — als monomorphe CRTP-Concept-Achsen, ohne Laufzeit-Fallback; die hybride `PrtArtSearchEngine<Ts...>`-API bleibt davon unberührt, und die Laufzeit-Anbindung erfolgt über den Execution-Engine-Adapter, nicht über eine eigene Such-Engine-Schicht. Geladen wird das Prüfling-Repo über den Plugin-Controller der Cache-Engine (`COMDARE_CE_PRUEFLINGE`): Sein Einstiegs-Snippet muss sich per `comdare_pruefling_deklarieren` ausdrücklich *zusichern* — mit der einzigen registrierten Fähigkeit `pruefling_slots_v1`, den Prüflings-Slots —, denn Anwesenheit darf nur widerlegen, nie erlauben: Die hinterlegten Beleg-Header können die Zusicherung nur brechen, niemals ein Gatter öffnen, und ein

angeforderter, aber nicht deklarierender Prüfling bricht die Konfiguration mit Fehlermeldung ab, statt still übersprungen zu werden; ein prüfling-eigener Adapter-Pfad ist deaktiviert und nicht Teil dieser Zusicherung. Was PRT-ART beiträgt, liegt auf drei Ebenen. Erstens die *Prüflings-Slots* im eigenen Repository: die Build-Achse Page-Type, der Prefetch, das Wert-Handle (ChainRef) und das Telemetrie-Tooling mit den Hypothesen-Metriken H1/H2/H3 (in der Dreiteilung eine Mess-Tooling-Unter-Achse, Abschnitt 3.4.1); die Bewertung der Hypothesen-Metriken enthalten die Kapitel 5 und 6. Zweitens die Referenz-Komposition der Cache-Engine, *PrtArtComposition*, die sich von der ART-Komposition in genau einer Achse unterscheidet — der Pfadkompression als Redirect-Organ, dem *R* in PRT-ART; das optimistische Lock-Coupling ist von ART geerbt, kein PRT-ART-Beitrag. Drittens die codegen-fähige Prüflings-Komposition, die sechzehn von achtzehn Achsen aus der ART-Komposition übernimmt und Prefetch sowie Wert-Handle durch die Prüflings-Slots ersetzt. Seine weiteren eigenen Bausteine, der 4+1+2-Pool-Allokator (Free-List-Unter-Achse) und das MultiLevel-Layout, liegen im Prüfling-Repo und laufen im Mess-Pfad über den Cache-Engine-Standard (Anbindung als nachfolgender Umsetzungsschritt vorgesehen, aber nicht implementiert); für die übrigen Achsen verwendet er bestehende SOTA-Bausteine über die Permutations-Konfiguration wieder. So belegt der Prüfling, dass das Mess-System tragfähig ist: Ein neuartiges Lebewesen entsteht aus wenigen eigenen Organen plus dem Standard-Baukasten (Abbildung 3.17) und ist über dasselbe Prüf-Dock messbar wie jedes rekonstruierte SOTA-Verfahren.

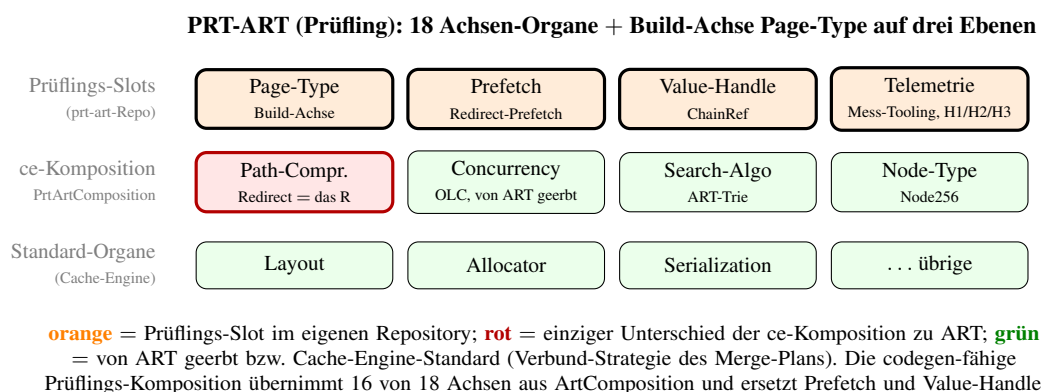


Abbildung 3.17: PRT-ART als Demonstrator im M-Modell auf drei Ebenen: die vier Prüflings-Slots im eigenen Repository (Page-Type als Build-Achse, Prefetch, Wert-Handle, Telemetrie als Mess-Tooling-Unter-Achse, Abschnitt 3.4.1); die Referenz-Komposition der Cache-Engine, die sich von ART in genau einer Achse — der Pfadkompression als Redirect-Organ — unterscheidet, während das optimistische Lock-Coupling von ART geerbt ist; und die codegen-fähige Prüflings-Komposition, die sechzehn von achtzehn Achsen aus der ART-Komposition übernimmt und Prefetch sowie Wert-Handle ersetzt. Die weiteren eigenen Bausteine (4+1+2-Pool-Allokator, MultiLevel-Layout) laufen im Mess-Pfad über den Cache-Engine-Standard (Anbindung: nachfolgender Umsetzungsschritt, nicht implementiert). So entsteht ein vollständiges, mess-fähiges Lebewesen aus wenigen eigenen plus vielen Standard-Organen.

3.6 Schärfung durch Heuristiken

Die Messreihen liefern mehr als Ranglisten: Aus ihnen lassen sich Heuristiken gewinnen, die das Mess-System schrittweise von einem Prüfstand zu einem sich selbst optimierenden Auslieferungs-System

schärfen. Dieser Abschnitt entwickelt diese Schärfung als Konzept in drei Stufen — die Heuristik-Schleife über gelernte Lastprofile, die vier Betriebsmodi des Builders bis zum Hybrid-Modus als Zielbild der Arbeit und das Messkurven-Typsystm samt Filterkette und Messfehler-Erkennung, das diese Modi mit Daten versorgt.

3.6.1 Heuristik-Schleife und gelernte Lastprofile

Aus den Messreihen lässt sich je Last- und Datentyp-Profil die beste Achsen-Komposition ableiten und als wiederverwendbares Profil ablegen.

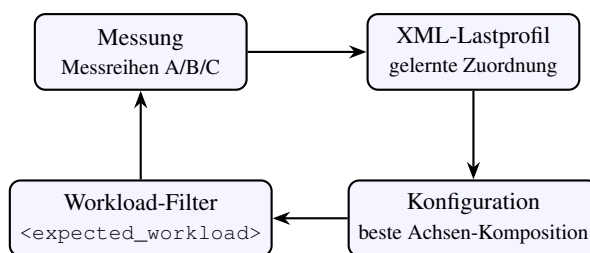


Abbildung 3.18: Die Heuristik-Schleife: aus den Messreihen wird je Lastprofil die beste Achsen-Komposition gelernt, als XML-Lastprofil abgelegt und über den Workload-Filter in die nächste Messung zurückgeführt.

Die volle Schleife ist ein Ausblick-Mechanismus; realisiert sind davon die automatische Auswahl und der Binary-Versand (Abschnitt 1.4). Der Selektor liest die Messwert-Mappe — die `xlsx`-Mappe ist die Ausgabe und der Standard des Apparats; die CSV-Form ist kein einzelnes Kind-Dokument, sondern ein aus der Mappe erzeugter, geordneter Baum aus CSV-Dateien mit denselben Spalten (ein Gesamtordner je Mappe, darunter ein Ordner je Blatt, darin die CSV-Dateien der einzelnen Unter-Achsen-Fahrten) —, rankt je Interface-Funktion und Metrik über die gültigen Zwei-Phasen-Zeilen (Median, Nearest-Rank), bestimmt bei konkurrierenden Zielgrößen die Pareto-Front der nicht-dominierten Kandidaten statt eines Einzelsiegers, löst die `binary_id` zur Binary im Lager auf und liefert Binary, Versions-Sidecar und Manifest als eigenständiges Artefakt aus. Die Heuristik-Kurven entstehen als natürliche kubische Splines über die Stützstellen je Achse und x -Dimension; ein Entscheidungs-Lambda-Baum liest die Break-Even-Schwellen aus den Spline-Schnittpunkten und wählt an einer Anfrage-Koordinate rückwärts die führende Binary — synthetische Stützstellen weist er als Gerüst aus, ohne gültige Kurven trifft er keine Entscheidung. Über die reine Rangbildung hinaus ist eine *Heuristik-Extraktion* als Folge-Phase nach **Persist** — der letzten Phase der Sieben-Phasen-Pipeline (Abschnitt 4.3.2) — vorgesehen, die aus den gesammelten Messungen automatisch ableitet, welche Achsen-Konfiguration je Workload und Datensatz die beste ist. Der angestrebte Mechanismus ist eine ML-Klassifikation in drei Schritten: Zunächst werden aus der Messwert-Mappe je Permutation Merkmale gewonnen — die Lastprofil- und Datensatz-Merkmale (Workload-Typ, Schlüssel-Verteilung, Schreib-/Lese-Anteil, Alphabet- und Präfix-Charakteristik sowie Schlüssel- und Wert-Datentyp) sowie die Achsen-Belegung —, mit der je Lastmuster gemessenen Bestleistung (etwa Durchsatz oder Tail-Latenz) als Zielgröße; darüber lernt ein *ML-Klassifikator* die Abbildung von Lastprofil-/Achsen-Merkmalen auf die beste Konfiguration; und dieser liefert je Workload-/Datensatz-Klasse die empfohlene Achsen-Komposition, ohne den gesamten Permutationsraum erneut zu fahren. Das Ergebnis wird als wiederverwendbares *XML-Lastprofil* persistiert — symmetrisch zu den Messreihen-XML-Konfigurationen — und kann ohne erneute Vollvermessung in

den `<expected_workload>`-Filter zurückgespeist werden; damit schließt sich der Kreis zwischen Messung, gelernter Heuristik und konfigurationsleitender Auswahl (Abbildung 3.18). Dieser *measure* $\rightarrow$ *profile* $\rightarrow$ *config* $\rightarrow$ *filter*-Kreislauf ist die klassische, etablierte Form des *empirischen, messgetriebenen Autotunings*: Er steht in der Linie von Autotuning-Frameworks wie OpenTuner [AKV⁺14] und den messen-im-Kreis-Autotunern FFTW [FJ98] und ATLAS [WPD01]; die lastprofil-getriebene Konfigurationswahl mit Workload-Filter findet ihre Datenbank-Entsprechung in OtterTune [APGZ17] und im Database Tuning Advisor mit Workload-Kompression [ACK⁺04], die online-laufzeitnahe Variante in Active Harmony [TCH02]; die Bezeichnung „Software-in-the-Loop“ selbst entstammt der X-in-the-Loop-Validierungstaxonomie [TMS⁺16]. Die ML-Klassifikation ist nicht implementiert und bleibt zukünftiger Arbeit vorbehalten (Kapitel 6); die Schleife selbst ist jedoch nur die erste Stufe eines größeren Betriebs-Zielbilds, das der folgende Unterabschnitt entwickelt.

3.6.2 Die vier Betriebsmodi des Builders

Die Heuristik-Schleife ist der Kern eines Betriebsregimes aus *vier* aufeinander aufbauenden Betriebsmodi des Builder-Interfaces. Von der Ablauf-Kette *build* $\rightarrow$ *measure* $\rightarrow$ *compare* $\rightarrow$ *release* (Abschnitt 3.3.2, Abbildung 3.10) sind diese Betriebs-Zielbilder zu unterscheiden, in denen der Apparat betrieben wird; die Zuordnung ist eindeutig: Der *Mess-Modus* umfasst *build* und *measure*, der *Auswertungs-Modus* ist *compare*, der *Arbeits-Modus* ist der *release* einer Einzel-Binary, und der *Hybrid-Modus* ist der *release* einer Hybrid-Rekombination, deren Kette sich auf der Hybrid-Ebene wiederholt. Der erste Modus ist der Betrieb des Apparats; die Modi zwei bis vier sind das erklärte Ziel dieser Arbeit und werden hier als Konzept festgehalten. Der Hybrid-Modus ist dabei ein erweitertes, optionales Konstrukt: Der Kern der Arbeit ist das Durchmessen und Erstellen der Tier-Binaries samt Break-Even-Analyse vor jeder Anwendung des Hybriden, und die Arbeit geht in der Kette nicht über die Vergleichsstufe der Hybrid-Ebene (*hybrid-compare*) nach der Auslieferungsstufe der Einzel-Binary (*single-release*) hinaus. Von diesen vier Betriebsmodi zu unterscheiden ist der in Teilfrage FF2.c (Abschnitt 1.3) erfragte *Paper-Isolations-Modus*: ein Betriebsmodus, der die Entwurfsbestandteile genau einer Veröffentlichung als *Achsen-Set* — die geschlossene Achsenmenge dieses einen Referenzentwurfs, bei weiterhin freistehenden Mess- und System-Achsen — isoliert anbietet, um die Veröffentlichung originalgetreu durchzumessen; er ist nicht implementiert und bleibt Ausblick (Abschnitt 6.3).

Im *Mess-Modus* — dem Betrieb, den Abschnitt 3.4 beschreibt — fährt der Builder die Messreihen über die zugelassenen Permutationen. Nach Abschluss der Messung kennt das Framework die Eigenschaften aller zugelassenen Achsen-Algorithmen gegen die Rekombination aller zugelassenen Lastprofil-Rahmenwerke und Workloads. Der *Auswertungs-Modus* verdichtet diese Messungen: Aus ihnen ergeben sich minimale Stränge der Permutationen aller Eigenschaften gegeneinander für bestimmte *Workload-Cluster* — für Gruppen ähnlicher Lasten bleibt der kleine Satz von Achsen-Kompositionen übrig, der sie dominiert. Workloads werden dafür nach möglichst ähnlichen Abfolgen gleicher Zugriffsmuster auf die Konzept- und Gattungs-Interfaces der Tier-Binaries geclustert: Jedes Last-Rahmenwerk durchläuft seine geplante Strategie einmal trocken, und Abfolge, Häufigkeit und Anforderungen seiner Interface-Aufrufe werden notiert — das, was ein Durchlauf des Rahmenwerks statisch von sich gibt. Daraus entsteht je Profil ein gerichteter Übergangsgraph der Interface-Zugriffe mit absoluten und relativen Folgewahrscheinlichkeiten (die Abbildung eines Interfaces auf sich selbst ist als Kante zugelassen); seine Kantenmatrix wird mit dem EM-Algorithmus [DLR77] ausgewertet, und die Abstände der Mittelpunkte der Wahrscheinlichkeits-

Punktwolken zueinander ergeben je Profil ein *Punkt-Wolken-Wort* variabler Länge (kürzer, wenn Interfaces nicht aufgerufen werden — null-gefiltert). Die lexikografische Sortierung dieser Wörter ist die Mess-Reihenfolge: Nebeneinanderliegende Wörter sind ähnliche Lasten und werden — gleich aus welchem Rahmenwerk sie stammen — hintereinander gefahren. Dieses Verfahren ist im Code nicht implementiert; es ist zugleich die vierte Auswertungs-Komponente des Hybrid-Tiers (Abschnitt 3.6.3).

Der *Arbeits-Modus* ist das Produktions-Ziel: Das Builder-Interface hält alle für die aktuell stochastisch häufig auftretende Workload-Last und Interface-Funktion relevanten Tier-Binaries vorgeladen im Arbeitsspeicher und wechselt intern die Tier-Binary mit ihren speziell optimalen Achsen-Permutationen am ABI-stabilen Interface — derselbe Mechanismus des Binary-Wechsels, den der Mess-Modus nutzt, nur last-getrieben statt mess-getrieben. Ebenen-richtig bleibt dabei der Übersetzungszeit-Grundsatz gewahrt: Es gibt keinen Laufzeit-Umschalter *im* Tier — die Binaries selbst bleiben übersetzungszeitlich permutiert; gewechselt wird *zwischen* vorgeladenen Binaries an der bewussten ABI-Grenze. Das Prüf-Dock der Gattung (Abschnitt 3.3.2) heißt in diesem Modus *Arbeits-Dock*: Dieselbe Schnittstelle, über die im Mess-Modus geprüft wird, liefert nun die produktiven Antworten. Die Dock-Schicht dieses Wechsels ist implementiert — die Dock-Verträge, das Prüf-Dock, die Dock-Fabrik als einziger Konstruktions-Ort, das Dock-Feld mit Anschluss und Trennung, der Parser der Hybrid-Sektion der XML und der Binary-Proxy, der eine Hybrid-Binary nach außen zu einer gewöhnlichen Tier-Binary macht —; Router, Eviction und der `.so`-Export des Wechsels sind nicht implementiert. Der Hybrid kompiliert stets mithilfe des `CacheEngineBuilders`, nie selbst.

Beim Wechsel von einer Tier-Binary zur anderen muss jede beteiligte Tier-Binary eine weitere *Zustands-Fläche* einkompiliert enthalten, die den Zustand im Hybrid-Modus liest und wieder setzt — sparse: Jede Binary behält ihren Zustand und zieht nur die versioniert geänderten Blöcke nach, ein Checkpoint-Verfahren nach dem *Memento*-Muster [GHJV94], das die Interfaces umgeht und den Speicher blockweise nur für geänderte Bereiche nachführt. Der Arbeitsspeicher vervielfacht sich dabei, weil alle beteiligten Binaries den Gesamtzustand halten, von denen stets eine vorn und aktuell ist. Die Zustands-Fläche ist eine einkompilierte, metaprogrammierte Sidecar-Bibliothek, die nur im Hybrid-Muster auftritt — im Einzelbetrieb enthält eine Tier-Binary sie nicht; deshalb werden zwei Release-Artefakte eingelagert, ein Release für den Einzelbetrieb ohne Sidecar und ein Release für den Hybrid-Betrieb mit Sidecar (Abschnitt 3.4.5). Das Hybrid-Binary organisiert die Zustände im Protokoll des Wechsels und setzt das Checkpointing für alle metaprogrammiert angebotenen Tier-Binaries durch: Der berechnete Wechsel je Hybrid-Anfrage erzeugt einen Speicher-Abgleich zwischen den Tier-Binaries und leitet die Anfrage dann an die gewählte Tier-Binary durch. Zustands-Fläche und Sidecar sind nicht implementiert.

Der *Hybrid-Modus* schließlich — das Ziel der Arbeit — verbindet Arbeitsmodus und Messung. Sein Kernstück ist, dass der Heuristik-Adapter eine eigene *Gattung* mit eigenem Gattung ist — die vierte Gattung `HeuristikAdapter` mit der `Reroute-Gattung` `FunctionInterfaceReroute` (Abschnitt 3.3.2): Er übernimmt per Metaprogrammierung die Gattungs- und Gattungs-Interfaces seiner Tier-Binaries, wird selbst zu einer Tier-Binary kompiliert und dockt am Prüf-Dock des Builders an wie jedes einzelne Tier; Befehle reicht er über das *Command*-Muster [GHJV94] an die ihm statisch zugewiesenen einzelnen Tier-Binaries weiter. Nach außen ist er von einer einzelnen Binary nicht zu unterscheiden, weil seine Binary dieselben Pflicht-Symbole exportiert wie jede andere und als Gattung die geerbte Ziel-Gattung meldet — Klassifikation (die Art des Reroutes) und Durchstellung liegen auf getrennten Ebenen. Die Rekombination der zu verwendenden Heuristik-Tier-Binaries ist dabei selbst permutierbar gegen die

existierenden einzelnen Tier-Binaries; der Heuristik-Adapter kann daher die Mess-Funktionskurven für Schätzungen verwenden und eine optimale mehrdimensionale Rekombination kompilieren, die durch ihr ABI-stabiles Gattungs-Interface eindeutig als Suchalgorithmus-Hülle verwendbar ist — nach außen ein *virtuelles ganzes Tier-Binary*. Zugleich misst der Hybrid-Modus mit: Über die optimierte Rekombination werden die kombinierte Wall-Clock-Zeit sowie Makro- und Mikro-Benchmarks per Mess-Command über alle Tier-Binaries und Operationen mitgeloggt.

Damit erbringt der Hybrid-Modus den *heuristischen Gegenbeweis*: Nach Eintritt des Arbeitsmodus wird das Heuristik-Tier-Binary erneut vermessen — sein Wall-Clock-Verhalten im Vergleich gegen die besten Einzel-Binaries und gegen die rekonstruierten Paper-Algorithmen entscheidet, ob die heuristische Rekombination die von den Kurven vorhergesagte Leistung erreicht. Die Kurven, die der Gegenbeweis prüft, sind die Spline-Kurven je Achse und x -Dimension mit ihren Break-Even-Schnittpunkten (Abschnitt 3.6.1); in der Stufe compare wird aus dem Messwertlager errechnet, welche Tier-Binary oder Hybrid-Rekombination laut Vermessung die beste sein muss, und in der Stufe release wird die maximal optimiert kompilierte Binary ohne einkompilierte Messfühler nochmals durchgemessen — ob ihre Wall-Clock ohne Mess-Einblick die zuvor vermessene Gesamtzeit sogar noch unterbietet. Der Vergleich gegen die rekonstruierten Paper-Algorithmen setzt je Veröffentlichung eine Wall-Clock-Referenz voraus, die als Datenquelle fehlt und je Verfahren aus der jeweiligen Veröffentlichung zu ermitteln ist; der Paper-Vergleich hat Vorrang vor der Pareto-Auswahl. Das Ziel der Arbeit ist damit doppelt formuliert: die beste Rekombination beziehungsweise das beste einzelne Binary gegenüber allen Achsen-Eigenschaften — besonders der Cache-Line-Bewusstheit — zu bestimmen und am Ende beides produktionsfähig bereitzustellen, die beste einzelne Binary *und* das Heuristik-Tier-Binary bestehend aus den besten gemischten cache-bewussten Binaries für die vermessenen Lasten. Alle vier Modi sollen dabei über die Auswertungs-Kette der Definitions-Schicht (Abschnitt 3.3.4) automatisch dokumentiert werden; die Messwerttabellen und Diagramme dieser Arbeit selbst sind das Erzeugnis der compare-Stufe dieser Kette, und die Bewertung des Gegenbeweises definiert Kapitel 5.

3.6.3 Messkurven-Typsystem, Filterkette und Messfehler-Erkennung

Damit der Auswertungs- und der Hybrid-Modus tragfähig sind, braucht die Auswertung mehr als flache Ergebnistabellen. Als Zielbild formuliert diese Arbeit dafür ein *Messkurven-Typsystem*; implementiert ist davon ein erster Ansatz — eine Kurvenanpassung über eine Achse und eine x -Dimension als natürlicher kubischer Spline (Abschnitt 3.6.1) —, während der Typsystem-Schlüssel Rahmenwerk $\times$ Workload $\times$ Operation $\times$ Größe nicht implementiert ist. Jede Tier-Binary-Gesamtpermutation erhält je Mess-Achse eine Wall-Clock-Messkurve über das Working-Set — etwa die Lookup-Latenz als Funktion der Füllgröße —, und zwar getrennt nach Lastprofil-Rahmenwerk, Workload-Typ, Working-Set-Größe, Operationstyp und Beobachter-Typ und je Mess-Ebene wallclock, macro und micro (Abschnitt 3.4); jede Achse wird im Experimentbaum übersetzungsstatisch wie laufzeit-dynamisch separat durchgefahren, sodass jede Achse eine vollständige Übersicht ihrer Eigenschaften und Einzelmessungen erhält. Über alle Permutationen zusammengenommen entsteht so je Tier-Binary eine mehrdimensionale Mess-Datenbank, aus der die Heuristik-Kurven der Schleife (Abschnitt 3.6.1) abgelesen werden; die synthetisierten Kurven werden als Dateien im Lager gehalten (Abschnitt 3.4.5). Die Kardinalität jeder Dimension dieses Typsystems ist strategisch in Klassen und Hierarchien einzuteilen: design-feste Konstanten (etwa die achtzehn Organ-Achsen), registrierte, dynamisch wachsende Größen (die Mess-Kategorien, Abschnitt 3.4.1), profil-

gebundene Produkte (was ein Experiment-Profil an Belegungen zulässt) und laufzeit-freie Iterationsräume (die dynamischen Stützstellen je statischer Binary) — so bleiben die System-Achsen in der Lage, die Auswertung über alle Ebenen hinweg zu stützen.

Die compare-Stufe setzt sich aus fünf Auswertungs-Komponenten zusammen, die Stufungen einer Kette sind, nicht Alternativen: erstens das Ranking je Metrik mit Pareto-Front und Versand (implementiert, Abschnitt 3.6.1); zweitens die Spline-Kurven mit der Break-Even-Entscheidung (als erster Ansatz implementiert); drittens die Filterkette über die Permutations-Familien (Mechanismus implementiert, Auswerte-Glieder nicht implementiert, nächster Absatz); viertens das Live-Ranking der vorgeladenen Tier-Binaries im Hybrid-Tier aus den zur Laufzeit mitgeschnittenen Interface-Zugriffen und ihren Übergangswahrscheinlichkeiten — dasselbe Cluster-Verfahren wie bei der Mess-Reihenfolge (Abschnitt 3.6.2), das die bekannten Interface-Eignungen einer Tier-Binary gegen die gemessenen Anforderungen hält und die Anfrage an die beste Binary durchreicht (nicht implementiert); und fünftens die Messfehler-Interpolation über die Mess-Konstellationen (nicht implementiert, übernächster Absatz).

Aus den Auswertungen ergibt sich eine *Filterkette*: Jedes Kettenglied prüft anhand der bisherigen Kurven, ob eine Permutations-Familie weiterverfolgt wird, und reicht die Entscheidung andernfalls an das nächste Glied weiter — das Entwurfsmuster *Chain of Responsibility* [GHJV94]. Die Kette ist im CacheEngine-Builder verortet, weil sie die *Generierung* der nächsten Bau-Runde kontrolliert und die Feedback-Kante von der Auswahl zur Bereitstellung schließt: Auswertung, Filter, dann die Entscheidung, welche Permutationen als Nächstes gebaut und welche verworfen werden. Da in compare build und measure bereits durchlaufen sind, filtert sie zugleich aus den Messwerten für den release, welche Binary nach den in der XML geforderten Eigenschaften statistisch die schnellste für einen Verwendungszweck ist — Vorgabe ist die Ausführungszeit, weitere Filter sind nicht Gegenstand dieser Arbeit. Implementiert sind der Mechanismus (mit den Urteilen Pass, Reject und Weiterreichen: Ein Kandidat überlebt genau dann, wenn kein Glied ihn verwirft) und ein Kern-Glied für die Wiederaufnahme unterbrochener Läufe; die Auswerte-Glieder — der Paper-Vergleich, dem die Referenzquelle fehlt, und die Pareto-Dominanz, der der Typsystem-Schlüssel fehlt — folgen als weitere Glieder, mit dem Vorrang des Paper-Vergleichs vor der Pareto-Auswahl.

Schließlich braucht der Hybrid-Modus eine Kontrolle des Messfehlers, den die Mess-Instanzen selbst eintragen — Beobachter kosten Zeit. Die drei Mess-Einrichtungen — die Wall-Clock im CacheEngineBuilder, die Gattungs-Interface-Zeit der Tier-Binary (macro) und die Achsen-Interface-Zeit (micro) — lassen sich als dreiphasige Vertragskonstruktion zwischen CEB und Tier-Binary in allen Anordnungen bauen: bei drei Einrichtungen $3! = 6$ Prüfdock-Konstellationen, die nur an der Tier-Binary gemessen werden müssen, mit eingeschobener Hybrid-Tier-Binary $4! = 24$ Konstellationen der Kette CEB-Hybrid-Tier; über mehrere hintereinander geschaltete Hybrid-Stufen (Hybrid-Meta-Meta-Stufen, übersetzungsstatisch über die XML erweiterbar) ist die Zahl dynamisch $n \times m$ und wird nie statisch vorgelegt. Aus den Differenzen aller Messwerte mit eingebauten Messfühlern gegen die jeweils genaueste verbleibende Gesamtzeit, bezogen auf die Zahl der feinsten Interface-Zugriffe, interpoliert ein mathematisches Verfahren über die Konstellations-Matrix den Messfehler und rechnet ihn aus den veröffentlichten Ergebnissen heraus — die fünfte Auswertungs-Komponente. Die drei zuvor benannten Bau-Varianten desselben Experiments — erstens Beobachter in allen Tier-Binaries *und* im Heuristik-Hybrid-Tier, zweitens Beobachter nur im Heuristik-Tier-Binary, drittens Beobachter in keinem, nur die Wall-Clock — sind drei dieser Konstellationen. Komplementär folgt ein *vierter Schritt* der finalen heuristischen Zusammenstellung: Die

dem Heuristik-Tier untergebenen Tier-Binaries (nicht die final optimierte Haupt-Binary) werden ohne Mikro- und Makro-Benchmarks und ohne Beobachter neu gebaut und rein an der Wall-Clock gegen alle bekannten Tier-Binary-Permutationen und gegen die bekannten Paper-Algorithmen verglichen — wobei der Paper-Vergleich Vorrang hat. Gültigkeits-Kriterium jeder Konstellation ist, dass die Messung mit Messfühlern langsamer ist als dieselbe ohne. Gelegt ist das Fundament: die Mess-Gates je Kompilat als Fingerprint-Glied, deren Deckung CEB aus, Tier an durch einen Test belegt ist, und die ausgewiesene Nichterhebung je Messwert (Abschnitt 3.4.1); die Interpolation und die Konstellations-Matrix sind nicht implementiert.

Damit ist klar getrennt, was implementiert und was Ziel ist. Implementiert sind der Mess-Apparat mit seinen Verbund-Stufen, die Auswertungs-Kette bis zu Tabellen und Diagrammen, die Kurvenanpassung als erster Ansatz samt Break-Even-Baum, die Pareto-Auswahl mit Versand, der Filterketten-Mechanismus mit Wiederaufnahme-Glied, die Dock- und Klassifikations-Schicht des Hybrid-Tiers samt XML-Parser und Binary-Proxy und die Mess-Gates je Kompilat. Die Messläufe der vollständigen Messreihe liegen nicht vor. Nicht implementiert sind die Auswerte-Glieder der Filterkette, Router, Eviction und .so-Export des Hybrid-Wechsels, die Zustands-Synchronisation mit Sidecar, die Workload-Clusterung, die Messfehler-Interpolation, der Typsystem-Schlüssel der Messkurven, die Erhebung je Achsen-Interface-Parametersatz und die Anbindung der P-/E-Kern-Messung an das Lauf-Profil. Die Bewertung des heuristischen Gegenbeweises nimmt Kapitel 5 auf; die Einordnung des Ausblicks über diese Arbeit hinaus gibt Kapitel 6. Zunächst vertieft Kapitel 4 die konkrete Implementierung der hier eingeführten Bausteine.

4 Implementierung und konkrete Algorithmusbestandteile

Dieses Kapitel steigt in das Detail: zuerst die Repository-Architektur mit der Träger-Stufenkette (Abschnitt 3.3.2) und den vier Arbeitsmodi einer Permutation, dann jede Schlüsselstelle von PRT-ART einzeln und bildhaft, anschließend die Achsen-Implementierung mit der *einen* Architektur im Code, der Code-Gestalt der Bau-Kette, den Ebenen E4–E0 im Code und den Vertrags-Flächen an den Träger-Binaries, danach die Qualitätssicherung über Contract-Tests je Schicht samt Zwei-Gate-Protokoll und die Code-Gestalt der Realm-Wurzeln — System- und Mess-Achsen einschließlich der Hardware-Erkennung — und zuletzt das Register der Entwurfsmuster, die Hybrid-Stufe und das Lager. Im Kern ist die *eine* Architektur eine selbst-optimierende Compiler-Compiler-Struktur: Der Planer erzeugt aus der Anwender-XML die Bau-Aufträge, der Builder erzeugt daraus die Kompilate, und die Auswertung der Messwerte speist die nächste Planung (Abschnitt 3.4.6). Damit steht der Apparat in der Linie der empirischen Autotuning-Systeme, die Kandidaten-Implementierungen erzeugen, vermessen und die Auswahl aus den Messwerten treffen [WPD01, FJ98, AKV⁺14, BDG⁺18]; der Unterschied liegt darin, dass hier nicht die Parameter einer festen Implementierung, sondern die Organ-Belegung eines Lebewesens permutiert und je Permutation eine eigene Binary erzeugt wird.

4.1 Vier-Repository-Architektur

Die Implementierung verteilt sich auf vier Git-Repositorien mit klar getrennten Verantwortlichkeiten:

- `comdare-cache-engine` — **wie** gemessen wird: die Bausteine-Bibliothek und der Builder (XML-Config-Parser, `generate_module_from_profile`-Codegenerierung, Permutations-Schleife, Modul-Loader, der siebenphasige `ExperimentDriver` mit den Phasen `phase1_enumerate` bis `phase7_export` und der Experiment-Planer `ExperimentPlanDirector`, Abschnitt 4.3.2); die ABI-Header unter `abi/` — der lebende ABI-Kern `module_abi_v1` sowie der vom Bau isolierte Test-Cluster aus `baustein_variants` und `resolve_baustein` (Abschnitt 4.3) —; die 33 SOTA-Profilen unter `algorithm_profiles/sota` als ausführbare XML — 30 Vollprofile und drei abstrakte Profile (Abschnitt 3.2) — und die Test-Suites. Die Bibliothek `search_engine` ist ein deklariertes, inhaltsleeres Skelett (siebzehn Dateien, davon acht Platzhalter und neun Bau-Beschreibungen ohne Quellcode), und `execution_engine` enthält drei Inhaltsdateien (Ergebnis-Aggregator und Experiment-Demonstration); die lebende Wurzel der Ausführungs-Engine liegt unter `libs/cache_engine/execution_engine` (Abschnitt 4.3.1). Der Quellbaum folgt einem Pitchfork-Layout (`apps/` für ausführbare Dateien, `libs/` für Bibliotheken, `libs/common` für querschnittlichen Code); hinzu treten `libs/traeger` mit den vier Träger-Stufen (unten), `libs/cache_engine/heuristik` mit sechs Headern (Achsen-Spline, Break-Even,

Messkurven-Lader, Offline-Clustering der Workloads, Merkmalsvektor, Optimierungs-Katalog) und `libs/test_infra` (Benchmark-Suite, Workload-Generator, Testdaten).

- `comdare-prt-art` — der **Prüfling**: die hybride `PrtArtSearchEngine` mit der 1/2/N-Parameter-Regel (ein Parameter fordert die Funktionsinterfaces eines Vektors, zwei die einer Map, mehr als zwei die einer Map auf ein Tupel), ihre ABI-Adapter und ihre Bausteine (4+1+2-Allokator-Pools — sieben Pools: vier Seiten-Pools mit den Rollen Trie-Hülle, Dense-Pages, MultiLevel und Decision-Span, ein Rest-Pool und zwei Wert-Pools für statische und dynamische Werte — , OLC mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, DensityTracker, Hypothesen-Metriken H1/H2/H3) sowie die prüflingseitigen Vergleichs-Re-Implementierungen der Rang-2/3-Arbeiten (`legacy_reimpl`, vierzehn Verzeichnisse P11–P27 ohne P15, P20 und P25), die im SOTA-Beitrittstest gegen die tatsächlichen Adapter der Cache-Engine verglichen werden — Rang 1 (Zentralitäts-Rang, Abschnitt 1.3) umfasst die acht eigenständigen Suchentwürfe — alle mit öffentlichem Quellcode und Original-Adapter — sowie die als Organe eingehenden Arbeiten P08 und P09; von den Rang-2/3-Arbeiten besitzen P20, P25, P29 und P30 ebenfalls Original-Adapter über `ext/`, die vierzehn Arbeiten in `legacy_reimpl` sind ohne öffentlichen Quellcode nach der Veröffentlichung re-implementiert (Abschnitt 3.2). Pool-, Prefetch- und Layout-Komponenten sind Deskriptor-Skelette ohne eigene Speicheranbindung; welche Prüflings-Bausteine den Mess-Apparat erreichen, entscheidet die Slot-Belegung (Abschnitt 4.2).
- `probst-Diplomarbeit-cache-engine` — **was** durch den Anwender getestet wird: die Experiment-Konfigurationen (XML samt XSD-Schema), der Mess-Treiber `messung_driver` (Binary `comdare-messung-driver`, die CEB-Rolle) und die Auswertungs-Werkzeugkette aus neun numerierten Programmen von `01_sample_data_generator` bis `09_tex_formatter` (Abbildung 4.8).
- das **Manuskript-Repository** — die versionierte Experimentdurchführung direkt im Dokument: Die Generator-Ausgabe der Werkzeugkette (Tabellen- und Diagramm-Dateien je Messreihe) wird per Commit unter `anhang/<sprache>/tabellen/` abgelegt und vom Manuskript eingebunden, das Repository besitzt eine eigene CI mit den Jobs `lint:secrets`, `lint:latex` und `thesis:pdf`, und das Projekt-Repository stößt den Manuskript-Bau nach einem Messlauf an (`trigger:thesis`). Ein Messwert im Dokument ist damit ein versionierter Artefakt-Stand mit Herkunft, kein abgeschriebener Wert — das ist der Grund, warum das Manuskript ein eigenes Git-Repository mit eigener Pipeline ist.

Die Vierer-Ordnung des Codes ist nicht die Repository-Grenze, sondern die *Träger-Stufenkette* `Planner` → `CEB` → `Tier` mit dem optional zwischen CEB-Prüf-Dock und `Tier` eingeschobenen Hybrid (Abschnitt 3.3.2): Der Stempel-Vertrag kennt genau vier Träger, und der Quellbaum bildet sie unter `libs/traeger` als vier Unterprojekte in Stufenform ab — `comdare_planner`, `comdare_ceb`, `comdare_tier_emission` und `comdare_hybrid` —, wobei in der Bau-Reihenfolge Stufe $N + 1$ nur an Stufe N hängt (ohne `Tier` kein Hybrid), nie umgekehrt, und jede Stufe mit einer strikten Konfigurationsprüfung auf ihre Vorstufe schließt. Umsetzungsstand dieser Ordnung: Die Unterprojekte bilden ein Gerüst (je Stufe Bau-Beschreibung und Beschreibungsdatei, `Planner` und `CEB` mit eigenem Include-Verzeichnis), und die eigentliche Implementierung liegt als Monolith unter `libs/cache_engine`; ihre

Aufteilung entlang der Stufen ist vorgesehen, aber nicht umgesetzt. Der Prüfling ist keine Stufe, sondern der quer liegende Lieferant von Organ-Bausteinen; Builder und CacheEngine teilen sich das Engine-Repository — der Builder als ausführbare Datei (apps/) über seiner Builder-Bibliothek (libs/cache_engine/builder), die CacheEngine als Bibliothek (libs/cache_engine). Die Stufenkette ist zugleich die Schicht, an deren Binaries die Verträge in Flächen zergliedert sind (Abschnitt 4.3.2); Abbildung 4.1 zeigt sie mit ihren Binaries, Legenden-Formen und Flächen. Quer dazu trennt die Terminologie

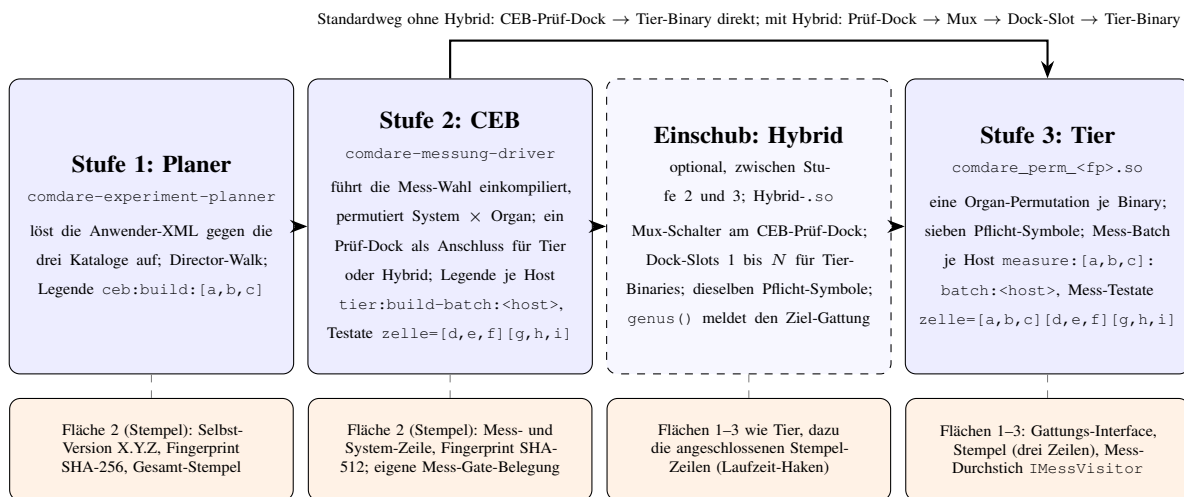


Abbildung 4.1: Die Träger-Stufenkette im Code: vier Träger, davon der Hybrid als optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Stufe (Stempel-Kette in fester Reihenfolge Planer → CEB → [Hybrid] → Tier), je Stufe die Binary, ihre Legenden-Form in der emittierten Pipeline und die an ihr ausgeprägten Vertrags-Flächen. In der Stempel-Kette hängt Stufe $N + 1$ nur an Stufe N ; der Standardweg führt vom CEB-Prüf-Dock direkt zur Tier-Binary, der Hybrid ist ein Mux-Schalter am Prüf-Dock mit Dock-Slots für Tier-Binaries (Bau-Abhängigkeit: ohne Tier kein Hybrid); nur die Mess-Testate enthalten alle drei Achsen-Klammern. Die Flächen zergliedern die Verträge (Abschnitt 4.3.2); die Unterprojekte unter `libs/traeger` sind ein Gerüst, die Implementierung liegt als Monolith unter `libs/cache_engine`.

zwei Engine-Begriffe, die nicht kurzgeschlossen werden dürfen. Die CacheEngine (libs/cache_engine) ist die *Bibliothek der Achsen-Bausteine*: Sie stellt je Achse die austauschbaren Entwurfsbestandteile als Concept-/CRTP-Komponenten bereit¹ und enthält *keine* Mess-Logik. Die ExecutionEngine hingegen ist die quer zu allen Achsen liegende *Ausführungs- und Mess-Infrastruktur*: Ihre abstrakte messbare Wurzel ist IExecutionEngine (Abschnitt 3.4) mit den drei Engine-Arten Anatomy, Virus und Hybrid; der konkrete Laufzeit-Adapter SearchAlgorithmAbiAdapter<A> liefert über die SearchAlgorithmAnatomy die OS-Primitiven (cache-line-ausgerichtete Allokation, NUMA-konformes Read-Pinning, kohärenz-bewusstes Schreiben); die Mess- und Observer-Kopplungen des Adapters werden über COMDARE_MEASUREMENT_ON geschaltet, und den Mess-Aggregator führt der ExperimentDriver host-seitig; die CMake-Option COMDARE_EXPERIMENT_MODE markiert Experiment-Kompilate und kodiert übersetzungszeitlich die Invariante, dass ein Experiment-Bau stets ein Mess-Bau ist — sie selbst schaltet keine Hooks, und ein Experiment-Bau ohne Mess-Modus bricht die

¹CRTP: das *Curiously Recurring Template Pattern* [Cop95], der statische Polymorphismus, bei dem eine Klasse von einer Klassenschablone abgeleitet wird, der sie sich selbst als Schablonenargument übergibt (Abschnitt 2.4); virtuelle Aufrufe werden so durch statisch aufgelöste ersetzt.

Konfiguration. Kurz: Die *CacheEngine* liefert die *Bausteine*, die *ExecutionEngine* die *Werkzeuge*; diese Trennung hält den Produktiv-Pfad mess-frei.

Was mit einer Permutation geschieht, ist eine Registry und kein Etikett: Die vier *Arbeitsmodi* (work modes) *build*, *measure*, *compare* und *release* sind als Unter-Achse des Mess-Realms geführt (Tabelle 4.1; Enum-Index, Ablauf-Ordnung und Registry-Zeile sind dieselbe Reihenfolge, Abschnitt 3.3.2). *build* baut die Binaries, ohne zu messen, und darf parallel laufen; *measure* ist der deterministische Ein-Thread-Messlauf und der einzige Modus mit eingeschalteter Messung; *compare* liest das Messwertlager und vergleicht bereits gemessene Sichten — die Stufe *vor* dem Release; *release* erzeugt die Referenz-Binary ohne Instrumentierung. Alle vier bauen als CMake-Release; daneben gelten die Bau-Varianten Debug, Messung und Release fort, und die Debug-Variante wird weiter ausgebaut — ein messendes Debug-Regime existiert jedoch nicht mehr, weil parallel erhobene Zahlen mit Ein-Thread-Messungen nicht vergleichbar sind, und der Verdrahtungs-Check, den es leistete, ist als Schalter *-debug* der Planer-Shell realisiert — ein Flag, kein Zustand; unter *-debug* fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces, und dieser indirekte Debug-Modus fällt mit dem Release-Bau zusammen und wird nicht separat geführt. Umsetzungsstand: *compare* ist ein wählbares Etikett mit *release*-gleicher Bausemantik; sein modus-spezifischer Ablauf (Replay-Vergleich statt Messen) ist für spätere Haupt-Wellen vorgesehen, aber nicht implementiert (Abschnitt 3.3.2).

Modus	CMake-Typ	misst	Mess-Threads	Zweck / Stand
<i>build</i>	Release	nein	—	Bau ohne Messung, parallel; der Schritt vor jeder Messung
<i>measure</i>	Release	ja	1	deterministischer Messlauf, Run-to-Run-stabil; füllt das Messwertlager
<i>compare</i>	Release	nein	—	liest das Messwertlager, Stufe vor <i>release</i> ; Ablauf vorgesehen, nicht implementiert (Etikett)
<i>release</i>	Release	nein	—	Referenz-Binary ohne Mess-Instrumentierung

Tabelle 4.1: Die vier Arbeitsmodi einer Permutation als registrierte Unter-Achse des Mess-Realms (*WorkMode*, vier Einträge; ein fünfter bricht die Übersetzung). Kein Modus misst parallel; nur *measure* schaltet die Messung ein.

4.2 PRT-ART im Detail

Aufbauend auf dem Stand der Technik beschreibt dieser Abschnitt jede Schlüsselstelle von PRT-ART einzeln und bildhaft; je Schlüsselstelle wird zuerst das verwandte Verfahren benannt, aus dem sie hervorgeht. Pool-Allokation mit rollenreinen Pools steht in der Linie der Größenklassen- und Thread-Heap-Allokatoren Hoard, jemalloc und mimalloc [BMBW00, Eva06, LZM19]; optimistisches Lock-Coupling ist das Synchronisationsverfahren des ART [LABN16]; Software-Prefetch entlang der Baumpfade folgt der Pfad-Bündel-Heuristik der Prefetching-B⁺-Bäume von Chen, Gibbons und Mowry [CGM01] (die verwandte Literatur des Bausteins im Code) und dem hierarchischen Prefetching [ZGH⁺25]; cache-line-bewusste Knoten-Layouts gehen auf den ART und auf FAST zurück [LKN13, KCS⁺10]; kohärenz-bewusste Telemetrie beantwortet das False-Sharing-Problem der Mehrprozessor-Caches [TLH94] in der Forschungslinie von Kuehn [Kue23]. Der DensityTracker und die Hypothesen-Metriken H1/H2/H3 sind

eigene Beiträge ohne Fremdvorlage.

Der Prüfling steuert sechs eigene Schlüsselstellen bei, zwei davon auf der Telemetrie-Achse, die in der Dreiteilung zweigeteilt ist — Sweep-Unter-Achse (sub-axis) des Mess-Toolings im Planer und Übersetzungszeit-Größe im Builder (Abschnitt 3.4.1). Auf der *Allokator-Achse* ersetzt ein *4+1+2-Pool-Allokator* — vier Seiten-Pools mit den Rollen Trie-Hülle, Dense-Pages, MultiLevel und Decision-Span, ein Rest-Pool und zwei Wert-Pools für statische und dynamische Werte — die algorithmus-fest verdrahtete Allokation durch eine cache-line-ausgerichtete, fragmentierungsarme Free-List. Auf der *Nebenläufigkeits-Achse* — selbst gegliedert in Pattern (8.1) und Locking-Mode (8.2) — trägt PRT-ART *optimistisches Lock-Coupling (OLC) mit reservierten Wert-Blöcken* bei (`olc_with_reserved_blocks`), koordiniert über einen Concurrency-Manager, der die Disziplinen OLC, Hardware- und Software-Transaktionsspeicher (HTM, STM), Lock-Free, Read-Copy-Update (RCU) und Hazard Pointers [Mic04a] führt; die Speicherfreigabe ist dabei eine Unter-Achse des Allokators (epoch-, RCU-, Hazard-Pointer- und QSBR-Strategien je Permutation wählbar). Auf der *Prefetch-Achse* liefert ein *pfadorientiertes Bundle-Prefetch* die heißen Suchpfade — die häufig durchlaufenen Knotenfolgen — vorab, auf der *Layout-Achse* ein *MultiLevel-Layout*. Besonders cache-kritisch ist die *Telemetrie-Achse*: Per-Node-Zähler verursachen auf Mehr-Kern-Systemen Cache-Line-Ping-Pong und verfälschen genau jene Messungen, die sie erheben. Der Kuehn-Forschungslinie [Kue23] folgend bietet PRT-ART daher cache-kohärenz-bewusste Strategien — *Leaf-Only-Zähler* (Hauptvariante), *Sampling* (jeder N -te Blattzugriff) und *Offline-Recompute* (Bottom-up-Aggregation vor dem Reordering) —, während der klassische Per-Node-Zähler nur als gekennzeichnetes Anti-Pattern erhalten bleibt. Den DensityTracker (`density_tracker`) und die Hypothesen-Metriken H1/H2/H3 ergänzt der Prüfling als eigene Telemetrie-Organen. Im Engine-Quellbaum liegen die Baustein-Implementierungen dieser drei Telemetrie-Strategien (TM1–TM3) und die ISA-Unter-Achsen (IS1–IS3) unter den Verzeichnissen der Organ-Achsen, während Telemetrie-Regime und Ziel-ISA in der Registry der System-Achsen geführt werden und die Permutations-Flag-Struktur eine eigene ISA- und Telemetrie-Bank besitzt (Abschnitt 4.3.2); diese doppelte Verortung ist Bestand des Codes, keine Zuordnungsfrage des Modells.

Diese Schlüsselstellen sind *Bausteine des Prüfling-Repos*; welche davon den Mess-Apparat erreichen, entscheidet die Slot-Belegung, nicht die Beitragsliste (Abschnitt 3.5). Der Prüfling stellt vier Slot-Hüllen: den Seitentyp der Build-Achse (`PrtArtBPlusPageType`), das Prefetch (`PrtArtRedirectPrefetch`, T7), das ChainRef-Wert-Handle (`PrtArtChainRefHandle`, T10) und einen Telemetrie-Wrapper mit dem Per-Node-Zähler als Anti-Pattern-Vergleichsbasis. Seine codegen-fähige Slot-Komposition `PrtArtCompositionDemo` übernimmt sechzehn der achtzehn Organe aus der ART-Referenz-Komposition der Cache-Engine und ersetzt genau zwei — Prefetch und Wert-Handle —; die Engine-seitige Referenz-Komposition `PrtArtComposition` unterscheidet sich von der ART-Komposition in genau einer Achse, dem Pfadkompressions-Redirect (T3, das *R* in PRT-ART), und erbt das optimistische Lock-Coupling von ART. Im Mess-Apparat überschrieben sind damit Pfadkompressions-Redirect, Prefetch und Wert-Handle (zusammen mit der Build-Achse Seitentyp); der 4+1+2-Pool-Allokator (T6) und das MultiLevel-Layout (T5) laufen im Mess-Pfad über den Cache-Engine-Standard, die reservierten OLC-Wert-Blöcke über die von ART geerbte OLC-Disziplin, und die Telemetrie-Strategien treten als Mess-Tooling-Unter-Achse auf, nicht als Kompositions-Slot. Die Anbindung von Pool-Allokator und Layout an den Mess-Pfad ist vorgesehen, aber nicht implementiert (Aktualisierung der Prüfling-Einbindung). Auf eine Galerie verdichtet zeigt Abbildung 4.2 die sechs

eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse verortet. Die verbleibenden zwei der acht Bausteinschichten aus der Beitragsliste (Abschnitt 1.4) — die Inline-/External-/ChainRef-Wert-Handles und die Signaling-Bit-Serialisierung — treten nicht als eigene Galerie-Punkte auf: Erstere stellen den im Apparat überschriebenen Wert-Handle-Slot (T10), Letztere ist als `VarLenSerialization`-Baustein der Serialisierungs-Achse (T9) mit Signalisierungsbits in Anhang D katalogisiert.

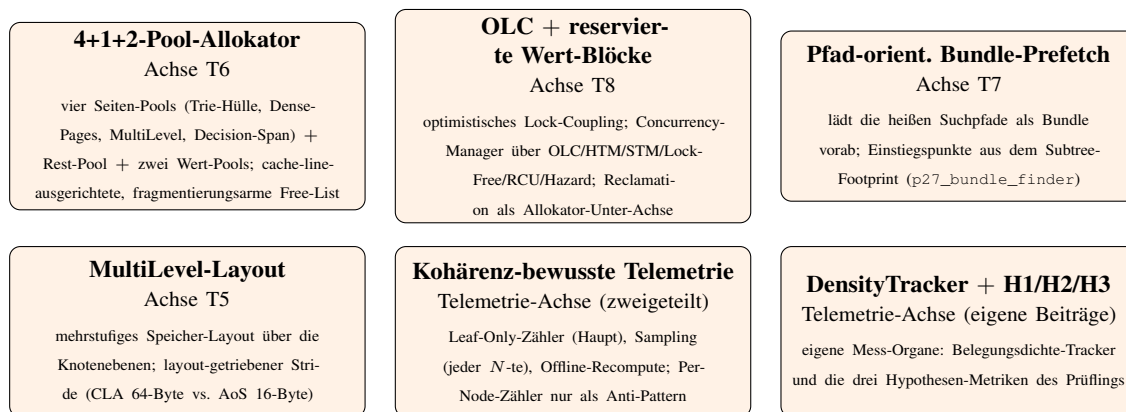


Abbildung 4.2: Galerie der sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse: ein 4+1+2-Pool-Allokator (T6), optimistisches Lock-Coupling mit reservierten Wert-Blöcken (T8), pfad-orientiertes Bundle-Prefetch (T7), ein MultiLevel-Layout (T5) und cache-kohärenz-bewusste Telemetrie samt DensityTracker und den Hypothesen-Metriken H1/H2/H3 — die Telemetrie-Achse ist zweigeteilt, Sweep-Unter-Achse des Mess-Toolings und Übersetzungszeit-Größe des Builders (Abschnitt 3.4.1). Die Kacheln zeigen die *Bausteine des Prüfling-Repos*; im Apparat überschrieben sind der Pfadkompressions-Redirect (T3, Engine-seitige Referenz-Komposition), das Prefetch (T7) und das ChainRef-Wert-Handle (T10, Slot-Komposition des Prüflings) sowie die Build-Achse Seitentyp; die Bausteine zu T5/T6 laufen im Mess-Pfad über den Cache-Engine-Standard, OLC ist von ART geerbt, und die übrigen Organ-Achsen bezieht die Slot-Komposition aus der ART-Referenz der Cache-Engine (sechzehn von achtzehn).

4.3 Achsen-Implementierung und Mess-System-Ebenen

Jede Achse wird als generalisiertes Interface implementiert; daneben liegen die Ebenen des Mess-Systems von der Codegenerierung bis zur Sieben-Phasen-Pipeline. Der Abschnitt schließt mit dem Ebenen-Bild E4–E0 im Code (Abbildung 4.9) und den Vertrags-Flächen an den Träger-Binaries; das Register der verwendeten Entwurfsmuster folgt am Kapitelende (Abbildung 4.12). Jede Achse ist ein Topic-Verzeichnis, das ein zweischichtiges *Concept* [SS12] (ein breites Topic-Concept und ein enges Achsen-Concept) sowie eine durch eine `requires`-Klausel abgesicherte *C RTP-Basisklasse* enthält. Der Live-Weg der Baustein-Auswahl ist monomorph: Je Achse führt eine vollständige Typliste aller bekannten Bausteine (`All*`), aus der ein Übersetzungszeit-Filter über das Enable-Flag jedes Bausteins (`mp_filter<is_enabled>`, Boost.Mp11 [Dim19]) die `Enabled*`-Liste bildet, und eine Composition belegt je Slot genau einen Typ dieser Liste. Der Hot-Path enthält weder `virtual`-Aufrufe noch Laufzeit-Switches — die zero-cost abstraction aus Abschnitt 3.3, im Sinne des C++-Grundsatzes, dass eine ungenutzte Abstraktion nichts kostet [Str12]. Jeder SOTA-Entwurf und jeder Allokator wird über einen dünnen *Adapter* integriert, der die Original-Implementierung hinter dem Engine-ABI bereitstellt: Allokator-Vendors werden über per-Vendor-

Include-Shims (`#if COMDARE_AXIS_06_USE_<X>` mit Forward-Stubs) angebunden, wobei `USE` das Produkt aus dem Enable-Flag der Konfiguration und dem Detektions-Flag `COMDARE_HAVE_<X>` des Bausystems ist — ein fehlender Vendor schaltet den Baustein ab, statt den Bau zu brechen —; SOTA-Bausteine werden über `constexpr`-Enable-Flags samt `mp_filter` (`COMDARE_AXIS_*_USE_-/ENABLE_-`-Flag-Familie) übersetzungszeitlich zu- oder abgeschaltet, jeweils mit sicherem, familienweise gewähltem Fallback: Allokator-Adapter ersetzen den fehlenden Vendor in `if constexpr`-Zweigen durch die portable ausgerichtete Allokation (`portable_aligned_alloc`), der Vendor-Shim liefert für den nie ausgeführten Zweig nur Forward-Stubs, und der Achsen-Wert `StdMalloc` ist ein eigener Baustein, keine Notlösung; Datenstruktur-Adapter fremder Suchverfahren besitzen hinter ihrem Enable-Flag eine eigenständige Re-Implementierung als Körper, während die Bindung an den Original-Code über den Bibliotheks-Bau mit dem Original-Compiler läuft — ein Stub, der zur Laufzeit eine Ausnahme wirft, kommt in den Organ-Achsen nicht vor (die dort geworfenen Typen sind Vertragsverletzungen wie `invalid_argument`, `length_error` oder `logic_error` und die Allokations-Fehler `bad_alloc` und `bad_array_new_length`, kein `runtime_error`) —, sodass der Build eigenständig lauffähig bleibt. Die vom Prüfling nicht belegten Organ-Achsen beziehen ihren Baustein auf demselben Weg aus der Cache-Engine (Abschnitt 4.2). Die Integration des hierarchischen Prefetching [ZGH⁺25] liefert zusätzlich einen Call-Graph-Analyzer zur Übersetzungszeit (`p27_bundle_finder`, ein C++23-Port des Original-Werkzeugs `hp_soft`, der Funktionen mit Subtree-Footprint oberhalb eines Schwellwerts als Bundle-Einstiegspunkte identifiziert).

Eine Design-Entscheidung grenzt diesen Live-Weg von einer verworfenen Alternative ab: Eine variant-basierte Baustein-Auflösung — `resolve_baustein` über `baustein_variants`, ein Concept-geprüftes `if constexpr` über Tag-Spezialisierungen — ist für statische Organ-Achsen ausdrücklich verboten, weil sie Laufzeit-Overhead und Code-Bloat erzeugt; sie existiert nur als vom Bau isolierter Test-Cluster mit einem Test als Konsument seines Auflösungs-Pfads (ein zweiter Test liest aus ihm nur eine Baustein-Kennung) und einer Striktheits-Prüfung, die jede Verdrahtung in den Codegen-Pfad bricht. Der Grund ist die Sparsamkeit des Kompilats: Eine Tier-Binary bildet genau die eine gefahrene Permutation ab; `std::variant`, `virtual` und Laufzeit-Switches sind in Tier- und Hybrid-Binaries verboten, weil sie zur Laufzeit auf mehrere Übersetzungszeit-Ziele abbilden — wer eine andere Permutation braucht, braucht ein anderes Kompilat. Die eine zugelassene variant-Stelle des Hauses ist das Dock-Array der Hybrid-Stufe (`HybridDockVariant`, Abschnitt 4.5.1), dessen Zone ein Test prüft, indem er die Quelldateien liest und genau ein Vorkommen zulässt; daneben duldet der Code `std::variant` in zwei achsenfremden Bausteinen (der Fehler-Domäne `axis_error` und dem Druck-Zustand `pressure_state`), die keinem Organ angehören, und in den drei Expected/Result-Nähten der Hardware-Sonden (NUMA-Seiten, NUMA-Kern-Bindung, Betriebssystem), die die Fehler-Summen von `axis_error` konsumieren; die CEB selbst enthält keine variant-Stelle: ihre Substanz unter `builder/` liegt im Zählbereich derselben Prüfung, und der eine Registry-Header ihres Träger-Gerüsts unter `libs/traeger/ceb` außerhalb dieses Bereichs ist ebenfalls variant-frei. Das Verbot ist damit strenger als die bloße Vermeidung virtueller Aufrufe [Ale01]: Es verbietet jede Form der Laufzeit-Verzweigung zwischen Bausteinen.

Innerhalb jeder Achsen-Kategorie gibt es Haupt-Achsen, Unter-Achsen (sub-axes) und Meta-Meta-Achsen. Die Kategorie ist im Code die Trichotomie `AxisCategory` mit der festen Ordnung `mess < system < organ` — eine Ordnung, die ein `static_assert` gegen stilles Umsortieren hält —, und die Familie einer Achse ist der Diskriminator `AxisKind` mit sechs Werten: Mess-System-Achse und Mess-Meta-

Meta, System-Konfigurations-Achse und System-Meta-Meta, Organ-Achse und Organ-Meta-Meta. Eine Unter-Achse ist dabei selbst eine Voll-Achse (die Rekursion des Schichtenmodells bleibt offen), und eine Unter-Achse kann zugleich Permutations-Achse sein — die beiden Rollen schließen sich nicht aus. Tabelle 4.2 zählt den Bestand je Kategorie: Der Organ-Realm umfasst achtzehn Haupt-Achsen, zwanzig Unter-Achsen-Header und die Organ-Meta-Meta `disk_io`; der System-Realm drei Haupt-Achsen, die Komplex-Achse `build_target_complex` als Command-Wrapper ihrer Rekombination mit der Unter-Achsen-Gruppe `build_toolchain` (Compiler, Optimierungsstufe, Atomik), das Scheduling mit fünf Dimensionen und die System-Meta-Metas unter dem Hub der Erweiterungs-Hardware; der Mess-Realm die Tooling-Haupt-Achse (wallclock/macro/micro), vier registrierte Unter-Achsen (sechzehn Kategorien, Arbeitsmodus, Mess-Framework, Rückschreib-Methode) und die Mess-Meta-Meta `load_framework`. Die Benennung Meta-Meta folgt der vierstufigen Meta-Ebenen-Architektur der Object Management Group (MOF) [Obj16], in der jede Ebene die Beschreibungssprache der darunterliegenden ist; die Lesart M0–M3 für diesen Apparat entwickelt Abschnitt 4.5.

Kategorie	Haupt-Achsen	Unter-Achsen	Meta-Meta-Achsen	Single-Source
Mess	Tooling (3: wall-clock, macro, micro)	16 Kategorien; Arbeitsmodus (4); Mess-Framework (1); Rückschreib-Methode (4)	<code>load_framework</code> (YCSB)	vier Registry-Header, Mess-Registry-XML
System	3: <code>target_isa</code> , <code>operating_system</code> , <code>external_utils</code>	Komplex-Achse <code>build_target_complex</code> mit Gruppe <code>build_toolchain</code> (3); <code>target_isa_complex</code> (3 Glieder); Scheduling (5 Dimensionen)	System-Meta-Metas unter dem Hub <code>external_utils</code>	<code>kSystemAxisOrder</code> , <code>System-Registry-XML</code>
Organ	18 (T0–T17)	20 Unter-Achsen-Header (z. B. SA1–SA4, AA1–AA7)	<code>disk_io</code> (<code>binary_id</code> = never)	<code>kOrganAxisCount</code> , <code>kCompositionAxisNames</code> , <code>Organ-Registry-XML</code>

Tabelle 4.2: Achsen-Typologie je Kategorie (`AxisCategory` mit der Ordnung `mess < system < organ`; Familien-Diskriminator `AxisKind` mit sechs Werten). Die Zahlen sind Strukturzahlen des Quellbaums (Registry-Header und generierte Registry-XML), keine Messwerte.

Die achtzehn Organ-Achsen (T0–T17, Abschnitt 3.4.1) sind dabei nicht abstrakt, sondern je ein konkretes *Strategie-Concept* mit mehreren austauschbaren Bausteinen, aus denen eine konkrete *Composition* je Slot genau einen wählt; ihre Zahl ist als `kOrganAxisCount` = 18 die Single-Source der Organ-Zeile, und ihre Namen und Slot-Reihenfolge führt das Array `kCompositionAxisNames`. Abbildung 4.3 fasst alle achtzehn als Galerie zusammen — je Achse der Kurzname, die Unter-Achsen-Familie und eine repräsentative Auswahl der Bausteine; die vollständige Bausteine-Matrix mit allen Herkunfts-Quellen führt Anhang D, die SOTA-Beiträge je Achse Tabelle 3.2. Neben den achtzehn kanonischen Haupt-Achsen steht die Organ-Meta-Meta `disk_io`, die die `binary_id` nie berührt (Abschnitt 3.4.1); eine neunzehnte, optionale Organ-Achse ist vorgesehen, aber nicht implementiert; erst ihre Implementierung führt sie als Slot T18. Dass keine *Composition* ein Organ auslässt, erzwingt das `IsComposition`-Concept: Es verlangt je Algorithmus die Belegung genau dieser achtzehn Organe — der fünfzehn Such-Achsen, der beiden Queuing-Achsen und des Persistenz-Ziels —, sodass keine *Composition* übersetzt, die eine Achse offenlässt; der vollständige Organ-Satz ist damit keine Konvention, sondern eine übersetzungszeitlich geprüfte Bedingung.

T0 Suchalgorithmus SA1-SA4 Array256, OriginalART/HOT, KAry, Interpol., Hash, SwissTable ... (22: 18 Basis + 4 per-K) <i>Std. Array256</i>	T1 Cache-Travers. CT1-CT2 Linear-Fanout, Hash- Lookup, Binär-Fanout <i>Std. Linear</i>	T2 Slot-Mapping MP1-MP2 Direkt-Platzierung, Pool-relativ <i>Std. Direkt</i>	T3 Pfadkompression PC1-PC3 Keine, Byte-weise (ART), Patricia <i>Std. Keine</i>
T4 Knotentyp NT1-NT3 Node4 / 16 / 48 / 256 <i>Std. Node256</i>	T5 Speicher-Layout HM1-HM3 AoS, Cache-Line-aligned, SoA, Bitmap (LOUDS), AoSoA <i>Std. Cache-Line</i>	T6 Allokator AA1-AA7 Hoard, Slab, Mimalloc, Jemalloc, TCMalloc, Buddy ... (26) <i>Std. Std-Malloc</i>	T7 Prefetch PF1-PF3 Keine, Distanz-Schätzer (ART), Hardware, Pfad-orient. <i>Std. Keine</i>
T8 Nebenläufigkeit CC1-CC2 Keine, Blocking, R/W, OLC, Lock-free, RCU, Hazard <i>Std. Keine</i>	T9 Serialisierung SR1-SR3 Roh-Binär, VarLen (LEB128), Succinct, LZ4/Delta <i>Std. Roh-Binär</i>	T10 Wert-Handle VH1-VH3 Inline, Extern-Pool, RCU- Shared, MVCC, Chain-Ref <i>Std. Inline</i>	T11 Index-Org. io1-io3 (index_organization) Clustered, Heap, IOT (em- bedded), Non-Clustered <i>Std. Heap</i>
T12 I/O-Dispatch IO1-IO3 (io_dispatch) In-Memory, Buffered, Di- rect (O_DIRECT), Mmap <i>Std. In-Memory</i>	T13 Migration MG1-MG3 Keine, Hot/Cold, Tier- basiert, Adaptiv (ML) <i>Std. Keine</i>	T14 Filter FT1-FT3 Bloom, Cuckoo, Range-SuRF (LLOUDS), Xor; None (dekla- riert, per Option abgeschaltet) <i>Std. Bloom</i>	T15 Op.-Puffer (Q1) QS1-QS6 Kein, Append, FIFO/LIFO, Ring, Priority ... (15) <i>Std. Kein-Puffer</i>
T16 Flush-Policy (Q2) FS1-FS4 Eager, Lazy, Watermark, Timed, Adaptiv-LSM <i>Std. Lazy</i>	T17 Persistenz-Ziel 18. Slot (ORG-18) In-Memory; Disk-Writeback (deklariert, per Option abgeschaltet) <i>Std. In-Memory</i>	Organ-Meta- Meta disk_io binary_id = never, Stufe ct; Laufzeit-Unter-Achse staging; stempelt im Klammer-Anhang der Organ-Zeile (Abschnitt 3.4.1)	Lesart Jede Achse = ein Concept mit austauschbaren Bausteinen (Strategy); eine Composition wählt je Slot genau einen.

Abbildung 4.3: Galerie der achtzehn Organ-Achsen (T0–T17, kanonische Slot-Reihenfolge): je Achse das Implementierungs-Konzept — Kurzname, Unter-Achsen-Familie und eine repräsentative Auswahl der austauschbaren Bausteine. Jede Achse ist ein `Concept`-Interface mit mehreren Strategie-Bausteinen (*Strategy*-Muster); eine konkrete *Composition* belegt je Slot genau einen. Der *Standard* („Std.“) ist eine abgeleitete Vergleichs-Baseline, kein Code-Flag. Die Zahlen in Klammern (T0, T6, T15) sind die *Kataloge* der vollständigen Typlisten (`All*`); das *Enabled-Inventar* einer Build-Konfiguration ist die vom Enable-Filter gebildete Teilmenge, die die generierte Registry-XML je Achse ausweist (etwa vier Suchalgorithmen und drei Allokatoren im Haus-Default). Die T-Nummern sind Slots der *Composition*, keine Verzeichnis-Nummern der Implementierung; die Kürzel io1–io3 und IO1–IO3 gehören zu verschiedenen Achsen (Index-Organisation und I/O-Dispatch). Neben den achtzehn Slots steht die Organ-Meta-Meta `disk_io`; das Persistenz-Ziel ist im kanonischen Referenz-Katalog auf seinen In-Memory-Vertreter festgelegt (Abschnitt 3.4.1). Die vollständige Matrix mit Herkunfts-Quellen führt Anhang D.

4.3.1 Die eine Architektur im Code

Die *eine* Architektur aus Abschnitt 3.3 ist im Code eine einzige Hierarchie. Ihre Wurzel ist `IExecutionEngine` mit drei Engine-Arten — Anatomy, Virus, Hybrid — und einem Vertrag aus sieben Methoden: `warm_up`, `run`, `reset` und `shutdown` als Mess-Lebenszyklus sowie `engine_name`, `engine_kind` und `lifecycle_state` als Auskünfte. Unter ihr liegen auf Ebene 1 die vier Konzepte Map, Container, Graph und HeuristikAdapter (Abschnitt 3.3.2) — Graph als deklarierter Stub ohne Gattung — und auf Ebene 2 die sechs Gattungen SearchAlgorithm, Set, Sequence, Adapter, View und FunctionInterfaceReroute; die Gattung ist im Code die Ableitung der Gattung (`gattung_of`), kein zweites Etikett. Das Lebewesen SearchAlgorithm und seine ABI-Laufzeit-Sicht — die „SearchEngine“ — sind zwei Sichten desselben Gegenstands über die Modulgrenze, kein paralleles Konstrukt: Der Körper ist `SearchAlgorithmAnatomy<C>`, die Komposition und Verdrahtung der achtzehn Organe, und die Sicht ist `SearchAlgorithmAbiAdapter<A>`, die ihn über die Modulgrenze exportiert. Die Schichtung `CacheEngine` → `ExecutionEngine` → `SearchEngine` (vgl. Abschnitt 4.1) trennt damit Rollen, nicht Klassenbäume: die `CacheEngine` als Achsen-Bausteine-Bibliothek, die `ExecutionEngine` als lebende Wurzel der Ausführungs- und Mess-Werkzeuge (OS-Primitiven; im Experiment-Modus der Mess-Aggregator über genau eine Beobachter-Schnittstelle) und die `SearchEngine` als die über genau diese Schnittstelle vermessene Such-API-Sicht — das gleichnamige Bibliotheks-Skelett `libs/search_engine` enthält keinen Code (Abschnitt 4.1). Achsenlose *Viren* stehen als Geschwister unter derselben Wurzel (im Bestand eine Graph-Breitensuche), besitzen aber keine Anatomie (Abbildung 4.4). In dieser Hierarchie ist der Compiler-Compiler-Charakter des Apparats verankert: Jede Composition ist ein Typ, aus dem der Builder je Permutation ein eigenes Kompilat erzeugt.

Konkret ist die Modulgrenze eine C-stabile Schnittstelle — die ABI (Application Binary Interface) des Apparats —, die je Permutation genau ein Lebewesen exportiert (Abbildung 4.5): Jede Tier-Binary enthält sieben Pflicht-Symbole mit C-Bindung — die vier Ur-Symbole ABI-Version, Magic-Zahl, Factory `comdare_create_anatomy` und Destroy, dazu die zwei über alle Gattungen buchstabengleichen Auskünfte `comdare_anatomy_gattung` und `comdare_anatomy_genus` sowie die Versions-Zeilen des Stempels. Die Buchstabengleichheit ist Bedingung, nicht Bequemlichkeit: Der konzeptunabhängige Loader lädt jede Binary über dieselbe Ladestrecke (POSIX `dlopen` [The18]) und kann das Konzept ermitteln, *bevor* er eine Instanz baut — konzeptunabhängige Namen mit konzept-spezifischen Werten; die Gattung ist dabei aus der Gattung abgeleitet. Die ABI führt die Major-Version 9, deren Naht das Gattungs-Interface ist. Die variadische 1/2/N-Hybrid-API gehört zum Prüfling: Seine `PrtArtSearchEngine<Ts...>` spezialisiert sich nach Parameterzahl — ein Parameter fordert alle Funktionsinterfaces eines `std::vector`, zwei die eines `std::map`, mehr als zwei die eines `std::map` auf ein Tupel —, und die Slot-Aritäten der Gattungen der Cache-Engine (achtzehn für das Gattung SearchAlgorithm, je eigene für Set, Sequence, Adapter und View aus der Gattungs-Bau-Zulassung) sind davon getrennt zu lesen. Komplexe Schlüssel reduziert eine Fingerprint-Komponente der Cache-Engine (`fixed_length_fingerprint`, FNV-1a-Basis) auf 16-Byte-Binärstrings fester Länge, die der Prüfling als Schlüsseltyp seines Speichermodells verwendet (`BinaryKey`); dieser Schlüssel-Fingerprint ist von den drei Identitäts-Fingerprints des Apparats zu unterscheiden — dem 64-Bit-Feld `permutation_fingerprint` im ABI-POD des Moduls, dem SHA-512-Tier-Fingerprint (Format 6, elf Glieder) und dem SHA-256-Planer-Fingerprint (Abschnitt 3.4.4). Wie diese und die übrigen

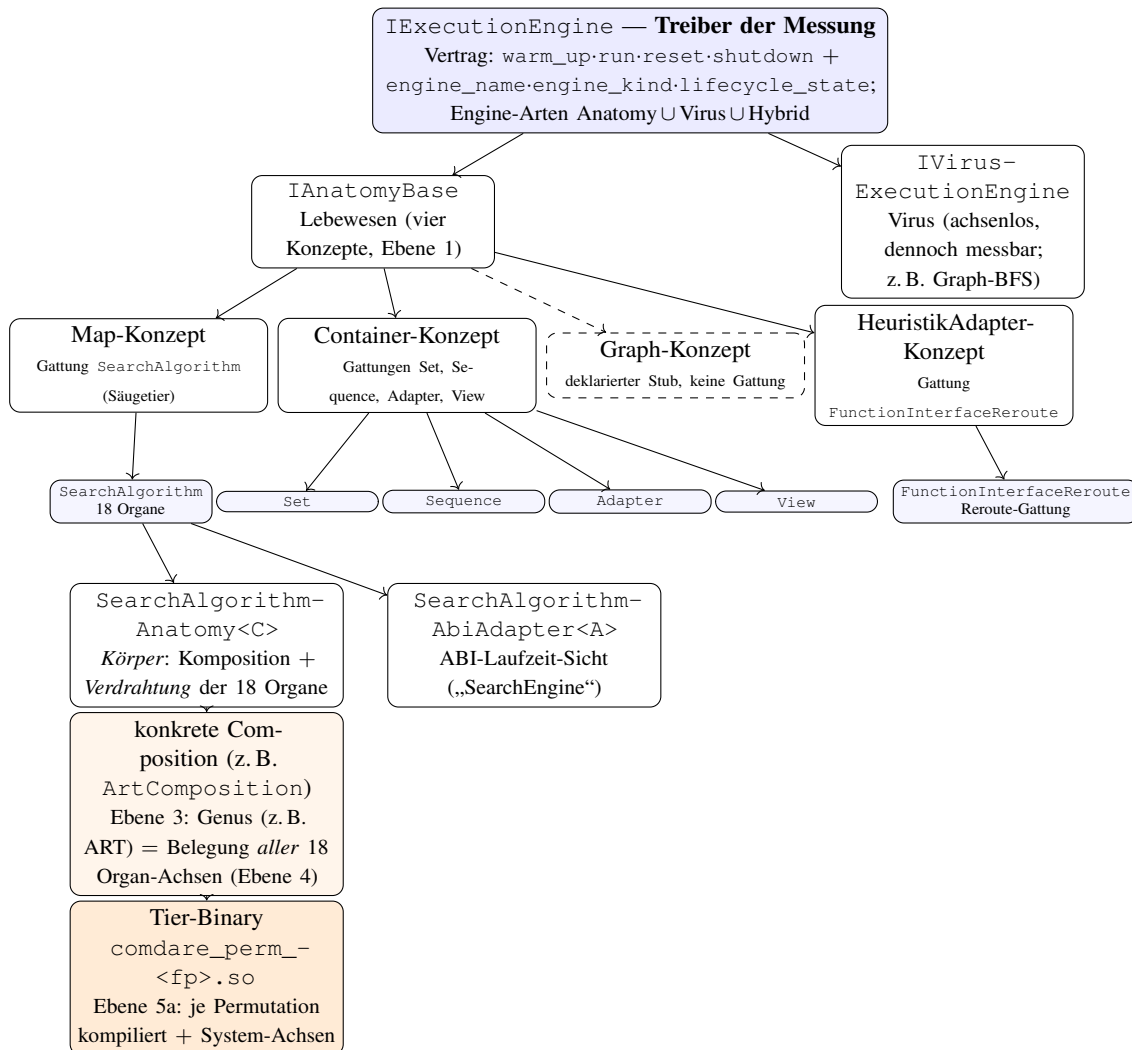


Abbildung 4.4: Die *eine* Architektur als einzige Hierarchie ohne Parallel-Bäume. An der Wurzel steht IExecutionEngine als *Treiber der Messung* — der universelle messbare Vertrag aus Mess-Lebenszyklus (warm_up/run/reset/shutdown) und Auskünften (engine_name/engine_kind/lifecycle_state) über alle drei Engine-Arten Anatomy, Virus und Hybrid; unter ihm verzweigen die anatomie-tragenden *Konzepte* (Ebene 1: Map, Container, Graph, HeuristikAdapter) mit ihren *Gattungen* (Ebene 2) und die achsenlosen *Viren*. SearchAlgorithm ist das fokale Lebewesen (Elefant); seine Anatomie ist sein *Körper* — die feste Komposition und Verdrahtung der 18 Organe —, und „SearchEngine“ nur dessen ABI-Laufzeit-Sicht. Der Elefanten-Ast (Gattung SearchAlgorithm) ist bis zum Individuum ausgezogen: ein Genus (Ebene 3) mit seiner konkreten Achsen-Composition (Ebene 4) wird je Permutation als eigenständige Tier-Binary kompiliert (Ebene 5a, das Individuum). Container besitzt einen eigenen Organsatz mit den Gattungen Set, Sequence, Adapter und View, deren Aritäten die Gattungs-Bau-Zulassung führt; Graph ist deklariert und besitzt keine Gattung; das Konzept HeuristikAdapter führt die Reroute-Gattung der Hybrid-Stufe.

Code-Interfaces im Detail zusammenhängen, fasst die UML-Landkarte in Abbildung 4.6 zusammen.

```
// C-stabile Modulgrenze: sieben Pflicht-Symbole je Tier-Binary
extern "C" {
    std::uint64_t comdare_anatomy_abi_version();    // ABI-Major 9
    std::uint64_t comdare_anatomy_abi_magic();    // Kompatibilitaets-Probe
    IAnatomyBase* comdare_create_anatomy();        // ein Lebewesen je Permutation
    void          comdare_destroy_anatomy(IAnatomyBase*);
    std::uint8_t  comdare_anatomy_gattung();       // buchstabengleich je Gattung
    std::uint8_t  comdare_anatomy_genus();         // Hybrid: meldet den Ziel-Genus
    AnatomyVersionLines const* comdare_anatomy_version_lines();
}
// Die Hybrid-.so exportiert dieselbe Menge: genau eine Ladestrecke.

// Pruefling-Sicht: variadische 1/2/N-API (Schluessel K, Werte V)
//   1 Parameter -> std::vector<V>
//   2 Parameter -> std::map<K, V>
//   N > 2       -> std::map<K, std::tuple<V1, ..., VN>>
template<class K, class First, class... Rest> class PrtArtSearchEngine;
```

Abbildung 4.5: Die ABI-stabile C-Modulgrenze (sieben Pflicht-Symbole; die Hybrid-.so exportiert dieselbe Menge und meldet über `genus` ihre Ziel-Gattung) und die variadische 1/2/N-API als Sicht des Prüflings. Komplexe Schlüssel reduziert eine Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge; die Identität einer Binary führen davon getrennt das `permutation_fingerprint`-Feld des ABI-POD, der SHA-512-Tier-Fingerprint und der SHA-256-Planer-Fingerprint.

4.3.2 Vom Profil zur Tier-Binary: Planer, Builder und Mess-Pipeline

Die Kette von der Anwender-XML zur Tier-Binary (Abschnitt 3.4.6) hat im Code eine benannte Gestalt aus zwei Binaries. Stufe 1, die Planer-Rolle, ist `comdare-experiment-planner` mit den Subkommandos `validate`, `plan`, `dump`, `ci`, `cmake`, `cache-key`, `fingerprint`, `status`, `check-size`, `simulate`, `version` und `help`. `validate` prüft rein lesend das per XML-Parser aufgelöste Profil gegen die einkompilierten Registry-Kataloge (eine unregistrierte Referenz ist ein harter Validierungsfehler, Abschnitt 3.4.6), baut dabei keine Binary und misst nicht. `plan` fährt den deterministischen Walk des `ExperimentPlanDirector` — nach den Entwurfsmustern *Director* und *Builder* [GHJV94] getrennt: Der Director besitzt den *einen* Enumerations-Walk über Mess-Kombinationen, System-Permutationen und Organ-Schritte, und je Ausgabe-Syntax (Text-Plan, CI-Pipeline-Beschreibung, CMake-Fragment) emittiert ein konkreter Builder über Hook-Methoden —; zwei Läufe sind byte-gleich, und der Plan-Kopf annotiert das Registry-Trio der drei Angebots-Kataloge Organ/System/Mess. `check-size` und `simulate` rechnen die Mess-Menge vor dem Lauf aus den Eingängen des Walks (Abschnitt 4.4). Stufe 2, die CEB-Rolle, ist der Mess-Treiber `comdare-messung-driver` im Projekt-Repository mit den Subkommandos `tier`, `ci`, `cmake`, `run`, `version` und `help`: Er emittiert die Sicht der zweiten Ketten-Stufe — System-Permutationen und Tier-Bau-Aufträge seines freigegebenen Bau-Raums — und fährt den Mess-Lauf; ein an ihn gerichtetes Planer-Thema beantwortet sein Dispatcher mit dem Verweis auf die Planer-Binary, statt es still auszuführen. Der Planer steuert die CEB-Aufträge, die CEB steuert die Tier-Aufträge: Die Rollentrennung ist damit nicht nur Oberfläche, sondern zwei Module. Die emittierten Bau- und Prüf-Testate halten die Layer-Trennung der Kette sichtbar — ihre Legenden besitzen drei Formen: `ceb:build:[a,b,c]` auf Stufe 1; auf Stufe 2 je Host ein Bau-

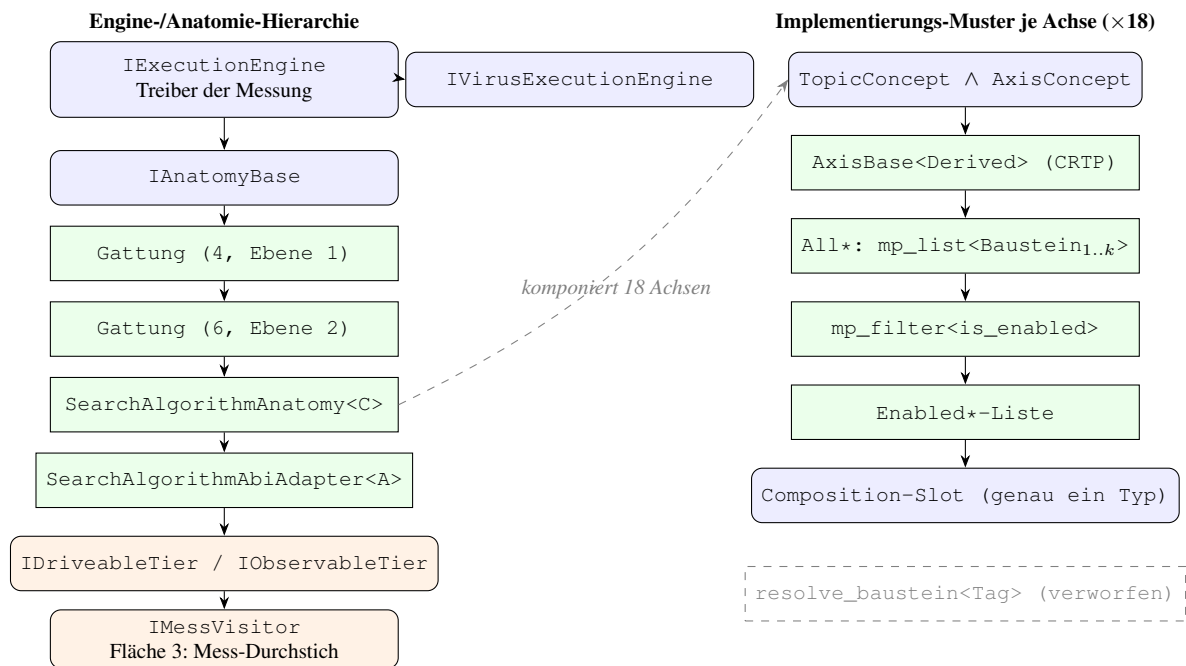


Abbildung 4.6: UML-Landkarte der Code-Interfaces: links die Engine-/Anatomie-Hierarchie — die messbare Wurzel `IExecutionEngine` als *Treiber der Messung* (Anatomy ∪ Virus ∪ Hybrid) über `IAnatomyBase`, den vier Konzepten und sechs Gattungen und der achsenlosen `IVirusExecutionEngine`, darunter der *Körper* `SearchAlgorithmAnatomy<C>` (Komposition und Verdrahtung der achtzehn Achsen) und seine ABI-Laufzeit-Sicht `SearchAlgorithmAbiAdapter<A>` („SearchEngine“), die über die Treib-/Beobachter-Verträge `IDriveableTier`/`IObservableTier` gemessen wird und deren Messung als Fläche 3 durch den Visitor `IMessVisitor` quert; rechts der je Achse wiederholte Live-Weg — ein zweischichtiges Concept, eine requires-gesicherte CRTP-Basis, die vollständige `mp_list`-Baustein-Familie, der Übersetzungszeit-Filter über das Enable-Flag und der Composition-Slot, der genau einen Typ enthält. Die variant-basierte Auflösung `resolve_baustein` ist eine verworfene, vom Bau isolierte Alternative (gestrichelt).

und Prüf-Batch `tier:build-batch:<host>`, dessen Testate je Schritt die getrennte System- und Organ-Klammer `zelle=[d,e,f][g,h,i]` enthalten und der die Organ-Bau-Volumina intern in Scheiben zu je 4096 Binaries bündelt (die Meilenstein-Batches in Abschnitt 5.1.7); und je Host ein Mess-Batch `measure:[a,b,c]:batch:<host>`, dessen Mess-Testate als einzige alle drei Klammern `zelle=[a,b,c][d,e,f][g,h,i]` enthalten (Abbildung 4.1); die Klammern nennen nur Haupt-Achsen, Unter-Achsen sind reine Laufzeit-Permutation und stehen in keiner Bau-Legende.

Die Anlage hat zwei Steuerpfade: den Planer per Shell direkt und eine API-Kommandozeile mit einem headless Dienst, der den Planer als Zelle auf Kubernetes oder auf bare metal bedient, sodass eine vollständige Messreihe ohne interaktive Sitzung angestoßen und beobachtet werden kann. Dazu tritt ein kommandozeilengeführter Erstellungsmodus für die Anwender-XML mit Hardware-Filter: Nicht verfügbare Optionen der Zielmaschine sind darin nicht wählbar, und der Modus ist optional — eine von Hand geschriebene XML bleibt gleichwertig. Bestand im Code sind die Planer-Hardware-Sonde, die zur Laufzeit den Host befragt, und die drei Maschinen-Signaturen der System-Registry; Umsetzungsstand: Der Shell-Pfad ist implementiert; Dienst und Erstellungsmodus sind vorgesehen, aber nicht implementiert.

Zwei Eigenschaften sichern den *einen* Hauptkanal ab. Erstens weist der Text-Plan (Format v1.1) je Bau-Schritt den tatsächlich erzeugten Datei-Stamm aus (`built_stem`) — erzeugt von *derselben* Orchestrator-Funktion, die auch der Bau-Orchestrator beim Bau aufruft: Der Plan behauptet keinen Namen, den der Bau nicht produziert, und ein nicht bestimmbarer Stamm wird als „-“ geschrieben statt synthetisiert. Zweitens gibt es *eine* XML-Quelle und *einen* Kanal, der sie baut: Die Variantenbildung liegt in der C++23-Metaprogrammierung, CMake ist auf die Rolle des Bauens beschränkt, und ein zweiter Emissions-Kanal für Permutations-Quelltexte existiert im Quellbaum nicht.

Innerhalb der CEB überführt der Builder jedes Profil in ein vorkompiliertes C++23-Modul. Jede Permutation ist durch eine bit-gepackte *Permutations-Flag*-Struktur identifiziert, die zehn Bänke zu je 64 Bit umfasst (Seitentyp, Knoten, Traversierung, Wert-Handle, Layout, Allokator, Prefetch, Nebenläufigkeit, ISA und Telemetrie) und hexadezimal serialisiert wird; die Allokator-Unter-Achsen AA1–AA7 führen eine eigene Flag-Bank. Ein Constraint-Filter der Flag-Struktur (`is_valid_combination`, vier Regeln, etwa: Leaf-Only-Zähler nur zusammen mit rückwirkender Aggregation) schließt inkohärente Kombinationen aus. Die achtzehn Organ-Achsen identifiziert der Mess-Pfad über den string-serialisierten Achtzehn-Segment-Pfad (die `binary_id`, Abschnitt 3.4.4) samt den drei Fingerprints; die Permutations-Zählung über die Achsen folgt einer Mixed-Radix-Kodierung, in der jede Achse eine Stelle mit ihrer eigenen Basis ist — die Gestalt eines Produktraums, wie ihn die Variabilitätsmodelle der Software-Produktlinien beschreiben [PBL05]. Über diese Bausteine läuft die *Sieben-Phasen-Mess-Pipeline* (Abbildung 4.7) der `ExperimentDriver`-Bibliothek je Reihe mit den Phasen des Codes: (1) ENUMERATE (XML-Configs parsen, Permutationsliste aufbauen), (2) GENERATE (ein `comdare_perm_<fp>.cpp` je Permutation plus Aggregator-CMakeLists, dazu `generate_module_from_profile` für die SOTA-Profile), (3) COMPILE (CMake-Konfiguration und Bau aller Permutations-Module), (4) LINK+LOAD (`dlopen` aller Module), (5) EXECUTE (`create_instance` + profilbewusstes `run_workload`), (6) MEASURE (binäre Mess-Records aggregieren) und (7) PERSIST (CSV und JSON, optional binär); zwei opt-in-Phasen kompilieren fehlende Module im laufenden Experiment nach (`phase3_hot_compile_missing`) und fahren je Modul einen Funktionstest — Erzeugen und Zerstören einer Instanz, ein `run_workload` mit leerem Workload-Deskriptor und die Prüfung der ABI-Version (`phase4b_functional_tests`). Die Messreihen-Modi `Defined`, `Full` und `FullSampled` (deterministische 1:n-Teilmenge des Vollraums, Abschnitt 3.4.6) wählen, welche Permutationen eine Reihe umfasst. Die nachgelagerte Auswertungs-Werkzeugkette (Abbildung 4.8) ist eine Folge von neun numerierten Programmen von `sample_data_generator` bis `tex_formatter`; ihre Ausgabe ist die `xlsx`-Mappe, der Standard; die CSV-Form ist kein einzelnes Kind-Dokument, sondern ein geordneter Baum aus CSV-Dateien — ein Gesamtordner je Mappe, darunter ein Ordner je Blatt, darin die CSV-Dateien der einzelnen Unter-Achsen-Fahrten —, der immer aus der Mappe erzeugt wird (`xlsx` nach `csv`, als Strategie derselben Factory) und per XML statt der Mappe oder zusätzlich zu ihr am selben Ziel liegt (vier registrierte Rückschreib-Methoden, Abschnitt 4.5). Umsetzungsstand: Die Naht schreibt die CSV-Projektion als eine flache Datei (`projektion_oeffnen` in `ergebnis_mappe_naht`); die Baumform ist vorgesehen, aber nicht implementiert. Die Mess-Kopplungen sind über das `COMDARE_MEASUREMENT_ON`-Gating vom Produktiv-Pfad getrennt (Abschnitt 4.1), und Continuous Integration [Fow06] läuft über alle vier Repositorien (GitLab CI; die Repositorien sind zusätzlich nach GitHub gespiegelt).

Die Ebenen E4–E0 aus Abschnitt 3.3.4 haben im Code je einen Ort, und die Grenzen zwischen ihnen besitzen je einen Contract-Test (Abschnitt 4.4); Abbildung 4.9 zeigt diese Code-Gestalt, ohne die Ebenen

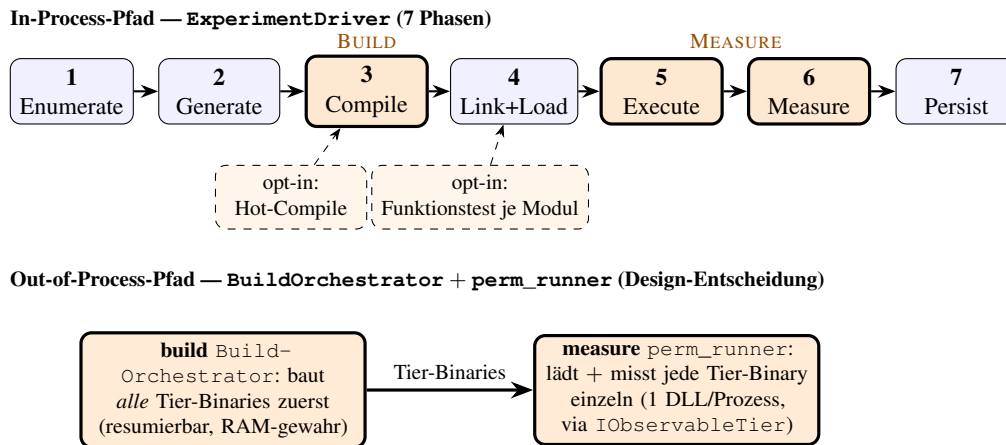


Abbildung 4.7: Die Mess-Pipeline in zwei Ausprägungen. Der **In-Process-Pfad** (oben) ist die Sieben-Phasen-Pipeline des `ExperimentDriver` je Messreihe (Enumerate → Generate → Compile → Link+Load → Execute → Measure → Persist; zwei opt-in-Phasen kompilieren fehlende Module im laufenden Experiment nach und fahren je Modul einen Funktionstest mit ABI-Versionsprüfung). Hervorgehoben sind die substantziellen Phasen: COMPILE (der *Build* der Tier-Binaries) und EXECUTE/MEASURE (die *Messung*); Generate/Link+Load/Persist sind Verdrahtung. Der **Out-of-Process-Pfad** (unten) ist eine Design-Entscheidung zur Prozess-Isolation: Der `BuildOrchestrator` baut *zuerst alle* Tier-Binaries (resumierbar, RAM-gewahr), danach lädt und misst `perm_runner` jede Binary einzeln über die Beobachter-Schnittstelle `IObservableTier` — eine DLL je Prozess, weil die DLL kein `main` enthält und der Runner das Executable ist, das sie lädt; kein Zustand wird zwischen Permutationen geteilt, und eine Cluster-Task fährt genau einen Runner mit genau einer DLL.

neu zu erklären. E4, die Definitions- und Auswertungs-Schicht, ist das XML-Profil mit dem Planer (`profile_facade/planner`) und den Auswertungs-Programmen 04 und 08 der Werkzeugkette; E3, der Permutations-B⁺-Baum, ist der Experiment-Baum (`experiment_tree`) mit der `binary_id` als Pfad; E2, die Tier-Binaries, sind der ABI-Adapter mit der Modul-ABI-Deklaration; E1, die Laufzeit-Konfiguration, ist das Interface `IResourceControllableTier` zusammen mit den dynamischen Dimensionen der Mess-Registry-XML; E0, die Infrastruktur, sind CI, Runner und Lager; der Querschnitt M ist die System-Achsen-Wurzel `SystemAxis` (Abschnitt 4.5).

Die Verträge zwischen den Träger-Stufen sind an den Binaries in Flächen zergliedert. Das Drei-Flächen-Modell (Abschnitt 3.3.2) ist im Code namentlich vorhanden: Fläche 1 ist das Gattungs-Interface der Tier-Binary (Laufzeit, Abstract Factory), Fläche 2 der Stempel (Übersetzungszeit, ABI-stabil) und Fläche 3 der Mess-Durchstich `IMessVisitor` — ein Visitor am `dlopen`-Übergang mit doppeltem Dispatch, der existiert, *damit* Konzept- und Gattungs-Interfaces unverändert bleiben: Die Messung quert als eigene Naht, statt Mess-Felder in die fachlichen Interfaces zu drängen. Der Stempel-Vertrag ist als Matrix aus sieben Interfaces und vier Trägern ausgeführt — 28 Zellen mit dreiwertiger Zulassung (Pflicht, Erlaubt, Verboten), von denen der Bestand nur Pflicht und Verboten belegt (Tabelle 4.3): Die statische Selbst-Version führt nur der Planer; Mess- und System-Zeile führen CEB, Tier und Hybrid; die Organ-Zeile führen nur Tier und Hybrid — die Absenz beim CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“ —; Fingerprint und Gesamt-Stempel führen alle vier, und die angeschlossenen Stempel-Zeilen führt nur der Hybrid als Laufzeit-Haken. Geprüft wird der Vertrag nicht in der Basis, sondern unter jeder

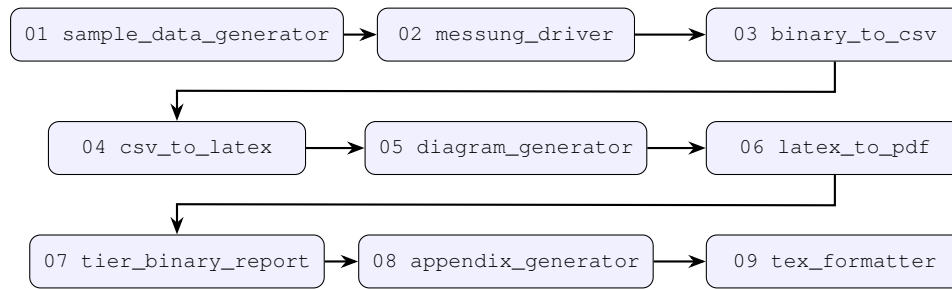


Abbildung 4.8: Die nachgelagerte Auswertungs-Werkzeugkette als Neun-Stufen-Kette: `sample_data_generator` erzeugt deterministische Testdaten ohne reale Hardware, `messung_driver` steuert die Messreihen (die CEB-Rolle), `binary_to_csv` deserialisiert die binären Mess-Records, `csv_to_latex` rendert die booktabs-Tabellen mit einem Baustein-Steckbrief je Permutations-ID und erzeugt bei leerer Eingabe keine Datei, `diagram_generator` die A4-bewussten TikZ-Diagramme (drei Grund-Plot-Typen Balken, Streudiagramm und Heatmap sowie die darauf aufbauenden Familien des Mess-Anhangs: ECDF-Treppe, Range-Balken, gestapelte Per-Achsen-Attribution, Sweep-Kurve, 3D-Rohdaten-Ansicht, Verhältnis-Matrix, normalisierte Balken, Pareto-Streuung und die Observer-Detail-Tabelle), `latex_to_pdf` setzt das Manuskript, `tier_binary_report` rastert je Tier-Binary den Interface-Vertrag gegen die geplante Menge des Text-Plans, `appendix_generator` erzeugt als In-Process-Fassade je Sprache die Kern- und Darstellungs-Dateien des Mess-Anhangs aus *einer* WIDE-Matrix, und `tex_formatter` normalisiert die erzeugten Dateien. Die Ausgabe der Kette ist die `xlsx`-Mappe; die CSV-Form ist kein einzelnes Kind-Dokument, sondern der aus der Mappe erzeugte, geordnete Baum aus CSV-Dateien (ein Ordner je Blatt), im Code als eine flache Datei geschrieben (die Baumform ist vorgesehen, aber nicht implementiert; Abschnitt 4.5).

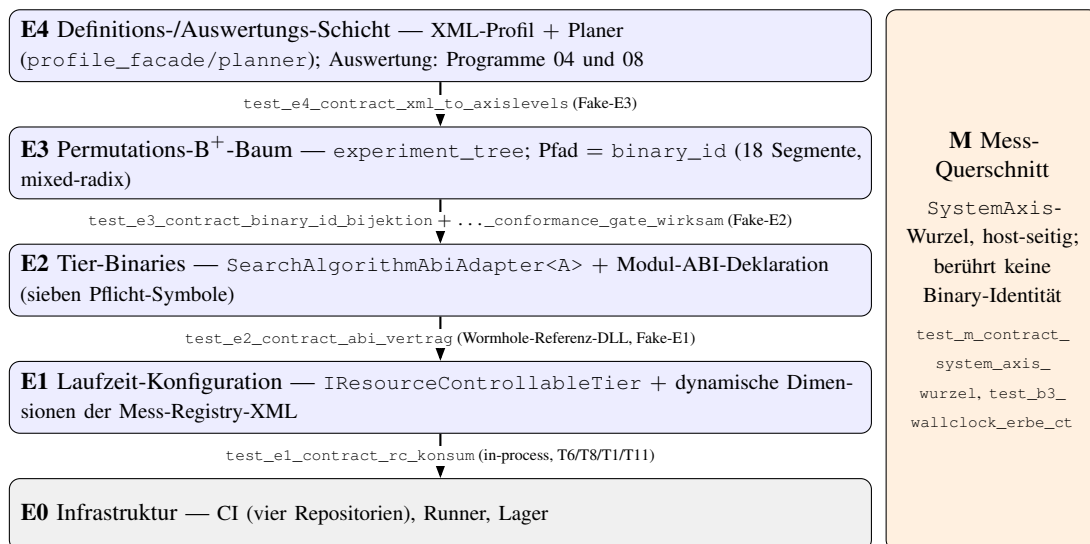


Abbildung 4.9: Die Ebenen E4–E0 des Schichtenmodells (Abschnitt 3.3.4) an ihrem Ort im Code, mit dem Contract-Test je Grenze (Fake der Nachbar-Ebene) und dem Mess-Querschnitt M, der alle Ebenen host-seitig quert, ohne die Binary-Identität zu berühren.

Erbin: Die Basis ist leer, und erst ein `static_assert (StempelVertrag<Erbin, Traeger>)` direkt unter der Definition macht eine unparsebare Zeile, eine fehlende Pflicht oder ein verbotenes Interface zum Übersetzungsfehler — eine Form des Consumer-Driven Contract [Rob06], in der die Erwartung des konsumierenden Trägers den Vertrag treibt.

Interface	Inhalt	Planer	CEB	Hybrid	Tier
VersionXyz	statische Selbst-Version X.Y.Z[.flag]	P	V	V	V
MessZeile	Stempel-Zeile des Mess-Realms	V	P	P	P
SystemZeile	Stempel-Zeile des System-Realms	V	P	P	P
OrganZeile	Stempel-Zeile des Organ-Realms	V	V	P	P
FingerprintSha	Fingerprint-Hex (Planer SHA-256, sonst SHA-512)	P	P	P	P
GesamtStempel	das eine Kompositum der Erbin	P	P	P	P
Angeschlossene	angeschlossene Stempel-Zeilen (Laufzeit-Haken)	V	V	P	V

Tabelle 4.3: Der Stempel-Vertrag als Matrix aus sieben Interfaces und vier Trägern (28 Zellen, dreiwertige Zulassung P = Pflicht, E = Erlaubt, V = Verboten; E ist im Bestand unbelegt). Die Verbots-Zelle der Organ-Zeile beim CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“; die Zellen werden je Erbin per `static_assert` geprüft. Die Spaltenordnung folgt wie Abbildung 3.9 der Verarbeitungs-Topologie mit der Hybrid-Tier-Binary als optionalem Einschub zwischen CEB und Tier-Binary.

4.4 Qualitätssicherung: Contract-Tests je Schicht

Das Schichtenmodell E4–E0 mit seinen festgeschriebenen Interface-Verträgen hat Abschnitt 3.3.4 als Konzept entwickelt; dieser Abschnitt beschreibt seine Umsetzung als Prüf-Infrastruktur. Der Grund liegt in der Schichtung selbst: Änderungen, die mehrere Schichten zugleich verändern, erzeugen Fehler aus Wechselwirkungen zwischen den Ebenen, die weder lokale Tests noch zeilenweises Review sichtbar machen (ADR-5, Anhang E). Für ein System, dessen Produktraum jede vollständige Materialisierung ausschließt (Abschnitt 3.4.6), sind Änderungen über mehrere Ebenen zugleich damit praktisch nicht abnehmbar; die Konsequenz ist, jede Schicht einzeln fertigzustellen — mit explizit festgeschriebenem Interface-Vertrag zur Nachbar-Ebene, eigenem Test-Harness und eigener Abnahme-Definition.

Das tragende Prüfprinzip lautet: *Jede Schicht wird gegen einen übersetzungszeitlichen Ersatz ihrer Nachbar-Ebene getestet.* Die Contract-Tests sind gewöhnliche Testziele der Build-Pipeline; die Nachbar-Ebene ersetzt dabei je ein *Fake* — eine Template- und Concept-basierte Attrappe im Test-Scope, kein Laufzeit-Umschalter im Produktiv-Code. So kann eine Schicht fertig und abgenommen sein, bevor die tiefere Ebene ihre Werte überhaupt konsumiert, und die Prüffrage jeder Grenze — reicht die Schicht ihre Definition *verlustfrei* an die nächste weiter? (Abschnitt 3.3.4) — wird vom Entwurfsgrundsatz zum ausführbaren Abnahme-Test.

Je Schicht konkretisiert sich das so (Abbildung 4.9): Für die Definitions- und Auswertungs-Schicht E4 prüft der Harness (`test_e4_contract_xml_to_axislevels`) die Übersetzung der deklarativen XML-Profile in die Achsen-Ebenen-Struktur — Golden-Fixtures vergleichen die aus einem Profil emittierten Achsen-Ebenen strukturell gegen eine erwartete Referenz, wobei ein Fake-E3 den Permutationsbaum ersetzt; weder Baum-Aufbau noch Bau-Aufträge sind nötig, um die Grenze E4→E3 abzunehmen. Für den Permutations-B⁺-Baum E3 prüft der Harness (`test_e3_contract_binary_id_bijektion`) die Bijektion der Binary-Identität: Aus Achsen-Ebenen-Fixtures wird die Permutations-

Enumeration gefahren und je Pfad der mixed-radix-kodierte Binary-Identifier hin- und zurückübersetzt (Zählung plus Signatur-Roundtrip) — gegen ein Fake-E2, das keine einzige Binary baut; dazu tritt ein Konformitäts-Gate im Voll-Lauf-Pfad (`test_e3_contract_conformance_gate_wirksam`), das jede importierte Binary vor der Messung gegen den festgeschriebenen Vertrag prüft (unter `-debug` zusätzlich gegen den Standard-Testsatz des Tier-Interfaces). Für die Tier-Binaries E2 ist der Vertrag die ABI selbst (`test_e2_contract_abi_vertrag`): Die Wormhole-Referenz-DLL wird geladen und muss die festgeschriebene Schnittstellen-Folge erfüllen — `IAatomyBase`, `IObservableTier` und `IResourceControllableTier` (an jedem Adapter vorhanden) samt fester Fähigkeits-Auskunft. Die Antwort einer Stellgrößen-Anfrage (`apply`) ist dabei kein Fehlercode, sondern die *Zahl der Achsen, die den Wert tatsächlich angenommen haben*: Eine Anfrage oberhalb der gemeldeten Fähigkeiten wird sichtbar, weil diese Zahl unter der Zahl der angefragten Achsen bleibt — im Wormhole-Piloten nehmen von fünf steuerbaren Achsen genau zwei (Batch-Größe und Inline-Schwelle) den Wert tatsächlich an, ein Nullzeiger liefert null —, und das ist die Vollständigkeitsregel der Stellgrößen: Nichts wird still angenommen, jede Rückmeldung zählt nur den realen Konsum. Das zugehörige Fake-E1 legt die Laufzeit-Stellgrößen an und appliziert sie, verlangt aber keine Konsum-Wirkung — die gehört zur Abnahme von E1. Für die Laufzeit-Konfiguration E1 schließlich prüfen Edge-Tests (`test_e1_contract_rc_konsum`, in-process über die Achsen T6, T8, T1 und T11) die Randfälle des dynamischen Regimes: dass ein wiederverwendetes Tier zwischen zwei Läufen vollständig zurückgesetzt wird, dass ein Pool-Budget unterhalb des lebenden Datenbestands als benannter Zustand behandelt statt still verletzt wird und dass jede applizierte Stellgröße als benannte Spalte in der persistierten Ergebnis-Datei sichtbar ist, dazu der Erhalt der Zwei-Phasen-Operationsschleife (verworfenen Warmlauf, Messlauf) als Invariante. Geprüft wird die Persistenz an der Ergebnis-Datei, nicht am In-Process-Zustand; Führung ist dabei die Standard-xlsx — die Rückschreib-Registry führt sie als vierte Methode und Default-Auswerteformat, der xlsx-Writer liegt in der Lager-Ablage —, und die CSV-Form ist kein einzelnes Kind-Dokument, sondern ein geordneter Baum aus CSV-Dateien (ein Gesamtordner je Mappe, darunter ein Ordner je Blatt, darin die CSV-Dateien der einzelnen Unter-Achsen-Fahrten), der immer aus der Mappe erzeugt wird und per XML statt der Mappe oder zusätzlich zu ihr am selben Ziel liegt. Umsetzungsstand: Die E1-Prüfung liest die CSV-Ausgabe der Persist-Phase, die die Naht als eine flache Datei schreibt (`projektion_oeffnen`); die Prüfung an der je Binary erzeugten xlsx und die Baumform der CSV-Ablage sind vorgesehen, aber nicht implementiert.

Oberhalb der Schicht-Harnesse sichert ein *Zwei-Gate-Protokoll* jedes Implementierungs-Paket ab. Sein Zweck ist die Trennung zweier Fragen: Gate 1 sichert das Engine-Repository gegen sich selbst — die Einheits- und Vertrags-Tests —, Gate 2 sichert die Integration in das Projekt-Repository, in dem die Kette XML → Binaries → Messung → Anhang tatsächlich läuft; erst beide zusammen weisen nach, dass ein Paket den Apparat nicht bricht (Continuous Integration und Delivery als Sicherungsstufen [Fow06, HF10]). *Gate 1* ist das scharfe Gate des Engine-Repositories: Die volle Test-Suite² läuft in vier Zellen, gcc und clang jeweils als Release- und Debug-Bau, ohne Änderungs-Skip, und akzeptiert wird nur ein Lauf ohne fehlgeschlagene Tests, ohne Ausnahmeliste. Der Doppellauf hat zwei Gründe: Die Compiler-Matrix zwingt jede statische Prüfung durch zwei Übersetzer, und die Wiederholung des Messlaufs — die Zwei-Phasen-Operationsschleife aus verworfenem Warmlauf und Messlauf im Ein-Thread-deterministischen Modus `measure` — macht die Run-to-Run-Stabilität der Zahl selbst zum

²Umfang der Test-Suite: 551 registrierte Tests (`ctest -N`) im gcc-Release-Baubaum vom 17. September 2026.

Prüfgegenstand. Bestandteil von Gate 1 ist die Golden-Verankerung des Permutationsraums, und hier sind zwei Begriffe zu trennen (Abschnitt 3.4.6): Der *Referenz-Katalog* des Golden-Tests hält $2^{17} = 131072$ Binary-Identitäten — siebzehn Achsen mit je zwei Vertretern, das Persistenz-Ziel auf einen festgelegt; eine vereinfachende, überholte Annahme, denn die Achsen tragen mehrere Auslegungen, der Katalog ist Regressions-Detektor, nicht Obergrenze — und wird gegen eine festgeschriebene CRC-64-Prüfsumme bestätigt (0x56F1B721C72DC10E, CRC-64/ECMA-182 über die Identitäten in kanonischer Reihenfolge; die Referenz ist literaler Werkzeug-Output, und ihre Änderung ist nur im Rahmen einer koordinierten Schnittstellen-Änderung zulässig), während ein kleiner, materialisierbarer Katalog von $4 \cdot 4 \cdot 5 \cdot 4 = 320$ Identitäten als byte-genaue Prüfung der Serialisierungsform dient — ein Charakterisierungs-Test im Sinne von Feathers [Fea04], der eine festgeschriebene Kardinalität als Regressions-Detektor nutzt. Die *Mächtigkeit* des in einer vollständigen Messreihe geplanten Raums ist davon verschieden und nie statisch: Der Planer zählt sie je Achse aus den in der XML deklarierten Möglichkeiten im Director-Walk (check-size, simulate) und multipliziert die Stufen über alle Achsen; sie unterscheidet sich je Plattform durch fehlende Freigaben, und jede Zahl der Rechnung kommt als Eingang herein, nie als Konstante. Gate 2 ist das Integrations-Gate des Projekt-Repositories: Seine Pipeline baut fehlerfrei ohne Ausnahmemenge — die Regel dieser Arbeit kennt keine geduldeten fehlschlagenden Tests; die beiden deklarierten Fehlschlag-Duldungen der Messmaschinen liegen im Engine-Repository und sind eine Plattform-Grenze (ein arm64-Smoke-Bau ohne verfügbaren Runner) und ein manueller Relock-Wartungsjob (er darf nicht ausgeführt bleiben), keine Testmenge.

Dieses Protokoll ist Methodik, kein Systembestandteil: Es beschreibt, *wie* Änderungen am Apparat abgenommen werden, nicht, was die Binaries enthalten. Die Verzeichnis- und Suite-Struktur, auf der es läuft, dokumentiert Anhang B.

Die Fertigstellung folgt der Schichtung top-down — E4, E3, E2, dann der Mess-Querschnitt M unmittelbar vor E1 —, und die Infrastruktur E0 bildet den letzten Schritt der Konsolidierung (Abschnitt 3.3.4). Der Reifegrad ist am Test-Bestand ablesbar: Alle vier Grenzen besitzen ihren Harness (oben), und der Querschnitt M besitzt einen eigenen, der ohne jede Tier-Binary auskommt (test_m_contract_system_axis_wurzel für die System-Achsen-Wurzel und die Mess-Visitierbarkeit, test_b3_wallclock_erbe_ct für das Wallclock-Erbe): Eine synthetische Mess-Quelle wird über die System-Achsen-Wurzel getrieben und ihre Werte bis in die Ergebnis-Spalten verfolgt. Die Mess-Achse treibt dabei die Freigabe der Mess-Quelle über die System-Achsen-Wurzel in Gestalt der CEB: Der Planer delegiert an das CEB-Interface, welche Binary-Rekombination gegen welche Lasten zu messen ist, und die CEB gibt die Quelle frei. Umsetzungsstand: Die Hardware-Erhebungskette läuft an der Freigabe-Naht des Builders (Abschnitt 4.5); der CEB-getriebene Freigabe-Pfad der Mess-Quelle ist vorgesehen, aber nicht implementiert. Die Code-Gestalt des Querschnitts beschreibt der folgende Abschnitt.

4.5 Realm-Wurzeln im Code: System- und Mess-Achsen

Vier Abstraktionsebenen ordnen die Begriffe dieses Abschnitts (Abbildung 4.10), in Anlehnung an die vierstufige Meta-Ebenen-Architektur der Object Management Group (MOF) [Obj16], in der M0 die Laufzeit-Instanzen, M1 das Modell, M2 das Metamodell und M3 das Meta-Metamodell bezeichnet: M3 sind die *Meta-Meta-Achsen* im Sinne der Indirektion einer Beschreibung — eine Achse, die beschreibt, wie andere Achsen beschrieben werden (die Mess-Meta-Meta load_framework, die Organ-Meta-Meta

`disk_io`, die System-Meta-Metas unter dem Hub der Erweiterungs-Hardware); M2 sind als Lesart die *Meta-Achsen*, die literale Klassifizierung der unter ihrem Begriff geführten Haupt- und Unter-Achsen — im Code die Registry-Zeile mit Kategorie, Familie und Bausteinliste; M1 sind Konzeption und Laufzeit-Implementierung der Haupt- und Unter-Achsen — die CRTP-Achse mit ihrem Concept; M0 ist der Binärcode — die Tier-Binary, in der genau eine Belegung einkompiliert ist. Im Stempel ist die Meta-Meta-Ebene eines Eintrags mechanisch seine Klammer-Tiefe: Die Bits 3–5 eines Eintrags kodieren die Ebene, eine Haupt-Achse ist klammerlos, eine Meta-Meta hängt als Klammer-Anhang an ihrer Zeile, und eine Tiefe über sieben bricht mit Fehlermeldung ab; der Renderer der Meta-Meta-Einträge ist zugleich der Engpass, an dem zwei Versionsprüfungen jedes gestempelte Glied prüfen, sodass keine neue Meta-Meta diese Prüfung umgehen kann. Die Kurzform `c{p.e}` eines Versions-Flags bezeichnet dabei `c` mit den Untergliedern `p` und `e`, nie ein flaches Kürzel. Je Realm gibt es einen eigenen Meta-Meta-Diskriminator (`measurement_meta_meta`, `system_meta_meta`, `organ_meta_meta`); die PMC-Erfassung — Performance Monitoring Counter, die Hardware-Zähler der CPU — ist eine Meta-Meta-Mess-Achse, weil sie beschreibt, mit welchem Zähler-Instrument andere Messungen erhoben werden.

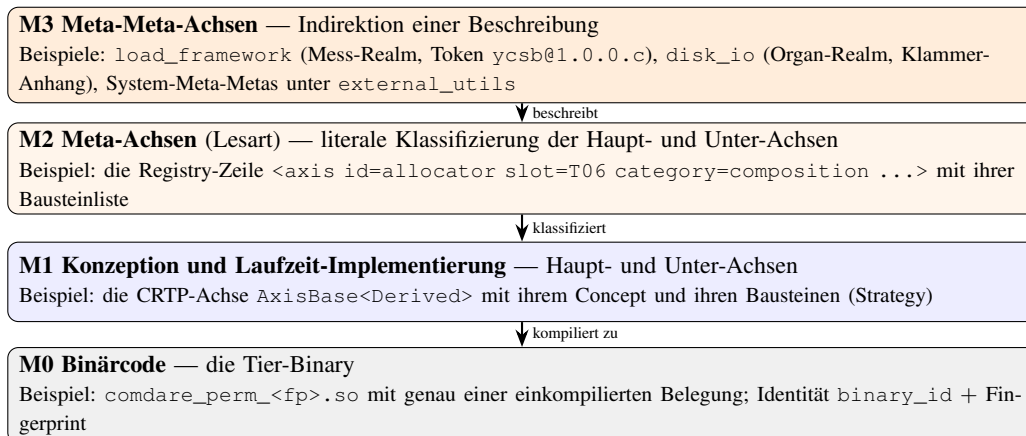


Abbildung 4.10: Die vier Abstraktionsebenen M0–M3 als Lesart der Achsen-Begriffe, in Anlehnung an die Meta-Ebenen-Architektur der MOF: Meta-Meta-Achsen (M3) beschreiben, wie andere Achsen beschrieben werden; die Registry-Zeile (M2) klassifiziert Haupt- und Unter-Achsen; die CRTP-Achse (M1) ist ihre Konzeption und Laufzeit-Implementierung; die Tier-Binary (M0) ist der Binärcode genau einer Belegung.

Die Dreiteilung der Achsen-Realms (Abschnitt 3.4.1) ist im Code als Typ-Trennung ausgeführt. Alle Achsen teilen ein gemeinsames Familiendach — `topics::Axis<Derived>`, eine leere, nicht-polymorphe CRTP-Basis, deren Concept-Prüfung (`is_empty` und nicht `is_polymorphic`) jede eingeschleppte `vtable` schon beim Übersetzen zurückweist — und deklarieren ihre Kategorie über die Trichotomie `AxisCategory` und ihre Familie über den Diskriminator `AxisKind` mit sechs Werten (Abschnitt 4.3). Je Familie existiert eine eigene CRTP-Wurzel: `OrganAxis` und `OrganMetaMetaAxis` für den Organ-Realm, `SystemAxis` für die Mess-System-Achsen des host-seitigen Querschnitts, `CebSystemAxis` für die Bau- und Steuer-Achsen der CEB-Konfiguration, `SystemMetaMetaAxis` für die System-Meta-Achsen unter dem Erweiterungs-Hub und `MeasurementMetaMetaAxis` für die Mess-Realm-Achsen des Planers. Eine Achse *ist* ihre Familie durch ihre Basisklasse, nicht durch ein umhängbares Etikett — `CebSystemAxis` liefert ihre Art `axis_kind()` als harten Rückgabewert `system_config`, weshalb der Mess-Realm eine eigene Wurzel besitzt statt einer wiederverwendeten:

Der System-Realm wurzelt an der CEB, der Mess-Realm am Planer (Abschnitt 3.4.1).

Auf der System-Seite ist die kanonische Ordnung selbst Code: Das konstante Drei-Element-Array `kSystemAxisOrder` hält die System-Haupt-Achsen `target_isa`, `operating_system` und `external_utils` in ihrer verbindlichen Reihenfolge, und jeder Konsument, der eine Ordnung braucht — die Stempel-Zeile, die Generator-Blockfolge, der XML-Kopf, die Suffix-Segmente —, muss sie dort lesen, statt sie nachzubilden (Ordnungs-Single-Source mit Drift-Prüfungen). Drei übersetzungszeitliche *Rückkehr-Sperren* halten den Schnitt der Haupt-Achsen fest: `static_assert`-Prüfungen weisen Compiler, Scheduling und Last-Rahmenwerk als System-Haupt-Achsen zurück — der Compiler ist eine Unter-Achsen-Gruppe `build_toolchain` des Komplex-Wrappers `build_target_complex` mit einer globalen Optimierungs-Identität im Toolchain-Glied des Fingerprints, das Scheduling eine Unter-Achse an `target_isa` mit fünf Dimensionen, das Last-Rahmenwerk eine Meta-Meta-Haupt-Achse des Mess-Realms —, und wer einen der drei wieder einträgt, bricht den Bau und muss die Zuordnung ausdrücklich widerrufen, statt sie still zu revidieren. Auch die Bau-Versionskennung der System-Zeile (Abschnitt 3.4.4) speist sich aus einer einzigen deklarativen Quelle: der geordneten Acht-Segment-Folge `kSuffixSegmentOrder` der Bau-Steuergrößen — Compiler-Kennung, Optimierung, SIMD-Erweiterung, CEB, Ziel-ISA, Telemetrie, Bau-Typ und Gate-Beitrag —, in der ein leeres Glied kein Segment erzeugt und das Gate-Segment am Ende steht, damit jeder Suffix-Vergleich einen unveränderten Anfang und ein Anhängsel sieht (Präfix-Stabilität); der vierteilige Zell-Pfad der Ablage folgt derselben Ordnung Glied für Glied, mit einer Prüfung, die beide Ordnungen aneinander bindet.

Auf der Mess-Seite besitzt der Planer seine eigene Meta-Meta-Wurzel `MeasurementMetaMetaAxis`; ihre erste Trägerin ist das Last-Rahmenwerk `load_framework` (Stempel-Token `ycsb@1.0.0.c`), und wie alle Mess-Realm-Achsen ist sie strikt `binary-id-neutral` (Registry-XML: `binary_id = never`). Die Mess-Tooling-Haupt-Achse hält genau drei versionierte Einträge — Wallclock, Makro, Mikro — in der einzig erlaubten Reihenfolge `wallclock/macro/micro` (Alias `w/ma/mi`, Abschnitt 3.4); ihre Prüfung macht ein viertes, unabgestimmtes Tooling zum Übersetzungsfehler. Sie fächert vier Unter-Achsen-Registries auf, je eine Datei (Tabelle 4.4): den Arbeitsmodus (`WorkMode`: `build`, `measure`, `compare`, `release`, Abschnitt 4.1), das Mess-Framework (`MeasurementFramework`: genau ein Eintrag, YCSB), die Rückschreib-Methode (`WritebackMethod`: `CSV`, LaTeX-Tabelle, Vergleichs-Metriken und `xlsx` — vier Werte, `xlsx` als Default-Auswerteformat, ausdrücklich kein PDF, weil die PDF-Kompilation kein Rückschreib-Kanal der Mess-Maschine ist) und die sechzehn Mess-Kategorien (unten); jede Registry ist ein `constexpr`-Katalog mit der statischen Prüfung `Index = Enum-Wert`, und ein weiterer Eintrag bricht die Übersetzung. Die Zähler-Instrumentierung steckt im Mikro-Tooling; eine PMC-Vendor-Registry ergänzt sie um die Hardwareform der Zähler (unten). Übersetzungsseitig spiegeln zwei getrennte Gates diese Tooling-Leiter: Das *Statistik-Gate* (`COMDARE_CE_ENABLE_STATISTICS`, ein `#ifdef`) schaltet die Observer-Statistik des Makro-Regimes, das wertbasierte *Segment-Timing-Gate* (`COMDARE_CE_ENABLE_SEGMENT_TIMING`, ein `#if`) die achtzehn Per-Achsen-Segment-Timer des Feinkorns. Ein nicht gesetztes Segment-Timing-Gate *erbt* den Zustand des Statistik-Gates — Bestandsbauten ändern durch die Trennung kein Byte —, und ein gesetztes Segment-Timing-Gate ohne Statistik-Gate bricht die Übersetzung mit Fehlermeldung ab, denn die Segment-Timer schreiben in Strukturen des Observer-Blocks. Der tatsächlich wirksame Gate-Zustand jeder Übersetzungseinheit wandert als eigenes Mess-Gate-Glied mit neun `;`-getrennten Feldern in das Fingerprint-Preimage des Stempels (Abschnitt 3.4.4) — im Aus-Zustand `mg=m0;s0;st0;x0;tw0;tm0;tmi0;hm0;hmi0`: Messung, Statistik, Segment-Timing,

Experiment-Modus, dann Wallclock, Makro und Mikro der Tier-Stufe und Makro und Mikro der Hybrid-Stufe auf eigenen Positionen —; der Stempel behauptet damit nie eine Ausstattung, die der Bau nicht fährt. Das Zielbild neben diesem Registry-Stand ist das Achsen-Interface-Modell aus Abschnitt 3.4.6: Mikro-Benchmarking vermisst eine Achse über ihr Achsen-Interface, und die drei Mess-Ebenen liefern die Gegenmessung; die Registry ist an dieses Modell angeglichen.

Registry (Datei)	Werte	Anzahl	Prüfung
run_ methodology_ registry	WorkMode: build, measure, compare, release	4	Index = Enum, Ablauf-Ordnung; nur measure misst
measurement_ framework_ registry	MeasurementFramework: Ycsb	1	ein zweites Framework bricht compile-time
writeback_ method_registry	WritebackMethod: Csv, LatexTable, ComparisonMetrics, Xlsx (Default)	4	eine fünfte Methode bricht compile-time; kein PDF
measurement_ axis_registry	sechzehn Mess-Kategorien mit Regime (9 Zeit/Observer, 7 PMC)	16	Index = Kategorie, kein leerer Name, Regime = Wurzel-Zuordnung

Tabelle 4.4: Die vier Unter-Achsen-Registries des Mess-Toolings: je eine `constexpr`-Datei mit Indexgleich-Enum-Prüfung; die Haupt-Achse selbst hält die drei Toolings wallclock, macro und micro.

Die Abschaltung der Messung ist ein Stufenmodell über den drei Mess-Ebenen (Abbildung 4.11). Das Messgerät `checkpoint_measure` erhebt die Eingangsparameter stets zu einem *Zeitpunkt*; die erste Schicht, die Wallclock, ist das Minimum eines zeitmessenden, sonst parameter-leeren Checkpoints und damit das minimale Messgerät der CEB. Sind Makro- oder Mikro-Benchmarking aktiv, vollzieht auch der Wallclock-Checkpoint die Parameter-Klammer — ein Übersetzungszeit-Sonderfall, weil in Makro und Mikro Parameter und Zeitmessung stets zusammenfallen. Die erste Abschaltungsstufe schaltet in Makro und Mikro die Parameter-Erhebung ab, sodass die Wallclock für `measure` und `compare` bestehen bleibt; die zweite, permutative Abschaltungsstufe lässt für `release` auch die Wallclock entfallen, und mit ihr entfallen die Flächen und Interfaces der Messung: Für den Release bleibt nur der Kern-Code der Binary stehen. Umsetzungsstand: Der Code enthält die Gates G2 und G3 und den Release-Modus ohne Instrumentierung (`measurement_on` aus); die zweistufige Abschaltung als Permutations-Größe ist vorgesehen, aber nicht implementiert.

Das PMC-Tooling ist mehrschichtig, weil die Hardware-Zähler je Kern-Sorte und Hersteller verschieden sind (Zugriff über die `perf_event_open`-Schnittstelle des Linux-Kernels [Lin26b]). Eine generische Ereignis-Liste (`pmc_event_set`) entscheidet, ob PMC überhaupt eingebaut wird: Die Host-Sonde des Planers prüft genau die Events, die die reale Quelle später öffnet, damit die Entscheidung und die Messung dieselbe Quelle lesen. Ein modellgebundener RAW-Katalog (`pmc_raw_event_katalog`) füllt die Kanäle für L2-Misses und Kohärenz je Mikroarchitektur: Für AMD Zen 5 (Family 0x1A, gemessen auf dem Ryzen 9 9950X3D von prod1) ist die Belegung gegengeprüft — Event 0x64/Umask 0x9 für die Demand-Misses in L2 und Event 0x43/Umask 0x14 für Demand-Fills aus dem Cache eines anderen CCX, gegen die benannten `perf`-Events mit identischen Zählerständen (17 314 023 gegen 17 314 023); für Intel liegt kein Eintrag vor, weil eine RAW-Belegung ohne Gegenprüfung durch Messung ein Rateversuch wäre [Int24b] — eine ausgewiesene Lücke, die als fehlender Wert erscheint.

	wallclock (w)	macro (ma)	micro (mi)
Stufe 0: volle Messung (measure)	Zeitpunkt + Parameter-Klammer (Übersetzungszeit-Sonderfall)	Zeit + Parameter je Gattungs-Interface-Funktion	Zeit + Erfolgs-Parameter je Achsen-Interface
Stufe 1: Parameter aus (measure, compare)	nur Zeitpunkt: parameter-leerer <code>checkpoint_measure</code> (Minimum)	Parameter-Erhebung abgeschaltet	Parameter-Erhebung abgeschaltet
Stufe 2: alles aus (release)	entfällt — keine Mess-Fläche, kein Mess-Interface	entfällt	entfällt: nur der Kern-Code der Binary

Abbildung 4.11: Die zwei Abschaltungsstufen der Messung über den drei Mess-Ebenen: `checkpoint_measure` erhebt zu einem Zeitpunkt; die Wallclock ist das parameter-leere Minimum, vollzieht bei aktivem Makro oder Mikro aber die Parameter-Klammer mit; Stufe 1 schaltet die Parameter-Erhebung ab, Stufe 2 lässt für den Release auch die Wallclock samt Mess-Flächen entfallen.

Die Semantik ist offengelegt: `cache_misses_l2` erfasst auf Zen 5 die Demand-Misses ohne L2-Prefetch, `coherence_invalidations` die Cache-zu-Cache-Fills aus einem fremden CCX — das beobachtbare Kohärenz-Maß des Kern-PMU, kein wörtlicher Invalidierungs-Zähler. Eine Vendor-Registry (`pmc_vendor_registry`) führt die PMC als *ein* Merkmal mit zwei Werten — AMD und Intel sind zwei Hardware-Komponenten, nicht ein PMC — mit je eigener Versionskennung, weil eine CEB auf genau einem Host gebaut wird und die eingebaute Zählerform ihren Fingerprint mitbestimmt; die Planer-Hardware-Sonde erkennt je Lauf auf jeder Maschine für sich, ob PMC existiert, und die CI fährt zwei Zähler-Pfade, `pmc:amd` auf `prod1` und `pmc:intel` auf `prod2`, dem Intel Core i9-12900K (Alder Lake) mit P- und E-Kernen. Die getrennte Messung der P- und E-Kerne ist als Scheduling-Dimension und über die Kern-Pinning-Bausteine angelegt (Sechs-Modus-Modell mit HybridAware als Pflicht-Modus, Abschnitt 3.4.1); Umsetzungsstand: Das Laufzeit-Pinning je Kern-Sorte ist vorgesehen, aber nicht implementiert.

Der host-seitige Mess-Querschnitt des System-Realms — der Querschnitt M, der alle Schichten durchdringt, ohne je die Binary-Identität zu berühren (Abschnitt 3.4.1) — wurzelt in der CRTP-Basis `SystemAxis` (Datei `system_axis.hpp`), die nur lesend an die host-seitigen Ergebnis-Strukturen andockt. Sie ist als zero-cost abstraction implementiert — eine leere, nicht-polymorphe Basis ohne virtuelle Aufrufe im Hot-Path, deren Konformität ein eigenes Concept erzwingt — und übersetzt die Konzept-Invarianten in übersetzungszeitliche Vollständigkeits-Gates: Ein `consteval`-Gate prüft, dass jede der sechzehn Mess-Kategorien genau einem der beiden Regimes `TimeObserver` und `PmcCounter` zugeordnet ist (neun zeit- und observer-basierte, sieben zählerbasierte PMC-Kategorien, Abschnitt 3.4.1), und ein zweites weist jede Achse zurück, die Kategorien beider Regimes mischt — das Regime-Misch-Verbot bricht so die Übersetzung statt erst die Auswertung.

Die sechzehn Mess-Kategorien führt eine einzige `constexpr`-Registry (`measurement_axis_registry.hpp`): je Kategorie der Name und das aus der einen Quelle abgeleitete Regime. Ein statisches Gate erzwingt, dass der Tabellen-Index mit dem Kategorie-Wert übereinstimmt, kein Name leer ist und kein Regime von der Wurzel-Zuordnung abweicht — eine siebzehnte Kategorie oder eine stille Index-Drift wird damit zum Übersetzungsfehler; das Wachstum der Kategorien-Menge läuft über die Kanal-Adressen

des Achsen-Interface-Modells (unten), nicht an dieser Prüfung vorbei. Konzeptionell sind die Kategorien eine *Unter*-Achse des Mess-Toolings und manifestieren sich als Spalten der Ergebnis-Tabellen (Abschnitt 3.4.1); die Registry-Namen sind zugleich das Berichts-Vokabular der Auswertungs-Schicht (die Spalten-Namen der E4-Berichte), und eine compile-time-Iteration über alle Einträge stellt die Registry der Metaprogrammierung zur Verfügung.

Die Erhebung abstrahiert eine vendor-neutrale Mess-Quellen-Schnittstelle (`i_measurement_source.hpp`). `IMeasurementSource` beschreibt eine Quelle über ihre *strukturellen* Fähigkeiten — welche der zehn Ereignis-Semantiken (Zyklen, Instruktionen, L1-/L2-/L3-Cache-, dTLB- und Branch-Misses, Speicher-Stalls, Kohärenz-Invalidierungen, Energie) sie überhaupt ausdrücken kann — und liefert je Messintervall ein `MeasuredDelta` mit *per-Ereignis-Gültigkeit*: Der Wert Null mit gesetztem Gültigkeits-Flag ist eine tatsächliche Null-Messung; ein nicht gesetztes Flag heißt *nicht gemessen*. Die Fähigkeits-Auskunft gibt dabei nie eine Mess-Garantie — die Gültigkeit je Messung führen allein die Gültigkeits-Flags. Als garantierter, ausdrücklich *degradiertes* Fallback existiert die Wall-Clock-Quelle (`WallClockSource`): Sie liefert nur einen Nanosekunden-Proxy für die Zyklen und lässt alle Hardware-Ereignisse ungültig; die Bestands-PMC-Familie bindet der Adapter `PmcSourceAdapter` an, der nur die sieben strukturell ausdrückbaren Kanäle bewirbt — L1d-, L2-, L3-, dTLB- und Branch-Misses, Kohärenz-Invalidierungen und Energie —, während Zyklen, Instruktionen und Speicher-Stalls im Zähler-POD strukturell nicht ausdrückbar sind und ungültig bleiben.

Im Achsen-Interface-Modell wird die Mess-Quellen-Schnittstelle für das Mikro-Benchmarking je Achsen-Interface speziell für dessen Parametersatz erhoben und in Stufen über das Makro-Benchmarking und die Wallclock in der CEB an Parametern zusammengetragen; die Kategorien-Menge wächst damit über die Kanal-Adressen Achse → Gattung → Kategorie (Hierarchie-Tiefe drei) auf hunderte Mess-Kategorien, je Achsen-Interface und Parametersatz eine. Umsetzungsstand: sechzehn Kategorien, durch die Registry statisch festgelegt (oben). Dazu tritt die *ungebremste Nachmessung*: In einem Experiment werden die Aktionen jeder Achse in Reihenfolge über den Checkpoint der zuerst aufgerufenen Achsen-Interface-Funktion hinter dem Gattungs-Interface durch eine eigene Steuerungsfläche zurückgerollt, damit anschließend alle Achsen vom Eintrittspunkt aus ohne Messung erneut vermessen werden können — die CEB übernimmt dazu im Einzelmodus den inneren Zustand einer Tier-Binary mit Messfühlern in die analoge Tier-Binary ohne Messfühler (nur der Wallclock-Checkpoint) und misst vom selben Punkt erneut, gestaffelt nach dem Last-Profil-Eingang der Tier-Binary je Makro-Benchmarking-Interface rekursiv. Ein einfaches erneutes Durchlaufen verschiedener Permutationen genügt dafür nicht, und ein kompletter Neustart weist einen neuen Speicherblock zu und verschlechtert die Vergleichbarkeit, wenn nicht exakt auf den letzten Checkpoint zurückgerollt werden kann. Umsetzungsstand: vorgesehen, aber nicht implementiert. Das Prüfkriterium der Nachmessung ist die Plausibilitäts-Invariante, dass die Binary mit Messfühlern langsamer ist als die ohne (Kapitel 5).

Der Grundsatz der ausgewiesenen Nichterhebung (Abschnitt 3.4.1) — ein fehlender Wert ist ein eigener Zustand, kodiert als Gültigkeits-Flag oder als NaN nach IEEE 754 [IEE19], nie ein Nullwert; in der Terminologie fehlender Daten ist er *fehlend*, nicht *null* [Rub76] — ist in diesen Bausteinen einprogrammiert statt dokumentiert (im Code als Vokabular `honest-0`, `honest-100` und `honest-empty` in Kommentaren und Prüf-Dock-Meldungen benannt; die Mess-Bausteine kodieren sie als Stichproben-Status `NotApplicable` und `SourceUnavailable` (`SampleStatus`) sowie als `*_source_available`-Flags im Zähler-POD). Die Wall-Clock-System-Achse verweigert es,

ihren Mittelwert als Perzentil zu etikettieren: Anfragen nach p50 bis p999 beantwortet sie ungültig, denn die durch die Operationszahl geteilte Gesamtzeit ist ein Mittelwert, und ein als Perzentil ausgegebener Mittelwert wäre ein Phantomwert — tatsächliche Perzentile entstehen erst nachgelagert in der Auswertung: nearest-rank über den erhobenen Rohlatenzen (die einzige Umrechnung, das Typ-1-Quantil nach Hyndman und Fan [HF96]), ergänzt um das HDR-Histogramm [Ten12] (High Dynamic Range), das bei drei signifikanten Stellen mit einer relativen Bucket-Toleranz von $1/1024 \approx 0,0977\%$ auflöst — hergeleitet aus seiner Bucket-Geometrie: $2 \cdot 10^3$ Werte mit Einheits-Auflösung, $\lceil \log_2 2000 \rceil = 11$, also 2^{10} Unter-Buckets je Hälfte —; im Messpfad selbst steht bewusst kein Histogramm-Bucket, weil er den Rohwert schon bei der Aufnahme unumkehrbar quantisierte. Die Cache-Line-Auslastung (CLU, cache line utilization) wird als tatsächliche Auslastungs-Ratio erhoben — genutzte Nutzdaten-Bytes gegen belegte Cache-Lines mal Zeilenbreite, die Felder `field_bytes`, `cache_lines` und `line_bytes` des Achsen-Schemas —, nicht als roher Zeilen-Zähler, der gerade die *inverse* Metrik wäre. Speicher-Fußabdruck und Fill-Buffer-Auslastung bleiben ungültig, bis eine tatsächliche Peak-Spalte beziehungsweise eine Hardware-Quelle existiert (der Software-Operations-Puffer der Pufferungs-Achse ist kein Proxy für den Hardware-Line-Fill-Buffer), und IPC/CPI bleibt ungültig, solange das Zähler-POD keine Instruktions- und Zyklen-Spalten besitzt.

Die Hardware-Erkennung ist die Erhebungskette der System-Haupt-Achse `target_isa` — ihre Unter-Achse `target_isa_complex` führt je benannter Rekombination die Glieder CPU-Fabrikation, RAM-Frequenz und CAS-Latenz — und zugleich eine *Mess*-Achse (Abschnitt 3.4.3), deren Planer-Sicht die Host-Sonde ist; sie liegt in beiden Stufen als Code vor. Die erste Stufe umfasst das Provenienz-Vokabular (die drei Vertrauensstufen `ConfiguredMeasured`, `SpdJedecBase` und `DeclaredNotMeasured` als eigener Ergebnis-Typ; „nicht erhoben“ ist bewusst keine vierte Stufe, sondern ein eigener Zustand), einen DDR5-SPD-Parser (Serial Presence Detect, der Modul-Datensatz nach JEDEC [JED25]) als reine, `constexpr`-fähige Byte-Funktionen über einem übergebenen Speicherbereich — 141 Zeilen, ohne Ein-/Ausgabe und ohne Fremdcode-Vendoring, mit `std::expected` als Ergebnis-Typ; Übertaktungs-Profilen (XMP, Extreme Memory Profile) erkennt er als separates Angebot, nie als Ist-Wert — sowie eine eigene Fehler-Domäne mit vier Klassen (`QuelleFehlt`, `QuelleUnlesbar`, `QuelleKorrupt`, `FormatUnbekannt`), damit eine korrupte Quelle nie still zur nächsten Stufe degradiert. Die zweite Stufe verdrahtet die Erhebung: Die Laufzeit-Kette Boot-Cache (die gefilterte SMBIOS/DMI-Tabelle des Systems [Dis24]) → SPD → Deklaration ist eine Chain of Responsibility [GHJV94] ohne virtuelle Aufrufe — ihre Glieder sind heterogene Typen, ihre Verdrahtung ist eine Typliste [Ale01], sodass die Reihenfolge (das Wesentliche einer solchen Kette) im Typ steht statt in einer Laufzeit-Liste, und eine statische Prüfung hält fest, dass jede Provenienz-Stufe genau einmal vorkommt; die Kette bricht bei der höchsten erreichbaren Vertrauensstufe ab, die liefernde Stufe bestimmt die Provenienz des Ergebnisses (kein Glied kann eine fremde Herkunft behaupten), sie bezieht ihre Quell-Pfade per Injektion (jede Stufe ist ohne die Zielpattform testbar) und läuft ausschließlich, einmalig gepuffert, an der Freigabe-Naht des Builders. Welche Kette gilt, entscheidet eine Übersetzungszeit-Factory (`hardware_probe_factory`) über der Zelle aus ISA-Komplex und Betriebssystem: Ihr primäres Template ist deklariert, aber nicht definiert — eine Zelle ohne bewusste Entscheidung ist ein unvollständiger Typ und bricht den Bau —, eine Totalitäts-Prüfung erfasst die volle Matrix aller deklarierten ISA-Komplexe und Betriebssysteme, und nicht implementierte Zellen (Windows, macOS) sind ausgewiesene Nur-Deklarations-Zellen mit ausdrücklich besetzten oberen Stufen, damit der Degradations-Pfad „gefragt, gibt es hier nicht“ meldet

statt „übersprungen“.

Den Abgleich zwischen Deklaration und Erhebung fällt ein benanntes Laufzeit-Verdikt mit genau sieben Ausgängen — Übereinstimmung, Abweichung, nur deklariert, nicht erhoben, keine Deklaration, unvergleichbare Stufen und Etikett-Widerspruch —, wobei nur provenienz-kompatible Größen überhaupt verglichen werden (ein festgeschriebenes DMI-Ergebnis nur gegen eine konfigurierte Messung, ein festgeschriebenes SPD-Ergebnis nur gegen die SPD-Basis) und eine Abweichung auf die Fehlerklasse „Hardware-Erweiterung fehlt“ abgebildet wird; kein Verdikt und kein Messwert erreicht je die Stempel- oder Identitäts-APIs (Abschnitt 3.4.4). Damit ist die in Abschnitt 3.4.3 geforderte automatisierte Wiedervorlage im Code umgesetzt: Der in der System-Registry deklarierte `prod2`-Wert von 4800 MT/s ist ein festgeschriebenes `dmidecode`-Ergebnis und bleibt als Notiz seines Ursprungs geführt (`DeclarationOrigin`), der Code legt nicht die Zahl fest, sondern nur die Präsenz eines Werts, und die Erhebung findet bei jedem Lauf erneut statt und führt ihre Vertrauensstufe mit — in der Messung liefert Stufe 1 auf `prod1` den konfigurierten Takt von 5600 MT/s bei aktivem XMP und auf `prod2` 4800 MT/s.

Am Auswertungs-Ende des Querschnitts liegt der erste Ansatz der Kurvenanpassung aus Abschnitt 3.6.3 als Gerüst vor (`curve_fit.hpp` im Builder): Eine Mess-Achsen-Kurve über die Working-Set-Größe wird per kleinsten Quadrate an das Modell $y = a \log_2 x + b$ angepasst — die für Cache-Hierarchie-Kurven erwartete Form, deren Stufen die Kapazitätsgrenzen der Hierarchie abbilden [HS89]. Auch hier gilt: kein Fit ohne gültige Daten. Der Fit-Status unterscheidet *keine Daten*, *unterbestimmt* (weniger als zwei Punkte oder keine Streuung der x -Werte) und *ungültige Daten* (undefinierter Logarithmus, nicht-endliche Werte) ausdrücklich vom Erfolgsfall — nie ein *Ok* mit NaN-Koeffizienten —, und der Ergebnis-Leser übernimmt nur streng numerische Zellen und zählt übersprungene Zeilen als Diagnose mit, statt sie still zu nicht belegten Punkten zu machen. Auf demselben Gerüst steht je Messkurve ein kubischer Spline über der Abszisse $\log_2 x$, wahlweise natürlich (zweimal stetig differenzierbar, aber überschwingfähig) oder als monotonie-erhaltender kubischer Hermite-Spline nach Fritsch und Carlson [FC80], der auf monotonen Daten keine Schein-Extrema und damit keine falschen Schnittpunkte erzeugt; ohne mindestens zwei Punkte mit x -Streuung entsteht kein Spline. Die Heuristik-Familie daneben (`heuristik/`) enthält den Achsen-Spline als Strategie-Naht mit übersetzungszeitlicher Verfahrenswahl, den Break-Even-Finder über den Schnittpunkten zweier Kurven derselben Achse, den Messkurven-Lader, die Offline-Clusterung der Workloads mit ihrem Merkmalsvektor und den Optimierungs-Katalog der Achsen; Break-Even-Kurven werden nur über die `w/ma/mi`-gemessenen Organ-Achsen-Parameter gebildet. Die Kurvenbildung selbst entwickelt Abschnitt 3.6.3; hier steht nur ihre Code-Gestalt.

Implementiert sind damit die Realm-Wurzeln samt Ordnung und Rückkehr-Sperren, die Registries, die Quellen-Abstraktion mit degradiertem Fallback, die zweistufige Hardware-Erkennung und der erste Fit-Ansatz mit Spline; die Anbindung dieser Querschnitts-Bausteine an Runner und Prüf-Dock (der Kollektor der Erfolgs-Parameter) und die Abbildung der Registry-Namen auf die Bestands-Spalten der Ergebnis-Dateien sind nicht implementiert. Die drei Konzept-Invarianten der System-Achsen lauten: Sie *permutieren* den Bau- und Mess-Raum — die Komplex-Achse `build_target_complex` ist der Command-Wrapper der Rekombination der drei Haupt-Achsen mit der Unter-Achsen-Gruppe `build_toolchain` und dem Scheduling mit fünf Dimensionen, und dieser System-Multiplikator vervielfacht den Organ-Raum je Plattform —; sie stehen nicht im Organ-Permutationsbaum der `binary_id`; und sie berühren die Binary-Identität nie (Registry-XML: `binary_id = never`; nur die achtzehn Organ-Haupt-Achsen gehen in den Achtzehn-Segment-Pfad ein). Das wahrt die Bauform selbst: Die

Wurzel dockt nur lesend an die host-seitigen Ergebnis-Strukturen an und bleibt der ABI-Grenze gegenüber unsichtbar.

4.5.1 Muster-Register, Hybrid-Stufe und Lager

Die Entwurfsmuster des Apparats sind kein Zufallsbestand, sondern ein Register (Abbildung 4.12); die Benennung folgt dem Katalog von Gamma et al. [GHJV94] („Gang of Four“, GoF), und jedes Muster ist hier übersetzungszeitlich statt laufzeit-polymorph ausgeführt, weil der Hot-Path keine Laufzeit-Verzweigung enthalten darf (Abschnitt 4.3). Das Muster jeder Achse ist *Strategy*: austauschbare Bausteine hinter einem Concept, über CRTP [Cop95] statt über eine virtuelle Basis gebunden. Die Plan-Emission übernehmen *Director* und *Builder*: ein Walk, je Ausgabe-Syntax ein Builder, dazu die Legenden-Formen als Formatierungs-Single-Source. Eine *Abstract Factory* konstruiert die Docks der Hybrid-Stufe an genau einem Ort und baut die Ergebnis-Mappen des Lagers (die `xlsx`-Mappe als Standard; der CSV-Baum — ein Ordner je Blatt, immer aus der Mappe erzeugt — als wählbare Strategie derselben Factory, statt der Mappe oder zusätzlich zu ihr; Umsetzungsstand: die CSV-Projektion ist als eine flache Datei implementiert, die Baumform ist vorgesehen, aber nicht implementiert). Die Mess-Naht `IMessVisitor` am `dlopen`-Übergang ist ein *Visitor*: Der Host reicht die Senke hinein, das Tier ruft die passende `visit`-Methode (doppelter Dispatch). Die Hardware-Erhebungskette ist eine *Chain of Responsibility* mit Typliste statt Handler-Zeigern, und ebenso die Auswahl-Filterkette der CEB, die Kandidaten mit den Verdikten `Pass`, `Reject` und `PassToNext` durchreicht. Ein *Memento* sichert und stellt den Zustand statusbehafteter Achsen je Tier wieder her (`memento_all` nach dem Warmlauf; als Plan die Zustands-Übernahme der Hybrid-Stufe, unten). Das Muster *Command* ist die übersetzungszeitliche Command-Basis der Achsen mit Mess-Visitor-Slot und Limitations-Auskunft — und die Komplex-Achse `build_target_complex` als Command-Wrapper der System-Rekombination. Eine *Facade* schließlich bündelt die Anhang-Erzeugung (Programm 08: eine Fassade über die Bibliotheken 04 und 05, in-process) und die rein lesende Plan-Emission des Planers.

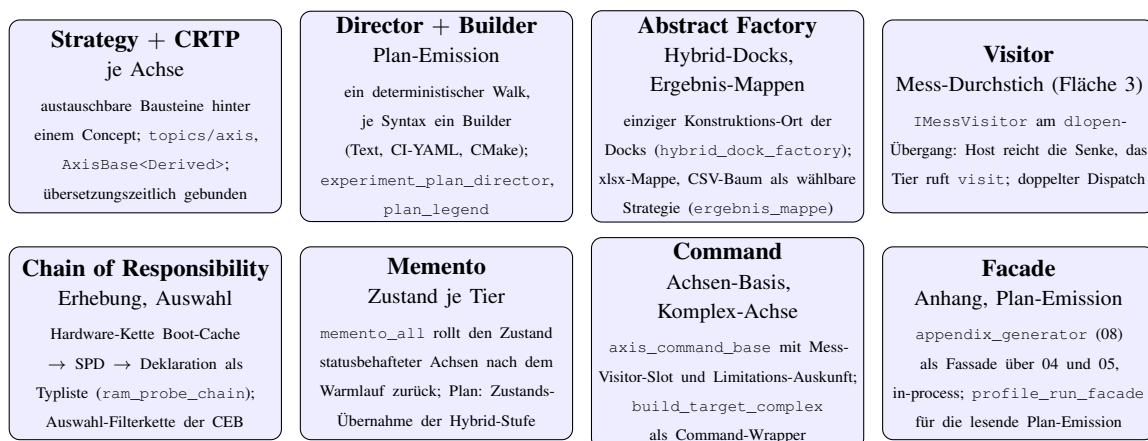


Abbildung 4.12: Das Register der Entwurfsmuster des Apparats (Benennung nach Gamma et al.): je Muster die Rolle und der Code-Ort. Alle acht sind übersetzungszeitlich ausgeführt — Bindung über Concepts, CRTP und Typlisten statt über virtuelle Aufrufe —, weil Tier-Binaries keine Laufzeit-Verzweigung zwischen Bausteinen enthalten dürfen.

Die Hybrid-Stufe ist der vierte Träger, als optionale Mux-Stufe zwischen CEB-Prüf-Dock und Tier-Binary eingeschoben (Abschnitt 3.6.2); ihr Code-Stand liegt unter `hybrid/` (Abbildung 4.13).

Implementiert sind die Dock-Verträge — vier Vertragsformen `standard`, `rollback`, `scan` und `resource_control` als `constexpr`-Registry mit sechzehn Status-Konstanten (`ok` und fünfzehn benannte Ablehnungsgründe) —, das minimale Prüf-Dock, die Dock-Factory als einziger Konstruktions-Ort, das Dock-Array mit `HybridDockVariant` (die eine erlaubte `variant`-Stelle, Abschnitt 4.3) und dem Laufzeit-Deckel `max_docks`, der das $(N + 1)$ -te Andocken mit „Array voll“ abweist, der Parser der `<hybrid_tier>`-Sektion (Zweiweg-Schalter `enabled`, exklusiv je Kette, `max_docks` Pflicht bei eingeschalteter Stufe) und der Binary-Proxy, der eine Hybrid-.so nach außen zu einer gewöhnlichen Tier-Binary macht: Er erfüllt `IAnatomyBase`, hält das Dock-Array und leitet den Antrieb an das Ziel durch, das an einem Dock steckt. Nicht implementiert sind der `.so`-Export der Hybrid-Stufe (er hängt an einer offenen Schichtungs-Auflage, weil eine Hybrid-.so Builder-Code linken müsste), die Eviction und der Router (er braucht tatsächliche Messkurven). Gemessen an der Dateiliste, die die README des Verzeichnisses `hybrid/` für die Dock-, Proxy- und Router-Schicht vorsieht, sind sechs der neun Dateien implementiert (Dock-Verträge, Prüf-Dock, Dock-Factory, Dock-Array, XML-Parser, Binary-Proxy) und drei nicht (das Tier-Modul des `.so`-Exports — die zwei Modulquellen unter `tests/unit` belegen nur die Baubarkeit des Modul-ABI —, die Eviction und der Router); daneben führt das Verzeichnis die vier Header der Klassifikations-Schicht (Klassifikation, Concept-Gate, Strategie je Gattung, Synthese-Matrix) und drei weitere Header für das Andocken, das Modul-ABI und die Stempel-Kette, insgesamt dreizehn Header. Die Hybrid Module ABI v1 exportiert dieselben Pflicht-Symbole wie die plain Tiere samt Konzept und Gattung, und `comdare_anatomy_genus` meldet den *Ziel*-Gattung (das, was das Modul für den Aufrufer ist), nie den *Reroute*-Gattung der Klassifikation — derselbe gattungsunabhängige Loader lädt die Hybrid-.so über die eine Ladestrecke; die Stempel-Kette der Hybrid-Stufe und die Hybrid-Positionen des Mess-Gate-Glieds (`hm/hmi`) führen ihre Identität, und die Gattungs-Bau-Zulassung führt sechs Einträge, den sechsten für die *Reroute*-Gattung mit dem Dock-Deckel (Default 32) als Arität. Das Hybrid-Live-Ranking ist als vierte Auswertungs-Komponente der `compare`-Stufe vorgesehen, und der Zustandsabgleich der Hybrid-Stufe ist per Memento mit zwei Release-Artefakten, `single` und `hybrid`, im Lager vorgesehen (Abschnitt 3.6.2); Umsetzungsstand: beides ist nicht implementiert, ebenso nicht die Behebung der Schema-Drift, dass das XSD die `<hybrid_tier>`-Sektion nicht führt. Der Hybrid-Modus kompiliert nur mithilfe der CEB.

Das Lager (Abschnitt 3.4.5) hat im Code ein übersetzungsstatisches Schlüssel-Schema (`bestand_schluesssel_schema`), das die vier Bestände deklariert, ohne selbst Lager-Mechanik zu implementieren: Bestand 1, die Binaries, ist über eine Schlüssel-Politik geschlüsselt, deren einziger SHA-512-Schlüssel mit dem Tier-Fingerprint identisch ist, die Zelle daneben geklammert; Bestand 2, die Messwerte, über den 128-Hex-Fingerprint der vermessenen Binary und die Hardware-Identität der messenden Maschine, die Zelle daneben geklammert und nie in den Digest fusioniert; Bestand 3, die Synthese-Batches, und Bestand 4, die Lösungen, sind als Arbeitsschritt des Lager-Vollausbaus deklariert (Tabelle 4.5). Drift zwischen Schema und Mechanik bricht beim Bau der Mechanik, nicht erst im Lager. Der Versions-Lock (`axis_version.lock`, Format v3, 719 Datensätze zu je drei Zeilen) bindet jede Quell-Datei der Achsen — die Heuristik-Header und den Overlay-Schnitt, über den das Overlay-Glied des Tier-Fingerprints hasht — an ihren Inhalts-Digest und ihre gestempelte Algorithmus-Version: Eine Abweichung zwischen Lock und Discovery ist ein Fehlschlag, eine Rückstufung eines versionstragenden Headers auf Digest-only ist ein Fehlschlag ohne Schreibweg, eine Aufwertung ist legitim und verlangt die bewusste Neuerzeugung des Locks; das Gate `contract:axis-version-lock` der CI erzwingt das. Aus dem Schema

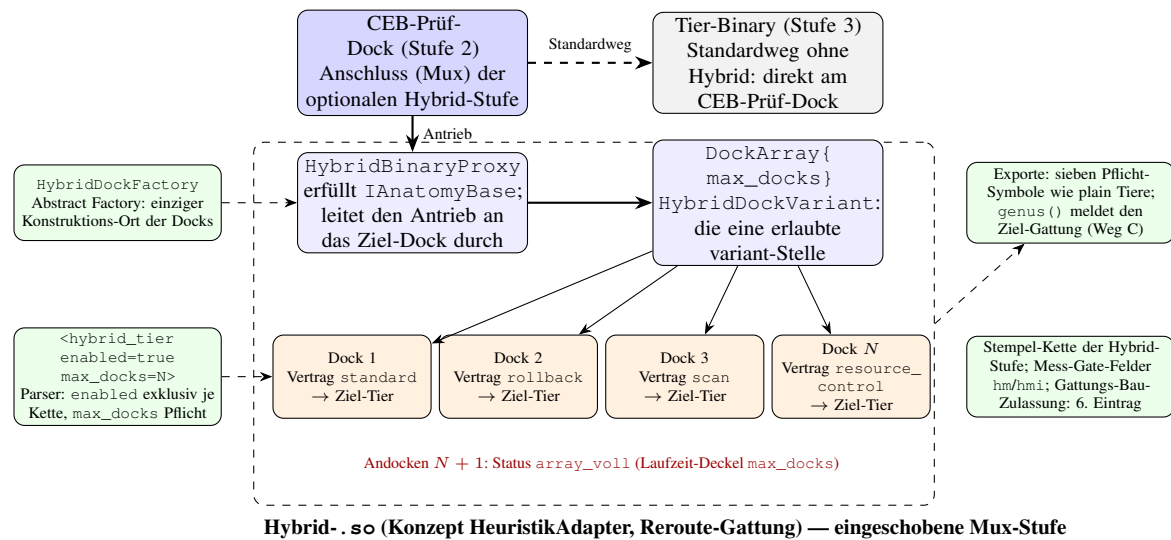


Abbildung 4.13: Die Hybrid-Stufe im Code: die optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Mux-Stufe, am CEB-Prüf-Dock angesetzt und mit Dock-Slots für Tier-Binaries (Standardweg ohne Hybrid: Prüf-Dock direkt zur Tier-Binary). Der Binary-Proxy macht die Hybrid-.so nach außen zu einer gewöhnlichen Tier-Binary und hält das Dock-Array mit dem Laufzeit-Deckel `max_docks`; je Dock gilt einer von vier Verträgen, die Dock-Factory ist der einzige Konstruktions-Ort, die `<hybrid_tier>`-Sektion der XML schaltet die Stufe und setzt den Deckel. Die Exporte sind buchstabengleich zu den plain Tieren, `genus()` meldet den Ziel-Gattung. Nicht implementiert: `.so`-Export, Eviction, Router.

folgen die Lager-Eigenschaften des Konzepts: Die Messung ist immer eingeschaltet, im Lager wird nur nachgemessen, was dort fehlt; ein Versionswechsel einer Achse ändert Stempel und Fingerprint und baut den betroffenen Lager-Stapel neu; und weil die Hardware-Identität im Messwert-Schlüssel steht, hat jeder Maschinentyp seinen eigenen Messwert-Bestand. Umsetzungsstand: Schema, Lock und die Bestände 1 und 2 sind implementiert; der Vollausbau — Planer-Blattfunktion, CEB-Vollplatzierung, Hybrid-Ablage, Commit-Skip, Neubau-Flag, XML-Zielwahl — und die Binaries mit und ohne Messfühler samt Kurven-Dateien als Lager-Einträge sind vorgesehen, aber nicht implementiert.

Bestand	Schlüssel-Komponenten	Quelle im Code	Umsetzungsstand
1 Binaries	der eine SHA-512-Schlüssel (= Tier-Fingerprint); Zell-Klammer daneben	Schlüssel-Politik des Bestandslogs	implementiert
2 Messwerte	128-Hex-Fingerprint der vermessenen Binary; Hardware-Identität der messenden Maschine; Zelle daneben geklammert, nie fusioniert	Messwert-Schlüsselquelle des Bestandslogs	implementiert
3 Synthese-Batches	Maschine; Voll-Stempel; Referenz auf Mess-Ebene und Kanal	Schema deklariert	nicht implementiert (Vollausbau)
4 Lösungen	kanonisierter Hash der Anwender-XML; Maschine; Referenzen auf die beteiligten Bestände	Schema deklariert	nicht implementiert (Vollausbau)

Tabelle 4.5: Die vier Lager-Bestände und ihre Schlüssel-Komponenten nach dem übersetzungsstatistischen Schlüssel-Schema; der Versions-Lock bindet die Achsen-Quellen an Digest und gestempelte Version.

Damit ist der Apparat vollständig beschrieben — von den vier Repositorien über den Prüfling, die achtzehn Organ-Achsen und die Träger-Stufenkette bis zur Kette vom Plan zur Tier-Binary, ihren Vertrags-

Flächen, der schichtweisen Qualitätssicherung, den Realm-Wurzeln im Code, dem Muster-Register, der Hybrid-Stufe und dem Lager. Kapitel 5 legt die Methodik offen, mit der dieser Apparat bewertet wird.

5 Evaluation

Die Evaluation legt zuerst die Methodik offen und wertet darauf aufbauend die Messreihen aus. Ihre Leitfrage ist die Cacheline-Awareness: Welche Entwurfsentscheidung verändert die cache-nahen Messgrößen — L1-, L2- und Last-Level-Misses, dTLB-Misses, Branch-Misses und die Latenz je Gattungs-Interface-Aufruf — auf welcher Plattform in welchem Maß? Drei Evaluationsobjekte ordnen das Kapitel: der *Entwurfsraum* (die Hypothesen H1–H3 und die Achsen-Sensitivitätsanalyse), der *Apparat* selbst (der heuristische Gegenbeweis und der Messfehler, den die Mess-Instanzen eintragen) und die *Automatisierung* der Evaluation aus der Experiment-XML, denn die Arbeit automatisiert im Kern genau diese Evaluation. Der Methodik-Teil (Abschnitt 5.1) entfaltet in acht Unterabschnitten Hypothesen und Messgrößen, die XML-getriebene Kette, Workloads und Datensätze, Plattformen und Regimes, Mess-Ebenen und Zähler, Bau-Umfang, Fairness sowie den Gegenbeweis; der Ergebnis-Teil (Abschnitt 5.2) führt in sieben Unterabschnitten die Identität der Ergebnisse, den Datenfluss, die Messreihen, die vorliegenden Messungen, die Sensitivitätsanalyse, die Gültigkeitsprüfung und die Zusammenfassung.

Die Evaluation folgt der in der empirischen Software- und Systemforschung üblichen Kette aus Forschungsfrage, Hypothese, Messgröße, Versuchsaufbau, Durchführung, Ergebnis, Gültigkeitsprüfung und Diskussion [WRH⁺12, Sha03]: Jede Hypothese benennt ihre Messgröße und ihr Annahmekriterium vor der Messung, jeder Versuchsaufbau seine Plattform, sein Regime und seine Wiederholungen, und jedes Ergebnis seine Unsicherheit [GBE07, KJ13]. Weil der Gegenstand ein Artefakt ist — der Mess-Apparat mit seinen Achsen —, tritt die Bewertung des Artefakts nach den Regeln der Design-Science-Forschung¹ hinzu: Nützlichkeit, Strenge und Nachvollziehbarkeit [HMPR04, PTRC07, Wie14]. Leistungsmessungen werden nach den Berichts-Regeln von Hoeffler und Belli [HB15] und Blackburn et al. [BDH⁺16] ausgewiesen: Bezugsgröße, Wiederholungen, Streuung, Plattform und nicht erhobene Werte (n/a) stehen an jedem Ergebnis. Die drei Evaluationsobjekte ordnen die Unterabschnitte; die Cacheline-Awareness ist ihre gemeinsame Leitachse.

Die Leitachse der Evaluation ist die Cacheline-Awareness: Verglichen werden die cache-nahen Messgrößen — L1-, L2- und Last-Level-Misses, dTLB-Misses, Branch-Misses, Energie sowie die Latenz-Perzentile je Gattungs-Interface-Funktion; sie sind sechzehn registrierte Mess-Kategorien mit einem Regime-Ordinal je Kategorie — je Achsen-Belegung, je Konzept- und Gattungs-Interface-Aufruf (die Kanal-Adresse lautet stets Achse → Gattung → Kategorie) und je Plattform, also je registrierter Maschinen-Signatur: prod1 (AMD Zen 5), prod2 (Intel Core i9-12900K, Alder Lake) und das ODROID-Board mit Intel-Gracemont-Kernen. Die Frage lautet nicht, welcher Algorithmus schneller ist, sondern welche Achse die cacheline-bezogenen Kosten auf welcher Mikroarchitektur bewegt. Die PMC-Kategorien (Performance Monitoring Counter, die Hardware-Zähler der Prozessoren) sind dafür die primären Messgrößen, die Zeit-Kategorien die Gegenprobe; weil Hardware-Zähler nur nach einer Gegenprüfung durch Messung vertrauenswürdig sind [WM08], legt Abschnitt 5.1.5 die Semantik jedes Zählers je Modell offen.

¹Design Science: die Forschungsrichtung der Informationssysteme, die ein konstruiertes Artefakt (hier den Mess-Apparat) selbst zum Gegenstand der Bewertung macht und dafür Nützlichkeit, Strenge des Vorgehens und Nachvollziehbarkeit verlangt [HMPR04].

5.1 Methodik

5.1.1 Hypothesen und Messgrößen

Drei *Hypothesen* leiten die Auswertung; jede nennt ihre Messgröße und ihr Annahmekriterium vor der Messung:

H1 (Page-Type-Kosten) Die mittleren Zugriffskosten je Page-Type-Variante unterscheiden sich systematisch nach Workload: dichte Page-Typen gewinnen erwartungsgemäß unter Zipf-Verteilung (YCSB-A), bereichsoptimierte Pages unter Bereichs-Scans (YCSB-E). Messgrößen sind die Latenz-Perzentile und die Cache-Miss-Kategorien je Workload; H1 gilt als gestützt, wenn die Rangfolge der Page-Type-Varianten zwischen YCSB-A und YCSB-E auf derselben Plattform kippt und der Abstand das Unsicherheitsband der Bau-Varianten (Abschnitt 5.1.8) übersteigt.

H2 (Code-Qualität pro Quelle) Der vom Original-Quell-Repository geerbte Code-Qualitäts-Score korreliert messbar mit dem erreichten Durchsatz. Zwei Instrumente liefern den Score: erstens das dokumentierte Hand-Audit je Quell-Repository über sieben Bewertungs-Achsen (Stil, Tests, Dokumentation, Wartbarkeit, Pflege-Status, Lizenz, Build-System; Skala 1–5) für 22 bewertete Quellen — eine Akte unter den Audits des Cache-Engine-Repositories —, zweitens das tool-berechnete Instrument, das tatsächlich in die Mess-Zeile eingeht: die gewichtete `cppcheck`-Befunddichte je tausend physischer Zeilen (Gewichte `error` 3, `warning` 2, `performance` 1, `portability` 1, `style` 0,5) über die vendorten Original-Quellen; die Akte führt 33 Einträge, davon 22 als `n/a`, weil die Rekonstruktion keine vendorte Original-Quelle besitzt, und der Wert reist als Spalte `h2_code_quality_score` des WIDE-Schemas (im Code als Feld `h2_score` geführt, Akte `sota_h2_scores.xml`) in jeder Mess-Zeile einer SOTA-Reihe mit. Ergänzt wird beides um die dokumentierte `is_original`-Markierung je Funktion (Anhang D). Annahmekriterium: H2 gilt als gestützt, wenn der tool-berechnete Score über die belegten SOTA-Profilen mit dem Durchsatz in derselben Zelle (Plattform, Workload, Regime) einen Rangkorrelations-Koeffizienten jenseits einer vor der Messung genannten Schwelle zeigt; `n/a`-Profile gehen nicht in die Korrelation ein, und das Hand-Audit ist die qualitative Einordnung.

H3 (Inline- vs. External- vs. Chain-Verteilung) Die beobachtete Verteilung der drei ValueHandle-Varianten korreliert mit der Page-Dichte: dichte Pages bevorzugen Inline, spärliche Pages External und PRT-ART-Heuristiken ChainRef. Die Registry der Achse `value_handle` führt fünf Bausteine (Inline, External-Pool, Immutable-Shared-Ref, Versioned-Pointer, Chain-Ref; nur Inline ist an den golden-Referenzkatalog gebunden); H3 betrachtet die drei für PRT-ART maßgeblichen Varianten als begründeten Ausschnitt. Messgröße ist der Anteil je Variante über der Page-Dichte; H3 gilt als gestützt, wenn der Anteil monoton mit der Dichte wandert.

Diese *Evaluations-Hypothesen* sind nicht zu verwechseln mit den gleichnamigen PRT-ART-Telemetrie-Metriken H1–H3 (Abschnitt 3.5): Erstere sind prüfbare Aussagen über den Entwurfsraum, Letztere Mess-Organen des Prüflings, die unter anderem die Daten für diese Prüfung liefern.

5.1.2 Vollautomatische Experiment-Evaluation aus der XML

Die Evaluation ist selbst Gegenstand der Automatisierung (Abbildung 5.1): Eine Anwender-XML mit der Wurzel `<comdare_experiment>` deklariert, *was* gemessen wird, *wo* und *wann* — nie das *Wie*. Für die Evaluation sind die Sektionen `workloads`, `datasets`, `measurement_tooling`, `measurement_categories`, `run_methodology`, `writeback_methods` und `output` maßgeblich; der Profil-Validator prüft sie vor jedem Lauf gegen die Registries (Achsen-Namen und -Werte, Verbund-Bindungen, Fairness-Modi, Akten). Aus der XML entsteht deterministisch der Plan — der Director-Walk des Planers, der als reine Text-Emission auch ohne Bau abrufbar ist —, daraus die Bau-Jobs des CacheEngineBuilders und die Tier-Binaries, deren Messung in der Mappe (xlsx) abgelegt wird; die CSV-Form ist der aus der Mappe erzeugte, geordnete Baum aus CSV-Dateien (ein Ordner je Blatt, darin die Dateien der einzelnen Unter-Achsen-Fahrten; kein einzelnes Kind-Dokument), und die Generatoren des Werkzeug-Repositories (Binary-zu-CSV, CSV-zu-LaTeX, Diagramm-Generator, Anhang-Generator) rendern Steckbriefe, Tabellen und TikZ-Diagramme je Sprache in ein `tabellen`-Verzeichnis, das das Manuskript einbindet. Kein Schritt dieser Kette wird von Hand ausgeführt; jede Zahl im Anhang führt den Lauf, der sie erzeugt hat (Anhang A). Umsetzungsstand: Die Kette ist in einem ersten durchgehenden Lauf nachgewiesen — die Smoke-Messreihe über 43 Permutationen im Anhang ist ihr Erzeugnis —; die XML-Sektion `<publish>` für den Lager-Rückschrieb, der kommandozeilengeführte Erstellungsmodus der XML (Abschnitt 4.3.2) und die dritte Ausgabe-Factory LaTeX (Abschnitt 5.2.2) sind vorgesehen, aber nicht implementiert.

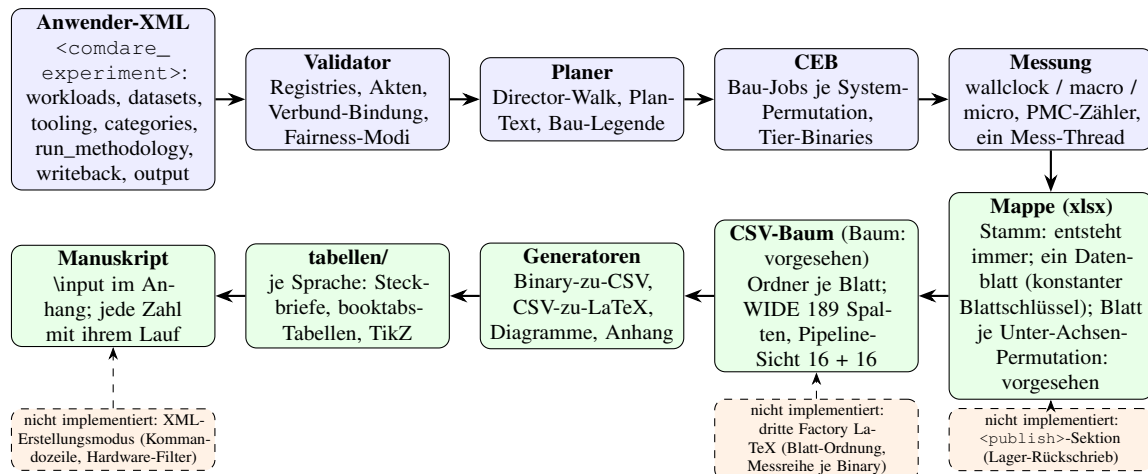


Abbildung 5.1: Die automatisierte Evaluations-Kette von der Anwender-XML bis zum Manuskript. Obere Reihe: Die XML deklariert Was, Wo und Wann; der Validator prüft sie gegen die Registries; der Planer emittiert den deterministischen Plan; die CEB baut je System-Permutation die Tier-Binaries; die Messung erhebt die drei Mess-Ebenen und die Zähler mit einem Mess-Thread. Untere Reihe: Die Mappe ist der Stamm (ein Datenblatt unter konstantem Blattschlüssel; die Auffächerung in ein Blatt je Unter-Achsen-Permutation ist vorgesehen, aber nicht implementiert), die CSV-Form ihr aus ihr erzeugter, geordneter Baum aus CSV-Dateien (ein Ordner je Blatt; kein einzelnes Kind-Dokument; implementiert ist eine flache Datei), die Generatoren rendern je Sprache in das `tabellen`-Verzeichnis, das Manuskript bindet die Dateien ein — jede Zahl führt ihren Lauf. Gestrichelt: nicht implementierte Teile (XML-Erstellungsmodus, `<publish>`-Sektion, dritte Factory).

5.1.3 Workloads und Datensätze

Phase 5 der Mess-Pipeline (Execute, Abschnitt 4.3.2) leitet den YCSB-Workload je Permutation als Fallback-Heuristik aus dem Traversal-Tag des Profils ab (Tabelle 5.1), sofern kein `<expected_workload>`-Tag des Profils die Heuristik überschreibt (Abschnitt 3.2); das Routing matcht dabei per Teilstring (RANGE, HOT_PATH) und nicht über eine geschlossene Tag-Liste, und der Wert des `<expected_workload>`-Tags wird gegen die sechs Token YCSB_A bis YCSB_F aufgelöst. Der globale Default-Workload ist die Permutation über alle verfügbaren Workloads; die Laufzeit-Dimension `workload` ist dafür als dynamische Dimension der Mess-Registry registriert, gespeist aus dem Last-Rahmenwerk YCSB [CST⁺10]. Umsetzungsstand: Der Treiber fällt für profil-lose Module auf genau einen Workload zurück, die Workload-Option des Laufs; die Voll-Permutation ist vorgesehen, aber nicht implementiert. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik (Tabelle 5.2), jeder mit einer Reproduzierbarkeits-Akte aus Quelle, Prüfsumme, Zeilenzahl, Vorverarbeitung und Seed-Regel für die synthetischen Miss-, Update- und Delete-Schlüssel; vier der sechs Datensätze (`url`, `protein`, `tpcds-id`, `trec-terms`) besitzen eine gegen die vorverarbeiteten Dateien tatsächlich berechnete Akte, `dna` die Akte der Pizza&Chili-Quelle, und `xml` besitzt eine Spezifikation ohne Prüfsumme, weil die Datei nicht lokal vorliegt — eine synthetische Prüfsumme gibt es nicht (n/a). Die Akten liegen als XML-Dateien je Datensatz im Werkzeug-Repository, und ein Kanon-Test prüft die berechneten Akten gegen den Sechser-Kanon der Tabelle.

Tabelle 5.1: Profil-bewusstes Workload-Routing (Fallback-Heuristik)

Traversal-Tag	YCSB-Workload	Charakteristik
RANGE_SCAN / RANGE_HOT_PATH	YCSB-E	Bereichsabfragen
HOT_PATH / PARTIAL_HOT_PATH	YCSB-A	50/50 Lesen/Aktualisieren, Zipf
STANDARD (Default)	YCSB-C	nur Lesen

Tabelle 5.2: Reale Datensätze, ihre Charakteristik und ihre Reproduzierbarkeits-Akte

Datensatz	Charakteristik	Akte
<code>url</code>	sehr lange Schlüssel, lange gemeinsame Präfixe, mittleres Alphabet	real
<code>dna</code>	kleines Alphabet, hohe Wiederholung, tiefe Präfix-Entropie	Pizza&Chili-Akte
<code>protein</code>	biologische Sequenzen, kleines bis mittleres Alphabet	real
<code>xml</code>	strukturierte Strings mit wiederkehrenden Präfixen	n/a (Datei nicht lokal)
<code>tpcds-id</code>	datenbanknahe IDs, sortiert und unsortiert	real
<code>trec-terms</code>	Wörterbuch-/Suchindex-Terme mit definierten Miss-Schlüsseln	real

5.1.4 Plattformen, Hersteller-Pfade und Regimes

Die Messungen zielen auf die registrierten Messmaschinen des Apparats: `prod1`, die AMD-Maschine mit einem Ryzen 9 9950X3D (Zen 5; Maschinen-Signatur `prod1_zen5` mit 22 Merkmalen inklusive AVX-512), zugleich die Entwicklungs-Workstation der Verifikation; `prod2`, die Intel-Maschine mit einem Core i9-12900K (Alder Lake; Signatur `prod2_alder_lake` mit neun Merkmalen, ohne AVX-512, mit P- und E-Kernen); und ein ODDROID-Board mit Intel-Gracemont-Kernen (Signatur `odroid_gracemont`, neun Merkmale). Der ZIH-Cluster der TU Dresden ist als Unter-Achse IS3 der Ziel-ISA deklariert

(Barnard, Capella, GraceHopper, N/A); der Barnard-Knoten besitzt Intel-Sapphire-Rapids-CPU's, zwei Xeon Platinum 8470 je Knoten [Zen26a], und Capella ist der GPU-Cluster mit AMD-Genoa-CPU's und NVIDIA H100. In keiner Maschinen-Signatur ist ein ZIH-Knoten registriert, und ein Lauf dort ist nicht gemessen (n/a) — gleichlautend mit Abschnitt 3.4. Eine HBM-Plattform ist in keiner Signatur und in keiner Achse als Plattform registriert; die Plattform-Sonde führt ein HBM-Merkmal und die Allokator-Achse kennt PIM-Sonderfälle, mehr nicht — sie ist vorgesehen, aber nicht registriert. Eine `IPlatformProbe` (Phase 1) meldet je Plattform die lauffähige Teilmenge der Achsen (etwa AVX-512 nur dort, wo verfügbar), wobei der Ausschluss einer nicht verfügbaren Erweiterung stets *maschinenseitig* fällt — mit Warnung und benannter Fehlerklasse, nie als stiller Profil-Filter, denn das Experiment-Profil bleibt für alle Maschinen gleich; je Plattform werden ein Scalar- und ein AVX2-Build vorbereitet, ein AVX-512-Build nur dort, wo die Signatur das Merkmal führt. Die verfügbaren Achsen-Optionen sind je Plattform über die Planer-Sonde prüfbar; ein kommandozeilengeführter XML-Erstellungsmodus mit Plattform-Filter ist vorgesehen, aber nicht implementiert (Abschnitt 4.3.2). Auf Hybrid-CPU's werden P- und E-Kerne getrennt vermessen statt zusammenaggregiert, auf Intel wie auf AMD; die Messmechanik dafür sind die Core-PMU-Domänen des Kernels (auf Intel die sysfs-Namen `cpu_core` und `cpu_atom`), während die Benennung der Unter-Achse vendorneutral bleibt — die sysfs-Namen stehen ausdrücklich auf der Verbotsliste der ISA-Unter-Achsen-Etiketten. Die Kern-Modi folgen dem Sechs-Modus-Modell aus Abschnitt 3.4.1: vier Pinning-Modi je Kern-Sorte (P- oder E-Kerne, einzelner Kern oder alle Kerne der Sorte), der ungepinnte Referenzlauf und *HybridAware* als sechster Modus (Scheduling-Dimension `hetero_core_dispatch`), der für die Cache-Line-Awareness-Messung verpflichtend ist; die Anbindung des Kern-Pinnings und der Kern-Modi an das Lauf-Profil ist vorgesehen, aber nicht implementiert (Abschnitt 4.5).

Der produktive Mess-Betrieb ist ein *Zwei-Maschinen-Betrieb* — die AMD-Maschine `prod1` neben der Intel-Maschine `prod2` —, und die zählerbasierte Erhebung führt dafür zwei je einzeln aktivierbare Hersteller-Pfade, `pmc:amd` auf `prod1` und `pmc:intel` auf `prod2`, mit je eigener PMU-Ressourcengruppe; die Pfade laufen absichtlich parallel, weil verschiedene Hosts verschiedene PMUs besitzen und eine gemeinsame Gruppe sie grundlos serialisieren würde. Die Vollständigkeit ist eine Auswertungs-Regel, kein CI-Gate: Eine Messreihe gilt erst als vollständig, wenn beide Pfade ihre Zähler-Voraussetzungen nachgewiesen und geliefert haben; ein einseitig gefahrener Pfad wird als Teil-Messung ausgewiesen, nie still als Ganzes geführt (Abschnitt 5.2.6). Der Messplan sieht auf den Produktions-Zielmaschinen zudem für jede Messung *zwei Betriebssystem-Regimes* vor, um Produktions-Treue von Mess-Tiefe zu trennen: Ein *immutable* Betriebssystem (Talos) spiegelt den Produktiv-Einsatz und erzwingt boot-/übersetzungsstatische Cache-Einstellungen (Abschnitt 2.2); ein *root-Linux mit vollem Hardware-Zähler-Zugriff* (`perf` und die modellspezifischen Register, MSR) eröffnet hingegen überhaupt erst den Zugang zu den zählerbasierten Mess-Kategorien (Abschnitt 3.2) und zu den laufzeit-konfigurierbaren cache-nahen Achsen-Parametern (Prefetch-Distanz, Hardware-Prefetcher). Das zweite Regime ist *entworfen, nicht umgesetzt*: Die berichteten Messungen stammen sämtlich vom root-Linux-Regime der beiden Produktionsmaschinen; das Talos-Regime ist vorgesehen, aber nicht gemessen.

5.1.5 Mess-Ebenen, Kollektoren und Zähler

Davon zu trennen ist, welche Zähler dieser Zugang tatsächlich füllt (Tabelle 5.3): Die Mess-Kette öffnet über `perf_event_open` [Lin26b, Mel10] vier generische Zähler — L1D-Misses, Last-Level-Misses,

dTLB-Misses und Branch-Misses (`PERF_COUNT_HW_BRANCH_MISSES`, portabel auf Intel und AMD). Für L2-Misses und Kohärenz-Invalidierungen gibt es keinen portablen generischen Zähler; beide werden modell-gebunden über einen RAW-Event-Katalog erhoben, dessen Kodierungen gegengeprüft sind: Für Zen 5² ist `l2_cache_req_stat.ic_dc_miss_in_l2` das Event 0x64 mit Maske 0x09 (Demand-Misses in L2 ohne L2-Prefetch) und `ls_dmnd_fills_from_sys.remote_cache` das Event 0x43 mit Maske 0x14 (Demand-Fills aus dem Cache eines anderen CCX, eines Kern-Komplexes mit eigenem L3); ein wörtlicher Invalidierungs-Zähler existiert im Core-Event-Satz nicht, die Spalte führt genau diese Belegung. Die Kodierungen stammen aus der Eventtabelle des Linux-Kernels für Zen 5 [Lin25], aus der `perf` die benannten Events bindet. Modelle ohne Katalog-Eintrag führen n/a, nie eine geratene Kodierung; für Intel liegt kein Eintrag vor, weil die Gegenprüfung auf einem Intel-Host nicht vorliegt. Das Last-Level-Event scheitert auf Zen 5 beim Öffnen (`ENOENT`). IPC/CPI ist als Kategorie deklariert, aber nicht angebunden — dem Zähler-Verbund fehlen die Quellen für Instruktionen und Zyklen —, und die Energie-Aufnahme wird best-effort aus der RAPL-Sysfs-Zone³ gelesen und bleibt ohne Zonen-Leserecht leer. Der Grundsatz der ausgewiesenen Nichterhebung ist als Feld-Flag realisiert: Je Zähler führt der Zähler-Verbund ein `*_source_available`-Flag, sieben Flags für sieben Zähler; eine Zahl bedeutet, dass die Quelle geliefert hat — auch eine gemessene Null —, n/a bedeutet, dass die Quelle fehlt, und diese Unterscheidung reist Ende-zu-Ende bis in Mappe, Tabellen und Diagramme. Hardware-Zähler gelten dabei nur nach Gegenprüfung als vertrauenswürdig [WM08]; die Gegenprüfung für Zen 5 ist durch Messung belegt, die für Intel liegt nicht vor.

Zeit- und Beobachter-Messung sind im Mess-Checkpoint (`checkpoint_measure`) vereint; ihre Verantwortlichkeiten trennt die Mess-Tooling-Haupt-Achse in genau dieser Reihenfolge: *wallclock* (w) — die Wall-Clock der Gesamtläufe im CacheEngineBuilder über die Last-Rahmenwerke, parameter-leer; *macro* (ma) — die Zeit der Gattungs-Interface-Funktionen der Tier-Binary; *micro* (mi) — die Zeit und die Erfolgs-Parameter je Achsen-Interface (Abschnitt 3.4, Abbildung 3.9); Teilmengen der drei Ebenen sind erlaubt, jede andere Reihenfolge ist syntaktisch falsch, und w/ma/mi ist der registrierte Alias. Drei Kollektoren übernehmen die Erhebung als Bausteine der Mess-Achse `collector` — Wall-Clock, Beobachter-Schnappschuss und PMC —, und die sechzehn Mess-Kategorien sind die Unter-Achse der Tooling-Achse, nicht ihre Haupt-Auffächerung. Die Auswertung hält die drei Ebenen getrennt: Achsen-Interface, Konzept- und Gattungs-Interface und die Wall-Clock des CacheEngineBuilders werden nie gemischt (Abschnitt 5.2.5); PMC-Messungen als eigener Abbildungstyp des Anhang-Generators sind vorgesehen, aber nicht implementiert. Umsetzungsstand: Registry und Ebenen-Kanon sind implementiert; die Ordnungs-Validierung an den drei Serialisierungs-Stellen und der Kollektor als eigenständiges Bauteil sind es nicht.

Die zeit- und beobachter-basierten Kategorien laufen unter beiden Regimes identisch und sichern die Vergleichbarkeit; die PMC-Kategorien werden nur unter dem privilegierten Regime erhoben. Je Kategorie führt die Registry das Regime als Ordinal (0 für Zeit und Beobachter, 1 für Zähler): neun Zeit-/Beobachter-Kategorien und sieben Zähler-Kategorien. Betriebssystem, Kernel und Zähler-Domäne sind je Plattform zu protokollieren: Die Betriebssystem-Sonde erhebt Kernel und Kennung des Systems als System-Achse, und die Zähler-Quelle hält je Zähler fest, auf welcher Core-PMU-Domäne er geöffnet hat (generisch oder,

²Zen 5: die AMD-Mikroarchitektur des Ryzen 9 9950X3D (Granite Ridge, CUID-Familie 26); prod1 besitzt zwei ungleich große L3-Komplexe.

³RAPL (Running Average Power Limit): die Energie-Zähler der Prozessoren, die das Betriebssystem über die Powercap-Schnittstelle in Mikrojoule ausliest [DGH⁺10, KHN⁺18]; ohne Leserecht auf die Zone bleibt der Wert leer.

Kategorie (Spalte)	Quelle	Intel-Stand	Gültigkeits-Flag
cache_misses_l1	generisches perf-Event (L1D-Read-Miss)	generisch: ja	eigenes Flag
cache_misses_l2	RAW-Katalog Zen 5: Event 0x64, Maske 0x09 (Demand-Misses in L2)	kein Katalog-Eintrag, Gegenprüfung nicht durchgeführt (n/a)	eigenes Flag
cache_misses_l3	generisches perf-Event (Last-Level); scheitert auf Zen 5 beim Öffnen (n/a)	generisch: ja	eigenes Flag
dtlb_misses	generisches perf-Event (dTLB-Read-Miss)	generisch: ja	eigenes Flag
branch_misses	generisches perf-Event (PERF_COUNT_HW_BRANCH_MISSES)	generisch: ja	eigenes Flag
coherence_invalidations	RAW-Katalog Zen 5: Event 0x43, Maske 0x14 (Fremd-CCX-Fills)	kein Katalog-Eintrag, Gegenprüfung nicht durchgeführt (n/a)	eigenes Flag
energy_micro_joules	RAPL-Sysfs-Zone, best-effort (Leserecht)	wie AMD (Leserecht)	eigenes Flag
IPC/CPI	deklariert, nicht angebunden (Quelle fehlt)	n/a	—

Tabelle 5.3: Die Zähler der Mess-Kette und ihre Quellen: vier generische perf-Events (portabel), zwei modell-gebundene RAW-Events aus dem Katalog für Zen 5 (durch Messung gegengeprüft), die RAPL-Energie als best-effort-Lesung und IPC/CPI als deklarierte, nicht angebundene Kategorie. Je Zähler führt der Zähler-Verbund ein `*_source_available`-Flag; eine Zahl heißt „Quelle hat geliefert“, n/a heißt „Quelle fehlt“. Der Intel-Stand nennt, was ohne Katalog-Eintrag auf `prod2` erhoben wird.

auf Hybrid-Maschinen, der `sysfs`-Name der Domäne); eine eigene Spalte der Mess-Ausgabe besitzen diese drei Angaben im WIDE-Schema nicht — die Spalten `platform` und `build_version` führen Plattform-Tag und Bau-Version je Zeile, die Protokoll-Spalte je Domäne ist nicht implementiert.

5.1.6 Bau-Umfang und Optimierungsstufen

Der Umfang des vollständigen Baus ist im Profil festgelegt. Das Profil deklariert die Bau-Steuergrößen: zwei Optimierungsstufen (O2, O3) mal zwei SIMD-Stufen (keine Erweiterung, AVX2), also vier Bau-Varianten je Permutation (Abschnitt 3.4.1). Beide Stufen sind in der System-Registry als IEEE-754-deterministisch deklariert; die Registry führt fünf Stufen (O0 bis O3 und Ofast), und ausgeschlossen aus dem Profil ist allein Ofast, weil es den Gleitkomma-Determinismus nach IEEE 754 und den Run-to-Run-Determinismus des Apparats sowie die CRC-64-Prüfsumme des golden-Referenzkatalogs bräche — die Registry markiert genau diese Stufe als nicht deterministisch. Die maximale Optimierung bleibt damit wählbar: Der Standard des Baus ist global O2, O3 wird über die XML gewählt und mitgebaut, und die gewählte Stufe ist über das Werkzeugketten-Glied des Fingerprints identitätswirksam, sodass O2- und O3-Bestände nie stillschweigend als Cache-Treffer verwechselt werden; O0 bis O3 als Anwender-Wahl in der XML und die Durchreichung weiterer Compiler-Flags sind vorgesehen, aber nicht implementiert. Je Bau-Variante wird der golden-Referenzkatalog gebaut: $2^{17} = 131072$ Binary-Identitäten, weil das golden-Profil für siebzehn Organ-Haupt-Achsen je zwei Vertreter aus dem Angebot der Organ-Registry aktiviert und die achtzehnte (`persistence_target`) festgelegt ist (Abschnitt 3.4.6) — eine vereinfachende, überholte Annahme, der Katalog ist Regressions-Detektor, nicht Bau-Umfang; die System-Achsen multiplizieren die Bau-Matrix, nie die `binary_id`. Die Obergrenze der Binary-Identitäten je System-Permutation

ist das Produkt der je Organ-Achse im XML-Experiment gewählten Auslegungen; der Planer bestimmt sie aus dem XML-Experiment, sie ist erst zur Ausführung bekannt, und die Mächtigkeit des realen Permutationsraums rechnet er dynamisch aus seinen Eingängen (Subkommando `simulate`); sie ist keine feste Anzahl. Ergebnisse der Läufe des vollständigen Baus liegen nicht vor (Abschnitt 5.2.4).

5.1.7 Fairness und Reproduzierbarkeit

Faire Vergleichbarkeit ist über feste Regeln gesichert. Jeder Vergleich trennt einen gemeinsamen Minimalmodus (Common-Denominator: externe Werte-Handles, keine PRT-ART-Spezialpfade) vom PRT-ART-Native-Modus (Inline-Umschaltung und adaptive Seitentyp-Wahl je Verzweigungspunkt); die Anwender-XML deklariert den Modus je SOTA-Reihe als `fairness_mode` mit den zwei erlaubten Werten `common_denominator` und `native`, und der Validator weist jeden anderen Wert ab. Umsetzungsstand: Der Modus ist Reihen-Metadatum ohne Mess-Einfluss — er reist als Tag-Spalte der Mess-Zeile und im Wiederaufnahme-Stempel, nie in der `binary_id` —, die Kompositions-Festlegung des Minimalmodus ist daten-gebunden, und zwei Reihen desselben Prüflings mit verschiedenem Modus erzeugen deshalb dieselbe `binary_id`, was der Validator als Warnung meldet. Alle Verfahren erhalten identische, normalisierte Byte-Schlüssel sowie identische Miss-, Update- und Delete-Mengen; der Warmup ist der Zwei-Phasen-Treiber des Lastprofil-Orchestrators (sichern, aufwärmen, zurückrollen, messen), sonst wird kalt gemessen, und jede Konfiguration wird in mehreren getrennten Läufen vermessen (Wiederholungs-Dimension `repetition`). Thread-Pinning: Der Aktuator — ein bereichsgebundenes Pinning des Mess-Threads — ist implementiert; die Kern-Wahl innerhalb des Kern-Komplexes ist Aktuator-Politik ohne Emissions-Vertrag, und die Mess-Emission des Planers weist das Pinning als vorgesehen und den Ist-Zustand als *ungepinnt* aus, weil der Aktuator nicht an das Lauf-Profil angebunden ist — auf `prod1` mit seinen zwei ungleich großen L3-Komplexen (96 und 32 MiB) ist ein ungepinnter Lauf nicht reproduzierbar, was die Emission wörtlich sagt. Fehlende Fähigkeiten einer Baseline — etwa Bereichs-Scans einer Hash-Tabelle — werden als *n/a* ausgewiesen statt verdeckt emuliert; die Mechanik dafür sind die Stichproben-Status *NotApplicable*, *SourceUnavailable* und *Failed* der Mess-Achsen, sodass fehlende Fähigkeiten und fehlende Quellen *n/a* führen, nie eine Null. Provenienz: Werkzeugkette (Compiler-Kennung und -Version), Flags, ISA-Zusammenfassung und die Commit-Stände der vier Repositories werden zur Konfigurationszeit als Bau-Provenienz erfasst und in das Provenienz-Manifest der Host-Exporte geschrieben; der Allokator ist als Organ-Achse Teil jeder `binary_id`; eine Provenienz je einzelner Fremdbibliothek — ihr eigener Compiler, ihre Flags, ihr Commit-Hash — wird nicht protokolliert (nicht implementiert). Die Latenz-Perzentile bestimmt der Auswertungs-Baum der Cache-Engine nearest-rank aus den Rohwerten — Hyndman und Fan, Typ 1: Rang $k = \lceil q \cdot n \rceil - 1$, ohne Interpolation [HF96] —, sodass jedes Perzentil ein tatsächlich gemessener Wert ist⁴; die Darstellungs-Werkzeuge des Werkzeug-Repositories besitzen eigene Kopien der Umrechnung (eine ausgewiesene Lücke), und das HDR-Histogramm [Ten12] ist ein eigenes Verfahren der Verteilungssicht (Abschnitt 4.5), keine Ergänzung des Nearest-Rank. Layout- und Umgebungs-Effekten, die ohne erkennbaren Fehler falsche Schlüsse erzeugen [MDHS09], begegnet der Apparat mit identischen Schlüsseln, identischer Bau-Matrix je Variante und der Wiederholungs-Dimension; eine randomisierte Layout-Stabilisierung im Sinne von Curtsinger und Berger [CB13] ist nicht Teil des Apparats und wird in Abschnitt 5.2.6 als Grenze geführt.

⁴Nearest-Rank: Das q -Quantil ist der kleinste Stichprobenwert, unter dem mindestens der Anteil q der Stichprobe liegt; interpolierende Verfahren (Typ 7) erzeugen dagegen Werte, die nie gemessen wurden.

Für fremde Bibliotheken gilt die Regel des Original-Compilers: Sezierte Achsen-Funktionen werden zur Übersetzungszeit eingebunden und zu einem wieder originalen Binary-Block kompiliert, sodass jeder sezierte Teilalgorithmus seine Leistungseigenschaften behält; wo umsetzbar oder vergleichbar, wird die Fremdquelle mit ihrem Original-Compiler gebaut. Der Nachweis der Originalität ist implementiert: die `is_original`-Markierung je Funktion (ein Makro der Cache-Engine samt Mixin-Basis der Achsen-Bausteine), der Validator als eigenes Programm und das manuelle Relock-Gate der CI, das die Lock-Dateien der Markierungen neu verankert; der Beleg je Baustein steht in Anhang D. Die H2-Akte markiert Rekonstruktionen ohne vendorte Originalquelle als n/a (22 von 33, Abschnitt 5.1.1). Umsetzungsstand: Der automatische Einzug der vendorten Quellen in den Bau mit ihrem Original-Compiler ist vorgesehen, aber nicht implementiert.

Auch der Mess-*Betrieb* ist Teil dieser Fairness-Regeln, und er läuft über das Lager. Das Lager ist der Bestand aller gebauten Tier-Binaries und ihrer Messwerte, adressiert über Übersetzungszeit-Schlüssel (Abschnitt 3.4.5; Abbildung 3.14 zeigt seine Struktur): Bestand 1 führt die Binary unter ihrem SHA-512-Fingerprint, Bestand 2 die Messwerte unter dem Paar aus dem Fingerprint der vermessenen Binary und der Hardware-Identität der messenden Maschine; die Zelle aus ISA, Optimierung und Erweiterung steht daneben geklammert, nie im Digest. Weil der Mess-Gate-Zustand ein Fingerprint-Glied ist, liegen die Binary mit und die Binary ohne Messfühler als zwei Einträge des Bestands 1 nebeneinander. Die Messung ist immer eingeschaltet; im Lager wird nur nachgemessen, was dort fehlt: Eine vorgefundene Binary wird übersprungen, wenn ihr Fingerprint bekannt und ihr bestandenes Prüf-Testat am Binary inventarisiert ist, sonst wird gebaut oder gemessen (Abbildung 5.2). Die CEB steuert die Parallelität, indem sie parallele Anfragen an die Tier-Binaries erzeugt — die Ausführungs-Threads entstehen in der CEB, in einem Worker-Pool mit deterministischer Ergebnis-Reihenfolge —, während das Threading der Gattung in der Tier-Binary bleibt. Umsetzungsstand: Bestand 1 und 2 sowie das Schlüssel-Schema aller vier Bestände sind implementiert; Bestand 3 und 4 (synthetisierte Kurven als Dateien, Lösungen je Anwender-XML), die Planer-Blattfunktion, die Vollplatzierung der CEB, die Hybrid-Ablage, der Commit-Skip, das rebuild-Flag und die XML-Sektion `<publish>` bilden den Lager-Vollausbau und sind nicht implementiert.

Die Messung bleibt strikt vom Bau getrennt: Mess-Läufe starten nie automatisch mit einem fehlerfreien Bau, sondern werden ausdrücklich ausgelöst — ein fehlerfreier Bau beweist Bau-Fähigkeit, kein Mess-Ergebnis. Die Ablaufmethodik kennt vier Arbeitsmodi — `build`, `measure`, `compare` und `release` (Abschnitt 4.3.2) —; der Mess-Modus misst mit genau *einem* Mess-Thread, eine parallele Mess-Ausführung ist über keine Registry-Zeile scharfgeschaltet und nur über die Zustands-Injektion des Laufeintrags erreichbar, und `-debug` ist ein Schalter der Planer-Shell für den Verdrahtungs-Check, der für Anwender gesperrt bleibt, kein Arbeitsmodus (unter `-debug` fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces; dieser indirekte Debug-Modus fällt mit dem Release-Bau zusammen und wird nicht separat geführt); das Threading der Gattung selbst bleibt dabei der Tier-Binary überlassen, statt vom Harness überschrieben zu werden. Ein im Lager vorgefundenes Binary wird nur dann übersprungen, wenn sein bestandenes Prüf-Testat am Binary inventarisiert ist (Abbildung 5.2), und vor jeder Messung steht die Konformitätsprüfung am Prüf-Dock (Abschnitt 4.4): das Konformitäts-Gate des Gattungs-Vertrags (unter `-debug` um den Standard-Testsatz des Tier-Interfaces erweitert) vor den Post-Compile-Testaten der Mess-Interfaces, daneben das Konsistenz-Gate der Messung; Bau- wie Mess-Läufe werden je Maschine in Meilenstein-Batches von je 4096 Binaries am Stück abgearbeitet — das Korn ist eine Übersetzungszeit-Konstante, deren Abweichung ein Übersetzungsfehler ist —, mit gleichverteilter

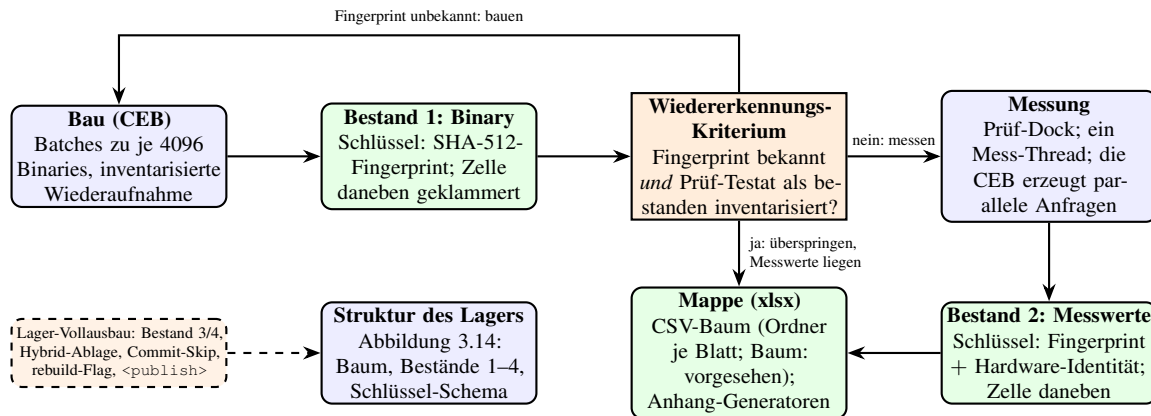


Abbildung 5.2: Der Lager-Kreislauf im Mess-Betrieb: Die CEB baut in Batches zu je 4096 Binaries; jede Binary liegt unter ihrem SHA-512-Fingerprint in Bestand 1; das Wiedererkennungskriterium überspringt, was mit bestandenem Prüf-Testat inventarisiert ist, und lässt sonst bauen oder messen; die Messung läuft am Prüf-Dock mit einem Mess-Thread, während die CEB die parallelen Anfragen erzeugt; die Messwerte liegen unter Fingerprint und Hardware-Identität in Bestand 2 und fließen in die Mappe (die CSV-Baumform mit einem Ordner je Blatt ist vorgesehen; implementiert ist eine flache Datei). Die Struktur des Lagers (Baum, vier Bestände, Schlüssel-Schema) zeigt Abbildung 3.14; gestrichelt der Lager-Vollausbau.

Zuteilung der gemeinsam baubaren Binaries über die Maschinen und inventarisierter Wiederaufnahme nach Unterbrechung.

5.1.8 Der heuristische Gegenbeweis als vierte Dimension

Über die drei Hypothesen hinaus erhält die Auswertung eine vierte, eigenständige *Evaluations-Dimension*: den heuristischen Gegenbeweis. Sein Konzept ist in Kapitel 3 entwickelt (Abschnitte 3.6.2 und 3.6.3): Die CacheEngineBuilder-Stufe kompiliert die heuristisch geschätzte Rekombination der Tier-Binaries zu einer Hybrid-Tier-Binary — der Hybrid-Modul-Emitter liegt im Builder-Baum der CEB —, und die Hybrid-Tier-Binary ist eine eigene Träger-Stufe, die optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschoben wird: Sie dockt am Prüf-Dock an, führt Dock-Slots für Tier-Binaries und erzeugt selbst keinen Code. Das Messkurven-Typsystem liefert die Schätzgrundlage (Abbildung 5.3): je Achse eine Kurve über ihren Parametern und je Mess-Ebene wallclock, macro und micro, daraus die Break-Even-Punkte und die geschätzte Rekombination; die Messfehler-Erkennung formuliert Abschnitt 3.6.3. Die Bewertungsfrage dieser Dimension lautet: „Ist die heuristisch geschätzte Rekombination der Tier-Binaries schneller als die existierenden Paper-Algorithmen?“ Anders als H1–H3, die Aussagen über den Entwurfsraum prüfen, prüft der Gegenbeweis den Apparat der Arbeit selbst: ob die aus den Messkurven geschätzte Rekombination im tatsächlichen Wall-Clock-Verhalten die von den Kurven vorhergesagte Leistung erreicht; er steht deshalb *neben* der Rangbildung der Sensitivitätsanalyse (Abschnitt 5.2.5), nicht in ihr. Umsetzungsstand: Klassifikations-Schicht, Dock-Schicht, XML-Parser und Binary-Proxy der Hybrid-Stufe sind implementiert (sechs der neun Dateien der in der README des Verzeichnisses `hybrid/` vorgesehenen Dock-, Proxy- und Router-Schicht, Abschnitt 4.5.1); Router und Eviction setzen Messkurven voraus und sind nicht implementiert.

Der Prüfplan hierfür ist zweistufig festgelegt; die zugehörigen Messläufe liegen nicht vor (Abschnitt 5.2.4).

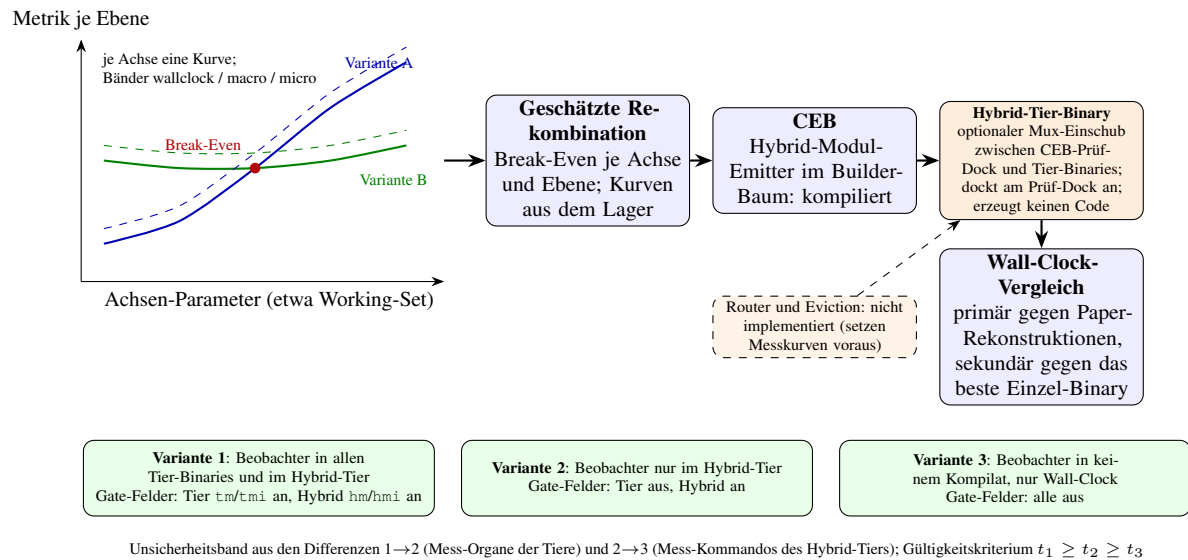


Abbildung 5.3: Das Messkurven-Typsystm im heuristischen Gegenbeweis. Links: je Achse eine Kurve der Metrik über dem Achsen-Parameter, getrennt nach Mess-Ebene (wallclock, macro, micro); der Schnittpunkt zweier Varianten ist der Break-Even-Punkt, aus dem die Rekombination geschätzt wird. Mitte und rechts: Die CEB kompiliert die geschätzte Rekombination über den Hybrid-Modul-Emitter; die Hybrid-Tier-Binary ist die optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Mux-Stufe, dockt am Prüf-Dock an, führt Dock-Slots für Tier-Binaries und erzeugt selbst keinen Code; verglichen wird an der Wall-Clock, primär gegen die Paper-Rekonstruktionen, sekundär gegen das beste Einzel-Binary. Unten: die drei Bau-Varianten des Messfehler-Prüfplans, auseinandergehalten über die Gate-Felder des Fingerprints. Gestrichelt: Router und Eviction (nicht implementiert).

Die erste Stufe quantifiziert den Messfehler, den die Mess-Instanzen selbst eintragen, denn die Schätzgrundlage der Heuristik sind *beobachtete* Kurven, ihr Anspruch aber gilt dem unbeobachteten Produktionsverhalten. Dazu wird dasselbe Experiment in drei Bau-Varianten aufgelegt:

1. Beobachter in *allen* Tier-Binaries *und* im Heuristik-Hybrid-Tier,
2. Beobachter *nur* im Heuristik-Tier-Binary,
3. Beobachter in *keinem* Kompilat — gemessen wird nur die Wall-Clock.

Auseinandergehalten werden die Varianten über das Mess-Gate-Glied des Fingerprints (Abschnitt 3.4.4): Es führt eigene Felder für die Tier-Ebene (t_m, t_{mi}) und die Hybrid-Ebene (h_m, h_{mi}); der Aus-Zustand aller Felder lautet $mg=m0; s0; st0; x0; tw0; tm0; tmi0; hm0; hmi0$, und weil das Glied in den Digest eingeht, liegen die drei Varianten als drei Binaries mit drei Fingerprints im Lager nebeneinander. Die Differenzen zwischen den drei Varianten beziffern den Beobachter-Overhead getrennt nach seinem Ursprung — Mess-Organen der untergebenen Tiere (Differenz von Variante 1 zu 2) gegen Mess-Kommandos des Hybrid-Tiers (Differenz von Variante 2 zu 3) — und ergeben je Workload und Plattform ein *Unsicherheitsband*, unterhalb dessen zwei Konfigurationen als nicht unterscheidbar gelten; das Band wird aus Wiederholungen mit Konfidenzintervall gebildet, nicht aus einem Einzellauf [GBE07]. Gültigkeitskriterium der Stufe: Mit Messfühler muss langsamer sein als ohne — für die Wall-Clock-Zeiten der drei Varianten gilt $t_1 \geq t_2 \geq t_3$; eine Verletzung ist ein Mess-Defekt der betroffenen Zelle, kein

Ergebnis, und führt zum Ausschluss der Zelle. Umsetzungsstand: Diese Plausibilitäts-Prüfung ist im Code nicht implementiert; sie schließt an die Messfehler-Interpolation als fünfte Auswertungs-Komponente der compare-Stufe an (Abschnitt 3.6.3), deren Konstellationen der Planer dynamisch aus der Gate-Belegung berechnet, nie als feste Anzahl.

Die zweite Stufe ist der komplementäre *vierte Schritt* der finalen heuristischen Zusammenstellung: Die dem Heuristik-Tier untergebenen Tier-Binaries — nicht die final optimierte Haupt-Binary — werden ohne Mikro- und Makro-Benchmarks und ohne Beobachter neu gebaut, sodass kein Mess-Artefakt im Kompilat verbleibt. Verglichen wird dann rein an der Wall-Clock, in zwei Vergleichsmengen mit festgelegtem Vorrang: primär gegen die rekonstruierten Paper-Algorithmen — die 33 Korpus-Arbeiten mit je einem Profil-XML, 30 Vollprofile und 3 abstrakte (Abschnitt 3.2); der Paper-Vergleich geht vor, denn er beantwortet die Bewertungsfrage wörtlich —, sekundär gegen alle bekannten Tier-Binary-Permutationen, insbesondere das beste Einzel-Binary; dieser zweite Vergleich ordnet ein, ob ein etwaiger Vorsprung der Heuristik als Ganzer oder schlicht einer einzeln überlegenen Permutation zuzuschreiben ist.

Daraus folgt die Entscheidungsregel: Der Gegenbeweis gilt je Workload-Klasse als *erbracht*, wenn die beobachterfrei neu gebaute Rekombination die schnellste Paper-Rekonstruktion in der Wall-Clock schlägt und der Abstand das Unsicherheitsband der Bau-Varianten übersteigt; er gilt als *verfehlt*, wenn ein Paper-Algorithmus oder das beste Einzel-Binary vorn liegt — auch dieses Ergebnis wäre ein Resultat, denn es bezifferte erstmals, wie viel Schätzfehler zwischen Kurven-Prognose und realem Verhalten liegt. Das Unsicherheitsband ist von einem Mess-Defekt zu unterscheiden: Zellen, die das Gültigkeitskriterium der ersten Stufe verletzen, gehen nicht in die Entscheidung ein. Die Messdaten dieser Dimension liegen nicht vor; sie werden mit den Reihen A–C in Abschnitt 5.2.4 ausgewiesen.

5.2 Ergebnisse und Analyse

5.2.1 Identität der Ergebnisse: Stempel, Lager, Mappe

Jede Ergebniszeile ist über das Stempelsystem identifizierbar (Abschnitt 3.4.4): die `binary_id` — der Pfad aus achtzehn `achse=wert`-Segmenten in Slot-Reihenfolge —, der SHA-512-Fingerprint über elf Glieder mit der Format-Kennung `fingerprint_format=6` und die drei einkompilierten Stempel-Zeilen je Realm (Organ, System, Mess). Diese Identität reist unverändert durch die Kette: als Lager-Schlüssel (Bestand 1 und 2), als Spalten `permutation_id` und `fingerprint` der Mess-Zeile, als Blatt-Schlüssel der Mappe und als Kennung in jedem Diagramm und jeder Tabelle des Anhangs — so bleibt jede dargestellte Zahl auf ihre Binary, ihre Achsen-Versionen und ihre Maschine zurückführbar. Die Misch-Strategie eines Prüflings wird nicht über ein Versions-Suffix ausgedrückt, sondern über die `merge`-Token `replace`, `merge` und `union` der Anwender-XML und die hierarchisch erweiterten Algorithmus-Namen der Organ-Zeile unter dem Namensraum-Präfix des Prüflings; das Flag `e` der Eintrags-Grammatik bezeichnet einen Efficiency-Core-Algorithmus (den E-Kern-Pfad einer Hybrid-CPU), kein Experiment und keine Mischung. Werte der Messung erscheinen nie im Stempel; gestempelt wird die Übersetzungszeit-Identität. Umsetzungsstand: Die Anhang-Tabellen führen `permutation_id` und `fingerprint` je Zeile; die volle Stempel-Identität je Diagramm ist vorgesehen, aber nicht implementiert.

5.2.2 Datenfluss von der Messung zum Manuskript

Je Reihen-Lauf schreibt die Messstufe die eine offizielle `measurements.csv` als Projektion der Mappe (implementiert ist eine flache Datei; vorgesehen ist der CSV-Baum mit einem Ordner je Blatt, unten); die `<output>`-Pfade der Anwender-XML (Muster `_runs/<Datum>/<Reihe>/`) löst der Laufeintrag als Lauf-Provenienz auf. Drei Spaltenlisten sind zu unterscheiden (Tabelle 5.4): das WIDE-Schema mit 189 Spalten (Trenner Semikolon; die Schema-Hoheit der Mess-Ausgabe, deren Kopf zur Laufzeit aus vier Einzelquellen entsteht und deshalb als Festschreibung verankert ist), die Pipeline-Sicht mit 16 Spalten plus 16 Zusatzspalten, also 32, für die LaTeX-Stufen (Trenner Komma) und die dreizehnspalrige Altform des Ergebnis-Aggregators über `ExecutionResult`. Die Pipeline-Sicht führt je Permutation `permutation_id`, `fingerprint`, `succeeded`, `workload_used`, `op_count`, `total_cycles`, `cache_misses_l1`, `cache_misses_l2`, `cache_misses_l3`, `dtlb_misses`, `coherence_invalidations`, `energy_micro_joules`, `bytes_allocated`, `bytes_in_use_peak`, `external_frag` und `internal_frag`; die volle Sicht ergänzt sechs Such-Zähler (`search_insert` bis `search_peak_occupancy`), `pmc_available`, `branch_misses`, `throughput_ops_per_sec` und die sieben `*_source_available`-Flags. `total_cycles` führt — unter seinem Legacy-Namen — die repräsentative p50-Latenz in Nanosekunden, keine Zyklen; die Umbenennung oder ein tatsächlicher Zyklen-Kanal ist eine dokumentierte Entwurfsentscheidung. Nicht erhobene Werte stehen als n/a, nie als Null.

Schema	Trenner	Spalten	Hoheit / Konsument	Quelle im Code
WIDE (E4)	Semikolon	189	Mess-Ausgabe; Kopf zur Laufzeit aus vier Einzelquellen, als Festschreibung verankert	Iterator des <code>CacheEngineBuilders</code> (<code>lazy_csv_header</code>); Schema-Festschreibung
PIPELINE	Komma	16 (+16 = 32)	LaTeX-Stufen 04/05/06; volle Sicht als Präfix-Superset	Pipeline-CSV-Schema: eine Liste, vier Schreiber
F15-ALT	Komma	13 (12 + 1)	Ergebnis-Aggregator über <code>ExecutionResult</code>	Ergebnis-Aggregator der Builder-Kommandos

Tabelle 5.4: Die drei Mess-Spaltenlisten der Cache-Engine und ihre Hoheit. `total_cycles` der Pipeline-Sicht führt die repräsentative p50-Latenz in Nanosekunden (Legacy-Name), keine Zyklen. F15-ALT umfasst zwölf Spalten und, hinten angehängt, degeneriert (12 + 1).

Die Mappe (xlsx) entsteht immer — im Speicher, bedingungslos, auch wenn ein Profil nur CSV persistiert; sie ist die Ausgabe und der Standard, und die CSV-Form ist kein einzelnes Kind-Dokument, sondern ein geordneter Baum aus CSV-Dateien — ein Gesamtordner je Mappe, darunter ein Ordner je Blatt, darin die CSV-Dateien der einzelnen Unter-Achsen-Fahrten —, der immer aus der Mappe erzeugt wird (xlsx nach csv, als Strategie derselben Factory) und als Darstellung ihr analog gespiegeltes Gegenstück ist: Eine Zeile erreicht den CSV-Baum nur, wenn die Mappe sie zuvor angenommen hat, und bricht die Mappen-Erzeugung, ist auch der CSV-Baum ungültig; was auf die Platte geht, entscheidet allein `<writeback_methods>` — xlsx, csv oder beide (die Mappe, der CSV-Baum oder beides am selben Ziel), als Strategie über den Ausgabe-Formaten mit xlsx als Default (Abschnitt 4.5); der xlsx-Writer liegt in der Lager-Ablage und schreibt über `libxlsxwriter`. Umsetzungsstand: Die Naht schreibt die CSV-Projektion als eine flache Datei (`projektion_oeffnen` in `ergebnis_mappe_naht`); die Baumform ist vorgesehen, aber nicht implementiert. Die Auswertung reichert die Projektion über Binary-zu-CSV um

Profil-ID, Paper-Referenz und Achsen-Belegung an, rendert CSV-zu-LaTeX-Steckbriefe je Permutations-ID (eine leere Eingabe erzeugt keine Datei, nie einen leeren Plot) und Diagramm-Generator-Diagramme als TikZ-Balken, Scatter-Plots und Heatmaps samt den darauf aufbauenden Familien des Mess-Anhangs (ECDF, Range-Balken, Sweep-Kurven, 3D-Rohdaten-Ansicht, Verhältnis-Matrizen, normalisierte Balken, Pareto-Streuung, Per-Achsen-Attribution, Observer-Detail-Tabelle) mit A4-Deckel und Größen-Prüfung (eine 1×1 -, $1 \times n$ - oder $n \times 1$ -Matrix erzeugt kein Bild); der Anhang-Generator schreibt je Sprache in ein `tabellen`-Verzeichnis, aus dem das Manuskript die Dateien einbindet — die deutsche Fassung bindet 38 erzeugte Dateien des Anhangs ein.

Die Mappe eines Laufs ist eine Datei mit einem Blatt je gewählter Unter-Achsen-Permutation und einem Info-Blatt (System-Informationen der messenden Maschine, verwendete Haupt-Achsen aller drei Realms, konstante Meta-Achsen); die Blätter sind in der bindenden Ordnung Mess $\rightarrow$ System $\rightarrow$ Organ nummeriert (S001 bis Snnn), und die Spalten stammen ausschließlich aus dem WIDE-Kopf; Spalten, deren Wert sich innerhalb eines Laufs nie ändert, wandern in die Metadaten des Info-Blatts, statt in jeder Zeile wiederholt zu werden. Die Contract-Tests der Persistenz prüfen an der CSV-Ausgabe; eine Prüfung an der Standard-xlsx ist vorgesehen, aber nicht implementiert (Abschnitt 4.4). Neben xlsx und csv ist eine dritte Factory LaTeX vorgesehen, die die Blatt-Ordnung der Mappe beim Ausrollen des Anhangs serialisiert und je Binary eine Tabellenzeile als zeitliche Messreihe führt. Umsetzungsstand: Writer und Projektion sind implementiert; die Naht schreibt ein Datenblatt unter konstantem Blattschlüssel, und die Projektion ist eine flache Datei. Nicht implementiert sind: die Auffächerung in ein Blatt je Unter-Achsen-Permutation plus Info-Blatt (Abschnitt 6.2), der CSV-Baum (ein Ordner je Blatt), die Prüfung an der je Binary erzeugten xlsx (die Persistenz-Prüfung liest die CSV-Ausgabe), das Export-Element mit Ziel-Filter und per-Binary-xlsx, die Chart-Funktionen, die LaTeX-Factory, die Warnungs-Sektion des Info-Blatts und der Nachzug der WIDE-Festschreibung.

5.2.3 Messreihen und Verbund-Strategien

Drei Messreihen werden ausgewertet (Tabelle 3.8): Reihe A — PRT-ART gegen den Stand der Technik mit den Evaluations-Hypothesen H1–H3 (Abschnitt 5.1.1); Reihe B — die systematische Variation der Cache-Engine-Permutationen, gespeist aus der Union der Stufe 3; Reihe C — Merge/Regression alt gegen neu, bau-übergreifend. Quer zu dieser Zuordnung deklariert die Anwender-XML je Achse den `merge`-Modus des Prüflings (Tabelle 5.5): `replace` (der Standard, auch bei leerem Attribut; die Prüflings-Achse ersetzt die Cache-Engine-Achse mit Rückfall, Verbund-Strategie `Verbund2_Replace`), `merge` (Cache-Engine und Prüfling hybrid je Prüfling, `Verbund2_Hybrid`) und `union` (die explizite, an Stufe 3 gebundene Vereinigung aller Achsen-Varianten ohne Verwerfen, `Verbund3_Union`); Stufe 1 ist `Verbund1_CeOnly`, die Abwesenheit jeder Prüflings-Direktive, und „Stufe 1 bis 3“ bleibt der Alias der Verbund-Namen, der Full Join der Alias von `union`. Reihe A arbeitet mit `replace` und `merge`, Reihe B mit `union`. Unterscheidbar bleiben die Stände an der Binary über die hierarchisch erweiterten Algorithmus-Namen der Organ-Zeile (Abschnitt 5.2.1); eine eigene Stempel-Zeile besitzen sie nicht, denn der Prüfling mischt gegen die Organ-Achsen und steht damit bereits in deren Zeile, und ein Versions-Suffix bezeichnet die Mischung nicht. Bei reinen Ersetzungs- wie reinen Cache-Engine-Bauten bleibt die Namens-Erweiterung folgerichtig aus. Umsetzungsstand: Das Token `merge` ist im Merge-Plan registriert und im golden-Kern-Profil deklariert; materialisiert sind die Strategien `Verbund1_CeOnly`, `Verbund2_Replace` und `Verbund3_Union`, während `Verbund2_Hybrid` ohne Produktions-

Aufrufer bleibt, weil seine Materialisierung im Strategie-Enum nicht implementiert ist.

merge-Token	Verbund-Strategie	Alias	Wirkung	Reihe
— (keine Direktive)	Verbund1_CeOnly	Stufe 1	nur Cache-Engine: Standard-Bausteine und SOTA-Profil	A (Baseline, auch C)
leer / replace	Verbund2_Replace	Stufe 2 (Ersetzen)	die Prüflings-Achse ersetzt die Cache-Engine-Achse, mit Rückfall	A
merge	Verbund2_Hybrid	Stufe 2 (Hybrid)	Cache-Engine und Prüfling hybrid je Prüfling; Materialisierung offen, kein Produktions-Aufrufer	A
union	Verbund3_Union	Stufe 3 (Full Join)	Vereinigung aller Achsen-Varianten ohne Verwerfen, an Stufe 3 gebunden	B

Tabelle 5.5: Die merge-Token der Anwender-XML und ihre Verbund-Strategien im Merge-Plan; die Stufen-Namen sind Aliasse, der Full Join ist der Alias von union. Die Reihen-Spalte folgt der Zuordnung aus Tabelle 3.8.

5.2.4 Vorliegende Messungen

Methodik und Auswertungs-Pipeline sind vorhanden und an realen Daten verifiziert: Eine Smoke-Messreihe über 43 Permutationen — DLL-Messung; 27 kartesische Kombinationen aus SIMD, Speicherlayout und Allokator plus 16 weitere über vier zweiwertige Größen — dokumentiert Anhang A samt ihren Limitierungen (Abschnitt A.10); sie variiert sieben Größen (SIMD ist eine System-Achse, Speicherlayout und Allokator sind Organ-Achsen, die übrigen vier Namen sind keine Registry-Bausteine und werden nicht gleichgesetzt), die vollständige Messreihe variiert alle achtzehn Kompositions-Achsen. Die vollständigen Läufe der Pflicht-Messreihen A–C auf den Zielplattformen liegen nicht vor (n/a), weil der vollständige Bau nicht durchgeführt ist (Abschnitt 6.2); dasselbe gilt für die Messdaten des Gegenbeweises (Abschnitt 5.1.8). Keine Zahl dieses Kapitels stammt aus einem anderen Lauf als dem im Anhang ausgewiesenen.

5.2.5 Achsen-Sensitivitätsanalyse

Den Kern der Auswertung bildet die *Achsen-Sensitivitätsanalyse*: Sie beantwortet das Trennbarkeits-Problem der Aufgabenstellung (Abschnitte 1.3 und 3.1), indem das Achsen-Framework jeden Bestandteil als austauschbare Achse über einer sonst konstanten Struktur isoliert und der Beitrag jeder Entwurfsentscheidung unter sonst gleichen Bedingungen zurechenbar wird. Die Rangbildung läuft über die drei Mess-Ebenen in Kanon-Reihenfolge: *wallclock* — das Gesamt-Benchmarking der YCSB-Lastprofile im CacheEngineBuilder —, *macro* — die Gattungs-Interface-Operationen der Tier-Binary — und *micro* — das Achsen-Interface, die Funktionsmenge, die alle Algorithmen einer Achse liefern müssen (Abschnitt 3.4.6); anhand der Zeilen je Permutation quantifiziert die Analyse auf der micro-Ebene, welche Achse am meisten zur Performance-Varianz beiträgt, die wallclock-Ebene prüft die interne micro-Messung von außen, und die Mess-Kategorien sind dabei die Unter-Achse der Tooling-Achse (Abschnitt 5.1.5). Leitachse ist die Cacheline-Awareness: welche Achse die cache-nahen Kategorien auf welcher Plattform bewegt. Aus der Rangbildung folgt die Empfehlung von Standard-Konfigurationen je Workload-Klasse; wo Verhältnisse über Workloads oder Plattformen zusammengefasst werden, geschieht das geometrisch,

nie arithmetisch gemittelt [FW86]. Der heuristische Gegenbeweis steht neben dieser Rangbildung (Abschnitt 5.1.8).

5.2.6 Gültigkeit und Grenzen

Die Gültigkeit der Evaluation wird nach den vier Klassen der empirischen Softwareforschung geprüft⁵ (Tabelle 5.6; Abschnitt 3.4 führt dieselben Klassen als Gütekriterien des Mess-Systems). Konstrukt-Validität: Messen die Kategorien die Cacheline-Awareness? Die PMC-Semantik ist je Modell offengelegt (Abschnitt 5.1.5), und Hardware-Zähler gelten nur gegengeprüft als vertrauenswürdig [WM08]. Interne Validität: Der Beobachter-Overhead ist über das Unsicherheitsband der drei Bau-Varianten gebunden (Abschnitt 5.1.8); Layout- und Umgebungs-Bias [MDHS09, CB13] begegnen identische Schlüssel, identische Bau-Matrix und Wiederholungen, eine Layout-Randomisierung fehlt; Warmup und Wiederholungen folgen Abschnitt 5.1.7 [KJ13]. Externe Validität: Die registrierten Messmaschinen umfassen zwei x86-64-Mikroarchitekturen und einen Gracemont-Knoten; ZIH und HBM sind deklariert, nicht gemessen (Abschnitt 5.1.4), die Intel-RAW-Belegung fehlt, und ein einseitig gefahrener Pfad gilt als Teil-Messung. Schluss-Validität: Perzentile nearest-rank ohne Interpolation, geometrische Aggregation, n/a statt Null. Die Limitierungen des Mess-Anhangs (Tabelle A.10, Abschnitt A.10) und des Fazits (Abschnitt 6.2) sind die belegte Liste dieser Grenzen; keine Zahl wird ohne Lauf genannt, und die Berichtsregeln für Leistungsmessungen [HB15, BDH⁺16] gelten für jede Zahl, die ein Lauf liefert.

5.2.7 Zusammenfassung der Evaluation

Die Evaluation hält je Evaluationsobjekt fest. Entwurfsraum: Die Hypothesen H1–H3 führen Messgröße und Kriterium, und die Sensitivitätsanalyse ist als Verfahren über die drei Mess-Ebenen festgelegt; Belege liegen für die Smoke-Reihe vor (43 Permutationen, sieben Größen, Anhang A), für die Pflicht-Messreihen nicht (n/a, vollständiger Bau nicht durchgeführt). Apparat: Der Gegenbeweis ist als vierte Dimension mit Prüfplan, Gültigkeitskriterium und Entscheidungsregel festgelegt, seine Messdaten liegen nicht vor; das Messfehler-Fundament — Gate-Felder im Fingerprint, Feld-Flags je Zähler, n/a-Regel — ist implementiert. Automatisierung: Die Kette von der Experiment-XML bis zum eingebundenen Anhang ist in einem ersten durchgehenden Lauf nachgewiesen und liefert jede Zahl mit ihrem Lauf. Die Ergebnisse werden vor dem Leitmotiv der Arbeit diskutiert — dem Übergang vom heuristischen zum informierten Cache-Management, also der Frage, ob telemetrie-getriebene, automatisch adaptierte Konfigurationen statisch gewählte übertreffen; der Cache-Line-Effekt aus den Synthese-Kurven ist der erste Auswertungsschritt nach dem vollständigen Lauf, datenabhängig und deshalb hier nicht vorweggenommen. Nach den Maßstäben für Forschungsbeiträge [Sha03] und für empirische Bewertungen [BDH⁺16] ist der Beitrag dieses Kapitels ein validierter Versuchsaufbau samt seiner Automatisierung, dessen Ergebniszahlen mit der vollständigen Messreihe vorliegen werden. Kapitel 6 beantwortet die Forschungsfragen auf dieser Grundlage.

⁵Threats to Validity: die in der empirischen Softwareforschung übliche Einteilung der Bedrohungen eines Schlusses in Konstrukt-Validität (misst die Messgröße das Gemeinte?), interne Validität (ist der Effekt kausal isoliert?), externe Validität (überträgt sich der Befund auf andere Plattformen und Lasten?) und Schluss-Validität (stützen Statistik und Aggregation den Schluss?) [WRH⁺12].

Klasse	Bedrohung	Gegenmaßnahme im Apparat	Umsetzungsstand
Konstrukt	Zähler messen nicht das Gemeinte (Modell-Semantik, Prefetch-Anteile)	RAW-Katalog je Modell mit Gegenprüfung; Semantik-Offenlegung je Spalte; n/a ohne Katalog-Eintrag	implementiert (Zen 5); Intel-Eintrag: nicht implementiert
Konstrukt	Zeit-Kategorie als Zyklen fehlgelesen	<code>total_cycles</code> als p50-Latenz in Nanosekunden deklariert	implementiert; Umbenennung: Entwurfsentscheidung
Intern	Beobachter-Overhead verfälscht die Kurven	drei Bau-Varianten, Unsicherheitsband, Gültigkeitskriterium $t_1 \geq t_2 \geq t_3$	Gate-Felder implementiert; Prüfung: nicht implementiert
Intern	Layout-/Umgebungs-Bias, Wechsel des L3-Komplexes	identische Schlüssel und Bau-Matrix; Wiederholungs-Dimension; Pinning als vorgesehen ausgewiesen	Aktuator implementiert, Anbindung: nicht implementiert; keine Layout-Randomisierung implementiert
Intern	Warmup- und Kaltstart-Effekte	Zwei-Phasen-Treiber (sichern, aufwärmen, zurückrollen, messen)	
Extern	Messmaschinen decken die Zielplattformen nicht ab	registrierte Signaturen <code>prod1</code> , <code>prod2</code> , ODROID; ZIH als IS3 deklariert	ZIH und HBM: deklariert, nicht gemessen (n/a)
Extern	einseitiger Hersteller-Pfad	Vollständigkeits-Regel der Auswertung; Teil-Messung wird ausgewiesen	Regel gesetzt; Intel-Pfad: Gegenprüfung nicht durchgeführt
Schluss	Perzentile synthetisch statt gemessen, Mittelwerte verzerrt	Nearest-Rank ohne Interpolation; geometrische Aggregation	implementiert (Auswertungs-Baum); Kopien in den Werkzeugen: ausgewiesene Lücke
Schluss	Null statt Nichterhebung	Feld-Flags je Zähler, n/a Ende-zu-Ende	implementiert

Tabelle 5.6: Gültigkeit der Evaluation nach den vier Klassen der Bedrohungen (Konstrukt, intern, extern, Schluss): je Bedrohung die Gegenmaßnahme im Apparat und ihr Umsetzungsstand — implementiert, deklariert und nicht gemessen (n/a) oder nicht implementiert. Keine Zeile nennt eine Zahl ohne Lauf.

6 Fazit und Ausblick

Dieses Kapitel zieht Bilanz: Es stellt je Forschungsfrage FF0–FF4 (Abschnitt 1.3) die Teilfragen voran, benennt den Stand ihrer Beantwortung in drei Stufen — beantwortet, strukturell beantwortet, offen —, führt je Frage die Beweiskette durch die Kapitel und ihre Folgen, ordnet die Hypothesen dem Zielpfad zu, benennt die Grenzen des Systems aus Sicht des Anwenders und skizziert die nächsten Schritte.

6.1 Beantwortung der Forschungsfragen

Die Antworten folgen dem Schema Behauptung, Beleg, Validierung [Sha03]: Eine Frage gilt als *beantwortet*, wenn ein Test oder eine Messung sie belegt; als *strukturell beantwortet*, wenn der Apparat, der die Antwort liefert, implementiert und getestet ist, die Messung aber nicht vorliegt; als *offen*, wenn weder Struktur noch Messung vorliegen. Drei Belegarten kommen vor: die Struktur (ein Code-Gegenstand, der die Antwort stützt), der Test (eine statische Prüfung oder ein Vertrags-Test, der sie sichert) und die Messung (eine Zahl mit ihrem Lauf); für die Frage nach dem Stand der Technik der Workloads (FF2.d) tritt der Literaturbeleg hinzu. Ein „strukturell beantwortet“ ersetzt die Zahl nicht: Ohne Messwerte bleibt die Hauptfrage unbeantwortet, auch wenn ihre Struktur tragfähig ist. Tabelle 6.1 führt Status, Beweis und Rest je Haupt- und Teilfrage; die Berichtsform folgt den Regeln der empirischen Software- und Systemforschung [WRH⁺12, RH09] und der Forderung, Bezugsgröße, Plattform und nicht erhobene Werte an jedem Ergebnis auszuweisen [BDH⁺16, HB15].

Quer zu allen fünf Fragen hat die Einleitung eine Kategorisierungs-Frage als Frage ausgelegt: ob sich die Achsen des Entwurfsraums sauber in Organ-, System- und Mess-Achsen trennen lassen — und wie eine solche Trennung zu bewerkstelligen ist, wo sie in einem Entwurf nicht angelegt ist. Beides ist beantwortet (Abschnitt 3.4.1): Die Achsen trennen sich entlang der drei Belange in drei *Realms* — Organ, System, Mess — mit je eigener Klassen-Wurzel, je eigenem Angebots-Katalog (drei generierte Registries: die Organ-Achsen-Registry der Cache-Engine, die System-Achsen-Registry und die Mess-Achsen-Registry) und je eigener Stempel-Zeile an der Binary (Abschnitt 3.4.4); die drei Zeilen-Träger des Fingerprints — Organ-, System- und Mess-Zeile — sind als Typen paarweise unkonstruierbar auseinander, sodass eine Vertauschung nicht übersetzt. In die `binary_id` gehen ausschließlich die Organ-Haupt-Achsen ein; System- und Mess-Achsen sind in ihren Registries als `binary-id-neutral` deklariert. Gehalten wird die Trennung durch übersetzungszeitliche *Rückkehr-Sperren* (Abschnitt 4.5): `static_assert`-Prüfungen weisen Compiler, Scheduling und Last-Rahmenwerk als System-Haupt-Achsen zurück — das Scheduling ist eine Unter-Achse der Ziel-ISA `target_isa` mit fünf Dimensionen, der Compiler eine Unter-Achsen-Gruppe der Komplex-Klammer `build_target_complex`, das Last-Rahmenwerk die Meta-Meta-Achse des Mess-Realms —, und die Flag-Grammatik der Versions-Stempel lässt nur katalogisierte Flag-Ketten zu. Diese Dreiteilung ist die zweite Gestalt des Trennbarkeits-Problems und stützt die Antworten auf die folgenden Teilfragen.

FF0 (Hauptfrage). Gefragt ist, wie stark aktive, messgetriebene cache-bewusste Entscheidungen — Seitentyp, Speicher-Layout, Value-Ablage, Prefetching — ihre passiven, statisch ausgelegten Gegenstücke auf unterschiedlichen CPU-Plattformen übertreffen; die Erweiterung FF0.E fragt, ob sich ein gemessener Gewinn eindeutig dem Organ-, dem System- oder dem Mess-Belang zuordnen lässt. Der *Status*: FF0 ist offen. Ohne die End-to-End-Läufe je Plattform gibt es keine Zahl; die Struktur, die die Zahl liefert, ist implementiert. Dieselbe achsen-konstante Struktur wird einmal aktiv — mit laufzeit-dynamischer Cache-Line-Politik (der Politik-Selektor leitet aus einem ausgewerteten Profil-Aggregat aus Operationsmix und Working-Set die Cache-Line-Anpassung ab) und messwert-getriebener Auswahl der besten Binary — und einmal passiv — mit statisch fixierter Organ-Wahl von Speicher-Layout, Prefetch, Value-Ablage und Knotentyp als Segmenten der `binary_id` — gebaut und unter identischer Compiler- und Flag-Basis gegenübergestellt (Standard `-O2`; `-O3` nur als Opt-in unter Warnung und mit eigener Toolchain-Identität). Vermessene Messmaschinen sind die drei registrierten Maschinen-Signaturen: `prod1` (AMD Zen 5, Ryzen 9 9950X3D), `prod2` (Intel Core i9-12900K, Alder Lake — eine Hybrid-CPU mit P- und E-Kernen, ohne AVX-512) und ein ODROID-Board mit Intel-Gracemont-Kernen; die in der Frage genannte Server-Plattform Sapphire Rapids ist in der Plattform-Erkennung angelegt (ZIH-Knoten Barnard, Xeon 8470), aber nicht gemessen (n/a; Abschnitt 5.1.4). FF0.E ist strukturell beantwortet, und die dafür nötige Trennung ist die Dreiteilung: Ein Organ-Effekt permutiert die `binary_id`, ein Plattform-Belang steht als System-Stempel-Zeile und Bau-Versionskennung an der Binary, ohne sie zu permutieren, und ein Mess-Belang bleibt `binary-id-neutral` — kein Mess-Instrument verändert, *was* gebaut wird. Als *Beleg* dienen Abschnitt 3.4.1 (Dreiteilung; Struktur), Abschnitt 4.5 (Rückkehr-Sperren; Test) und Abschnitt 5.1 (Messplan; Struktur). Die *Folge*: Eine Bezifferung setzt den vollständigen Bau und den Lauf der Reihe A voraus (Abschnitt 6.3); der Effekt der Cache-Line-Bewusstheit aus den synthetisierten Mess-Kurven ist der erste Auswertungsschritt nach dem ersten Messlauf (Abschnitt 3.6.3).

FF1 (Komposition). Gefragt ist, wie sich ein Suchalgorithmus-Entwurf kanonisch in orthogonale Achsen zerlegen und in einer Permutationsmatrix systematisieren lässt, sodass Stand-der-Technik-Verfahren möglichst aus ihrem Originalcode bias-frei als Profile rekonstruierbar und beliebig permutierbar werden; FF1.a fragt nach dem Kategorien-Schnitt der Zerlegung, FF1.b nach den Organ-Kopplungen und dem Verhalten unter allein variiertter ISA-/SIMD-Achse. Der *Status*: FF1 ist strukturell beantwortet, FF1.a beantwortet, FF1.b in seinem empirischen Teil offen. Die kanonische Zerlegung ist tragfähig: *Achtzehn* Organ-Haupt-Achsen in fester Slot-Reihenfolge T0–T17 — als konstantes Namens-Array der Achsen-Pfad-Serialisierung Code, nicht Konvention — spannen die Permutationsmatrix auf (Abschnitt 3.4.1); dazu tritt eine optionale *Organ-Meta-Meta-Achse* für die Festplatten-Ein-/Ausgabe `disk_io`, in der Organ-Registry als neunzehnter Eintrag geführt (übersetzungszeitliche Stufe mit der Laufzeit-Option `staging`, `binary-id-neutral`), per Experiment-Profil aber nicht wählbar, weil der Profil-Parser ihren Registry-Block nicht liest; der Profil-Anschluss ist vorgesehen, aber nicht implementiert. Sie ist das Beispiel der Arbeit für die Meta-Meta-Ebene M3, die Indirektion einer Beschreibung: Sie stempelt im Klammer-Anhang der Organ-Zeile, und die Klammer-Tiefe eines Eintrags ist seine Meta-Meta-Ebene (Abschnitt 3.4.4). Zwei Bezugsgrößen sind zu trennen: Der golden-Referenzkatalog variiert siebzehn Achsen je zwei und legt das Persistenz-Ziel auf einen Wert fest — $2^{17} = 131072$ Binary-Identitäten je System-Permutation unter der überholten Annahme zweier Auslegungen je Achse, als Regressions-Detektor (nicht

Obergrenze) mit einer festgeschriebenen CRC-64-Prüfsumme¹ über alle Identitäten (Abschnitt 4.4) —, während die Mächtigkeit eines geplanten Raums keine Konstante ist, sondern die Rechnung des Planers je Profil und Plattform (Subkommando `simulate`) als Obergrenze, die erst zur Ausführung bekannt ist; alle achtzehn Achsen sind für die Variation vorgesehen (nicht gemessen). Die 33 Korpus-Arbeiten P01–P33 besitzen je ein Profil-XML, davon 30 Vollprofile eigenständiger Suchentwürfe (acht Rang-1 und 22 Rang-2/3) und drei abstrakte Profile (OLC, LOUDS, VAMPIR), die nur einzelne Achsen belegen (Abschnitt 3.2); die Registry der Paper-Prüflinge hält diese Übersetzung übersetzungszeitlich gegen Dubletten und Lücken. Die variadische Abbildung — ein Template-Parameter für die `std::vector`-API, zwei für `std::map`, $N > 2$ für `std::map<K, std::tuple<...>`» (Anhang F) — macht algorithmus- wie container-orientierte Muster auf einer Engine ausdrückbar, und die Gattungs-Dreiteilung Map, Container, Graph ordnet die Gattungen darunter (Abschnitt 3.3.2). FF1.a ist bejaht: Die Zerlegung bleibt nur orthogonal, weil Plattform- und Mess-Belange aus der Permutationsmatrix herausgehalten sind, und die Rückkehr-Sperren halten diesen Schnitt (Abschnitt 4.5). Zu FF1.b: Die von der Frage genannte Kopplung Knotentyp $\times$ SIMD $\times$ Pfadkompression zerfällt — Knotentyp $\times$ Pfadkompression bleibt eine nicht voll-orthogonale Organ-Kopplung, die als solche markiert und bei der Permutation eingeschränkt wird, während SIMD keine Organ-Achse ist, sondern eine Bau-Steuergröße der System-Achse `external_utils` (Laufzeit-Unter-Achse mit den Optionen `no_extension`, `avx2` und `avx512` und einem Merkmals-Katalog von 23 Flags, `binary-id-neutral`): Ein und dieselbe `binary_id` wird unter mehreren System-Permutationen gebaut und bei sonst unveränderter Struktur verglichen; die empirische Antwort zum Cache-Update- und Cache-Line-Füllverhalten ist eine Messung und liegt nicht vor. Als *Beleg* dienen Abschnitt 3.4.1 (Zerlegung und Realms; Struktur), Abschnitt 3.4.6 (Katalog und Bezugsgröße; Struktur) und Abschnitt 4.4 (Katalog-Prüfungen und Golden-Prüfsumme; Test). Die *Folge*: FF1 strukturell beantwortet; FF1.b (Füllverhalten) setzt den vollständigen Bau voraus.

FF2 (Mess-Methodik). Gefragt ist, wie eine reproduzierbare Mess-Pipeline mit Hardware-Zähler-Erhebung gestaltet sein muss, damit jede Achsen-Permutation fair — unter gleicher Compiler- und Flag-Basis — in drei Messreihen über drei Granularitäten gemessen wird, und unter welchen plattformkalibrierten Kriterien eine Plattform als verifiziert gilt; FF2.a fragt, ob die Mess-Instrumente selbst eigene Achsen sein müssen, FF2.b nach der Nachmessung widersprüchlicher Literaturbefunde, FF2.c nach der Erweiterbarkeit, FF2.d nach den Workloads, die gerade für Suchalgorithmen zum Stand der Technik gehören. Der *Status*: FF2 ist strukturell beantwortet, FF2.a und FF2.d beantwortet. Die Messung ist in einen Produktiv- und einen Experiment-Modus geteilt, geschaltet über den Master-Schalter der Mess-Übersetzung (`COMDARE_MEASUREMENT_ON`): Der Experiment-Modus misst stets — ein Release-Bau ohne Messfühler ist ein eigenes, auslieferbares Artefakt, das den Schalter erzwungen abschaltet, kein abgeschalteter Messlauf —, und im Lager wird nur nachgemessen, was dort fehlt; das übersetzungsstatische Schlüssel-Schema der vier Bestände ist der Entwurf dieser Wiedererkennung, der Lager-Vollausbau ist nicht implementiert (Abschnitt 3.4.5). Den Ablauf steuert der `ExperimentDriver` host-seitig: Der Runner lädt die Module und führt die Phasen Laden, Ausführen, Messen und Abbau, der Treiber führt Ausführung und Messung je Modul und sammelt die Ergebnisse im `ResultAggregator`. Jede Reihe wird auf drei *Mess-Ebenen* erhoben, in genau dieser Reihenfolge: der Wallclock-Ebene (`wallclock`; die Ge-

¹CRC-64: eine zyklische Redundanzprüfung mit 64-Bit-Prüfsumme (hier die Variante ECMA-182), berechnet über alle Identitäten in kanonischer Reihenfolge; eine geänderte Identität ändert die Prüfsumme.

samtläufe im CacheEngineBuilder über die Last-Rahmenwerke), der Makro-Ebene (`macro`; die Konzept- und Gattungs-Interface-Operationen der Tier-Binary) und der Mikro-Ebene (`micro`; Achsen-Interface und Erfolgs-Parameter); `w/ma/mi` ist der registrierte Alias, und die drei Ebenen sind die Mess-Tooling-Haupt-Achse, die je Wahl in eine CacheEngineBuilder-Strecke einkompiliert wird (Abschnitt 3.4.6). FF2.a ist bejaht: Die Mess-Instrumente sind ein eigener Realm im Planer — das Last-Rahmenwerk als Meta-Meta-Achse, das Mess-Tooling als Haupt-Achse, die Mess-Kategorien als Unter-Achse (sechzehn Bausteine in der Registry; vorgesehen ist die Erhebung je Achsen-Interface-Parametersatz, deren Katalog je Achse nicht implementiert ist) —, stets präsent, sobald gemessen wird, und `binary-id-neutral`. Bewerkstelligt wird die Trennung durch den doppelten Wurzel-Schnitt: Der System-Realm wurzelt am CacheEngineBuilder, der Mess-Realm am Planer, und die Mess-Achse treibt über die System-Achsen-Wurzel die Freigabe der Mess-Quelle in Gestalt des CacheEngineBuilder-Typs — die Legenden-Kette Mess-Achse → CacheEngineBuilder-Typ → System-Achsen → Tier-Binary ist als Namensschema Code (Abschnitt 3.4.1). Beim Workload reicht der Treiber einen Standard-Workload je Modul durch, und ein Profil bringt seinen eigenen mit; vorgesehen ist die Voll-Permutation über alle verfügbaren Workloads, damit kein Entwurf nur unter seinem selbstgewählten Lastprofil gewinnt — der *Lastprofil-Selektionsverzerrung*² begegnet der Apparat mit der Vollmatrix aller Lastprofile gegen alle Lebewesen-Binaries, weil schon unauffällige Setup-Entscheidungen Messergebnisse systematisch verfälschen [MDHS09]; Umsetzungsstand: Die Voll-Permutation ist nicht implementiert.

Zu FF2.b (Organ-Achsen): Die widersprüchlichen Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe — eine Cache-Line je Knoten bei CSS- und CSB⁺-Bäumen [RR99, RR00] gegenüber sechzehn und mehr Cache-Lines bei Hankins und Patel [HP03] (Knoten von 512 Byte und größer bei 32-Byte-Cache-Lines) sowie mehreren je Knoten bei den Prefetching-B⁺-Bäumen von Chen, Gibbons und Mowry [CGM01] — werden nicht aus ihren Original-Setups übernommen, sondern achsen-isoliert unter sonst gleichen Bedingungen nachgemessen: Knotentyp, Speicher-Layout und Prefetch sind Organ-Achsen, die einzeln variiert werden, während System- und Mess-Achsen konstant bleiben; die Zahl ist eine Messung und liegt nicht vor. Zu FF2.c (Erweiterbarkeit): Ein neuer Prüfling gliedert sich als Lebewesen seiner Gattung ins Gattung ein — Gattung ist die erste Ebene (Map, Container, Graph), Gattung die zweite, und die Gattungs-Bau-Zulassung führt die Slot-Sätze je Gattung als übersetzungszeitliches Urteil —; er dockt am Prüf-Dock an, durchläuft die Drei-Stufen-Prüfung (Abschnitt 3.4.6), und sein Verbund mit der Cache-Engine führt einen der registrierten Namen: `Verbund1_CeOnly` (keine Prüflings-Direktive), `Verbund2_Replace` (die Prüflings-Achse ersetzt die Cache-Engine-Achse mit Rückfall), `Verbund2_Hybrid` (Cache-Engine und Prüfling hybrid je Prüfling; deklariert, nicht materialisiert — der Validator sperrt das Token `merge`, solange der Strategie-Wert nicht implementiert ist) oder `Verbund3_Union` (Vereinigung aller Varianten; vorbereitete Schnittstelle, Abschnitt 5.2.3). An der Binary bleibt der Verbund über die hierarchisch erweiterten Algorithmus-Namen und Flags der Organ-Zeile unterscheidbar — ein Prüflings-Organ führt sein Namensraum-Präfix im Eintrag —; eine eigene Stempel-Zeile braucht es nicht, und ein Versions-Suffix bezeichnet die Mischung nicht: Das Flag `e` bezeichnet einen Efficiency-Core-Algorithmus (Abschnitt 3.4.4). Ob der Prüfling eine bestehende Gattung trifft, eine neue begründet oder eine Mischform darstellt, entscheidet der Gattungs-Schnitt (Abschnitt 3.3.2); ein eigenes Konzept entstünde erst bei grundlegend anderem Basis-Achsen-Satz. Der

²Lastprofil-Selektionsverzerrung (workload selection bias): die Verzerrung eines Vergleichs dadurch, dass die Lastprofile so gewählt sind, dass ein Entwurf unter ihnen bevorzugt gut abschneidet; die Benchmark-Literatur verlangt darum eine breite, begründete Lastprofil-Menge [BGH⁺06, BDH⁺16].

in FF2.c erfragte Betriebsmodus, der die Entwurfsbestandteile genau einer Veröffentlichung als geschlossenes Achsen-Set isoliert anbietet, ist nicht implementiert und bleibt Gegenstand des Ausblicks (Abschnitt 6.3). Zu FF2.d (Workload-Stand der Technik): Den Stand der Technik der Workloads für Suchalgorithmen bilden die Schlüssel-Wert-Konvention YCSB mit ihren sechs Lastprofilen A–F (je ein Operations-Mix über einer Schlüssel-Zugriffsverteilung, von 50/50 Lesen/Aktualisieren Zipf-verteilt bis zu kurzen Bereichs-Scans und Read-Modify-Write), der Index-Benchmark SOSD, die TPC-Familie (TPC-C, TPC-DS), die Integration in reale Systeme (SuRF in RocksDB), reale String-Korpora (URLs, DNA, Wörterbuch-Terme) sowie SPEC CPU und CloudSuite bei den hardware-nahen Arbeiten (Abschnitt 2.3); diese Rahmenwerke erzeugen untereinander nicht unmittelbar vergleichbare Lasten, und jede Veröffentlichung wählt typischerweise das Lastprofil, unter dem ihr eigener Entwurf am besten abschneidet. Die Arbeit übernimmt daraus YCSB als das registrierte Last-Rahmenwerk (die Meta-Meta-Achse `load_framework` des Mess-Realms) mit der Laufzeit-Dimension `workload` über die sechs Token YCSB_A bis YCSB_F, geroutet je Permutation aus dem Traversal-Tag des Profils oder seinem `<expected_workload>`-Tag, und fährt sie über reale String-Datensätze mit Reproduzierbarkeits-Akte (`url`, `protein`, `tpcds-id`, `trec-terms`, `dna`, `xml`; Abschnitt 5.1.3), wobei `xml` nur eine Spezifikation ohne Prüfsumme besitzt, weil die Datei nicht lokal vorliegt (n/a); die übrigen Konventionen sind als Last-Rahmenwerk nicht registriert (n/a). Umsetzungsstand: Der Treiber fällt für profil-lose Module auf genau einen Workload zurück, die Workload-Option des Laufs; die Voll-Permutation über alle Lastprofile ist vorgesehen, aber nicht implementiert (Abschnitt 5.1.3). Zur Plattform-Verifikation: Eine Plattform gilt als verifiziert, wenn das Konformitäts-Gate je geladener Tier-Binary die siebzehn in Anhang F markierten `std::map`-Operationen (unter `-debug` zusätzlich den Standard-Testsatz des Tier-Interfaces) — die Operationsmenge seiner zwölf Kern-Prüfblöcke, darunter `at`, `contains`, `count`, `insert_or_assign`, `erase`, `clear`, `lower_bound` und `upper_bound` — gegen `std::map` als Orakel bestätigt und die Plattform-Probe die gemeldete Achsen-Teilmenge deckt; die fünf Antriebs-Operationen der Modulgrenze (`insert`, `lookup`, `erase`, `clear`, `size`) sind die ABI-Primitive, aus denen der Host die übrigen Operationen komponiert, nicht der Prüfumfang, und der Vollausbau auf die volle `std::map`-Schnittstelle ist nicht implementiert; die plattform-kalibrierten Schwellwerte liegen nicht vor, ihre Festlegung setzt die Messläufe voraus. Als *Beleg* dienen Abschnitt 3.4.6 (Drei-Stufen-Prüfung und Verbund; Struktur), Abschnitt 3.4 (Mess-Ebenen und Realm; Struktur), Abschnitt 3.3.2 (Gattungs-Schnitt; Struktur), Anhang F (Vertrags-Teilmenge; Test) sowie für FF2.d Abschnitt 2.3 (Workload-Konventionen; Literatur) und Abschnitt 5.1.3 (Workload-Routing und Akten; Struktur). Die *Folge*: FF2 strukturell beantwortet, FF2.c konzeptionell beantwortet, FF2.d beantwortet; der Isolations-Modus, die Zahl zu FF2.b und die Voll-Permutation der Workloads sind offen.

FF3 (Prüfling-Vergleich). Gefragt ist, ob der Prüfling PRT-ART gegenüber den acht Rang-1-Entwürfen — ART [LKN13], HOT [BZP⁺18], Masstree, CoCo-Trie, START, B²-Baum, Wormhole, SuRF — und der Rang-2/3-SOTA *nicht* durch asymptotische Neuheit, sondern durch bessere Mikroarchitektur-Eigenschaften gewinnt — höhere Cache-Line-Auslastung, weniger LLC- und dTLB-Misses, geringere Branch-Kosten, kleinere heiße Suchpfade — bei Punktsuche, Prefix-Lookup und Prefix-Enumeration, gemessen in den Pflicht-Messreihen A, B und C; FF3.a fragt nach der günstigsten lokalen Seitendarstellung je Verzweigungspunkt, FF3.b nach dem Compile-Time-Zwang. Der *Status*: FF3 ist offen, FF3.b beantwortet, FF3.a strukturell beantwortet (Vorgabe ohne Messbeleg). Der Vergleich liegt

nicht vor und setzt die ersten Mess-Läufe voraus; der Apparat dafür ist implementiert: das PRT-ART-Profil, die drei Pflicht-Messreihen — die Reihe wird mechanisch aus der Verbund-Stufe abgeleitet (Verbund1 und Verbund2 liefern Reihe A, Verbund3 Reihe B, Reihe C ist bau-übergreifend; Tabelle 3.8) —, der ExperimentDriver mit Auto-Pickup der 33 Profil-XML aus dem Profil-Verzeichnis und der ABI-konforme Adapter mit dem `std::map`-Kernvertrag als host-seitiger Kompositions-Schicht über dem Punkt-Such-Primitiv der Modulgrenze, flankiert von Einfügen und Löschen, mit der geordneten Iteration als additiver Scan-Fähigkeit (Anhang F). Das Konformitäts-Gate prüft die siebzehn in Anhang F markierten Operationen in zwölf Kern-Prüfblöcken gegen `std::map` als Orakel und unter `-debug` zusätzlich den Standard-Testsatz des Tier-Interfaces (generischer Gattungs-Teil und Genus-spezifischer Teil); der Vollausbau auf die volle Schnittstelle ist vorgesehen, aber nicht implementiert. Der Benachrichtigungs-Haken der Mess-Schicht ist ein Single-Slot-Hook je Achse, standardmäßig nicht einkompiliert (Opt-in), und kein Teil des Vertrags. Zu FF3.a: Punktseite — die Direktindex-Seiten `Array256` und `Array65535` — gegenüber Kompaktseite — die Vektor-Seiten `Vector<u8,u8>` und `Vector<u16,u16>` — je Verzweigungspunkt und die Umschaltregeln nach Belegungsdichte und Zugriffsart sind Teil des PRT-ART-Profils; der Prüfling schaltet nach festen Dichte-Schwellen von 25, 50 und 75 Prozent um (Abschnitt 3.5), und die plattform-kalibrierten Regeln sind Vorgaben ohne Messbeleg, solange die Läufe sie nicht belegen. FF3.b ist bejaht und zur Bauform geworden: Jede Achse gliedert sich in eine übersetzungszeit-fixierte *Haupt-Achse* und laufzeit-variable *Unter-Achsen* — die Haupt-Achsen sind statisch einkompiliert, bei Organ-Achsen als `binary_id`-Segment, bei System-Achsen als Stempel samt Bau-Versionskennung, während Unter-Achsen mit der Stufe `runtime` ihre Parameter ohne Neuübersetzung variieren; die Organ-Registry führt die Stufe je Unter-Achse, die System-Registry die Unter-Achsen-Gruppe `build_toolchain` der Komplex-Klammer (Abschnitt 3.4). Als *Beleg* dienen Abschnitt 3.5 (Prüfling und Seitendarstellung; Struktur), Abschnitt 3.2 (33 Profile; Struktur), Abschnitt 3.4 (Haupt- und Unter-Achse; Struktur) und Anhang F (Kernvertrag; Test). Die *Folge*: Die Zahl setzt die Reihe A voraus; die Hypothesen H1–H3 liefern die Auswerte-Kriterien.

Hypothesen und Zielpfad. Die Evaluations-Hypothesen H1–H3 (Abschnitt 5.1.1) sind vor dem Apparat formuliert worden; der über die Kapitel erarbeitete Zielpfad — Dreiteilung der Realms, Stempel-Identität, Mess-Ebenen — hat sie nicht als Fundament gelegt, sondern ordnet sie als Prüfsteine der Auswertung ein: H1 (Seitentyp-Kosten) und H3 (Verteilung der Wert-Ablage) werden in Reihe A je Lastprofil geprüft, H2 (Code-Qualität pro Quelle) über die tool-berechnete Qualitäts-Akte. Deren Datenbasis ist die gewichtete `cppcheck`-Befunddichte je tausend physischer Zeilen über die vendorten Originalquellen; die Akte führt 33 Einträge, davon 22 als n/a, weil eine Rekonstruktion keine vendorte Originalquelle besitzt, und PRT-ART besitzt kein Original-Repository (n/a). Die Hypothesenprüfung folgt der Vorgabe, Kriterium und Schwelle vor der Messung zu nennen [WRH⁺ 12], und die Wiederholungszahl je Zelle richtet sich nach der geforderten Konfidenz [KJ13]. H2 gilt als angenommen, wenn über die Vollprofile mit vendorter Originalquelle die Rangkorrelation nach Spearman zwischen Befunddichte und gemessenem Durchsatz je Lastprofil und Zelle (Plattform, Workload, Regime) negativ ist mit $|\rho| \geq 0,5$ bei $\alpha = 0,05$; als abgelehnt bei $|\rho| < 0,3$; dazwischen als nicht entschieden; unterhalb von acht Profilen mit Score ist H2 nicht prüfbar. Das Hand-Audit über sieben Bewertungs-Achsen bleibt die qualitative Einordnung. Die PRT-ART-Telemetrie-Metriken H1–H3 sind Mess-Organen des Prüflings, nicht diese Hypothesen (Abschnitt 5.1.1); und die Variante ChainRef der Wert-Ablage ist Gegenstand von H3, sie ist

kein Abbruchkriterium für H2.

FF4 (Compile-Time vs. Laufzeit). Gefragt ist, welche Achsen-Eigenschaften zur Übersetzungszeit fixiert werden müssen, um plattform-optimierte, provenienz-nachweisbare Binaries je Permutation zu erzeugen, und welche zur Laufzeit gewählt werden können bzw. müssen (bit-codierte Binary-Identität je Permutation, laufzeit-adaptive Layout-Wahl), ohne die Vergleichbarkeit (gleiche Compiler- und Flag-Basis) zu verletzen — und ob es durch die Messung möglich wird, ISA+OS-optimierte Binaries für eine Suchalgorithmus-Rekombination zu erzeugen, deren heuristische Optimierung perfekt auf ihre Laufzeitumgebung zugeschnitten ist; FF4.a fragt, ob sich der gesamte Ablauf — vom plattformbezogenen Bau aller Binary-Permutationen über den Messlauf und die Auswahl der besten gemessenen Binary oder Binary-Rekombination bis zur Auswertung in CSV/xlsx-Dateien, LaTeX und Diagrammen — durchgehend aus einer XML-Beschreibung des Experiments erzeugen lässt; die Erweiterung FF4.E fragt, ob die Grenze entlang der Achsen-Kategorien verläuft. Der *Status*: FF4 ist strukturell beantwortet, FF4.a strukturell beantwortet, FF4.E beantwortet. Die Grenze verläuft entlang der Kategorien: Die drei System-Haupt-Achsen `target_isa`, `operating_system` und `external_utils` samt ihrer Komplex-Klammer `build_target_complex` (Abschnitt 3.4.2; keine vierte Haupt-Achse) gehören zur Übersetzungszeit-Identität der Binary; sie erscheinen als System-Stempel-Zeile und Bau-Versionskennung, nie als `binary_id`-Segment. Die Provenienz sichert der Tier-Fingerprint: ein SHA-512 über elf Glieder (Fingerprint-Format 6), darunter das Overlay-Glied als ein SHA-512 über die konkatenierten Quelldateien des Overlay-Schnitts — je Achse die Dateimenge ihrer eigenen Implementierung plus die gemeinsame Anatomie-Substanz —, sodass eine reine Quelltext-Änderung eine neue Identität ergibt, und das Toolchain-Glied mit der Bau-Compiler-Kennung und der Optimierungs-Identität; der Planer-Fingerprint ist ein SHA-256 über sein eigenes Preimage. Die Abfrage `get_compiler()` einer Achse liefert dagegen den Original-Compiler der Veröffentlichung aus dem Manifest der vendorten Quelle — eine Herkunftsangabe, keine Bau-Kennung. Mess-Wahlen bleiben laufzeit-variabel; Layout- und Seiten-Selektion bleiben über die mixed-radix-Ordnung der `binary_id` (die statische Binary-Sicht) und die laufzeit-dynamische Cache-Line-Politik wählbar, ohne die gemeinsame Compiler- und Flag-Basis zu verletzen. Wo eine Achse beide Seiten der Grenze braucht, ist sie zweigeteilt: Last-Rahmenwerk, Telemetrie und Hardware-Erkennung laufen über die Laufzeit des Planers *und* die Übersetzungszeit des Builders — die Unter-Achsen-Freigabe der Vorstufe wird zur Haupt-Achsen-Annahme der Folgestufe (Abschnitt 3.4.1) —, und die Hardware-Erkennung setzt die Übersetzungszeit-Stempel der Tier-Binaries aus den zur Builder-Laufzeit erhobenen Ist-Eigenschaften (Abschnitt 3.4.3). Gestempelt wird stets die Plattform-Klasse, nie ein Messwert; der *Passungs-Stempel*, der die gestempelte Klasse bei jedem Mess-Lauf gegen das Laufzeit-Verdict der Hardware-Erkennung prüft, ist entworfen, aber nicht implementiert (Abschnitt 6.3). Die vorwärtsgerichtete Teilfrage, ob die Messung selbst ISA- und betriebssystem-optimierte Binaries für eine Rekombination zuschneiden kann, greift der Ausblick als Heuristik-Automation auf; die Betriebssystem-Dimension ist mit drei deklarierten Werten der System-Achse (`linux`, `windows`, `macos`, je mit eigener Probe) und den zwei Mess-Regimes im Messplan angelegt — das zweite Regime, ein immutablees Betriebssystem, ist entworfen, nicht umgesetzt (Abschnitt 5.1.4). Zu FF4.a (deklarative Kette): Die Kette ist durchgehend deklarativ angelegt und als zwei Binaries gebaut. Der Planer `comdare-experiment-planner` prüft die Anwender-XML rein lesend gegen die einkompilierten Registry-Kataloge und emittiert aus dem einen Director-Walk den determinis-

tischen Plan — zwei Läufe sind byte-gleich, der Plan-Kopf annotiert das Registry-Trio, und jeder Bau-Schritt weist den tatsächlich erzeugten Datei-Stamm aus derselben Orchestrator-Funktion aus, die auch der Bau aufruft —; der Mess-Treiber `comdare-messung-driver` emittiert die System-Permutationen und Tier-Bau-Aufträge seines freigegebenen Bau-Raums und fährt die Sieben-Phasen-Mess-Pipeline von `ENUMERATE` bis `PERSIST` (CSV und JSON); die nachgelagerte Neun-Stufen-Werkzeugkette rendert daraus die `xlsx`-Mappe, die `booktabs`-Tabellen, die `TikZ`-Diagramme und die Anhang-Dateien, die das Manuskript einbindet — eine XML-Quelle, ein Bau-Kanal, kein Schritt von Hand (Abschnitt 4.3.2, Abbildung 5.1). Die Auswahl der besten Binary oder Rekombination ist als Arbeitsmodus `compare` der Ablauf-Kette registriert und validierbar; sein `modus`-spezifischer Ablauf und der `release` sind vorgesehen, aber nicht implementiert — beide sind wählbare, validierbare Etiketten mit `release`-gleicher Bausemantik (Abschnitt 3.3.2). Umsetzungsstand: Die Kette ist in einem ersten durchgehenden Lauf nachgewiesen — die Smoke-Messreihe über 43 Permutationen im Anhang ist ihr Erzeugnis —; der headless Dienst, der XML-Erstellungsmodus, die `<publish>`-Sektion und die CSV-Baumform sind vorgesehen, aber nicht implementiert. Als *Beleg* dienen Abschnitt 3.4.2 (Komplex-Klammer; Struktur), Abschnitt 3.4.3 (zweigeteilte Erkennung; Struktur), Abschnitt 3.4.4 (Stempel-Zeilen; Struktur), Abschnitt 4.5 (Rückkehr-Sperren und Ordnungs-Prüfungen; Test) sowie für FF4.a Abschnitt 4.3.2 (Planer, Treiber und Mess-Pipeline; Struktur) und Abschnitt 3.4.6 (Kette von der XML zur Tier-Binary; Struktur). Die *Folge*: FF4 und FF4.a strukturell beantwortet; Heuristik-Automation, Passungs-Stempel sowie der Ablauf von `compare` und `release` offen.

Umsetzungsstand. Der vollständige Bau ist nicht durchgeführt; sein Umfang ist kein deklarierter fester Wert: Die Obergrenze der Binary-Identitäten je System-Permutation ist das Produkt der je Organ-Achse im XML-Experiment gewählten Auslegungen (mehrere je Achse, nicht an/aus), vom Planer aus dem XML-Experiment bestimmt und erst zur Ausführung bekannt; die Planungs-Simulation (Unterkommando `simulate`) rechnet heute dieses Produkt mal System-Permutationen arithmetisch, die Planer-Erweiterung, die Obergrenze und Batch-Größe eines Experiments als Grundlage der Last-Abschätzung korrekt erfasst, ist geplant und steht aus (die 131072 Organ-Permutationen des golden-Referenzkatalogs mit dem Katalog-Kommentar von rund 224 GB und rund 24 Stunden je Bau sind eine historische Schätzung unter der überholten Annahme zweier Auslegungen je Achse, kein Bau-Umfang); die End-to-End-Läufe des Mess-Treibers über die Zielplattformen (x86-64 mit AVX2 und AVX-512, aarch64 mit NEON und SVE³) liegen nicht vor, ebenso die Abgabe-Messung über den 320er-Katalog. Belegt mit Lauf sind zwei Teil-Durchführungen: der erste durchgehende Lauf der Kette von der Experiment-XML über Bau und Messung bis zum eingebundenen Anhang (Abschnitt 5.1.2) — ein Mess-Job der Smoke-Stufe auf der AMD-Zen-5-Maschine⁴ mit 10000 Operationen und 671,5 ns je Operation (p50 der `lookup`-Operation 750 ns) als realer Messwert des Weges — und der Kalibrierlauf auf der AMD-Maschine, der 250 von 250 Tier-Binaries gebaut und den Bau-Takt auf 0,521 s je Binary sowie den Platzbedarf auf rund 457 kB je Binary kalibriert hat; daraus folgt als Rechenbeispiel des historischen golden-Profiles (vier System-Permutationen mal 2^{17} , zwei Optimierungsstufen mal zwei SIMD-Optionen) ein Lagerbedarf

³NEON und SVE: die SIMD-Erweiterungen der ARM-Architektur — NEON mit fester 128-Bit-Breite, SVE (Scalable Vector Extension) mit implementierungsabhängiger Vektorbreite.

⁴Provenienz des Smoke-Laufs: Pipeline 15800, Mess-Job 377503, Host prod1 (Anhang A); ein vorausgehender erster durchgehender Lauf derselben Kette (Pipeline 15764, Job 376333) ergab bei 10000 Operationen 1199,047 ns je Operation, seine messende Maschine ist nicht dokumentiert und die Differenz nicht bewertet.

Frage	Teilfrage	Status	Beweis (Abschnitte; Belegart)	Rest
FF0	Hauptfrage: aktiv, messgetrieben gegen passiv, statisch	offen	3.4.1 (Struktur), 4.5 (Test), 5.1 (Messplan)	Zahl aus Reihe A nach dem vollständigen Bau
FF0.E	Zuordenbarkeit eines Gewinns zu Organ, System, Mess	strukturell beantwortet	3.4.1, 3.4.4 (Struktur)	Messung, die die Zuordnung ausübt
FF1	kanonische Zerlegung, Permutationsmatrix, Profile	strukturell beantwortet	3.4.1, 3.4.6 (Struktur), 4.4 (Test)	Lauf über alle achtzehn Achsen; Profil-Anschluss der Meta-Meta-Achse
FF1.a	Kategorien-Schnitt der Zerlegung	beantwortet	3.4.1 (Struktur), 4.5 (Test)	—
FF1.b	Organ-Kopplungen, ISA-/SIMD-Variation	offen (Messung)	3.4.1 (Struktur)	Cache-Line-Füllverhalten je System-Permutation
FF2	Mess-Pipeline, drei Reihen, drei Ebenen, Verifikation	strukturell beantwortet	3.4.6, 3.4 (Struktur)	Voll-Permutation der Workloads; kalibrierte Schwellwerte
FF2.a	Mess-Achsen als eigener Realm	beantwortet	3.4.1 (Struktur), 4.5 (Test)	Katalog der Erfolgs-Parameter je Achse
FF2.b	Knoten-/Cache-Line-Größe achsen-isoliert	strukturell beantwortet	3.4 (Struktur)	Messung über Knotentyp, Layout, Prefetch
FF2.c	Erweiterbarkeit, Eingliederung neuer Prüflinge	beantwortet (konzeptionell)	3.3.2, 3.4.6 (Struktur), F (Test)	Paper-Isolations-Modus; Materialisierung von Verbund2_Hybrid
FF2.d	Workload-Stand der Technik	beantwortet	2.3 (Literatur), 5.1.3 (Struktur)	Voll-Permutation der Workloads; nur YCSB als Last-Rahmenwerk registriert
FF3	PRT-ART gegen SOTA in den Reihen A–C	offen	3.5, 3.2 (Struktur)	Reihe A; Vollausbau des Vertrags
FF3.a	Seitendarstellung, Umschaltregeln	strukturell beantwortet (Vorgabe ohne Messbeleg)	3.5 (Struktur)	plattform-kalibrierte Regeln aus den Läufen
FF3.b	Compile-Time-Zwang	beantwortet	3.4 (Struktur), 4.4 (Test)	—
FF4	Übersetzungszeit gegen Laufzeit, Provenienz	strukturell beantwortet	3.4.4, 3.4.3 (Struktur), 4.5 (Test)	Passungs-Stempel; Heuristik-Automation
FF4.a	deklarative Kette von der XML bis zur Auswertung	strukturell beantwortet	4.3.2, 3.4.6 (Struktur)	Ablauf von <code>compare</code> und <code>release</code> ; Dienst, Erstellungsmodus, CSV-Baum
FF4.E	Grenze entlang der Achsen-Kategorien	beantwortet	3.4.2, 3.4.4 (Struktur)	—

Tabelle 6.1: Die Beweiskette je Haupt- und Teilfrage: Status in drei Stufen (beantwortet, strukturell beantwortet, offen), die stützenden Abschnitte mit ihrer Belegart (Struktur, Test, Messung; für FF2.d der Literaturbeleg) und der offene Rest ohne Zahl.

von rund 240 GB für 524288 Tier-Binaries — kein Bau-Umfang des Systems. Die Smoke-Reihe — 43 Permutationen in zwei Teilblöcken: 27 kartesische SIMD×Layout×Allokator-Kombinationen plus 16 über Knotenbreite, Pfadkompression, interne Suche und Telemetrie — liegt in Anhang A vor; ihre Provenienz ist der Datenstand, unter dem sie erhoben wurde: sieben variierte Achsen jener Zählung, von denen SIMD und Telemetrie in der kanonischen Ordnung keine Organ-Achsen mehr sind. Sie approximiert Achsen-Effekte teils über Wall-Clock-Proxys, und einzelne Effekte sind Instrumentierungs-Artefakte⁵ statt Organ-Effekte; die Vorbehalte je Reihe führt Tabelle A.10 (Abschnitt A.10). Die apparat-seitige Abnahme je Paket läuft über das Zwei-Gate-Protokoll (Abschnitt 4.4). Die *Folge*: FF0 und FF3 bleiben offen, solange Reihe A nicht gelaufen ist; alle achtzehn Achsen sind für die Variation vorgesehen (nicht gemessen).

6.2 Limitierungen

Die folgenden Grenzen beschreiben, was ein Anwender mit dem System nicht erreichen kann — abgesehen von den natürlichen und selbst gewählten Grenzen der Maschine, auf der er arbeitet; Vorbehalte einzelner Messreihen führt der Mess-Anhang (Tabelle A.10), Implementierungsstände das Kapitel Implementierung (Kapitel 4), und der Umsetzungsstand von vollständigem Bau und Smoke-Reihe steht in Abschnitt 6.1. Zwei Grenzen folgen aus dem Lager-Prinzip, die übrigen sind aus der Architektur abgeleitet: Maschinenspezifische empirische Optimierung ist der Wesenszug cache-bewusster Verfahren — ATLAS [WPD01] und FFTW [FJ98] vermessen die Maschine, bevor sie den Code wählen —, und cache-oblivious Verfahren [FLPR99] sind der Gegenpol ohne Maschinenbindung; ein Apparat, der die Maschinenbindung zum Prinzip macht, erbt ihre Kosten. Die Modul-Bodies sind keine Grenze: Der Codegenerator des Mess-Pfads emittiert je Permutation die achtzehn voll qualifizierten Achsen-Typen samt Umbrella-Include, der Alt-Pfad mit dem Prüflings-Stub ist durch eine strikte Sperre vom Mess-Pfad getrennt, und die Nullaussagen fest zugeordneter Pool-Familien sind explizit kodierte Nicht-Erhebung mit Grund (Abschnitt 4.3.2; Tabelle A.10).

- **Versionswechsel eines Achsen-Algorithmus:** Ändert der Anwender die Version eines Achsen-Algorithmus, wird jede Tier-Binary, deren Overlay-Schnitt die Datei enthält, zu einer neuen Identität — das Overlay-Glied des Fingerprints hasht alle Quellen des Schnitts, und der Versions-Lock verlangt für jede Änderung an einer verzeichneten Datei den bewussten Regenerations-Commit (Abschnitt 4.4); der Lager-Bestand dieser Binaries samt Messung und Auswertung ist neu zu erzeugen, denn der Fingerprint des CacheEngineBuilders ist Provenienz, kein Wiederverwendungs-Kriterium. Was der Anwender nicht kann: einen einzelnen Algorithmus austauschen und die Messungen der übrigen Binaries behalten.
- **Lager je Maschinentyp und Hardware-Konfiguration:** Messwerte sind an die Hardware-Identität der messenden Maschine geschlüsselt — Bestand 2 des Lagers ist über den Fingerprint der vermessenen Binary und die Hardware-Identität adressiert, die Plattform-Zelle daneben geklammert (Abschnitt 3.4.5) —; das gesamte Lager ist je Maschinentyp und Hardware-Konfiguration einzeln zu messen. Was der Anwender nicht kann: Messwerte einer Maschine auf eine andere übertragen.

⁵Instrumentierungs-Artefakt (probe effect): die Veränderung des beobachteten Verhaltens durch das Mess-Instrument selbst [Gai86].

Das ist der Preis der maschinenspezifischen Optimierung (ATLAS [WPD01], FFTW [FJ98]) und der Grund für die drei Maschinen-Signaturen der System-Registry.

- **Deckung der Hardware-Zähler je Mikroarchitektur:** Im Mess-Pfad ist die Zähler-Erhebung aktiv — die emittierten Mess-Jobs setzen den PMC-Schalter, nur der Test-Bau baut ohne Hardware-Zugriff. Erhoben werden vier generische Zähler (L1-, Last-Level-, dTLB- und Branch-Misses über `perf_event_open`⁶) und zwei modell-gebundene Rohzähler (L2-Demand-Misses und Fills aus dem Cache eines fremden CCX als Kohärenz-Beobachtbare), Letztere nur, wo die Kodierung durch Messung gegengeprüft ist — AMD Zen 5; ohne Katalog-Eintrag führen die Felder eine explizit kodierte Nicht-Erhebung⁷ (n/a), nie eine Null, und für Intel gibt es keine Rohbelegung, bis ein Intel-Host die Gegenprüfung liefert. Das Last-Level-Ereignis scheitert auf Zen 5 beim Öffnen (keine generische Abbildung), RAPL⁸ ist best-effort und ohne Zonen-Leserecht leer, und IPC ist als Kategorie deklariert, nicht erhoben. In der Smoke-Reihe sind die Hardware-Spalten nicht erhoben (n/a); die tabellierten Nullen der Anhang-Tabellen sind Darstellung, nicht Messung (Tabelle A.10, Zeile 1). Was der Anwender nicht kann: L2- und Kohärenz-Zähler auf anderen Modellen als Zen 5 erhalten (Abbildung 3.13).
- **Hardware-Passung ohne automatische Prüfung:** Die Hardware-Erkennung liegt in drei Funktionen vor: der Erhebung (die ISA-Komplex-Achse mit SPD-/XMP-Deklaration der Speichertaktung und die Planer-Probe, die je Lauf den Host nach seinen Zählern fragt), der Übersetzungszeit-Factory (die Tier-Stempel aus den Ist-Eigenschaften) und dem Laufzeit-Verdikt; die Maschinen-Deklarationen der System-Registry sind Deklarationen mit konservierter Quelle (`dmidecode`). Nicht vorhanden sind die Anbindung an Runner und Prüf-Dock, der additive Ergebnis-Rückschrieb, der Provenienz-Roundtrip der Deklarationen und der Passungs-Stempel. Was der Anwender nicht kann: die Passung zwischen Maschine und Binary automatisch prüfen lassen (Abschnitte 3.4.3 und 4.5).
- **Ausgabe ohne Blatt-Auffächerung:** Die Ergebnis-Mappe entsteht bedingungslos im Speicher; `xlsx` ist der Standard, die CSV-Form ihr wählbares, immer aus ihr erzeugtes Gegenstück — kein einzelnes Kind-Dokument, sondern ein geordneter Baum aus CSV-Dateien mit einem Ordner je Blatt —, beide zusammen sind wählbar (Abschnitt 5.2.1). Die Mappe besitzt einen konstanten Blattschlüssel — alle Zeilen in einem Blatt —; die Auffächerung in ein Blatt je Unter-Achsen-Permutation plus Info-Blatt ist als Filterkette der Auswertung vorgesehen, aber nicht implementiert; die CSV-Projektion ist eine flache Datei, die Baumform nicht implementiert. Was der Anwender nicht kann: Ergebnisse je Unter-Achsen-Permutation als eigene Blätter erhalten oder die CSV-Ablage als Baum erhalten.
- **Permutationsraum und Budget:** Ein vollständiger Bau des golden-Referenzkatalogs kostet je System-Permutation nach der Schätzung des Katalogs rund 224 GB und rund 24 Stunden, nach der Kalibrierung rund 457 kB und 0,521 s je Tier-Binary (Abschnitt 6.1); der Planer rechnet die Mächtigkeit je Profil und bietet im Subkommando `simulate` Budget-Eingänge (Bytes je

⁶`perf_event_open(2)`: der Linux-Systemaufruf, der einen Hardware- oder Software-Zähler als Dateideskriptor öffnet; generische Ereignisse bildet der Kernel je Mikroarchitektur ab, modellspezifische verlangen eine rohe Kodierung.

⁷Explizit kodierte Nicht-Erhebung: Der fehlende Wert ist ein eigener Zustand (n/a mit Grund), nie eine Null; die Statistik spricht von geplant fehlenden Daten, deren Fehlmechanismus bekannt ist und darum keine Verzerrung einträgt [GTOC06, Gra09].

⁸RAPL (Running Average Power Limit): die Energie-Zähler der Prozessor-Pakete, unter Linux über die `powercap-sysfs`-Zone lesbar; die Zone ist oft nur mit Root-Rechten lesbar.

Binary, Lager-Budget, Mess-Teilmenge, Kandidaten). Was der Anwender nicht kann: den vollen Produktraum aller Achsen-Kataloge materialisieren — er wählt Kataloge oder Teilmengen, und die Wiederholungszahl je Messung bleibt eine Abwägung von Aufwand und Konfidenz [KJ13].

- **Kein Laufzeit-Wechsel zwischen Tier-Binaries:** Die Hybrid-Stufe ist als Klassifikation, Dock-Schicht, XML-Parser und Binary-Proxy implementiert (Abschnitt 4.5.1); der `.so`-Export der Hybrid-Stufe, die Eviction und der Router sind nicht implementiert — Reroute und Router setzen Messkurven voraus. Was der Anwender nicht kann: ein Hybrid-Tier betreiben, das zur Laufzeit zwischen vorgeladenen Binaries wechselt; auch die Zustands-Übergabe beim Wechsel — der Memento-Abgleich mit den zwei Release-Artefakten `single` und `hybrid` im Lager — ist entworfen, nicht implementiert.
- **Betriebssystem-Regime:** Die System-Achse `operating_system` ist mit `linux`, `windows` und `macos` deklariert und besitzt je eine Probe; gemessen wird unter root-Linux mit vollem Zähler-Zugriff, das zweite Regime (immutable Betriebssystem) ist entworfen, nicht umgesetzt (Abschnitt 5.1.4), und ein Windows-Zählertreiber liegt nicht vor (Tabelle A.10, Zeile 1). Was der Anwender nicht kann: Messreihen unter Windows oder macOS oder ohne `perf`-Rechte (`perf_event_paranoid`, RAPL-Zonen) erheben.
- **Keine isolierte Nachmessung einer einzelnen Veröffentlichung:** Ein Betriebsmodus, der die Entwurfsbestandteile genau einer Veröffentlichung als geschlossenes Achsen-Set anbietet, ist nicht implementiert; je Veröffentlichung existieren das Profil-XML (33) und der Ersetzungs-Verbund, nicht die Isolation gegen alle übrigen Achsen. Was der Anwender nicht kann: eine Veröffentlichung originalgetreu isoliert durchmessen (Abschnitt 6.3).
- **Keine automatische Auswahl der besten Konfiguration:** Die heuristik-gesteuerte Vollautomatisierung (Klassifikator und Binary-Packager) ist nicht implementiert; vorhanden sind das messwert-getriebene Inkrement der Binary-Auswahl (Rangierung je Metrik aus der Auswertungs-Ausgabe) und die laufzeit-dynamische Cache-Line-Politik. Was der Anwender nicht kann: aus Lastprofil- und Hardware-Merkmalen automatisch eine Achsen-Konfiguration empfehlen lassen; die Empfehlung bleibt manuelle Ableitung aus der Auswertung (Abschnitt 6.3).

6.3 Ausblick

Kurzfristig stehen die folgenden Schritte an; implementierte Teile treten mit ihrem offenen Rest auf, entworfene Teile mit ihrem Umsetzungsstand:

- (1) der vollständige Bau nach dem vom Planer aus dem XML-Experiment bestimmten Umfang (Abschnitt 5.1.6) und die Abgabe-Messung über den 320er-Katalog;
- (2) der End-to-End-Lauf der drei Pflicht-Messreihen auf den registrierten Messmaschinen samt Erzeugung der Mess-Diagramme;
- (3) der Lager-Vollausbau: die Binaries *mit* und *ohne* Messfühler als zwei eingelagerte Release-Artefakte und die synthetisierten Funktions-Kurven als Dateien — die Messung ist immer eingeschaltet, im Lager wird nur Fehlendes nachgemessen (Abschnitt 3.4.5); der Ausbau ist nicht implementiert;

- (4) die Hardware-Erkennung: Anbindung an Runner und Prüf-Dock, Ergebnis-Rückschrieb, Provenienz-Roundtrip und Passungs-Stempel;
- (5) die Blatt-Auffächerung der *xlsx*-Mappe je Unter-Achsen-Permutation plus Info-Blatt — der Mappen-Writer ist implementiert, die Auffächerung ist nicht implementiert;
- (6) die Gegenprüfung der Intel-Rohzähler auf *prod2*, damit L2- und Kohärenz-Zähler auch dort erhoben werden;
- (7) das zweite Betriebssystem-Regime (immutables Betriebssystem) und der Windows-Zählertreiber;
- (8) der Paper-Isolations-Modus aus *FF2.c*: ein Betriebsmodus, der die Entwurfsbestandteile genau einer Veröffentlichung als geschlossenes Achsen-Set isoliert anbietet, um sie — bei freistehenden Mess- und System-Achsen — originalgetreu durchzumessen;
- (9) der Profil-Anschluss der Organ-Meta-Meta-Achse *disk_io* und der Permutationsbau über die Scheduling-Unter-Achse;
- (10) die Voll-Permutation der Workloads gegen alle Lebewesen-Binaries (*FF2*);
- (11) die Auswertung des H2-Kriteriums nach dem Lauf der Reihe A.

Heuristik-Automation und Binary-Delivery. Der in der Aufgabenstellung (Teilaufgabe 7) geforderte Mechanismus, der aus den Messergebnissen automatisch die beste Konfiguration ableitet, ist in seiner heuristik-gesteuerten *Voll*automatisierung nicht implementiert und Gegenstand künftiger Arbeit; ein erstes Inkrement der automatischen Auswahl und des Binary-Versands liegt als eigenständiges Werkzeug (*best_binary_selector*) vor, das je Metrik die beste Permutation aus der Auswertungs-Ausgabe rangiert und ausliefert — die *xlsx*-Mappe ist die Ausgabe und der Standard, die CSV-Form ihr aus ihr erzeugter, geordneter Baum aus CSV-Dateien (ein Ordner je Blatt, kein einzelnes Kind-Dokument), und das Werkzeug liest die CSV-Projektion, die die Naht als eine flache Datei schreibt (die Baumform ist vorgesehen, aber nicht implementiert). Neben der Auswahl (Zusage 1) umfasst derselbe Werkzeug-Strang die laufzeit-dynamische Cache-Line-Anpassung (Zusage 2), die für die Cache-Line-Bewusstheit der Arbeit einschlägig ist: Aus einem ausgewerteten Profil-Aggregat leitet der Politik-Selektor die Cache-Line-Politik ab. Der Vollausbau setzt drei Komponenten voraus; die zweite liegt vor, und zur ersten wie zur dritten stellt das Werkzeug ein erstes, rein messwert-getriebenes Inkrement: (i) einen *Heuristik-/ML-Klassifikator*, der aus den Lastprofil- und Achsen-Features (Operationsmix, Schlüssel-Charakteristik, Hardware-Kennung) auf die empfohlene Achsen-Konfiguration abbildet; (ii) den implementierten *XML-Parser* (Import und Export samt Profil-Lader) für die als XML-Lastprofil-Ergebnis abgelegten, je Architekturfokus wiederverwendbaren Heuristik-Richtlinien; und (iii) einen *Binary-Packager*, der den so ausgewählten Gesamtalgorithmus als eigenständige Binary hinter dem einheitlichen Suchalgorithmus-Interface ausliefert. Voraussetzung für alle drei ist, dass die End-to-End-Messreihen vollständig vorliegen — erst die belastbare Datenbasis über alle Permutationen und Lastprofile macht eine messgetriebene, automatisch gesteuerte Auswahl sinnvoll; ohne diese Automatisierung werden die Konfigurations-Empfehlungen des Auswertungs-Schemas aus Abschnitt 5.2 nach Vorliegen der Daten manuell abgeleitet.

Mittel- bis langfristig: die Messung der PIM-Allokator-Familie auf PIM-Hardware⁹ — der Allokator A16 ist als Achsen-Baustein integriert und fällt ohne PIM-Hardware auf die portable ausgerichtete Allokation zurück [VIA26] —; die formale Verifikation ausgewählter Permutations-Klassen im Stil von StarMalloc [RFP24]¹⁰ — die Allokator-Familie A13 ist integriert, die Verifikation selbst ist offen —; die Übertragung des Anatomie-Prinzips auf Beschleuniger-Speicherhierarchien (GPU), ohne Zusage einer Cluster-Verfügbarkeit; und die Hybrid-Komponenten Live-Ranking, Zustands-Übergabe und Messfehler-Interpolation (Abschnitt 3.6.2). Die Vier-Repository-Architektur (Abschnitt 4.1) — Cache-Engine, Prüfling, Super-Repository, Manuskript — erlaubt, weitere Prüflinge analog zu PRT-ART einzubringen, ohne die Bausteine-Bibliothek zu ändern: Der Prüflings-Kontrakt der Bau-Konfiguration prüft den Code-Weg und den Registry-Weg gegeneinander, damit beide Hälften der Integration auf denselben Ort zeigen. Über die hier behandelten baumartigen Suchstrukturen hinaus ist das Anatomie-Prinzip über höherwertige Abstraktionen auf die weiteren Strukturklassen aus Abschnitt 2.1 erweiterbar — hash-basierte, räumliche und flache Strukturen —, sodass derselbe Mess- und Permutationsapparat ohne Neubau auf neue Klassen angewendet werden kann; darin liegt ein wesentlicher Hebel für künftigen experimentellen Fortschritt.

⁹PIM (Processing-in-Memory): Speicherbausteine mit eigenen Rechenwerken, etwa UPMEM-DPUs oder Samsungs HBM-PIM; ein PIM-Allokator verlangt solche Hardware.

¹⁰StarMalloc ist in F* mit dem Rahmenwerk Steel verifiziert, einer Separationslogik für nebenläufige Programme; die Verifikation eines Achsen-Bausteins in diesem Stil ist Ausblick.

Literaturverzeichnis

- [ABKS13] APEL, Sven ; BATORY, Don ; KÄSTNER, Christian ; SAAKE, Gunter: *Feature-Oriented Software Product Lines: Concepts and Implementation*. 1. Springer, 2013
- [ACK⁺04] AGRAWAL, Sanjay ; CHAUDHURI, Surajit ; KOLLÁR, Lubor ; MARATHE, Arunprasad P. ; NARASAYYA, Vivek R. ; SYAMALA, Manoj: Database Tuning Advisor for Microsoft SQL Server 2005. In: *30th International Conference on Very Large Data Bases (VLDB)*, 2004, S. 1110–1121
- [ADHW99] AILAMAKI, Anastassia ; DEWITT, David J. ; HILL, Mark D. ; WOOD, David A.: DBMSs on a Modern Processor: Where Does Time Go? In: *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, 1999, S. 266–277
- [Adv23] ADVANCED MICRO DEVICES. *Software Optimization Guide for the AMD Zen Microarchitecture (Family 19h)*. Advanced Micro Devices, Santa Clara, CA, USA. 2023
- [AKLS15] AIGNER, Christoph ; KIRSCH, Christoph ; LIPPAUTZ, Michael ; SOKOLOVA, Ana: Fast, Multicore-Scalable, Low-Fragmentation Memory Allocation. In: *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2015, S. 513–531
- [AKV⁺14] ANSEL, Jason ; KAMIL, Shoaib ; VEERAMACHANENI, Kalyan ; RAGAN-KELLEY, Jonathan ; BOSBOOM, Jeffrey ; O'REILLY, Una-May ; AMARASINGHE, Saman: OpenTuner: An Extensible Framework for Program Autotuning. In: *23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2014, S. 303–316
- [Ale01] ALEXANDRESCU, Andrei: *Modern C++ Design: Generic Programming and Design Patterns Applied*. 1. Addison-Wesley, 2001
- [AMH08] ABADI, Daniel J. ; MADDEN, Samuel R. ; HACHEM, Nabil: Column-Stores vs. Row-Stores: How Different Are They Really? In: *ACM SIGMOD International Conference on Management of Data*, 2008, S. 967–980
- [Apa26] APACHE SOFTWARE FOUNDATION. *Maven — POM Reference*. <https://maven.apache.org/pom.html>, abgerufen am 01.08.2026. 2026
- [APGZ17] AKEN, Dana V. ; PAVLO, Andrew ; GORDON, Geoffrey J. ; ZHANG, Bohan: Automatic Database Management System Tuning Through Large-scale Machine Learning. In: *ACM SIGMOD International Conference on Management of Data*, 2017, S. 1009–1024
- [APK⁺22] ANDERSCH, Michael ; PALMER, Greg ; KRASHINSKY, Ronny ; STAM, Nick ; MEHTA, Vishal ; BRITO, Gonzalo ; RAMASWAMY, Sridhar. *NVIDIA Hopper Architecture In-Depth*.

- NVIDIA Technical Blog, 22.03.2022, <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>, abgerufen am 19.09.2026. 2022
- [App24] APPLE INC. *Apple Silicon CPU Optimization Guide*. Apple Inc., Cupertino, CA, USA, Version 4. 2024
- [APV02] ARGE, Lars ; PROCOPIUC, Octavian ; VITTER, Jeffrey S.: Implementing I/O-efficient Data Structures Using TPIE. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 88–100
- [Arm24] ARM LIMITED. *Arm Architecture Reference Manual for A-profile Architecture (DDI 0487)*. Arm Limited, Cambridge, UK, <https://developer.arm.com/documentation/ddi0487/latest>, abgerufen am 18.09.2026. 2024
- [Aßm03] ASSMANN, Uwe: *Invasive Software Composition*. 1. Springer, 2003
- [AV88] AGGARWAL, Alok ; VITTER, Jeffrey S.: The Input/Output Complexity of Sorting and Related Problems. In: *Communications of the ACM* 31 (1988), Nr. 9, S. 1116–1127
- [AVL62] ADELSON-VELSKY, Georgy ; LANDIS, Evgenii: An Algorithm for the Organization of Information. In: *Doklady Akademii Nauk SSSR* 146 (1962), S. 263–266
- [BA01] BONWICK, Jeff ; ADAMS, Jonathan: Magazines and Vmem: Extending the Slab Allocator to Many CPUs and Arbitrary Resources. In: *Proceedings of the USENIX Annual Technical Conference (ATC)* USENIX, 2001
- [Bay72] BAYER, Rudolf: Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms. In: *Acta Informatica* 1 (1972), Nr. 4, S. 290–306
- [BBB⁺11] BINKERT, Nathan ; BECKMANN, Bradford ; BLACK, Gabriel ; REINHARDT, Steven K. ; SAIDI, Ali ; BASU, Arkaprava ; HESTNESS, Joel ; HOWER, Derek R. ; KRISHNA, Tushar ; SARDASHTI, Somayeh ; SEN, Rathijit ; SEWELL, Korey ; SHOAIB, Muhammad ; VAISH, Nilay ; HILL, Mark D. ; WOOD, David A.: The gem5 Simulator. In: *ACM SIGARCH Computer Architecture News* 39 (2011), Nr. 2, S. 1–7
- [BBF09] BLAIR, Gordon ; BENCOMO, Nelly ; FRANCE, Robert B.: Models@run.time. In: *Computer* 42 (2009), Nr. 10, S. 22–27
- [BCD⁺02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin ; ZITO, Jack: Two Simplified Algorithms for Maintaining Order in a List. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 152–164
- [BCDFC02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Scanning and Traversing: Maintaining Data for Traversals in a Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 139–151

- [BDFC00] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *Proceedings of the 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2000, S. 399–409
- [BDFC02] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Efficient Tree Layout in a Multilevel Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 165–173
- [BDFC05] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *SIAM Journal on Computing* 35 (2005), Nr. 2, S. 341–358
- [BDG⁺18] BALAPRAKASH, Prasanna ; DONGARRA, Jack ; GAMBLIN, Todd ; HALL, Mary ; HOLLINGSWORTH, Jeffrey K. ; NORRIS, Boyana ; VUDUC, Richard: Autotuning in High-Performance Computing Applications. In: *Proceedings of the IEEE* 106 (2018), Nr. 11, S. 2068–2083
- [BDH⁺16] BLACKBURN, Stephen M. ; DIWAN, Amer ; HAUSWIRTH, Matthias ; SWEENEY, Peter F. ; AMARAL, José N. ; BRECHT, Tim ; BULEJ, Lubomír ; CLICK, Cliff ; EECKHOUT, Lieven ; FISCHMEISTER, Sebastian ; FRAMPTON, Daniel ; HENDREN, Laurie J. ; HIND, Michael ; HOSKING, Antony L. ; JONES, Richard E. ; KALIBERA, Tomas ; KEYNES, Nathan ; NYSTROM, Nathaniel ; ZELLER, Andreas: The Truth, The Whole Truth, and Nothing But the Truth: A Pragmatic Guide to Assessing Empirical Evaluations. In: *ACM Transactions on Programming Languages and Systems* 38 (2016), Nr. 4, S. 1–20
- [Ben75] BENTLEY, Jon L.: Multidimensional Binary Search Trees Used for Associative Searching. In: *Communications of the ACM* 18 (1975), Nr. 9, S. 509–517
- [BFTV24] BOFFA, Antonio ; FERRAGINA, Paolo ; TOSONI, Francesco ; VINCIGUERRA, Giorgio: CoCo-trie: Data-aware Compression and Indexing of Strings. In: *Information Systems* 120 (2024), S. 102316
- [BGH⁺06] BLACKBURN, Stephen M. ; GARNER, Robin ; HOFFMANN, Chris ; KHANG, Asjad M. ; MCKINLEY, Kathryn S. ; BENTZUR, Rotem ; DIWAN, Amer ; FEINBERG, Daniel ; FRAMPTON, Daniel ; GUYER, Samuel Z. ; HIRZEL, Martin ; HOSKING, Antony ; JUMP, Maria ; LEE, Han ; MOSS, J. Eliot B. ; PHANSALKAR, Aashish ; STEFANOVIĆ, Darko ; VANDRUNEN, Thomas ; VON DINCKLAGE, Daniel ; WIEDERMANN, Ben: The DaCapo Benchmarks: Java Benchmarking Development and Analysis. In: *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 2006, S. 169–190
- [BH23] BERTHOLD, Johannes ; HABICH, Dirk: VAMPIR/OTF2 Profiling and Performance Analysis for Database Workloads. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, 2023. – Poster presentation
- [Blo70] BLOOM, Burton H.: Space/Time Trade-offs in Hash Coding with Allowable Errors. In: *Communications of the ACM* 13 (1970), Nr. 7, S. 422–426

- [BM72] BAYER, Rudolf ; MCCREIGHT, Edward M.: Organization and Maintenance of Large Ordered Indexes. In: *Acta Informatica* 1 (1972), Nr. 3, S. 173–189
- [BMBW00] BERGER, Emery D. ; MCKINLEY, Kathryn S. ; BLUMOFÉ, Robert D. ; WILSON, Paul R.: Hoard: A Scalable Memory Allocator for Multithreaded Applications. In: *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*, ACM, 2000, S. 117–128
- [BO92] BATORY, Don ; O'MALLEY, Sean: The Design and Implementation of Hierarchical Software Systems with Reusable Components. In: *ACM Transactions on Software Engineering and Methodology* 1 (1992), Nr. 4, S. 355–398
- [Bon94] BONWICK, Jeff: The Slab Allocator: An Object-Caching Kernel Memory Allocator. In: *Proceedings of the USENIX Summer 1994 Technical Conference* USENIX, 1994, S. 87–98
- [Bri59] DE LA BRIANDAIS, René: File Searching Using Variable Length Keys. In: *Western Joint Computer Conference*, 1959, S. 295–298
- [BSB⁺23] BERTHOLD, A. ; SCHMIDT, L. ; BARTUSCHKA, A. ; LEHNER, W. ; HABICH, D. ; SCHIRMEIER, H. *VAMPIR: Virtualized Non-Functional Memory Properties for Data-Pipeline Scheduling*. Poster, 29th ACM Symposium on Operating Systems Principles (SOSP 2023), 24.10.2023; Posterliste <https://sosp2023.mpi-sws.org/cfpo.html>, Autorenliste nach der Publikationsliste der Fakultät Informatik der TU Dresden, beide abgerufen am 19.09.2026. 2023
- [BVA21] BOYARINOV, Konstantin ; VINOGRADOV, Sergey ; ARUTYUNYAN, Ruslan. *Heterogeneous erasure overloads for associative containers*. ISO/IEC JTC1/SC22/WG21 Dokument P2077R3, 2021-10-15, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2077r3.html>, abgerufen am 19.09.2026. 2021
- [BVA23] BOYARINOV, Konstantin ; VINOGRADOV, Sergey ; ARUTYUNYAN, Ruslan. *Extending associative containers with the remaining heterogeneous overloads*. ISO/IEC JTC1/SC22/WG21 Dokument P2363R5, 2023-02-10, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2363r5.html>, abgerufen am 19.09.2026. 2023
- [BZP⁺18] BINNA, Robert ; ZANGERLE, Eva ; PICHL, Martin ; SPECHT, Günther ; LEIS, Viktor: HOT: A Height Optimized Trie Index for Main-Memory Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 521–534
- [CB13] CURTSINGER, Charlie ; BERGER, Emery D.: STABILIZER: Statistically Sound Performance Evaluation. In: *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013, S. 219–228
- [CE00] CZARNECKI, Krzysztof ; EISENECKER, Ulrich W.: *Generative Programming: Methods, Tools, and Applications*. 1. Addison-Wesley, 2000

- [CGM01] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Improving Index Performance through Prefetching. In: *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, ACM, 2001, S. 235–246
- [CGM02] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Fractal Prefetching B+-Trees. In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, ACM, 2002, S. 157–168
- [CHL99] CHILIMBI, Trishul M. ; HILL, Mark D. ; LARUS, James R.: Cache-Conscious Structure Layout. In: *Proceedings of the ACM SIGPLAN 1999 Conference on Programming Language Design and Implementation (PLDI)*, 1999, S. 1–12
- [CHP71] COURTOIS, Pierre-Jacques ; HEYMANS, Frans ; PARNAS, David L.: Concurrent Control with “Readers” and “Writers”. In: *Communications of the ACM* 14 (1971), Nr. 10, S. 667–668
- [Cli93] CLIFF, Norman: Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. In: *Psychological Bulletin* 114 (1993), Nr. 3, S. 494–509
- [CLP22] CROTTY, Andrew ; LEIS, Viktor ; PAVLO, Andrew: Are You Sure You Want to Use MMAP in Your Database Management System? In: *12th Conference on Innovative Data Systems Research (CIDR 2022)*, Chaminade, CA, USA, January 9-12, 2022, www.cidrdb.org, 2022
- [CLP23] CHOCKCHOWWAT, Supawit ; LIU, Wenjie ; PARK, Yongjoo: AirIndex: Versatile Index Tuning Through Data and Storage. In: *Proceedings of the ACM on Management of Data* 1 (2023), Nr. 3, S. 204:1–204:26. – Artikel-Nr. 204; präsentiert auf der ACM-SIGMOD-Konferenz 2024
- [Com79] COMER, Douglas: The Ubiquitous B-Tree. In: *ACM Computing Surveys* 11 (1979), Nr. 2, S. 121–137
- [Cop95] COPLIEN, James O.: Curiously Recurring Template Patterns. In: *C++ Report* (1995), Februar, S. 24–27. – <https://dl.acm.org/doi/10.5555/229227.229229>
- [cpp26a] CPPREFERENCE.COM. *std::map* — Online-Referenz zur C++-Standardbibliothek. <https://en.cppreference.com/w/cpp/container/map>, abgerufen am 19.09.2026. 2026
- [cpp26b] CPPREFERENCE.COM. *std::vector* — Online-Referenz zur C++-Standardbibliothek. <https://en.cppreference.com/w/cpp/container/vector>, abgerufen am 19.09.2026. 2026
- [Cre13] CREATIVE COMMONS. *Creative Commons Attribution 4.0 International Public License*. Legal Code, Version 4.0 vom 25.11.2013, <https://creativecommons.org/licenses/by/4.0/legalcode>, abgerufen am 19.09.2026. 2013
- [CST⁺10] COOPER, Brian F. ; SILBERSTEIN, Adam ; TAM, Erwin ; RAMAKRISHNAN, Raghu ; SEARS, Russell: Benchmarking Cloud Serving Systems with YCSB. In: *1st ACM Symposium on Cloud Computing (SoCC)*, 2010, S. 143–154

- [Des14] DESROCHERS, Cameron. *moodycamel::ConcurrentQueue: A Fast Multi-Producer, Multi-Consumer Lock-Free Queue for C++*. <https://github.com/cameron314/concurrentqueue>. 2014
- [DGH⁺10] DAVID, Howard ; GORBATOV, Eugene ; HANE BUTTE, Ulf R. ; KHANNA, Rahul ; LE, Christian: RAPL: Memory Power Estimation and Capping. In: *Proceedings of the 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, 2010, S. 189–194
- [Dij65] DIJKSTRA, Edsger W.: Solution of a Problem in Concurrent Programming Control. In: *Communications of the ACM* 8 (1965), Nr. 9, S. 569
- [Dim19] DIMOV, Peter. *Boost.Mp11: A C++11 metaprogramming library*. Dokumentation der Boost-Bibliothek Mp11 (Copyright 2017–2019), <https://www.boost.org/doc/libs/release/libs/mp11/doc/html/mp11.html>, abgerufen am 19.09.2026. 2019
- [Dio19] DIONNE, Louis. *Making std::vector constexpr*. ISO/IEC JTC1/SC22/WG21 Dokument P1004R2, 2019-07-19, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1004r2.pdf>, abgerufen am 19.09.2026. 2019
- [Dis24] DISTRIBUTED MANAGEMENT TASK FORCE: System Management BIOS (SMBIOS) Reference Specification, Version 3.8.0 / DMTF. 2024 (DSP0134). – Forschungsbericht. https://www.dmtf.org/sites/default/files/standards/documents/DSP0134_3.8.0.pdf, abgerufen am 19.09.2026
- [DLR77] DEMPSTER, Arthur P. ; LAIRD, Nan M. ; RUBIN, Donald B.: Maximum Likelihood from Incomplete Data via the EM Algorithm. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 39 (1977), Nr. 1, S. 1–22
- [DPT⁺13] DEBRABANT, Justin ; PAVLO, Andrew ; TU, Stephen ; STONEBRAKER, Michael ; ZDONIK, Stan: Anti-Caching: A New Approach to Database Management System Architecture. In: *Proceedings of the VLDB Endowment* 6 (2013), Nr. 14, S. 1942–1953
- [Dre07] DREPPER, Ulrich: What Every Programmer Should Know About Memory / Red Hat, Inc. 2007. – Forschungsbericht. White paper, <https://www.akkadia.org/drepper/cpumemory.pdf>
- [DSST89] DRISCOLL, James R. ; SARNAK, Neil ; SLEATOR, Daniel D. ; TARJAN, Robert E.: Making Data Structures Persistent. In: *Journal of Computer and System Sciences* 38 (1989), Nr. 1, S. 86–124
- [DWKN20] DIETRICH, Robert ; WINKLER, Frank ; KNÜPFER, Andreas ; NAGEL, Wolfgang E.: PIKA: Center-Wide and Job-Aware Cluster Monitoring. In: *2020 IEEE International Conference on Cluster Computing (CLUSTER)*, IEEE, 2020, S. 424–432
- [EB75] VAN EMDE BOAS, Peter: Preserving Order in a Forest in Less Than Logarithmic Time. In: *16th Annual Symposium on Foundations of Computer Science (FOCS)*, 1975, S. 75–84

- [ED18] EMMERICH, Michael T. M. ; DEUTZ, André H.: A Tutorial on Multiobjective Optimization: Fundamentals and Evolutionary Methods. In: *Natural Computing* 17 (2018), Nr. 3, S. 585–609
- [Ehr05] EHRGOTT, Matthias: *Multicriteria Optimization*. 2nd. Berlin, Heidelberg : Springer, 2005. – ISBN 978-3-540-21398-7
- [Eva06] EVANS, Jason: A Scalable Concurrent malloc(3) Implementation for FreeBSD. In: *Proceedings of BSDCan 2006* BSDCan, 2006. – <https://people.freebsd.org/~jasone/jemalloc/bsdcan2006/jemalloc.pdf>
- [FAK⁺12] FERDMAN, Michael ; ADILEH, Almutaz ; KOCBERBER, Onur ; VOLOS, Stavros ; ALISAFEE, Mohammad ; JEVDJIC, Djordje ; KAYNAK, Cansu ; POPESCU, Adrian D. ; AILAMAKI, Anastasia ; FALSAFI, Babak: Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware. In: *17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2012, S. 37–48
- [FAKM14] FAN, Bin ; ANDERSEN, David G. ; KAMINSKY, Michael ; MITZENMACHER, Michael: Cuckoo Filter: Practically Better Than Bloom. In: *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT '14)*, ACM, 2014, S. 75–88
- [FB74] FINKEL, Raphael A. ; BENTLEY, Jon L.: Quad Trees: A Data Structure for Retrieval on Composite Keys. In: *Acta Informatica* 4 (1974), Nr. 1, S. 1–9
- [FC80] FRITSCH, F. N. ; CARLSON, R. E.: Monotone Piecewise Cubic Interpolation. In: *SIAM Journal on Numerical Analysis* 17 (1980), Nr. 2, S. 238–246
- [Fea04] FEATHERS, Michael C.: *Working Effectively with Legacy Code*. 1. Upper Saddle River, NJ : Prentice Hall PTR, 2004. – ISBN 978-0-13-117705-5
- [FJ98] FRIGO, Matteo ; JOHNSON, Steven G.: FFTW: An Adaptive Software Architecture for the FFT. In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* Bd. 3, 1998, S. 1381–1384
- [FJKN20] FENT, Philipp ; JUNGMAIR, Michael ; KIPF, Andreas ; NEUMANN, Thomas: START: Self-Tuning Adaptive Radix Tree. In: *36th IEEE International Conference on Data Engineering Workshops (ICDEW)*, 2020, S. 147–153
- [FLPR99] FRIGO, Matteo ; LEISERSON, Charles E. ; PROKOP, Harald ; RAMACHANDRAN, Sridhar: Cache-Oblivious Algorithms. In: *40th Annual Symposium on Foundations of Computer Science (FOCS)*, 1999, S. 285–297
- [Fow06] FOWLER, Martin. *Continuous Integration*. <https://martinfowler.com/articles/continuousIntegration.html>, Fassung vom 01.05.2006, revidiert 18.01.2024, abgerufen am 19.09.2026. 2006

- [Fre60] FREDKIN, Edward: Trie Memory. In: *Communications of the ACM* 3 (1960), Nr. 9, S. 490–499
- [Fre24] FREE SOFTWARE FOUNDATION EUROPE E.V. *REUSE Specification — Version 3.3*. Stand 2024-11-14, <https://reuse.software/spec-3.3/>, abgerufen am 19.09.2026. 2024
- [Fre26] FREE SOFTWARE FOUNDATION. *GNU Coding Standards, Abschnitt 7.2.6 Standard Targets for Users*. Stand 27.04.2026, https://www.gnu.org/prep/standards/html_node/Standard-Targets.html, abgerufen am 19.09.2026. 2026
- [FW86] FLEMING, Philip J. ; WALLACE, John J.: How Not to Lie with Statistics: The Correct Way to Summarize Benchmark Results. In: *Communications of the ACM* 29 (1986), Nr. 3, S. 218–221
- [FW93] FREDMAN, Michael L. ; WILLARD, Dan E.: Surpassing the Information Theoretic Bound with Fusion Trees. In: *Journal of Computer and System Sciences* 47 (1993), Nr. 3, S. 424–436
- [Gai86] GAIT, Jason: A Probe Effect in Concurrent Programs. In: *Software: Practice and Experience* 16 (1986), Nr. 3, S. 225–233
- [GBE07] GEORGES, Andy ; BUYTAERT, Dries ; EECKHOUT, Lieven: Statistically Rigorous Java Performance Evaluation. In: *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, S. 57–76
- [GHJV94] GAMMA, Erich ; HELM, Richard ; JOHNSON, Ralph ; VLISSIDES, John: *Design Patterns: Elements of Reusable Object-Oriented Software*. 1. Addison-Wesley, 1994
- [Git26] GIT PROJECT. *Git — gitmodules Documentation*. Git Reference Documentation, Stand Git 2.54.0 (20.04.2026), <https://git-scm.com/docs/gitmodules>, abgerufen am 19.09.2026. 2026
- [GL01] GRAEFE, Goetz ; LARSON, Per-Åke: B-Tree Indexes and CPU Caches. In: *Proceedings of the 17th IEEE International Conference on Data Engineering (ICDE)*, IEEE, 2001, S. 349–358
- [GL20] GRAF, Thomas M. ; LEMIRE, Daniel: Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters. In: *ACM Journal of Experimental Algorithmics* 25 (2020), Nr. 1, S. 1.5:1–1.5:16
- [GM05] GHEMAWAT, Sanjay ; MENAGE, Paul: TCMalloc: Thread-Caching Malloc / Google. 2005. – Forschungsbericht. Design documentation
- [GMUW09] GARCIA-MOLINA, Hector ; ULLMAN, Jeffrey D. ; WIDOM, Jennifer: *Database Systems: The Complete Book*. 2. Upper Saddle River, NJ, USA : Pearson Prentice Hall, 2009

- [Goo18] GOOGLE. *Abseil Swiss Tables* (flat_hash_map). <https://abseil.io/about/design/swisstable>. 2018
- [Göt13] GÖTZ, Sebastian: *Multi-Quality Auto-Tuning by Contract Negotiation*, Technische Universität Dresden, Diss., 2013. – URN: urn:nbn:de:bsz:14-qucosa-119938
- [Gra09] GRAHAM, John W.: Missing Data Analysis: Making It Work in the Real World. In: *Annual Review of Psychology* 60 (2009), Nr. 1, S. 549–576
- [GS78] GUIBAS, Leo J. ; SEDGEWICK, Robert: A Dichromatic Framework for Balanced Trees. In: *19th Annual Symposium on Foundations of Computer Science (FOCS)*, 1978, S. 8–21
- [GTOC06] GRAHAM, John W. ; TAYLOR, Bonnie J. ; OLCHOWSKI, Allison E. ; CUMSILLE, Patricio E.: Planned Missing Data Designs in Psychological Research. In: *Psychological Methods* 11 (2006), Nr. 4, S. 323–343
- [Gut84] GUTTMAN, Antonin: R-Trees: A Dynamic Index Structure for Spatial Searching. In: *ACM SIGMOD International Conference on Management of Data*, 1984, S. 47–57
- [HB15] HOEFLER, Torsten ; BELLI, Roberto: Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results. In: *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015
- [HBHR11] HERTER, Jörg ; BACKES, Peter ; HAUPENTHAL, Florian ; REINEKE, Jan: CAMA: A Predictable Cache-Aware Memory Allocator. In: *23rd Euromicro Conference on Real-Time Systems (ECRTS)*, 2011, S. 23–32
- [Her91] HERLIHY, Maurice: Wait-Free Synchronization. In: *ACM Transactions on Programming Languages and Systems* 13 (1991), Nr. 1, S. 124–149
- [HF96] HYNDMAN, Rob J. ; FAN, Yanan: Sample Quantiles in Statistical Packages. In: *The American Statistician* 50 (1996), Nr. 4, S. 361–365
- [HF10] HUMBLE, Jez ; FARLEY, David: *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. 1. Addison-Wesley, 2010. – ISBN 978-0-321-60191-9
- [HIKY12] HALIM, Felix ; IDREOS, Stratos ; KARRAS, Panagiotis ; YAP, Roland H. C.: Stochastic Database Cracking: Towards Robust Adaptive Indexing in Main-Memory Column-Stores. In: *Proceedings of the VLDB Endowment* 5 (2012), Nr. 6, S. 502–513
- [HKT⁺21] HUNTER, A. H. ; KENNELLY, Chris ; TURNER, Paul ; GOVE, Darryl ; MOSELEY, Tipp ; RANGANATHAN, Parthasarathy: Beyond malloc Efficiency to Fleet Efficiency: A Hugepage-Aware Memory Allocator. In: *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021. – <https://www.usenix.org/conference/osdi21/presentation/hunter>, S. 257–273

- [HMPR04] HEVNER, Alan R. ; MARCH, Salvatore T. ; PARK, Jinsoo ; RAM, Sudha: Design Science in Information Systems Research. In: *MIS Quarterly* 28 (2004), Nr. 1, S. 75–106
- [Hol79] HOLM, Sture: A Simple Sequentially Rejective Multiple Test Procedure. In: *Scandinavian Journal of Statistics* 6 (1979), Nr. 2, S. 65–70
- [HP03] HANKINS, Richard A. ; PATEL, Jignesh M.: Effect of Node Size on the Performance of Cache-Conscious B+-Trees. In: *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, ACM, 2003, S. 120–131
- [HP19] HENNESSY, John L. ; PATTERSON, David A.: *Computer Architecture: A Quantitative Approach*. 6th. Morgan Kaufmann, 2019. – ISBN 978-0-12-811905-1
- [HS89] HILL, Mark D. ; SMITH, Alan J.: Evaluating Associativity in CPU Caches. In: *IEEE Transactions on Computers* 38 (1989), Nr. 12, S. 1612–1630
- [IEE19] IEEE. *IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019)*. IEEE, New York, NY, USA. 2019
- [IKM07] IDREOS, Stratos ; KERSTEN, Martin L. ; MANEGOLD, Stefan: Database Cracking. In: *3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007, S. 68–78
- [Int24a] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation, Santa Clara, CA, USA. 2024
- [Int24b] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide*. Intel Corporation, Santa Clara, CA, USA, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, abgerufen am 18.09.2026. 2024
- [Int26] INTEL CORPORATION. *Intel Architecture Instruction Set Extensions and Future Features Programming Reference*. Document Number 319433-061, Intel Resource and Documentation Center, Dokument-ID 915637, <https://cdrdv2.intel.com/v1/dl/getContent/915637>, abgerufen am 19.09.2026. 2026
- [ISO24] ISO/IEC. *ISO/IEC 14882:2024 — Programming Languages — C++*. International Organization for Standardization, Geneva, Switzerland, <https://www.iso.org/standard/83626.html>, abgerufen am 18.09.2026. 2024
- [IZA⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; ATHANASSOULIS, Manos ; DAYAN, Niv ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi ; MAAS, Lukas M. ; QIN, Wilson ; WASAY, Abdul ; SUN, Yiyou: The Periodic Table of Data Structures. In: *IEEE Data Engineering Bulletin* 41 (2018), Nr. 3, S. 64–75
- [IZH⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi: The Data Calculator: Data Structure Design and Cost Synthesis from First Principles, and Learned Cost Models. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 535–550

- [Jac89] JACOBSON, Guy J.: Space-Efficient Static Trees and Graphs. In: *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1989, S. 549–554
- [Jan17] JANSSON, Mattias. *rpmalloc: A Public Domain License Per-thread Memory Allocator*. GitHub Repository. 2017
- [JED20] JEDEC SOLID STATE TECHNOLOGY ASSOCIATION: DDR5 SDRAM Standard (JESD79-5) / JEDEC Solid State Technology Association, Arlington, VA, USA. 2020. – Forschungsbericht. Norm
- [JED25] JEDEC SOLID STATE TECHNOLOGY ASSOCIATION: DDR5 Serial Presence Detect (SPD) Contents / JEDEC. 2025 (JESD400-5D). – Forschungsbericht. <https://www.jedec.org/standards-documents/docs/jesd400-5d01>, abgerufen am 19.09.2026
- [JNC22] JABOT, Corentin ; NIEBLER, Eric ; CARTER, Casey. *Conversions from ranges to containers*. ISO/IEC JTC1/SC22/WG21 Dokument P1206R7, 2022-01-21, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p1206r7.pdf>, abgerufen am 19.09.2026. 2022
- [JTJE10] JALEEL, Aamer ; THEOBALD, Kevin B. ; JR., Simon C. S. ; EMER, Joel: High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In: *37th Annual International Symposium on Computer Architecture (ISCA)*, 2010, S. 60–71
- [KAS21] KOYLE, Neal ; AYYAPUREDDI, Sujeet ; SEIBERT, Mike: Micron DDR5: Client Module Features / Micron Technology, Inc. 2021 (CCM004676576390-11566, Rev. A). – White Paper. Online-Ressource
- [KAZ18] KOPP, Oliver ; ARMBRUSTER, Anita ; ZIMMERMANN, Olaf: Markdown Architectural Decision Records: Format and Tool Support. In: *Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS 2018)*, Dresden, Germany, February 8–9, 2018 Bd. 2072, CEUR-WS.org, 2018. – <https://ceur-ws.org/Vol-2072/paper9.pdf>, abgerufen am 19.09.2026, S. 55–62
- [KBC⁺18] KRASKA, Tim ; BEUTEL, Alex ; CHI, Ed H. ; DEAN, Jeffrey ; POLYZOTIS, Neoklis: The Case for Learned Index Structures. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 489–504
- [KBD⁺08] KNÜPFER, Andreas ; BRUNST, Holger ; DOLESCHAL, Jens ; JURENZ, Matthias ; LIEBER, Matthias ; MICKLER, Holger ; MÜLLER, Matthias S. ; NAGEL, Wolfgang E.: The Vampir Performance Analysis Tool-Set. In: *Tools for High Performance Computing*. Berlin, Heidelberg : Springer, 2008, S. 139–155
- [KCH⁺90] KANG, Kyo C. ; COHEN, Sholom G. ; HESS, James A. ; NOVAK, William E. ; PETERSON, A. S.: Feature-Oriented Domain Analysis (FODA) Feasibility Study / Software Engineering Institute, Carnegie Mellon University, Pittsburgh, PA, USA. 1990 (CMU/SEI-90-TR-021). – Forschungsbericht. Technical Report

- [KCS⁺10] KIM, Changkyu ; CHHUGANI, Jatin ; SATISH, Nadathur ; SEDLAR, Eric ; NGUYEN, Anthony D. ; KALDEWEY, Tim ; LEE, Victor W. ; BRANDT, Scott A. ; DUBEY, Pradeep: FAST: Fast Architecture Sensitive Tree Search on Modern CPUs and GPUs. In: *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, S. 339–350
- [Kha11] KHAN, Minhaj A.: Data Cache Prefetching With Dynamic Adaptation. In: *The Computer Journal* 54 (2011), Nr. 5, S. 815–823
- [KHA⁺19] VAN DER KOUWE, Erik ; HEISER, Gernot ; ANDRIESSE, Dennis ; BOS, Herbert ; GIUFFRIDA, Cristiano: SoK: Benchmarking Flaws in Systems Security. In: *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, S. 310–325
- [KHN⁺18] KHAN, Kashif N. ; HIRKI, Mikael ; NIEMI, Tapio ; NURMINEN, Jukka K. ; OU, Zhong-hong: RAPL in Action: Experiences in Using RAPL for Power Measurements. In: *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 3 (2018), Nr. 2, S. 1–26
- [Kit26] KITWARE, INC. *CMake Documentation: cmake(1)*. <https://cmake.org/cmake/help/latest/manual/cmake.1.html>, abgerufen am 01.08.2026. 2026
- [KJ13] KALIBERA, Tomas ; JONES, Richard: Rigorous Benchmarking in Reasonable Time. In: *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, 2013, S. 63–74
- [KM17] KHUONG, Paul-Virak ; MORIN, Pat: Array Layouts for Comparison-Based Searching. In: *ACM Journal of Experimental Algorithmics* 22 (2017), S. 1.3:1–1.3:39
- [KMR⁺19] KIPF, Andreas ; MARCUS, Ryan ; VAN RENEN, Alexander ; STOIAN, Mihail ; KEMPER, Alfons ; KRASKA, Tim ; NEUMANN, Thomas: SOSD: A Benchmark for Learned Indexes. In: *NeurIPS Workshop on Machine Learning for Systems*, 2019
- [Kno65] KNOWLTON, Kenneth C.: A Fast Storage Allocator. In: *Communications of the ACM* 8 (1965), Nr. 10, S. 623–625
- [KNR11] KREPS, Jay ; NARKHEDE, Neha ; RAO, Jun: Kafka: A Distributed Messaging System for Log Processing. In: *6th International Workshop on Networking Meets Databases (NetDB)*, 2011, S. 1–7
- [Knu98] KNUTH, Donald E.: *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd. Addison-Wesley, 1998
- [Kro81] KROFT, David: Lockup-Free Instruction Fetch/Prefetch Cache Organization. In: *Proceedings of the 8th Annual Symposium on Computer Architecture (ISCA)*, 1981, S. 81–87
- [Kue23] KUEHN, Lena: Towards Data-Based Cache Optimization of B+-Trees. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2023, S. 8:1–8:8

- [LABN16] LEIS, Viktor ; ANDERSEN, Michael ; BIRKEDAL, Erik ; NEUMANN, Thomas: The ART of Practical Synchronization. In: *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2016, S. 8:1–8:8
- [Lam77] LAMPORT, Leslie: Concurrent Reading and Writing. In: *Communications of the ACM* 20 (1977), Nr. 11, S. 806–811
- [Lam83] LAMPORT, Leslie: Specifying Concurrent Program Modules. In: *ACM Transactions on Programming Languages and Systems* 5 (1983), Nr. 2, S. 190–222
- [Lam13] LAMETER, Christoph: NUMA (Non-Uniform Memory Access): An Overview. In: *ACM Queue* 11 (2013), Nr. 7, S. 40–51
- [LC86] LEHMAN, Tobin J. ; CAREY, Michael J.: A Study of Index Structures for Main Memory Database Management Systems. In: *12th International Conference on Very Large Data Bases (VLDB)*, 1986, S. 294–303
- [LC01] LEWIS, Steff ; CLARKE, Mike: Forest Plots: Trying to See the Wood and the Trees. In: *BMJ* 322 (2001), Nr. 7300, S. 1479–1480
- [Lc21] LEIJEN, Daan ; CONTRIBUTORS. *mimalloc-bench: Suite for Benchmarking Malloc Implementations*. <https://github.com/daanx/mimalloc-bench>. 2021
- [LCL⁺25] LI, Meng ; CHAI, Huayi ; LUO, Siqiang ; DAI, Haipeng ; GU, Rong ; ZHENG, Jiaqi ; CHEN, Guihai: VEGA: An Active-tuning Learned Index with Group-Wise Learning Granularity. In: *Proceedings of the ACM on Management of Data* 3 (2025), Nr. 1, S. 86:1–86:26. – SIGMOD 2025; Artikel-Nr. 86
- [LDPB19] LIETAR, Paul ; DREYER, Derek ; PARKINSON, Adam ; BORNAT, Richard: snmalloc: A Message Passing Allocator. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, ACM, 2019, S. 122–135
- [Lea96] LEA, Doug. *A Memory Allocator*. Design article and C implementation. 1996
- [Lin25] LINUX KERNEL COMMUNITY. *Linux Kernel Source Tree, Tag v6.17: perf PMU Event Tables for AMD Zen 5 (tools/perf/pmu-events/arch/x86/amdzen5, Dateien l2-cache.json und load-store.json)*. <https://github.com/torvalds/linux/tree/v6.17/tools/perf/pmu-events/arch/x86/amdzen5>, abgerufen am 19.09.2026. 2025
- [Lin26a] LINUX KERNEL ORGANIZATION. *RISC-V Hardware Probing Interface*. Linux Kernel Documentation, <https://docs.kernel.org/arch/riscv/hwprobe.html>, abgerufen am 18.09.2026. 2026
- [Lin26b] LINUX MAN-PAGES PROJECT. *perf_event_open(2) — Linux Manual Page*. https://man7.org/linux/man-pages/man2/perf_event_open.2.html, abgerufen am 19.09.2026. 2026

- [LJY25] LI, Ruihao ; JOHN, Lizy K. ; YADWADKAR, Neeraja J.: Old is Gold: Optimizing Single-Threaded Applications with Exgen-Malloc. In: *IEEE Computer Architecture Letters* 24 (2025), Nr. 2
- [LKN13] LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas: The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In: *29th IEEE International Conference on Data Engineering (ICDE)*, 2013, S. 38–49
- [LLS13] LEVANDOSKI, Justin J. ; LOMET, David B. ; SENGUPTA, Sudipta: The Bw-Tree: A B-tree for New Hardware Platforms. In: *29th IEEE International Conference on Data Engineering (ICDE 2013), Brisbane, Australia, April 8-12, 2013*, IEEE Computer Society, 2013, S. 302–313
- [LNOM08] LINDHOLM, Erik ; NICKOLLS, John ; OBERMAN, Stuart ; MONTRYM, John: NVIDIA Tesla: A Unified Graphics and Computing Architecture. In: *IEEE Micro* 28 (2008), Nr. 2, S. 39–55
- [LR18] LEITE, Ricardo ; ROCHA, Rui: LRMalloc: A Modern and Competitive Lock-Free Dynamic Memory Allocator. In: *Proceedings of the 13th International Conference on Parallel Processing and Applied Mathematics (VECPAR)*, Springer, 2018, S. 188–200
- [LYTL19] LI, Tianlin ; YAO, Yiping ; TANG, Wenjie ; LIN, Zhongwei: HMalloc: A Hybrid, Scalable, and Lock-Free Memory Allocator for Multi-Threaded Applications. In: *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, 2019, S. 406–409
- [LZM19] LEIJEN, Daan ; ZORN, Benjamin ; DE MOURA, Leonardo: Mimalloc: Free List Sharding in Action. In: *Proceedings of the 17th Asian Symposium on Programming Languages and Systems (APLAS)*, Springer, 2019, S. 172–192
- [LZS⁺23] LUO, Xuchuan ; ZUO, Pengfei ; SHEN, Jiacheng ; GU, Jiazhen ; WANG, Xin ; LYU, Michael R. ; ZHOU, Yangfan: SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory. In: *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023
- [MBL25] MÜLLER, Marcus ; BENSON, Lawrence ; LEIS, Viktor: B-Trees Are Back: Engineering Fast and Pageable Node Layouts. In: *Proceedings of the ACM on Management of Data* 3 (2025), Nr. 1
- [McK01] MCKENNEY, Paul E.: Read-Copy Update. In: *Proceedings of the 2001 Ottawa Linux Symposium (OLS)*, 2001, S. 338–367
- [MDHS09] MYTKOWICZ, Todd ; DIWAN, Amer ; HAUSWIRTH, Matthias ; SWEENEY, Peter F.: Producing Wrong Data Without Doing Anything Obviously Wrong! In: *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009, S. 265–276
- [Mel10] DE MELO, Arnaldo C.: The New Linux perf Tools. In: *Linux Kongress*, 2010. – <http://oldvger.kernel.org/~acme/perf/lk2010-perf-paper.pdf>, S. 1–42

- [Mey05] MEYERS, Scott: *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*. 3. Addison-Wesley Professional, 2005. – ISBN 978–0–321–33487–9
- [MF02] MANZINI, Giovanni ; FERRAGINA, Paolo: Engineering a Lightweight Suffix Array Construction Algorithm. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 698–710
- [Mic04a] MICHAEL, Maged M.: Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects. In: *IEEE Transactions on Parallel and Distributed Systems* 15 (2004), Nr. 6, S. 491–504
- [Mic04b] MICHAEL, Maged M.: Scalable Lock-Free Dynamic Memory Allocation. In: *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2004, S. 35–46
- [Mit16] MITTAL, Sparsh: A Survey of Recent Prefetching Techniques for Processor Caches. In: *ACM Computing Surveys* 49 (2016), Nr. 2, S. 35:1–35:35
- [MKM12] MAO, Yandong ; KOHLER, Eddie ; MORRIS, Robert T.: Cache Craftiness for Fast Multicore Key-Value Storage. In: *7th ACM European Conference on Computer Systems (EuroSys)*, 2012, S. 183–196
- [MMP18] MOKHOV, Andrey ; MITCHELL, Neil ; PEYTON JONES, Simon: Build Systems à la Carte. In: *Proceedings of the ACM on Programming Languages* 2 (2018), Nr. ICFP, S. 79:1–79:29
- [Mor68] MORRISON, Donald R.: PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric. In: *Journal of the ACM* 15 (1968), Nr. 4, S. 514–534
- [MPU⁺23] MINERVINI, Francesco ; PALOMAR, Oscar ; UNSAL, Osman ; REGGIANI, Enrico ; QUIROGA, Josue ; MARIMON, Joan [u. a.]: Vitruvius+: An Area-Efficient RISC-V Decoupled Vector Coprocessor for High Performance Computing Applications. In: *ACM Transactions on Architecture and Code Optimization* 20 (2023), Nr. 2
- [MS96] MICHAEL, Maged M. ; SCOTT, Michael L.: Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In: *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, ACM, 1996, S. 267–275
- [MS08] MEHLHORN, Kurt ; SANDERS, Peter: *Algorithms and Data Structures: The Basic Toolbox*. 1. Springer, 2008. – ISBN 978–3–540–77977–3
- [MTL78] MCGILL, Robert ; TUKEY, John W. ; LARSEN, Wayne A.: Variations of Box Plots. In: *The American Statistician* 32 (1978), Nr. 1, S. 12–16
- [MW47] MANN, Henry B. ; WHITNEY, Donald R.: On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. In: *The Annals of Mathematical Statistics* 18 (1947), Nr. 1, S. 50–60
- [MWR25] MAHLING, Fabian ; WEISGUT, Marcel ; RABL, Tilmann: Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory. In: *Proceedings of*

- the 21st International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2025
- [NR21] NIKOLAEV, Ruslan ; RAVINDRAN, Binoy: Brief Announcement: Crystalline: Fast and Memory Efficient Wait-Free Reclamation. In: *35th International Symposium on Distributed Computing (DISC)* Bd. 209, 2021, S. 60:1–60:4
- [NR24] NIKOLAEV, Ruslan ; RAVINDRAN, Binoy: A Family of Fast and Memory Efficient Lock- and Wait-Free Reclamation. In: *Proceedings of the ACM on Programming Languages* 8 (2024), Nr. PLDI
- [NTSA16] NADERAN-TAHAN, Mahmood ; SARBAZI-AZAD, Hamid: Why Does Data Prefetching Not Work for Modern Workloads? In: *The Computer Journal* 59 (2016), Nr. 2, S. 244–259
- [Nyg11] NYGARD, Michael. *Documenting Architecture Decisions*. Blog-Beitrag, Cognitect, 15.11.2011, <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, abgerufen am 19.09.2026. 2011
- [Obj16] OBJECT MANAGEMENT GROUP. *Meta Object Facility (MOF) Core Specification, Version 2.5.1*. OMG Document formal/19-10-01, <https://www.omg.org/spec/MOF/2.5.1>, abgerufen am 19.09.2026. 2016
- [OCGO96] O’NEIL, Patrick E. ; CHENG, Edward ; GAWLICK, Dieter ; O’NEIL, Elizabeth J.: The Log-Structured Merge-Tree (LSM-Tree). In: *Acta Informatica* 33 (1996), Nr. 4, S. 351–385
- [PBL05] POHL, Klaus ; BÖCKLE, Günter ; VAN DER LINDEN, Frank: *Software Product Line Engineering*. 1. Berlin, Heidelberg : Springer, 2005. – ISBN 978–3–540–24372–4
- [PIA78] PERL, Yehoshua ; ITAI, Alon ; AVNI, Haim: Interpolation Search - A Log Log N Search. In: *Communications of the ACM* 21 (1978), Nr. 7, S. 550–553
- [Pik18] PIKE, Colby. *cxx-pfR1: The Pitchfork Layout (PFL)*. Spezifikation data/spec.bs im Repositorium vector-of-bool/pitchfork, <https://github.com/vector-of-bool/pitchfork>, abgerufen am 19.09.2026. 2018
- [PM12] PHARR, Matt ; MARK, William R.: ispc: A SPMD compiler for high-performance CPU programming. In: *2012 Innovative Parallel Computing (InPar)*, IEEE, 2012, S. 1–13
- [PR04] PAGH, Rasmus ; RODLER, Flemming F.: Cuckoo Hashing. In: *Journal of Algorithms* 51 (2004), Nr. 2, S. 122–144
- [PTRC07] PEFERS, Ken ; TUUNANEN, Tuure ; ROTHENBERGER, Marcus A. ; CHATTERJEE, Samir: A Design Science Research Methodology for Information Systems Research. In: *Journal of Management Information Systems* 24 (2007), Nr. 3, S. 45–77
- [Pug90] PUGH, William W.: Skip Lists: A Probabilistic Alternative to Balanced Trees. In: *Communications of the ACM* 33 (1990), Nr. 6, S. 668–676

- [QJP⁺07] QURESHI, Moinuddin K. ; JALEEL, Aamer ; PATT, Yale N. ; JR., Simon C. S. ; EMER, Joel: Adaptive Insertion Policies for High Performance Caching. In: *34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, S. 381–391
- [Rei05] REINDERS, James: *VTune Performance Analyzer Essentials: Measurement and Tuning Techniques for Software Developers*. 1. Hillsboro, OR, USA : Intel Press, 2005. – ISBN 978-0-9743649-5-7
- [RFP24] REITZ, Antonin ; FROMHERZ, Aymeric ; PROTZENKO, Jonathan: StarMalloc: Verifying a Modern, Hardened Memory Allocator. In: *Proceedings of the ACM on Programming Languages* 8 (2024), Nr. OOPSLA2
- [RH09] RUNESON, Per ; HÖST, Martin: Guidelines for Conducting and Reporting Case Study Research in Software Engineering. In: *Empirical Software Engineering* 14 (2009), Nr. 2, S. 131–164
- [RIS21] RISC-V INTERNATIONAL. *RISC-V V Vector Extension, Version 1.0*. https://docs.riscv.org/reference/isa/extensions/vector/_attachments/riscv-v-spec.pdf. 2021
- [RLK⁺18] VAN RENEN, Alexander ; LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas ; HASHIDA, Takushi ; OE, Kazuichi ; DOI, Yoshiyasu ; HARADA, Lilian ; SATO, Mitsuru: Managing Non-Volatile Memory in Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 1541–1555
- [Rob06] ROBINSON, Ian. *Consumer-Driven Contracts: A Service Evolution Pattern*. <https://martinfowler.com/articles/consumerDrivenContracts.html>, 12.06.2006, abgerufen am 19.09.2026. 2006
- [RR99] RAO, Jun ; ROSS, Kenneth A.: Cache Conscious Indexing for Decision-Support in Main Memory. In: *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, 1999, S. 78–89
- [RR00] RAO, Jun ; ROSS, Kenneth A.: Making B+-Trees Cache Conscious in Main Memory. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ACM, 2000, S. 475–486
- [Rub76] RUBIN, Donald B.: Inference and Missing Data. In: *Biometrika* 63 (1976), Nr. 3, S. 581–592
- [SBB⁺17] STEPHENS, Nigel ; BILES, Stuart ; BOETTCHER, Matthias ; EYOLE, Mbou ; GABRIELLI, Giacomo ; HORSNELL, Matt ; MAGKLIS, Grigorios ; MARTINEZ, Alejandro ; PREMILLIEU, Nathanael ; REID, Alastair ; RICO, Alejandro ; WALKER, Paul: The ARM Scalable Vector Extension. In: *IEEE Micro* 37 (2017), Nr. 2, S. 26–39
- [Sch06] SCHMIDT, Douglas C.: Guest Editor’s Introduction: Model-Driven Engineering. In: *Computer* 39 (2006), Nr. 2, S. 25–31

- [SDF⁺00] SRINIVASAN, Jagannathan ; DAS, Souripriya ; FREIWALD, Chuck ; CHONG, Eugene I. ; JAGANNATH, Mahesh ; YALAMANCHI, Aravind ; KRISHNAN, Ramkumar ; TRAN, Anh-Tuan ; DEFAZIO, Samuel ; BANERJEE, Jayanta: Oracle8i Index-Organized Table and its Application to New Domains. In: *26th International Conference on Very Large Data Bases (VLDB)*, 2000, S. 285–296
- [SGL09] SCHLEGEL, Benjamin ; GEMULLA, Rainer ; LEHNER, Wolfgang: k-Ary Search on Modern Processors. In: *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN 2009)*, ACM, 2009, S. 52–60
- [Sha03] SHAW, Mary: Writing Good Software Engineering Research Papers. In: *Proceedings of the 25th International Conference on Software Engineering (ICSE)*, 2003, S. 726–736
- [Sid26] SIDERO LABS. *Talos Linux — Linux OS for Kubernetes*. Produktseite, <https://www.siderolabs.com/talos-linux/> (talos.dev leitet dorthin weiter), abgerufen am 19.09.2026. 2026
- [SKK⁺25] SCHMIDT, Lennart ; KÜHN, Roland ; KRAUSE, Matti ; TEUBNER, Jens ; LEHNER, Wolfgang ; HABICH, Dirk: To Stride or Not to Stride the Memory Access? In: *Proceedings of the 3rd Workshop on Disruptive Memory Systems (DIMES)*, ACM, 2025, S. 19–26
- [SPB05] SAMUEL, Curt ; PEDERSEN, Torben B. ; BONNET, Philippe: Processor-Conscious Indexing. In: *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, ACM, 2005, S. 235–246
- [SS12] SUTTON, Andrew ; STROUSTRUP, Bjarne: Design of Concept Libraries for C++. In: *Software Language Engineering (SLE 2011), Lecture Notes in Computer Science 6940*. Berlin, Heidelberg : Springer, 2012, S. 97–118
- [SSL⁺21] SCHMEISSER, Josef ; SCHÜLE, Maximilian E. ; LEIS, Viktor ; NEUMANN, Thomas ; KEMPER, Alfons: The B²-Tree: Cache-Friendly String Indexing within B-Trees. In: *Datenbanksysteme für Business, Technologie und Web (BTW)*, 2021, S. 39–58
- [SSS08] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Binary Trees. In: *Fifth IFIP International Conference on Theoretical Computer Science (TCS 2008)* Bd. 273, Springer, 2008
- [SSS16] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Dynamic Binary Trees. In: *The Computer Journal* 59 (2016), Nr. 5, S. 630–649
- [Sta17] STANDARD PERFORMANCE EVALUATION CORPORATION. *SPEC CPU 2017 Benchmark Suite*. <https://www.spec.org/cpu2017/>. 2017
- [Sto81] STONEBRAKER, Michael: Operating System Support for Database Management. In: *Communications of the ACM* 24 (1981), Nr. 7, S. 412–418
- [Str12] STROUSTRUP, Bjarne: Foundations of C++. In: *Programming Languages and Systems (ESOP 2012), Lecture Notes in Computer Science 7211*. Berlin, Heidelberg : Springer, 2012, S. 1–25

- [SVS16] SPERTUS, Mike ; VALI, Faisal ; SMITH, Richard. *Template argument deduction for class templates (Rev. 6)*. ISO/IEC JTC1/SC22/WG21 Dokument P0091R3, 2016-06-24, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0091r3.html>, abgerufen am 19.09.2026. 2016
- [TCH02] ȚĂPUȘ, Cristian ; CHUNG, I-Hsin ; HOLLINGSWORTH, Jeffrey K.: Active Harmony: Towards Automated Performance Tuning. In: *ACM/IEEE Conference on Supercomputing (SC)*, 2002, S. 1–11
- [Ten12] TENE, Gil. *HdrHistogram: A High Dynamic Range (HDR) Histogram*. <https://github.com/HdrHistogram/HdrHistogram>. 2012
- [TFB⁺11] THOMPSON, Martin ; FARLEY, Dave ; BARKER, Michael ; GEE, Patricia ; STEWART, Andrew: Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads / LMAX. 2011. – Technical Paper. Version 1.0
- [The18] THE OPEN GROUP. *dlopen — Open a Symbol Table Handle. The Open Group Base Specifications Issue 7, 2018 Edition (IEEE Std 1003.1-2017)*. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/dlopen.html>, abgerufen am 19.09.2026. 2018
- [TLH94] TORRELLAS, J. ; LAM, H. S. ; HENNESSY, J. L.: False Sharing and Spatial Locality in Multiprocessor Caches. In: *IEEE Transactions on Computers* 43 (1994), Nr. 6, S. 651–663
- [TMS⁺16] TIBBA, Ghizlane ; MALZ, Christoph ; STOERMER, Christoph ; NAGARAJAN, Natarajan ; ZHANG, Licong ; CHAKRABORTY, Samarjit: Testing Automotive Embedded Systems under X-in-the-Loop Setups. In: *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016, S. 1–8
- [Tra24] TRANSACTION PROCESSING PERFORMANCE COUNCIL. *TPC Benchmark C and TPC-DS Specifications*. Transaction Processing Performance Council, San Francisco, CA, USA. 2024
- [TWH16] TALBOT, Alan ; WAKELY, Jonathan ; HINNANT, Howard ; DENNETT, James. *Splicing Maps and Sets (Revision 5)*. ISO/IEC JTC1/SC22/WG21 Dokument P0083R3, 2016-06-24, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0083r3.pdf>, abgerufen am 19.09.2026. 2016
- [UHL⁺17] UNGETHUEM, Sigmund ; HABICH, Dirk ; LEHNER, Wolfgang [u. a.]: Hardware Acceleration for Database Systems: A Comprehensive Survey. In: *ACM Computing Surveys* 50 (2017), Nr. 1, S. 3:1–3:35
- [VIA26] VIA RESEARCH: PIM-malloc: A Fast and Scalable Dynamic Memory Allocator for Processing-In-Memory Systems. In: *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2026. – Accepted to HPCA 2026; also available as arXiv:2505.13002 (2025)

- [VJG17] VANDEVOORDE, David ; JOSUTTIS, Nicolai M. ; GREGOR, Douglas: *C++ Templates: The Complete Guide*. 2nd. Addison-Wesley, 2017. – ISBN 978–0–321–71412–1
- [VRM⁺18] VIETRI, Giuseppe ; RODRIGUEZ, Liana V. ; MARTINEZ, Wendy A. ; LYONS, Steven ; LIU, Jason ; RANGASWAMI, Raju ; ZHAO, Ming ; NARASIMHAN, Giri: Driving Cache Replacement with ML-based LeCaR. In: *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018
- [Vyu11] VYUKOV, Dmitry. *Bounded MPMC Queue*. <https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue>. 2011
- [WF09] WILKINSON, Leland ; FRIENDLY, Michael: The History of the Cluster Heat Map. In: *The American Statistician* 63 (2009), Nr. 2, S. 179–184
- [WG68] WILK, Martin B. ; GNANADESIKAN, Ramanathan: Probability Plotting Methods for the Analysis of Data. In: *Biometrika* 55 (1968), Nr. 1, S. 1–17
- [Wie14] WIERINGA, Roel J.: *Design Science Methodology for Information Systems and Software Engineering*. 1. Berlin, Heidelberg : Springer, 2014. – ISBN 978–3–662–43838–1
- [Wil64] WILLIAMS, J. W. J.: Algorithm 232: Heapsort. In: *Communications of the ACM* 7 (1964), Nr. 6, S. 347–348
- [Wil83] WILLARD, Dan E.: Log-Logarithmic Worst-Case Range Queries are Possible in Space $\Theta(N)$. In: *Information Processing Letters* 17 (1983), Nr. 2, S. 81–84
- [WLLM13] WAKELY, Jonathan ; LAVAVEJ, Stephan T. ; LÓPEZ MUÑOZ, Joaquín M. *Adding heterogeneous comparison lookup to associative containers (rev 4)*. ISO/IEC JTC1/SC22/WG21 Dokument N3657, 2013-03-19, <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2013/n3657.htm>, abgerufen am 19.09.2026. 2013
- [WM95] WULF, William A. ; MCKEE, Sally A.: Hitting the Memory Wall: Implications of the Obvious. In: *ACM SIGARCH Computer Architecture News* 23 (1995), Nr. 1, S. 20–24
- [WM08] WEAVER, Vincent M. ; MCKEE, Sally A.: Can Hardware Performance Counters be Trusted? In: *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, 2008, S. 141–150
- [WNJ19] WU, Xingbo ; NI, Fan ; JIANG, Song: Wormhole: A Fast Ordered Index for In-Memory Data Management. In: *14th ACM European Conference on Computer Systems (EuroSys)*, 2019
- [WPD01] WHALEY, R. C. ; PETITET, Antoine ; DONGARRA, Jack J.: Automated Empirical Optimization of Software and the ATLAS Project. In: *Parallel Computing* 27 (2001), Nr. 1–2, S. 3–35
- [WRH⁺12] WOHLIN, Claes ; RUNESON, Per ; HÖST, Martin ; OHLSSON, Magnus C. ; REGNELL, Björn ; WESSLÉN, Anders: *Experimentation in Software Engineering*. 1. Berlin, Heidelberg : Springer, 2012. – ISBN 978–3–642–29043–5

- [Xu24] XU, Chuanqi: Modules ABI Requirement / ISO/IEC JTC1/SC22/WG21. 2024 (P3092R0). – Forschungsbericht. Online-Ressource
- [YZZ⁺23] YANG, Hanmei ; ZHAO, Xin ; ZHOU, Jin ; WANG, Wei ; KUNDU, Sandip ; WU, Bo ; GUAN, Hui ; LIU, Tongping: NUMAlloc: A Faster NUMA Memory Allocator. In: *2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, 2023
- [ZDL⁺13] ZAHARIA, Matei ; DAS, Tathagata ; LI, Haoyuan ; HUNTER, Timothy ; SHENKER, Scott ; STOICA, Ion: Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In: *24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, S. 423–438
- [Zen26a] ZENTRUM FÜR INFORMATIONSDIENSTE UND HOCHLEISTUNGSRECHNEN (ZIH), TU DRESDEN. *HPC Compendium: HPC Resources (Hardware Overview) und CPU Cluster Barnard*. https://compendium.hpc.tu-dresden.de/jobs_and_resources/hardware_overview/, abgerufen am 19.09.2026. 2026
- [Zen26b] ZENTRUM FÜR INFORMATIONSDIENSTE UND HOCHLEISTUNGSRECHNEN (ZIH), TU DRESDEN. *HPC Compendium: NVIDIA Grace Hopper Superchip (Testsystem)*. https://compendium.hpc.tu-dresden.de/jobs_and_resources/nvidia_grace_hopper/, abgerufen am 19.09.2026. 2026
- [ZGH⁺25] ZHANG, Tingji ; GROT, Boris ; HE, Wenjian ; LV, Yashuai ; QU, Peng ; SU, Fang ; WANG, Wenxin ; ZHANG, Guowei ; ZHANG, Xuefeng ; ZHANG, Youhui: Hierarchical Prefetching: A Software-Hardware Instruction Prefetcher for Server Applications. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, ACM, 2025, S. 529–544
- [ZGV⁺24] ZHOU, Zhuangzhuang ; GOGTE, Vaibhav ; VAISH, Nilay ; KENNELLY, Chris ; XIA, Patrick ; KANEV, Svilen ; MOSELEY, Tipp ; DELIMITROU, Christina ; RANGANATHAN, Parthasarathy: Characterizing a Memory Allocator at Warehouse Scale. In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, S. 192–206
- [ZL77] ZIV, Jacob ; LEMPEL, Abraham: A Universal Algorithm for Sequential Data Compression. In: *IEEE Transactions on Information Theory* 23 (1977), Nr. 3, S. 337–343
- [ZLL⁺18] ZHANG, Huanchen ; LIM, Hyeontaek ; LEIS, Viktor ; ANDERSEN, David G. ; KAMINSKY, Michael ; KEETON, Kimberly ; PAVLO, Andrew: SuRF: Practical Range Query Filtering with Fast Succinct Tries. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 323–336
- [ZSZ⁺24] ZHANG, Qian ; SONG, Haoyun ; ZHOU, Kaiyan ; WEI, Jianhao ; XIAO, Chuqiao: A Prefetching Indexing Scheme for In-Memory Database Systems. In: *Future Generation Computer Systems* 156 (2024), S. 179–190

Abbildungsverzeichnis

1.1	Die „Cache-Wand“: typische Größenordnungen der Zugriffslatenz je Ebene der Speicher-Hierarchie (Literaturwerte nach [HP19, Dre07]); über die Hierarchie wächst die Latenz um rund zwei Größenordnungen.	5
1.2	Das Trennbarkeits-Problem: als Monolith bleibt unklar, welche Entwurfsentscheidung wirkt; erst die Zerlegung in orthogonale Achsen macht jeden Beitrag isoliert messbar. .	6
2.1	Die Landschaft der Suchstrukturen: fünf Klassen mit typischen Vertretern.	13
2.2	Cache-Line-Auslastung: eine Cache-Line wird stets vollständig geladen; werden nur wenige ihrer Bytes genutzt, sinkt die effektive Bandbreite — cache-bewusstes Layout hebt die Auslastung.	16
3.1	Der Entwurfsraum: die Achsen spannen ein multidimensionales kartesisches Produkt auf; ein konkreter Algorithmus ist ein Punkt darin. Zur Veranschaulichung sind drei der achtzehn Organ-Achsen gewählt — das reale Speicherorgan aus Knotentyp (T4), Speicher-Layout (T5) und Allokator (T6); der volle Entwurfsraum hat achtzehn Organ-Dimensionen; dazu tritt eine optionale neunzehnte Organ-Achse für die Festplatten-Ein-/Ausgabe, die nicht eine Belegung beschreibt, sondern beschreibt, wie andere Achsen beschrieben werden (Abschnitt 3.4.1).	26
3.2	Galerie der wichtigsten rekonstruierten Suchstrukturen: je Verfahren (mit Paper-ID) seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet. Trie-, Hybrid-, succinct- und cache-sensitive Verfahren stehen neben einer nicht-baumartigen Hash-Gegenprobe (SwissTable) — alle als Punkte im selben Achsen-Entwurfsraum. Die vollständige Achsen-Sezierung führt Tabelle 3.2.	35
3.3	Die Anatomie-Brücke: wie ein Lebewesen aus zusammenwirkenden Organen besteht, ist ein Suchalgorithmus eine Komposition von Achsen-Organen; das Blut verbindet die Organ-Achsen, der Lebensraum (die System-Achsen) trägt das Tier, und das Mess-System übernimmt die Rolle des Labors, das jedes Organ einzeln vermisst.	39
3.4	Synthese Stand der Technik → Achsen: die Entwurfsbausteine der bekannten Verfahren werden in orthogonale Achsen zerlegt; jede Achse sammelt die Beiträge des gesamten Korpus — die Umkehrung von Abbildung 3.1 (dort ist ein Algorithmus ein Punkt, hier eine Achse ein Sammelbecken).	40
3.5	Achse = Organ, Anatomie = Verdrahtung: das Speicher-Organ (Komposition aus Knotentyp, Layout und Allokator) exportiert ein generalisiertes Speicher-Organ-Interface (im Code <code>LayoutAwareChunkedStore</code>), über das die Suche läuft (durchgezogen, <i>verpflichtend</i>); weitere Organe wie die Telemetrie <i>können</i> es über optionale Beobachter-Hooks nutzen (gestrichelt, <i>optional</i>), müssen aber nicht.	40

- 3.6 Inter-Organ-Nutzungs-Graph („welche Achse nutzt welche“): Es gibt genau *einen* kanonischen Hub — das Speicher-Organ `LayoutAwareChunkedStore`, das Knotentyp (T4), Layout (T5) und Allokator (T6) komponiert. *Verpflichtend* (durchgezogen) konsumiert der Store diese drei Achsen, und die Suche (T0) läuft über ihn. Alle übrigen Kopplungen sind *optional* (gestrichelt): Prefetch (T7) und die Telemetrie (Mess-Instrumentierung, Abschnitt 3.4.1) lesen den Store nur über `requires`-gegate Beobachter-Hooks — eine Achse *kann* fremde Interfaces nutzen, *muss* aber nicht. Die Kopplung der Nebenläufigkeit (T8) an den Store ist nicht angebunden (gepunktet). Der Graph ist eine Beispiel-Verdrahtung des Speicher-Organs, keine im Code ausgewertete Struktur. 41
- 3.7 Das Drei-Ebenen-Modell: die drei Interface-Ebenen Konzept (Ebene 1, Tierklasse, Mess-Surface: Prüf-Dock) → Gattung (Ebene 2, Tier-Interface, Template nach außen) → Genus (Ebene 3, Unterart: Implementierung des Gattungs-Interfaces, ggf. mit erweiterten Spezial-Interfaces); darunter die Organe/Achsen (Ebene 4, eigene Klassifikation), darüber das Reich (Ebene 0). Vier Konzepte: `Map` (Säugetier) besitzt als einzige Gattung `SearchAlgorithm` (Elefant), deren Genera die konkreten Implementierungen sind (Mammut = ART, afrikanischer Elefant = HOT, indischer Elefant = START); `Container` (Vogel) die Gattungen `Set/Sequence/Adapter/View` (Specht/Storch/Kakadu/Falke), die aus demselben Achsen-Pool gebildet werden; `Graph` (Pflanze) ist als Interface angelegt, eine Gattung ist nicht implementiert; `HeuristikAdapter` (Insekt) besitzt die `Reroute`-Gattung `FunctionInterfaceReroute` (Ameise) des Hybrid-Tiers. Heute trägt der Code das Genus nur als Achsen-Belegung; die Genus-Implementierungsklasse ist ein erkannter Architekturfehler (Behebung nach der Verteidigung). Tier-Binary (Ebene 5a) und Hybrid-Tier (Ebene 5b) sind die Nachphase der Kompilation (Abbildung 3.8). 42
- 3.8 Die vier Konzepte und die Tier-Hierarchie bis zum kompilierten Binary 43
- 3.9 Träger-Stufen und Interface-Flächen: die dreiwertige Zulassungsmatrix der sieben Stempel-Interfaces über die vier Träger-Stufen Planer, `CacheEngineBuilder` (CEB), Hybrid-Tier-Binary (optionaler Einschub) und Tier-Binary (28 Zellen; grün = Pflicht, grau = verboten; die Zelle Organ-Zeile/CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“) und darunter die Interface-Flächen je Stufe — Fläche 1 Gattungs-Interface (Laufzeit), Fläche 2 Stempel (Übersetzungszeit, ABI-stabil), Fläche 3 Mess-Durchstich. Die Konfiguration wandert von der XML über Planer und CEB bis zur Tier-Binary; die Hybrid-Tier-Binary ist eine optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Stufe (gestrichelt: Mux-Schalter am Prüf-Dock mit Dock-Slots für Tier-Binaries; die Spaltenordnung der Matrix folgt der Verarbeitungs-Topologie mit dem Hybrid an seiner Einschub-Position, der Bogen ist der Standardweg ohne Hybrid); keine Stufe durchläuft Schichten, jede enthält ihre Schicht einkompiliert. 45

- 3.10 Die Ablauf-Kette der vier Arbeitsmodi build → measure → compare → release als registrierte Unter-Achse des Mess-Realms, kumulativ in dieser Reihenfolge und auf der Hybrid-Ebene synchron wiederholt; compare liest das Messwertlager und errechnet die beste Binary bzw. Rekombination, release erzeugt die voll optimierte Binary ohne Messfühler und misst ihre Wall-Clock nach. Ein Debug-Zustand existiert nicht; unter `-debug` fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces, ein indirekter Debug-Modus, der mit dem Release-Bau zusammenfällt. Die Zuordnung zu den Betriebs-Zielbildern Mess-, Auswertungs-, Arbeits- und Hybrid-Modus gibt Abschnitt 3.6.2. 46
- 3.11 Entwurfsmuster als Bausteine der Architektur: oben die im Code durchgängig verwendeten klassischen GoF-Muster mit ihrer jeweiligen Rolle, in der Mitte die zwei in dieser Arbeit benannten neuen Übersetzungszeit-Muster, unten die code-seziert *Drei-Stufen-Concept-Kette* je Achse (Topic-Concept → Axis-Concept → CacheEngine-Permutations-Concept → CRTP-Basis → `mp_list`-Registry) als Verfeinerung von *Strategy/CRTP*; ein literaler `std::variant` tritt in keiner Organ-Achse auf (Verbot), sondern nur in einem vom Bau isolierten Test-Bestand. Ihre Komposition bildet die design-pattern-getriebene, mess-orientierte Architektur (Kapitel 4). 46
- 3.12 Die fünf Interface-Schichten der Experiment-Pipeline: von der deklarativen Definitions- und Auswertungs-Schicht E4 (Anwender-Sicht) über den Permutations-B⁺-Baum (E3) und die Tier-Binaries (E2) zur Laufzeit-Konfiguration (E1); die Infrastruktur (E0) stützt alles; ihre Fertigstellung bildet den letzten Schritt. Der Rückkanal führt die Messwerte zur Auswertung nach E4 zurück; das Mess-System durchdringt als Querschnitt M alle Schichten (Labor-Ausstattung, Abschnitt 3.4). Jede Schicht wird einzeln fertiggestellt, mit festgeschriebenem Interface-Vertrag zur Nachbar-Ebene. 48
- 3.13 Das Mess-Trade-off-Dreieck: jede Achsen-Permutation spannt sich gegen die drei gegenläufigen Mess-Komponenten Latenz, Durchsatz und Speicherverbrauch auf und ist ein Punkt in seinem Inneren (vgl. den Entwurfsraum in Abbildung 3.1, wo dieselbe Permutation ein Punkt ist); die Pareto-Front der nicht-dominierten Permutationen ist die Brücke von den drei Größen zur „besten Binary“. Die für cache-bewusste Suchstrukturen zentrale vierte Komponente — das Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im Mess-Pfad verbindlich eingebaut; offen sind der Intel-Ereignis-Katalog, der Last-Level-Zähler auf Zen 5 und die nur modellgebundene L2-/Kohärenz-Belegung. 55
- 3.14 Das Lager: der Lager-Baum ist nach Konzept, Gattung und Realm gegliedert; darunter liegen vier Bestände unter Stempel-Schlüsseln — Bestand 1 die Tier-Binaries (SHA-512-Fingerprint; die Binary mit und die ohne Messfühler sind zwei Einträge, weil der Mess-Gate-Zustand ein Fingerprint-Glied ist), Bestand 2 die Messwerte (Fingerprint der Binary plus Hardware-Identität der Maschine), Bestand 3 die Synthese-Batches mit den synthetisierten Mess-Kurven als Dateien und Bestand 4 die Lösungen je Anwender-XML. Der Apparat erkennt Vorhandenes über den Fingerprint wieder und misst nur Fehlendes nach. 58

- 3.15 Das M-Modell mit Fokus auf die Build- und Mess-Phasen. Die äußere `messung_driver`-Schleife (Was, Diplomarbeit-Code) speist die Konfigurationen der drei Messreihen A/B/C ein; der `CacheEngineBuilder` (Wie, Bibliothek) zerfällt in die beiden eigentlich interessanten Phasen: **build** (`BuildOrchestrator/ExperimentTree` baut je Konfiguration eine eigenständige Tier-Binary, resumierbar und RAM-gewahr) und **measure** (`perm_runner` treibt und misst jede Tier-Binary einzeln über die Beobachter-Schnittstelle `IObservableTier`). Die `CacheEngine` liefert die Standard-Achsen-Bausteine, der Prüfling PRT-ART seine eigenen Organe; ihre bidirektionale Beziehung ((a) instantiiert, (b) ruft Dienste) begründet die Multi-Prüfling-Fähigkeit. Die `ExecutionEngine` bleibt die gemeinsame messbare Wurzel. 60
- 3.16 Die Drei-Stufen-Prüfung des Builders: jede Stufe erzeugt nicht-redundante Permutations-Binaries; ihre Zuordnung zu den Messreihen A/B/C zeigt Tabelle 3.8. 60
- 3.17 PRT-ART als Demonstrator im M-Modell auf drei Ebenen: die vier Prüflings-Slots im eigenen Repository (Page-Type als Build-Achse, Prefetch, Wert-Handle, Telemetrie als Mess-Tooling-Unter-Achse, Abschnitt 3.4.1); die Referenz-Komposition der Cache-Engine, die sich von ART in genau einer Achse — der Pfadkompression als Redirect-Organ — unterscheidet, während das optimistische Lock-Coupling von ART geerbt ist; und die codegen-fähige Prüflings-Komposition, die sechzehn von achtzehn Achsen aus der ART-Komposition übernimmt und Prefetch sowie Wert-Handle ersetzt. Die weiteren eigenen Bausteine (4+1+2-Pool-Allokator, MultiLevel-Layout) laufen im Mess-Pfad über den Cache-Engine-Standard (Anbindung: nachfolgender Umsetzungsschritt, nicht implementiert). So entsteht ein vollständiges, mess-fähiges Lebewesen aus wenigen eigenen plus vielen Standard-Organen. 63
- 3.18 Die Heuristik-Schleife: aus den Messreihen wird je Lastprofil die beste Achsen-Komposition gelernt, als XML-Lastprofil abgelegt und über den Workload-Filter in die nächste Messung zurückgeführt. 64
- 4.1 Die Träger-Stufenkette im Code 73
- 4.2 Galerie der sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse: ein 4+1+2-Pool-Allokator (T6), optimistisches Lock-Coupling mit reservierten Wert-Blöcken (T8), pfad-orientiertes Bundle-Prefetch (T7), ein MultiLevel-Layout (T5) und cache-kohärenzbewusste Telemetrie samt `DensityTracker` und den Hypothesen-Metriken H1/H2/H3 — die Telemetrie-Achse ist zweigeteilt, Sweep-Unter-Achse des Mess-Toolings und Übersetzungszeit-Größe des Builders (Abschnitt 3.4.1). Die Kacheln zeigen die *Bausteine des Prüfling-Repos*; im Apparat überschrieben sind der Pfadkompressions-Redirect (T3, Engine-seitige Referenz-Komposition), das Prefetch (T7) und das ChainRef-Wert-Handle (T10, Slot-Komposition des Prüflings) sowie die Build-Achse Seitentyp; die Bausteine zu T5/T6 laufen im Mess-Pfad über den Cache-Engine-Standard, OLC ist von ART geerbt, und die übrigen Organ-Achsen bezieht die Slot-Komposition aus der ART-Referenz der Cache-Engine (sechzehn von achtzehn). 76

- 4.3 Galerie der achtzehn Organ-Achsen (T0–T17, kanonische Slot-Reihenfolge): je Achse das Implementierungs-Konzept — Kurzname, Unter-Achsen-Familie und eine repräsentative Auswahl der austauschbaren Bausteine. Jede Achse ist ein *Concept-Interface* mit mehreren Strategie-Bausteinen (*Strategy-Muster*); eine konkrete Composition belegt je Slot genau einen. Der *Standard* („Std.“) ist eine abgeleitete Vergleichs-Baseline, kein Code-Flag. Die Zahlen in Klammern (T0, T6, T15) sind die *Kataloge* der vollständigen Typlisten (*All**); das *Enabled-Inventar* einer Build-Konfiguration ist die vom Enable-Filter gebildete Teilmenge, die die generierte Registry-XML je Achse ausweist (etwa vier Suchalgorithmen und drei Allokatoren im Haus-Default). Die T-Nummern sind Slots der Komposition, keine Verzeichnis-Nummern der Implementierung; die Kürzel io1–io3 und IO1–IO3 gehören zu verschiedenen Achsen (Index-Organisation und I/O-Dispatch). Neben den achtzehn Slots steht die Organ-Meta-Meta *disk_io*; das Persistenz-Ziel ist im kanonischen Referenz-Katalog auf seinen In-Memory-Vertreter festgelegt (Abschnitt 3.4.1). Die vollständige Matrix mit Herkunfts-Quellen führt Anhang D. 79
- 4.4 Die *eine* Architektur als einzige Hierarchie ohne Parallel-Bäume. An der Wurzel steht *IExecutionEngine* als *Treiber der Messung* — der universelle messbare Vertrag aus Mess-Lebenszyklus (*warm_up/run/reset/shutdown*) und Auskünften (*engine_name/engine_kind/lifecycle_state*) über alle drei Engine-Arten Anatomy, Virus und Hybrid; unter ihm verzweigen die anatomie-tragenden *Konzepte* (Ebene 1: Map, Container, Graph, HeuristikAdapter) mit ihren *Gattungen* (Ebene 2) und die achsenlosen *Viren*. *SearchAlgorithm* ist das fokale Lebewesen (Elefant); seine Anatomie ist sein *Körper* — die feste Komposition und *Verdrahtung* der 18 Organe —, und „*SearchEngine*“ nur dessen ABI-Laufzeit-Sicht. Der Elefanten-Ast (Gattung *SearchAlgorithm*) ist bis zum Individuum ausgezogen: ein Genus (Ebene 3) mit seiner konkreten Achsen-Composition (Ebene 4) wird je Permutation als eigenständige Tier-Binary kompiliert (Ebene 5a, das Individuum). Container besitzt einen eigenen Organsatz mit den Gattungen Set, Sequence, Adapter und View, deren Aritäten die Gattungs-Bau-Zulassung führt; Graph ist deklariert und besitzt keine Gattung; das Konzept HeuristikAdapter führt die Reroute-Gattung der Hybrid-Stufe. 81
- 4.5 Die ABI-stabile C-Modulgrenze (sieben Pflicht-Symbole; die Hybrid-.so exportiert dieselbe Menge und meldet über *genus* ihre Ziel-Gattung) und die variadische 1/2/N-API als Sicht des Prüflings. Komplexe Schlüssel reduziert eine Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge; die Identität einer Binary führen davon getrennt das *permutation_fingerprint*-Feld des ABI-POD, der SHA-512-Tier-Fingerprint und der SHA-256-Planer-Fingerprint. 82

4.6	UML-Landkarte der Code-Interfaces: links die Engine-/Anatomie-Hierarchie — die messbare Wurzel <code>IExecutionEngine</code> als <i>Treiber der Messung</i> (<code>Anatomy ∪ Virus ∪ Hybrid</code>) über <code>IAatomyBase</code> , den vier Konzepten und sechs Gattungen und der achsenlosen <code>IVirusExecutionEngine</code> , darunter der <i>Körper</i> <code>SearchAlgorithmAnatomy<C></code> (Komposition und Verdrahtung der achtzehn Achsen) und seine ABI-Laufzeit-Sicht <code>SearchAlgorithmAbiAdapter<A></code> („SearchEngine“), die über die Treib-/Beobachter-Verträge <code>IDriveableTier/IObservableTier</code> gemessen wird und deren Messung als Fläche 3 durch den Visitor <code>IMessVisitor</code> quert; rechts der je Achse wiederholte Live-Weg — ein zweischichtiges Concept, eine requires-gesicherte CRTP-Basis, die vollständige <code>mp_list</code> -Baustein-Familie, der Übersetzungszeit-Filter über das Enable-Flag und der Composition-Slot, der genau einen Typ enthält. Die variant-basierte Auflösung <code>resolve_baustein</code> ist eine verworfene, vom Bau isolierte Alternative (gestrichelt).	83
4.7	Die Mess-Pipeline in zwei Ausprägungen. Der In-Process-Pfad (oben) ist die Sieben-Phasen-Pipeline des <code>ExperimentDriver</code> je Messreihe (<code>Enumerate</code> → <code>Generate</code> → <code>Compile</code> → <code>Link+Load</code> → <code>Execute</code> → <code>Measure</code> → <code>Persist</code> ; zwei opt-in-Phasen kompilieren fehlende Module im laufenden Experiment nach und fahren je Modul einen Funktionstest mit ABI-Versionsprüfung). Hervorgehoben sind die substanziellen Phasen: <code>COMPILE</code> (der <i>Build</i> der Tier-Binaries) und <code>EXECUTE/MEASURE</code> (die <i>Messung</i>); <code>Generate/Link+Load/Persist</code> sind Verdrahtung. Der Out-of-Process-Pfad (unten) ist eine Design-Entscheidung zur Prozess-Isolation: Der <code>BuildOrchestrator</code> baut <i>zuerst alle</i> Tier-Binaries (resumierbar, RAM-gewahr), danach lädt und misst <code>perm_runner</code> jede Binary einzeln über die Beobachter-Schnittstelle <code>IObservableTier</code> — eine DLL je Prozess, weil die DLL kein <code>main</code> enthält und der Runner das Executable ist, das sie lädt; kein Zustand wird zwischen Permutationen geteilt, und eine Cluster-Task fährt genau einen Runner mit genau einer DLL.	85
4.8	Die nachgelagerte Auswertungs-Werkzeugkette	86
4.9	Die Ebenen E4–E0 im Code	86
4.10	Die vier Abstraktionsebenen M0–M3	90
4.11	Die zwei Abschaltungsstufen der Messung	93
4.12	Das Register der Entwurfsmuster	97
4.13	Die Hybrid-Stufe im Code	99

5.1	Die automatisierte Evaluations-Kette von der Anwender-XML bis zum Manuskript. Obere Reihe: Die XML deklariert Was, Wo und Wann; der Validator prüft sie gegen die Registries; der Planer emittiert den deterministischen Plan; die CEB baut je System-Permutation die Tier-Binaries; die Messung erhebt die drei Mess-Ebenen und die Zähler mit einem Mess-Thread. Untere Reihe: Die Mappe ist der Stamm (ein Datenblatt unter konstantem Blattschlüssel; die Auffächerung in ein Blatt je Unter-Achsen-Permutation ist vorgesehen, aber nicht implementiert), die CSV-Form ihr aus ihr erzeugter, geordneter Baum aus CSV-Dateien (ein Ordner je Blatt; kein einzelnes Kind-Dokument; implementiert ist eine flache Datei), die Generatoren rendern je Sprache in das <code>tabellen</code> -Verzeichnis, das Manuskript bindet die Dateien ein — jede Zahl führt ihren Lauf. Gestrichelt: nicht implementierte Teile (XML-Erstellungsmodus, <code><publish></code> -Sektion, dritte Factory). .	103
5.2	Der Lager-Kreislauf im Mess-Betrieb: Die CEB baut in Batches zu je 4096 Binaries; jede Binary liegt unter ihrem SHA-512-Fingerprint in Bestand 1; das Wiedererkennungskriterium überspringt, was mit bestandenem Prüf-Testat inventarisiert ist, und lässt sonst bauen oder messen; die Messung läuft am Prüf-Dock mit einem Mess-Thread, während die CEB die parallelen Anfragen erzeugt; die Messwerte liegen unter Fingerprint und Hardware-Identität in Bestand 2 und fließen in die Mappe (die CSV-Baumform mit einem Ordner je Blatt ist vorgesehen; implementiert ist eine flache Datei). Die Struktur des Lagers (Baum, vier Bestände, Schlüssel-Schema) zeigt Abbildung 3.14; gestrichelt der Lager-Vollausbau.	110
5.3	Das Messkurven-Typsystm im heuristischen Gegenbeweis. Links: je Achse eine Kurve der Metrik über dem Achsen-Parameter, getrennt nach Mess-Ebene (wallclock, macro, micro); der Schnittpunkt zweier Varianten ist der Break-Even-Punkt, aus dem die Rekombination geschätzt wird. Mitte und rechts: Die CEB kompiliert die geschätzte Rekombination über den Hybrid-Modul-Emitter; die Hybrid-Tier-Binary ist die optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Mux-Stufe, dockt am Prüf-Dock an, führt Dock-Slots für Tier-Binaries und erzeugt selbst keinen Code; verglichen wird an der Wall-Clock, primär gegen die Paper-Rekonstruktionen, sekundär gegen das beste Einzel-Binary. Unten: die drei Bau-Varianten des Messfehler-Prüfplans, auseinandergehalten über die Gate-Felder des Fingerprints. Gestrichelt: Router und Eviction (nicht implementiert).	111
A.1	Zyklen je Lebewesen-Permutation (v5_pipeline_demo)	170
A.2	Zyklen je Lebewesen-Permutation (cartesian_smoke43)	172
A.3	3D-Rohdaten-Ansicht: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)	174
A.4	3D-Rohdaten-Ansicht: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	174
A.5	Working-Set-Sweep: Gesamt-Latenz (ns/op)	176
A.6	Working-Set-Sweep: Lookup-Latenz p50 (ns/op)	179
A.7	Gestapelte Per-Achsen-Latenz-Attribution	180
A.8	Latenz-Spanne p50-p99 je Suchalgorithmus x Operation; Punkt=p50, Whisker=p99.	181
A.9	ECDF der Gesamt-Latenz – Verteilung ueber Konfigurationen	182

A.10 Pareto-Streuung: Median- gegen Tail-Latenz (ein Punkt je Konfiguration und Op-Art; Diagonale = kein Tail-Aufschlag)	183
---	-----

Tabellenverzeichnis

2.1	Cache-nahe Struktur-Entwürfe und die Hardware-Parameter, die sie explizit in den Entwurf binden.	20
3.1	Dialektische Aneignung der Stand-der-Technik-Instanzen je Kompositions-Achse (in kanonischer Slot-Reihenfolge T0–T17; Telemetrie und Ziel-ISA gehören zur System-Sphäre bzw. zum Mess-Tooling, Abschnitt 3.4.1. Cluster-Kürzel: A Trie/Knoten, B Hybrid/Layout, C Allokation, D Prefetching, E Synchronisation, F Hardware/Messung/Engineering)	27
3.2	Achsen-Sezierung der Suchverfahren: achtzehn Organ-Achsen in kanonischer Slot-Reihenfolge (T0–T17), die Größen der System- und Mess-Sphäre, die Prüfling-Build-Achse und die optionale Organ-Meta-Meta-Achse; je Achse die ein Organ beitragenden Quellen.	30
3.3	Allokator-Profil (23, alle mit <code>expected_workload</code> -Tag) mit Lizenz, Workload-Affinität und Anbindungsstand	31
3.4	In dieser Arbeit zugewiesene Ziel-Hardware- und Scheduling-Profil je rekonstruiertem SOTA-Verfahren (Auswahl; die Profile sind Rekonstruktions-Ziele dieser Arbeit, nicht zwingend Original-Paper-Eigenschaften; die Spalte Hardware-Strategie folgt dem Vokabular des im Code konservierten, konsumentenlosen Achsen-Entwurfs <code>hardware_strategy.hpp</code> , die Ziel-ISA jeder Rekonstruktion steht im Feld <code><isa></code> ihres SOTA-Profiles (System-Sphäre, im Code eine Build-Konstante; P03 und P31: <code>X86_AVX2</code> , P27 und P32: <code>X86_AVX512</code>); der Achsen-Katalog HW1–HW4 mit den Plattform-Profilen folgt in Anhang D, Abschnitt „Build-HW“, eine Paper-für-Paper-Liste gibt es nicht).	32
3.5	Benchmark-Frameworks und nutzende Veröffentlichungen	34
3.6	Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)	34
3.7	SOTA-Profil (33: 30 Vollprofile, 3 abstrakte) mit Workload-Affinität	37
3.8	Zuordnung der Drei-Stufen-Prüfung zu den Pflicht-Messreihen	61
4.1	Die vier Arbeitsmodi einer Permutation als registrierte Unter-Achse des Mess-Realms (<code>WorkMode</code> , vier Einträge; ein fünfter bricht die Übersetzung). Kein Modus misst parallel; nur <code>measure</code> schaltet die Messung ein.	74
4.2	Achsen-Typologie je Kategorie (<code>AxisCategory</code> mit der Ordnung <code>mess < system < organ</code> ; Familien-Diskriminator <code>AxisKind</code> mit sechs Werten). Die Zahlen sind Strukturzahlen des Quellbaums (Registry-Header und generierte Registry-XML), keine Messwerte.	78

4.3	Der Stempel-Vertrag als Matrix aus sieben Interfaces und vier Trägern (28 Zellen, dreiwertige Zulassung P = Pflicht, E = Erlaubt, V = Verboten; E ist im Bestand unbelegt). Die Verbots-Zelle der Organ-Zeile beim CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“; die Zellen werden je Erbin per <code>static_assert</code> geprüft. Die Spaltenordnung folgt wie Abbildung 3.9 der Verarbeitungs-Topologie mit der Hybrid-Tier-Binary als optionalem Einschub zwischen CEB und Tier-Binary.	87
4.4	Die vier Unter-Achsen-Registries des Mess-Toolings: je eine <code>constexpr</code> -Datei mit Index-gleich-Enum-Prüfung; die Haupt-Achse selbst hält die drei Toolings wallclock, macro und micro.	92
4.5	Die vier Lager-Bestände und ihre Schlüssel-Komponenten nach dem übersetzungsstatischen Schlüssel-Schema; der Versions-Lock bindet die Achsen-Quellen an Digest und gestempelte Version.	99
5.1	Profil-bewusstes Workload-Routing (Fallback-Heuristik)	104
5.2	Reale Datensätze, ihre Charakteristik und ihre Reproduzierbarkeits-Akte	104
5.3	Die Zähler der Mess-Kette und ihre Quellen: vier generische perf-Events (portabel), zwei modell-gebundene RAW-Events aus dem Katalog für Zen 5 (durch Messung gegengeprüft), die RAPL-Energie als best-effort-Lesung und IPC/CPI als deklarierte, nicht angebundene Kategorie. Je Zähler führt der Zähler-Verbund ein <code>*_source_available</code> -Flag; eine Zahl heißt „Quelle hat geliefert“, n/a heißt „Quelle fehlt“. Der Intel-Stand nennt, was ohne Katalog-Eintrag auf prod2 erhoben wird.	107
5.4	Die drei Mess-Spaltenlisten der Cache-Engine und ihre Hoheit. <code>total_cycles</code> der Pipeline-Sicht führt die repräsentative p50-Latenz in Nanosekunden (Legacy-Name), keine Zyklen. F15-ALT umfasst zwölf Spalten und, hinten angehängt, degeneriert (12 + 1).	113
5.5	Die merge-Token der Anwender-XML und ihre Verbund-Strategien im Merge-Plan; die Stufen-Namen sind Aliasse, der Full Join ist der Alias von <code>union</code> . Die Reihen-Spalte folgt der Zuordnung aus Tabelle 3.8.	115
5.6	Gültigkeit der Evaluation nach den vier Klassen der Bedrohungen (Konstrukt, intern, extern, Schluss): je Bedrohung die Gegenmaßnahme im Apparat und ihr Umsetzungsstand — implementiert, deklariert und nicht gemessen (n/a) oder nicht implementiert. Keine Zeile nennt eine Zahl ohne Lauf.	117
6.1	Die Beweiskette je Haupt- und Teilfrage: Status in drei Stufen (beantwortet, strukturell beantwortet, offen), die stützenden Abschnitte mit ihrer Belegart (Struktur, Test, Messung; für FF2.d der Literaturbeleg) und der offene Rest ohne Zahl.	127
A.1	Artefakte des Mess-Anhangs: Kennzahl, Mess-Ebene (w/ma/mi) und die im vorliegenden Korpus enthaltene Datenbasis; Zahlen nur aus den eingebundenen Artefakten	168
A.2	Messreihe <code>v5_pipeline_demo</code> : vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, <code>run_workload</code>)	169
A.3	Messreihe <code>cartesian_smoke43</code> : vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, <code>run_workload</code>)	171

A.4	Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)	172
A.5	Nicht erhobene Kennzahlen der Heatmap-Familie <code>lc_surface_*</code> (Suchalgorithmus × Lastprofil): sechs Vermerke statt Flächen — zwei Kennzahlen gemessen, aber als 1×1-Matrix nicht darstellbar, vier nicht erhoben	173
A.6	Nicht erhobene Kennzahlen der 3D-Familie <code>lc_surface3d_*</code> (Suchalgorithmus × Lastprofil): vier Vermerke statt Ansichten; die zwei gemessenen Kennzahlen (Gesamt-Latenz, Lookup-Latenz p50) zeigen die zwei Ansichten dieses Abschnitts mit je einem Stützpunkt	175
A.7	Nicht erhobene Kennzahlen der Sweep-Familie <code>ld_sweep_*</code> (Latenz über der Arbeitsmengen-Größe <i>n</i>): ein Vermerk statt Kurve; die zwei gemessenen Kennzahlen (Gesamt-Latenz, Lookup-Latenz p50) zeigen die zwei Kurven dieses Abschnitts mit je einem Stützpunkt	175
A.8	Per-Achsen-Observer-Detailwerte je Tier-Binary (nur echt gemessene Zaehler)	176
A.9	Nicht erhobene Kennzahlen der Verhältnis-Familie <code>lc_surface_ratio_*</code> (Quotient Median/Referenz-Median): sechs Vermerke statt Matrizen, weil die Referenz-Ausprägung <code>search_algo/linear_scan</code> im Korpus nicht mitgemessen ist	181
A.10	Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)	184
E.1	Übersicht der zwanzig Architekturentscheidungen: Kurztitel, belegender Abschnitt und Umsetzungsstand — <i>umgesetzt</i> (implementiert und belegt), <i>teilweise</i> (implementiert, mit ausgewiesenem offenem Gegenstand) oder <i>Entwurf</i> (nur entworfen, nicht implementiert); die Begründungen stehen in den Datensätzen, nicht in der Tafel	263
F.1	Vollständige <code>std::map</code> -Schnittstelle (C++23)	274
F.2	Vollständige <code>std::vector</code> -Schnittstelle (C++23)	277
F.3	Zusammengesetzte Funktionen (interne Komposition primitiver Operationen)	280

A Vollständige Messreihen

Dieser Anhang dokumentiert die Messreihen und Auswertungs-Artefakte der Mess-Kette (Abschnitt 3.4.6). Tabelle A.2 listet je vorkonfigurierter Lebewesen-Permutation die Kennzahlen des 16-Spalten-Schemas: die Operationszahl (`op_count`), die repräsentative p50-Latenz in Nanosekunden — im Schema unter dem Legacy-Spaltennamen `total_cycles` geführt und in der Tabelle mit „Zyklen“ beschriftet; es sind keine Taktzyklen¹ — sowie die Spalten der Cache-Misses L1–L3, die in dieser Reihe nicht erhoben sind (`pmc_available` = 0) und deshalb als 0 tabelliert stehen: Sie sind als explizit kodierte Nicht-Erhebung zu lesen (Grundsatz der ausgewiesenen Nichterhebung, Abschnitt 3.4.1), nicht als gemessene Null; eine Ausgabe dieser Zellen als n/a ist nicht implementiert. Mit gesetztem Schalter `COMDARE_ENABLE_PMC` (Bau-Option, Vorgabe aus, damit die Unit-Tests ohne Hardware-Zugriff bauen; die CI setzt ihn in den PMC-Pfaden) erhebt die Mess-Kette L1D-, Last-Level-, dTLB- und Branch-Misses generisch über `perf_event_open`² [Mel10] sowie L2-Demand-Misses und Fills aus dem Cache eines fremden CCX³ modellgebunden über den RAW-Event-Katalog — für die AMD-Mikroarchitektur Zen 5 (CUID-Familie 26) durch Messung gegengeprüft [Lin25, WM08]; Modelle ohne Katalog-Eintrag (alle Intel-Modelle) führen n/a (Limitierung 1 in Abschnitt A.10; Abschnitt 5.1.5). Abbildung A.1 stellt dieselbe Kennzahl grafisch gegenüber; ihre Achsenbeschriftung „Zyklen“ folgt dem Legacy-Spaltennamen. Die hier dokumentierten Reihen sind Smoke-Nachweise des vollständigen Ketten-Durchlaufs mit Achsen-Variation; sie ersetzen die Pflicht-Messreihen A–C (Abschnitt 5.2.4, Tabelle 3.8) nicht, deren vollständige Läufe nicht vorliegen (n/a), weil der vollständige Bau nicht durchgeführt ist.

Die zwei Permutations-Tabellen (Tabellen A.2 und A.3) entstammen dem In-Process-Pfad (`run_workload`) eines Ketten-Durchlaufs auf der Entwicklungsmaschine (Abschnitt 5.2.4). Alle übrigen Artefakte dieses Anhangs stammen aus *einem* Smoke-Lauf der Mess-Pipeline auf der AMD-Zen-5-Maschine — Stufe `measure:smoke` des ersten durchgehenden Laufs der Kette⁴, System-Permutation Optimierungsstufe O3 mit SIMD-Option `no_extension` —, und dieser Lauf enthält genau *eine* Zelle: ein Suchverfahren (`k_ary`), ein Lastprofil (`y_csb_c`), ein Tier-Binary, eine Arbeitsmenge $n = 4096$, $N = 1$. Die vermessene Komposition ist über ihre `binary_id` mit 18 Segmenten identifiziert (Kopfzeile von Tabelle A.8); Stempel-Versionszeile, Fingerprint, Host, Datum und Lauf-Kennung führen die Generator-Ausgaben nicht mit (n/a im Artefakt) — eine Provenienz-Zeile je Artefakt, die die Identitäts-Stempel aus Abschnitt 3.4.4 in Tabellen und Diagrammen lesbar macht, ist im Generator nicht implementiert; der Beleg ist die Beweiskette dieses Absatzes. PMC-Kennzahlen

¹`total_cycles`: Das Feld des Snapshot-Schemas enthält die repräsentative Latenz der Messung in Nanosekunden (den zusammengeführten p50-Wert), nicht gezählte Prozessortakte; der Spaltenname ist ein Erbe der ersten Schema-Fassung; die Spalte heißt daher „Zyklen“ und bezeichnet die p50-Latenz in Nanosekunden.

²`perf_event_open(2)`: der Linux-Systemaufruf, der einen Hardware- oder Software-Zähler als Dateideskriptor öffnet; generische Ereignisse bildet der Kernel je Mikroarchitektur ab, modellspezifische verlangen die rohe Kodierung des Ereignisses (`PERF_TYPE_RAW`: Ereignisnummer und Maske, wie die Hardware sie zählt).

³CCX (Core Complex): ein Kern-Komplex mit eigenem L3-Cache; Demand-Fills aus dem Cache eines anderen CCX sind die Kohärenz-Beobachtbare des Core-PMU, ein wörtlicher Invalidierungs-Zähler existiert im Ereignis-Satz nicht.

⁴Provenienz des Smoke-Laufs: Pipeline 15800, Mess-Job 377503, Host prod1 (AMD Zen 5); diese Angaben stammen aus dem Lauf-Protokoll der Pipeline, die Artefakte selbst enthalten keine davon.

erscheinen in diesem Anhang nur als Tabellenspalten (nicht erhoben); ein PMC-Abbildungstyp ist im Generator nicht vorhanden. Jede Zahl dieses Anhangs stammt aus diesen Artefakten; keine andere Messung wird zitiert.

Die Kennzahlen dieses Anhangs verteilen sich auf die drei Mess-Ebenen der Mess-Tooling-Haupt-Achse — wallclock (w), macro (ma) und micro (mi) in genau dieser Reihenfolge (Abschnitt 3.4): `ns_per_op` = `total_ns/timed_ops` ist die Host-Wall-Clock der Messung über das Last-Rahmenwerk (w); `op_<art>_p50` und `op_<art>_p99` sind die Latenzen der Gattungs-Interface-Operationen `insert`, `lookup`, `erase`, `scan` und `rmw` der Tier-Binary (ma); `seg_<achse>_ns` und `stat_<achse>_<feld>` sind die Per-Achsen-Zeiten und Observer-Zähler über die Achsen-Interfaces (mi); die PMC-Spalten gehören zur Micro-Instrumentierung (mi). Dieser Anhang definiert die Ebenen nicht neu, er ordnet zu: Tabelle A.1 nennt je Artefakt die Kennzahl, ihre Ebene und die Datenbasis, die der vorliegende Korpus enthält. In jedem Abschnitt ist die Design-Aussage (was eine Darstellung leistet) von der Korpus-Aussage (was der vorliegende Lauf enthält) getrennt.

Tabelle A.1: Artefakte des Mess-Anhangs: Kennzahl, Mess-Ebene (w/ma/mi) und die im vorliegenden Korpus enthaltene Datenbasis; Zahlen nur aus den eingebundenen Artefakten

Artefakt	Kennzahl	Ebene	Datenbasis im Korpus
Tabelle A.2, Abbildung A.1	<code>op_count</code> ; p50-Latenz (ns) unter „Zyklen“; HW-Spalten L1–L3	w (HW-Spalten: mi)	6 Permutationen, 2000 Operationen je Zeile; HW-Spalten 0 = nicht erhoben
Tabelle A.3, Abbildung A.2	wie oben	w (HW-Spalten: mi)	43 Permutationen (27 + 16), 1000 Operationen je Zeile; HW-Spalten 0 = nicht erhoben
Bias-Bruch-Matrix (Tabelle A.4)	Median <code>ns_per_op</code> (nearest-rank), nur <code>two_phase_valid</code>	w	1×1 (<code>k_ary</code> × <code>y_csb_c</code>): 672 ns/op
Heatmaps <code>lc_surface_*</code> (6)	<code>ns_per_op</code> ; <code>op_*_p50</code>	w; ma	<code>ns_per_op</code> und <code>lookup</code> gemessen, aber 1×1 unter dem 2×2-Minimum; <code>insert</code> , <code>erase</code> , <code>scan</code> , <code>rmw</code> nicht ausgeführt — sechs Vermerke, keine Fläche (bis zur vollständigen Messreihe durch Tabelle A.5 vertreten)
3D-Ansichten <code>lc_surface3d_*</code> (6)	wie oben	w; ma	zwei Ansichten mit je einem Stützpunkt (671,5 ns/op; 750 ns), vier Vermerke (nicht ausgeführt) (bis zur vollständigen Messreihe durch Tabelle A.6 vertreten)
Sweep-Kurven <code>ld_sweep_*</code> (3)	<code>ns_per_op</code> , <code>op_insert_p50</code> , <code>op_lookup_p50</code> über <i>n</i>	w; ma	ein gemessenes <i>n</i> (4096): je ein Messpunkt für <code>ns_per_op</code> und <code>lookup</code> ; <code>insert</code> nicht ausgeführt (Vermerk) (bis zur vollständigen Messreihe durch Tabelle A.7 vertreten)
Per-Achsen-Attribution <code>seg_attribution</code>	<code>seg_<achse>_ns</code> gegen <code>seg_run_total_ns</code>	mi	ein Balken (<code>k_ary</code>); 19 Segmente = 18 Organ-Achsen + Framework-Rest
Observer-Detail (Tabelle A.8)	<code>stat_<achse>_<feld></code>	mi	ein Tier-Binary (<code>binary_id</code> mit 18 Segmenten)

Fortsetzung nächste Seite

Tabelle A.1 — Fortsetzung

Artefakt	Kennzahl	Ebene	Datenbasis im Korpus
Range-Balken <code>latency_range</code>	<code>op_*_p50</code> und <code>op_*_p99</code>	ma	eine Kategorie (<code>k_ary/lookup</code>): <code>p50</code> 750 ns, <code>p99</code> 1170 ns
ECDF <code>latency_ecdf</code>	<code>ns_per_op</code> über die Konfigurationen	w	$N = 1$: einstufige Treppe bei 671,5 ns/op
Pareto-Streuung <code>latency_</code> <code>tradeoff</code>	<code>op_*_p50</code> gegen <code>op_*_p99</code>	ma	ein Punkt (750/1170 ns); keine Front
Austauschbarkeit <code>ld_</code> <code>exchange_*</code> (4 Langtabellen)	Median rel. Δ ns/op je Geschwister-Paar, IQR	w; ma	0 von 480/480/480/640 Geschwister-Paaren: leere Hüllen (bis zur vollständigen Messreihe durch eine Ersatzzeile vertreten)
Forest-Plots <code>exchange_forest,</code> <code>exchange_forest_vs_</code> <code>reference</code>	wie oben, aggregiert	w	keine Datei (0 Paare; Referenz nicht gemessen): Ersatztext
Verhältnis-Matrizen <code>lc_</code> <code>surface_ratio_*</code> (6)	Quotient Median/Referenz-Median	w; ma	Referenz <code>linear_scan</code> nicht gemessen: sechs Vermerke (bis zur vollständigen Messreihe durch Tabelle A.9 vertreten)
Normalisierte Balken <code>lc_</code> <code>normbar_*</code> (6)	wie oben, je Suchalgorithmus	w; ma	keine Datei (Referenz nicht gemessen): Ersatztexte
Limitierungen (Tabelle A.10)	Vorbehaltsliste, keine Messzahl	—	16 Zeilen

Permutation	Ops	Zyklen	L1	L2	L3
ArtComposition_0	2000	825053	0	0	0
HotComposition_1	2000	740176	0	0	0
StartComposition_2	2000	805750	0	0	0
SurfComposition_3	2000	873976	0	0	0
WormholeComposition_4	2000	696350	0	0	0
MasstreeComposition_5	2000	770871	0	0	0

Tabelle A.2: Messreihe `v5_pipeline_demo`: vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, `run_workload`)

A.1 Kartesische Smoke-Messreihe (43 Permutationen, DLL-Messung)

Die folgende Messreihe entstand aus dem vollständigen Ketten-Durchlauf: 43 gebaute Permutations-DLLs in zwei Teilblöcken — 27 kartesische Kombinationen `SIMD` $\times$ `Speicherlayout` $\times$ `Allokator` (`avx2`, `scalar`, `sse4` $\times$ `aos`, `hybrid`, `soa` $\times$ `jemalloc`, `mimalloc`, `std`) und 16 Kombinationen über vier zweiwertige Größen (Knotenbreite `compact/wide`, Pfadkompression `lazy/none`, interne Suche `binary/linear`, Telemetrie `leaf_only/sampling`; 2^4) — wurden am Prüf-Dock gemessen, als binäre Mess-Records persistiert, über die Pipeline-Stufe 03 in das 16-Spalten-Schema konvertiert und automatisch eingebunden. Die Variantennamen sind die Bau-Matrix dieser Reihe; im Achsenmodell des Codes ist `SIMD` eine System-Achse (Option `avx2` der Registry der System-Achsen), `Speicherlayout` und `Allokator` sind Organ-Achsen (`memory_layout`, `allocator`), und Telemetrie ist keine

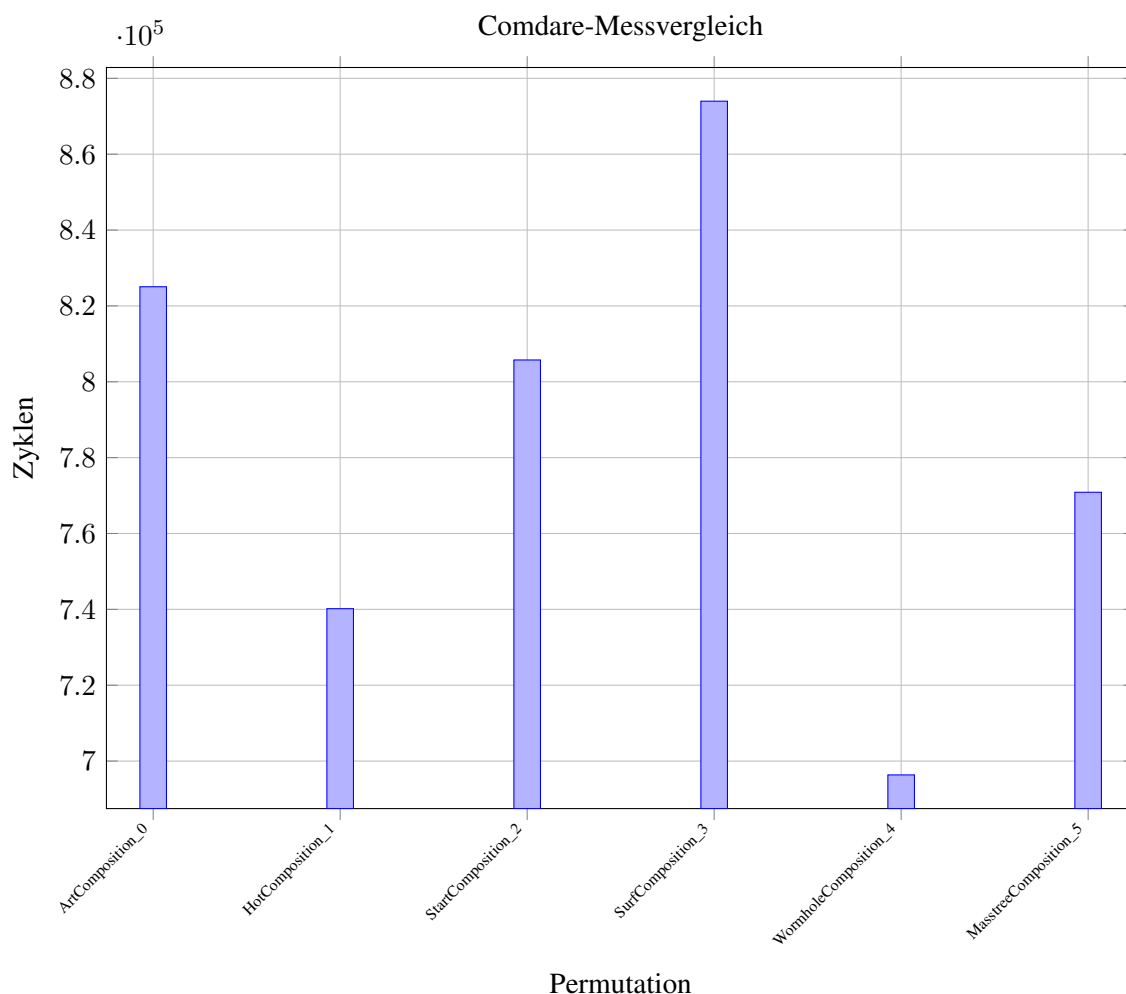


Abbildung A.1: Zyklen je Lebewesen-Permutation (v5_pipeline_demo)

binary_id-Achse (Limitierung 2 in Tabelle A.10); die übrigen Namen sind nicht namensgleich mit Registry-Bausteinen und werden nicht gleichgesetzt. Die Reihe variiert damit sieben Größen; die vollständige Messreihe variiert alle 18 Kompositions-Achsen, dazu die optionale Organ-Meta-Meta-Achse disk_io ohne binary_id-Anteil. Tabelle A.3 und Abbildung A.2 zeigen die Ergebnisse; die HW-Spalten stehen auf 0 = nicht erhoben, und die Achsenbeschriftung „Zyklen“ der Abbildung folgt dem Legacy-Spaltennamen (siehe oben).

A.2 Bias-Bruch-Matrix

Tabelle A.4 fasst die je Suchverfahren und Lastprofil gemessene Median-Latenz (ns/op, nearest-rank⁵) über alle dynamischen Einstellungen und Wiederholungen zusammen; berücksichtigt sind ausschließlich die als two_phase_valid⁶ markierten Läufe. Die Matrix macht sichtbar, bei welchen Lastprofilen das jeweilige Verfahren seinen systematischen Bias bricht. Im vorliegenden Korpus enthält die Matrix eine

⁵Nearest-Rank: Das q -Quantil ist der kleinste Stichprobenwert, unter dem mindestens der Anteil q der Stichprobe liegt (Hyndman und Fan, Typ 1); es wird nicht interpoliert, jedes Perzentil ist ein tatsächlich gemessener Wert [HF96].

⁶two_phase_valid: das Gültigkeits-Flag der Zwei-Phasen-Operationsschleife — nur eine Messung, deren zweite (warme) Phase gültig abgeschlossen wurde, geht in die Auswertung ein (Abschnitt 3.4).

Permutation	Ops	Zyklen	L1	L2	L3
avx2_aos_jemalloc	1000	8025	0	0	0
avx2_aos_mimalloc	1000	7995	0	0	0
avx2_aos_std	1000	7995	0	0	0
avx2_hybrid_jemalloc	1000	315164	0	0	0
avx2_hybrid_mimalloc	1000	243930	0	0	0
avx2_hybrid_std	1000	230759	0	0	0
avx2_soa_jemalloc	1000	423930	0	0	0
avx2_soa_mimalloc	1000	400634	0	0	0
avx2_soa_std	1000	398265	0	0	0
scalar_aos_jemalloc	1000	6975	0	0	0
scalar_aos_mimalloc	1000	7005	0	0	0
scalar_aos_std	1000	6975	0	0	0
scalar_hybrid_jemalloc	1000	1366319	0	0	0
scalar_hybrid_mimalloc	1000	1279365	0	0	0
scalar_hybrid_std	1000	1356329	0	0	0
scalar_soa_jemalloc	1000	440835	0	0	0
scalar_soa_mimalloc	1000	394200	0	0	0
scalar_soa_std	1000	438960	0	0	0
sse4_aos_jemalloc	1000	16845	0	0	0
sse4_aos_mimalloc	1000	14160	0	0	0
sse4_aos_std	1000	14324	0	0	0
sse4_hybrid_jemalloc	1000	1281180	0	0	0
sse4_hybrid_mimalloc	1000	1275404	0	0	0
sse4_hybrid_std	1000	1279350	0	0	0
sse4_soa_jemalloc	1000	397095	0	0	0
sse4_soa_mimalloc	1000	392294	0	0	0
sse4_soa_std	1000	396780	0	0	0
pa_compact_lazy_binary_leaf_only	1000	348105	0	0	0
pa_compact_lazy_binary_sampling	1000	375600	0	0	0
pa_compact_lazy_linear_leaf_only	1000	445815	0	0	0
pa_compact_lazy_linear_sampling	1000	423090	0	0	0
pa_compact_none_binary_leaf_only	1000	360765	0	0	0
pa_compact_none_binary_sampling	1000	346980	0	0	0
pa_compact_none_linear_leaf_only	1000	398939	0	0	0
pa_compact_none_linear_sampling	1000	391875	0	0	0
pa_wide_lazy_binary_leaf_only	1000	309300	0	0	0
pa_wide_lazy_binary_sampling	1000	311805	0	0	0
pa_wide_lazy_linear_leaf_only	1000	445605	0	0	0
pa_wide_lazy_linear_sampling	1000	492495	0	0	0
pa_wide_none_binary_leaf_only	1000	307110	0	0	0
pa_wide_none_binary_sampling	1000	311925	0	0	0
pa_wide_none_linear_leaf_only	1000	445140	0	0	0
pa_wide_none_linear_sampling	1000	478635	0	0	0

Tabelle A.3: Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (In-Process-Pfad, run_workload)

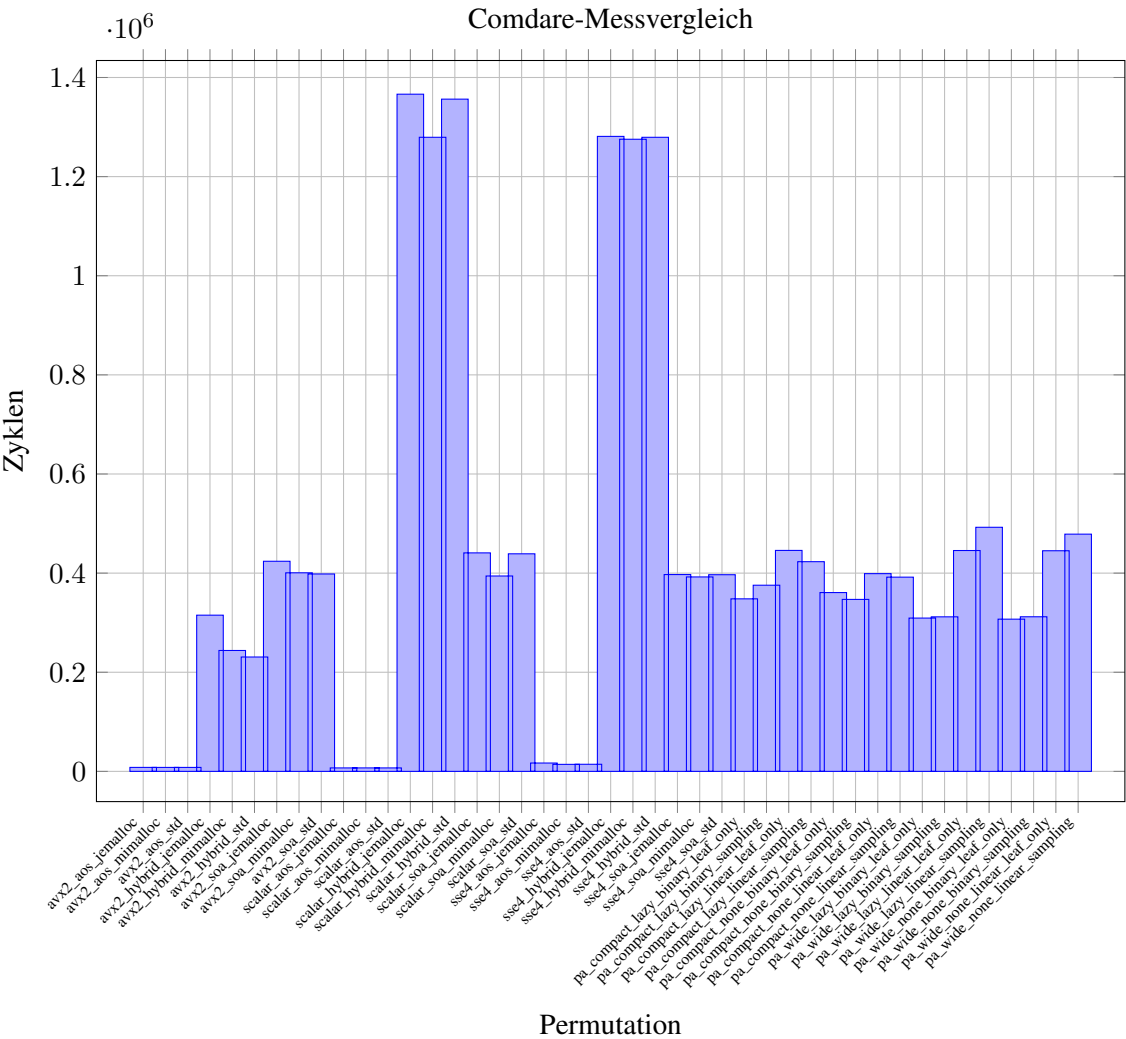


Abbildung A.2: Zyklen je Lebewesen-Permutation (cartesian_smoke43)

Zelle ($k_{\text{ary}} \times y_{\text{csb_c}}$, 672 ns/op — ganzzahlig formatiert; dieselbe Zelle steht in den Diagrammen als 671,5 ns/op); ein Bruch-Muster über Lastprofile ist mit einer 1×1 -Matrix nicht ablesbar — eine Aussagekraft setzt die vollständige Messreihe voraus (n/a).

Suchverfahren	$y_{\text{csb_c}}$
	672

Tabelle A.4: Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)

A.3 Latenz-Surfaces je Schnittstellen-Funktion

Für jede der sechs Kennzahlen — die Gesamt-Latenz (`ns_per_op`) sowie die p50-Latenzen der Einzeloperationen `insert`, `lookup`, `erase`, `scan` und `rmw` — ist eine Latenz-Oberfläche als Heatmap⁷ Suchalgorithmus $\times$ Lastprofil vorgesehen. Im vorliegenden Korpus enthält keine der sechs eine darstellbare Fläche, aus zwei verschiedenen Gründen: Die Gesamt-Latenz und die p50-Latenz von `lookup` sind gemessen, liegen aber als 1×1 -Matrix unter dem 2×2 -Minimum der `pgfplots`-Flächenform; die Operationen `insert`, `erase`, `scan` und `rmw` wurden im Korpus nicht ausgeführt (`op_<art>_n = 0`). Beide Fälle stehen als explizit kodierte Nicht-Erhebung im Artefakt — als Vermerk, der den Grund nennt —, nie als Nullfläche.

Tabelle A.5 fasst die sechs Vermerke zusammen; die Vermerk-Artefakte des Generators bleiben erhalten und erscheinen nach der vollständigen Messreihe als Heatmaps an dieser Stelle.

Kennzahl	Status im Korpus	Grund (Vermerk des Artefakts)
Gesamt-Latenz (<code>ns_per_op</code>)	gemessen, nicht darstellbar	1×1 -Matrix (<code>k_ary $\times$ ycsb_c</code>) unter dem 2×2 -Minimum der <code>pgfplots</code> -Flächenform; Messwert im Korpus vorhanden (Tabelle A.4)
Insert-Latenz p50 (<code>op_insert_p50</code>)	nicht erhoben	Operation im Korpus nicht ausgeführt (<code>op_insert_n = 0</code>)
Lookup-Latenz p50 (<code>op_lookup_p50</code>)	gemessen, nicht darstellbar	1×1 -Matrix unter dem 2×2 -Minimum der <code>pgfplots</code> -Flächenform; Messwert im Korpus vorhanden (Tabelle A.1)
Erase-Latenz p50 (<code>op_erase_p50</code>)	nicht erhoben	Operation im Korpus nicht ausgeführt (<code>op_erase_n = 0</code>)
Scan-Latenz p50 (<code>op_scan_p50</code>)	nicht erhoben	Operation im Korpus nicht ausgeführt (<code>op_scan_n = 0</code>)
RMW-Latenz p50 (<code>op_rmw_p50</code>)	nicht erhoben	Operation im Korpus nicht ausgeführt (<code>op_rmw_n = 0</code>)

Tabelle A.5: Nicht erhobene Kennzahlen der Heatmap-Familie `lc_surface_*` (Suchalgorithmus $\times$ Lastprofil): sechs Vermerke statt Flächen — zwei Kennzahlen gemessen, aber als 1×1 -Matrix nicht darstellbar, vier nicht erhoben

A.4 Dreidimensionale Rohdaten-Ansicht

Dieselben sechs Kennzahlen sind zusätzlich als 3D-Rohdaten-Ansicht über der Ebene (Suchalgorithmus $\times$ Lastprofil) vorgesehen. Im Korpus enthalten zwei Ansichten je einen Stützpunkt (`ns_per_op`: 671,5 ns/op; `lookup-p50`: 750 ns) und vier den Vermerk der nicht ausgeführten Operation; eine Fläche spannt der Korpus nicht auf. Beide Flächen-Achsen sind *nominal*: die aufgespannte Fläche enthält bewusst *keine* Interpolations- oder Trendaussage und ist ausschließlich als Rohdaten- und Prüfansicht neben der Heatmap zu lesen.

Tabelle A.6 fasst die vier Vermerke zusammen; die Vermerk-Artefakte des Generators bleiben erhalten und erscheinen nach der vollständigen Messreihe als 3D-Ansichten an dieser Stelle.

⁷Heatmap: eine Matrix, deren Zellen den Wert farbkodiert darstellen; die Form geht auf die geclusterte Wärmekarte der Statistik zurück [WF09].

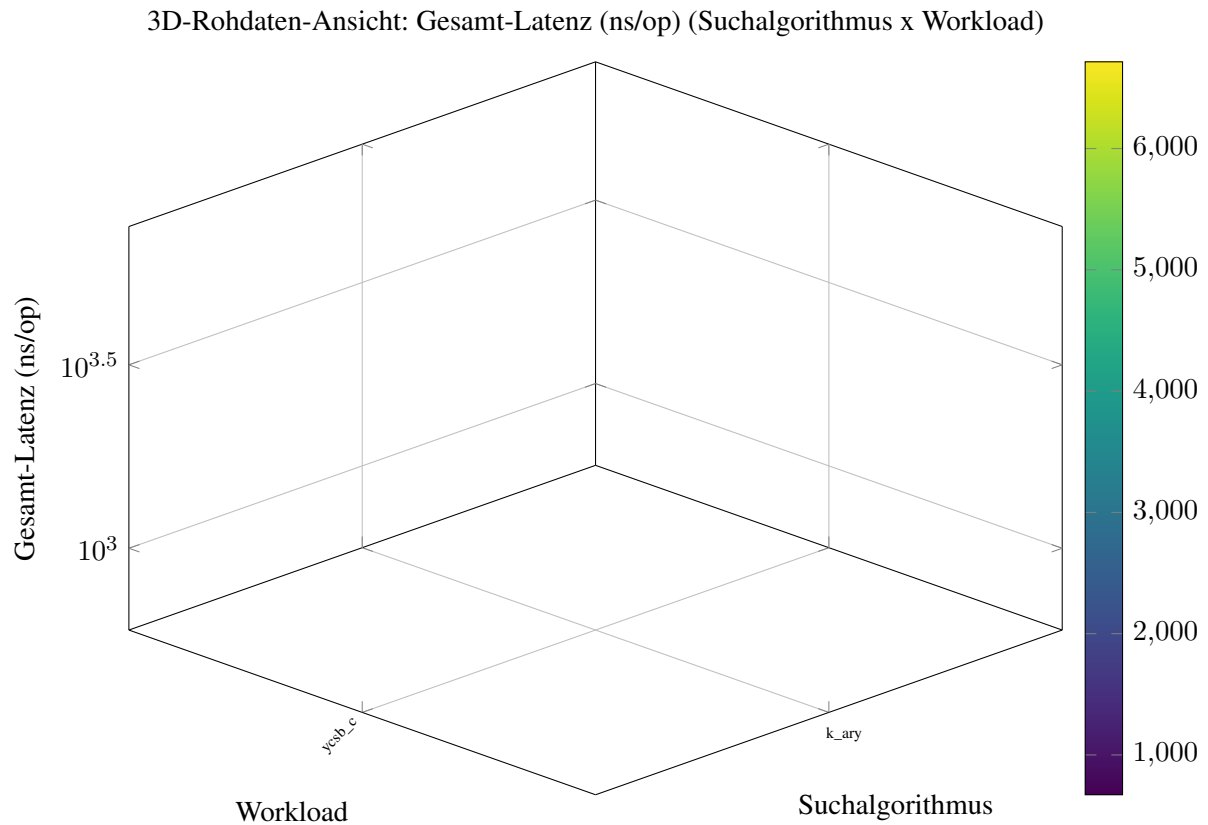


Abbildung A.3: 3D-Rohdaten-Ansicht: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)

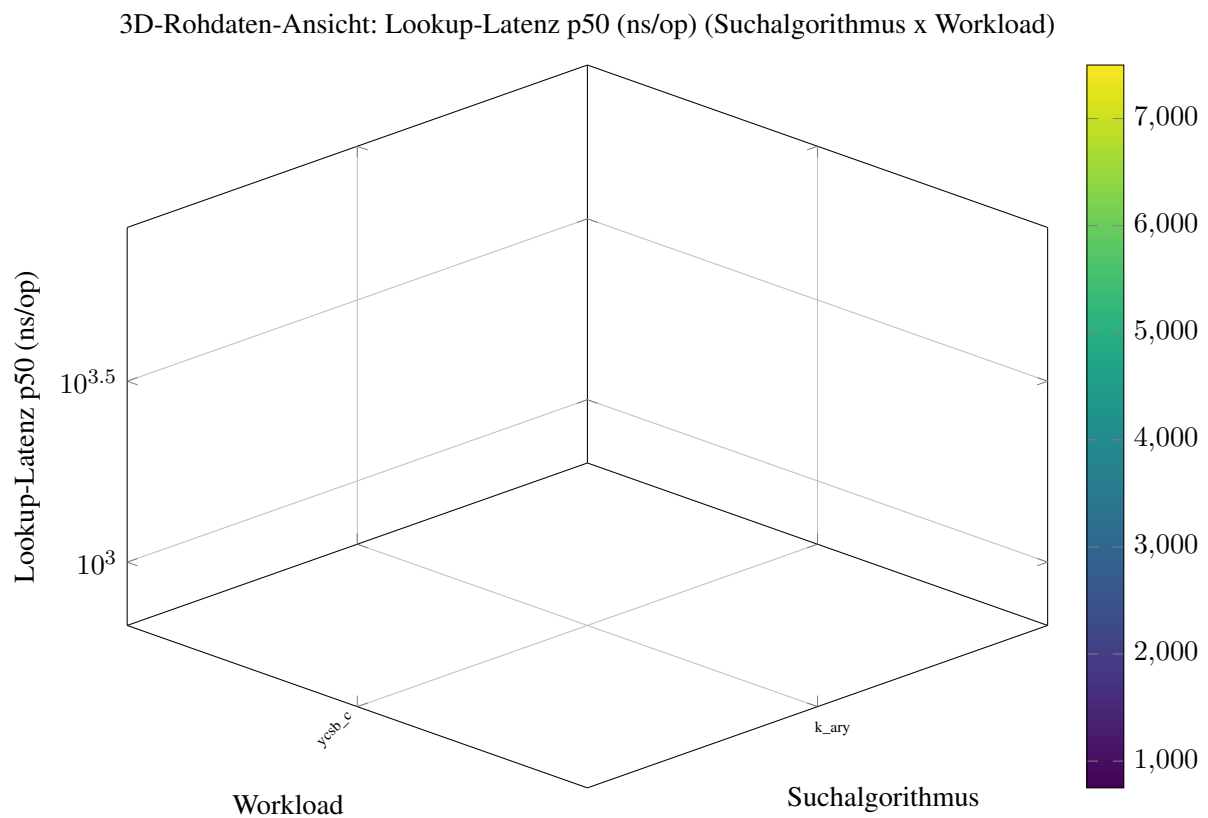


Abbildung A.4: 3D-Rohdaten-Ansicht: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

Kennzahl	Status im Korpus	Grund (Vermerk des Artefakts)
Insert-Latenz p50 (op_insert_p50)	nicht erhoben	Operation im Korpus nicht ausgeführt (op_insert_n = 0)
Erase-Latenz p50 (op_erase_p50)	nicht erhoben	Operation im Korpus nicht ausgeführt (op_erase_n = 0)
Scan-Latenz p50 (op_scan_p50)	nicht erhoben	Operation im Korpus nicht ausgeführt (op_scan_n = 0)
RMW-Latenz p50 (op_rmw_p50)	nicht erhoben	Operation im Korpus nicht ausgeführt (op_rmw_n = 0)

Tabelle A.6: Nicht erhobene Kennzahlen der 3D-Familie `lc_surface3d_*` (Suchalgorithmus $\times$ Lastprofil): vier Vermerke statt Ansichten; die zwei gemessenen Kennzahlen (Gesamt-Latenz, Lookup-Latenz p50) zeigen die zwei Ansichten dieses Abschnitts mit je einem Stützpunkt

A.5 Arbeitsmengen-Sweep

Die folgenden Kurven tragen die Latenz über der Arbeitsmengen-Größe n (Schlüssel) auf — je eine Kurve je gesweepter Achsen-Ausprägung, und zwar für die Gesamt-Latenz (`ns_per_op`) sowie die p50-Latenzen von `insert` und `lookup`. Im Korpus liegt ein gemessener Wert von n vor (4096): `ns_per_op` und `lookup-p50` enthalten je einen Stützpunkt („1 Messpunkt“), `insert` wurde nicht ausgeführt und führt den Vermerk statt einer Kurve. Ohne Messwert entsteht kein Stützpunkt: Eine Messreihe ohne `working_set_n`-Spalte erzeugt bewusst *keine* Kurve, eine nicht ausgeführte Operation (`op_<art>_n = 0`) keinen Punkt.

Tabelle A.7 nennt den Vermerk der nicht erhobenen Sweep-Kennzahl; das Vermerk-Artefakt des Generators bleibt erhalten und erscheint nach der vollständigen Messreihe als Kurve an dieser Stelle.

Kennzahl	Status im Korpus	Grund (Vermerk des Artefakts)
Insert-Latenz p50 über n (op_insert_p50)	nicht erhoben	Operation im Korpus nicht ausgeführt (op_insert_n = 0): kein Stützpunkt, keine Kurve

Tabelle A.7: Nicht erhobene Kennzahlen der Sweep-Familie `ld_sweep_*` (Latenz über der Arbeitsmengen-Größe n): ein Vermerk statt Kurve; die zwei gemessenen Kennzahlen (Gesamt-Latenz, Lookup-Latenz p50) zeigen die zwei Kurven dieses Abschnitts mit je einem Stützpunkt

A.6 Per-Achsen-Latenz-Attribution

Die folgende Abbildung zerlegt die Segment-Laufzeit je Suchverfahren in die Beiträge der einzelnen Organ-Achsen (gestapelte Balken). Das 100 %-Ganze je Balken ist die Wall-Clock des separaten Segment-Laufs (`seg_run_total_ns`), *nicht* die Real-Workload-Latenz (`total_ns`); die 19 Segmente — die 18 Organ-Achsen der kanonischen Kompositions-Ordnung plus der Framework-Rest — summieren sich exakt zu dieser Basis (Coverage $\approx 1,0$). Der Korpus enthält einen Balken (`k_ary`).

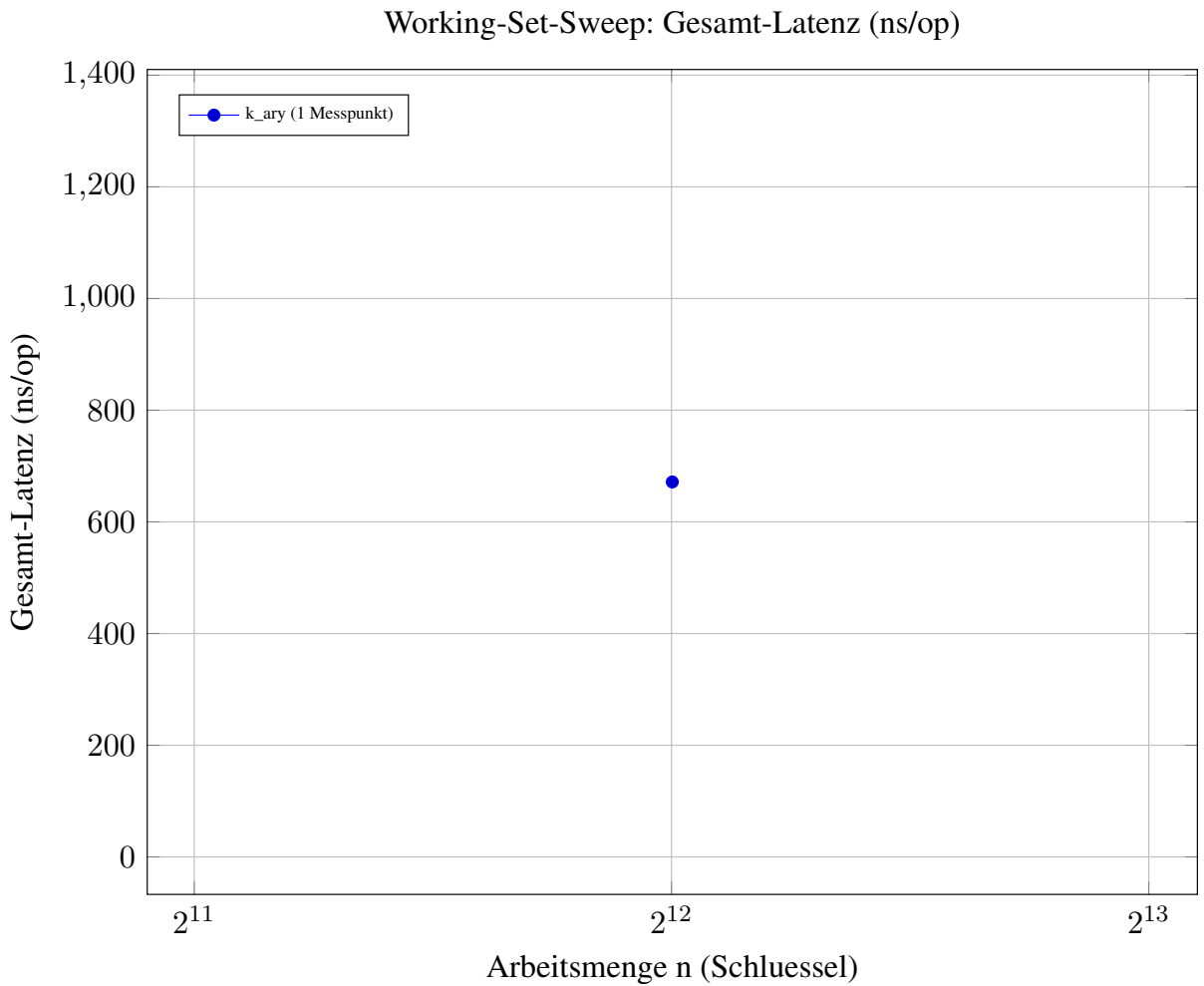


Abbildung A.5: Working-Set-Sweep: Gesamt-Latenz (ns/op)

A.6.1 Per-Achsen-Observer-Detail

Die folgende Langtabelle listet je Tier-Binary die gemessenen Observer-Zähler (stat_*-Spalten aus dem DLL-seitigen Achsen-Schema) auf — je Achse und Observer-Feld den gemessenen uint64-Zählwert; nicht erhobene Felder (n/a) werden weggelassen (kein synthetisch ergänzter 0-Wert). Nullen in der Tabelle sind gemessene Zählerstände inaktiver Teilfunktionen fest gewählter Strategien (etwa budget_reject, fail, register; Limitierung 4 in Tabelle A.10), keine nicht erhobenen Felder — Letztere fehlen in der Tabelle. Das Achsen-Schema ist single-source (18 Achsen mal 8 Feld-Slots, DLL-seitig emittiert, host-seitig header-getrieben gelesen); der Korpus enthält ein Tier-Binary, dessen binary_id die Kopfzeile von Tabelle A.8 nennt.

Tabelle A.8: Per-Achsen-Observer-Detailwerte je Tier-Binary (nur echt gemessene Zaehler)

Achse	Observer-Feld	Wert
search_algo=k_ary/cache_traversal=linear_fanout/mapping=direct_placement/path_compression=path_compression_none/ node_type=node4/memory_layout=memory_layout_cache_line_aligned/allocator=std_malloc/prefetch=prefetch_none/ concurrency=concurrency_none/serialization=serialization_raw_binary/value_handle=value_handle_inline/ index_organization=index_org_heap/io_dispatch=io_in_memory_only/migration_policy=migration_none/filter=filter_bloom/ queuing_q1=no_buffer/queuing_q2=eager_flush/persistence_target=persistence_memory_only [ycsb_c]		

Fortsetzung nächste Seite

Tabelle A.8 – Fortsetzung

Achse	Observer-Feld	Wert
allocator	alloc_cnt	2099211
	budget_reject	0
	bytes_alloc	537444328
	bytes_in_use	286720
	dealloc_cnt	2098186
	fail	0
cache_traversal	batch_size	4096
	batch_visited	32768
	peak_tracked	0
	register	0
	resolve	20000
	resolve_hit	20000
concurrency	resolve_miss	0
	unregister	0
	acquire	20000
	contention	0
	pattern_id	0
	release	20000
filter	validation_fail	0
	checksum	18432
	hash_probes	16384
	neg	0
	pos	4096
	probe	1
index_organization	checksum	16378
	indirect_lookups	0
	predicate_evals	4096
	records	4096
	scan	1024
	align_adjusts	0
io_dispatch	bytes	262144
	checksum	16378
	dispatch_cnt	4096
	rounds	1024
	indirect_steps	20000
	peak_mapped	0
mapping	register	0
	resolve	20000
	resolve_hit	20000
	resolve_miss	0
	reverse_lookup	0
	cache_lines	4096
memory_layout	checksum	16378
	field_bytes	32768
	line_bytes	64
	records	4096
	scan	1024
	cold_votes	0
migration_policy	decisions	4096
	hot_votes	0
	migrations	0
	tier_moves	0
	checksum	40496
	find	1
node_type	keys_stored	4
	queries	4096

Fortsetzung nächste Seite

Tabelle A.8 – Fortsetzung

Achse	Observer-Feld	Wert
path_compression	bytes_saved	70000
	checksum	7
	compress	10000
	cuts	10000
	prefix_len	70000
persistence_target	bytes_staged	0
	checksum	0
	device_flushes	0
	records_staged	36864
	rounds	9
prefetch	addrs_enqueued	0
	hot_path_hints	0
	max_queue_depth	0
	suggestions	0
	trigger	0
queuing_q1	get	0
	overflow	0
	peak_size	0
	put	0
	underflow	20000
queuing_q2	decisions	0
	flush_complete	0
	full_flush	0
	no_flush	0
	partial_flush	0
search_algo	erase	0
	hit	10000
	insert	0
	lookup	10000
	miss	0
serialization	peak	0
	bytes	262144
	checksum	16378
	records	4096
	serialize	1024
value_handle	access	4096
	indirect_deref	0
	peak_chain_depth	1
	version_strips	0

A.7 Latenz-Verteilung

Die Range-Balken zeigen je Suchverfahren und Operation die p50–p99-Spanne (Punkt bei p50, Whisker zu p99); da je Operation nur die Perzentile p50, p99 und p999 gemessen werden und das Schema keine Quartilsgrenzen trägt, wird bewusst *kein* Box-Plot⁸ mit interpolierten Quartilen gezeichnet. Die ECDF⁹ stellt die Verteilung der Gesamt-Latenz *über die Konfigurationen* dar (Anteil der Permutationen mit $ns_per_op \leq x$) — keine Per-Operation-CDF, da die Rohsamples je Operation nicht vorliegen. Im Korpus liegt eine Kategorie vor ($k_ary/lookup$: p50 750 ns, p99 1170 ns) und eine Konfiguration

⁸Box-Plot: die Verteilungsdarstellung aus Median, Quartilen und Whiskern nach Tukey; sie setzt die Quartilsgrenzen voraus, die das Schema nicht trägt [MTL78].

⁹ECDF (empirische Verteilungsfunktion): der Anteil der Beobachtungen, die kleiner oder gleich x sind, als Treppenfunktion über x ; die Wahrscheinlichkeits-Darstellung von Rohdaten ohne Verteilungsannahme [WG68].

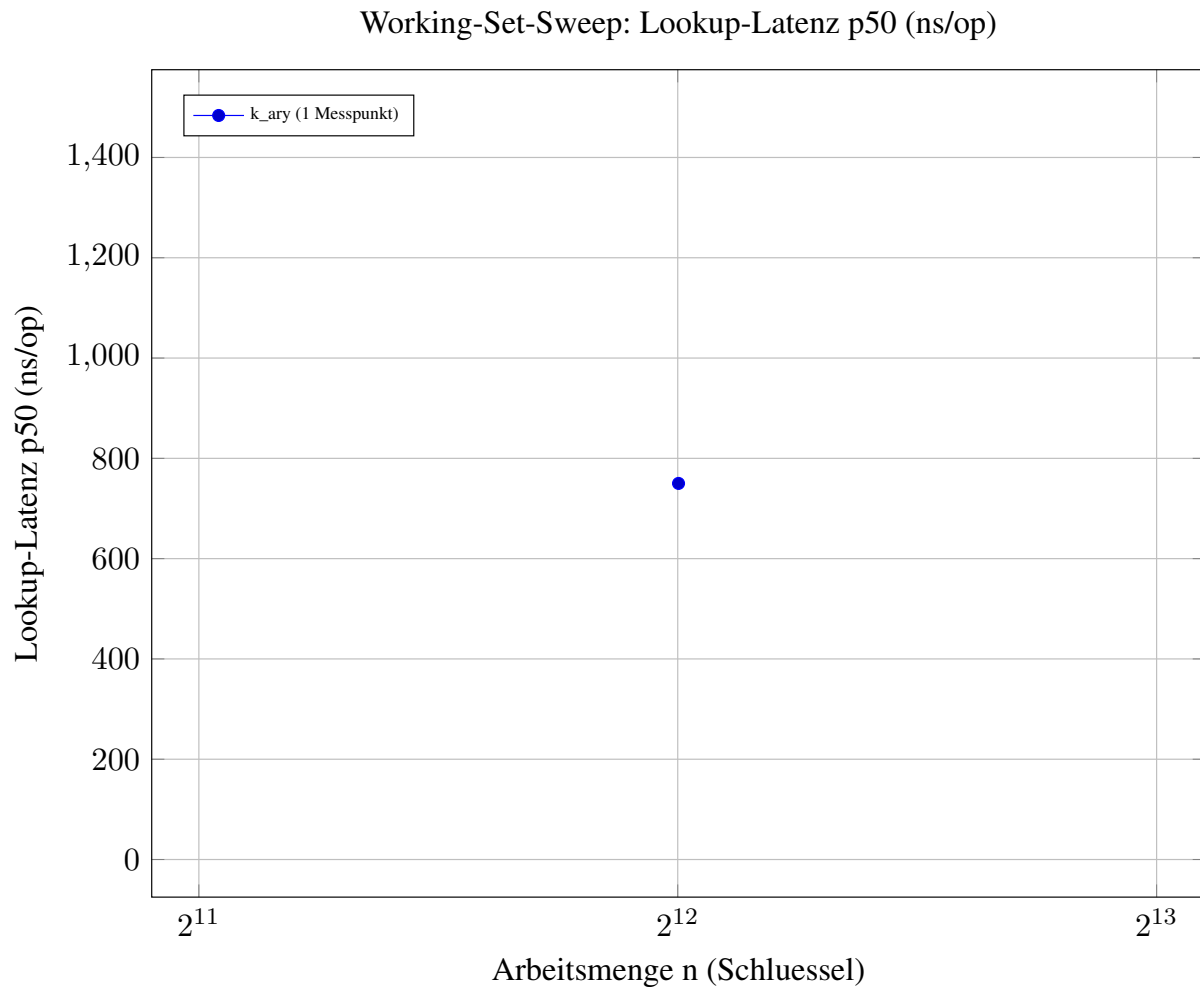


Abbildung A.6: Working-Set-Sweep: Lookup-Latenz p50 (ns/op)

($N = 1$): Die ECDF ist eine einstufige Treppe bei 671,5 ns/op.

Die Pareto-Streuung trägt schließlich je Konfiguration und Operationsart die Median- gegen die Tail-Latenz auf (p50 gegen p99). Die untere linke Front wäre die Menge der Konfigurationen, die von keiner anderen in beiden Kennzahlen zugleich übertroffen werden (Pareto-Dominanz¹⁰). Im Korpus liegt ein Punkt vor ($k_ary/lookup$: 750/1170 ns), eine Front ist damit nicht aussagbar. Die drei Latenz-Zahlen dieses Anhangs sind drei Kennzahlen derselben Zelle: die Gesamt-Latenz je Operation über den Lauf (671,5 ns/op), die p50-Latenz der `lookup`-Operation (750 ns) und deren p99 (1170 ns, als $750 + 420$ im Range-Balken).

A.8 Referenzbezogene Auswertung

Die folgenden Darstellungen setzen alle Messwerte zu *einer* Referenz ins Verhältnis. Diese Referenz ist eine gemessene Achsen-Ausprägung des Korpus selbst (Voreinstellung `search_algo/linear_scan`) und ausdrücklich *keine* externe Bibliotheks-Baseline: eine gemessene `std::map`-Leistungsserie existiert nicht, denn jedes `std::map` im Korpus ist das Konformitäts-Orakel des Prüf-Docks und damit ein

¹⁰Pareto-Dominanz: Eine Lösung dominiert eine andere, wenn sie in keinem Ziel schlechter und in mindestens einem Ziel besser ist; die nicht-dominierten Lösungen bilden die Pareto-Front [ED18, Ehr05].

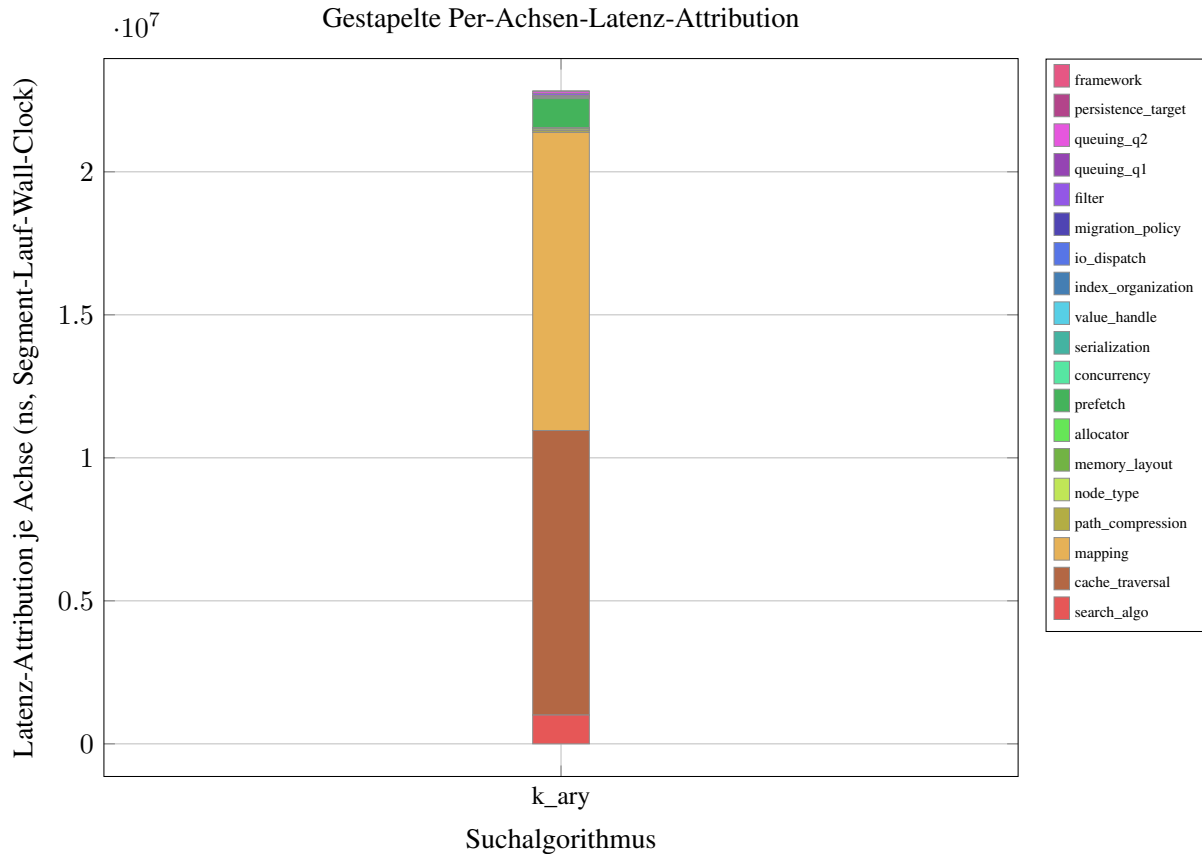


Abbildung A.7: Gestapelte Per-Achsen-Latenz-Attribution

Korrekttheits-, kein Leistungs-Datenpunkt. Kommt die Referenz-Ausprägung in einer Messreihe nicht vor, führen die Darstellungen dieses Abschnitts den Vermerk der fehlenden Referenz beziehungsweise ihren Ersatztext; im vorliegenden Korpus ist `linear_scan` nicht mitgemessen, alle Verhältnis-Darstellungen sind daher ohne Nenner (n/a).

A.8.1 Verhältnis-Matrizen

Je Schnittstellen-Funktion zeigt eine Matrix den Quotienten aus gemessenem Median und Referenz-Median. Die divergente Farbskala ist um die Gleichheit (Verhältnis 1) zentriert, sodass Vorteil und Nachteil gegenüber der Referenz ohne Zahlenvergleich unterscheidbar sind. In der Analyse-Rolle löst diese Matrix die Heatmap aus Abschnitt A.3 ab; die rohe Heatmap bleibt als Rohwert-Ansicht daneben bestehen. Im vorliegenden Korpus trägt weder die Verhältnis-Matrix noch die Heatmap Daten (Referenz nicht gemessen; Fläche unter 2×2). Wo Verhältnisse zusammengefasst werden, geschieht das geometrisch, nie als arithmetisches Mittel von Quotienten¹¹ — das gilt auch für die normalisierten Balken des nächsten Unterabschnitts.

Tabelle A.9 fasst die sechs Vermerke zusammen; die Vermerk-Artefakte des Generators bleiben erhalten und erscheinen nach der vollständigen Messreihe mit gemessener Referenz als Verhältnis-Matrizen an dieser Stelle.

¹¹Das arithmetische Mittel normalisierter Verhältnisse hängt von der Wahl der Referenz ab und kann die Rangfolge umkehren; das geometrische Mittel ist dagegen referenz-invariant [FW86].

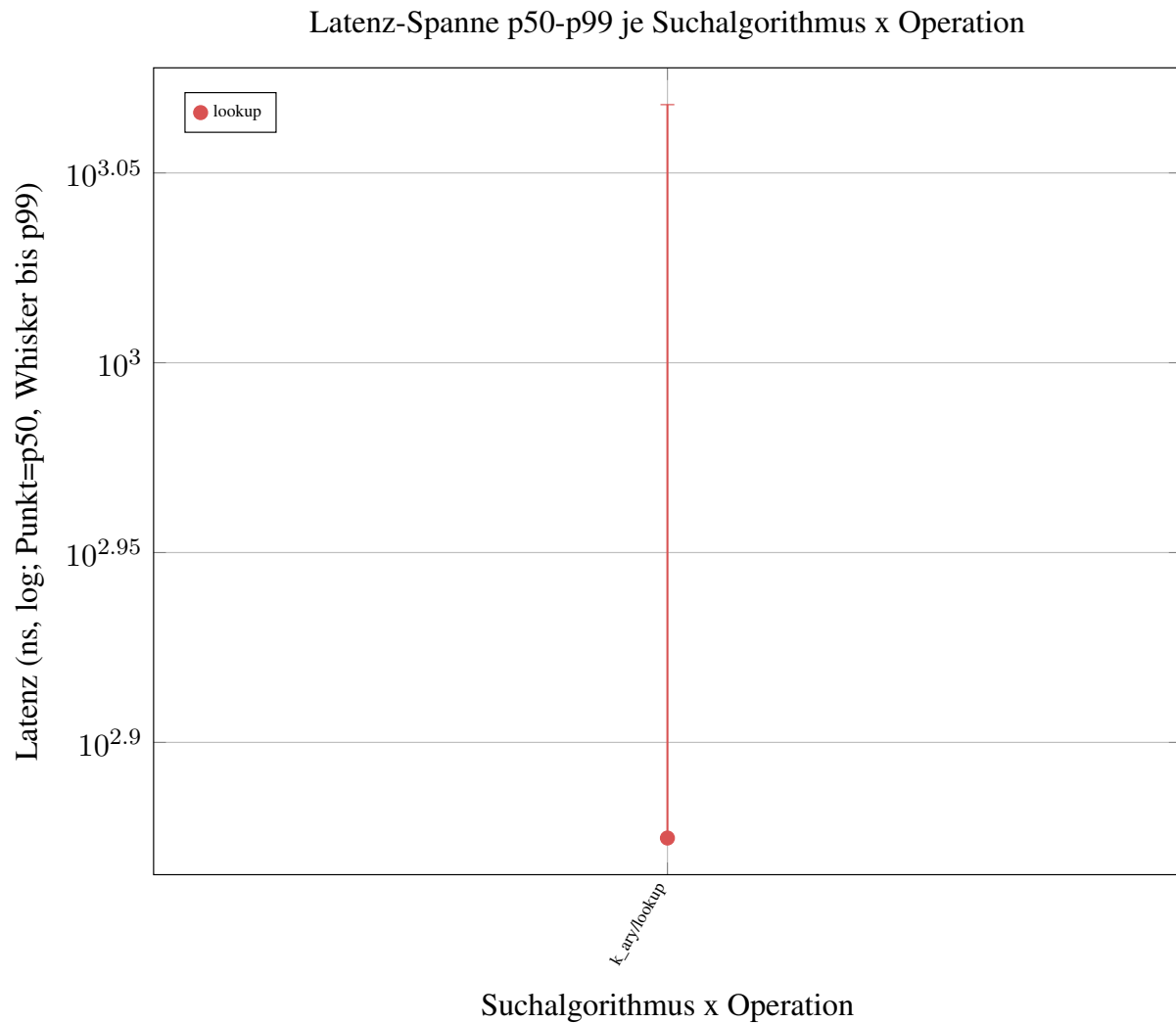


Abbildung A.8: Latenz-Spanne p50-p99 je Suchalgorithmus x Operation; Punkt=p50, Whisker=p99.

Kennzahl	Status im Korpus	Grund (Vermerk des Artefakts)
Gesamt-Latenz (ns_per_op)	n/a (ohne Nenner)	Referenz <code>linear_scan</code> nicht gemessen
Insert-Latenz p50 (op_insert_p50)	n/a (ohne Nenner)	Referenz <code>linear_scan</code> nicht gemessen; Operation zudem nicht ausgeführt
Lookup-Latenz p50 (op_lookup_p50)	n/a (ohne Nenner)	Referenz <code>linear_scan</code> nicht gemessen
Erase-Latenz p50 (op_erase_p50)	n/a (ohne Nenner)	Referenz <code>linear_scan</code> nicht gemessen; Operation zudem nicht ausgeführt
Scan-Latenz p50 (op_scan_p50)	n/a (ohne Nenner)	Referenz <code>linear_scan</code> nicht gemessen; Operation zudem nicht ausgeführt
RMW-Latenz p50 (op_rmw_p50)	n/a (ohne Nenner)	Referenz <code>linear_scan</code> nicht gemessen; Operation zudem nicht ausgeführt

Tabelle A.9: Nicht erhobene Kennzahlen der Verhältnis-Familie `lc_surface_ratio_*` (Quotient Median/Referenz-Median): sechs Vermerke statt Matrizen, weil die Referenz-Ausprägung `search_algo/linear_scan` im Korpus nicht mitgemessen ist

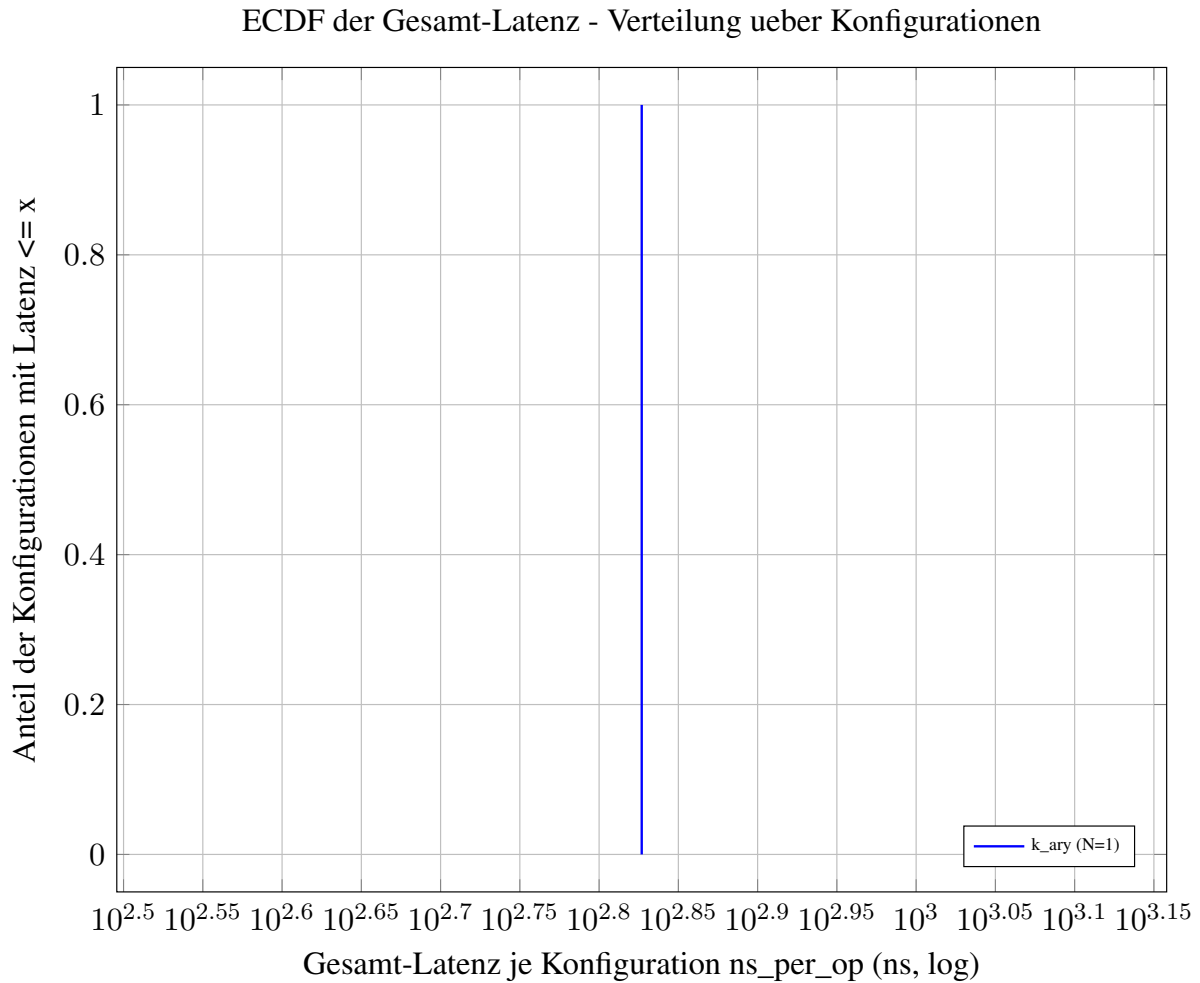


Abbildung A.9: ECDF der Gesamt-Latenz – Verteilung ueber Konfigurationen

A.8.2 Normalisierte Balken

Dieselbe Aussage über die Lastprofile hinweg verdichtet: ein Balken je Suchalgorithmus, die Referenzlinie liegt bei 1. Wo die Verhältnis-Matrix das Lastprofil-Muster zeigt, gibt der Balken die Gesamtaussage je Suchalgorithmus.

*(Hier stünden die normalisierten Balken der **Gesamt-Latenz** (ns_per_op) — ein Balken je Suchalgorithmus, Referenzlinie bei 1. Sie liegen nicht vor: sie teilen den Nenner der Verhältnis-Matrizen, und die Referenz-Ausprägung ist in den bisher vorliegenden Messreihen nicht mitgemessen.) (Hier stünden dieselben Balken für die **p50-Latenz von insert**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von lookup**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von erase**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von scan**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.) (Hier stünden dieselben Balken für die **p50-Latenz von rmw**; auch ihnen fehlt die mitgemessene Referenz-Ausprägung.)*

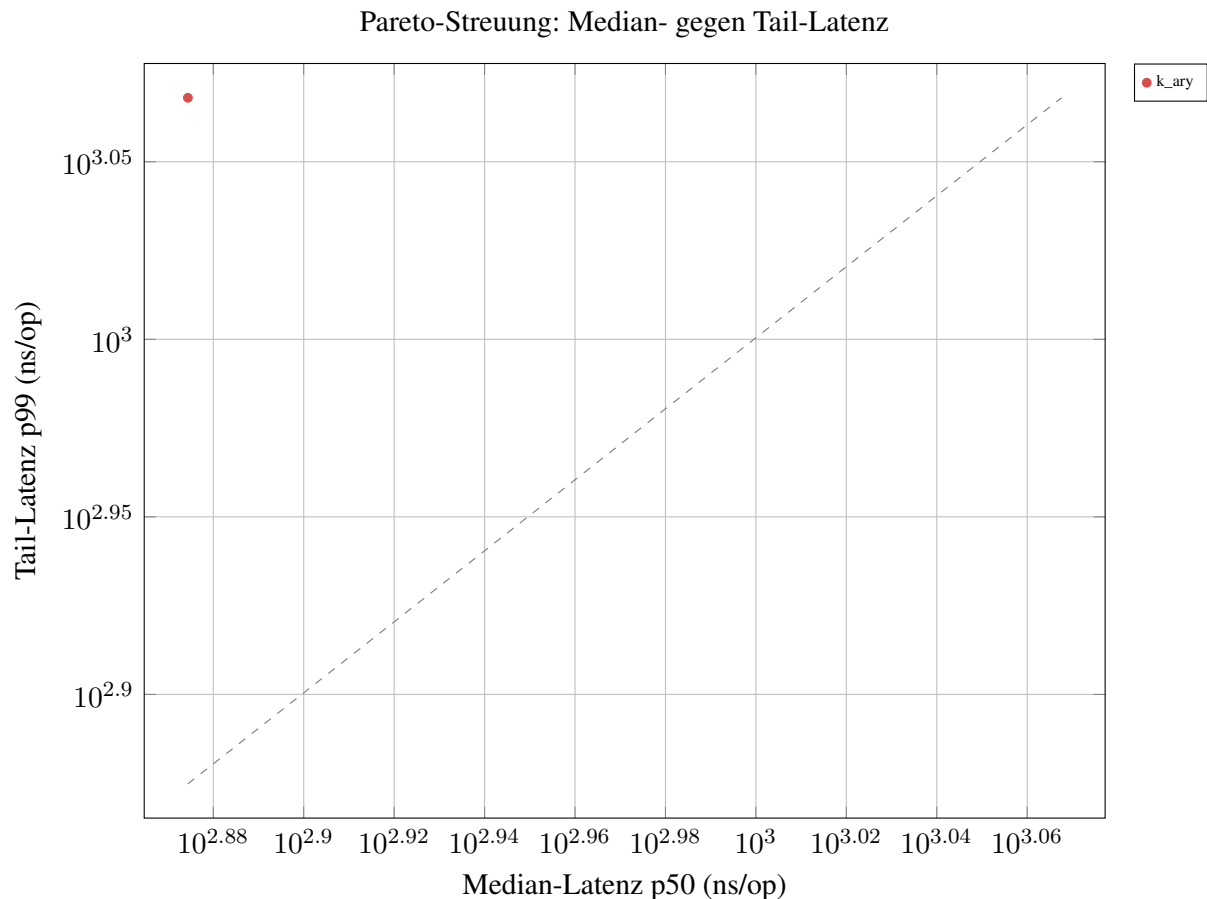


Abbildung A.10: Pareto-Streuung: Median- gegen Tail-Latenz (ein Punkt je Konfiguration und Op-Art; Diagonale = kein Tail-Aufschlag)

A.9 Achsen-Austauschbarkeit

Die vier Langtabellen der Generator-Familie `ld_exchange_*` sind die Träger des Austauschbarkeits-Nachweises je variabler Achse anhand der Geschwister-Paar-Differenzen (Median rel. Δ ns/op bezüglich v): `search_algo`, `node_type`, `memory_layout` und `prefetch`; das Aggregat je Wertepaar und Interface-Funktion ist der nearest-rank-Median des relativen Deltas mit seiner IQR¹² über alle Geschwister-Paare und validen Lastprofile. Ein Nachweis setzt Geschwister-Paare voraus — der Generator erwartet 480/480/480/640 (`search_algo`, `node_type`, `prefetch`, `memory_layout`); der vorliegende Korpus liefert 0, die vier Tabellen sind daher leer und belegen nichts; derselbe Generatorlauf füllt sie, sobald die vollständige Messreihe vorliegt. Dass `search_algo` und `prefetch` am wenigsten konfundiert sind, ist ein Vorbehalt der Messanordnung (Limitierung 10 in Tabelle A.10), kein Messergebnis. Bis zur vollständigen Messreihe steht an der Stelle der vier Tabellen die folgende Zeile.

(0 Geschwister-Paare im Korpus (Stand der Messreihe); die vier Langtabellen `ld_exchange_*` erscheinen an dieser Stelle nach dem Generator-Neulauf der vollständigen Messreihe.)

Der folgende Forest-Plot¹³ fasst die Austauschbarkeit der `ns_per_op`-Kennzahl zusammen (Punkt = Median rel. Δ , Whisker = IQR, 0-Referenzlinie; kleine- n -Zeilen ausgegraut). Der Whisker liegt

¹²IQR (Interquartilsabstand): die Breite zwischen dem 25 %- und dem 75 %-Perzentil, hier als $p75 - p25$ des relativen Deltas.

¹³Forest-Plot: die Darstellung mehrerer Effektschätzer als Punkte mit Unsicherheits-Whiskern übereinander an einer gemeinsamen Referenzlinie, aus der Meta-Analyse übernommen [LC01].

symmetrisch um den Median (über die IQR-Breite), da das Aggregat nur die IQR-Breite, nicht die Quartilsgrenzen trägt.

(Hier stünde der Forest-Plot der `ns_per_op`-Austauschbarkeit. Er liegt nicht vor: ohne Geschwister-Paare (0 von 480/480/480/640) schreibt der Generator keine Datei — keine leere Abbildung, kein synthetisch erzeugtes Aggregat.)

Der zweite Forest-Plot zeigt dieselben Aggregate, dreht aber alle Paare auf *eine* feste Referenz-Achsenausprägung im Sinne von Abschnitt A.8, sodass der Maßstab des Vergleichs eindeutig benannt ist statt offen zu bleiben. Die Drehung rechnet das relative Delta exakt um — aus d wird $d' = -d/(1 + d)$, denn mit dem Bezugswert wechselt auch der Nenner; eine bloße Vorzeichen-Umkehr wäre hier ein Rechenfehler. Punkte, die sich so nicht abbilden lassen, werden ausgelassen und nicht ersetzt.

(Der referenzbezogene Forest-Plot liegt nicht vor: er setzt Geschwister-Paare und eine mitgemessene Referenz-Ausprägung voraus; im vorliegenden Korpus fehlt beides.)

A.10 Limitierungen

Tabelle A.10 führt die im Generator gepflegten, nicht behobenen Vorbehalte der Mess-Artefakte auf (16 Zeilen), damit jeder Vorbehalt dokumentiert bleibt; die Limitierungen der Arbeit aus Nutzer-Sicht stehen im Fazit (Abschnitt 6.2).

Tabelle A.10: Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)

#	Vorbehalt	Status / Konsequenz
1	Cache-Misses (Kernmetrik): L2 + Coherence modell-gebunden real (Zen 5: <code>PERF_TYPE_RAW 0x964/0x1443</code>), sonst n/a; L1/L3/dTLB nur mit gesetztem PMC-Schalter erhoben; Energy best-effort	Ohne <code>COMDARE_ENABLE_PMC</code> (Build-Default OFF, damit <code>test:unit</code> ohne HW-Zugriff baut) liefert <code>NullPmcSource</code> <code>available=false</code> , und alle HW-Spalten bleiben 0 – so entstanden die tabellierten Nullen. Mit dem Schalter erhebt <code>LinuxPerfPmcSource</code> L1-, L3- (Last-Level) und dTLB-Misses real über <code>perf_event_open(2)</code> (Kernel-Syscall, keine Vendor-Lib, kein PAPI nötig – PAPI bleibt optionaler Sekundärpfad). Für L2 und Coherence-Invalidierungen existiert kein portabler generischer Zähler; sie werden modell-gebunden über den RAW-Event-Katalog erhoben (<code>pmc_raw_event_katalog.hpp</code> ; Zen 5, Family 26: <code>l2_cache_req_stat.ic_dc_miss_in_l2 = PERF_TYPE_RAW 0x964</code> = Demand-IC+DC-Misses in L2 ohne L2-Prefetch; <code>ls_dmnd_fills_from_sys.remote_cache = PERF_TYPE_RAW 0x1443</code> = Demand-Fills aus dem Cache eines anderen CCX, der Kohärenz-Beobachtbare des Core-PMU – ein wörtlicher Invalidierungszähler existiert im <code>amdzen5</code> -Satz nicht); Modelle ohne Katalog-Eintrag tragen ehrlich n/a (per-Feld-Flag), nie eine geratene Kodierung. L3 kann auf einzelnen AMD-Plattformen zusätzlich beim Öffnen des Zählers scheitern; Energie läuft best-effort über <code>RAPL-sysfs</code> und bleibt ohne Zonen-Leserecht leer. Der Intel-PCM-Windows-Treiber steht für die Windows-Lane weiterhin aus (#26/P4). Auf Modellen ohne Katalog-Eintrag (heute trägt nur Zen 5 einen) bleibt die L2/Coherence-Lücke nur indirekt über den Wall-Clock-Proxy (<code>seg_memory_layout_ns/ns_per_op</code>) beobachtbar.

Fortsetzung nächste Seite

Tabelle A.10 – Fortsetzung

#	Vorbehalt	Status / Konsequenz
2	14 gepinnte Achsen = 0 Austauschbarkeits-Belege	Nur 4 der 18 Kompositions-Achsen variieren (search_algo, node_type, memory_layout, prefetch). Gepinnt (je 1 Wert): cache_traversal, mapping, path_compression, allocator, concurrency, serialization, value_handle, index_organization, io_dispatch, migration_policy, filter, queuing_q1, queuing_q2, persistence_target. (isa ist seit INC-2d die System-Achse target_isa und telemetry seit INC-2c keine binary_id-Kompositions-Achse mehr – beide zählen daher nicht mehr als gepinnte Organ-Achsen.)
3	Observer-Proben-Zähler (*_find/*_probe/*_get) by-design ≈ 1	Erwartetes Verhalten, kein Phantom: je Lookup eine Probe. Keine Aussage über interne Iterationen.
4	Honest-0 inaktiver Sub-Features gepinnter Strategien	concurrency/migration/io/value_handle melden 0 (Feature inaktiv), nicht Mess-Fehler.
5	Wall-Clock nicht bit-reproduzierbar	Seed steuert Keys (deterministisch), NICHT das CPU-Timing; Wall-Clock variiert lauf-zu-lauf.
6	RC-Dimension nur nominal	K1: $\times 18 \rightarrow \times 3$ degeneriert; RC-Achse liefert keine volle Ausprägungs-Breite.
7	2/21 Scan-Profile = No-Op (ausgeschlossen)	ycsb_e und lp_range_scan führen keinen echten Scan aus; bei scan-Diffs ausgeschlossen (19 valide Lastprofile).
8	Insert-Profile messen Upserts; stat_*-Spalten enthalten Load-Phase	Insert-Pfad ist faktisch Upsert; die stat_*-Zähler akkumulieren die Lade-Phase mit.
9	prefetch misst Key-Werte als Pseudo-Adressen (K9)	Prefetch-Achse interpretiert Schlüssel-Werte als Adressen; kein echter Speicher-Prefetch.
10	node_type + memory_layout: Q2-Schritt-4-Beschattung	Apparat-Artefakt möglich (search_organ_-Beschattung); search_algo + prefetch sind am wenigsten konfundiert.
11	container_-Spiegel (Weg-B): O(n)-Flatten+Rebuild als Zusatz-Apparat	Weg-B-Lebewesen (SOTA-Bäume/Tries/Hash) halten neben dem echten Such-Organ einen sortierten container_-Spiegel; dessen O(n)-Flatten+Rebuild (bei NEU-Inserts + erfolgreichen Erases; #211, bis-M3 ungefixt) ist ZUSÄTZLICHER Apparat-Aufwand, den die echte Baumstruktur nicht hat $\rightarrow$ die gemessene ns_per_op der Weg-B-Tiere ist um diesen Spiegel-Anteil überhöht. (Bei der Weg-A-Array-Familie ist derselbe O(n)-Rebuild der ECHTE Struktur-Kostenanteil, kein Zusatz-Artefakt.) Behebung mit #188/#211 + Nachfolge-Neubau (#215).
12	memory_layout-Effekt teils sub-noise	Teil der Layout-Differenzen liegt unter dem Mess-Rauschen $\rightarrow$ PMC extern-gated (#26).
13	Sekundäre Indirektions-Organen (cache_traversal/mapping): uint16-Schlüssel-Sättigung	Der uint64-Primärschlüssel wird beim Auto-Koppeln in die schmalere slot_index/key_type-Felder der GEPINNTEN cache_traversal-/mapping-Organen verengt (low-bits); bei records > Typbreite kollidieren deren Observer-Einträge. Betrifft NICHT den Primär-Lookup (uint64); reine Observer-Limitierung gepinnter Achsen (vgl. Zeile 2/4).
14	Kapsel-internes (verschlucktes) OOM in einer Mess-Op	Nach aussen geworfenes OOM entwertet die Messung jetzt hart (two_phase_valid=false). Ein vom Tier-Code INTERN per catch(...) gefangenes OOM (z.B. defensiver Memento-Verzicht) ist von aussen nicht beobachtbar $\rightarrow$ konservativ als Restvorbehalt dokumentiert, nicht erzwungen.
15	Timer-Overhead (clock::now()) nicht kalibriert	Die Wall-Clock-Segmentzeiten enthalten den nicht subtrahierten Eigen-Overhead der std::chrono::steady_clock::now()-Aufrufe (eine additive, lauf-stabile ns-Komponente je gemessenem Segment). Es findet KEINE Leer-Schleifen-Kalibrierung statt $\rightarrow$ dokumentierter Vorbehalt; betrifft den absoluten ns-Nullpunkt, nicht die relativen Achsen-Vergleiche (G5-Audit, MESS-MINOR-7).

Fortsetzung nächste Seite

Tabelle A.10 – Fortsetzung

#	Vorbehalt	Status / Konsequenz
16	Entwurfs-Etiketten = Namenskonvention, kein Mess-Einfluss	Einzelne Pattern-Etiketten im Code/Doku sind Benennungs-/Entwurfsvorbehalte ohne Mess-Wirkung: „hybrider Visitor“ (real ein Memento, kein <code>accept()</code> /Host-Besuch), sowie Interpreter-/Template-/Decorator-Etiketten und <code>reserve-if</code> <code>constexpr</code> -Hygiene. Reine Naming-Konvention → als Vorbehalt dokumentiert, kein Einfluss auf Messwerte (G5-Audit, PATTERN-MINOR-1..7).

Zeile 1 benennt die Lage der Kernmetrik Cache-Misses: L1D-, Last-Level- und dTLB-Misses nur mit gesetztem PMC-Schalter; L2-Demand-Misses und Fremd-CCX-Fills modellgebunden über den RAW-Event-Katalog — für die AMD-Mikroarchitektur Zen 5 (CUID-Familie 26) durch Messung gegengeprüft (`PERF_TYPE_RAW 0x964` und `0x1443`) [Lin25], für Modelle ohne Katalog-Eintrag n/a (alle Intel-Modelle); Energie best-effort über RAPL [DGH⁺10]. Ein Neuaufsatz der Tabelle mit dreiwertigem Status je Zeile und eine Überarbeitung der Begriffe ihrer Zeilen 4 und 10 sind im Generator nicht umgesetzt.

B Code-Struktur

Dieser Anhang dokumentiert die Quellcode-Struktur der vier Repositorien aus Abschnitt 4.1 nachprüfbar auf Verzeichnis-Ebene (zwei bis drei Ebenen tief). Maßgeblich ist je Repository der Zweig `development`: für die beiden Code-Repositories in dem Commit, den das Super-Repository als Submodul-Zeiger (Gitlink¹) führt, für das Super-Repository sein eigener Stand und für das Manuskript-Repository der Commit, aus dem die CI das PDF baut: `comdare-cache-engine eb69abc1`, `comdare-prt-art b539d6ee`, `probst-Diplomarbeit-cache-engine d989e008` und `20260931-overleaf-diplomarbeit d32cc492`. Build-Artefakte (`build*`), IDE-Verzeichnisse und LaTeX-Bauartefakte sind ausgeblendet; der vendorte Fremdcode unter `ext/` und die Vorspann-Unterlagen der Arbeit erscheinen als Sammel-Einträge. Alle genannten Pfade existieren im genannten Stand; Pfadangaben sind relativ zur jeweiligen Repository-Wurzel. Die Lizenz-Basis jedes Repositoriums (`LICENSE`, `NOTICE`, `LICENSES/`, `REUSE.toml`²) ist Teil der Struktur; die Lizenzaussage der Arbeit selbst trifft der Vorspann.

comdare-cache-engine — Bausteine-Bibliothek, Planer und Builder

Das Repository folgt dem Pitchfork-Layout³: `apps/` enthält die ausführbaren Programme, `libs/` die Bibliotheken, `libs/common` den querschnittlichen Code und `libs/traeger` die Aufteilung der vier Träger-Stufen; der vendorte Fremdcode liegt gesammelt unter `ext/` und wird hier nicht aufgefächert. Dem CMake-Bau ist ein GNU-Bauweg vorgelagert: `configure.sh` und ein statisches Makefile mit den GNU-Standardzielen⁴ reichen an das bauende CMake-Skript weiter.

Die Achsen liegen in drei Realms (Achsen-Heimaten; Anhang C) je Kategorie: `organ_axes/` (22 Verzeichnisse: die 18 Kompositions-Haupt-Achsen, `organ_meta_meta/` sowie `cacheline/`, `simd/` und `telemetry_axis/`), `system_axes/` (`target_isa`, `operating_system` und `external_utils` samt ihren Unter-Achsen sowie `compiler`, `scheduling` und `extension_hardware`; Unter-Achsen u.a. `optimization_level`, `compiler_atomic` und `simd`) und `mess_axes/` (Mess-Tooling-Registry der drei Mess-Ebenen `wallclock`, `macro` und `micro`). Jeder Realm besitzt eine generierte Registry-XML als maschinenlesbares Abbild ihres Bestands: `algorithm_profiles/cache_engine_axis_registry.xml`, `include/`

¹Ein Submodul ist ein in ein übergeordnetes Repository eingebettetes Git-Repository; `.gitmodules` deklariert je Submodul `path` und `url`, optional `branch` und `update`. Der Gitlink ist der im übergeordneten Repository verzeichnete Commit des Submoduls [Git26].

²REUSE ist eine Spezifikation der Free Software Foundation Europe für maschinenlesbare Lizenz- und Urheberangaben: Die Lizenztexte liegen unter `LICENSES/` mit dem SPDX-Bezeichner als Dateiname, und `REUSE.toml` ordnet Dateien ihren Lizenzen zu [Fre24].

³Konvention für die Verzeichnisstruktur nativer C/C++-Projekte (Pike, *cxx-pfIR1*; Spezifikation im Repository `vector-of-bool/pitchfork`): `apps/` für Programme, `libs/` für Bibliotheken, daneben `tools/`, `tests/`, `docs/` und `external/` als benannte Wurzelverzeichnisse [Pik18].

⁴`all`, `install`, `check`, `installcheck`, `uninstall`, `clean`, `distclean`, `mostlyclean` und `maintainer-clean` nach den GNU Coding Standards, Abschnitt *Standard Targets for Users* [Fre26]; das Test-Ziel heißt dort `check`, nicht `test`.

```

| | |-- mess_axes/      Mess-Realm (Mess-Tooling-Registry
| | |                  wallclock/macro/micro)
| | |-- mess/          Mess-/Steuer-Naht (MessVisitor, Genus-Kaskade,
| | |                  Steuer-Docks Planer -> CEB)
| | |-- hybrid/        Hybrid-Tier-Stufe (Dock-Vertraege, Dock-Array,
| | |                  Dock-Factory, XML-Parser, Modul-ABI v1,
| | |                  Stempel-Kette, Binary-Proxy, Heuristik-Adapter)
| | |-- topics/, anatomy/, compositions/, profile_facade/
| | |                  Achsen-Dach (AxisKind), Anatomie-Kern,
| | |                  Kompositions- und Katalog-Schicht;
| | |                  profile_facade/planner/ = Planer-Substanz
| | |                  (ExperimentPlanDirector, Simulation, Version,
| | |                  Legende)
| | |-- execution_engine/ ExecutionEngine-Wurzel ueber AnatomyBase
| | |                  (Mess-Schicht)
| | |-- builder/        codegen/, permutation_loop/, module_loader/,
| | |                  experiment_driver/ (ExperimentDriver,
| | |                  Phase 1-7), experiment_tree/ (Achsen-Pfad-
| | |                  Serialisierung, Genus-Bindung, CEB-Generator),
| | |                  build_orchestrator/, pruef_dock/, bestandslog/,
| | |                  lager_ablage/, measure_storage/, curve_fit/,
| | |                  overlay_source_set.hpp (Overlay-Quellmenge)
| | |                  u. a.
| | |-- heuristik/, subsystems/, concurrency_manager/, reclamation/,
| | |                  harness/, anatomy_drive/, src/, virus/
| | |                  Kurven/Spline/Break-Even, zwoelf Subsystem-
| | |                  Engines (c01..c12) + Plattform-Profiler,
| | |                  Nebenlaeufigkeits-Manager, RCU-Reclamation,
| | |                  Mess-Harness, Antriebe, SHA-Kerne,
| | |                  Graph-Algorithmen
| | |-- include/cache_engine/abi/ ABI-Header (stempel_basis.hpp,
| | |                  baustein_variants.hpp, resolve_baustein.hpp,
| | |                  module_abi_v1.hpp, anatomy_version_stamp.hpp,
| | |                  anatomy_fingerprint.hpp,
| | |                  anatomy_stamp_entries.hpp u. a.)
| | |-- include/cache_engine/measurement/ Realm-Wurzeln + Registries
| | |                  (system_axis.hpp, measurement_*_registry.hpp,
| | |                  system_axis_registry.xml,
| | |                  measurement_axis_registry.xml,
| | |                  ram_frequency_reading.hpp, spd_ddr5_parser.hpp,
| | |                  ram_probe_chain.hpp, hardware_probe_factory.hpp)
| | |-- include/cache_engine/lager/, naming/
| | |                  Lager-Schluessel-Schema, Begriffs-Alias-Registry
| | |-- include/cache_engine/{api, concepts, allocators, platform,
| | |                  platform_probe, fingerprint, indexes, hbm}/
| | |                  uebrige oeffentliche Header (Sammel-Eintrag)
| | |-- algorithm_profiles/ sota/ (33 Profil-Dateien *.profile.xml +
| | |                  sota_h2_scores.xml), cache_engine_axis_registry.xml
| | |                  (generiert), paper_experiments/, allocators/,
| | |                  load_profiles/, thesis_profiles/,
| | |                  permutation_axes.xml
| | |-- traeger/        Traeger-Stufen planner/ -> ceb/ -> tier/ -> hybrid/

```

```

| | (Bau-Reihenfolge; CMake-Ziele;
| | Substanz in libs/cache_engine/)
| |-- common/ Querschnitt: config/, measurement/, platform/,
| | serialization/ (xml_config_parser/), succinct/
| |-- execution_engine/ Ergebnis-Aggregation (result_aggregator,
| | experiment_demo)
| |-- search_engine/ deklariertes Skelett: node/, page/, traversal/,
| | operations/, memory_layout/, value_handle/ u. a.
| | (nur CMake-Eintraege, keine Quelldateien)
| '-- test_infra/ benchmark_suite/, test_data/, workload_generator/
|-- ext/ vendorte Fremd-Repositorien der A-/P-Serie nach allocator/,
| prefetch/, queuing/, traversal/ gegliedert; daneben io/
| (liburing, libxlswriter, zlib) und A05-jemalloc
| (Sammel-Eintrag)
|-- tests/ unit/, integration/, generic_module_tests/,
| module_specific_tests/, fixtures/,
| pruefling_fixture/, deprecated/ (Archiv geparkter
| Tests)
|-- tools/ Registry-Generatoren axis_registry_gen/,
| system_axis_registry_gen/, measurement_axis_registry_gen/;
| axis_version_lock/ (Versions-Lock); overlay_source_hash_gen/;
| vor_push_gate/; abi_stability_test/, gen_golden_fullpilot/,
| best_binary_selector/, p27_bundle_finder/, ycsb_cli/,
| mess_report/, latex_anhang/, latex_toolchain/,
| compiler_provisioning/, prerequisites/, gitlab_ci_zih_push/,
| socks5_zih_delivery/
|-- benchmarks/, scripts/ Mikro-/Plattform-/YCSB-Benchmarks; CI-Pruefungen,
| Lint- und Lande-Gate-Skripte (Shell)
|-- modules/, prerequisites/, *.py nur README bzw. vier Werkbank-Hilfsskripte
| (Python, ausserhalb der Buildchain)
'-- docs/ Architektur-, Plan-, Status-, Ledger-, Audit-,
Lizenz- und Arbeitsdokumentation

```

Komponente	Pfad	Zweck
CacheEngineBuilder	apps/cache_engine_builder/	Builder-Executable (Konfiguration zu Permutations-Modulen)
ExperimentDriver	libs/cache_engine/builder/ experiment_driver/	Phase-1-bis-7-Pipeline (host-seitig): phase1_ enumerate bis phase7_ export, zwei Opt-in-Schritte
Codegenerierung	libs/cache_engine/builder/codegen/	generate_module_ from_profile samt Templates
Permutations- Schleife	libs/cache_engine/builder/ permutation_loop/	Aufzählung der Achsen-Permutationen
Modul-Loader	libs/cache_engine/builder/ module_loader/	Laden der erzeugten Module

Komponente	Pfad	Zweck
ABI-Header	libs/cache_engine/include/ cache_engine/abi/	u. a. stempel_basis.hpp (CRTP-Vertrag aller Stempel-Träger), baustein_ variants.hpp, resolve_baustein.hpp, anatomy_ fingerprint.hpp, anatomy_stamp_ entries.hpp
XML-Config-Parser	libs/common/serialization/ xml_config_parser/	Parser der Profil- und Experiment-XMLs
Achsen-Realms	libs/cache_engine/{organ_axes, system_axes,mess_axes}/	je Kategorie ein Realm; organ_axes/ je Kompositions-Haupt-Achse ein Unterverzeichnis (18) sowie organ_meta_meta/ 33 Profil-Dateien (* .profile.xml): 30 Vollprofile eigenständiger Suchentwürfe (Abschnitt 3.2) und drei abstrakte Profile olc, louds und vampir, dazu sota_h2_scores.xml
Adapter A-/P-Serie	adapters/	ABI-Adapter über dem ext/-Fremdcode (10 A-, 12 P-Einträge)
Planer	apps/experiment_planner/	Planer-Rolle als eigene Binary (comdare-experiment- planner); Subkommando-CLI mit u. a. validate, plan, simulate, check-size, status, version, help
Plan-Director	libs/cache_engine/profile_facade/ planner/	deterministischer Director-Walk (experiment_plan_ director.hpp), Planungs-Simulation (planner_ simulation.hpp), Planer-Version und Fingerprint (planner_version.hpp)

Komponente	Pfad	Zweck
Träger-Stufen	libs/traeger/	Stufenkette planner, ceb, tier, hybrid (in dieser Bau-Reihenfolge; in der Verarbeitungskette ist der Hybrid der optionale Einschub zwischen CEB-Prüf-Dock und Tier) als CMake-Ziele; die Substanz liegt in libs/
Achsen-Registries	libs/cache_engine/ algorithm_profiles/ cache_engine_axis_registry.xml, libs/cache_engine/include/ cache_engine/measurement/{system, measurement}_axis_registry.xml	generierte Abbilder der drei Realms (tools/ *_registry_gen)
Versions-Lock	tools/axis_version_lock/	axis_version.lock über den Overlay-Schnitt (im genannten Stand 719 Records), geprüft vom CI-Gate contract: axis-version-lock
Overlay- Quellmenge	libs/cache_engine/builder/ overlay_source_set.hpp, tools/ overlay_source_hash_gen/	Dateimenge und Hash des Fingerprint-Glieds [7]
Hybrid-Stufe	libs/cache_engine/hybrid/	Dock-Verträge, Dock-Array, Dock-Factory, XML-Parser der <hybrid_tier>-Sektion, Modul-ABI v1, Stempel-Kette, Binary-Proxy, Heuristik-Adapter; Router und Eviction nicht implementiert
Mess-/Steuer-Naht	libs/cache_engine/mess/	MessVisitor-Naht (mess_naht.hpp), Gattungs-Kaskade (genus_kaskade.hpp), Steuer-Docks vom Planer zur CEB (steuer_dock.hpp)
Reproduktionsweg	configure.sh, Makefile	GNU-Bauweg vor CMake: configure, make, make check, make install; Makefile statisch mit den GNU-Standardzielen, reicht an CMake weiter

Komponente	Pfad	Zweck
Lizenz-Basis	LICENSE, NOTICE, LICENSES/, REUSE.toml	Comdare Research License 1.0 (SPDX LicenseRef-Comdare-Research-1.0), Change Date 2031-08-10, danach Apache-2.0; REUSE-Zuordnung je Fremdpaket

comdare-prt-art — Prüfling

Der Prüfling ist als Header-Bibliothek⁵ unter `prt_art/include/prt_art/` organisiert; jeder Baustein liegt in einem eigenen thematischen Unterverzeichnis (14), `prt_art/src/` ist ein leeres Implementierungsverzeichnis (header-only; der gesamte Bibliotheks-Code liegt unter `include/`). Die Cache Engine lädt ihn als Plugin: `comdare_pruefling.cmake` registriert den Code-Weg über `COMDARE_CE_PRUEFLINGE`, die generierte `prt_art_axis_registry.xml` ist der Registry-Weg für den Planer (`COMDARE_PRT_ART_DIR`); eine Gegenprüfung im Prüflings-Kontrakt der Cache Engine meldet es, wenn nur einer der beiden Wege gesetzt ist, und bricht die Konfiguration ab, wenn beide auf verschiedene Orte zeigen (Abschnitt 4.2). Im Stempel führt der Prüfling seine Achsen unter eigenem Namensraum (Präfix `prt-art.`).

```
comdare-prt-art/                                Zweig development @ b539d6ee
|-- CMakeLists.txt, comdare_pruefling.cmake      Build-Einstieg (CMake) + Plugin-
|                                                  Registrierungs-Snippet fuer die ce
|-- LICENSE, NOTICE, LICENSES/, REUSE.toml      Lizenz-Basis (Comdare Research
|                                                  License 1.0)
|-- README.md, CITATION.cff, CONTRIBUTING.md, SECURITY.md, CODE_OF_CONDUCT.md,
| CHANGELOG.md, PROJECT_LAYER_MAP.md, CMakePresets.json  (Sammel-Eintrag)
|-- prt_art/
|   |-- include/prt_art/      Bausteine des Prueflings (Header, 14 Verzeichnisse)
|   |   |-- identity/        prt_art_search_engine.hpp (PrtArtSearchEngine),
|   |   |                    prt_art_execution_engine_adapter.hpp,
|   |   |                    prt_art_pruefling_factory.hpp,
|   |   |                    prt_art_identity.hpp, status.hpp
|   |   |-- allocator/       pool_descriptor.hpp, pool_router.hpp,
|   |   |                    pool_set.hpp
|   |   |-- concurrency/     olc_with_reserved_blocks.hpp
|   |   |-- prefetch/        path_oriented_prefetch.hpp,
|   |   |                    redirect_prefetch.hpp, distance_estimator.hpp
|   |   |-- memory_layout/   multi_level_layout.hpp u. a.
|   |   |-- measurement/     density_tracker.hpp, hypothesis_metrics.hpp
|   |   |-- telemetry/       leaf_only_counter.hpp
|   |   |-- slots/           Slot-Wrapper der vier Achsen-Beitraege
|   |   |                    (axis_01_page_type, axis_07_prefetch,
```

⁵Header-only: Bibliothek, deren Definitionen vollständig in Header-Dateien liegen und beim Einbinden übersetzt werden; eine separat übersetzte Bibliotheksdatei gibt es nicht.

```

| | | axis_11_telemetry, axis_14_value_handle)
| | | + Kompositions-Demo
| | | '-- ... nodes/, default_lookup/, value_handle/,
| | | value_buffer/, internal_search/, serialization/
| |-- legacy_reimpl/ 14 Re-Implementierungen der Arbeiten P11-P27
| | (ohne P15, P20, P25; P11-CSS-tree ...
| | P27-Zhang-ASPLOS-Hierarchical)
| |-- algorithm_profiles/ prtart_pruefling.profile.xml,
| | permutation_axes_extension.xml,
| | prt_art_axis_registry.xml (generiert von
| | registry_gen/, eigener Namensraum engine=prt_art)
| |-- codegen/ + permutations_codegen/ Codegen-Vorlagen des Prueflings
| |-- registry_gen/ Registry-Generator + Roundtrip-Pruefung
| | (registry_roundtrip.cmake)
| |-- src/ leeres Implementierungsverzeichnis (header-only)
| |-- tests/unit/ modulnahe Tests
|-- tests/unit/ repositoriumsweite Unit-Tests
|-- cmake/ Build-Hilfsdateien
'-- docs/ architektur/, ledger-sections/ (repo-lokales
Ledger), sessions/

```

Komponente	Pfad	Zweck
PrtArtSearchEngine	prt_art/include/prt_art/identity/ prt_art_search_engine.hpp	hybride Engine des Prüflings: Vector-API bei einem, Map-API bei zwei, Map-Tuple-API bei mehr als zwei Parametern
ExecutionEngine- Adapter	prt_art/include/prt_art/identity/ prt_art_execution_engine_ adapter.hpp	ABI-Anbindung an die Mess-Infrastruktur
Pool-Allokator	prt_art/include/prt_art/allocator/	Deskriptoren der sieben Pools: vier Seiten-Pools A–D, ein Rest-Pool R, zwei Wert-Pools (4+1+2)
OLC	prt_art/include/prt_art/ concurrency/	OLC mit reservierten Blöcken
Prefetch	prt_art/include/prt_art/prefetch/	pfadorientiertes Prefetch, Redirect-Prefetch
MultiLevel-Layout	prt_art/include/prt_art/ memory_layout/	multi_level_ layout.hpp u. a.
DensityTracker, H1–H3	prt_art/include/prt_art/ measurement/	density_tracker.hpp, hypothesis_ metrics.hpp
legacy_reimpl	prt_art/legacy_reimpl/	14 Vergleichs-Re- Implementierungen (P11–P27, ohne P15, P20, P25)
Prüflings-Profil	prt_art/algorithm_profiles/	Profil- und Achsen-Erweiterungs-XML

Komponente	Pfad	Zweck
Registry-Abzug	prt_art/algorithm_profiles/ prt_art_axis_registry.xml, prt_art/ registry_gen/	generierte Achsen-Registry des Prüflings (engine=prt_art) mit Roundtrip-Prüfung; liegt als Fixture auch in der Cache Engine (tests/unit/ thesis_tiere/)
Slot-Beitrag	prt_art/include/prt_art/slots/	vier Slot-Wrapper und Kompositions-Demo (Slot-Merge in die Achsen 01, 07, 11, 14)
Plugin-Einbindung	comdare_pruefling.cmake	Registrierungs-Snippet für den Plugin-Controller der Cache Engine (COMDARE_CE_ PRUEFLINGE; Kontrakt cmake/pruefling_ kontrakt.cmake der Cache Engine)
Repo-Ledger	docs/ledger-sections/	repo-lokales Architektur- und Offene-Punkte-Ledger (Vier-Ledger-Struktur)
Lizenz-Basis	LICENSE, NOTICE, LICENSES/, REUSE.toml	Comdare Research License 1.0 (SPDX LicenseRef- Comdare- Research-1.0)

probst-Diplomarbeit-cache-engine — Super-Repository

Das Super-Repository bündelt Mess-Treiber, Experiment-Konfigurationen, die neunstufige Auswertungs-Pipeline (01 bis 09), CI-Prüfungen und die Einbindung der drei anderen Repositorien als Submodule [Git26]. Unter `Code/external/` liegen drei Einträge: die beiden Code-Repositories und das Manuskript-Repository als statischer Fundort der aus der Messung erzeugten LaTeX-Anlage neben dem übrigen Quellcode, der auch den Betrieb ohne Netz abdeckt. Dasselbe Manuskript-Repository ist zusätzlich als Baupfad `thesis/diplomarbeit` eingebunden, in dem die CI das PDF baut; `.gitmodules` führt damit vier Submodule.

[illegible]

```

|-- README.md, MANUAL_RUN.md, CITATION.cff, CONTRIBUTING.md, SECURITY.md,
| CODE_OF_CONDUCT.md, CHANGELOG.md, PROJECT_LAYER_MAP.md    Projekt-Dokumente
|-- Code/
| |-- 01_sample_data_generator/    Erzeugung von Beispieldaten
| |-- 02_messung_driver/          Mess-Treiber (main.cpp,
| |                               v32_orchestrator.hpp, plugin_loader.hpp,
| |                               measurement_writer.hpp, ...)
| |-- 03_binary_to_csv/           Auswertungs-Stufe 1: binaere Messdaten
| |                               -> CSV
| |-- 04_csv_to_latex/            Auswertungs-Stufe 2: CSV -> LaTeX-Tabellen
| |-- 05_diagram_generator/       Auswertungs-Stufe 3: Diagramm-Erzeugung
| |                               (TikZ)
| |-- 06_latex_to_pdf/            Manuskript-Build (build_thesis.sh/.bat)
| |-- 07_tier_binary_report/       Bericht ueber die Tier-Mess-Binaries
| |-- 08_appendix_generator/       In-Process-Generator des Mess-Anhangs (linkt
| |                               csv_to_latex und diagram_generator direkt)
| |-- 09_tex_formatter/           satz-neutraler LaTeX-Quelltext-Formatierer
| |-- common/                     gemeinsamer Perzentil-Kanon
| |                               (percentile_canon.hpp)
| |-- ci_wachen/                  CI-Pruefungen in C++ (CI-Invarianten,
| |                               YAML-Scanner, Gitlink-Paritaet,
| |                               Lizenz-Konsistenz, Mutationslauf,
| |                               XML-Wohlgeformtheit, WIDE-Aggregat)
| |-- experiment_config/          messreihen.xml (Messreihen-Katalog:
| |                               A_defined, A_full, B_CacheEngine_Permis,
| |                               C_Merge_Alt_Neu), thesis_profiles/
| |                               (fl_durchstich), sample_data/
| |-- test_data_xml/              XML-Schemata experiment_schema.xsd und
| |                               messreihe_v32_schema.xsd, golden-XML,
| |                               Testdaten-XML
| |-- external/                   Submodule: comdare-cache-engine,
| |                               comdare-prt-art,
| |                               20260931-overleaf-diplomarbeit (statischer
| |                               Fundort)
| |-- tests/, cmake/              repoweite Tests, Build-Hilfsdateien
| `-- CMakeLists.txt, CMakePresets.json, USAGE.md, MANUAL_RUN.md, README.md
|-- thesis/diplomarbeit/          Manuskript-Repositorium als Submodul
| |                               (Baupfad; Struktur im eigenen Abschnitt);
| |                               daneben ZIHLatexVorlage/
| |                               (Dokumentklassen-Vorlage)
|-- ci/, scripts/                 CI-Pruefungen (Shell) und Hygiene-Skripte
| |                               (Diff-/ASCII-Pruefung, Worktree-Pruefung,
| |                               Testdaten-Beschaffung)
|-- docs/                         Ledger (Vollfassung und dichte Fassung),
| |                               Architektur- und Plan-Dokumente, Termine,
| |                               Glossar, Mess-Planung mit Erstbelegen
| |                               (docs/architektur/measurement/),
| |                               Arbeitsdokumentation
|-- Forschungsarbeiten/           Literatur-Sammlung je Achse (Sammel-Eintrag)
|-- measurement/                  thesis_compiles/: persistierte PDF-Kompilate
| |                               des CI-Rueckschriebs
|-- <Vorspann-Unterlagen>         Exposés, Skizzen, Anmeldeunterlagen und

```

Vorarbeiten der Arbeit (Sammel-Eintrag)

Komponente	Pfad	Zweck
messung_driver	Code/02_messung_driver/	Mess-Treiber: lädt
Experiment-Konfigurationen	Code/experiment_config/	Permutations-Module, orchestriert die Messreihen messreihen.xml (Messreihen-Katalog des Treibers: A_defined, A_full, B_ CacheEngine_Permits, C_ Merge_Alt_Neu), thesis_profiles/ (Profil des ersten durchgehenden Laufs der Kette), sample_data/ Auswertungs-Pipeline, Stufe 1 Auswertungs-Pipeline, Stufe 2, mit gemeinsamem Perzentil-Kanon (Code/ common/)
binary_to_csv	Code/03_binary_to_csv/	Diagramm-Erzeugung aus CSV-Daten
csv_to_latex	Code/04_csv_to_latex/	Manuskript-Build-Skripte
diagram_generator	Code/05_diagram_generator/	Bericht über die Tier-Mess-Binaries
latex_to_pdf	Code/06_latex_to_pdf/	Auswertungs-Stufe 4: In-Process-Erzeugung des Mess-Anhangs (je Sprache Kern- und Darstellungs-.tex aus einer WIDE-Matrix)
tier_binary_report	Code/07_tier_binary_report/	satz-neutraler LaTeX-Quelltext-Formatierer
appendix_generator	Code/08_appendix_generator/	experiment_ schema.xsd (Experiment-Profil), messreihe_v32_ schema.xsd (Messreihe), golden-XML und Testdaten-XML
tex_formatter	Code/09_tex_formatter/	
XML-Schemata	Code/test_data_xml/	

Komponente	Pfad	Zweck
CI-Prüfungen	Code/ci_wachen/, ci/, scripts/	Prüfungen in C++ (Gitlink-Parität, Lizenz-Konsistenz, CI-Invarianten) und Shell (Anhang-Vorwärts-Kanal, Frische-, Bestandslog-, Diff-/ASCII-Prüfung)
Submodule	Code/external/	Einbindung der beiden Code-Repositories und des Manuskript-Repositories (statischer Fundort); zweiter Zeiger <code>thesis/</code> <code>diplomarbeit</code> als Baupfad
Submodul- Deklaration	<code>.gitmodules</code>	vier Submodule; Baupfad <code>thesis/diplomarbeit</code> mit <code>branch</code> und <code>update</code> , statischer Fundort bewusst ohne
Manuskript	<code>thesis/diplomarbeit/</code>	Manuskript-Repository als Submodul (Baupfad der PDF-CI)
Mess-Rückschrieb	<code>measurement/thesis_compiles/</code> , <code>docs/</code> <code>architektur/measurement/</code>	persistierte PDF-Kompilate des CI-Rückschriebs; Mess-Planung mit den Erstbelegen der Rohdaten
Reproduktionsweg	<code>configure.sh</code> , <code>Makefile</code>	GNU-Bauweg vor CMake (wie in der Cache Engine)
Lizenz-Basis	<code>LICENSE</code> , <code>NOTICE</code> , <code>LICENSES/</code> , <code>REUSE.toml</code>	Apache-2.0 für den Eigencode; Submodule und Fremdkomponenten je eigene Lizenz (21 Lizenztexte unter <code>LICENSES/</code>)

20260931-overleaf-diplomarbeit — Manuskript-Repository

Das Manuskript-Repository enthält die LaTeX-Quellen beider Sprachfassungen (Schalter `\thesislang` in `diplomarbeit.tex`), die Generator-Ausgabe der Messung unter `anhang/{de,en}/tabellen/` und eine eigene CI mit den Jobs `lint:secrets`, `lint:latex` und `thesis:pdf`. Die Mess-CI des Super-Repositories legt die Generator-Ausgabe per Commit in diesem Repository ab und stößt den PDF-Bau an; dieser Rückschreib-Kanal ist der Grund, warum das Manuskript ein eigenes, versioniertes Repository ist (Abschnitt 4.1). Im Super-Repository ist es zweifach eingebunden: als Baupfad `thesis/diplomarbeit` und als statischer Fundort `Code/external/20260931-overleaf-diplomarbeit`. Das Aufgabenblatt liegt als eigenes Kapitel unter `aufgabenstellung/`; die Lizenz-Basis ist CC BY 4.0 (`LICENSES/CC-BY-4.0.txt`) [Cre13],

die Lizenzaussage steht im Vorspann.

```

20260931-overleaf-diplomarbeit/      Zweig development @ d32cc492
|-- diplomarbeit.tex                 Rahmendokument: Sprachschalter, Vorspann,
|                                   Lizenzabsatz, Einbindung von Kapiteln und
|                                   Anhaengen
|-- aufgabenstellung/               de.tex, en.tex (offizielles Aufgabenblatt
|                                   als Kapitel)
|-- kapitel/{de,en}/                sechs eingebundene Kapitel 01_einleitung ..
|                                   06_fazit sowie nicht eingebundene
|                                   Quelldateien
|-- anhang/{de,en}/                 Anhaenge A .. F
|   '-- tabellen/                   Generator-Ausgabe der Messung (*.tex, u. a.
|                                   le_limitierung.tex)
|-- config/                          language.tex (Sprachumschaltung)
|-- literatur.bib                   Literaturdatenbank (BibTeX)
|-- zihpub.cls, alphadin.bst, plaindin.bst, zih_logo_de_sw.{eps,pdf}
|                                   Dokumentklasse, Bibliographie-Stile, Logo
|-- Makefile, build.ps1, build_all.ps1
|                                   lokale Bau-Einstiege (pdflatex/bibtex;
|                                   PowerShell)
|-- .gitlab-ci.yml                   CI: lint:secrets, lint:latex, thesis:pdf
|-- LICENSE, NOTICE, LICENSES/, REUSE.toml   Lizenz-Basis (CC BY 4.0)
|-- docs/, sessions/, tools/        repo-lokales Ledger, Quellhinweise,
|                                   Arbeitsdokumentation, TeX-Formatierer
'-- <PDF-Fassung>                   gesicherte PDF-Fassung der Arbeit
                                   (Sammel-Zeile)

```

Komponente	Pfad	Zweck
Rahmendokument	diplomarbeit.tex	Vorspann, Lizenzabsatz, \thesislang-Schalter, Include-Liste der Kapitel und Anhänge
Kapitel	kapitel/de/ (kapitel/en/)	sechs eingebundene Kapitel; die englische Fassung folgt der deutschen als Übersetzung
Anhänge	anhang/de/ (anhang/en/)	A Messwerte, B Code-Struktur, C Glossar, D Bausteinmatrix, E Architekturentscheidungen, F Vergleichs-Schnittstellen
Generator-Ausgabe	anhang/{de,en}/tabellen/	von 04_csv_to_latex, 05_diagram_generator und 08_appendix_generator erzeugte .tex (Tabellen, Diagramme, Limitierungen); Ort der Messwerte (Anhang A)
Aufgabenblatt	aufgabenstellung/	de.tex, en.tex

Komponente	Pfad	Zweck
Literatur	<code>literatur.bib</code>	BibTeX-Datenbank der Arbeit
Klasse und Stile	<code>zihpub.cls</code> , <code>alphadin.bst</code> , <code>plaindin.bst</code>	Dokumentklasse und Bibliographie-Stile
CI	<code>.gitlab-ci.yml</code>	<code>lint:secrets</code> , <code>lint:latex</code> , <code>thesis:pdf</code> (DE und EN mit Warnungs-Gates)
Lizenz-Basis	<code>LICENSE</code> , <code>NOTICE</code> , <code>LICENSES/</code> , <code>REUSE.toml</code>	CC BY 4.0 (SPDX CC-BY-4.0), Copyright beim Autor
Repo-Ledger	<code>docs/ledger-sections/</code>	repo-lokales Ledger (Vier-Ledger-Struktur)

C Glossar

Dieses Glossar fasst die in der Arbeit verwendeten Abkürzungen und zentralen Begriffe zusammen. Jedes Fachbegriff-Lemma nennt zuerst den englischen Musterbegriff, dann die Definition und zuletzt die in dieser Arbeit gleichbedeutend gebrauchten Aliase; Registry- und Code-Bezeichner stehen in Schreibmaschinenschrift. Die Verweise zeigen auf den Abschnitt, der den Begriff einführt.

ABI Application Binary Interface — die binär-stabile Schnittstelle, über die der Builder die Permutations-Module (Lebewesen) lädt.

Abschaltungsstufe (*instrumentation shut-off level*) Stufung, in der die Messeinrichtung einer Tier-Binary zurückgenommen wird: Stufe 1 schaltet in der Makro- und der Mikro-Ebene die Erhebung der Achsen-Parameter ab, sodass die Wall-Clock-Messung für `measure` und `compare` bestehen bleibt; Stufe 2 (permutativ, `release`) entfernt auch die Wall-Clock-Messung und mit ihr die Mess-Flächen und -Interfaces aus dem Kompilat. Umsetzungsstand: Der Code enthält die Mess-Gates G2 und G3, deren Zustand ein Fingerprint-Glied ist, und den Release-Modus ohne Instrumentierung; die zweistufige Abschaltung als Permutations-Größe ist vorgesehen, aber nicht implementiert (Abbildung 4.11, Abschnitt 4.5).

Achse (*axis*) Eine orthogonale Teilentscheidung („Organ“) eines Suchalgorithmus. Das Framework ordnet seine Achsen drei Realms zu: achtzehn Organ-Haupt-Achsen (Kompositions-Achsen in fester Slot-Reihenfolge T0–T17, Registry-Etikett T00–T17) und eine optionale Organ-Meta-Meta-Achse `disk_io`; drei System-Haupt-Achsen (`target_isa`, `operating_system`, `external_utils`) mit ihrem Komplex-Wrapper; und den Mess-Realm mit der Mess-Tooling-Haupt-Achse (vgl. Abschnitt 3.4.1). Jede Achse ist Haupt-, Unter- oder Meta-Meta-Achse (siehe die Lemmata); in die `binary_id` gehen ausschließlich die achtzehn Organ-Haupt-Achsen ein.

Achsen-Erfolgs-Parameter (*axis success parameter*) Die je Achsen-Interface erhobenen Kenngrößen, an denen ein Interface-Aufruf von Anfang bis Ende gemessen wird; das Messgerät ist der Mess-Checkpoint `checkpoint_measure` (Mikro-Ebene), und ab der Makro-Ebene werden alle Parameter einschließlich der PMC-Zähler zusammengeführt (vgl. Abschnitt 3.4). Der Katalog der Parameter je Achse liegt nicht vor (Gegenstand künftiger Arbeit).

Adaptiv vs. dynamisch vs. messgetrieben Begriffsabgrenzung für die Anpassung des Algorithmus-Verhaltens. *Adaptiv*: Anpassung anhand laufzeit-beobachteter Datenmerkmale (z. B. adaptive Knotengrößen in ART). *Dynamisch*: im allgemeinen Sprachgebrauch jede zur Laufzeit getroffene Entscheidung; in dieser Arbeit enger die Laufzeit-Auswahl des fertig kompilierten Permutations-Moduls über die Binary-Identität `binary_id` (Modul-Ladung) — alle achtzehn Organ-Achsen sind je Binary zur Übersetzungszeit festgelegt; hinzu kommen laufzeit-variierebare Parameter dynamischer Unter-Achsen (*sub-axes*: Prefetch-Distanz, Stapelgröße, Inline-Schwelle,

Pool-Budget; vgl. Abschnitt 3.4), keine Laufzeit-Wahl einzelner Achsen-Strategien. *Messgetrieben* (*measurement-driven*): Auswahl der Konfiguration auf Basis empirisch erhobener Messreihen — das Leitmotiv dieser Arbeit; der zugehörige Auswahl-Mechanismus ist in einem ersten Inkrement implementiert (`best_binary_selector`: Ranking und Auslieferung der besten Permutation je Metrik aus den Mess-CSVs), die vollautomatische, lastprofilübergreifende Auswahl ist Gegenstand des Ausblicks (vgl. Abschnitt 6.3).

Alder Lake Intel-Mikroarchitektur mit heterogenen Performance- und Efficiency-Kernen; die zweite Messplattform dieser Arbeit (`prod2`, Intel Core i9-12900K) gehört ihr an. In der System-Registry ist sie die Rekombination `prod2_alder_lake` (CPU-Fabrikation GenuineIntel/6/151/2, RAM-Nennrate 4800 MT/s deklariert, CAS-Latenz nicht erhoben).

Anatomie (*anatomy*) Der *Körper* eines Lebewesens: die feste Komposition seiner achtzehn Achsen-Organen und vor allem deren *Verdrahtung* — die generalisierten Interfaces, über die ein Organ die Dienste der übrigen nutzt (etwa eine Container-Gattung als Knoten-Substrat der Achse `node_type` oder als Index-Organisation). Kein zweites Modell neben dem Suchalgorithmus, sondern dessen Innenstruktur (vgl. Abschnitt 3.3.1).

Antwort-Status (*answer status*) Die je Forschungs-Haupt- und -Teilfrage im Fazit ausgewiesene Stufe der Beantwortung: *beantwortet* (ein Test oder eine Messung belegt die Antwort), *strukturell beantwortet* (der Apparat, der die Antwort liefert, ist implementiert und getestet, die Messung liegt nicht vor) oder *offen* (weder Struktur noch Messung liegen vor). Ohne Messwerte und konkrete Belege gilt eine Frage als nicht beantwortet, auch wenn ihre Struktur tragfähig ist (vgl. Abschnitt 6.1).

AoSoA Array of Structures of Arrays — Speicher-Layout, das Datensätze in Blöcke fester Breite zerlegt und innerhalb eines Blocks feldweise (SoA) ablegt; es verbindet die Cache-Line-Dichte des SoA-Layouts mit der Lokalität des AoS-Layouts. In dieser Arbeit eine Ausprägung der Organ-Achse `memory_layout`.

Arbeitsmodus (*work mode*) Die vier Modi der Ablauf-Kette, in dieser Reihenfolge: `build` (Bau der Tier-Binaries), `measure` (der deterministische Mess-Lauf), `compare` (Vergleich der Mess-Ebenen und Permutationen aus dem Messwertlager) und `release` (Auslieferung ohne Messfühler); der modus-spezifische Ablauf von `compare` und `release` ist für spätere Haupt-Wellen vorgesehen. Der Arbeitsmodus ist eine Unter-Achse des Mess-Realms (`work_mode`), keine Eigenschaft der Binary-Identität; das Modus-Token `compare` ist von dem gleichlautenden Alias der Wallclock-Ebene in der Mess-Konfiguration zu unterscheiden. Alias: Modus, Ablauf-Kette.

ART Adaptive Radix Tree [LKN13]; Referenzentwurf mit adaptiven Knotengrößen (Node4/16/48/256).

Autonome Heuristik-Profil-Auswertung (*autonomous profile-driven heuristic evaluation*) Der vorgesehene Mechanismus, der aus den Messreihen je Lastprofil automatisch die beste Achsen-Konfiguration ableitet und als wiederverwendbare Heuristik-Richtlinie ablegt (Ziel-Mechanismus: ML-Klassifikation von Lastprofil-/Achsen-Features auf Konfigurationen). Erste Bausteine sind implementiert — die automatische Best-Binary-Auswahl aus Messreihen (`best_binary_selector`) und die Extraktion/Ablage als XML-Lastprofil-Ergebnis —; die ML-Klassifikation selbst ist Ausblick (vgl. Abschnitt 6.3).

AVX, AVX-512 x86-SIMD-Erweiterungen (256- bzw. 512-Bit).

Bestand (*stock*) Eine der vier Ablage-Klassen des Lagers, übersetzungsstatisch im Schlüssel-Schema festgeschrieben: Bestand 1 die gebauten Tier-Binaries, geschlüsselt über ihren SHA-512-Fingerprint, die Mess-Zelle daneben geklammert; Bestand 2 die Messwerte, geschlüsselt über den Fingerprint der vermessenen Binary und die Hardware-Identität der messenden Maschine, die Plattform-Zelle daneben geklammert und nie in den Digest fusioniert; Bestand 3 die Synthese-Batches mit den synthetisierten Mess-Kurven als Dateien; Bestand 4 die Lösungen je Anwender-XML (vgl. Abschnitt 3.4.5).

binary_id Der Identitäts-Pfad einer Tier-Binary im Permutationsraum: die Belegung der achtzehn Organ-Haupt-Achsen in kanonischer Reihenfolge (achtzehn Segmente, das letzte `persistence_target`). System- und Mess-Achsen stehen nie in der `binary_id` (Registry-Attribut `binary_id` mit dem Wert `never`); sie wirken über Bau-Version, Stempel-Zeilen und CSV-Spalten.

built_stem Feld der Plan-Textform (Fassung v1.1), das je Bau-Schritt den tatsächlich erzeugten Datei-Stamm ausweist — erzeugt von derselben Funktion, die auch der Bau-Orchestrator beim Bau aufruft; Beleg für den *einen* Hauptkanal (vgl. Abschnitt 4.3.2).

CacheEngineBuilder (CEB) Das autonome Plattform-Ausmess-System der Kette und die zweite der vier Träger-Stufen; Wurzel des System-Realms. Der CEB empfängt vom Planer den freigegebenen System-Achsen-Raum, permutiert System- und Organ-Achsen zu Tier-Binaries, baut und stempelt sie je Plattform und führt die Sieben-Phasen-Pipeline über den `ExperimentDriver` aus. Sein Stempel enthält Mess- und System-Zeile, aber keine Organ-Zeile (die statische Sperre); sein SHA-512-Fingerprint ist Provenienz, kein Wiederverwendungs-Kriterium (vgl. Abschnitte 3.4.6 und 4.3.2).

CEB siehe *CacheEngineBuilder*.

checkpoint_measure Das zentrale Messinstrument des Systems: ein Messpunkt, der zu einem Zeitpunkt die Eingangsparameter und die Zeit erhebt. Auf der Wall-Clock-Ebene ist er das parameterleere Minimum; sind Makro- oder Mikro-Ebene aktiv, vollzieht er zusätzlich die Parameter-Klammer — Zeit und Beobachter sind im Checkpoint vereint, die Verantwortung ist je Mess-Ebene getrennt. Die Zeilen werden in der Mess-Arena abgelegt, die Verschachtelung führt die Stapel-Arena. Alias: Mess-Checkpoint.

CLU Cache-Line-Utilization — Anteil genutzter Bytes je geladener Cache-Line.

CPI, IPC Cycles per Instruction bzw. Instructions per Cycle.

CRTP Curiously Recurring Template Pattern — C++-Idiom für statischen Polymorphismus: Eine Klasse leitet von einer Klassenschablone ab, die sie selbst als Schablonen-Argument erhält, sodass die Basis zur Übersetzungszeit auf die Erbin zugreift [Cop95]. Im Framework die Bauform des einen Stempel-Vertrags aller Träger-Stufen (die Schablone `StempelBasis` mit der Erbin und der Träger-Stufe als Parametern) und der Achsen-Konzepte.

CSV-Baum (*CSV tree*) Die CSV-Form der Ergebnis-Mappe: kein einzelnes Kind-Dokument, sondern ein geordneter Baum aus CSV-Dateien — ein Gesamtordner je Mappe, darunter ein Ordner je Blatt, darin die CSV-Dateien der einzelnen Unter-Achsen-Fahrten —, immer aus der *xlsx*-Mappe erzeugt (*xlsx* nach *csv*, als Strategie derselben Factory) und als Darstellung ihr analog gespiegeltes Gegenstück; per XML wird die Mappe, der CSV-Baum oder beides am selben Ziel abgelegt. Umsetzungsstand: Die Naht schreibt die CSV-Projektion als eine flache Datei; die Baumform ist nicht implementiert (Abschnitt 5.2.2).

Dichte-Schwellen (*density thresholds*) Die drei festen Umschaltregeln des Prüflings PRT-ART zwischen seinen vier Seitendarstellungen nach Belegungsdichte, in der Reihenfolge Array_{256} (bis 25 %) $\rightarrow \text{Array}_{65535}$ (25 bis 50 %) $\rightarrow \text{Vector}\langle u8, u8 \rangle$ (50 bis 75 %) $\rightarrow \text{Vector}\langle u16, u16 \rangle$ (über 75 %); der Prüfling schaltet nach diesen festen Schwellen um; die plattform-kalibrierten Regeln liegen nicht vor, ihre Festlegung setzt die Messläufe voraus (Teilfrage FF3.a, Abschnitt 1.3; Abschnitt 3.5). Die Schwelle bei 50 % ist zugleich die Seitentyp-Grenze zwischen Punkt- und Kompaktseite (siehe *Punktseite*, *Kompaktseite*), keine weitere Umschaltregel.

Drei-Ebenen-Modell (*three-level model*) Die drei Interface-Ebenen der einen Architektur: *Konzept* (Ebene 1, die Tierklasse als Mess-Surface — Map, Container, Graph, HeuristikAdapter), *Gattung* (Ebene 2, das Tier-Interface als Template — SearchAlgorithm, Set, Sequence, Adapter, View, FunctionInterfaceReroute) und *Genus* (Ebene 3, die Unterart als Implementierung des Gattungs-Interfaces — ART, HOT, START). Darunter liegen die *Organe/Achsen* (Ebene 4) als eigene Klassifikation, darüber das Reich (Ebene 0); Tier-Binary (Ebene 5a) und Hybrid-Tier (Ebene 5b) sind die Nachphase der Kompilation. Siehe Abschnitt 3.3.2.

dTLB Daten-Translation-Lookaside-Buffer; cacht Daten-Adressübersetzungen.

ECDF Empirische Verteilungsfunktion (*empirical cumulative distribution function*) — Treppenfunktion, die jedem Wert den Anteil der Beobachtungen zuordnet, die ihn nicht überschreiten [WG68]; im Mess-Anhang die Darstellung der Latenz-Verteilung über die Konfigurationen (bei einer einzigen Konfiguration eine einstufige Treppe).

Explizit kodierte Nicht-Erhebung (*explicitly coded non-observation*) Grundsatz der Mess-Kette: Ein nicht erhobener Wert ist ein eigener Zustand (NaN nach IEEE 754 bzw. ein Gültigkeits-Flag [IEE19]) und wird nie als Null oder Phantomwert geführt — weder im Schema noch in CSV, *xlsx*, Tabelle oder Diagramm; eine Messreihe ohne gültige Zeile erzeugt keine Ausgabe statt einer leeren. Der fehlende Wert bleibt damit als eigene Kategorie auswertbar [Gra09]. Alias: Grundsatz der ausgewiesenen Nichterhebung; im Code das Vokabular `honest-0/honest-empty` der Kommentare und Prüf-Dock-Meldungen, im Diagramm-Generator des Werkzeug-Repositories (`diagram_generator`) die Platzhalter-Routine `write_honest_empty_placeholder` (vgl. Abschnitt 3.4.1).

Eytzinger-Layout (*Eytzinger layout*) Anordnung eines sortierten Feldes in der Ebenen-Reihenfolge eines impliziten vollständigen Binärbaums (Kinder von Index i bei $2i$ und $2i + 1$), sodass die binäre Suche cache-line- und prefetch-freundlich verläuft [KM17]. In dieser Arbeit eine Ausprägung der Speicher-Layout-Achse.

Fingerprint (*fingerprint*) Der kryptografische Identitäts-Hash je Träger-Stufe: Der Planer trägt einen SHA-256 über Version, ISA und Betriebssystem (64 Hex-Zeichen); die Tier-Binary einen SHA-512 über elf Glieder in fester Ordnung (Format 6: Format-Kennung, Mess-, System- und Organ-Zeile, Unter-Achsen-Werteset, Toolchain, Enabled-Mengen-Signatur der Bau-Achsen, Overlay-Quellmenge, Mess-Gates, Hybrid-Komposit, Bau-Versions-Basis; 128 Hex-Zeichen); der CEB einen SHA-512 als Provenienz. Der Fingerprint ist der Lager-Schlüssel des Bestands 1 und das Wiedererkennungs-Kriterium einer Permutation.

First-Touch (*first-touch policy*) NUMA-Platzierungsregel des Betriebssystems: Eine Speicherseite wird dem Knoten zugeordnet, dessen Prozessor sie zuerst beschreibt; deshalb entscheidet der initialisierende Thread über die Speicher-Lokalität [Lam13].

Forest-Plot (*forest plot*) Darstellung mehrerer Effektschätzer mit ihren Unsicherheitsintervallen als übereinander gestellte Punkt-Balken-Zeilen [LC01]; im Mess-Anhang die Sicht auf den Achsen-Austausch je Konfiguration gegen eine Referenz.

Gattung (*family*; Code: `AnatomyGenus`) Die zweite Ebene des Frameworks (Ebene 2) und die erste Tier-Interface-Ebene unter einem Konzept: das Interface, das eine Gattung als UND-Schnittmenge der Interfaces aller ihrer Genera nach außen als Template trägt (im Metapher-Kanon der Elefant = `SearchAlgorithm` unter dem Säugetier Map). Sechs Gattungen: `SearchAlgorithm` unter Map, Set, Sequence, Adapter und View unter Container (Specht, Storch, Kakadu, Falke) sowie `FunctionInterfaceReroute` (Ameise) unter HeuristikAdapter. Die Gattung ist die einzige gepflegte Quelle der Einordnung; das Konzept wird zur Laufzeit aus ihr ermittelt. Je andockender Gattung gibt es genau ein Prüf-Dock (fünf Docks für `SearchAlgorithm`, Set, Sequence, Adapter und View), während die klassifizierende Gattung `FunctionInterfaceReroute` keines hat und ihre Binaries am Dock der gemeldeten Ziel-Gattung andocken. Die Tier-Binary zeigt dem Prüf-Dock ihr Gattungs-Interface; Docks verschiedener Gattungen werden nie gemischt. Die Gattungs-Tests des Konformitäts-Gates sind generisch für alle Genera der Gattung. Code-Brücke: `AnatomyGenus` (Umbenennung im Code nach dem Vortrag; siehe *Konzept, Genus, Drei-Ebenen-Modell*).

Genus (*genus*) Die dritte Ebene (Ebene 3) und zweite Tier-Interface-Ebene: die Unterart, die Implementierung des Gattungs-Interfaces mit ggf. erweiterten Spezial-Interfaces (im Metapher-Kanon die Elefanten-Arten Mammut = ART, afrikanischer Elefant = HOT, indischer Elefant = START unter der Gattung `SearchAlgorithm`; die Giraffe ist reserviert, CoCo hat kein Code-Objekt). Unter dem Genus liegen eine oder mehrere Suchstrategien (einfach = Belegung einer einzelnen Achse, komplex = Komposition mehrerer Achsen); die Organe sind eine eigene Klassifikation (Ebene 4), die die Genus-Implementierung bindet. Das Genus ist weder die Suchstrategie-Achse noch die Komposition seiner Organe (die Zusammensetzung aus Achsen ist Sache des Versions-Stempels, Abschnitt 3.4.4, nicht der Taxonomie). Die Spezial-Interfaces des Genus prüft das Konformitäts-Gate nur unter `-debug` (Genus-Tests); dem Prüf-Dock zeigt die Tier-Binary ihr Gattungs-Interface, nicht das Genus. Die Genus-Implementierung führt je Funktion einen Hauptalgorithmus, der ausschließlich über Achsen-Interface-Aufrufe arbeitet, und ruft fremde Achsen als erste auf. Ist-Stand: Der Code trägt das Genus heute nur als Achsen-Belegung; die Genus-Implementierungsklasse fehlt (erkannter Architekturfehler, Behebung nach dem Vortrag). Das Hybrid-Artefakt klassifiziert die

Gattung `FunctionInterfaceReroute`; `genus()` einer Hybrid-Binary meldet dagegen die geerbte Ziel-Gattung, die Umleitung bleibt nach außen transparent. Code-Brücke: `AnatomyGenus` bezeichnet im Code die *Gattung* dieser Arbeit, nicht das Genus (Umbenennung nach dem Vortrag; siehe *Gattung*).

Gitlink (*gitlink*) Der im übergeordneten Repositorium verzeichnete Commit eines Submoduls (Eintragsmodus 160000); `.gitmodules` deklariert je Submodul Pfad, URL und optional den Zweig [Git26]. Das Super-Repositorium führt so den maßgeblichen Stand von Engine, Prüfling und Manuskript.

GoF „Gang of Four“ — die kanonischen Entwurfsmuster nach Gamma et al. [GHJV94].

golden-Referenzkatalog (*golden reference catalogue*) Der feste Regressions-Detektor des Ganzsystems: siebzehn Organ-Achsen mit je zwei Ausprägungen, `persistence_target` auf den In-Memory-Vertreter festgelegt, ergeben $2^{17} = 131072$ Identitäten, über eine CRC-64-Prüfsumme statt über die materialisierte Liste verankert — unter der vereinfachenden, überholten Annahme genau zweier Auslegungen je Achse; die Achsen tragen mehrere Auslegungen, der Katalog bleibt Regressions-Detektor, nicht Obergrenze. Er ist von der Obergrenze der Binary-Identitäten je System-Permutation zu trennen: dem Produkt der je Achse im XML-Experiment gewählten Auslegungen, das der Planer je Profil und Plattform dynamisch aus den aktivierten Katalogen rechnet und das erst zur Ausführung bekannt ist (vgl. Abschnitt 3.4.6).

H2-Akte (*H2 dossier*) Die werkzeug-berechnete Datenquelle der Evaluations-Hypothese H2 (Code-Qualität je Quelle): eine gewichtete `cppcheck`-Befunddichte je kLOC über die vendorten Originalquellen der Stand-der-Technik-Entwürfe; ohne vendorte Originalquelle führt die Akte *n/a* mit Grund. Nicht zu verwechseln mit den PRT-ART-Telemetrie-Metriken H1–H3.

Haupt-Achse (*main axis*) Eine übersetzungsstatische Achse, deren Wahl fest in den CEB bzw. die Tier-Binary einkompiliert wird und in der Bau-Legende erscheint: die achtzehn Organ-Haupt-Achsen (identitätsbildend), die drei System-Haupt-Achsen und die Mess-Tooling-Haupt-Achse. Ihre Unter-Achsen permutiert der Planer zur Laufzeit. Im Stempel-Eintrag ist die Haupt-Achse die klammerlose Ebene 0.

HBM High-Bandwidth Memory.

HDR-Histogramm (*high dynamic range histogram*) Histogramm mit logarithmisch gestuften Klassen und garantierter relativer Genauigkeit über viele Größenordnungen [Ten12]; in der Auswertung die Verteilungssicht auf die Latenzen, während die Perzentile *nearest-rank* aus den Rohwerten bestimmt werden (siehe *nearest-rank-Median*).

header-only Bibliotheksform, deren gesamter Code in Header-Dateien liegt und beim Einbinden übersetzt wird; es entsteht kein eigenes Bibliotheks-Artefakt. Der Prüfling PRT-ART und die Hybrid-Stufe liegen in dieser Form vor.

HOT Height Optimized Trie [BZP⁺18].

Hybrid-Tier-Binary (*hybrid tier binary*) Die vierte Träger-Stufe (Ebene 5b: zwischen dem CEB-Prüfdock und den Tier-Binaries eingeordnet als Multiplexer mit Slots) in der Bau-Reihenfolge, in der Verarbeitungskette der optionale Mux-Einschub zwischen CEB-Prüf-Dock und Tier-Binary: ein Modul der Konzept HeuristikAdapter (Gattung FunctionInterfaceReroute), das zur Übersetzungszeit die Interfaces einer Gattung und einer Gattung erbt und Aufrufe nach heuristischen Kriterien an angeschlossene Tier-Binaries durchstellt; es exportiert dieselben Pflicht-Symbole wie ein Tier, meldet nach außen die Ziel-Gattung und wird ausschließlich mithilfe des CEB gebaut. Umsetzungsstand: Klassifikation, Dock-Schicht, XML-Parser und Binary-Proxy sind implementiert; der Modul-Export als eigenes Kompilat, der Reroute-Router, die Verdrängung, die Zustands-Synchronisation (Memento-Sidecar) und das Live-Ranking sind nicht implementiert (vgl. Abschnitt 4.5.1).

HybridAware Der sechste Kern-Modus des Mess-Modells und Wert HybridAware der Scheduling-Dimension `hetero_core_dispatch` (neben `None`, `PCoresOnly` und `ECoresOnly`): Der Lauf kennt die heterogene Kern-Topologie, pinnt seine Threads zur Laufzeit über die Affinitäts-Schnittstelle des Betriebssystems auf ausgewählte Kerne beider Sorten und misst dort mit dem zur Kern-Sorte gehörenden Zähler-Satz; Voraussetzung der Cache-Line-Awareness-Messung und deshalb Pflicht-Bestandteil des Mess-Modells (vgl. Abschnitt 3.4.1).

Instrumentierungs-Artefakt (*probe effect*) Die Veränderung des beobachteten Verhaltens durch das Mess-Instrument selbst [Gai86]; in dieser Arbeit der Unterschied zwischen der Tier-Binary mit und ohne Messfühler, der über die Plausibilitäts-Invariante (mit Messfühler langsamer als ohne) geprüft und über die Messfehler-Interpolation herausgerechnet wird.

Interface-Fläche (*interface surface*) Eine der drei Vertragsflächen, die jede Träger-Stufe bietet: Fläche 1 das Gattungs-Interface (Laufzeit, Abstract Factory), Fläche 2 der Stempel (Übersetzungszeit, ABI-stabil), Fläche 3 der Mess-Durchstich (`IMessVisitor`), der die Messung als eigene Naht quert, damit Konzept- und Gattungs-Interfaces unverändert bleiben (Abbildung 3.9). Alias: Vertragsfläche, Träger-Fläche.

IQR Interquartilsabstand (*interquartile range*) — Differenz zwischen drittem und erstem Quartil ($p75 - p25$); im Mess-Anhang das Streuungsmaß der Geschwister-Paar-Differenzen in den Austauschbarkeits-Langtabellen und den Forest-Plots (Whisker = IQR); ein Box-Plot der Latenzen wird dort bewusst nicht gezeichnet, weil das Schema je Operation nur die Perzentile $p50$, $p99$ und $p999$, aber keine Quartilsgrenzen ($p25/p75$) trägt [MTL78].

Komplex-Achse (*complex axis*) Klammer über einer festen Rekombination mehrerer Haupt-Achsen, die gemeinsam wie *eine* Achse auftreten: die äußere Klammer über Ziel-ISA, Betriebssystem und Erweiterungs-Hardware sowie die innere Komplex-Achse `target_isa` (RAM-Frequenz, CAS-Latenz, CPU-Fabrikation); im Identitäts-Stempel belegt eine Komplex-Achse genau ein Feld (vgl. Abschnitt 3.4.2). Ein nicht deklariertes Glied wird als nicht deklariert gestempelt, nie geraten; die CAS-Latenz ist für keine Messmaschine erhoben (n/a: die SMBIOS-Struktur Typ 17 führt sie nicht).

Konformitäts-Gate (*conformance gate*) Die unbedingte Prüfung vor jeder Messung: Das Gate prüft am Prüf-Dock den `std::map`-Vertrag der Konformitäts-Teilmenge (Anhang F), damit nur Vergleichbares verglichen wird. Unter `-debug` fährt es zusätzlich den Standard-Testsatz des Tier-Interfaces:

den generischen Gattungs-Teil (gültig für alle Genera) und den Genus-spezifischen Teil (die erweiterten Spezial-Interfaces); dieser indirekte Debug-Modus fällt mit dem Release-Bau zusammen und wird nicht separat geführt (siehe *Gattung*, *Genus*).

Konsument (*consumer*) Die Rolle des Diplomarbeits-Codes gegenüber der Engine: Ein Konsument linkt ausschließlich gegen die Cache-Engine und benutzt eine Tür (`get_cache_engine()`); der Mess-Treiber ist zugleich ihr Treiber. Davon zu trennen ist der Bediener — der Mensch, der Planer und Anlage steuert. Alias: formaler Konsument und Treiber.

Konzept (*concept*; Code: `AnatomyGattung`) Die oberste Ebene unter dem Reich (Ebene 1): die Tierklasse als Mess-Surface — Map (Säugetier), Container (Vogel), Graph (Pflanze), HeuristikAdapter (Insekt) —, das nach außen sichtbare Mess-Interface, das die Messbarkeit definiert; je Gattung materialisiert ein Prüf-Dock sie. Das Konzept wird nicht als zweites Datum gepflegt, sondern zur Laufzeit aus der Gattung ermittelt, über ein konzeptunabhängiges C-Symbol (das Symbol trägt für jedes Konzept denselben Namen). In der Produktlinien-Lesart ist das Konzept die Feature-Kern-Komposition, die Gattung die Feature-Spezial-Komposition. Lesehinweis: „Adapter“ ohne Präfix meint stets die Container-Gattung, das Hybrid-Konzept heißt immer HeuristikAdapter. Code-Brücke: `AnatomyGattung`, ABI-Symbol `comdare_anatomy_gattung` (Umbenennung im Code nach dem Vortrag; siehe *Drei-Ebenen-Modell*, *Gattung*, *Genus*).

Lager (*repository*) Der persistente Bestand des Apparats, in dem Gebautes und Gemessenes unter Stempel-Schlüsseln liegt (siehe *Bestand*): Der Lager-Baum ist nach Konzept, Gattung und Realm gegliedert, eine Permutation wird über ihren Fingerprint wiedererkannt statt neu gebaut, die Messung ist immer eingeschaltet und misst im Lager nur nach, was dort fehlt. Jede Maschinen-Konfiguration hat ihren eigenen Messwert-Bestand; ein Algorithmus-Versionswechsel löst den Neubau des betroffenen Lager-Stapels samt Neumessung aus. Umsetzungsstand: Bestand 1 und 2 sowie das Schlüssel-Schema aller vier Bestände sind implementiert; die Mechanik von Bestand 3 und 4, die Kurven-Facette, der Ablage-Pfad des Hybrid-Tiers und die Zielwahl über die XML bilden den Lager-Vollausbau und sind nicht implementiert (Abbildung 3.14, Abschnitt 3.4.5).

Lastprofil-Selektionsverzerrung (*workload selection bias*) Die Verzerrung eines Vergleichs dadurch, dass die Lastprofile so gewählt sind, dass ein Entwurf unter ihnen bevorzugt gut abschneidet [BDH⁺16]; ihr begegnet die Arbeit mit dem globalen Standard-Workload als Permutation über alle verfügbaren Workloads und mit je Veröffentlichung reproduzierten Profilen.

Lebewesen (*living being*) Ein vollständig komponiertes Tier-Modul eines Konzepts — alle Gattungen und Genera sind Lebewesen; die Gattung `SearchAlgorithm` ist der Fall mit der vollen Achtzehn-Achsen-Anatomie, und nur die ABI-Adapter-Sicht verengt den Begriff auf ihn (technischer Identifier dort: `SearchAlgorithm-Instanz`). Die Anwender-XML führt ein Lebewesen mit einem oder mehreren Tieren. Abgrenzung: Achsenlose Graph-Module sind Viren (siehe *Virus*).

LOUDS Level-Ordered Unary Degree Sequence — succincte (platzsparend nahe der informationstheoretischen Schranke) Baum-Kodierung, die die Knotengrade ebenenweise unär ablegt [Jac89].

M0–M3 (*meta-levels M0–M3*) Die vier Abstraktionsstufen, nach denen diese Arbeit ihre Achsen-Begriffe ordnet, in Anlehnung an die vierstufige Meta-Ebenen-Architektur der Object Management Group

(MOF) [Obj16]: M3 die Meta-Meta-Achsen als Indirektion einer Beschreibung; M2 die Meta-Achsen als literale Klassifizierung der unter ihrem Begriff geführten Haupt- und Unter-Achsen; M1 Konzeption und Laufzeit-Implementierung; M0 der Binärcode. Die Zuordnung von M3 ist gesetzt, der Schnitt zwischen M2 und M1 ist Lesart (Abbildung 4.10, Abschnitt 4.5).

MESI, MOESI, MESIF Cache-Kohärenzprotokolle (Grund-Protokoll bzw. AMD-/Intel-Varianten).

Mess-Ebene (*measurement level*) Eine der drei Ebenen der Mess-Tooling-Haupt-Achse, in genau dieser Reihenfolge: *wallclock* (w) — die Wall-Clock der Gesamtläufe im CacheEngineBuilder über die Last-Rahmenwerke; *macro* (ma) — die Zeit der Gattungs-Interface-Funktionen der Tier-Binary; *micro* (mi) — Zeit und Erfolgs-Parameter je Achsen-Interface. Teilmengen sind erlaubt, jede andere Reihenfolge ist syntaktisch falsch (vgl. Abschnitt 3.4). Alias: w/ma/mi; Wallclock-, Makro-, Mikro-Ebene.

Messfühler (*probe*) Die in eine Tier-Binary einkompilierte Messeinrichtung (Mess-Durchstich, Beobachter, PMC-Zähler); ihre Belegung ist im Mess-Gates-Glied des Fingerprints kodiert, sodass die Binary mit und die Binary ohne Messfühler verschiedene Identitäten besitzen und beide im Lager liegen. Der Unterschied ihrer Laufzeiten ist das Instrumentierungs-Artefakt.

Mess-Realm (*measurement realm*) Der dritte Achsen-Realm neben Organ- und System-Achsen: die Belange des Messens selbst, verwurzelt am Experiment-*Planer*. Seine Haupt-Achse ist das Mess-Tooling (wallclock/macro/micro), die Auffächerung des CEB-Typs; seine Unter-Achsen sind die sechzehn Mess-Kategorien (CSV-Spalten), der Arbeitsmodus, Workloads und Datensätze sowie die Rückschrieb-Methoden; das Last-Rahmenwerk ist seine Meta-Meta-Haupt-Achse. Alle Mess-Realm-Achsen sind strikt binary-id-neutral (vgl. Abschnitt 3.4.1).

Meta-Meta-Achse (*meta-meta axis*) Eine Achse der Stufe M3: Sie beschreibt nicht eine Belegung, sondern die Indirektion einer Beschreibung, und stempelt als Klammer-Anhang ihrer Realm-Zeile; die Meta-Meta-Ebene eines Stempel-Eintrags ist seine Klammer-Tiefe. Je Realm gibt es einen eigenen Diskriminator: `organ_meta_meta (disk_io)`, `system_meta_meta` (die Familien-Achsen unter dem Hub der Erweiterungs-Hardware, etwa die SIMD-Familie) und `measurement_meta_meta (load_framework)`; die PMC-Erfassung zählt zu den Meta-Meta-Mess-Achsen.

Metapher-Kanon Die biologische Lesart der Architektur, in dieser Arbeit einheitlich gebraucht: Reich = Animalia (IAAnatomyBase); Tierklasse = Konzept („Säugetier“ = Map, „Vogel“ = Container, „Pflanze“ = Graph — botanisch das Reich Plantae, die Metapher greift hier bewusst über das Reich Animalia hinaus —, „Insekt“ = HeuristikAdapter); Art = Gattung („Elefant“ = SearchAlgorithm, „Specht“ = Set, „Storch“ = Sequence, „Kakadu“ = Adapter, „Falke“ = View, „Ameise“ = FunctionInterfaceReroute); Unterart = Genus („Mammut“ = ART, „afrikanischer Elefant“ = HOT, „indischer Elefant“ = START); Organ = Achse; Anatomie = der Körper eines Lebewesens; Tier = ein Lebewesen einer Gattung, das Individuum die Tier-Binary; Virus = achsenloses Graph-Modul; Blut = das flüssige Organ, das alle Organ-Achsen verbindet und im Planer zu einer Übersetzungszeit-Konfiguration verdichtet wird; Lebensraum = die System-Achsen; Labor = die Mess-Achsen (vgl. Abschnitt 3.3.1). Wo der Text den Mess-Querschnitt des Hosts meint, sagt er das ohne Metapher.

- MLP** Memory-Level Parallelism — Anzahl gleichzeitig ausstehender Speicherzugriffe, die ein Kern überlappen kann; begrenzt durch die Miss-Status-Holding-Register (siehe *MSHR*).
- MPKI** Misses per Kilo-Instructions — Cache- oder TLB-Fehlzugriffe je tausend ausgeführter Instruktionen; normiert Fehlzugriffszahlen über Programme verschiedener Länge.
- MSHR** Miss Status Holding Register — Puffer eines Caches, der ausstehende Fehlzugriffe verwaltet und damit die maximale Zahl gleichzeitig offener Zugriffe (MLP) begrenzt.
- nearest-rank-Median** (*nearest-rank median*) Perzentil-Definition ohne Interpolation: Das p -Quantil ist der Wert an Rang $\lceil p \cdot n \rceil$ der sortierten Stichprobe (Typ 1 nach Hyndman und Fan [HF96]); alle Perzentile der Auswertung werden so aus den Rohwerten bestimmt, die HDR-Verteilung ergänzt die Sicht.
- NINE** Non-Inclusive-Non-Exclusive — Inklusions-Politik eines gemeinsamen Last-Level-Cache, der gegenüber den privaten Caches weder Inklusion noch Exklusion garantiert.
- NUMA** Non-Uniform Memory Access.
- OLC** Optimistic Lock Coupling — optimistische Nebenläufigkeits-Technik (ARTSync, P08) [LABN16].
- Organ** (*organ*) Metaphorischer Begriff für eine *Achse* (Ebene 4 unterhalb des Drei-Ebenen-Modells); ein Suchalgorithmus besitzt achtzehn Organ-Achsen. Neben den achtzehn Organ-Haupt-Achsen führt der Organ-Realm eine optionale Organ-Meta-Meta-Achse (*disk_io*), die nur bei einem Platten-Rückschreibe-Ziel wirksam wird (siehe *Achse*, *Meta-Meta-Achse*).
- Overlay-Quellmenge** (*overlay source set*) Die eine Menge der Quelldateien, über die das Overlay-Glied des Tier-Fingerprints hasht: je Achse die Dateien ihrer eigenen Implementierung plus die gemeinsame Tier-Substanz, in fester Ordnung je Achsen-Kategorie konkateniert und einmal per SHA-512 gehasht; dieselbe Menge ist die Grundgesamtheit des Versions-Locks.
- Pareto-Front** (*Pareto front*) Menge der Konfigurationen, die in keiner Metrik verbessert werden können, ohne sich in einer anderen zu verschlechtern (Pareto-Dominanz) [ED18]; im Mess-Anhang die Sicht auf den Latenz-Durchsatz-Speicher-Kompromiss der Permutationen.
- Passungs-Stempel** *Entwurf* eines Abgleichs, der die einkompilierte Plattform-Klasse einer Tier-Binary bei jedem Mess-Lauf gegen das Laufzeit-Verdikt der Hardware-Erkennung prüft; nicht implementiert (vgl. Abschnitt 6.1).
- P-Core, E-Core** (*performance core, efficiency core*) Die zwei Kern-Sorten hybrider Prozessoren (auf Intel seit Alder Lake); die Messung pinnt Threads je Kern-Sorte und erhebt die PMC-Zähler getrennt. Die Kern-Zuteilung ist die Dimension `hetero_core_dispatch` der Scheduling-Unter-Achse an `target_isa`, die Kern-Klasse die Laufzeit-Unter-Achse `core_class`; das Mess-Modell führt sechs Kern-Modi (vier Pinning-Modi je Kern-Sorte, der ungepinnte Referenzlauf und HybridAware als sechster Modus), die Anbindung an das Lauf-Profil ist nicht implementiert (vgl. Abschnitt 3.4.1).

Pitchfork-Layout (*Pitchfork layout*) Konvention für die Verzeichnisstruktur nativer C-/C++-Projekte (apps/, libs/, tools/, tests/, docs/, external/ — in der Engine als ext/), der die Engine folgt [Pik18].

Planer (*planner*) Der Experiment-Planer (`ExperimentPlanDirector`): löst die Anwender-XML gegen die drei Angebots-Kataloge auf, wählt die Mess-Achsen und steuert die Builder-Aufträge der Kette; Wurzel des Mess-Realms und erste der vier Träger-Stufen, als Werkzeug ein Subkommando-Programm (validate, plan, cache-key, fingerprint, status, check-size, simulate, version, help); sein Stempel trägt als einziger die Versionszeile und einen SHA-256-Fingerprint (vgl. Abschnitt 3.4.6).

PMC (Hardware) Performance Monitoring Counter. Im Mess-Realm ist der Zählerpfad ein Baustein der Kollektor-Achse (Regime `PmcCounter`); die modellgebundenen Roh-Ereignisse liefert ein Katalog je Mikroarchitektur, und in der Achsen-Typologie zählt PMC zu den Meta-Meta-Mess-Achsen.

Provenienz-Stufe Vertrauensstufe eines erhobenen Hardware-Werts — konfiguriert-gemessen, SPD-JEDEC-Basiswert oder deklariert-nicht-gemessen —, die jedes Ergebnis samt Erhebungszeitpunkt mitführt; eine niedrigere Stufe verdrängt eine höhere nur mit mitwandernder Kennzeichnung (vgl. Abschnitt 3.4.3).

Prüf-Dock (*probe dock*) Die builder-seitige Mess-Orchestrierung je Gattung: Das Konzept definiert die Messbarkeit, je Gattung materialisiert ein Prüf-Dock sie. Genau ein Dock je Gattung (SearchAlgorithm, Set, Sequence, Adapter, View) lädt ein Lebewesen, treibt es durch die Interface-Operationen und vermisst es; Hybrid-Docks binden angeschlossene Tiere. Das Dock ist keine ABI-Grenze — die ABI-Grenze bleibt das gattungseigene Antriebs-Sub-Interface samt POD-Snapshot; die Persistenz der Ergebnisse übernehmen der Ergebnis-Einzug in den Experiment-Baum und der Rückschrieb (vgl. Abschnitt 3.3.2).

Prüfling Der testweise einzugliedernde neue Suchalgorithmus (hier PRT-ART); wird vom Builder gegen die Kernbestandteile der Cache-Engine kompiliert.

PRT-ART Probst-Redirect-Tree-/ART-Hybrid — der Prüfling dieser Arbeit.

Punktseite, Kompaktseite (*array-indexed page, vector page*) Die zwei Seitenformen des PRT-ART- B^+ -Knotens nach Belegungsdichte: Die Punktseite adressiert Einträge direkt über ein Feld (8- bzw. 16-Bit-Index, Dichte bis 50 %), die Kompaktseite hält sortierte Schlüssel-Wert-Vektoren (Dichte über 50 %); die Grenze bei 50 % ist die Seitentyp-Grenze, die drei Umschaltregeln des Prüflings nennt *Dichte-Schwellen*.

Rang (*rank*) Zentralitäts-Einordnung der Stand-der-Technik-Veröffentlichungen (Rang-1/2/3); erklärt in Abschnitt 1.3. Nicht zu verwechseln mit der Rangbildung der Messung (siehe *Rangbildung*).

Rangbildung (*ranking*) Die Ordnung der vermessenen Permutationen nach einer Metrik über die drei Benchmarking-Granularitäten (Aufgabenstellung, Teilaufgabe 7); das erste Inkrement ist das Ranking je Metrik aus den Mess-CSVs mit Auslieferung der besten Binary (`best_binary_selector`).

RCU Read-Copy-Update — Reklamations-Schema (McKenney, P29) [McK01].

Realm (*realm*) Einer der drei Bereiche der Achsen mit je eigener Registry, Registry-XML und Stempel-Zeile: der Organ-Realm (Wurzel: die Anatomie des Tiers), der System-Realm (Wurzel: der CEB) und der Mess-Realm (Wurzel: der Planer); die kanonische Ordnung lautet Mess-, System-, Organ-Achsen. Alias: Achsen-Heimat, Achsen-Realm.

Registry-XML (*registry XML*) Das je Realm per Übersetzungszeit-Reflexion generierte, maschinenlesbare Abbild des Achsen-Bestands (Achsen, Bausteine, Unter-Achsen, Maschinen-Signaturen); es wird nie von Hand editiert und ist das Angebot, gegen das der Planer die Anwender-XML auflöst.

REUSE, SPDX REUSE-Spezifikation (Free Software Foundation Europe): Konvention, jede Datei eines Repositoriums per SPDX-Lizenzbezeichner und `REUSE.toml` einer Lizenz zuzuordnen; SPDX ist der standardisierte Bezeichner-Katalog für Lizenzen. Die Zuordnung lizenziert nichts um — maßgeblich bleiben die Lizenzdateien [Fre24].

RRIP, DRRIP (Dynamic) Re-Reference Interval Prediction — Cache-Ersetzungs-Heuristik [JTJE10].

Seitentyp (*page type*) Der Prüfling-Slot `page_type` (`prt-art axis_01`): die Knotenform des PRT-ART-B⁺-Baums; im Kanon der Engine entspricht er der Organ-Haupt-Achse `node_type` (Slot T04). Die Slot-Nummern des Prüflings folgen nicht der Slot-Reihenfolge T00–T17 der Engine.

SIMD Single Instruction, Multiple Data.

SOSD Search On Sorted Data — Index-Benchmark [KMR⁺19].

SOTA State of the Art — der Stand der Technik.

SPD, JEDEC Serial Presence Detect — der von JEDEC (dem Normungsgremium der Speicherindustrie) standardisierte Kennungsspeicher eines Speichermoduls, der u. a. die JEDEC-Basisfrequenz trägt [JED25]; in der Hardware-Erkennung die mittlere Provenienz-Stufe (SPD-JEDEC-Basiswert) zwischen konfiguriert-gemessen und deklariert-nicht-gemessen (siehe *Provenienz-Stufe*).

Stempel (*stamp*) Die einkompilierte, ABI-stabile Identitäts-Beschriftung eines Trägers: je Realm eine Zeile aus Einträgen der Form `achse=algo@X.Y.Z` mit optionalen Flags und rekursiven Klammer-Gruppen (`c{p.e}` ist `c` mit den Untergliedern `p` und `e`, nie ein flaches Kürzel), dazu Versionszeile und Fingerprint. Vier Träger-Stufen und sieben Stempel-Interfaces bilden eine dreierwertige Zulassungs-Matrix von 28 Zellen (Pflicht, Erlaubt, Verboten); der CEB trägt keine Organ-Zeile, nur der Planer eine Versionszeile, nur der Hybrid angeschlossene Tiere. Der Stempel ist zugleich Identität, Lager-Schlüssel und Wiedererkennungskriterium; der Versions-Ausdruck des Treibers dient dagegen nur der Einordnung (Abbildung 3.9). Alias: Identitäts-Stempel, Stempelsystem.

Subsystem (*subsystem*) In dieser Arbeit die vier Träger-Subsysteme der Kette (Mess-Treiber, CacheEngineBuilder, CacheEngine, Prüfling); im Code dagegen die zwölf `ISubEngine`-Stufen der Empfehlungs-Pipeline hinter der Fassade `ICacheEngine`. Die beiden Bedeutungen werden im Text nicht vermischt.

SuRF Succinct Range Filter [ZLL⁺18].

SVE, NEON, RVV ARM- (SVE, NEON) bzw. RISC-V- (RVV) SIMD-Erweiterungen.

Teilmenge (*subset*) In dieser Arbeit in zwei festen Bedeutungen: die zulässige Teilmenge der Mess-Ebenen (jede Teilmenge von wallclock/macro/micro in Kanon-Reihenfolge) und die Konformitäts-Teilmenge der Interface-Operationen, die das Konformitäts-Gate vor jeder Messung prüft (unter `-debug` zusätzlich der Standard-Testsatz des Tier-Interfaces; Anhang F).

Tier-Binary (*tier binary*) Die dritte Träger-Stufe (Ebene 5a der Konzept-Hierarchie: die Nachphase der Kompilation, das Individuum): das vom CEB je Permutation gebaute, dynamisch ladbare Modul eines Tiers, das genau eine Organ-Belegung (`binary_id`) samt System- und Mess-Wahl fest einkompiliert enthält, über buchstabengleiche C-Symbole (die vier Ur-Symbole, die Konzept- und Gattungs-Abfrage und die Versionszeilen) geladen wird und als Bestand 1 im Lager liegt. Alias: Tier, Permutations-Modul.

TLB Translation Lookaside Buffer.

Träger-Stufe (*carrier stage*) Eine der vier Stufen, die die Anatomie tragen, in Bau-Reihenfolge: Planer, CacheEngineBuilder (CEB), Tier-Binary, Hybrid-Tier-Binary (jede Stufe erzeugt die nächste, ohne Tier kein Hybrid); in der Verarbeitungskette sitzt die Hybrid-Tier-Binary als optionaler Mux-Einschub zwischen CEB-Prüf-Dock und Tier-Binary. Keine Stufe durchläuft Schichten — jede enthält ihre Schicht fest einkompiliert; nur die Konfiguration wandert von der Anwender-XML bis zur fertigen Tier-Binary. Jede Stufe führt einen Stempel und bietet drei Interface-Flächen (Abbildung 3.9).

two_phase_valid Gültigkeits-Flag einer Messzeile: Nur eine Messung, deren Workload über das zweiphasige Lastprofil (Aufbau, dann Messphase) gefahren wurde, gilt als valide; Zeilen mit `two_phase_valid` gleich `false` gehen nicht in Median, Matrix oder Diagramm ein.

Unter-Achse (*sub-axis*) Eine Achse unterhalb einer Haupt-Achse, im Regelfall dynamisch und zur Laufzeit vom Planer permutiert: Ihre Werte reisen als Einstellungen über das Prüf-Dock in die Tier-Binary, stehen in keiner Bau-Legende und nie in der `binary_id`. Beispiele: die Laufzeit-Parameter Prefetch-Distanz, Stapelgröße, Inline-Schwelle, Pool-Budget und Thread-Zahl sowie die sechzehn Mess-Kategorien, der Workload und der Arbeitsmodus (Mess-Realm); Optimierungsstufe, Atomik-Fähigkeit, SIMD-Familie, NUMA-Knoten, Seitengröße und Kern-Klasse (System-Realm). Einzelne Unter-Achsen sind übersetzungsstatische Tupel ihrer Haupt-Achse (etwa Scheduling und Compiler) und wirken über die Bau-Version. Registry-Token `sub_axis`. Alias: Sub-Achse. Unter-Achse und Permutations-Achse schließen sich nicht aus.

Verbund-Strategie (*merge strategy*) Die per Achse deklarierte Art, wie ein Prüfling-Slot mit der Engine-Achse verbunden wird: `Verbund1_CeOnly` (nur die Engine, keine Prüflings-Direktive), `Verbund2_Replace` (die Prüfling-Achse ersetzt die Engine-Achse, mit Rückfall; der Standard), `Verbund2_Hybrid` (Engine und Prüfling hybrid je Prüfling) und `Verbund3_Union` (die nicht-redundante Vereinigung); die XML-Token lauten `replace`, `merge` und `union`. Alias: Stufe 1 bis 3 (Altnamen), Full Join (für `union`).

Versions-Lock (*version lock*) Die eingetragene Sperrdatei, die je Quelldatei der Overlay-Quellmenge Kategorie, Achsen-Version und SHA-256 verzeichnet; jede Änderung an einer verzeichneten Datei

verlangt den bewussten Neu-Erzeugungs-Commit, eine Abweichung gegen die Entdeckung ist ein Fehlschlag, und ein CI-Gate prüft sie.

Virus (*virus*) Ein achsenloses Graph- oder Rein-Mathematik-Modul an derselben Wurzel wie die Lebewesen: ohne Achsen-System, ohne Komposition und außerhalb des Permutationsbaums, aber mit eigenem flachem Mess-POD vermessbar (Latenz, Durchsatz, modul-eigene Zähler). Plural: Viren.

Vollprofil, abstraktes Profil (*full profile, abstract profile*) Die zwei Typen der Stand-der-Technik-Profile: Ein Vollprofil beschreibt einen eigenständigen Suchentwurf vollständig; ein abstraktes Profil stellt nur einzelne Organe (OLC als Nebenläufigkeits-Organ [LABN16], LOUDS als Speicher-Layout-Organ [Jac89], VAMPIR als Mess-Technologie [BH23]) und überlässt die übrigen dem Standard-Baukasten — derselbe Typ, dem der Prüfling PRT-ART entspricht. Von den 33 Profilen sind 30 Vollprofile und 3 abstrakte.

working_set_n Die Spalte der Arbeitsmengen-Größe (Anzahl Schlüssel) einer Messreihe; sie ist die Abszisse der Sweep-Kurven des Kurvenladers, und die Zell-Token *n/a* und *failed* sind keine Zahlen — ohne Messwert entsteht kein Stützpunkt.

XML-Lastprofil-Ergebnis (*XML load-profile result*) Das in der Aufgabenstellung (Teilaufgabe 8, Automatisierung der Systemerkennung und Binary-Auswahl) geforderte Persistenz-Format, in dem die für einen Gesamtalgorithmus extrahierte, je Architekturfokus wiederverwendbare Heuristik-Richtlinie unter Referenz der angewendeten Lastprofile abgelegt wird; dient als Eingabe der autonomen Heuristik-Profil-Auswertung. Import-Parser und Export-Writer samt Extraktor sind implementiert (round-trip-fähiges Schema `comdare_load_profile`); Ausblick ist die darauf aufsetzende autonome Heuristik-Auswertung (ML-Klassifikation).

YCSB Yahoo! Cloud Serving Benchmark [CST⁺10].

Zen 5 AMD-Mikroarchitektur (CPUID-Familie 26); die erste Messplattform dieser Arbeit (prod1, AMD Ryzen 9 9950X3D, Granite Ridge, zwei Kern-Komplexe) gehört ihr an, und ihr Roh-Ereignis-Katalog ist durch Messung gegengeprüft. In der System-Registry ist sie die Rekombination `prod1_zen5`. Der Begriff bezeichnet in dieser Arbeit ausschließlich die Mikroarchitektur.

Zwei-Gate-Protokoll (*two-gate protocol*) Das Abnahme-Protokoll je Implementierungs-Paket: Gate 1 ist das Doppellauf-Gate des Engine-Repositories — die volle Test-Suite läuft in vier Zellen (gcc und clang, jeweils als Release- und Debug-Bau) ohne Änderungs-Skip, und akzeptiert wird nur ein Lauf ohne fehlgeschlagene Tests, ohne Ausnahmeliste; Gate 2 ist das Integrations-Gate des Projekt-Repositories (Super-Repository), dessen Pipeline fehlerfrei ohne Ausnahmemenge baut (vgl. Abschnitt 4.4).

D Bausteine-Matrix

Dieser Anhang enthält die vollständige Angebots-Matrix der Bausteine — die `All*`-Listen des Codes —, auf die der Stand der Technik (Abschnitt 3.2) verweist; das je Bau-Konfiguration aktivierte Inventar nennt jede Sektion gesondert. Jede Achse ist dabei ein *Organ* des Lebewesens: Ein konkreter Suchalgorithmus (technisch `SearchAlgorithm`, metaphorisch ein Lebewesen) ist die feste Komposition seiner achtzehn Organ-Achsen (T0–T17), von denen jede einem Organ entspricht; die je Achse aufgeführten Baustein-Varianten sind die austauschbaren Ausprägungen dieses Organs, und ihre Permutation ist im Metapher-Kanon dieser Arbeit der Austausch von Organen am Lebewesen. Die *Anatomie* dieses Lebewesens ist dabei die Verdrahtung *zwischen* den Organen: Achsen-Algorithmen nutzen die Interfaces anderer Achsen (so stellt die Allokations-Achse ein gemeinsames Allokations-Interface für die übrigen Organe bereit). Je Achse sind der Zweck, die *statischen* Unter-Achsen (übersetzungszeitlich wählbare Varianten-Familien) und die *dynamischen* Unter-Achsen (Laufzeit-Parameter) sowie die konkreten Baustein-Varianten mit ihrer Quelle (Paper-Kennung oder Code/Baseline) aufgeführt. Über die achtzehn Kompositions-Slots hinaus führt die Organ-Registry eine optionale Organ-Meta-Meta-Achse¹ `disk_io`, die keinen neunzehnten Kompositions-Slot bildet und nie in die `binary_id` eingeht (Abschnitt D.19). Die Angaben sind aus dem Code der `comdare-cache-engine` und gegen die Achsen-Algorithmus-Paper-Code-Zuordnung des `ce-Repository`s (Datei `docs/architecture/18_achsen_algorithmus_paper_code_map.md`) verifiziert.

Über die achtzehn Organ-Slots hinaus dokumentiert dieser Anhang die beiden übrigen Achsen-Kategorien der Dreiteilung (vgl. Abschnitt 3.4.1). Der *System-Realm* umfasst genau drei Haupt-Achsen in kanonischer, im Code geprüfter Ordnung — Ziel-ISA (`target_isa`), Betriebssystem (`operating_system`) und Erweiterungs-Hardware (`external_utils`) — zuzüglich der äußeren Komplex-Klammer `build_target_complex`, die ausdrücklich keine vierte Haupt-Achse ist; die Reihenfolge ist dabei kein redaktioneller Entschluss, sondern folgt der einen Ordnungs-Quelle des Codes. Der *Mess-Realm* umfasst die Mess-Kategorien, das Mess-Tooling und das Last-Rahmenwerk als Meta-Meta-Achse am Realm-Ende. In der Anatomie-Metapher sind System- und Mess-Achsen kein weiteres Organ und auch nicht das Blut (das flüssige Organ, das die Organ-Achsen verbindet), sondern der *Lebensraum* des Tiers (System-Achsen) und die Ausstattung des *Labors* (Mess-Achsen; Abschnitt 3.3.1); beide sind nicht Teil des Permutationsbaums der `binary_id` (`binary_id=never`) und strikt neutral gegenüber der Binary-Identität; ihre Unter-Achsen werden gleichwohl vom Planer permutiert (`stage=runtime`). Die Angaben beider Sphären sind zusätzlich gegen die drei Achsen-Registries des Codes verifiziert (`cache_engine_axis_registry.xml` für den Organ-Realm, `system_axis_registry.xml` und `measurement_axis_registry.xml`). Drei konservierte Angebots-Kataloge außerhalb des bau-treibenden Angebots — der SIMD-Katalog Build-SIMD unter der Erweiterungs-Hardware sowie der

¹Meta-Meta im Sinne der M3-Ebene der Vier-Ebenen-Schichtung der Meta Object Facility [Obj16]: die Indirektion einer Beschreibung. Meta-Achsen (M2) klassifizieren Haupt- und Unter-Achsen, M1 ist die Konzeption beziehungsweise die Laufzeit-Implementierung, M0 der Binärcode. Die Zuordnung von M3 ist gesetzt; der Schnitt zwischen M2 und M1 ist eine Lesart dieser Arbeit.

Seitentyp-Katalog Build-PG und die Plattform-Hardware-Strategie Build-HW — sind als solche markiert und in Abschnitt D.30 (Konservierte Entwürfe) am Ende dieses Anhangs zusammengefasst.

D.1 T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)

Diese Achse ist das *Such-Organ* (T0) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2). Systematische Permutation von 22 konkreten Suchalgorithmen und Such-Strukturen (S01–S22): 17 Basis-Bausteine — Array-basiert, sortierte Vektoren, Original-Paper-Wrapper, Such-Methoden wie k-ary/ Interpolation sowie geordnete Strukturen wie Skip-Liste/B-Baum — sowie fünf standardmäßig abgeschaltete Opt-in-Bausteine, nämlich die vier per-K-k-ary-Wrapper K2/K4/K8/K16 (S18–S21) und die Swiss-Table-Hash-Struktur (S22), zur messbaren Quantifizierung des Einflusses der Such-Paradigma-Wahl auf Performance-Charakteristiken im einheitlichen `std::map`-Interface. *Registry-Stand*: Das aktivierte Inventar dieser Bau-Konfiguration umfasst vier Bausteine (`k_ary`, `interpolation`, `eytzinger`, `linear_scan`); der Katalog S01–S22 ist das Angebot. Unter-Achsen klassifizieren nach Zugriffsdichte (SA1 dense, SA2 sparse, SA3/SA4 multilevel), ermöglichen Compliance-Nachweise über Paper-Mixins (Originalcode-Nachweis, Anforderung der Betreuung: `is_original`-Markierung pro Funktion/ Wrapper), und identifizieren SIMD-fähige sowie store-traversierbare Varianten.

Statische Unter-Achsen

ID	Name	Beschreibung
SA1	Direkt-adressiert hohe Dichte (Dense Access)	Array-Familie: direkt-adressiert ($O(1)$ lookup), hohe Füllungsdichte ≥ 50 %. Umfasst <code>Array256SearchAlgo</code> (256-Slot compact array, ART Node256 Re-Impl) und <code>Array65535SearchAlgo</code> (uint16-basiert, mid-density 25-50 % direkt-adressiert).
SA2	Sparse/Geordnet (Sparse Access)	Sortierte Vektoren, geordnete Such-Methoden und tree/hash-Strukturen: niedrige Füllungsdichte, Range-Scan-Support, variable $O(\log n)$ - $O(1)$ Lookup. Umfasst <code>VectorU8U8SearchAlgo</code> , Original-Paper-Wrapper (ART/HOT/START/Wormhole/SuRF), Such-Methoden (k-ary, Interpolation, Eytzinger, Hash, Swiss-Table) sowie Strukturen (SkipList, BST, B-Baum).
SA3	Multi-Byte-Diskriminatoren (Multilevel Access)	Mehrbyteige Such-Strukturen mit indizierten Sub-Diskriminatoren (z.B. START mit Cost-DP-Mehrebenen). <code>VectorU16U16SearchAlgo</code> (u16-Keys, sorted lower_bound) und <code>OriginalStartSearchAlgo</code> (Veröffentlichung P05) gehören hier.
SA4	Direkt-adressiert mittlere Dichte (Direct Multibyte)	Teilt die Direkt-Adressierungs-Zugriffsmuster (wie SA1) mit Multi-Byte-Granularität (uint16-Größe) und mittlerer Füllungsdichte 25–50 % (im Gegensatz zu SA1 ≥ 50 %). Beispiel: prt-art <code>Array65535</code> . Diese Unter-Achse erfasst die Dichte-Variante, ohne die SA1-Abstraktion zu brechen.

Bausteine

Variante	Beschreibung	Quelle
Array256SearchAlgo (S01)	ART Node256 direkt-adressierte Re-Implementierung: <code>std::array<std::optional<u64>, 256></code> mit $O(1)$ Lookup/Insert/Erase. Cache-Line-aligned, SIMD-fähig, kein Heap-Allocation. Baseline für dichte Zugriffe.	Code/Baseline (Leis ICDE 2013 ART, Veröffentlichung P01)
VectorU8U8SearchAlgo (S02)	Sparse sortiertes Key-Value-Vektor-Paar: <code>std::vector<u8> keys + std::vector<u64> values</code> mit <code>std::lower_bound</code> . $O(\log n)$ Lookup, $O(n)$ Insert/Erase. Konzept-äquivalent zu HOT-k-Constrained. SIMD-fähig (Bitmask-Scan).	Code/Baseline (Binna SIGMOD 2018 HOT, Veröffentlichung P02)
VectorU16U16SearchAlgo (S03)	Sparse sortiertes u16-Vektor-Paar: <code>std::vector<u16> keys + std::vector<u64> values</code> . Vereinfachtes START-Konzept (Multi-Byte-Diskriminatoren ohne Cost-DP). $O(\log n)$ Lookup, $O(n)$ Insert/Erase.	Code/Baseline (Fent ICDEW 2020 START, Veröffentlichung P05)
Array65535SearchAlgo (S09)	Mid-Density direkt-adressiert uint16-basiert (25–50 % Füllungsdichte). Aus prt-art migriert. $O(1)$ Lookup, aber mit Speicher-Overhead vs. SA1 densen Arrays.	Code/Baseline (aus prt-art migriert)
OriginalArtSearchAlgo (S04)	ART Paper-Bindung (<code>unodb::db::is_original_module()=true</code> (4/4 Funktionen original). Paper-Mixin via SHA256-validierte Tool-Codegenerierung. Originalcode-Nachweis (Anforderung der Betreuung): <code>get_compiler()= 'gcc-9.5'</code> , all insert/lookup/erase/clear marked original.	P01 (Leis/Kemper/Neumann ICDE 2013, Apache-2.0 unodb)
OriginalHotSearchAlgo (S05)	HOT Paper-Bindung (HOTRowex): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, aber erase/clear Lücken (HotRowex ist append-only; Re-Impl fallback). Veröffentlichung P02.	P02 (Binna et al. SIGMOD 2018, ISC)
OriginalStartSearchAlgo (S06)	START Paper-Bindung (sosd-Adapter): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, erase/clear Lücken (kein remove im sosd-Adapter). Veröffentlichung P05, Multi-Byte Diskriminatoren.	P05 (Fent/Jungmair/Kipf/Neumann ICDEW 2020, MIT)
OriginalWormholeSearchAlgo (S07)	Wormhole Paper-Bindung: <code>is_original_module()=false</code> (3/4 original). put/get/del original, clear Lücke. Hybrid Hash+Trie+B+ Index. GPL-3.0-lizenziert (blockiert Original-Code-Linking für Thesis).	P07 (Wu/Ni/Jiang EuroSys 2019, GPL-3.0)
OriginalSurfSearchAlgo (S08)	SuRF Paper-Bindung (FST mit LOUDS-Bitmap): <code>is_original_module()=false</code> (1/4 original, nur lookup). Succinct Range Filter, Multi-Header-Bibliothek. Veröffentlichung P10.	P10 (Zhang/Lim/Andersen SIGMOD 2018, Apache-2.0)
KArySearchAlgo (S10)	k-ary Search Such- <i>Methode</i> (Schlegel/Gemulla/Lehner Da MoN 2009): K-Wege-Partition statt Halbierung bei der Binärsuche. K ist <code>iterable_aspect</code> ($K \in \{2,4,8,16\}$). $O(\lceil \log_{K+1} n \rceil)$ Iterationen, parallele ILP-Vergleiche. Array-Familie aber kein treues dediziertes Traversal-Organ (SA2-Klassifikation). C++23-Re-Impl, <code>is_original=false</code> .	Schlegel et al. DaMoN 2009 [SGL09]
InterpolationSearchAlgo (S11)	Interpolationssuche Such- <i>Methode</i> (Perl/Itai/Avni CACM 1978): verteilungsbewusste Positionsschätzung statt blinder Halbierung. $O(\log \log N)$ avg bei Gleichverteilung, $O(N)$ worst-case. Nicht SIMD-vektorisierbar (datenabhängiger Index). C++23-Re-Impl.	Perl/Itai/Avni CACM 1978 [PIA78]
EytzingerSearchAlgo (S12)	Eytzinger/BFS-Layout branch-free Suche (Khuong/Morin JEA 2017): Cache-Layout-Such- <i>Methode</i> mit branch-free Kern. Keys in BFS-Ordnung des impliziten Suchbaums. Für große Arrays laut Veröffentlichung das schnellste Allzweck-Layout. C++23-Re-Impl (Experiment-Harness ohne Standard-Lizenz).	Khuong/Morin JEA 2017 [KM17]

Variante	Beschreibung	Quelle
SkipListSearchAlgo (S13)	Skip-Liste probabilistische geordnete <i>Struktur</i> (Pugh CACM 1990): $O(\log n)$ erwartet search/insert/erase ohne explizites Rebalancing. Jeder Knoten hat 1..kMaxLevel Forward-Zeiger (Münzwurf Probability 0.5). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, value-semantisch). Lehrbuch-Algorithmus, C++23-Re-Impl.	Pugh CACM 1990 [Pug90]
HashSearchAlgo (S14)	Open-Addressing-Hashtabelle <i>ungeordnet</i> (Knuth TAOCP 3 §6.4): Fibonacci-Hash, Linear Probing, Tombstone-Erase bei Resize. $O(1)$ erwartet lookup/insert bei load_factor < 0.7. Keine Range-Scan-Unterstützung (distinkte Klassifikation). Korrekte Tombstone-Markierung zur Probe-Ketten-Integrität. C++23-Re-Impl.	Knuth TAOCP 3 §6.4 [Knu98]
LinearScanSearchAlgo (S15)	Unsortierter linearer Scan Baseline (Leis ICDE 2013 ART Node4): $O(n)$ lookup auf unsortiertem KV-Array. Einfachste Vergleichs-Nullpunkt-Struktur: insert $O(1)$ -amortisiert, erase $O(1)$ via swap-and-pop. Store-traversierbar (Array-Familie). Keine Range-Scan-Unterstützung.	Code/Baseline (Leis ICDE 2013 ART, Veröffentlichung P01, Node4)
BinarySearchTreeSearchAlgo (S16)	Unbalancierter Binärer Suchbaum (Knuth TAOCP 3 §6.2.2 Hibbard-Deletion): deterministische Vergleichs-Baum-Baseline (vs. SkipList probabilistisch). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, Free-List). Hibbard-erase (3 Fälle: Blatt/1 Kind/2 Kinder via In-Order-Nachfolger). $O(n)$ worst-case bei sortiertem Input.	Knuth TAOCP 3 §6.2.2 [Knu98]
BTreeSearchAlgo (S17)	Balancierter Block-orientierter Mehrwege-B-Baum (Bayer/McCreight Acta Inf. 1972 / CLRS Kap. 18): $t=4$, daher [3..7] Keys pro Knoten, bis zu 8 Kinder. Immer balanciert ($O(\log n)$ guaranteed). Index-basiert, <code>alignas(64)</code> -Knoten (Cache-Line-Alignment). Top-Down Splitten beim Insert, Borrow/Merge beim Erase. Die Standardstruktur für Datenbankindizes.	Bayer/McCreight Acta Inf. 1972 / CLRS [BM72]

Bausteine (Opt-in, standardmäßig abgeschaltet)

Die folgenden fünf Bausteine sind im Katalog registriert und konform, in der Bau-Konfiguration aber standardmäßig abgeschaltet; die Registrierung baut nichts, sie macht die Varianten selektierbar.

Variante	Beschreibung	Quelle
KArySearchAlgoK2 (S18)	Übersetzungszeit-K-Wrapper der k-ary-Suche mit $K=2$ (Binärsuch-Baseline): je K eine eigene Tier-Binary (<code>KArySearchAlgoK<N> → KAryTraversal<K></code>) mit eigenem Aktivierungs-Flag (Default OFF) und eigenem Namen <code>k_ary_k2</code> (Trennung in der <code>binary_id</code>).	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
KArySearchAlgoK4 (S19)	$K=4$ (Paper-Default, 5-Wege-Partition); Name <code>k_ary_k4</code> , sonst wie S18.	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
KArySearchAlgoK8 (S20)	$K=8$; Name <code>k_ary_k8</code> , sonst wie S18.	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)
KArySearchAlgoK16 (S21)	$K=16$; Name <code>k_ary_k16</code> , sonst wie S18.	Schlegel et al. DaMoN 2009 [SGL09] (Code/Baseline)

Variante	Beschreibung	Quelle
SwissTableSearchAlgo (S22)	CE-native Re-Implementierung des Swiss-Table-Prinzips: offene Adressierung mit einem Control-Byte je Slot (leer, gelöscht, 7-Bit-H2-Fingerprint), Gruppen von 16 Slots, H1/H2-Split und Tombstones für erase; ungeordnet, kein Range-Scan (SA2). Allokation ausschließlich über die Allokator-Achse (vier Puffer in einem Allokations-Buch). Standardmäßig abgeschaltet und ohne Organ-Backing nicht als Swiss-Table-Messung zu lesen; <code>is_original=false</code> .	Google Abseil Swiss Tables [Goo18] (Vortrag CppCon 2017); C++23-Re-Impl

Beitragende Quellen: P01, P02, P05, P07, P10, k-ary (Schlegel et al. 2009), Interpolation (Perl et al. 1978), Eytzinger (Khuong/Morin 2017), Skip-List (Pugh 1990), Hashing/BST (Knuth TAOCP 3), B-Baum (Bayer/McCreight 1972), Swiss Tables (Google/Abseil 2018) [LKN13, BZP⁺18, FJKN20, WNJ19, ZLL⁺18, SGL09, PIA78, KM17, Pug90, Knu98, BM72, Goo18].

D.2 T1 Cache-Speicher-Traversierung (Fanout-Lookup)

Diese Achse ist das *Cache-Traversierungs-Organ* (T1) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Die Achse `axis_03b` definiert drei distinkte Strategien zur Schlüssel-Auflösung in kleinen Fanout-Arrays (typisch: innere B+-Tree- oder Trie-Knoten). Sie spannt ein Spektrum von linearer Suche ($O(N)$, cache-freundlich, klein), über multiplikatives Hashing mit Open-Addressing ($O(1)$ amortisiert, Knuth-Fibonacci), bis zur sortierten Binärsuche ($O(\log N)$, geordnete Traversierung, B+-Tree-Standard). Alle drei sind Lehrbuch-Baselines ohne Original-Code-Linking, implementiert als reine C++23-Re-Implementierungen.

Statische Unter-Achsen

ID	Name	Beschreibung
CT1	Lineare Zugriffe	Linear-scan-basierte Lookup-Methoden auf kleinen, unsortierten Fanout-Arrays. Charakterisiert durch $O(N)$ -Auflösung mit einfachen Vergleichen, typisch für Knoten mit Fanout < 32 in B+-Trees. Unter-Achse für <code>LinearFanout</code> und <code>BinarySearchFanout</code> (beide: <code>axis_tag = linear_access_tag</code>).
CT2	Hash-basierte Zugriffe	Hash-Lookup mit Open-Addressing und linearem Probing. $O(1)$ amortisierte Auflösung, charakterisiert durch Fibonacci-Multiplikator ($11400714819323198485 \approx 2^{64}/\varphi$), Power-of-2 Bucket-Count, dynamisches Rehashing bei Load-Factor > 0.7 . Nur Hash Lookup-Wrapper. Unter-Achse: <code>axis_tag = hash_access_tag</code> .

Bausteine

Variante	Beschreibung	Quelle
LinearFanout	Unsortierte Fanout-Traversal via <code>std::find_if</code> und linearem Scan. $O(N)$ Lookup, kein Hash-Overhead, cache-freundlich für kleine N (< 32). Klassische B-Tree-Baseline (Bayer/McCreight 1972). Gehört zu Unter-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)
HashLookup	Fibonacci-Hash open-addressing mit linearem Probing (Knuth TAOCP Vol. 3, §6.4). $O(1)$ amortisiert, dynamisches Rehashing (verdoppelt Capacity bei $\text{load} \geq 0.7$), iterable Laufzeit-Kapazität (<code>kIterableInitialCapacities: {8, 16, 64, 256, 1024}</code>). Standalone C++23-Re-Impl, keine Original-Code-Bindung. Gehört zu Unter-Achse CT2.	Code/Baseline — Knuth 1998 (The Art of Computer Programming Vol.3)
BinarySearch Fanout	Sortierte Fanout-Traversal via <code>std::lower_bound</code> . $O(\log N)$ Lookup bei sortierter Ordnung, bewahrt Schlüssel-Sequenzierung für Range-fähige Routing-Layer. Klassische Suche in inneren B+-Tree-Knoten (Bayer/McCreight 1972), dritte distinkte Strategie der Achse. C++23-Re-Impl, <code>is_original=false</code> . Gehört zu Unter-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)

Beitragende Quellen: Bayer/McCreight 1972, Knuth TAOCP Vol. 3 [BM72, Knu98].

Verwandte Literatur: P17, P21, P32 [BDFC05, CGM01, SKK⁺25].

D.3 T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)

Diese Achse ist das *Mapping-Organ* (T2) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Vermittlung zwischen logischen Slot-Indizes und physischen Speicher-Offsets in inneren Trie-Knoten und Speicher-Pools. Die Achse bietet zwei grundsätzlich verschiedene Zuordnungsstrategien: direkte absolute Offsets (lineare Packing) oder pool-relative Offsets mit automatischem Rebase-Mechanismus zur Unterstützung von Speicher-Umallokation und Persistent-Data-Structures.

Statische Unter-Achsen

ID	Name	Beschreibung
MP1	Direct-Access-Mapping (MP1)	Direkter Slot-zu-Absolute-Offset-Mapping ohne Indirection. Wrapper-Familie für klassisches Array-of-Pointers-Layout wie in inneren B-Tree-Knoten (Bayer/McCreight 1972). Einfache Cache-Locality, $O(N)$ Lookup per <code>std::find_if</code> .
MP2	Pool-Relative-Mapping (MP2)	Slot-zu-Pool-Relativem-Offset-Mapping mit Based-Pointer-Arena-Pattern. Alle Offsets relativ zu dynamischem <code>pool_base_address</code> gespeichert. Ermöglicht $O(1)$ Rebase bei Speicher-Umallokation — zentrale Eigenschaft für Persistent-Data-Structures (Driscoll et al. 1989) und CustomAligned Structure-Pattern in prt-art.

Bausteine

Variante	Beschreibung	Quelle
DirectPlacement	Wrapper für direktes Slot-zu-Absolute-Offset-Mapping (lineares Packing). <code>std::vector<pair<slot_index_type, offset_type>;</code> <code>resolve_offset/register_slot</code> via <code>linear</code> <code>std::find_if</code> . Verwendungskontext: prt-art <code>Inode::placement_page_ +</code> Mapping-Table. Speicher: 16-bit Slot-Index + <code>size_t</code> Offset pro Entry. Keine Pool-Basis-Abhängigkeit.	CE-Baseline
PoolRelative	Wrapper für Pool-Relatives-Offset-Mapping mit $O(1)$ -Rebase-Fähigkeit. <code>std::vector<pair<slot_index_type, relative_offset></code> + <code>mutable pool_base_address</code> . Bei Pool-Umallokation: <code>rebase(new_pool_base)</code> setzt nur <code>pool_base_neu</code> — alle relativen Offsets bleiben gültig. Verwendet bei prt-art CustomAligned Structure + Versioning-Szenarien (Persistent Data Structures nach Driscoll/ Sarnak/Sleator/Tarjan 1989).	CE-Baseline

Beitragende Quellen: — (CE-Baseline, eigene Wrapper).

Verwandte Literatur: Bayer/McCreight 1972, Driscoll et al. 1989, P03, P07 [BM72, DSST89, MKM12, WNJ19].

D.4 T3 Pfadkompression

Diese Achse ist das *Pfadkompressions-Organ* (T3) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Definiert die Kompressions-Granularität und -Strategie für Schlüsselpfade in Trie-Strukturen. Steuert, wie innere Knoten redundante Byte-Sequenzen oder signifikante Bits gemeinsamer Präfixe speichern und navigieren, um Speicher- und Cache-Effizienz zu optimieren.

Statische Unter-Achsen

ID	Name	Beschreibung
PC1	Kompressions-Granularität	Definiert die minimale Einheit der Pfad-Kompression: Bit-weise (Patricia), Byte-weise (ART) oder keine Kompression (raw path). Bestimmt die Trie-Höhe und navigationsoptimale Codeblöcke im Traversal. Code-Token: <code>granularity_tag</code> .
PC2	Sprung-Strategie	Definiert das Navigations-Pattern bei inneren Knoten: Linear über 8 Bytes (ByteWise), Single-Bit-Vergleich (Patricia) oder direkter Sprung ohne Zwischenschritte. Beeinflusst Branch-Dichte und Prefetch-Verhalten. Code-Token: <code>skip_strategy_tag</code> .
PC3	Decode-Komplexität	Klassifiziert die Laufzeit-Komplexität beim Extrahieren von Präfix-Informationen: $O(1)$ für direkte Extraktion, $O(\log n)$ für struktur-basierte Suche, $O(n)$ für sequenzielle Prüfung. Wichtig für Operationen wie <code>common_prefix_len()</code> und Schlüssel-Vergleiche. Code-Token: <code>decode_complexity_tag</code> .

Bausteine

Variante	Beschreibung	Quelle
PathCompression None	Keine Pfad-Kompression; rohe Schlüsselpfade werden vollständig und unkomprimiert in Knoten gespeichert. Dient als Mess-Baseline mit <code>compression_ratio=1.0</code> . Verwendet als Kontrollfall zur Quantifizierung der Speicher- und Höhen-Einsparungen komprimierender Strategien.	Code/Baseline
ByteWisePath Compression	Byte-für-Byte-Kompression (ART-Stil); speichert bis zu 7 Präfix-Bytes pro Knoten in einem kompakten <code>u64</code> -Wort mit Längenetikett im High-Byte. Reduktion der Trie-Höhe um typischerweise Faktor 2 (<code>compression_ratio=0.5</code>). Ursprung: Leis et al., ART ICDE 2013 (P01). Verwendet Goldstandard-Operationen <code>common_prefix_len()</code> und <code>cut()</code> für Navigations-Optimierung.	P01
PatriciaPath Compression	Single-Bit-Split-Pfadkompression (Patricia-Trie); navigiert bitweise statt byteweise, reduziert Trie-Höhe um typischerweise Faktor 3 (<code>compression_ratio=0.3</code>). Ursprung: Morrison PATRICIA JACM 1968; modern implementiert in HOT (Binna SIGMOD 2018, P02) und Wormhole (Wu EuroSys 2019, P07). F15-operative Primitiven: <code>key_split_bit()</code> für MSB-first Bit-Extraktion, <code>path_descend_scan()</code> für goldstandard-konforme <code>_scan</code> -Signatur mit schlüssel-abhängiger variabler Tiefe.	P02

Beitragende Quellen: P01, P02, P07, Morrison 1968 [LKN13, BZP⁺18, WNJ19, Mor68].

Verwandte Literatur: P04, P05, P09 [BFTV24, FJKN20, Jac89].

D.5 T4 Knotentypen (Node-Type-Achse)

Diese Achse ist das *Knotentyp-Organ* (T4) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Die Achse `axis_04_node_type` definiert die vier Knotenformate des Adaptive Radix Tree (ART) als austauschbare Varianten: `Node4`, `Node16`, `Node48` und `Node256`. Jeder Knotentyp bietet eine andere Balance zwischen Speicher, Cache-Effizienz und Suchgeschwindigkeit. Diese Achse bildet die Basis für adaptive Baum-Reorganisationen und ist zentral für das Design der Suchalgorithmen.

Statische Unter-Achsen

ID	Name	Beschreibung
NT1	Kapazitätsklasse (NT1)	Klassifizierung nach Fanout-Größe und Slotanzahl: klein (<code>Node4</code> mit 4 Slots, linear search), mittel (<code>Node16</code> mit 16 Slots, SIMD-binary-search) bis maximal (<code>Node256</code> mit 256 Slots, direct-addressed).
NT2	Zugriffsmuster (NT2)	Art der Schlüsselsuche innerhalb eines Knotens: linearer Scan (<code>Node4</code>), binäre Suche mit SIMD (<code>Node16</code>), indirekte Indizierung (<code>Node48</code>) oder direkte Adressierung (<code>Node256</code>).
NT3	Kompaktheit (NT3)	Speicherlayout und Dichte: spärlich und cache-aligned (<code>Node4</code>), dicht mit Indirection (<code>Node48</code>) oder maximal direkt (<code>Node256</code>), mit Auswirkungen auf Speicherverbrauch und Cache-Line-Misses.

Bausteine

Die Tafel führt drei Gegenstände aus demselben Achsen-Verzeichnis; nur der erste Block ist die Wertemenge der Achse.

Variante	Beschreibung	Quelle
<i>Achsen-Varianten (AllNodeTypes, vier)</i>		
Node4NodeType	Kleiner ART-Knoten mit 4 Slots und linearer Suche. Ideal für spärlich besetzte obere Bauebenen, passt komplett in eine Cache-Line (64 Byte). Implementiert als CE-Reimplementierung der ART-Spezifikation mit <code>node_find_scan()</code> als linear scan über sortierte Schlüssel.	P01
Node16NodeType	Mittlerer ART-Knoten mit 16 Slots, optimiert für SIMD-basierte binäre Suche (SSE2 PCMPSTRM / AVX2 VPCMPSTRM). Balance zwischen Speicher und Suchgeschwindigkeit; größte Kapazität ohne Indirection. CE-Re-Impl der Leis-ICDE-2013-Spezifikation.	P01
Node48NodeType	Großer ART-Knoten mit 48 Slots, nutzt Indirection: 256-Byte ChildIndex (Byte→Slot-Mapping, 0=leer) plus 48er ChildArray. Trade-off zwischen Speicher (vs Node256 direct) und Cache-Misses (vs Node4 linear). Doppelter Indirektions-Zugriffsmuster (Index + Child).	P01
Node256NodeType	Maximaler ART-Knoten mit 256 Slots, direkt adressiert ohne Index. O(1) Lookup (kein Scan, kein Index-Touch). Hoher Speicherverbrauch (256 Pointer), aber optimale Suchgeschwindigkeit für dicht besetzte Knoten. Default für untere Bauebenen.	P01
<i>Paper-Knotenformate (Rekonstruktions-Ziele dieser Arbeit, keine CE-Baseline)</i>		
Internal/Border	Knotentyp-Paar des Masstree (Trie aus B ⁺ -Bäumen): Internal-Knoten navigieren die 8-Byte-Schlüssel-Slices der B ⁺ -Ebene, Border-Knoten bilden die Blatt-Ebene mit den Werten bzw. dem Übergang zur nächsten Trie-Schicht. Paper-Knotenformat (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P03 (Mao et al. 2012) [MKM12]
Decision/Span	Knotentyp-Paar des B ² -Baums (B ⁺ mit eingebetteten binären In-Knoten, cache-bewusst): Decision-Knoten enthalten die binäre Verzweigungs-Entscheidung innerhalb der Seite, Span-Knoten fassen zusammenhängende Schlüsselbereiche zusammen. Paper-Knotenformat (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P06 (Schmeisser et al. 2022) [SSL ⁺ 21]
LOUDS-Dense/Sparse	Knotentyp-Paar des succinct Range-Filters SuRF: LOUDS-Dense kodiert die oberen, dicht besetzten Trie-Ebenen bitmap-basiert mit direkter Adressierung, LOUDS-Sparse die unteren, spärlich besetzten Ebenen byte-sequenziell succinct. Paper-Knotenformat (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P10 (Zhang et al. 2018) [ZLL ⁺ 18]
<i>Speicher-Organ der Komposition (Storage-Organ, keine node_type-Werte)</i>		
NodeTypeSlotStore	Storage-Organ: <code>node_type</code> -getriebener boundierter Slot-Speicher. Erstreckt sich über <code>std::array</code> mit Kapazität <code>N::max_capacity()</code> über einen gemeinsamen uint64-Key. Erfüllt das StorageOrgan-Concept und ist Drop-in-Substrat für ComposedSearch. Baseline-Variante ohne Layout/Allocator-Integration.	Code/Baseline
ComposedStore	3-Achsen-Storage-Organ: <code>node_type(N) ⊕ layout(L) ⊕ allocator(A)</code> . Unboundierter vector über Allocator A; layout fließt als statisches <code>cache_line_size</code> und via optionale <code>organ_scan_field_sum()</code> ein. Messbar (Anforderung F15).	Code/Baseline
NodeChunkedStore	3-Achsen-Storage-Organ mit Laufzeitwirkung der <code>node_type</code> -Achse: Slots liegen in Chunks der Kapazität <code>N::max_capacity()</code> . Chunk-Count = Zahl der node-großen Allokationen; damit ist die <code>node_type</code> -Achse messbar (Node4 viele Chunks, Node256 wenige). Beleg für Delegation.	Code/Baseline
LayoutAware ChunkedStore	Layoutgetreue Variante von NodeChunkedStore: speichert Records am layout-getriebenen Stride (<code>eff_stride</code>). Jeder Record belegt <code>eff_stride</code> Bytes (Key 0..8, Value 8..16, Rest Padding). Macht die <code>memory_layout</code> -Achse messbar, OOB-sicher für scan-Methoden.	Code/Baseline
ObservableNodeType	ObservableAxis-Hülle um eine <code>node_type</code> -Strategie. Reicht die statische API durch (<code>max_capacity/name/topic_tag</code>) + bietet <code>observe_node_find()</code> als Instanz-Driver. Trackt <code>find_count</code> , <code>keys_stored</code> , <code>queries_run</code> , <code>last_checksum</code> . Format-divergente Lookup-Latenz messbar (Out-of-Process-Pfad).	Code/Baseline

Beitragende Quellen: P01 [LKN13], P03 [MKM12], P06 [SSL⁺21], P10 [ZLL⁺18].

D.6 T5 Speicher-Layout (Memory Layout)

Diese Achse ist das *Speicher-Layout-Organ* (T5) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Diese Achse klassifiziert die räumliche Organisation von Datensätzen im Speicher und beeinflusst damit Cache-Effizienz, SIMD-Vectorisierbarkeit und False-Sharing-Charakteristiken. Sie erfasst orthogonal vier Design-Dimensionen, von denen drei als Unter-Achsen-Tags im Code stehen (HM1–HM3): (1) Alignment-Strategien (Cache-Line-Padding vs. Strict-Packing), (2) Datenorganisation (zeilenorientiert AoS vs. spaltenorientiert SoA vs. Hybrid AoSoA), (3) Packing-Dichte (dense vs. succinct bit-packed); die vierte, das Stride-Muster (HM4, sequenziell vs. interleaved), ist eine geplante Dimension ohne tragenden Layout-Baustein, für die der Code keinen Tag führt (ein permutierbarer Stride-Wert brähe die auf fünf Layout-Bausteine festgelegte *AllLayouts*-Liste des 320er-Katalogs). Die Layout-Wahl ist ein primärer Treiber für Cache-Lokalität und SIMD-Fähigkeit bei Index-Traversal.

Statische Unter-Achsen

ID	Name	Beschreibung
HM1	Alignment-Strategie	Entscheidung zwischen Cache-Line-gepaddetem Layout (False-Sharing-Vermeidung) und Strict-Packing (Speicherkompaktheit). Bestimmt Stride-Overhead bei sequenzieller Traversal.
HM2	Daten-Organisation	Grundlegende Speicher-Ordnung: Array-of-Structs (AoS, zeilenorientiert), Struct-of-Arrays (SoA, spaltenorientiert), oder Hybrid AoSoA (Block-basierte Mischung). Kritisch für Cache-Line-Ausnutzung bei Feld-selektiven Scans.
HM3	Packing-Dichte	Dichtegrad der Speichernutzung: von Standard-Records (8–64 Byte) bis zu kompaktem Bit-Packing (z.B. LOUDS-Bitmaps mit ~1 Bit pro Slot). Beeinflusst Decode-Latenz und L1/L2-Auslastung.
HM4	Stride-Muster (geplant, kein Code-Tag)	Muster der Speicheradressierung bei Traversal: Sequenzielle Strides (aufeinanderfolgende Records), Interleaved (z.B. um Hot-Cold-Daten zu trennen) oder Cache-Aware-Layouts (z.B. Eytzinger BFS). Optimiert Prefetch-Effizienz. Im Code ohne tragenden Layout-Baustein und ohne Tag: Ein permutierbarer Stride-Wert brähe die auf fünf Layout-Bausteine festgelegte <i>AllLayouts</i> -Liste des 320er-Katalogs.

Dynamische Unter-Achsen

ID	Parameter	Beschreibung
CacheLineSize	Cache-Line-Größe (Dimension Line-Größe der <i>cacheline</i> -Unter-Achse)	Parameter des Planers für die Zielplattform-Cache-Zeile: 32 Byte (embedded), 64 Byte (Standard x86-64/ARM64), 128 Byte (unter anderem manche Power-ISAs) oder 256 Byte (zum Beispiel s390x). Der Wertebereich der <i>cacheline</i> -Unter-Achse umfasst 4 Line-Größen $\times$ 3 Alignment-Werte $\times$ 5 Prefetch-Hinweis-Werte = 60 Konfigurationen; die Wahl wird je Bau als <i>NTTP CacheLineConfig</i> eingesetzt und beeinflusst Alignment/Padding-Overhead aller alignierten Layouts.

Bausteine

Variante	Beschreibung	Quelle
AoSStrictMemoryLayout	Array-of-Structs, kompakt gepackt ohne Cache-Line-Alignment. Vorteil: minimale Speichernutzung (record_size-Strides, kompakt). Nachteil: False-Sharing bei concurrent Writes, schlechte Cache-Hit-Quote bei schmalen Feldscans. Typisch für Wormhole-Index.	Code/Baseline
CacheLineAlignedMemoryLayout	AoS mit 64-Byte-Alignment per Record. Vermeidet False-Sharing durch Padding (Stride = round_up(record_size, 64)). Standard für ART/HOT/Masstree/START. Bau-Konstante: Bei record_size=48 (Mess-Build) entsteht 16 Byte Padding pro Record; eine messbar höhere Latenz als bei aos_strict ist der erwartete Cache-Effekt (Messbeleg im Korpus: n/a, die vollständige Messreihe liegt nicht vor).	Code/Baseline
SoAMemoryLayout	Struct-of-Arrays: Alle Felder spaltenweise hintereinander. Vorteil: Kontiguierliche Feldscans (z.B. nur Keys ohne Values) nutzen maximal Cache-Lines (16× weniger Cache-Lines als AoS bei Einfeld-Scan). Ideal für SIMD-Vectorization und OLAP-Indizes. Vorteil beschrieben in Abadi et al. SIGMOD 2008.	Abadi et al. 2008 [AMH08]
PackedBitmapMemoryLayout	Bit-packed succinct Layout (LOUDS / SuRF FST). Speichert n Einträge mit $\sim n \cdot \lg(\sigma)$ Bits statt bytes. Extreme Kompaktheit (32× weniger Cache-Lines als AoS), aber höherer Decode-Overhead per Lookup. Fundament: Jacobson LOUDS FOCS 1989 (P09). Moderne Impl: SuRF Apache-2.0.	P09 (Jacobson LOUDS FOCS 1989)
AoSoAMemoryLayout	Array-of-Structures-of-Arrays: Hybrid-Layout mit Block-Breite W=8 (AVX2-Lanes). Innerhalb eines Blocks: SoA (Felder kontiguierlich). Über Blocks: AoS (Blocks sind gestrided). Vereint SIMD-Vectorisierbarkeit (SoA innerhalb mit räumlicher Lokalität (AoS über). Standard in HPC/SIMD-Engines. Konvention; kein einzelnes Ursprungspaper, aber Idiom des SPMD-Übersetzers ispc [PM12].	Code/Baseline (HPC/SIMD-Konvention; ispc [PM12])

Beitragende Quellen: P09, Abadi et al. 2008, ispc (Pharr/Mark 2012) [Jac89, AMH08, PM12].

Verwandte Literatur: P06, P11, P12, P32 [SSL⁺21, RR99, RR00, SKK⁺25].

D.7 T6 Allokator-Achse (T6)

Diese Achse ist das *Allokations-Organ* (T6) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Diese Achse stellt die Speicherverwaltungsstrategie für die Cache-Engine bereit. Sie definiert 26 konkrete Allocator-Wrapper für verschiedene Algorithmen und Techniken (von klassischen Baselines wie Buddy und Slab über moderne Research-Allokatoren wie `mimalloc`, `jemalloc`, `snmalloc` bis zu spezialisierten Varianten für NUMA, PIM und formal verifizierte Systeme sowie einer Speichertyp-Virtualisierung nach dem VAMPIR-Entwurf, A24). Alle Wrapper erfüllen standardisierte Concepts und sind orthogonal nach 7 Compile-Time-Varianten-Familien (Freelist-Topologie, Size-Class-Schema, Thread-Lokalität, Synchronisation, Allokationspolicy, Reklamation, Fragmentierungsstrategie) strukturierbar.

Statische Unter-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Wie sind freie Speicherblöcke organisiert? Beispiele: Per-Page-Sharding (mimalloc A04), Per-Thread-Caching (snmalloc A07), Buddy-Tree (A19), Single-Global-Free-List (dlmalloc A20), Spans (scalloc A08), hybride Strategien (hmalloc A15).
AA2	Size-Class-Schema	Wie werden Block-Größen klassifiziert? Beispiele: Slab pro Objektgröße (A02), dlmalloc Bins (A20), jemalloc Größenklassen (A05), scalloc Spans (A08), Buddy Power-of-2 (A19), formal verifizierte Bins (StarMalloc A13), std::pmr pools (A22).
AA3	Thread-Lokalität	Wie lokal pro Thread/Core ist die Allokation? Beispiele: tcmalloc thread-local-caches (A06), rpmalloc thread-heaps (A10), snmalloc message-passing (A07), tcmalloc-warehouse hyperscale (A14), Hoard per-heap-locks (A01).
AA4	Synchronisation	Welcher Concurrency-Mechanismus? Beispiele: Lock-Free CAS (Michael A03, LRMalloc A11), Per-Heap-Locks (Hoard A01), Single-Threaded (Exgen A18), Wait-Free (Crystalline A17).
AA5	Allokationspolicy	Wie werden Allokationsanfragen geroutet? Beispiele: NUMA-Origin-Awareness (NUMAAlloc A09), Cache-Set-Awareness (CAMA A12), NUCA-Awareness (TCMalloc-Warehouse A14), PIM-Awareness (PIM-malloc A16), polymorphe Memory-Resource (std::pmr A22), NFP-Speichertyp-Virtualisierung (VampirNfp A24).
AA6	Reklamation	Wie werden gelöschte Blöcke zurückgegeben? Beispiele: Magazine-Cache (Slab A02), Page-Heap-Release (TCMalloc A06), Lock-Free-Reclamation (Crystalline A17), Deferred-Free (mimalloc A04), Vmem-Magazine-Extension (A23).
AA7	Fragmentierungsstrategie	Wie wird Fragmentierung vermieden? Beispiele: dlmalloc Coalescing (A20), Object-Coloring (Slab A02), Buddy-Splitting (A19), Page-Local-Sharding (mimalloc A04), Span-Pooling (scalloc A08).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierbarkeit. Berger et al. (ASPLOS 2000). Synchronisierungskonzept via Per-Heap-Locks + lokale Heap-Caches pro Thread.	P— (OSS, Apache-2.0)
SlabAllocator (A02)	Object-Caching-Kernel-Allokator mit Magazine-Cache-Strategie. Bonwick (USENIX 1994, 2001). Klassisches Lehrbuch-Paradigma ohne eigenständigen OSS-Code.	P— (Baseline/Textbook)
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allokator mit Hazard-Pointers. Maged Michael (PLDI 2004). Re-Implementierung mit IBM-Patent-Hintergrund.	P— (LGPL Re-Impl)
MimallocAllocator (A04)	Free-List-Sharding mit deferred-free und page-local management. Leijen/Zorn/de Moura (MSR-TR-2019-18, APLAS 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
JemallocAllocator (A05)	Arena-basierte Größenklassen mit per-CPU Arenas und tcache Layer. Evans (BSDCan 2006). Standard-Allokator von FreeBSD und Facebook. Original Code-Linking (is_original=2/2).	P— (OSS, BSD-2-Clause, is_original)
TCMallocAllocator (A06)	Thread-Caching Malloc mit zentralem Page-Heap. Ghemawat/Menage (Google ca. 2005) + TEMERAIRE (OSDI 2021). Google-Produktions-Standard.	P— (OSS, Apache-2.0)
SnmallocAllocator (A07)	Message-Passing-Allokator mit asymmetrischen Heap-Operationen. Paul Liétar et al. (ISMM 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)

Variante	Beschreibung	Quelle
ScallocAllocator (A08)	Span-basierte Free-List mit Virtual Spans für Multi-Core-Skalierbarkeit. Aigner/Kirsch/Lippautz/Sokolova (OOPSLA 2015). Low-Fragmentierung via Span-Pooling.	P— (OSS, BSD-3-Clause)
NUMAllocAllocator (A09)	NUMA-Origin-Awareness mit schnellerer Speicherallokation. UTSASRG (ISMM 2023). Optimiert für Multi-Socket-Architekturen.	P— (OSS, Lizenz nicht ermittelt)
RPMallocAllocator (A10)	Per-Thread Span-Caching ohne Dependency zu anderen Allokator-Bibliotheken. Mattias Jansson (2017). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/MIT, is_original)
LRMallocAllocator (A11)	Lock-Free Allocator mit Hazard-Pointers für Multi-Core-Skalierbarkeit. Leite/Rocha (VECPAR 2018, LNCS 11333). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator mit vorhersehbarem Verhalten. Herter/Backes/Haupenthal/Reineke (ECRTS 2011, Saarland). Keine eigenständige OSS-Implementierung.	P— (Baseline/Research)
StarMallocAllocator (A13)	Formal verifizierte härtete Memory-Allocator mit F*/Steel. Inria-Prosecco: Reitz/Fromherz/Protzenko et al. (OOPSLA 2024). Mathematisch verifizierter Allocator.	P— (OSS, Apache-2.0)
TCMallocWarehouse Allocator (A14)	TCMalloc-Hyperscale-Erweiterung mit Huge Page-Unterstützung für Warehouse-Scale-Computing. TEMERAIRE (Hunter OSDI 2021) + Warehouse-Charakterisierung (Zhou ASPLOS 2024). NUCA-aware TCMalloc-Variante.	P— (OSS, Apache-2.0)
HMallocAllocator (A15)	Hybrid Lock-Free Allocator mit skalierbare Speicherverwaltung. Li/Yao/Tang/Lin (IEEE ICPADS 2019). Keine eigenständige OSS-Implementierung.	P— (Research Baseline)
PIMMallocAllocator (A16)	Processing-In-Memory Allocator für heterogene HW mit schnellem lokalen Speicher. VIA-Research (HPCA 2026, arXiv:2505.13002). Spezialisiert auf PIM-Hardware.	P— (OSS, MIT)
CrystallineReclamation (A17)	Wait-free Safe-Memory-Reclamation-Familie (Nikolaev/Ravindran DISC 2021 / PLDI 2024). Reclamation-Policy, kein eigenständiger Allokator.	P— (Research)
ExgenAllocator (A18)	Single-Threaded Specialized Allocator mit Optimierungen für Single-Threaded Workloads. UT Austin (IEEE CAL 2025, arXiv:2510.10219). „Old is Gold“ Prinzip.	P— (Research, Lizenz nicht ermittelt)
BuddyAllocator (A19)	Buddy-System mit Power-of-2-Splitting für einfache Fragmentierungskontrolle. Knuth (TAoCP Vol.1, 1968). Klassisches Lehrbuch-Paradigma.	P— (Baseline/Textbook)
DlmallocAllocator (A20)	Doug Lea Malloc mit Bins-basierter Klassifizierung. Lea (1987–2012). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/CC0, is_original)
PtMalloc2Allocator (A21)	ptmalloc2 (glibc malloc) mit expliziter Wrapper-Kontrolle. Wolfram Gloger / Ulrich Drepper (1990er-2000er). Lizenz-blockiert (LGPL-2.1+).	P— (OSS, LGPL-2.1+, blockiert Linking)
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis) — portable Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen.	P— (Standard-Library)
PmrResourceAllocator (A22b)	std::pmr::memory_resource (polymorphe Memory-Ressource). Halpern (N3916, C++17). Abstrakte Schnittstelle ohne verhaltens-distinkte Default-Implementierung.	P— (Standard-Library, C++17)
PoolResourceAllocator (A22c)	std::pmr::unsynchronized_pool_resource mit eigenem Size-Class-Pool. Halpern (N3916, C++17). Verhaltens-distinkt ohne Vendor-Linking (F15-operativ).	P— (Standard-Library, C++17, F15-operativ)
VmemMagazinesAllocator (A23)	Vmem + Magazines als Erweiterung des Slab-Allocators für Multi-Core-Skalierung. Bonwick (USENIX ATC 2001). Keine eigenständige OSS-Implementierung.	P— (Baseline/Textbook)

Variante	Beschreibung	Quelle
VampirNfpAllocator (A24)	Allokator-Front nach dem VAMPIR-Entwurf (Virtualisierung nicht-funktionaler Speichereigenschaften): Allokationen laufen durch eine NFP-etikettierte Speicherstufe (Ziel-Stufe DRAM), der NFP-Deskriptor (<code>bandwidth_class</code> , <code>latency_class</code> , <code>capacity</code>) wird offengelegt. Strukturelle Rekonstruktion allein des baubaren Speichertyp-Virtualisierungs-Teils: Poster und Review liefern weder Code noch Kostenformel; Energie ist bewusst nicht modelliert (kein Zähler, sonst eine leere Metrik); Kompensation, transparente OS-Migration und globales Pipeline-Scheduling sind nicht Gegenstand. <code>is_original=false</code> , standardmäßig abgeschaltet.	P33 (VAMPIR-Poster SOSP 2023 [BSB ⁺ 23], TU Dresden; Rekonstruktion, <code>is_original=false</code>)

Registry-Stand: Die generierte Organ-Registry führt für die Allokator-Achse drei aktivierte Bausteine — `std_malloc`, `pmr_resource` und `pool_resource`; der Katalog A01–A24 bleibt als Angebots-Katalog erhalten, die übrigen Wrapper sind je Vendor-Verfügbarkeit und Aktivierungs-Flag zuschaltbar.

Beitragende Reclamation-Quellen: P29, P30 [McK01, Mic04a]; vollständiger Allokator-Korpus A01–A24 des Codes (der Literatur-Korpus A01–A23 aus Abschnitt 3.2 plus die aus dem SOTA-Profil P33 rekonstruierte Allokator-Front A24 ohne eigenes Allokator-Profil) siehe Abschnitt „A-Korpus“.

D.8 T7 Prefetch-Strategie

Diese Achse ist das *Prefetch-Organ* (T7) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2). Die Prefetch-Achse bestimmt die Strategie zum vorausschauenden Laden von Knoten und Adressen entlang des erwarteten Such-Pfads in einer Trie-Datenstruktur. Sie unterstützt vier konkrete Strategien mit unterschiedlichen Triggermechanismen und Heuristiken: keine Prefetch (Mess-Baseline), distanzgeschätzte Software-Prefetch auf Basis von Knoten-Dichte und Cache-Latenz (ART-Standard), explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard) und pfadorientierte Prefetch mit Bundle-Granularität (`prt-art`, diese Arbeit).

Statische Unter-Achsen

ID	Name	Beschreibung
PF1	Triggermechanismus	Art der Kontrolle über den Prefetch-Moment: ob kein Mechanismus (Baseline), durch Compiler-Hinweise via Distance-Estimator, durch explizite Hardware-Prefetch-Instruktionen oder durch pfadorientierte Vorhersage
PF2	Distance-Heuristik	Berechnungsweise für die Prefetch-Distanz (wie weit voraus geladen wird): linear, adaptiv basierend auf Knoten-Dichte und Tier-Latenz, oder keine Heuristik
PF3	Granularität	Prefetch-Größe pro Trigger: Cache-Line (64 Byte, einzeln), Page (4K, Betriebssystem-Fenster) oder Bundle (mehrere Cache-Lines, Trie-pfadorientiert)

Bausteine

Variante	Beschreibung	Quelle
NonePrefetch	Keine Prefetch-Strategie (Baseline). Dient als Vergleichs-Referenz für Mess-Reihen. Trigger: keine; Heuristik: keine; Granularität: N/A. <code>is_active()</code> gibt false zurück.	Code/Baseline
DistanceEstimator Prefetch	Distanzgeschätzte Software-Prefetch (ART-Standard nach Leis ICDE 2013). Schätzt Cache-Distance zum nächsten Knoten basierend auf Knoten-Dichte [%] und Tier-Latenz [cycles] mit Heuristik: <code>estimate = clamp(1 + sparseness*8 + latency_factor, [1,16])</code> . Trigger: Compiler-Hinweise via <code>__builtin_prefetch</code> ; Heuristik: adaptiv (Dichte + Latenz); Granularität: Cache-Line. Setzt auf ART-Pattern (unodb/libart).	P01 (ART/unodb, Leis ICDE 2013)
HardwarePrefetch	Explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard nach Wu EuroSys 2019). Nutzt PREFETCHT0/T1/T2/NTA auf Hash-Anchor-Chain ohne Software-Heuristik; CPU schätzt Distance autonom. Trigger: explizite Hardware-Instruktion; Heuristik: Hardware-managed; Granularität: Hardware-bestimmt. Lizenz GPL-3.0 blockiert <code>is_original-Linking</code> .	P07 (Wormhole, Wu EuroSys 2019)
PathOriented Prefetch	Pfadorientierte Prefetch-Heuristik (prt-art, diese Arbeit). Pre-loadet alle Knoten entlang vorhergesagtem Trie-Pfad bevor Suche startet. Tracker verfolgt Suchpfad und empfiehlt nächste Adresse via linearer Schritt-Extrapolation. Hot-Path-Hints: interpretiert erste 8 Byte eines Schlüssels als uint64-Adresse. Trigger: pfadorientierte Vorhersage; Heuristik: linear (Schritt-Extrapolation); Granularität: Bundle (mehrere Cache-Lines). Verwandte Literatur: Chen/Gibbons/Mowry SIGMOD 2001 (pB+-Trees Prefetching) + Mowry 1994.	P21 (verwandt: Chen/Gibbons/Mowry SIGMOD 2001); diese Arbeit

Beitragende Quellen: P01, P07, P21 [LKN13, WNJ19, CGM01].

Verwandte Literatur: P23, P25, P26, P27 [Kha11, MWR25, ZSZ⁺24, ZGH⁺25].

D.9 T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)

Diese Achse ist das *Nebenläufigkeits-Organ* (T8) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Diese Achse regelt Synchronisationsmuster und sichere Speicherreclamation für nebenläufige Zugriffe auf Baum-/Indexstrukturen. Die Achse stellt compile-time wählbare Synchronisations-Patterns (None/Blocking/Reader-Writer/Optimistic/Lock-Free/Wait-Free) bereit sowie orthogonale Reclamation-Schemata (RCU/Hazard-Pointer) für sichere Speicherfreigabe in lock-freien Strukturen.

Statische Unter-Achsen

ID	Name	Beschreibung
CC1	Synchronisations-Pattern (CC1)	Compile-time Varianten-Familie der Synchronisationsmuster: None (single-threaded Baseline), Blocking (pessimistic Mutex), ReaderWriter (shared_mutex), Optimistic (lock-free Versionsvalidierung), LockFree (CAS-Loops), WaitFree (gebundene Schritte). Jede Familie hat distinkte Laufzeit-Charakteristiken (0-Overhead bis Mutex-Overhead).

ID	Name	Beschreibung
CC2	Reclamation-Schema (CC2)	Orthogonale Compile-time Varianten-Familie für sichere Speicherfreigabe in lock-freien Strukturen: RCU (deferred grace-period, via userspace-rcu), Hazard-Pointer (per-thread in-use-Marken). Kombinierbar mit lock-freien Patterns, optional bei Blocking/Optimistic.

Bausteine

Variante	Beschreibung	Quelle
NoneConcurrency	Keine Synchronisation, single-threaded Baseline. Beide acquire() und release() sind no-ops (volatile Dummy). Vergleichs-Nullpunkt für Concurrency-Overhead-Messung (F15).	Code/Baseline
Blocking Concurrency	Pessimistisches Mutex-Locking mit std::mutex. Globaler coarse-grained Lock. Serialisiert alle Zugriffe. Obergrenze des Locking-Overheads. Veröffentlichung: Dijkstra CACM 1965.	Code/Baseline
ReaderWriter Concurrency	Single-Writer/Multi-Reader via std::shared_mutex. Lock-Sharing bei read-lastigen Workloads. Mittelgewicht zwischen pessimistischen und optimistischen Mustern. Veröffentlichung: Courtois et al. CACM 1971.	Code/Baseline
OlcOptimistic Concurrency	Optimistic Lock-Coupling (ART-Sync): Versions-Counter pro Knoten, lock-freie Reader mit Retry-on-Conflict. Billig bei niedrigen Konflikten. Veröffentlichung: Leis DaMoN 2016 / DEBULL 2019 (P08, is_original=true, Apache-2.0, unodb).	P08
LockFree Concurrency	Lock-free via Compare-And-Swap (std::atomic CAS-Loops). System-weiter Fortschritt. Generisches CAS-Pattern ohne Blockierung. Veröffentlichung: Michael/Scott PODC 1996 (Referenz-Paper).	Code/Baseline
WaitFree Concurrency	Wait-free: Jeder Thread schließt in beschränkter Schrittzahl ab (stärkste Garantie). Ohne Retry-Loop wie LockFree. Realisiert via Sequence-Lock / Fetch-Add. Veröffentlichung: Herlihy TOPLAS 1991 (Theorie-Referenz).	Code/Baseline
RcuConcurrency	Read-Copy-Update (McKenney OLS 2001): Lock-freie Reader via per-Thread Nesting-Zähler (RELAXED-Order, <i>kein</i> Memory-Barrier auf Read-Side). Reclamation-Schema orthogonal zu Pattern. Lokale Re-Impl (P29 userspace-rcu LGPL-2.1, is_original=false).	P29
HazardPointer Concurrency	Hazard Pointers (Michael TPDS 2004, P30): Pro-Thread markierte in-use-Zeiger verhindern vorzeitige Speicherfreigabe (ABA-safe). Reclamation-Schema. Lokale Re-Impl (P30 NO LICENSE, is_original=false). SEQ_CST-Store auf acquire().	P30
OlcReservedBlocks Concurrency	OLC + Cache-Line-Reservierte Value-Blocks (Multi-Writer Cache-Coherence-Storm-Vermeidung). Spezialisierung des Optimistic-Patterns mit zusätzlicher atomarer Block-ID-Reservierung (fetch_add). prt-art-Option (is_original=false).	Code/Baseline

Beitragende Quellen: Dijkstra 1965, Courtois et al. 1971, P08, Michael/Scott 1996, Herlihy 1991, P29, P30 [Dij65, CHP71, LABN16, MS96, Her91, McK01, Mic04a].

D.10 T9 Serialisierung (Kodierung und Kompression)

Diese Achse ist das *Serialisierungs-Organ* (T9) des Lebewesens SearchAlgorithm (vgl. Abschnitt 3.2).

Die Achse axis_10 (Organ-Slot T9) bestimmt, wie Daten innerhalb des Cache-Engine-Systems kodiert und komprimiert werden: vom Raw-Binary-Speicherlayout über variable Längenkodierung bis

zur Succinct-Bit-Packing. Die Wahl der Serialisierungsstrategie beeinflusst direkt CPU-Kosten (Delta-Encoding, Bit-Manipulation, LEB128-Varint), Speicher-Overhead und IO-Bandbreitennutzung — kritisch für LSM-Trees und Read-Heavy-Workloads mit Approx-Filter (SuRF/LOUDS).

Statische Unter-Achsen

ID	Name	Beschreibung
SR1	Byte-Order und Wortformat	Wie werden Bytes innerhalb eines Datensatzes angeordnet? Raw (memcpy Roh-Layout), Varlen (LEB128-Varint mit Signalisierungsbits), oder Packed (Bit-für-Bit-Packing)?
SR2	Dichte (Speicherauslastung)	Speicher-Overhead pro Datensatz: Sparse (vollständige Records), Dense (mit Padding), oder Succinct (Information-theoretisch dicht, $n \cdot H + o(n)$ bits)?
SR3	Kompression und Vorverarbeitung	Block-Kompression (LZ4/Snappy, LZ77-basiert) oder Delta-Encoding (Zigzag-Transform für negative Deltas)? Oder keine Kompression?

Bausteine

Variante	Beschreibung	Quelle
RawBinary Serialization	Roh-Byte-Speicherlayout (memcpy). Baseline ohne Transformation. Minimaler CPU-Aufwand. Typ. für In-Memory-Workloads ohne Kompression. Variante: <code>serialisierte_scan()</code> liest 32-Bit Felder + Order-sensitive Checksum.	Code/Baseline
VarLen Serialization	Variable-Längen-Kodierung mit Signalisierungsbits (LEB128-VarInt). Standard für ART (Leis ICDE 2013 P01). Kleine Werte = wenige Bytes. Signalisierungsbits markieren Byte-Länge. Datenabhängige Schleife (1-5 Bytes). FNV-1a-Mix der Varint-Bytes.	P01
Succinct Serialization	Bit-für-Bit-Packing mit minimaler Bitbreite (LOUDS/SuRF). Erreicht Information-theoretisches Minimum $n \cdot H + o(n)$ bits. Read-Only nach Build. Optimal für Approx-Filter + Read-Heavy. Höchster Per-Element-CPU-Aufwand (Bit-Manipulation). Variante: <code>serialize_scan()</code> mit 64-Bit Akkumulator.	P10,P09
Compressed Serialization	Block-Kompression (LZ4/Snappy) auf Roh-Binary aufgesetzt. Trade-off: CPU-Kosten vs IO/Memory-Bandbreite. Variante: Delta-Encoding gegen Vorgänger + Zigzag-Transform (negatives Delta → kleine unsigned). Typisch für LSM-Trees. Mittlerer CPU-Aufwand.	Code/LZ77 (Ziv/Lempel 1977) [ZL77]

Beitragende Quellen: P01, P09, P10, Ziv/Lempel 1977 [LKN13, Jac89, ZLL⁺18, ZL77].

D.11 T10 Wert-Handle (Value-Handle)

Diese Achse ist das *Wert-Handle-Organ* (T10) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Die Wert-Handle-Achse definiert, wie Datenwerte (Values) im Cache-Index gespeichert und zugegriffen werden: inline im Knoten-Slot, extern in einem Pool, mittels immutable-shared References (RCU-Style), versioned Pointer (MVCC), oder verkettete externe Referenzen (Multi-Value). Diese Achse charakterisiert die Speicherlayout-Strategie für Values und bestimmt damit die Zugriffslatenz (direkt vs. indirekt) und Concurrency-Semantik (Snapshot vs. Versioning).

Statische Unter-Achsen

ID	Name	Beschreibung
VH1	Speicherort (Storage-Location)	Compile-Time Varianten für die physische Platzierung des Value: inline im Knoten-Slot (kombiniert Key/Value in einem Slot ohne Indirektion) oder extern in einem separaten Pool (der Knoten hält nur Offset/Pointer).
VH2	Eigentümer-Semantik (Ownership)	Compile-Time Varianten für Ownership und Sharing-Strategie: owned (exclusive per Key), shared (immutable reference-counted via RCU-style), weak (borrowing ohne Increment).
VH3	Versionierung (Versioning)	Compile-Time Varianten für Concurrency-Kontrolle: keine Versionierung (simple Ownership), Version-Tags in Pointer (MVCC für Reader-Snapshot-Isolierung).

Bausteine

Variante	Beschreibung	Quelle
InlineValueHandle	Value wird direkt im Knoten-Slot eingebettet (keine Indirektion). Standard für kleine Values wie uint64_t. Minimale Zugriffslatenz (1 direkter Read), aber Knoten-Speicher wird durch größere Values beansprucht. Quelle: ART (Leis ICDE 2013, P01).	P01
ExternalPoolValueHandle	Value extern in Pool gespeichert; der Knoten hält nur den Pool-Offset. Ermöglicht variable Value-Größen und kompaktere Knoten, kostet aber 1 zusätzlichen Cache-Miss pro Lookup (Pointer-Chasing). Quelle: Wormhole (Wu EuroSys 2019, P07).	P07
ImmutableSharedRefValueHandle	Value via reference-counted Pointer (RCU-Style Snapshot-Sharing). Optimal für Reader-Concurrency: Leser sehen stabile Snapshots, Writer erstellen neue immutable Snapshots. Requires RefCount-Acquire/Release pro Zugriff. Quelle: RCU (McKenney OLS 2001, P29).	P29
VersionedPointerValueHandle	Pointer mit Version-Tag im oberen Byte (MVCC). Ermöglicht Multi-Version-Concurrency-Control: Reader bekommt Snapshot-Version zur Sichtbarkeitskontrolle, Writer erstellt neue Version. Tombstone-Erkennung via Versions-Bit. Quelle: Masstree (Mao EuroSys 2012, P03) und SMART (OSDI 2023).	P03
ChainRefValueHandle	Multi-Value verkettete externe Referenz (Linked-List). Der Knoten hält den Offset zum Chain-Head im Pool; jeder Chain-Knoten hält (value_offset, next_offset). Teuerste Variante mit 2 abhängigen Derefs pro Value-Zugriff. Quelle: prt-art (lokale Re-Implementierung).	Code/Baseline

Beitragende Quellen: P01, P03, P07, P29, SMART (OSDI 2023) [LKN13, MKM12, WNJ19, McK01, LZS⁺23].

D.12 T11 Index-Organisation

Diese Achse ist das *Index-Organisations-Organ* (T11) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2).

Klassifizierung der Datenspeicherung und Indexstrukturierung nach drei orthogonalen Dimensionen: Storage-Order (Heap/Clustered/Ungeordnet), Index-Count (keine/einzeln/mehrfach) und Data-Embedding (separate Speicherung vs. in Leaf-Pages eingebettet). Diese Achse modelliert fundamentale DBMS-Entwurfsentscheidungen, die Cache-Verhalten, Zugriffsmuster und I/O-Charakteristiken auf Basis von Index-Order vs. Storage-Order-Alignment entscheidend beeinflussen. Die drei Unter-Achsen führen die

Kürzel IO1–IO3 (Index-Organisation); die I/O-Dispatch-Achse (T12) verwendet dieselben Kürzel für andere Gegenstände.

Statische Unter-Achsen

ID	Name	Beschreibung
IO1	Storage-Order	Anordnung der Datensätze im Speicher: ungeordnet (Insert-Order im Heap), sortiert nach Primary-Key (Clustered), oder mit Pointer-Indirektion (Non Clustered). Definiert die sequenziellen oder zufälligen Zugriffsmuster bei Range-Scans. Code-Token: <code>storage_order_tag</code> .
IO2	Index-Multiplizität	Anzahl unterstützter Indizes pro Tabelle: keine (Heap-Baseline), Single-Index (Clustered, MySQL InnoDB Primärschlüssel), oder Multiple Secondary-Indizes (NonClustered, SQL Server NONCLUSTERED INDEX). Code-Token: <code>index_count_tag</code> .
IO3	Daten-Einbettung	Speicherplatzierung der Daten relativ zum Index: getrennt im Row-Store (Clustered/NonClustered mit Pointer-Hop), oder direkt in Index-Leaf-Pages eingebettet (IOT/Trie-Style wie ART/HOT). Code-Token: <code>data_embedding_tag</code> .

Bausteine

Variante	Beschreibung	Quelle
ClusteredIndex Organization	Speicher-Reihenfolge entspricht Index-Reihenfolge (1 Primary-Key). Optimiert für Range-Scans entlang des Primary-Key mit sequenziellen Disk-Reads (cache-freundlich). Standard: MySQL InnoDB, PostgreSQL CLUSTER, SQL Server CLUSTERED INDEX. Zugriffsmuster: sequenzielle Forward-Iteration ohne Predicate-Evaluation, O(n) Summen-Scan.	Bayer & McCreight, Acta Informatica 1972 (10.1007/BF00288683)
HeapIndex Organization	Storage-Reihenfolge = Insert-Order, <i>kein</i> Index (Baseline). Lookup mittels O(n) Full-Scan mit Predicate-Evaluation pro Record (datenabhängiger Branch, kein früher Abbruch). Standard: PostgreSQL Heap-Tabellen, HBase. Zugriffsmuster: ungeordneter sequenzieller Scan mit Vergleichslast und Branch-Misprediction.	Garcia-Molina, Ullman & Widom, Database Systems (Textbook Baseline)
IotIndexOrganization	Daten direkt in Index-Leaf-Pages eingebettet (Oracle IOT, SQL Server CLUSTERED INDEX style). Spart Pointer-Indirektion vs. NonClustered. Äquivalent zu B+Tree-Leaves mit eingebetteten Daten oder Trie-Knoten (ART/HOT/Wormhole). Zugriffsmuster: sequenziell wie Clustered, aber mit Zusatzlast durch Daten im Leaf (zwei Felder pro Record).	Srinivasan et al., VLDB 2000 (Oracle8i IOT) [SDF ⁺ 00]

Variante	Beschreibung	Quelle
NonClusteredIndex Organization	Index-Reihenfolge $\neq$ Storage-Reihenfolge, mehrere Secondary-Indizes pro Tabelle. Lookup über Index + Pointer-Hop zur Daten-Row (Random-Stride, Cache-Misses). Standard: SQL Server NONCLUSTERED INDEX, PostgreSQL CREATE INDEX. Zugriffsmuster: Index-navigation gefolgt von zufälligen Storage-Hops (LCG-determiniert, tatsächliche cache-Miss-Latenz durch TLB/L2-Invalidierung), anschließend ein Residual-Predicate-Vergleich pro geholtem Record (datenabhängiger Branch auf der geholten Daten-Row nach dem Pointer-Hop).	Comer, ACM Computing Surveys 1979 (The Ubiquitous B-Tree, 10.1145/356770.356776)

Beitragende Quellen: Bayer & McCreight 1972, Comer 1979, Srinivasan et al. 2000, Garcia-Molina, Ullman & Widom (Textbook) [BM72, Com79, SDF⁺00, GMUW09].

D.13 T12 I/O-Dispatch-Achse

Diese Achse ist das *I/O-Dispatch-Organ* (T12) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Modellierung und Auswahl von I/O-Strategien für die Persistierung und den Speicherzugriff im Index: Speicherung im RAM-only (Baseline), gepuffert via OS-Page-Cache (Standard), direkter I/O mit `O_DIRECT`-Flag (NVMe-Optimierung), oder Memory-Mapped File I/O (Persistent Memory). Diese Achse ermöglicht die Entkopplung der Such- und Baum-Strukturen von der Persistierungs-Implementierung und gestattet A/B-Messungen der Dispatch-Overhead-Profile unter identischen Workloads. Die drei Unter-Achsen führen die Kürzel IO1–IO3 (I/O); die Index-Organisations-Achse (T11) verwendet dieselben Kürzel für andere Gegenstände.

Statische Unter-Achsen

ID	Name	Beschreibung
IO1	Persistierungsmodell	Klassifiziert die Speicherart: RAM-only (keine Persistierung, reine Mess-Baseline), oder Disk/Persistent-Memory (Datei-gesicherte Indizes mit verschiedenen Caching-Strategien).
IO2	Caching-Strategie	Charakterisiert die OS-Puffer-Nutzung: Buffered (OS Page-Cache, read-ahead + write-back, Standard), Direct I/O (<code>O_DIRECT</code> , Page-Cache-Bypass, 512B/4KB-Alignment), oder Memory-Mapped (file-backed <code>mmap</code> mit automatischer OS-Pagination und page-fault-getriebenem Zugriff).
IO3	Schreib-Dauerhaftigkeit	Legt die Konsistenz-Garantien fest: keine <code>fsync</code> (höchste Performance, keine Absicherung), <code>fsync</code> nach Batch (Kompromiss, Batch-Commits), oder atomare Schreibvorgänge (höchste Sicherheit, niedrigste Performance). In dieser Implementierung hauptsächlich als konzeptionelle Unter-Achse vorhanden; operative Umsetzung erfolgt über die Caching-Strategie.

Bausteine

Variante	Beschreibung	Quelle
InMemoryOnly	Rein RAM-basierter Index ohne Persistierung. Direkter, sequentieller Record-Zugriff ohne Dispatch-Overhead. Schnellster Pfad und Vergleichs-Nullpunkt der Achse. Verwendet für Mess-Baseline und Performance-Referenzierung unter Bedingungen ohne I/O-Latenz.	Code/Baseline
BufferedIo	Standard <code>read()/write()</code> via OS-Page-Cache mit automatischem read-ahead und write-back. Default für allzweck-DBMS. Trade-off: bequem, aber Double-Buffering (App-Cache + OS-Cache führt zu erhöhter Speichernutzung). Referenz-Veröffentlichung: Stonebraker CACM 1981 (Konzept-/Position-Paper Operating System Support for Database Management, DOI 10.1145/358699.358703). Einordnung: Engineering-Pattern, kein Algorithmus-Code der Referenz.	P-Stonebraker-1981
DirectIo	Bypass des OS-Page-Cache mittels <code>O_DIRECT</code> -Flag (<code>open(2)</code>), NVMe-SSD-optimal. 512B/4KB-Alignment erforderlich. Ermöglicht DBMS-eigene Cache-Strategien (RocksDB, MySQL). Höhere CPU-Kosten, aber keine Cache-Pollution durch Background-Activity. Keine dedizierte Veröffentlichung als Quelle (Linux-API seit 2.4.10); wird als bewusste Engineering-Baseline behandelt.	Code/Baseline
MmapIo	Memory-Mapped File I/O (<code>mmap</code>) mit Persistent Memory oder Read-Heavy Workloads. OS pagiert automatisch, kein Copy zwischen User- und Kernel-Space. Trade-off: page-fault-Latenz-Spitzen vs. Reuse-Freundlichkeit. Referenz-Veröffentlichung: Crotty et al. CIDR 2022 (Anti-Pattern-Baseline Are You Sure You Want to Use MMAP in Your DBMS?, db.cs.cmu.edu/.../cidr2022-p13-crotty.pdf). Bewusst als Anti-Pattern-Referenz zitiert, um Grenzen und Overhead zu dokumentieren. OSS-Referenz MIT-lizenziert.	P-Crotty-2022

Beitragende Quellen: Stonebraker-1981, Crotty-2022 [Sto81, CLP22].

D.14 T13 Migrations-Strategie-Achse

Diese Achse ist das *Migrations-Organ* (T13) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2). Die Migrations-Strategie-Achse (T13, *axis_migration*) kapselt Entscheidungslogiken zur Daten-Platzierung über mehrschichtige Speicherhierarchien (RAM/SSD/HDD, Hot/Cold-Tiers). Sie modelliert, wann und wie häufig zugriffene Daten zu schnellerem Storage promoviert bzw. selten zugriffene demoviert werden, ohne eine tatsächliche Tier-Migration durchzuführen (Entscheidungslogik-Messung nur). Unterstützt vier konkrete Strategien: static (Baseline), Heat/Cold-Separation, Multi-Tier-Hierarchie, ML-driven Adaptive.

Statische Unter-Achsen

ID	Name	Beschreibung
MG1	Trigger-Mechanismus (MG1)	Bestimmt, wann eine Migrations-Entscheidung ausgelöst wird: keine (static), Schwellwert-basiert (Threshold), beobachtungsgesteuert (Observation). Compile-Time Variante pro Strategy.
MG2	Migrations-Richtung (MG2)	Bestimmt die Richtung von Daten-Bewegung: keine (static), aufwärts (RAM→SSD), abwärts (SSD→RAM), bidirektional. Compile-Time Variante.

ID	Name	Beschreibung
MG3	Migrations-Granularität (MG3)	Spezifiziert die Skalierungseinheit für Migrations-Entscheidungen: keine (static), Page, Block, Object (einzelnes Item); die Größe der Page- und Block-Einheit legt der Code nicht fest (reine Tag-Typen <code>granularity_tag</code> ; die Migrations-Bausteine messen die Entscheidungslogik ohne realen Block-Transfer). Compile-Time Kategorie.

Dynamische Unter-Achsen

Die drei Größen sind Entwurfs-Parameter der Achse; der Code führt sie als Übersetzungszeit-Konstanten innerhalb der Entscheidungs-Scans (`w_freq = 3` und `w_rec = 1` in `axis_migration_adaptive.hpp`, `kTiers = 3` in `axis_migration_tier_based.hpp`; der Hot/Cold-Scan entscheidet per Recency-Vergleich ohne Schwellwert), ein Laufzeit- oder Benutzer-Parameter ist für keine der drei implementiert, und die generierte Registry führt für T13 keine Laufzeit-Unter-Achse.

ID	Parameter	Beschreibung
D1	Entscheidungs-Schwellwert (Decision Threshold)	Laufzeit-Parameter (Plan; kein Code-Parameter): Hot/Cold-Klassifikations-Grenzwert (z.B. Recency-Counter-Limit für LRU-basierte Promotions/Demotions), Benutzer-konfigurierbar; der Hot/Cold-Scan im Code entscheidet je Record per Recency-Vergleich mit dem zuletzt gesehenen Feldwert ohne Grenzwert.
D2	Gewichtungen für Feature-Scoring	Laufzeit-Parameter (Plan; im Code Konstante) für AdaptiveMigration: Gewichte (<code>w_freq</code> , <code>w_rec</code>) für Häufigkeits- und Recency-Features in der ML-Linear kombination, im Code fest als <code>w_freq = 3</code> und <code>w_rec = 1</code> . Selbstadaption der Gewichte vorgesehen, aber nicht implementiert; implementiert ist ein gleitender Score-Akkumulator bei festen Gewichten.
D3	Tier-Konfiguration	Laufzeit (Plan; im Code Konstante): Anzahl Speicher-Tiers (z.B. <code>kTiers=3</code> für RAM/SSD/HDD in TierBasedMigration; im Code die Übersetzungszeit-Konstante <code>kTiers = 3</code>), Kapazitäten pro Tier und Promotions/Demotion-Schwellen (Plan; ohne Code-Objekt).

Bausteine

Variante	Beschreibung	Quelle
NoMigration	Statische Platzierung ohne Migration (Baseline). Daten werden einmalig positioniert, keine Bewegung zwischen Tiers. Benchmark-Nullpunkt für Vergleich mit aktiven Strategien.	Code/Baseline
HotColdMigration	Heat/Cold-Separation via LRU + probabilistisches Recency-Counting. Häufig zugriffene Daten (hot) werden zu schnellerem Cache promoviert, selten zugriffene (cold) zu langsamem Storage demoviert. Standard für LRU-K und LFU-Cache-Systeme.	Anti-Caching (PVLDB 2013) [DPT ⁺ 13]
TierBased Migration	Multi-Ebenen-Migration über die Stufen RAM→SSD→HDD (<code>kTiers = 3</code> im Code; die Paper-Code-Zuordnung des <code>ce-Repositorys</code> nennt die Stufenfolge RAM→NVM→SSD→HDD, eine NVM-Stufe ist im Code nicht implementiert) mit Modulo-basiertem Tier-Mapping. Jedes Datenitem wird anhand eines Feldwertes modulo-auf Ziel-Tier gemappt. Standard für RocksDB, LSM-Trees und Tiered-Cache-Systeme mit konfigurierbaren Schwellwerten; der Baustein selbst besitzt keinen Schwellwert-Parameter (Tabelle der dynamischen Unter-Achsen).	Managing NVM in DBMSs (SIGMOD 2018) [RLK ⁺ 18]

Variante	Beschreibung	Quelle
AdaptiveMigration	ML-driven adaptive Migration mit Online-Learning (Cachelib/ LeCaR-Stil). Gewichtete Linearkombination von Häufigkeits- und Recency-Features berechnet adaptiven Score; gleitender Lern-Akkumulator modelliert Online-Learning. Höherer Overhead vs. optimale Hit-Rate.	ML-based LeCaR (USENIX HotStorage 2018) [VRM ⁺ 18]

Beitragende Quellen: Anti-Caching (PVLDB 2013), LeCaR (HotStorage 2018), Managing NVM (SIGMOD 2018) [DPT⁺13, VRM⁺18, RLK⁺18].

D.15 T14 Filter-Achse

Diese Achse ist das *Filter-Organ* (T14) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2). Die Filter-Achse T14 bietet wählbare probabilistische Membership-Query-Filter (Punkt- und Bereichs-abfragen; alle vier Bausteine antworten approximativ, ihre Konstruktion ist deterministisch). Sie deckt klassische Bloom-Filter mit k Hash-Funktionen, den deletable-Cuckoo-Filter mit Bucket-Hashing und Fingerprints, den succinct Range-Filter SuRF mit LOUDS-Trie-Navigation sowie den platzsparenden Xor-Filter mit konstanter 3er-XOR-Latenz ab. Zweck: Trade-offs zwischen Raum, Abfragelatenz, Mutability und Falsch-Positiv-Rate direkt im Cache-Engine-Experiment messbar machen. Als exakter Vergleichs-Nullpunkt ist zusätzlich die Durchreich-Variante *NoneFilter* registriert (E14), in der Bau-Konfiguration standardmäßig abgeschaltet.

Statische Unter-Achsen

ID	Name	Beschreibung
FT1	Abfrage-Typ	Unterscheidung zwischen Punkt-Abfragen (membership: key enthalten?) und Bereichs-Abfragen (range: keys in [a,b]?). Punkt-Filter sind Bloom/ Cuckoo/Xor; nur RangeSurfFilter unterstützt Range-Semantik.
FT2	Mutierbarkeit	Unterteilung in unveränderbare Filter (Bloom, RangeSurf, Xor nach Build immutable) vs. Mutable (Cuckoo mit delete-Operation). Bestimmt Design des Wrapper und Cachingmöglichkeiten während Laufzeit.
FT3	Fehler-Profil	Charakterisierung der fehlertoleranten Semantik: False-Positive-Only (Bloom, Cuckoo, Xor — Filter sagt „ja“ falsch), vs. lossless (theoretisch deterministische Antwort). Bloom: approx. $k \cdot n$ Hash-Tests; Xor: exakt 3 XOR-Operationen (branch-frei).

Bausteine

Variante	Beschreibung	Quelle
BloomFilter	Klassischer Bloom-Filter nach Bloom 1970: m-Bit-Bitmap, k unabhängige Hash-Funktionen (double-hashing nach Kirsch/Mitzenmacher 2006). Punkt-Abfragen nur; False-Positives möglich, keine Deletes. Implementierung mit tatsächlicher persistenter m-Bit-Bitmap (8 KiB, m=65536, k=4 double-hashing), die im Out-of-Process-Mess-Pfad über die tatsächlich eingefügten Schlüssel geprobt wird; die als solche ausgewiesene Pseudo-Bitmap-Probe (k=4 Hash-Tests pro Query, charakteristische early-exit-Latenz bei Mismatch) verbleibt nur im In-Process-Segment-Timing-Scan.	Bloom 1970 (CACM 13:422-426)
CuckooFilter	Cuckoo Filter nach Fan et al. CoNEXT 2014: Bucketed Cuckoo-Hashing mit 1-Byte-Fingerprints. Unterstützt Deletes und Lookup-Counting (Vorteil vs Bloom). Probe-Charakteristik: 2 Bucket-Lookups + Fingerprint-Vergleich (partielle-Schlüssel-Cuckoo nach CoNEXT 2014 §3: $i2 = i1 \text{ XOR } \text{hash}(fp)$). Höhere Insert-Kosten, aber flexible Mutierbarkeit.	Fan et al. CoNEXT 2014 (DOI:10.1145/2674005.2674994)
RangeSurfFilter	SuRF (Succinct Range Filter) nach Zhang et al. SIGMOD 2018: LOUDS-Bitmap-Trie für Range-Abfragen. Einziger Filter mit Range-Semantik (lower_bound/upper_bound auf Byte-Prefix-Ebene). Probe-Charakteristik: mehrstufiger Trie-Abstieg mit succinct-LOUDS-Navigation (bis kMaxDepth=6), nicht exakt 1 Hash-Test, daher latenz-variabel je Prefix-Länge. Praktisch in RocksDB SST-Optimierungen.	P10 (Zhang et al. SIGMOD 2018)
XorFilter	Xor Filter nach Graf+Lemire 2020 (JEA 25): XOR-basierter Filter mit ca. 9 bits/key (vs. 10 bits/key Bloom bei gleicher FP-Rate). Immutable nach Build. Probe-Charakteristik: exakt 3 unabhängige Tabellen-Zugriffe (3 disjunkte Tabellen-Drittel $h0, h1, h2$) + 3er-XOR, branch-frei, konstante Latenz ohne early-exit. Sicherheit gegen Wegoptimierung, kleinster Speicherfootprint aller Filter.	Graf+Lemire 2020 (JEA 25:Article 1.5, DOI:10.1145/3376122)
NoneFilter	Durchreich-Variante ohne Filterung (E14): jede Probe besteht unbedingt (Eingabe gleich Ausgabe), 0 Bit Struktur-Speicher und 0 Hash-Tests je Probe — der Vergleichs-Nullpunkt für den Raum/Zeit-Trade-off der vier Filter (Familie 5, End-Append, golden-320-neutral). Im Katalog registriert und konform, in der Bau-Konfiguration aber standardmäßig abgeschaltet (Option <code>COMDARE_AXIS_FILTER_ENABLE_NONE</code> , Default OFF), sodass das aktivierte Inventar der Achse vier Bausteine umfasst.	Code/Baseline

Beitragende Quellen: Bloom 1970, Fan et al. CoNEXT 2014, Zhang et al. SIGMOD 2018, Graf+Lemire JEA 2020 [Blo70, FAKM14, ZLL⁺18, GL20].

D.16 T15 Pufferung Q1 (Operations-Puffer)

Diese Achse ist das *Pufferungs-Organ (Q1)* (T15) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Die Achse `axis_q1_queuing` realisiert verschiedene Pufferungsstrategien für Operationen im Cache-Engine-Kontext. Sie umfasst Lehrbuch-ADTs (FIFO, LIFO, Append-Only, Priority-Heap), spezialisierte Versioning-Techniken (Delta-Chain, Copy-on-Write, Epoch, Tombstone) sowie moderne Lock-Free-Queues (SPSC/MPMC nach Lamport und Vyukov). Die Strategien werden übersetzungszeitlich über die Achsen-Registry (`mp_list` der Achsen-Registry, Enable-Flag-gefiltert) pro Permutations-Binary selektiert (kein Runtime-Switch) und sind klassifizierbar nach Zugriffsmuster (sequenziell, sortiert, zyklisch, versioniert, batched, lock-free).

Statische Unter-Achsen

ID	Name	Beschreibung
QS1	Sequenzieller Zugriff	FIFO/LIFO/Append-Only: Zugriff am Anfang oder Ende, typisch Write-Coalescing + MemTable-Pufferung
QS2	Sortierter Zugriff	Skip-List/Priority-Heap: Sortierter Drain, typisch LSM-Compact und Hot-Key-Eviction
QS3	Zyklischer Zugriff	Bounded Ring-Buffer: Feste Kapazität, Disruptor-Pattern, wrap-around mit Overflow-Handling
QS4	Versionierter Zugriff	Delta-Chain/Tombstone/CoW/Epoch: Multi-Version Concurrency Control, Snapshot-Isolation, Reclamation-Windows
QS5	Batch-Operationen	Batched-Insert-Buffer: Sammlung von Operationen zu Sub-Batches, SIMD-Vectorization, Cache-Locality-Optimierung
QS6	Lock-Free Concurrent	SPSC/MPMC: CAS-Loop oder wait-free Lamport-Modell, Lock-Freedom ohne Mutex

Bausteine

Variante	Beschreibung	Quelle
NoBuffer	Passthrough/Identität (Baseline): <code>put()</code> ist no-op, <code>get()</code> liefert immer <code>std::nullopt</code> . Sinnvoll als Vergleichspunkt und in klassischen B+/ART-Szenarien mit direktem Leaf-Zugriff. Family Q01, QS1 <code>sequential_access</code> .	Code/Baseline
AppendOnlyBuffer	Append-Only Linear Buffer (LSM-MemTable, Bw-Tree-Delta): monoton wachsendes <code>std::vector</code> ohne mittiges Löschen. Optimiert für reine Append + Bulk-Drain. Family Q02, QS1 <code>sequential_access</code> . Literatur: Levandoski/Lomet/Sengupta Bw-Tree ICDE 2013, O'Neil LSM 1996.	Code/Baseline
FIFOQueueBuffer	FIFO-Queue (<code>std::deque</code>): First-In-First-Out via <code>deque::push_back / deque::pop_front</code> . Unbounded, kann mittig effizient wachsen. Lehrbuch-ADT, typisch für Write-Coalescing. Family Q03, QS1 <code>sequential_access</code> .	Code/Baseline
LIFOStackBuffer	LIFO-Stack (<code>std::vector</code>): Last-In-First-Out via <code>vector::push_back / vector::pop_back</code> . Unbounded, optimiert für Hot-Path-Wiederverwendung und Versions-Tombstones. Family Q04, QS1 <code>sequential_access</code> .	Code/Baseline
BoundedRingBuffer	Bounded Ring-Buffer (Disruptor-Pattern): Feste Kapazität, zyklischer Zugriff mit wrap-around. Overflow: <code>drop_oldest</code> (default). <code>iterable_aspect_t</code> : Capacity als Laufzeit-Loop. Family Q05, QS3 <code>cyclic_access</code> . Literatur: LMAX Disruptor 2011.	LMAX 2011
PriorityHeapBuffer	Priority-Heap (<code>std::priority_queue</code>): Max-Heap, <code>get()</code> liefert höchste Priorität zuerst. Erste Strategie mit <code>supports_priority_ordering()=true</code> . Unbounded, Lehrbuch-ADT (Williams 1964). Anwendung: LRU-Approximation, Hot-Key-Promotion, Eviction-Queues. Family Q06, QS2 <code>ordered_access</code> .	Code/Baseline
DeltaChainBuffer	Delta-Chain Buffer (Bw-Tree, Levandoski 2013): Erste versioned-Strategie (<code>is_versioned()=true</code>). Jeder <code>put()</code> hängt Delta-Record mit aufsteigender Version-ID an Spitze der Chain an. <code>get()</code> LIFO-Drain mit Versions-ID. Append-Versioning-Paradigma. Family Q07, QS4 <code>versioned_access</code> .	Bw-Tree (Levandoski et al. ICDE 2013) [LLS13]
SkiplistBuffer	Skip-List Buffer (sortierter Buffer, RocksDB-Stil): <code>put()</code> fügt in sortierter Reihenfolge ein via <code>std::set</code> . <code>get()</code> ASCENDING-drain (minimum first), typisch für LSM-Compact-Output. Unterschied zu <code>PriorityHeapBuffer</code> : <code>Skiplist=ascending</code> , <code>Heap=descending</code> . Literatur: Pugh CACM 1990. Family Q08, QS2 <code>ordered_access</code> .	Pugh 1990, RocksDB

Variante	Beschreibung	Quelle
TombstoneBuffer	Tombstone-Marker Buffer (LSM + MVCC): <code>put()</code> erzeugt Live-Eintrag, deleted-Einträge bleiben als Tombstones (<code>std::nullopt</code>) bis Compact-Lauf. <code>get()</code> überspringt Tombstones FIFO-artig, <code>is_versioned()</code> = true (Erase-Versioning, anders als Delta-Chain Append-Versioning). <code>size()</code> zählt nur Live-Slots. Family Q09, QS4 <code>versioned_access</code> . Literatur: O’Neil LSM-tree 1996.	O’Neil 1996
CopyOnWriteBuffer	Copy-on-Write Snapshot-Buffer (Persistent Data Structures): <code>put()</code> / <code>get()</code> inkrementiert Snapshot-Version, frühere Snapshots bleiben logisch erhalten via <code>shared_ptr</code> . Snapshot-Versioning-Paradigma (anders als Delta-Chain Append, Tombstone Erase). <code>is_versioned()</code> = true. Family Q10, QS4 <code>versioned_access</code> . Literatur: Driscoll/Sarnak/Sleator/Tarjan JCSS 1989.	Driscoll et al. 1989
EpochBuffer	Epoch-based Buffer / QSBR (Quiescent-State-Based-Reclamation): Einträge in aktueller Epoche akkumuliert. Nach <code>epoch_threshold</code> Übergang => frühere Epoche kann reclaimed werden. <code>iterable_aspect_t</code> : <code>epoch_threshold</code> als Laufzeit-Parameter. <code>is_versioned()</code> = true (Reclamation-Window-Versioning). Family Q11, QS4 <code>versioned_access</code> . Literatur: McKenney RCU OLS 2001, Masstree EuroSys 2012, SMART OSDI 2023.	P29 (RCU OLS 2001), Masstree 2012
BatchedInsertBuffer	Batched-Insert Buffer (OLAP-Indexing, ART-Bulk-Insert): <code>put()</code> puffert in Sub-Batches der Größe <code>batch_size_</code> , vollständige Batches werden übergeben (bulk-Drain). Ziel: amortisierter Insert-Cost durch Bulk-Operations, SIMD-Vectorization, Cache-Locality. Erste QS5-Strategie. Family Q12, QS5 <code>batched_access</code> . Literatur: Leis ART ICDE 2013 §6.	P01 (Leis ART ICDE 2013)
LockFreeSPSCBuffer	Lock-Free SPSC Ring-Queue (Lamport 1977, 1983): Single-Producer/Single-Consumer. <code>put()</code> wait-free (1 atomic.store + 1 atomic.load), <code>get()</code> wait-free. Kein CAS nötig dank disjoint Modified-Sets. <code>ProgressGuarantee</code> : LockFree (sogar wait-free). <code>iterable_aspect_t</code> : Capacity. Overflow: dropping. Pflicht-Vertrag: genau 1 Producer + 1 Consumer. Family Q13a, QS6 <code>lock_free_access</code> .	Lamport 1977, 1983
LockFreeMPMCBuffer	Lock-Free Bounded MPMC Queue (Vyukov 2010–11): Multi-Producer/Multi-Consumer via per-cell atomic Sequence-Number und CAS-Loop. <code>Cell.sequence == pos</code> (Producer schreibt), dann <code>pos+1</code> (Consumer liest). Retries bei Contention (wait-free pro Op <i>nicht</i> garantiert). Capacity <i>muss</i> Power-of-2 sein. <code>iterable_aspect_t</code> . Family Q13b, QS6 <code>lock_free_access.is_original_eligible=true</code> (Vyukov BSD-2).	Vyukov 1024cores 2010–11, Michael/Scott PODC 1996
OriginalLockFreeMpmcConcurrentQueue	<code>moodycamel::ConcurrentQueue</code> (Cameron Desrochers, Block-Based MPMC): Nachweis-Wrapper für Originalcode mit Original-Code-Linking (2/6 Methoden original: <code>put</code> , <code>get</code> ; 4/6 Lücken: <code>emplace</code> , <code>peek_front</code> , <code>peek_back</code> , <code>clear</code> sind Re-Impl). <code>is_original_module()</code> = false wegen Lückenfüllungen. Header-only, BSD-2/BSL-1.0 dual. Analog <code>LockFreeMPMCBuffer</code> , aber mit tatsächlicher Submodule-Bindung. Family Q15, QS6 <code>lock_free_access.is_original_eligible=true</code> (moodycamel BSD-2).	moodycamel 2014, BSD-2/BSL-1.0

Die Familien-Nummern laufen Q01–Q13b und Q15; Q14 ist im Katalog nicht belegt, die beiden Lock-Free-Warteschlangen teilen sich die Familie Q13 (Q13a SPSC, Q13b MPMC), der Original-Wrapper trägt Q15.

Beitragende Quellen: LMAX 2011, P01 (Leis ART ICDE 2013), Bw-Tree (Levandoski et al. ICDE 2013), P29 (RCU OLS 2001), Lamport 1977, 1983, Vyukov 1024cores 2010–11, Michael/Scott PODC 1996, Pugh CACM 1990, O’Neil LSM-tree 1996, Driscoll et al. 1989, Masstree EuroSys 2012, SMART OSDI 2023, Williams 1964, moodycamel 2014 [TFB⁺11, LKN13, LLS13, McK01, Lam77, Lam83, Vyu11,

MS96, Pug90, OCGO96, DSST89, MKM12, LZS⁺23, Wil64, Des14].

D.17 T16 Pufferung Q2 (Flush-Policy)

Diese Achse ist das *Pufferungs-Organ (Q2)* (T16) des Lebewesens *SearchAlgorithm* (vgl. Abschnitt 3.2).

Die Achse Q2 definiert automatische Spülstrategien für Schreibpuffer. Sie entscheidet, *wann* ein gefüllter Buffer sein Ausgabeergebnis an die nächste Stufe propagiert — nach je einer Operation (Eager), gar nicht (Lazy), bei Schwellwertüberschuss (Watermark), nach Zeitfenster (Timed) oder adaptiv nach Workload-Häufigkeit (EWMA-gesteuert LSM-Stil). Dies ist zentral für den Speicher-/Durchsatz-Tradeoff und die Batching-Effizienz in Cache-Engines.

Statische Unter-Achsen

ID	Name	Beschreibung
FS1	Ereignisgetriggert (Event-Triggered)	Flush wird durch explizite Ereignisse (put/get-Operation oder Eviction) ausgelöst. Umfasst EagerFlush (nach jeder Operation) und LazyFlush (nur bei Eviction).
FS2	Schwellwertgetriggert (Threshold-Triggered)	Flush wird ausgelöst, wenn die Pufferfüllung einen Schwellwert (Default 75%) überschreitet. Reduziert unnötige Spülungen bei sparsamem Traffic.
FS3	Zeitgetriggert (Time-Triggered)	Flush wird periodisch nach einem Zeitfenster (Micro-Batching) ausgelöst, unabhängig von Pufferfüllung. Klassisches Hadoop/Spark/Kafka-Pattern für bounded-latency Guarantees.
FS4	Adaptiv (Adaptive-Triggered)	Flush-Schwellwert wird adaptiv aus Workload-Häufigkeit via EWMA gelernt. Hohe Write-Rate → früher spülen (60% Schwelle); niedrige Rate → später spülen (95% Schwelle). Erste selbstadaptive Q2-Strategie.

Bausteine

Variante	Beschreibung	Quelle
EagerFlush	Pessimistische Write-Through-Policy: spült nach <i>jeder</i> <code>put()</code> -Operation vollständig (FullFlush). Niedrigste Latenz, aber maximale Spülfrequenz und höchster Durchsatzaufwand — klassisches Synchronous-Write-Pattern.	Lehrbuch Write-Through Policy / Code Baseline
LazyFlush	Optimistische Write-Back-Policy: spült <i>nie</i> automatisch, nur bei expliziter Eviction aus dem Buffer. Maximales Batching, minimale Spülfrequenz — klassisches Deferred-Write-Pattern. <code>should_flush()</code> gibt <i>immer</i> NoFlush zurück.	Lehrbuch Write-Back Policy / Code Baseline
WatermarkFlush	Schwellwert-gesteuerte Flush-Policy (Default 75% Kapazität). Iterable Aspekt mit 5 Varianten (50%, 65%, 75%, 85%, 95%) für Laufzeit-Permutation. Verwandt mit LSM-Tree Memtable-Trigger (O’Neil 1996).	LSM-Tree O’Neil 1996 (verwandt) / Code Implementation
TimedFlush	Zeit-Fenster-basierte Micro-Batching-Policy (iterable Aspekt: 10/100/1000/10000 ms). Inspiriert von Kafka (NetDB 2011) und Spark D-Streams (SOSP 2013) für Bounded-Latency-Guarantees. Unabhängig von Pufferfüllung.	Kafka NetDB 2011 / Spark SOSP 2013 / Code Implementation

Variante	Beschreibung	Quelle
AdaptiveLsm Flush	Selbstadaptive Watermark-Policy mit EWMA über Burst-Rate. Threshold interpoliert zwischen 60% (hohe Write-Rate) und 95% (niedrige Rate). Konzeptuell RocksDB Dynamic Leveled Compaction + O’Neil LSM 1996; EWMA-Logik ist CE-eigen. Erste Q2-Strategie mit <code>is_adaptive=true</code> .	RocksDB Dynamic Leveled Compaction / O’Neil LSM-Tree 1996 / Code Implementation

Beitragende Quellen: O’Neil et al. 1996 (LSM-Tree), Kafka (NetDB 2011), Spark D-Streams (SOSP 2013) [OCGO96, KNR11, ZDL⁺13].

D.18 T17 Persistenz-Ziel (Persistence-Target)

Diese Achse ist das *Persistenz-Ziel-Organ* (T17) des Lebewesens `SearchAlgorithm` (vgl. Abschnitt 3.2); sie ist der achtzehnte Organ-Slot (T17).

Die Achse macht das Ablage-Ziel der Struktur als eigenen Organ-Slot explizit, statt eine In-Memory-Annahme implizit mitzuführen. Im golden-Referenzkatalog ist sie auf ihren In-Memory-Vertreter festgelegt: siebzehn Achsen mit je zwei Ausprägungen und `persistence_target` mit einer ergeben $2^{17} = 131072$ Referenz-Identitäten je System-Permutation (vereinfachende, überholte Annahme zweier Auslegungen je Achse; Regressions-Detektor, nicht Obergrenze); jede `binary_id` führt gleichwohl das achtzehnte Segment `persistence_target=persistence_memory_only`. Die Mächtigkeit einer vollständigen Messreihe ist davon verschieden: sie ist keine statische Zahl, sondern wird je Profil aus dem Director-Walk gezählt, als Obergrenze aus den je Achse im XML-Experiment gewählten Auslegungen, die erst zur Ausführung bekannt ist (Abschnitt 3.4.6).

Bausteine

Variante	Beschreibung	Quelle
MemoryOnlyTarget	In-Memory-Ablage als Standard-Ziel; einziger aktivierter Vertreter des kanonischen Vermessungs-Katalogs. Kein Geräte-Schreibpfad (<code>has_device_writeback_path()=false</code>).	Code/Baseline
DiskWriteback Target	Platten-Rückschreibe-Ziel; als Baustein vollständig vorhanden (Typ, Registry, Unter-Achsen PT1–PT2), im kanonischen Katalog jedoch per Option abgeschaltet, sodass die Achse dort einelementig bleibt. Er ist zugleich der Träger der Organ-Meta-Meta-Achse <code>disk_io</code> (Abschnitt D.19), deren Stempel-Anhang nur an Kompositionen mit diesem Ziel tritt.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.19 Organ-Meta-Meta-Achse: Festplatten-E/A (`disk_io`)

Diese Größe ist die einzige Organ-Meta-Meta-Achse des Organ-Realms (Meta-Meta im Sinne der M3-Ebene, vgl. die Fußnote zu Beginn dieses Anhangs). Sie ist kein neunzehnter Kompositions-Slot: die Komposition bleibt bei achtzehn Organ-Achsen, und die Achse geht nie in die `binary_id` ein (`binary_`

`id=never, stage=ct, baustein_count=1`); die Organ-Registry führt sie als Kategorie `organ_meta_meta` neben den achtzehn Slots.

Die Achse führt die stempelnde Identität des Glieds `disk_io`: ihre Achsen-Code-Version (`1.0.0.c`) tritt als Klammer-Anhang an das Ende der Organ-Zeile (Glieder [3] des Tier-Fingerprints), und zwar ausschließlich für Kompositionen mit dem Persistenz-Ziel `DiskWritebackTarget` (Träger-Achse `persistence_target`, Träger-Wert `persistence_disk_writeback`); Kompositionen mit `MemoryOnlyTarget` bleiben byte-identisch, ihr Anhang ist leer und erzeugt kein leeres Klammer-paar. Die Form folgt dem System-Vorbild: die Organ-Zeile eines E/A-Trägers endet auf `; [disk_io=code@1.0.0.c]`. Die Bindung Strategie-Typ → Meta-Meta liegt in der ABI-Schicht, nicht im Achsen-Header; Träger der Selektion sind die achtzehn vorhandenen Slot-Aliase der Komposition. Für das Kanalwerk ist `disk_io` als eigener Gattungs-Kanal deklariert; der Kanal selbst ist als Konzept deklariert, aber nicht implementiert. Damit ist die Stempel-Mechanik der Achse implementiert, der Kanal-Bau nicht; ebenfalls nicht implementiert ist der Profil-Anschluss: der Profil-Parser liest den Registry-Block der Achse nicht, und eine Referenz auf sie in Organ-Position weist die Profil-Validierung ab, sodass die Achse per Experiment-Profil nicht wählbar und abgeschaltet ist; im golden-Referenzkatalog bleibt ihr Anhang leer (Abschnitt 3.4.1).

Dynamische Unter-Achsen

ID	Parameter	Beschreibung
<code>disk_io</code>	Bereitstellungs-Stufe der Festplatten-E/A	<code>stage=runtime, value_type=token, option_count=1</code> . Einzigste Option <code>staging: DiskWritebackTarget</code> misst ausschließlich die Rückschreib-Vorbereitung (kein Geräte-Schreibpfad, <code>has_device_writeback_path()=false</code>); ein Wert <code>device</code> folgt erst mit dem tatsächlichen Gerätepfad — die Registry enthält nur belegte Optionen. Die Familie spannt ihre gleichnamige Unter-Achse (Unter-Achsen-Label = Achsen-Label).

Bausteine

Variante	Beschreibung	Quelle
<code>DiskIoOrganMeta</code> <code>Meta</code>	Baustein <code>disk_io</code> , aktiviert (<code>enabled=true</code>); erfüllt das Konzept <code>OrganMetaMetaAxis</code> (Kategorie-Verzeichnis <code>organ_axes/organ_meta_meta/</code> , Wurzel-Vorbild ist die SIMD-Familie des System-Hubs); Achsen-Art <code>organ_meta_meta</code> , ausdrücklich keine Organ-Haupt-Achse (<code>static_assert</code>).	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigener Baustein).

D.20 System-Sphäre: Ziel-ISA (`target_isa`)

Diese Größe ist die System-Haupt-Achse `target_isa` — selbst eine Komplex-Achse (vgl. Abschnitte 3.4.1 und 3.4.2).

Die Achse spezifiziert die Ziel-CPU-Instruktionssatz-Architektur als Compile-Time-Konstante und ermöglicht damit Cross-Platform-Optimierungen für vier dominante Serverarchitekturen: x86-64 (Intel/

AMD), AArch64 (ARM), Power-ISA (IBM) und RISC-V (Open-Source). Jede Variante definiert eine hardware-spezifische SIMD-Baseline (SSE2, NEON, VSX, skalar) und charakterisiert damit die Vektorbeschleunigungspotenziale für Cache-Engine-Operationen wie `simd_field_sum` (Akkumulation von 32-bit-Wort-Feldern).

Statische Unter-Achsen

ID	Name	Beschreibung
IS1	Wortbreite (Word-Size)	CPU-Datenpfadbreite: 32-bit, 64-bit oder 128-bit. In der Cache-Engine auf 64-bit standardisiert für alle vier ISA-Profile.
IS2	CPU-Hersteller/Familie (Vendor)	Instruktionssatz-Familie: <code>intel_amd</code> (x86-64), <code>arm</code> (AArch64), <code>ibm</code> (Power-ISA), <code>open</code> (RISC-V). Bestimmt SIMD-Extension-Kompatibilität: SSE2/AVX2/AVX-512 (x86-64), NEON/SVE/SVE2 (AArch64), VSX (Power), RVV (RISC-V).
IS3	ZIH-Cluster-Zugehörigkeit	Produktionsmaschinen des ZIH der TU Dresden [Zen26a]: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids, x86-64), Capella (AMD EPYC 9334, x86-64), das Grace-Hopper-Superchip-Testsystem (72 Arm-Neoverse-V2-Kerne, AArch64; Testsystem mit begrenzter Verfügbarkeit [Zen26b]), N/A (RISC-V und Power: nicht unter den aktuellen HPC-Ressourcen des ZIH-Kompodiums). Ermöglicht Hardware-spezifische Experimente mit Produktionscluster-Profilen.

Über diese drei Entwurfs-Dimensionen hinaus führt die bau-treibende System-Registry fünf Unter-Achsen am Komplex-Wrapper: zwei übersetzungszeitliche (`target_isa_complex` und `scheduling`, je `stage=ct` mit `kind=fixed_enum_tuple`) sowie drei dynamische (`numa_node`, `page` und `core_class`). Die erste bündelt die benannten Rekombinationen der Zielmaschinen (`complex_count=2`) mit je drei festen Gliedern.

Rekombination	Glied	Deklaration
<code>prod1_zen5</code> (<code>isa=x86_64</code>)	<code>ram_frequency_mhz</code>	<code>declared=false</code> — keine deklarierte Zahl.
<code>prod1_zen5</code>	<code>cas_latency_cl</code>	<code>declared=false</code> — keine deklarierte Zahl.
<code>prod1_zen5</code>	<code>cpu_fabrication</code>	<code>declared=true, value=AuthenticAMD/26/68/0</code> (Hersteller/Familie/Modell/Stepping).
<code>prod2_alder_lake</code> (<code>isa=x86_64</code>)	<code>ram_frequency_mhz</code>	<code>declared=true, value=4800, declaration_source=dmidecode 2026-07.</code>
<code>prod2_alder_lake</code>	<code>cas_latency_cl</code>	<code>declared=false</code> — keine deklarierte Zahl.
<code>prod2_alder_lake</code>	<code>cpu_fabrication</code>	<code>declared=true, value=GenuineIntel/6/151/2.</code>

Notiz: Die Werte stammen aus `kDeclaredMachines`, das seinerseits den `<machines>`-Block der Anwender-XML spiegelt. `declaration_source` konserviert den festgeschriebenen Ursprung einer deklarierten Zahl als Notiz, nicht als Provenienz-Stufe; Laufzeit-Stufen stehen niemals in dieser XML. Die zweite übersetzungszeitliche Unter-Achse `scheduling` führt fünf feste Glieder. Sie hängt als `sub_axis` am `target_isa`-Komplex-Wrapper und ist nach der Entwurfsentscheidung dieser Arbeit zugleich Permutations-Achse des Planers — Unter-Achse und Permutations-Achse schließen sich nicht aus. Im Bau-Angebot ist sie `stage=ct` mit einem festen Fünfer-Tupel; die Permutation über den Planer ist als Entwurf festgeschrieben, aber nicht implementiert.

Glied	Registry-Wert	Beschreibung
worker_pool_layout	value_ ordinal=0	Aufteilung des Arbeiter-Pools über die Kerne.
simd_worker_count_limit	value=2	Obergrenze gleichzeitiger SIMD-Arbeiter.
hetero_core_dispatch	value_ ordinal=0	Verteilung auf heterogene Kerne (P-/E-Kerne).
co_routine_strategy	value_ ordinal=0	Koroutinen-Strategie der Auftrags-Abwicklung.
batch_granularity	value_ ordinal=0	Körnung der Stapel-Abarbeitung.

Notiz: Die Ordinale sind rohe Enum-Werte ohne übersetzungszeitliches Label-Vorbild; die Kennungs-Zeichenketten sind an die Zugriffs-Aufrufe von `concepts::scheduling_strategy.hpp` compile-gekoppelt.

Dynamische Unter-Achsen

ID	Parameter	Beschreibung
numa_node	NUMA-Knoten	stage=runtime, value_type=token, option_source=machine_resolved. Spiegel des Organ-Members <code>alloc_hw.numa_node</code> ; die zulässigen Werte sind die Knoten-Kennungen der konkreten Maschine und zur Bauzeit nicht erkennbar — daher eine deklarierte Options-Herkunft statt eines Options-Katalogs.
page	Seitengrößen-Freigabe	stage=runtime, value_type=token, option_source=machine_resolved. Die Freigabe der Seitengrößen (4k/2m/1g) als System-Capability; die Durchsetzung liegt organ-seitig im <code>AllocPageHint</code> -NTTP, diese Achse ist die System-Seite der Freigabe.
core_class	Kern-Klasse	stage=runtime, value_type=token, option_source=machine_resolved. Die Klasse des ausführenden Kerns (P-/E-Kern auf hybriden Prozessoren) als Ausführungs-Lokalität der Messung — Geschwister von <code>numa_node</code> und <code>page</code> , die die Speicher-Lokalität erfassen; abgegrenzt von <code>scheduling</code> , das die übersetzungszeitlich festgelegte Politik enthält: <code>core_class</code> sagt, ob und welche Kern-Klassen die Maschine führt. Zulässige Werte sind maschinen-aufgelöst; die getrennte Messung auf P- und E-Kernen ist damit im Katalog als System-Seite des Kern-Pinnings benannt, die Messung selbst leistet das Mess-System (Abschnitt 3.4).

Namenskollision (statisch geprüft): Die Unter-Achse heißt `page` und ist nicht `page_type`. `axis_01_page_type` ist der Baum-Knoten-PageKind der Organ-Welt und eine völlig andere Achse; der `static_assert` in `target_isa_sub_axes.hpp` hält das fest. Der Baustein-Katalog des Seitentyps steht im konservierten Angebots-Katalog „Build-PG“.

Bausteine

Variante	Beschreibung	Quelle
Amd64Isa	x86-64 / Intel 64 ISA (AMD/Intel Server+Desktop). Dominante Serverplattform mit SSE2-SIMD-Baseline (4 uint32-Lanes pro 128-bit-Register). Implementiert SSE2-Vektorisierung via <code>_mm_loadu_si128 + _mm_add_epi32</code> auf nativen x86-64-Hosts; skalarer Fallback auf Nicht-x86-Build-Hosts. ZIH-Präsenz [Zen26a]: Barnard (Intel Xeon Platinum 8470, Sapphire Rapids), Capella (AMD EPYC 9334), Alpha Centauri (AMD EPYC 7352), Romeo (AMD EPYC 7702).	Vendor-Spec (AMD 2000 / Intel fortlaufend)
Aarch64Isa	ARMv8-A 64-bit ISA (ARM Holdings). Wachsende Serverrelevanz (Apple M-Series, AWS Graviton, Ampere Altra). SIMD-Baseline: NEON (128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert NEON-Breite ohne ARM-Intrinsics auf x86-Build-Hosts); native NEON-Vektorisierung auf ARMv8-Hosts. ZIH-Präsenz: Grace-Hopper-Superchip-Testsystem (72 Arm-Neoverse-V2-Kerne; Testsystem mit begrenzter Verfügbarkeit [Zen26b]).	Vendor-Spec (ARM Ltd., ab 2011/2013)
PowerPcIsa	Power ISA (POWER9/10, ppc64le Little-Endian). IBM Power Systems (AC922, IC922, S1022). SIMD-Baseline: VSX (Vector-Scalar Extension, 128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert VSX-Breite ohne Power-Intrinsics auf x86-Build-Hosts). Relevant für HPC-Workloads mit NVLink-GPU-Anbindung (AC922). ZIH-Präsenz: im ZIH-Kompendium nur als abgeschaltetes Archiv-System (Power9) geführt, nicht unter den aktuellen HPC-Ressourcen.	Vendor/Standards-Spec (IBM/ OpenPOWER, v3.0 2017, v3.1 2020)
RiscVIsa	RISC-V RV64GC Open-Source ISA (RV64 General + Compressed). Wachsende Bedeutung (SiFive, Alibaba Xuantie, Andes, ETH Zürich Cores). SIMD-Baseline: <i>keine</i> (RVV ist optional, nicht Teil von RV64GC baseline). Cache-Engine-Implementierung: bewusst plain skalarer Pfad (nicht entrollt) – modelliert fehlende Vektoreinheit der Basis-ISA und erzeugt damit reale, langsamere Laufzeit gegenüber x86-64 und ARM. ZIH-Präsenz: im ZIH-Kompendium nicht gelistet.	Standards-Spec (RISC-V Intl.; UCB TR ab 2011)

Registry-Stand: Die bau-treibende System-Registry führt zwei aktivierte Bausteine — `x86_64` (nativ, ohne Triple- und March-Flags) und `aarch64` (Kreuz-Übersetzung mit `triple=-target aarch64-linux-gnu` und `march=-march=armv8-a`). Der oben aufgeführte Vier-ISA-Katalog bleibt als *Angebots*-Katalog erhalten: Power-ISA und RISC-V sind als Bausteine beschrieben, im Bau-Angebot aber nicht aktiviert.

Beitragende Quellen: —

D.21 System-Sphäre: Betriebssystem (operating_system)

Diese Größe ist die zweite System-Haupt-Achse und damit Teil des *Lebensraums* und keines Organs (vgl. Abschnitt 3.4.1); in der kanonischen, im Code geprüften Ordnung steht sie an Platz zwei von drei.

Die Haupt-Achse führt ausschließlich die *Klassen*-Identität: genau drei Betriebssystem-Familien (`baustein_count=3`), dazu `stage=ct` und `binary_id=never`. Die *Instanz* — Distribution, Release, Kernel und Patch-Stand — liegt dagegen vollständig in den drei Unter-Achsen; eine Distribution als vierte Ausprägung der Haupt-Achse einzutragen, verfehlte diesen Schnitt. Die vorhandenen Maschinen (Windows 11 und Windows Server 2022, die Linux-Docker-Matrix, zwei macOS-Maschinen) fallen vollständig in die drei Familien.

Dynamische Unter-Achsen

ID	Name	Beschreibung
os_version	Distributions-/ Release-Identität	Die Release-Identität der Betriebssystem-Instanz innerhalb ihrer Familie: unter Linux ID und VERSION_ID aus <code>/etc/os-release</code> , unter Windows die Produktlinie, unter macOS die Produktversion.
kernel	Kernel-Release	Das Kernel-Release der Instanz: unter Linux und macOS das <code>uname</code> -Release, unter Windows das NT-Kernel-Tripel.
build	Build- und Patch-Stand	Der Build- bzw. Patch-Stand der Instanz <i>einschließlich</i> des Update-Zustands (Entwurfsentscheidung dieser Arbeit: Der Update-Zustand ist Bestandteil der Build-Kennung.) Ein separates <code>update_zustand</code> ist damit ausdrücklich <i>keine</i> vierte Unter-Achse, sondern Bestandteil dieser.

Notiz: Alle drei sind `stage=runtime` mit `value_type=token` und `option_source=machine_resolved`: Sie haben keinen übersetzungszeitlichen Options-Katalog, weil ihre zulässigen Werte an der konkreten Maschine hängen und zur Bauzeit nicht erkennbar sind. Die Laufzeit-Erhebung ist in eigenen Probe-Dateien implementiert (`operating_system_probe_{linux,macos,windows}.hpp`); die Einhängung in die Registry deklariert nur die Optionen. Die drei Namen sind Teil der generierten Registry und damit festgeschrieben.

Bausteine

Variante	Beschreibung	Quelle
linux	Linux-Familie (<code>LinuxOperatingSystem, posix=true</code>). Deckt die gesamte Docker-Distributions-Matrix ab; die einzelne Distribution ist Sache der Unter-Achse <code>os_version</code> , nicht dieser Ausprägung.	Code/Baseline
windows	Windows-Familie (<code>WindowsOperatingSystem, posix=false</code>). Deckt Desktop- und Server-Produktlinien ab (Windows 11, Windows Server 2022).	Code/Baseline
macos	macOS-Familie (<code>MacosOperatingSystem, posix=true</code>). Deckt die vorhandenen Apple-Silicon- und Intel-Maschinen ab.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

Stempel-Neutralität: Die drei Unter-Achsen sind Laufzeit-Unter-Achsen und stehen deshalb niemals im Binary-Stempel. Ihre Einhängung ändert weder die Stempel-Zeilen noch den Bau-Versions-Suffix, und sie darf den Achsen-Code-Stand von `operating_system` nicht anheben (Prüfung gegen Versionssprung): Ein solcher Sprung verschöbe den SHA512 aller Neubauten gegen den Bestand und erzwänge genau den Voll-Neubau aller Alt-Binaries, den die Entwurfsentscheidung dieser Arbeit ausschließt.

D.22 System-Sphäre: Erweiterungs-Hardware (external_utils)

Diese Größe ist die dritte System-Haupt-Achse; sie ist ausschließlich der Sammel-Hub der Hardware-Meta-Meta-Achsen — SIMD- und Vektor-Erweiterungen, GPU, FPGA und NPU sowie externe Beschleuniger (vgl. Abschnitt 3.4.1).

Der Hub trägt *nur* System-Meta-Metas; das Last-Rahmenwerk gehört ausdrücklich nicht dazu. Weil die Meta-Meta-Achsen keine eigene Stempel-Zeile bekommen, sondern als Klammer-Anhang an das Ende der

jeweiligen Realm-Zeile treten, schließt dieser Hub den System-Realm ab: Der Anhang wird typgetrieben aus der Glied-Typliste des Hubs zusammengefasst (`ExternalUtilsHub` → `SimdExternalUtilsFamily`), die Ebene steckt in der Klammer-Tiefe und nicht im Namen. Im bau-treibenden Angebot führt die Achse `baustein_count=1`, `stage=ct` und `binary_id=never`.

Dynamische Unter-Achsen

ID	Name	Beschreibung
<code>simd</code>	Vektor-Erweiterungs-Stufe	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=3</code> . Die Familien-Kennung ist zugleich das Unter-Achsen-Label (Schreibweise <code>external_utils.simd=...</code>); die je Dialekt reflektierten Übersetzer-Flags liefert die Option, nicht die Familie.

Die drei Optionen der Unter-Achse mit ihren Flags je Übersetzer-Dialekt:

Option	Flag (gpp und clang)	Flag (msvc)
<code>no_extension</code>	— (leer)	— (leer)
<code>avx2</code>	<code>-mavx2</code>	<code>/arch:AVX2</code>
<code>avx512</code>	<code>-mavx512f</code>	<code>/arch:AVX512</code>

Notiz: `simd` materialisiert als Flag der Übersetzungs-Naht und erscheint damit im Bau-Versions-Suffix (vgl. Abschnitt 3.4.4) — niemals als Laufzeit-Typ-Umschalter und niemals in der `binary_id`.

Zur Unter-Achse gehört ein feiner Merkmals-Katalog `simd_feature_catalog` mit `count=23` Merkmalen, gestuft nach `tier`: vierzehn Merkmale der Stufe `avx512`, vier der Stufe `avx256`, drei Begleit-Merkmale (`companion`) und zwei skalare (`scalar`). Die Flags sind für `gpp` und `clang` durchgängig identisch.

Merkmal (cpuinfo)	Flag (gpp und clang)	Stufe (tier)
<code>avx512f</code>	<code>-mavx512f</code>	<code>avx512</code>
<code>avx512cd</code>	<code>-mavx512cd</code>	<code>avx512</code>
<code>avx512vl</code>	<code>-mavx512vl</code>	<code>avx512</code>
<code>avx512dq</code>	<code>-mavx512dq</code>	<code>avx512</code>
<code>avx512bw</code>	<code>-mavx512bw</code>	<code>avx512</code>
<code>avx512ifma</code>	<code>-mavx512ifma</code>	<code>avx512</code>
<code>avx512vbmi</code>	<code>-mavx512vbmi</code>	<code>avx512</code>
<code>avx512_vbmi2</code>	<code>-mavx512vbmi2</code>	<code>avx512</code>
<code>avx512_vnni</code>	<code>-mavx512vnni</code>	<code>avx512</code>
<code>avx512_bitalg</code>	<code>-mavx512bitalg</code>	<code>avx512</code>
<code>avx512_vpopcntdq</code>	<code>-mavx512vpopcntdq</code>	<code>avx512</code>
<code>avx512_vp2intersect</code>	<code>-mavx512vp2intersect</code>	<code>avx512</code>
<code>avx512_bf16</code>	<code>-mavx512bf16</code>	<code>avx512</code>
<code>avx512_fp16</code>	<code>-mavx512fp16</code>	<code>avx512</code>
<code>avx2</code>	<code>-mavx2</code>	<code>avx256</code>
<code>fma</code>	<code>-mfma</code>	<code>avx256</code>
<code>f16c</code>	<code>-mf16c</code>	<code>avx256</code>
<code>avx_vnni</code>	<code>-mavxvnni</code>	<code>avx256</code>
<code>gfni</code>	<code>-mgfni</code>	<code>companion</code>
<code>vaes</code>	<code>-mvaes</code>	<code>companion</code>
<code>vpclmulqdq</code>	<code>-mvpclmulqdq</code>	<code>companion</code>
<code>bmi2</code>	<code>-mbmi2</code>	<code>scalar</code>

Merkmal (cpuinfo)	Flag (gpp und clang)	Stufe (tier)
popcnt	-mpopcnt	scalar

Maschinen-Signaturen (*count=3*): Die Registry hält je Zielmaschine die Teilmenge der tatsächlich vorhandenen Merkmale fest — `prod1_zen5` (`host_isa=x86_64`) mit 22 Merkmalen, `prod2_alder_lake` mit 9 und `odroid_gracemont` mit 9. Nur die Zen-5-Maschine besitzt die `avx512`-Stufe; die beiden anderen enden bei `avx256` samt Begleit- und Skalar-Merkmalen.

Bausteine

Variante	Beschreibung	Quelle
<code>simd</code>	Vektor-Erweiterungs-Familie (<code>SimdExternalUtilsFamily</code> , <code>family_id=simd</code> , aktiviert). Einziges aktiviertes Glied des Hubs; die Familien für GPU, FPGA und NPU sind im Zuschnitt vorgesehen, aber nicht Teil des bau-treibenden Angebots.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.23 System-Sphäre: Äußere Komplex-Klammer (`build_target_complex`) und Bau-Werkzeugkette (`build_toolchain`)

Diese Größe ist der Befehls-Klammer-Wrapper über der festen Rekombination der drei System-Haupt-Achsen (`member_count=3`: `target_isa`, `operating_system`, `external_utils`) und schließt damit den System-Realm-Block ab (vgl. Abschnitt 3.4.2).

Als Klammer ist er ausdrücklich *keine* vierte Haupt-Achse: Die kanonische System-Achsen-Ordnung bleibt bei dreien. Drei statische Ausschluss-Prüfungen (`static_assert`) halten fest, dass `compiler`, `scheduling` und `load_framework` keine System-Haupt-Achsen sind. Das Last-Rahmenwerk ist auch kein *Glied* dieser Klammer: Es gehört dem Mess-Realm, während die System-Meta-Metas unter dem Hub `external_utils` geführt werden. Die Achse führt `stage=ct` und `binary_id=never`.

Statische Unter-Achsen

An der Klammer hängt die Unter-Achsen-Gruppe `build_toolchain` (`sub_axis_count=3`). Die Registry nennt die drei Glieder flach; die Verschachtelung darunter zeigt die tatsächliche Eltern-Kette.

Glied	Eltern-Bezug	Beschreibung
<code>compiler</code>	<code>build_target_complex</code>	<code>stage=ct</code> , <code>baustein_count=2</code> . Die Übersetzer-Wahl selbst; sie ist Glied der Unter-Achsen-Gruppe <code>build_toolchain</code> der äußeren Klammer.
<code>opt_level</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=5</code> . Optimierungsstufe je Übersetzer-Dialekt.
<code>atomic128</code>	<code>compiler</code>	<code>stage=runtime</code> , <code>value_type=token</code> , <code>option_count=2</code> . Freigabe der 128-Bit-Vergleichs-Tausch-Instruktion.

Notiz: Die Gruppe hängt an `build_target_complex` und nicht an `target_isa`, weil sie den *Bau* aus der Rekombination aller drei Haupt-Achsen beschreibt und nicht eine Eigenschaft der Ziel-ISA allein.

Bausteine

Variante	Beschreibung	Quelle
gcc	GNU-Übersetzer (GccCompilerAxis, Treiber g++-16, supports_fno_gnu_unique=true).	Code/Baseline
clang	LLVM-Übersetzer (ClangCompilerAxis, Treiber clang++-22, supports_fno_gnu_unique=false).	Code/Baseline

Angebot und Bau-Minimum: Die Treiber-Kennungen g++-16 und clang++-22 sind die Vorgabe-Treiber des Angebots (`driver_default` der Übersetzer-Bausteine); der Bau selbst fordert g++ $\geq$ 15.3 als Minimum und Standard der Messmaschinen und legt bewusst keine Version fest (neuere g++ zugelassen); die CI-Testzellen bevorzugen g++-15, wo vorhanden, und der clang-Vollbau läuft auf clang 22. Clang und MSVC haben kein Versions-Minimum.

Die fünf Optionen der Unter-Achse `opt_level` mit ihren Flags je Dialekt; die Spalte `ieee754_deterministic` weist aus, ob die Stufe die Gleitkomma-Semantik unangetastet lässt:

Option	Flag (gpp und clang)	Flag (msvc)	ieee754_deterministic
O0	-O0	/Od	true
O1	-O1	/O1	true
O2	-O2	/O2	true
O3	-O3	/O2	true
Ofast	-Ofast	/O2	false

Der Standard des Baus ist O2; O3 ist über die XML wählbar, und die Wahl geht als Feld in den Bau-Versions-Suffix ein.

Die zwei Optionen der Unter-Achse `atomic128`:

Option	Flag (gpp und clang)	Flag (msvc)
no_cx16	— (leer)	— (leer)
cx16	-mcx16	— (leer)

Bau-Versionskennung: `opt_level` und `simd` materialisieren als Flags der Übersetzungs-Naht und erscheinen als Segmente (`+opt=`, `+ext=`) im Bau-Versions-Suffix der Binary — einer geordneten Acht-Segment-Folge, in der ein leeres Glied schlicht kein Segment erzeugt (vgl. Abschnitt 3.4.4); `atomic128` materialisiert ebenfalls als Flag der Übersetzungs-Naht, steht aber nicht im Suffix, sondern als glied-eigenes Feld des Toolchain-Glieds im Fingerprint-Preimage (neben `vendoropt`, ADR-17 in Anhang E). In der `binary_id` stehen sie niemals.

Versions-Flag-Grammatik: Die Version eines jeden Achsen-Algorithmus ist einheitlich dreistellig; ein v-Präfix ist ausgeschlossen, Kurzformen sind verboten. Übersetzungszeitliche Prüfungen setzen die Form durch:

```
version := UINT '.' UINT '.' UINT [ '.' flag ]*
flag    := basis [ '{' sub [ '.' sub ]* '}' ] | companion
```

```

basis      := 'c' | 'g' | 'f' | 'n' | 'x128' | 'x256' | 'x512'
            | 'm64'
sub        := token [ '{' sub [ '.' sub ] * '}' ] (rekursiv)

```

Vor jedem Flag steht ein Punkt, auch vor dem ersten (1.0.0.c, nie 1.0.0c); die Basis hängt dabei direkt an ihrer öffnenden Klammer, ohne eigenen Punkt dazwischen (c{p.e}, nie c.{p.e}). Das Suffix *e* bedeutet *Efficiency Core* und ist ein Sub-Flag wie jedes andere unter der Basis *c*; die Basis *c* ohne Klammer ist semantisch *c{p}*; ein Sub-Flag für experimentell eingebrachte Prüflings-Algorithmen existiert nicht. Die Basen *c*, *g*, *f* und *n* benennen die Ziel-Hardware-Klasse (CPU, GPU, FPGA und NPU, wobei nur *c* produziert wird und die drei anderen reserviert sind), *x128*, *x256* und *x512* die drei Sichten auf dieselbe Vektor-Registerdatei (XMM, YMM, ZMM), und *m64* die Medien-Basis der MMX/3DNow!-Familie auf den x87-aliasierten MM-Registern, deren Sub-Flags die Katalogprüfung nur unter dieser Basis passieren lässt (1.0.0.c.m64{mmx.mmxext.3dnow}, nie als bloße Token ohne Basis). Beispiel-form: `memory_layout=SoaMemoryLayout@1.0.0.c{p.e}.x512{f.vl.bw.dq}.gfnl`.

D.24 Mess-Sphäre: Mess-Kategorien (measurement_category)

Diese Größe gehört zum dritten Realm — den Mess-Achsen, der Ausstattung des *Labors* (vgl. Abschnitt 3.4.1).

Sie führt `stage=ct`, `binary_id=never` und `baustein_count=16`. Ihre Werte manifestieren sich als Spalten der Ergebnis-Tabellen und als Einstellungs-Etikett eines Laufs, niemals in der Binary-Identität. Im Kapitel-Modell sind die sechzehn Kategorien die *Unter*-Achse des Mess-Toolings; die Mess-Registry führt sie zusätzlich als eigenen Angebots-Block, weil das Tooling seine Kategorien nicht selbst erfindet, sondern aus diesem Katalog wählt. Die Spalte `regime_ordinal` trennt das zeit- und observer-basierte Regime (0) vom zählerbasierten PMC-Regime (1). Die sechzehn Kategorien sind der Angebots-Katalog der Kategorien-Achse; die Zahl der je Achsen-Schnittstelle und Parametersatz erhobenen Mess-Quellen ist davon verschieden und nicht Gegenstand dieses Katalogs (Abschnitt 3.4).

Bausteine

Kategorie	category_ordinal	Regime (regime_ordinal)
CLU	0	0 — Zeit/Observer
CACHE_MISS_L1	1	1 — PMC
CACHE_MISS_L2	2	1 — PMC
CACHE_MISS_L3	3	1 — PMC
DTLB_MISS	4	1 — PMC
MEMORY_FOOTPRINT	5	0 — Zeit/Observer
BRANCH_MISS	6	1 — PMC
IPC_CPI	7	1 — PMC
LATENCY_MEAN	8	0 — Zeit/Observer
LATENCY_P50	9	0 — Zeit/Observer
LATENCY_P95	10	0 — Zeit/Observer
LATENCY_P99	11	0 — Zeit/Observer
LATENCY_P999	12	0 — Zeit/Observer
THROUGHPUT	13	0 — Zeit/Observer
ENERGY_J	14	1 — PMC

Kategorie	category_ordinal	Regime (regime_ordinal)
FILL_BUFFER_OCCUPANCY	15	0 — Zeit/Observer

Regime-Zweiteilung: Neun Kategorien liegen im Zeit- und Observer-Regime (CLU, MEMORY_FOOTPRINT, die fünf Latenz-Kategorien, THROUGHPUT, FILL_BUFFER_OCCUPANCY), sieben im PMC-Regime (die drei Cache-Miss-Stufen, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J). Die Aufteilung ist deckungsgleich mit der Zweiteilung „neun/sieben“ aus Abschnitt 3.4.1.

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.25 Mess-Sphäre: Mess-Tooling/Collector (collector)

Diese Größe ist die Instrumenten-Achse des Mess-Realms: Sie hält fest, mit welchem Werkzeug eine Kategorie erhoben wird (vgl. Abschnitt 3.4.1).

Sie führt `stage=ct`, `binary_id=never` und `baustein_count=3`. Jeder Baustein trägt sein Regime und die Menge der Kategorien, die er bedient; die Vereinigung der drei Mengen deckt zehn der sechzehn Kategorien ab. Von den übrigen sechs werden die vier Latenz-Perzentil-Kategorien (LATENCY_P50 bis LATENCY_P999) nicht von einem Erhebungs-Werkzeug, sondern von der nachgelagerten Auswertung bedient (nearest-rank über den Rohwerten, ergänzt um das HDR-Histogramm, Abschnitt 4.5); MEMORY_FOOTPRINT und FILL_BUFFER_OCCUPANCY sind im Angebots-Katalog beschrieben, aber keinem Erhebungs-Werkzeug zugeordnet.

Bausteine

Variante	Regime	Bediente Kategorien
WallClockSystemAxis	0	LATENCY_MEAN, THROUGHPUT
ObserverSnapshotSystemAxis	0	CLU
PmcSystemAxis	1	CACHE_MISS_L1, CACHE_MISS_L2, CACHE_MISS_L3, DTLB_MISS, BRANCH_MISS, IPC_CPI, ENERGY_J

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.26 Mess-Sphäre: Telemetrie-Bausteine (konservierter Katalog)

Diese Größe ist eine *zweigeteilte* Haupt-Größe — Sweep-Unter-Achse des Mess-Toolings im Planer *und* Übersetzungszeit-Hauptgröße im Builder (vgl. Abschnitt 3.4.1). Ihre Baustein-Katalogisierung bleibt hier als konservierter Katalog erhalten.

Definiert Strategien zur Messung von Index-Operationen (Insert, Lookup, Erase) mit verschiedenen Overhead-Profilen. Unterstützt Density-Tracking für adaptive Knotentyp-Wahl, globale Insert-Counter für Durchsatz-Messungen, HDR-Histogramme für Latenz-Quantile und leaf-only-Counter zur Vermeidung von Cache-Line-False-Sharing in den Messzyklen.

Statische Unter-Achsen

ID	Name	Beschreibung
TM1	Scope (Geltungsbereich)	Gültigkeitsbereich der Zählerverwaltung: leaf-only (nur Blatt-Knoten), per-node (alle Knoten), oder global (einzelner Akkumulator für die ganze Struktur). Compile-Time-Wahl, beeinflusst Overhead durch Konkurrenz-Druck auf gemeinsame Cache-Linien.
TM2	Metrik-Typ (Messgröße)	Art der gemessenen Metrik: counter (Zähler für Ereignisse), histogram (Verteilung mit Quantilen), oder density (Slot-Belegung pro Knoten). Beeinflusst den Speicher-Footprint und die Mess-Granularität.
TM3	Overhead-Level (Messlast)	Größenordnung des Mess-Overheads: low (atomare Operationen), medium (per-node-Datenstrukturen), high (Histogramm-Logging). Bestimmt die Eignung für verschiedene Workload-Szenarien (Benchmarking vs. Produktion).

Bausteine

Variante	Beschreibung	Quelle
LeafOnly Counter	Zähler <i>nur</i> in Blatt-Knoten (vermeidet Cache-Line-Ping-Pong durch Verzicht auf Updates innerer Knoten). Niedriger Overhead bei leaf-only Scope. Entspricht dem kohärenz-schonenden Zähler-Konzept aus Kuehn DaMoN 2023, obwohl das Original-Paper kein Leaf-Counter-Algorithmus ist — CE-Baseline für False-Sharing-Vermeidung.	P28 (Kuehn DaMoN 2023, Kontext; Code/Baseline)
DensityTracker	Density/Slot-Occupancy-Tracking je Knoten für adaptive Knotentyp-Wahl (4→16→48→256). Kontextuelle Telemetrie zur ART-Struktur (Leis ICDE 2013), misst Slot-Belegung aller Knoten. Höherer Overhead als LeafOnly, liefert aber Adaption-Signal für dynamische Rebalancing-Entscheidungen.	P01 (ART/Leis ICDE 2013, Kontext; Code/Baseline)
InsertCounter	Globaler atomarer Insert-Zähler mit minimalem Overhead (1 atomic increment pro Insert). Kontextuelle Telemetrie zur HOT-Struktur (Binna SIGMOD 2018), global Scope. Ausreichend für Durchsatz-Mess-Reihen, keine Latenz-Granularität.	P02 (HOT/Binna SIGMOD 2018, Kontext; Code/Baseline)
Latency Histogram	HDR-Histogramm zur Lookup-Latenz-Verteilung (p50/p95/p99 Quantile). Basiert auf HdrHistogram (Gil Tene), CC0-1.0 Lizenz. Höchster Overhead (pro Lookup), liefert aber vollständige Latenz-Statistik für SLA-orientierte Messreihen und Messreihen mit wissenschaftlichem Anspruch. Einziger is_original-Kandidat der Achse.	HdrHistogram (github.com/HdrHistogram/HdrHistogram_c, CC0-1.0, is_original)
Probability-Hints	Wahrscheinlichkeits-Hinweise der cache-oblivious Bender-Linie (P16/P17): verteilungs- bzw. wahrscheinlichkeitsbezogene Belegungs-Hinweise, die als Eingabe der Layout-Entscheidungen dienen. Paper-Baustein der Achsen-Sezierung (Rekonstruktions-Ziel dieser Arbeit, keine CE-Baseline).	P16/P17 (Bender 2000/2005) [BDFC00, BDFC05]

Die CRTP-Organ-Registry der Telemetrie (`AllTelemetries`) führt genau die vier Strategien der ersten vier Zeilen; daneben enthält der Code die `vtable`-Familie `ITelemetryStrategy` mit sechs Arten (Per-Node-Zähler, Leaf-Only-Zähler, gesampelter Leaf-Only-Zähler, retroaktive Aggregation, Pfad-Lese-Zähler P26 und die Probability-Hints dieser Arbeit), die keine Organ-Registry ist und von der Telemetrie-Engine (`c05_telemetry_engine`) und ihren Tests konsumiert wird. Zwei weitere Telemetrie-Ziele mit ZIH-Bezug — eine OTF2-Trace-Anbindung an das Leistungsanalyse-Werkzeug Vampir [KBD⁺08] und eine Anbindung an den zentrumsweiten Job-Monitor PIKA [DWKN20] — sind als Marker-Typen eines nur von Tests konsumierten, vom Bau isolierten Varianten-Katalogs vorgesehen und nicht Teil

des bau-treibenden Angebots; die Rekonstruktion des VAMPIR-Speichertyp-Entwurfs (A24) ist davon unabhängig.

Beitragende Quellen: HdrHistogram (Gil Tene) [Ten12], P16/P17 (Bender) [BDFC00, BDFC05].

Verwandte Literatur: P01, P02, P28 [LKN13, BZP⁺18, Kue23].

D.27 Mess-Sphäre: Last-Rahmenwerk (`load_framework`) — Meta-Meta-Achse

Diese Größe ist die Meta-Meta-Haupt-Achse des Mess-Realms und steht deshalb an dessen Ende (vgl. Abschnitt 3.4.1).

Sie führt `category=measurement_meta_meta`, `stage=ct`, `binary_id=never` und `baustein_count=1`. Sie ist die Meta-Meta-Haupt-Achse des Planers im Mess-Realm; unter den Hub der System-Meta-Metas gehört sie ausdrücklich nicht. Im Stempel erhält sie *keine* eigene Meta-Meta-Zeile: Ihre Kennung tritt als dynamischer Klammer-Anhang an das *Ende* der bestehenden Mess-Realm-Zeile. Der Speiseweg ist hier eintragsgetrieben — die gewählte Kennung samt Version kommt aus der Rahmenwerk-Registry des Mess-Realms und steht deshalb nicht im Typ, anders als beim typgetriebenen System-Hub.

Dynamische Unter-Achsen

Die Registry führt sieben Sweep-Dimensionen (`resource_control_version=1`); sechs davon sind `stage=runtime`, die siebte (`work_mode`) ist `stage=plan`.

Dimension	Herkunft (source)	Wert-Typ und Bedeutung
<code>workload</code>	<code>measurement:load_framework</code>	token — das Lastprofil; Einzelquelle ist das Unter-Achsen-Etikett des Last-Rahmenwerks selbst.
<code>thread_count</code>	<code>resource_control_pod</code>	uint — Zahl der Arbeiter-Fäden.
<code>prefetch_distance</code>	<code>resource_control_pod</code>	uint — Vorhol-Abstand.
<code>pool_budget_bytes</code>	<code>resource_control_pod</code>	uint — Speicher-Budget des Pools.
<code>batch_size</code>	<code>resource_control_pod</code>	uint — Stapelgröße je Operation.
<code>inline_threshold_bytes</code>	<code>resource_control_pod</code>	uint — Schwelle der Inline-Ablage.
<code>work_mode</code>	<code>measurement:run_methodology</code>	token — die Ablauf-Methode eines Laufs (<code>build, measure, compare, release</code>), <code>stage=plan</code> ; Einzelquelle ist die übersetzungszeitliche Registry der Run-Methodik, kein handgeschriebener Wert.

Vermerk zur Nicht-Erhebung: Die Wiederholungszahl eines Laufs steht bewusst *nicht* in dieser Liste: sie ist eine reine Etikett-Dimension ohne übersetzungszeitliche Entsprechung und wird erst nach ihrer Typisierung reflektierbar; ein manueller Eintrag wäre ein nicht belegter Wert — die Registry führt stattdessen eine explizit kodierte Nicht-Erhebung. Die Ablauf-Methoden dagegen sind typisiert: es sind *vier* — Bau-, Mess-, Vergleichs- und Release-Durchführung (`build, measure, compare, release`) —, gestützt auf eine übersetzungszeitliche Registry ohne Laufzeit-Verzweigung; die Vergleichs-Durchführung besitzt die Wählbarkeit und liest das Messwertlager, ohne selbst Referenzwerte zu erheben. Sie stehen als Dimension `work_mode` in der Liste.

Bausteine

Variante	Beschreibung	Quelle
ycsb	Das YCSB-Lastprofil-Rahmenwerk; aktiviert, mit dem Unter-Achsen-Etikett <code>workload</code> . Einziger aktivierter Baustein der Achse.	Code/Baseline

Beitragende Quellen: — (CE-Baseline, eigene Bausteine).

D.28 A-Korpus Allokator-Korpus (A01–A23 sowie A24)

Die Allokator-Achse dokumentiert 26 Speicherverwaltungs-Strategien (Kennungen A01–A21, die drei Standardbibliotheks-Baselines A22a–A22c, A23 sowie A24), die das dynamische Speichermanagement in Such-Workloads beeinflussen. Sie umfasst klassische Lehrbuch-Allokatoren (Buddy, Slab), industrielle Standards (dlmalloc, jemalloc, TCMalloc, mimalloc, snmalloc), spezialisierte Varianten (NUMA-aware, PIM-Malloc, Cache-Aware), moderne gehärtete Implementierungen (StarMalloc) und eine Speichertyp-Virtualisierung nach dem VAMPIR-Entwurf (A24). Dabei bilden A01–A23 den Literatur-Korpus der 23 Arbeiten aus Abschnitt 3.2 mit je einem Allokator-Profil (Tabelle 3.3), während A24 die aus dem SOTA-Profil P33 rekonstruierte Allokator-Front ohne eigenes Allokator-Profil ist. Alle 26 Wrapper erfüllen das `AllocatorStrategy`-Konzept und ermöglichen systematische Messung des Speichermanagement-Einflusses auf Suchbaum-Performance (F15-Operativität). Das aktivierte Inventar dieser Bau-Konfiguration umfasst drei Wrapper (siehe den Registry-Stand-Vermerk im Abschnitt T6).

Statische Unter-Achsen

Die Unter-Achsen AA1–AA7 sind im Abschnitt T6 definiert; die folgende Tafel ordnet ihnen die Korpus-Beispiele zu.

ID	Name	Beschreibung
AA1	Free-List-Topologie	Struktur der freien Speicher-Blöcke: Per-Page-Sharding (mimalloc), Per-Thread-Caching (snmalloc, rpmalloc, TCMalloc), Buddy-Baum (Buddy-Allocator), Single-Global-Free-List (dlmalloc), Per-Heap-Locks (Hoard).
AA2	Size-Class-Schema	Klassifizierung von Block-Größen: Slab-pro-Objektgröße (Slab), dlmalloc Bins, jemalloc Size-Classes, scallocc Spans, Buddy-Power-of-2.
AA3	Thread-Lokalität	Grad der Thread-Isolation: tcmalloc thread-local-caches, rpmalloc per-thread-spanning, snmalloc message-passing, Hoard per-heap-locks.
AA4	Synchronisation	Concurrency-Mechanismus: Lock-Free CAS (Michael, LRMalloc), Per-Heap-Locks (Hoard), Wait-Free (Crystalline), Mutex-basiert.
AA5	Allokations-Richtlinie	Routing von Allokationsanfragen: NUMA-origin-aware (NUMAalloc), Cache-Set-aware (CAMA), NUCA-aware (TCMalloc-Warehouse), PIM-aware (PIM-Malloc), NFP-Speichertyp-Virtualisierung (VampirNfp).
AA6	Rückgewinnung	Freigabe gelöschter Blöcke: Magazine-Cache (Slab), Page-Heap-Release (TCMalloc), Lock-Free-Reclamation (LRMalloc), Deferred-Free (mimalloc).
AA7	Fragmentierungsstrategie	Vermeidung von Fragmentierung: dlmalloc Coalescing, Object-Coloring (Slab), Buddy-Splitting, Page-Local-Sharding (mimalloc), Span-Pooling (scallocc).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierung. Drei-Ebenen-Struktur: Thread-lokale Heaps, globale Heap für Cross-Thread-Migration, False-Sharing-Reduktion durch segregierte Allocations-Anfragen.	A01 [BMBW00] (Berger/McKinley/ Blumofe/Wilson, ASPLOS 2000, 10.1145/378993.379232) + Code/ Baseline
SlabAllocator (A02)	Object-Caching Kernel-Allocator. Slab-pro-Objektgröße mit Magazine-Cache pro CPU (Bonwick 1994/2001). Klassische Solaris-Kernel-Implementierung für vorallokierte Objekt-Pools.	A02 [Bon94] (Bonwick, USENIX 1994) + Code/Baseline
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allocator nach Michael 2004. Hazard-Pointer-geschützte Lock-Free-Datenstrukturen, LGPL-Re-Implementierung mit IBM-Patent-Hintergrund.	A03 [Mic04b] (Michael, PLDI 2004, 10.1145/996841.996848) + Code/ Baseline
MimallocAllocator (A04)	Free-List-Sharding (3 Free-Lists pro Page). Microsoft Research 2019: deferred_free Hooks, zeitlicher Takt, Page-lokale Sharding. is_original: MIT-Direktbindung.	A04 [LZM19] (Leijen/Zorn/de Moura, APLAS 2019, 10.1007/ 978-3-030-34175-6_13) + is_original (github.com/microsoft/mimalloc, MIT)
JemallocAllocator (A05)	Arena-based Size-Class-Allocator. Evans 2006 / Free BSD. Pro-CPU Arenas mit Tcache-Layer für Thread-lokale Caches. Facebook-Standard. is_original: BSD-2-Clause-Direktbindung.	A05 [Eva06] (Evans, BSDCan 2006, papers.freebsd.org/.../jemalloc) + is_ original (github.com/jemalloc/jemalloc, BSD-2-Clause)
TCMallocAllocator (A06)	Thread-Cache + Central-Cache + Page-Heap. Google gperftools 2005. Später TEMERAIRE (OSDI 2021) mit Warehouse-Scale-Optimierungen.	A06 [HKT ⁺ 21] (TCMalloc 2005; TEMERAIRE, OSDI 2021) + Code/ Baseline
SnmallocAllocator (A07)	Message-Passing-Allocator. Paul Liétar et al. ISMM 2019: Free-Operationen via Lock-Free-Queue an Allocating-Thread, Cross-Thread-Lock-Contention-Vermeidung. is_original: MIT-Direktbindung.	A07 [LDPB19] (Liétar, ISMM 2019, 10.1145/3315573.3329980) + is_ original (github.com/microsoft/snmalloc, MIT)
ScallocAllocator (A08)	Span-basiert, Virtual-Spans-Technik. Aigner/Kirsch/Lippautz/Sokolova OOPSLA 2015. Multicore-skalierbar mit niedriger Fragmentierung.	A08 [AKLS15] (Aigner et al., OOPSLA 2015) + Code/Baseline
NUMAAllocator (A09)	NUMA-Origin-Aware Allocator. UTSASRG ISMM 2023. Knoten-Hinweis für lokale NUMA-Allokation, Multi-Knoten-Systeme optimiert.	A09 [YZZ ⁺ 23] (Yang et al., ISMM 2023) + Code/Baseline
RPMallocAllocator (A10)	Per-Thread Span-Caching. Mattias Jansson 2017, Open-Source-Implementierung ohne Veröffentlichung. Sonderfall: Pflicht-Init (rpmalloc_initialize, rpmalloc_thread_initialize). is_original: Public-Domain/MIT-Direktbindung.	Code/Baseline (github.com/mjansson/ rpmalloc, Public-Domain/MIT) + is_ original
LRMallocAllocator (A11)	Lock-Free + Hazard-Pointers. Leite/Rocha VECPAR 2018 (LNCS 11333). Moderner Lock-Free-Allocator mit ABA-Protection. is_original: MIT-Direktbindung.	A11 [LR18] (Leite/Rocha, VECPAR 2018, 10.1007/978-3-030-15996-2_17) + is_original (github.com/ricleite/ lrmalloc, MIT)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator. Herter/Backes/Hauptenthal/Reineke ECRTS 2011 (Saarland). Vorhersagbare Cache-Behavior, keine öffentliche Code-Referenz.	A12 [HBHR11] (Herter et al., ECRTS 2011) + Code/Baseline
StarMalloc Allocator (A13)	Formal Verified Hardened Allocator. Inria-Prosecco (Reitz/Fromherz/Protzenko et al.) OOPSLA 2024. F*/Steel-formale Verifikation, Apache-2.0-Lizenz.	A13 [RFP24] (Reitz et al., OOPSLA 2024) + Code/Baseline

Variante	Beschreibung	Quelle
TCMallocWarehouse Allocator (A14)	TCMalloc-Warehouse / Hyperscale. Google Cloud OSDI 2021 / ASPLOS 2024. Hupage-aware Variante, Produktvariante ohne eigenständiges Artefakt.	A14 [HKT ⁺ 21, ZGV ⁺ 24] (Hunter OSDI 2021 / Zhou ASPLOS 2024) + Code/Baseline
HMallocAllocator (A15)	Hybrid, Scalable, Lock-Free Memory Allocator. Li/Yao/Tang/Lin ICPADS 2019. Hybrid-Ansatz für Skalierbarkeit, keine öffentliche Code-Referenz.	A15 [LYTL19] (Li/Yao/Tang/Lin, ICPADS 2019, 10.1109/ICPADS47876.2019.00064) + Code/Baseline
PIMMalloc Allocator (A16)	Processing-In-Memory Allocator. VIA-Research HPCA 2026 (arXiv:2505.13002). Speicherverteilung über Rechenknoten, PIM-Hardware-optimiert.	A16 [VIA26] (VIA-Research, HPCA 2026, arxiv.org/abs/2505.13002) + Code/Baseline
Crystalline Reclamation (A17)	Wait-free bzw. lock-/wait-free Safe-Memory-Reclamation-Familie für nicht-blockierende Datenstrukturen. Kein eigenständiger malloc/free-Allokator, sondern Reclamation-Policy.	A17 [NR21, NR24] (Nikolaev/Ravindran, DISC 2021 / PLDI 2024) + Code/Baseline
ExgenAllocator (A18)	Single-Threaded Specialized Allocator. UT Austin IEEE CAL 2025 (arXiv:2510.10219). „Old is Gold“-Optimierungen für Einthread-Workloads, keine öffentliche Code-Referenz.	A18 [LJY25] (UT Austin, IEEE CAL 2025, arxiv.org/abs/2510.10219) + Code/Baseline
BuddyAllocator (A19)	Buddy System (Power-of-2-Splitting). Knuth TAOCP Vol.1 (1968), CACM 1965. Klassischer Lehrbuch-Allokator, Binärbaum-Splitting ohne OSS-Code-Referenz.	A19 [Kno65] (Knowlton, CACM 1965, 10.1145/365628.365655) + Code/Baseline
DlmallocAllocator (A20)	Doug Lea Malloc (Bins-based). Lea 1987–2012, Public-Domain (CC0). Klassiker seit 1987, Bins-Size-Class-Schema, älteste produktive Implementierung. is_original: Public-Domain-Direktbindung.	A20 [Lea96] (Lea, gee.cs.oswego.edu/.../malloc.html, 1996) + is_original (github.com/ennorehling/dlmalloc, Public-Domain/CC0)
PtMalloc2 Allocator (A21)	ptmalloc2 (glibc malloc). Wolfram Gloger 1990er-2000er. LGPL-2.1+ Copyleft, Linking blockiert (no is_original). Standard-Basis für glibc-basierte Systeme.	Code/Baseline (malloc.de/en/) + License restriction
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis), portabler Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen ohne Vendor-Linking.	Code/Baseline (C++ Standard Library) + Baseline
PmrResource Allocator (A22b)	std::pmr::memory_resource. WG21 N3916 (2014 → C++17). Polymorphic Memory Resources Standard-Bibliothek, Fallback auf new/delete-Allokator.	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + Baseline
PoolResource Allocator (A22c)	std::pmr::unsynchronized_pool_resource. WG21 N3916 (C++17). Eigener Size-Class-Pool, F15-operativ (erste Wrapper mit eigenem Speicherhaltungsverhalten ohne Vendor-Linking).	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + is_operative
VmemMagazines Allocator (A23)	Vmem + Magazines (Slab-Erweiterung). Bonwick 2001 USENIX ATC. Per-CPU-Magazine für Skalierung, Kernel-Klassiker ohne öffentliche Code-Referenz.	A23 [BA01] (Bonwick, USENIX ATC 2001, usenix.org/.../bonwick.html) + Code/Baseline

Variante	Beschreibung	Quelle
VampirNfp Allocator (A24)	Speichertyp-Virtualisierung nach dem VAMPIR-Entwurf (Virtualized Non-Functional Memory Properties for Data-Pipeline Scheduling): Allokator-Front über einer NFP-etikettierten Speicherstufe (Ziel DRAM) mit offengelegtem NFP-Deskriptor (bandwidth_class, latency_class, capacity); strukturelle Rekonstruktion des baubaren Teils, keine Kostenformel, kein Scheduling, keine Energie-Metrik; standardmäßig abgeschaltet.	A24 [BSB ⁺ 23] (VAMPIR-Poster, SOS 2023; TU Dresden) + Code/ Baseline (is_original=false)

Beitragende Quellen: je Allokator A01–A24 einzeln in der Quellenspalte der Bausteine-Tabelle nachgewiesen.

D.29 Achsen-Inventar aus den generierten Registries

Die folgende Tabelle wird nicht von Hand gepflegt: der Anhang-Generator (Code/08_appendix_generator) leitet sie aus den drei generierten Achsen-Registries des Codes ab — Organ-, System- und Mess-Realm — und schreibt sie, sofern der Lauf mit mindestens einer der drei Registry-Optionen (`--organ-registry`, `--system-registry`, `--measurement-registry`) aufgerufen wird, je Sprache nach `anhang/<lang>/tabellen/`. Sie ist damit eine Ableitung aus der übersetzungszeitlichen Reflektion und kein zweiter, gepflegter Stand neben den Registries. Enthalten die Registries keine Achse, entsteht bewusst keine Datei; eine leere Tabelle wird nicht synthetisch erzeugt.

(Das Achsen-Inventar liegt für diesen Stand nicht vor; ein Generatorlauf mit Registry-Optionen erzeugt es.)

D.30 Konservierte Entwürfe

Die drei folgenden Angebots-Kataloge sind konservierte Entwürfe: lebender Code mit eigenen Registries, der außerhalb des bau-treibenden Angebots der drei Achsen-Registries steht — der SIMD-Katalog Build-SIMD unter der Erweiterungs-Hardware, der Seitentyp-Katalog Build-PG außerhalb der achtzehn Kompositions-Slots und die Plattform-Hardware-Strategie Build-HW, die die System-Registry ausdrücklich ausschließt. Sie bleiben hier vollständig aufgeführt — mit Zweck, Unter-Achsen, Bausteinen und Quellen — und sind vom Angebot der Registries zu unterscheiden, das die vorangehenden Abschnitte dokumentieren (vgl. Abschnitt 3.4.1).

D.30.1 Build-SIMD: Vektor-/Accelerator-Erweiterungs-Achse (Hardware-Beschleuniger)

Konservierter Angebots-Katalog außerhalb des bau-treibenden Angebots: Die Achse `axis_09b_simd_extension` ist ein Angebots-Katalog unter der Haupt-Achse `external_utils`, deren Unter-Achse `simd` die bau-treibende Wahl trägt; ihre acht Bausteine und die Unter-Achsen SE1–SE3 bleiben vollständig aufgeführt.

Die Achse `axis_09b_simd_extension` modelliert die verfügbaren SIMD-/Accelerator-Erweiterungen der CPU-Instruktionssatz-Architektur als Unter-Achse zu `axis_09` (Haupt-ISA). Sie ermöglicht Compile-Time-Auswahl zwischen verschiedenen Vektor-Erweiterungen (SSE2/AVX2/AVX-512 für x86; NEON/SVE2 für ARM; RVV für RISC-V) sowie Baseline (Skalar) und GPU-Beschleunigung (CUDA), mit rückwärts-kumulativer Schichthierarchie und hardwarespezifischer Topologie (SIMD-Units pro Sockel, P/E-Core-Zugriff).

Statische Unter-Achsen

ID	Name	Beschreibung
SE1	Vector-Width (Vektorbreite)	Compile-Time-bekannte maximale Vektorbreite der SIMD-Extension in Bits: 0 (keine), 128, 256, 512, oder -1 (skalierbar). Bestimmt Lane-Breite und Parallelisierungsgrad.
SE2	Accelerator-Type (Beschleuniger-Typ)	Kategorisiert die Beschleuniger-Klasse: none (Baseline), simd (Vektor-SIMD), matrix (Matrix-Engine), gpu (GPU), npu (Neural Processing Unit). Beeinflusst die Komposition mit der Hardware-Topologie (<code>axis_12</code>).
SE3	Compat-Family (Kompatibilitäts-Familie)	Constraint zur Haupt-ISA (<code>axis_09</code>): none (universell), x86, arm, riscv, cuda. Sichert Compile-Time-Validierung: z.B. <code>Sse2SimdExtension</code> kompatibel nur mit <code>Amd64Isa</code> .

Bausteine

Variante	Beschreibung	Quelle
<code>NoSimdExtension</code>	Baseline-Variante ohne SIMD-Beschleunigung. Skalare Fallback-Strategie für alle Permutationen ohne Accelerator. Universell kompatibel zu allen Haupt-ISAs (x86/ARM/RISC-V/PowerPC). Pflicht-Vergleichspunkt.	Code/Baseline
<code>Sse2SimdExtension</code>	Intel SSE2 128-bit Vektor-Extension (x86_64 ABI-Pflicht seit 2003). Nur kompatibel mit <code>Amd64Isa</code> . Baseline auf x86_64; bietet 4× uint32-Lanes/Iteration. Standard auf Cluster-Knoten Barnard/Capella.	Intel ISA-Erweiterungs-Referenz 319433 [Int26]
<code>Avx2SimdExtension</code>	Intel AVX2 256-bit Vektor-Extension (Haswell+ 2013). Nur x86_64. Rückwärts-kumulativ: umfasst alle SSE+AVX. Default-Optimierungs-Stufe für Server-DBMS. 2 AVX2-Units/Sockel. ZIH-Standard auf Barnard (Intel Sapphire Rapids) und Capella (AMD EPYC 9334).	Intel ISA-Erweiterungs-Referenz 319433 [Int26]
<code>Avx512SimdExtension</code>	Intel AVX-512 512-bit Vektor-Extension (Skylake-X+, Ice Lake Server, Zen 4+). Rückwärts-kumulativ: alle SSE/AVX/AVX2 + 15 separate Flags (VNNI für Vector-Indexes, VPOPCNTDQ für Bitmap-Indexes). Nicht auf allen x86_64-CPU's (etwa Haswell ohne AVX-512; auf Intels hybriden Alder-Lake-, Raptor-Lake- und Meteor-Lake-Prozessoren ist AVX-512 wegen der E-Kerne im BIOS abgeschaltet — so auch auf prod2, dessen Signatur kein <code>avx512</code> -Merkmal enthält). 1 AVX-512-Unit/Sockel; E-Cores kein Zugriff (Intel Hybrid).	Intel ISA-Erweiterungs-Referenz 319433 [Int26]
<code>NeonSimdExtension</code>	ARM NEON / Advanced SIMD 128-bit (AArch64 ABI-Pflicht ARMv8+). Kompatibel nur mit <code>Aarch64Isa</code> . Baseline auf allen AArch64-CPU's (Apple M-Series, AWS Graviton 2/3/4, NVIDIA Grace). 1 NEON-Unit/Sockel; alle Cores zugänglich (big.LITTLE-Architektur).	ARM ARM DDI 0487 / Cortex-A8 TRM

Variante	Beschreibung	Quelle
Sve2SimdExtension	ARM SVE2 (Scalable Vector Extension 2) ARMv9-Pflicht (skalierbar 128–2048 bit). Kompatibel nur mit Aarch64Isa. ZIH-Testsystem NVIDIA Grace Hopper Superchip (Grace: 72 Arm-Neoverse-V2-Kerne mit SVE2; Testsystem mit begrenzter Verfügbarkeit [Zen26b]). Skalierbare Lane-Breite; 1 Unit/Socket; alle Cores.	SVE [SBB ⁺ 17] (IEEE Micro 2017); SVE2/ Armv9 per ARM ARM [Arm24]
RvvSimdExtension	RISC-V Vector Extension (RVV v1.0, skalierbar VLEN 128–65536 bit). Kompatibel nur mit RiscvIsa. Optional (nicht Baseline wie SSE2/NEON). Wachsende Relevanz: SiFive Performance P870 (128-bit), T-Head Xuantie C908 (128-bit). ZIH-Präsenz: im ZIH-Kompodium nicht gelistet.	RVV-v1.0-Spec [RIS21]; Vitruvius+ (TACO 2023) [MPU ⁺ 23]
CudaGh200SmdExtension	NVIDIA CUDA auf Hopper GH200 (Grace Hopper Superchip GPU-Beschleuniger). Hopper-GPUs am ZIH stehen in Capella (4 × H100-SXM5 je Knoten auf AMD-EPYC-9334-Hosts [Zen26a]) und im Grace-Hopper-Superchip-Testsystem (Hopper-GPU mit 96 GB HBM3, NVLink-C2C 900 GB/s [Zen26b]). Kompatibel mit Aarch64Isa (Grace CPU) und Amd64Isa (Hopper-Host). Massiv parallel: 16 896 FP32-CUDA-Kerne je H100-SXM5-GPU [APK ⁺ 22]. units_per_socket () = -1 (GPU ist separate Device).	NVIDIA Tesla [LNOM08] (IEEE Micro 2008); NVIDIA Hopper Architecture In-Depth [APK ⁺ 22]

Quellen: ISA-Spezifikationen und Whitepaper (Intel, ARM, IEEE Micro, ACM TACO, NVIDIA); Einzelnachweise in der Quellenspalte der Bausteine.

D.30.2 Build-PG Seitentyp-Achse (PAGE-TYPE)

Abgrenzung und Konservierung: Diese Sektion beschreibt den PageKind der Organ-Welt (axis_01_page_type) und ist von der System-Unter-Achse page der Ziel-ISA (Seitengrößen-Freigabe 4k/2m/1g) verschieden — beide Größen führen ähnliche Namen, aber weder gemeinsame Bausteine noch gemeinsame Semantik. Der Katalog ist ein konservierter Angebots-Katalog außerhalb der achtzehn Kompositions-Slots und bleibt vollständig aufgeführt.

Die Seitentyp-Achse definiert die Struktur- und Indexierungsstrategien von Branch-Seiten im Trie-Index: von dicht-indiziertem Direktadressierungsindex (DenseByte), über mehrstufige Dichte-Varianten (ExtendedDense, SparsePatricia) bis zu Pfadkollaps-Optimierungen (Redirect, CustomCache) und B+-artige sortierte Seiten (BPlus). Alle Seitentypen sind Cache-Line-bewusst und implementieren das Concept-Interface PageTypeStrategy mit garantierter Austauschbarkeit (F15-Permutation).

Statische Unter-Achsen

ID	Name	Beschreibung
PG1	Struktur-Rolle (Branching)	Klassifizierung der Verzweigungsseiten-Architektur: Branch-Seite mit voller Verzweigung, kollabierter Single-Path-Knoten (Redirect) oder spezielle Cache-optimierte Variante.
PG2	Dichte-Klasse (Density)	Grad der Slot-Auslastung und daraus resultierende Indexierungsstrategie: dicht besiedelt (0-256 Slots), erweitert (>256 Slots) oder sparse mit Patricia-Kompression.
PG3	Pfad-Kollaps-Strategie	Handhabung eindeutiger Restpfade: keine Kompression (Standard), oder Kollaps durch Redirect/CoCo-Technik zur Speicherminimierung.

Bausteine

Variante	Beschreibung	Quelle
DenseBytePageType	Dichte byte-indizierte Verzweigungsseite mit direkter 256-Slot-Adressierung (direct-addressed). Seitentyp 1: direkte Abbildung von Schlüssel-Byte auf Slot-Index ohne Umleitung. Quelle: Adaptive Radix Tree (ART), Leis et al. 2013.	P01
ExtendedDensePageType	Erweiterte dichte Seite für >256 Einträge mit mehrstufigem Indexierungsschema (z.B. 2-Level-Lookup oder variable k-Byte-Span). Seitentyp 2: Dichte-Klasse für komplexere Slot-Anordnungen. Quelle: Adaptive Radix Tree Erweiterungen, PRT-ART.	P01 (extended)
SparsePatriciaPageType	Spärlich besetzte Seite mit Patricia-Pfadkompression (Single-Bit-Split). Seitentyp 3: Dichte-Klasse mit Bit-Vektor-Traversal für niedrige Branchingness. Quelle: HOT (Height Optimized Trie, Binna et al. 2018, basierend auf Patricia-Trie, Morrison 1968).	P02
RedirectPageType	Kollabierter eindeutiger Restpfad (Single-Path-Knoten). Seitentyp 4: Pfad-Kollaps-Strategie zur Speicheroptimierung bei monopol eindeutigen Nachfolgersequenzen. Weder klassischer Branch noch Leaf, speichert nur das Suffix bis zum nächsten tatsächlichen Verzweigungs- oder Wert-Knoten. Quelle: CoCo-Trie (Boffa et al. 2024).	P04
CustomCachePageType	Cache-Line-orientierte Custom-Seite mit explizitem Layout-Design für 64-Byte-Alignment. Seitentyp 5: Struktur-Rolle Branch mit Hardware-bewusster Geometrie. Quelle: PRT-ART Eigenentwicklung mit TLB-Offset + Cache-Line-Aligned Memory Layout (Achse 5).	PRT-ART
BPlusPageType	B+-artige sortierte Verzweigungsseite mit Range-Walk-Traversal. Seitentyp 6: Struktur-Rolle Branch, aber mit sortierter Key-Array-Organisation statt Byte-Indexierung. Quelle: Masstree (Mao et al. 2012), B+-Tree-Familie.	P03

Beitragende Quellen: P01, P02, P03, P04, Morrison 1968, PRT-ART [LKN13, BZP⁺18, MKM12, BFTV24, Mor68].

Verwandte Literatur: P05, P06, P10, P11, P12, P13, P20 [FJKN20, SSL⁺21, ZLL⁺18, RR99, RR00, HP03, MBL25].

D.30.3 Build-HW — Plattform-Hardware-Strategie

Status (konservierter Angebots-Katalog): Diese Achse ist aus dem bau-treibenden Angebot der System-Registry ausgeschlossen. Die Registry hält den Ausschluss wörtlich fest: `extension_hardware_system_axis.hpp` ist als veraltet markiert, `hardware_isa_system_axis.hpp` ein reiner Host-Deskriptor, der den Bau nicht steuert — beide gehören nicht zum bau-treibenden Angebot. Die statischen Plattform-Profile mit ihren festen Konstanten für Cache-Zeilen-Größe, Seiten-Größe und SIMD-Bit-Breite sind konzeptionell durch die Laufzeit-Hardware-Erkennung des `CacheEngineBuilder` abgelöst — eine Fabrik über ISA und Betriebssystem, die Hardware-Werte erhebt statt sie statisch zu setzen (vgl. Abschnitt 3.4.3). Die vier Unter-Achsen HW1–HW4 und die drei Hardware-Profile bleiben als Angebots-Katalog vollständig aufgeführt.

Definiert die Plattform-Hardware-Familie (CPU-Architektur, SIMD-Capabilities, Cache-Topologie, Memory-Paging) als Compile-Time-Konstanten für alle Cache-Engine-Algorithmen. Die Achse erfasst, welche Plattformen zur Build-Zeit konfigurierbar sind, und liefert statische Properties (Cache-Line-Größe, Page-Größe, SIMD-Bit-Breite, NUMA-Fähigkeit, HugePage-Fähigkeit) für algorithmengerechte Datenstrukturen.

Statische Unter-Achsen

Die vier Unter-Achsen stehen orthogonal zueinander; eine Plattform-Familie ordnet ihre primäre Charakteristik genau einem Tag zu, und der `CacheEngineBuilder` kann Plattformen nach Unter-Achsen gruppieren (etwa $HW1 \times HW2$ über alle Plattformen).

ID	Name	Beschreibung
HW1	CPU-Architektur-Familie (<code>cpu_family</code>)	x86_64 (Intel/AMD), aarch64 (ARM/Apple), riscv64 (RISC-V), generic (plattformunabhängig).
HW2	SIMD-Instruktionssatz-Familie (<code>simd_capability</code>)	SSE/AVX/AVX2/AVX-512 (x86), NEON/SVE (ARM), RVV (RISC-V), scalar (kein SIMD).
HW3	NUMA- und Cache-Hierarchie (<code>memory_topology</code>)	UMA-single-socket, NUMA-multi-socket, Apple-unified-memory, HBM-on-package (Sapphire Rapids HBM, Grace Hopper), CXL-attached.
HW4	Seiten-Konfiguration (<code>page_topology</code>)	Standard- und HugePage-Konfiguration: 4K-standard (POSIX), 16K-apple-silicon, 2M-hugepage (x86), 64K-arm-server, 1G-gigantic-hugepage.

Bausteine

Variante	Beschreibung	Quelle
X86_64Hardware Profile	Moderne x86-64 Desktop/Server-Plattform (Intel/AMD). Build-Time-Konstanten: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=256</code> (AVX2 konservativ), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Basis für desktop- und server-seitige Experimente.	Code/Baseline
Aarch64Hardware Profile	ARM64/AArch64 Server und Mobile Plattformen (Ampere Altra, Apple Silicon M-Serie, Linux ARM). Build-Time-Konstanten: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=128</code> (NEON Standard), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Ermöglicht Cross-Plattform-Vergleiche.	Code/Baseline
GenericHardware Profile	Konservatives Fallback-Profil für plattformunabhängige Konfigurationen. Build-Time-Konstanten: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=0</code> (scalar-only), <code>numa_capable=false</code> , <code>huge_page_capable=false</code> . Kleinster gemeinsamer Nenner, wenn kein spezialisierter Adapter passt.	Code/Baseline

Quellen: Plattform-Profile als Code/Baseline (keine externen Veröffentlichungen).

E Architekturentscheidungen

Dieser Anhang sammelt die tragenden Entwurfsentscheidungen der Arbeit als kompakte *Architecture Decision Records*¹ (ADRs): je Entscheidung der Kontext, in dem sie fiel, die Entscheidung selbst und ihre wichtigste Konsequenz. Ziel ist die schnelle Nachvollziehbarkeit — *warum* der Apparat so gebaut ist, wie ihn die Kapitel beschreiben; die ausführliche Herleitung und Begründung liegt jeweils im per *Beleg* referenzierten Abschnitt des Haupttextes und wird hier nicht wiederholt. Wo Entwurf und Implementierung auseinanderliegen, weist die ADR den Umsetzungsstand an genau einer Stelle als viertes Feld aus. Der Bestand umfasst zwanzig Entscheidungen: die zwölf Entscheidungen der Konzeption (ADR-1 bis ADR-12) und acht Entscheidungen der Identität, der Schnittstellen, des Betriebs und des Baus (ADR-13 bis ADR-20); Tabelle E.1 gibt die Übersicht.

Tabelle E.1: Übersicht der zwanzig Architekturentscheidungen: Kurztitel, belegender Abschnitt und Umsetzungsstand — *umgesetzt* (implementiert und belegt), *teilweise* (implementiert, mit ausgewiesenem offenem Gegenstand) oder *Entwurf* (nur entworfen, nicht implementiert); die Begründungen stehen in den Datensätzen, nicht in der Tafel

ADR	Entscheidung (Kurztitel)	Beleg (Abschnitt)	Umsetzungsstand
ADR-1	C-stabile Modulgrenze statt C++-Modul-ABI	Abschn. 4.3.1	umgesetzt
ADR-2	Compile-Time-Permutation über Typlisten	Abschn. 4.3	teilweise (Aufschaltung der achtzehnten Achse)
ADR-3	Drei-Ebenen-Modell als die eine Architektur	Abschn. 3.3.2	umgesetzt
ADR-4	Durchreich-Regel und erzwungene Vollbelegung	Abschn. 4.3	teilweise (Durchreich-Baustein des Knotentyps)
ADR-5	Schichtenmodell E4–E0 mit festgeschriebenen Verträgen	Abschn. 3.3.4	umgesetzt
ADR-6	Explizit kodierte Nicht-Erhebung statt Phantomwert	Abschn. 3.4	umgesetzt
ADR-7	Vier-Repository-Architektur mit orthogonalen Träger-Stufen	Abschn. 4.1	umgesetzt
ADR-8	Familienweise Adapter-Fallbacks	Abschn. 4.3	umgesetzt
ADR-9	Mess-Gating in drei Schichten und vier Arbeitsmodi	Abschn. 4.3.2	teilweise (Replay-Ablauf compare/release)
ADR-10	<code>std::map</code> -Vertrag mit Konformitäts-Gate	Abschn. 3.4	teilweise (volle <code>std::map</code> -Schnittstelle, Container-Hülle)
ADR-11	Entwurfsmuster-Register mit belegten Rollen	Abschn. 3.3.3	umgesetzt
ADR-12	Mess-Regime-Doppel Talos und root-Linux	Abschn. 5.1	teilweise (Talos-Regime)
ADR-13	Vier Träger-Stufen und ein Stempel-Vertrag	Abschn. 3.4.4	teilweise (Stempel-Export Hybrid, SOTA-Zweig)

Fortsetzung auf der nächsten Seite

¹Architecture Decision Record (ADR): eine kurze, nummerierte Aufzeichnung einer Entwurfsentscheidung mit Kontext, Entscheidung und Konsequenzen — eingeführt von Nygard [Nyg11] als kurze Textdatei in einem Format nach Art eines Alexander-Musters und als Werkzeug-Format beschrieben von Kopp, Armbruster und Zimmermann [KAZ18].

Tabelle E.1 — Fortsetzung

ADR	Entscheidung (Kurztitel)	Beleg (Abschnitt)	Umsetzungsstand
ADR-14	Identitäts-Schnitt: nur Organ-Achsen bilden die <code>binary_id</code>	Abschn. 3.4.1	umgesetzt
ADR-15	Mess-Visitor-Naht (Fläche 3) statt Sidecar	Abschn. 3.3.2	umgesetzt
ADR-16	Lager als Bestand mit Übersetzungszeit-Schlüsseln	Abschn. 3.4.5	teilweise (Lager-Vollausbau)
ADR-17	Optimierungsstufe als Anwender-Wahl, Bau-Welt identitätswirksam	Abschn. 5.1.6	teilweise (O0/O1-Wahl, Flag-Durchreichung)
ADR-18	Ein offizieller Bauweg als GNU-Vorbau über CMake	Anhang B	teilweise (Framework-Export)
ADR-19	Vier CI-Zellen ohne Skip	Abschn. 4.4	teilweise (Mini-Pipelines je Träger-Stufe)
ADR-20	XML als einzige Steuerung	Abschn. 4.3.2	teilweise (Erstellungsmodus, headless Steuerpfad, Schema-Drift)

ADR-1 — C-stabile Modulgrenze statt C++-Modul-ABI

Kontext: Jede Permutation wird als eigenständige Tier-Binary kompiliert und zur Laufzeit geladen; C++23-Module strukturieren zwar die Übersetzungseinheiten, garantieren für sich genommen aber keine portable ABI-Stabilität [Xu24]. **Entscheidung:** Eine einheitliche C-Modulgrenze exportiert je Permutation genau ein Lebewesen — eine `IAnatomyBase`-Instanz — über sieben gattungsunabhängig benannte Pflicht-Symbole: die vier Ur-Symbole ABI-Version, Magic, Factory (`comdare_create_anatomy()`) und Destroy, die zwei Identitäts-Symbole Konzept und Gattung, wobei das Konzept nicht als zweites Datum gepflegt, sondern übersetzungsstatisch aus der Gattung abgeleitet wird, und das Stempel-Zeilen-Symbol. Die Namensgleichheit über alle Gattungen ist das Stabilitäts-Argument: Der Lader kann die Gattung über ein konzeptunabhängiges Symbol ermitteln, ohne sie vorab zu kennen. Dahinter liegen zwei Engine-Begriffe: die `CacheEngine` als Bibliothek der Achsen-Bausteine und die `ExecutionEngine` als Ausführungs- und Mess-Infrastruktur mit der Wurzel `IExecutionEngine`, unter der `IAnatomyBase` steht; die API-Fassade ist `ICacheEngine`; der Name „SearchEngine“ bezeichnet die ABI-Laufzeit-Sicht der Gattung `SearchAlgorithm`, kein drittes Kettenglied; das gleichnamige Auslagerungs-Modul ist deklariert, aber nicht angebunden. **Konsequenz:** Die ABI-Stabilität stellt die C-Grenze her, nicht das Modulsystem (ABI-Major 9); der Lader prüft jedes Modul beim Laden fail-closed gegen die Symbol-Pflichtmenge und die Identitäts-Ableitung — ein Modul mit widersprüchlicher Identität oder ohne Stempel-Symbol wird abgewiesen, statt als falsche Identität in Stempel, Lager und Messreihe zu wandern. **Beleg:** Abschnitte 1.4, 3.3.3 und 4.3.1; Abbildung 4.4.

ADR-2 — Compile-Time-Permutation über Typlisten statt Laufzeit-Adaption

Kontext: Achtzehn Organ-Kompositions-Achsen zählen schon im golden-Referenzkatalog $2^{17} = 131072$ Binary-Identitäten je System-Permutation — eine davon, das Persistenz-Ziel, auf ihren In-Memory-Vertreter festgelegt, unter der überholten Annahme zweier Auslegungen je Achse —, die Obergrenze je System-Permutation ist das Produkt der je Achse im XML-Experiment gewählten Auslegungen (erst zur Ausführung bekannt), und der rohe Varianten-Produktraum über alle Achsen-Kataloge samt Unter-Achsen liegt Größenordnungen darüber; der Hot-Path

darf weder `virtual`-Aufrufe noch Laufzeit-Switches enthalten. Die Festlegung ist eine Budget-Entscheidung (laut Katalogkopf rund 224 GB und 24 h vollständiger Bau statt rund 448 GB und 48 h); vorgesehen ist die Variation aller achtzehn Achsen, und die Mächtigkeit eines konkreten Plans rechnet der Planer je XML im Director-Walk, nie statisch. **Entscheidung:** Die Permutation läuft übersetzungszeitlich über Boost.Mp11-Typlisten²: je Achse eine Gesamt-Liste, per Filter über das Aktivierungs-Prädikat zur Enabled-Liste, gebunden an den Kompositions-Slot, dazu concept-geprüfte Achsen-Deskriptoren mit dem Inventar `variants` und eine generische Registry-zu-Baum-Reflektion; der golden-Referenzkatalog wählt je Slot die ersten Vertreter der Enabled-Liste — kein Runtime-Tag, kein `std::variant`. `std::variant`, `virtual` und Laufzeit-Switches sind in Tier- und Hybrid-Binaries grundsätzlich ausgeschlossen; eine statische Prüfung erzwingt das, und der variant-basierte Baustein-Cluster (`baustein_variants`, `resolve_baustein`) ist ein vom Bau isolierter Test-Bestand ohne Bibliotheks-Konsumenten. **Konsequenz:** Die Achsen-Abstraktion bleibt eine zero-cost abstraction; in die Binary-Identität gehen ausschließlich die achtzehn Organ-Achsen ein (ADR-14). **Umsetzungsstand:** Die Aufnahme der achtzehnten Achse in den Referenzkatalog ist nicht umgesetzt; sie verlangt zwei Schritte zugleich — den Katalog-Grad zwei für das Persistenz-Ziel und den Rückschreib-Schalter der Persistenz-Achse. **Beleg:** Abschnitte 1.4, 3.3.3, 4.3 und 3.4.6; Abbildung 3.11.

ADR-3 — Drei-Ebenen-Modell als die *eine* Architektur

Kontext: Anatomie-Metapher und technische Engine-Sicht drohen zu parallelen Modellbäumen auseinanderzulaufen. **Entscheidung:** Es gibt genau eine Hierarchie ohne Parallel-Bäume — Konzept (Ebene 1, Mess-Surface: die Prüf-Dock-Konzepte Map, Container und Graph sowie das vierte Konzept HeuristikAdapter, das kein eigenes Dock hat, weil seine Binaries per `genus()` die Ziel-Gattung melden und am Dock des Ziels andocken) → Gattung (Ebene 2, das Tier-Interface mit festem Achsen-Satz; die sechs Gattungen `SearchAlgorithm`, `Set`, `Sequence`, `Adapter`, `View` und `FunctionInterfaceReroute`) → Genus (Ebene 3, die Implementierung des Gattungs-Interfaces, z. B. ART, HOT, START) → Achsen-Organe (Ebene 4, eigene Klassifikation) — unter `IExecutionEngine` als gemeinsamer messbarer Wurzel; ein eigenes Konzept entstünde erst, wenn sich der Basis-Achsen-Satz grundlegend unterschiede. **Konsequenz:** Die Gattung `SearchAlgorithm` unter dem Konzept Map und ihre ABI-Laufzeit-Sicht („SearchEngine“) sind zwei Sichten desselben Lebewesens, kein zweites Modell. **Beleg:** Abschnitte 3.3, 3.3.2 und 4.3.1; Abbildungen 3.7 und 3.8.

ADR-4 — Durchreich-Regel und erzwungene Vollbelegung

Kontext: Für nicht-baumartige Verfahren (etwa die Hash-Gegenprobe) sind baum- und trie-spezifische Achsen wie Pfadkompression oder Knotentyp nicht sinnvoll belegbar. **Entscheidung:** Solche Achsen fallen nicht weg, sondern werden als Durchreich-Varianten (`None`) belegt — ein Durchreich-Wert ist ein Algorithmus mit leerem Pfad, kein Weglassen; implementiert ist der Durchreich-Baustein für die Pfadkompression (`PathCompressionNone`), der Knotentyp besitzt keine `None`-Variante und wird regulär mit einem der vier ART-Knotenformate belegt, die ein nicht-baumartiger Baustein nicht nutzt (Entwurf; ohne Code-Baustein); das Pflicht-Concept

²Boost.Mp11: eine C++11-Metaprogrammier-Bibliothek zur übersetzungszeitlichen Manipulation von Datenstrukturen, die Typen enthalten; ihr Listentyp ist `mp_list` [Dim19].

IsComposition der Gattung SearchAlgorithm erzwingt übersetzungszeitlich die Belegung aller achtzehn Organe (achtzehn Typ-Forderungen und ein Organ-Zähler von achtzehn mit Prüfung auf ABI-Bruch), sodass keine Komposition dieser Gattung mit offener Achse übersetzt. **Konsequenz:** Nicht-baumartige Verfahren erfordern kein eigenes Konzept, sondern ein eingeschränktes Achsen-Subset innerhalb derselben Gattung. **Beleg:** Abschnitte 3.2, 3.3.2 und 4.3.

ADR-5 — Schichtenmodell E4–E0 mit festgeschriebenen Verträgen und Contract-Tests

Kontext: Bei einem Produktraum, der jede vollständige Materialisierung ausschließt, sind Änderungen über mehrere Ebenen zugleich praktisch nicht abnehmbar; deshalb wird je Ebene einzeln fertiggestellt. **Entscheidung:** Die Experiment-Pipeline gliedert sich in die fünf Schichten E4–E0 (E4 Definition und Auswertung, E3 Permutations-Baum, E2 Tier-Binaries, E1 Laufzeit-Konfiguration, E0 Infrastruktur) und den Querschnitt M; die E-Kennungen sind eine Konzept-Ordnung mit je einem Code-Ort. Jede Schicht wird einzeln fertiggestellt, mit explizit festgeschriebenem Interface-Vertrag zur Nachbar-Ebene (die Festschreibung des Vertrags der Schicht E1 im ABI-Adapter: Signaturen, Datenlayout und Fähigkeiten festgeschrieben, Semantik-Änderungen nur additiv), und gegen einen übersetzungszeitlichen Ersatz (Fake) der Nachbar-Ebene contract-getestet. **Konsequenz:** Jede Ebene ist einzeln abnehmbar, bevor die nächste beginnt; die Grenze E3→E2 ist ohne Modul-Bau gegen einen Fake der Schicht E2 contract-getestet, geprüft wird allein die statische Baum- und Sichtstruktur samt ihrer Mixed-Radix-Bijektion. **Beleg:** Abschnitte 3.3.4 und 4.4; Abbildungen 3.12 und 4.9.

ADR-6 — Explizit kodierte Nicht-Erhebung statt Phantomwert

Kontext: Nicht wirklich erhebbare Werte (Zähler ohne Hardware-Zugriff, Achsen ohne tatsächlichen Laufzeit-Parameter) laden zu plausiblen Phantomwerten ein, die die Heuristik-Ableitung systematisch vergiften würden. **Entscheidung:** Das System meldet explizit ungültig oder leer statt eines Phantomwerts und unterscheidet je Messwert drei Zustände: gültiger Messwert, explizit kodierte Nicht-Erhebung (nicht anwendbar oder Quelle nicht verfügbar) und Messfehler³. Je Ereignis reist ein Gültigkeits-Flag mit, sodass eine gemessene Null ein Messergebnis bleibt; die Wall-Clock-Quelle bietet nur Mittelwert und Durchsatz an und verweigert es, ihren Mittelwert als Perzentil zu etikettieren, und eine negative Dauer ist ein Messfehler, kein Nullwert; ein PMC-Katalog ohne Modell-Eintrag liefert „nicht anwendbar“, nie eine geratene Kodierung; die Generatoren erzeugen für eine Messreihe ohne gültige Zeile keine Ausgabe statt einer leeren. **Konsequenz:** Lücken bleiben sichtbar und gezielt schließbar; der Grundsatz ist in den Bausteinen einprogrammiert statt nur dokumentiert. **Beleg:** Abschnitte 3.4, 4.5 und 3.4.1.

ADR-7 — Vier-Repository-Architektur mit orthogonalen Träger-Stufen

Kontext: Mess-Apparat, Prüfling, Experiment-Definition samt Auswertung und das Manuskript haben getrennte Verantwortlichkeiten und Lebenszyklen; die Messwerte werden versioniert direkt in das Manuskript eingebaut, weshalb das Manuskript ein eigenes Repositorium mit eigener Pipeline ist. **Entscheidung:** Vier Repositorien — `comdare-cache-engine` (wie gemessen wird; Bausteine-Bibliothek und Builder im Pitchfork-Quellbaum-Layout, Anhang C), `comdare-prt-art` (der

³Das Code-Vokabular `honest-0/honest-100/honest-empty` ist der Implementierungsname dieses Grundsatzes (Anhang C); methodisch ist die Nicht-Erhebung eine ausgewiesene Klasse fehlender Werte im Sinne der Missing-Data-Literatur [Gra09], nie ein imputierter Wert.

Prüfling), das Projekt-Repository `probst-Diplomarbeit-cache-engine` (was durch den Anwender getestet wird: Experiment-Definition in XML und XSD, Mess-Treiber, Auswertungs-Werkzeugkette) und das Manuskript-Repository mit eigener CI, in das der Vorwärts-Kanal die Generator-Ausgabe als Tabellen- und Diagramm-Dateien per Commit einlegt —; orthogonal zu dieser Aufteilung liegen die vier Träger-Stufen in Bau-Reihenfolge Planer → CacheEngineBuilder → Tier-Binary → Hybrid-Tier-Binary, wobei Stufe $N + 1$ nur an Stufe N hängt (Verarbeitungskette: CEB-Prüf-Dock → [optionaler Hybrid-Mux] → Tier-Binary); Planer und CEB sind zwei Binaries — der Planer `comdare-experiment-planner` und der Mess-Treiber `comdare-messung-driver` —, und die zwei Rollen erscheinen als Planer-Zeile und CEB-Vertragszeile im Selbst-Stempel des Mess-Treibers (`-version`) in einem Treiber-Binary; den Mess-Aggregator führt der `ExperimentDriver` host-seitig. **Konsequenz:** Builder und CacheEngine teilen sich das Engine-Repository — der Builder als ausführbare Datei über seiner Builder-Bibliothek, die CacheEngine als Bibliothek —; die Rollen bleiben klar getrennt, und der Prüfling ist keine Stufe, sondern der quer liegende Lieferant von Organ-Bausteinen. **Beleg:** Abschnitt 4.1 und Anhang B; Abbildung 4.1.

ADR-8 — Familienweise Adapter-Fallbacks für eigenständig lauffähige Builds

Kontext: Fremd-Code (Allokator-Vendors, SOTA-Bausteine) ist nicht auf jeder Plattform und in jedem Build verfügbar oder aktiviert. **Entscheidung:** Übersetzungszeitliche Flag-Familien schalten Adapter zu oder ab — je Vendor ein Familien-Schalter, dessen Nutzung aus Freigabe und Verfügbarkeit folgt, hinter 23 Vendor-Shim-Headern —, jeweils mit sicherem, familienweise gewähltem Fallback: Die Allokator-Adapter ersetzen den fehlenden Vendor in `if constexpr`-Zweigen durch die portable ausgerichtete Allokation (`portable_aligned_alloc`), der Vendor-Shim liefert für den nie ausgeführten Zweig nur Forward-Stubs, und der Achsen-Wert `StdMalloc` ist ein eigener Baustein, keine Notlösung; Datenstruktur-Adapter fremder Suchverfahren schalten über einen Familien-Schalter und besitzen eine eigenständige Re-Implementierung als Körper, während die Bindung an den Original-Code über den Bibliotheks-Bau mit dem Original-Compiler läuft. **Konsequenz:** Der Build bleibt ohne jede Fremd-Aktivierung eigenständig lauffähig. **Beleg:** Abschnitt 4.3.

ADR-9 — Mess-Gating in drei Schichten und vier Arbeitsmodi

Kontext: Mess- und Observer-Kopplungen kosten Zeit und dürfen den Produktiv-Pfad nicht belasten. **Entscheidung:** Die Mess-Kopplungen des Laufzeit-Adapters werden über die Master-Option `COMDARE_MEASUREMENT_MODE` geschaltet, die `COMDARE_MEASUREMENT_ON` definiert und die der Release-Modus erzwungen abschaltet; die schaltbare Fläche zerfällt in drei trennbare Schichten — Basis-Zeit, Observer-Zähler und Feinkorn-Segment-Timer je Achse —, die Abschaltungsstufen der Messung. Die Ablaufmethodik kennt vier Arbeitsmodi derselben Permutation, als Registry und nicht als Etikett: `build` (bauen ohne Messen), `measure` (deterministischer Ein-Thread-Messlauf, der einzige Modus mit Messung), `compare` (liest das Messwertlager, die Stufe vor `release`) und `release` (Voll-Optimierung ohne Mess-Instrumentierung); der Arbeitsmodus ist eine Unter-Achse des Mess-Realms und geht nicht in die Binary-Identität ein, einen eigenen Debug-Arbeitsmodus gibt es nicht (`-debug` ist ein Flag der Planer-Shell und kein eigener Arbeitsmodus der Kette `build/measure/compare/release`; es schaltet den indirekten Debug-Modus

der Debug-Variante ein, den nicht offiziellen fünften Wartungsmodus, die weiter ausgebaut wird: unter `-debug` fährt das Konformitäts-Gate zusätzlich den Standard-Testsatz des Tier-Interfaces, ein indirekter Debug-Modus, der mit dem Release-Bau zusammenfällt; die Bau-Varianten Debug, Messung und Release gelten fort), und den Mess-Aggregator führt der `ExperimentDriver` host-seitig. **Konsequenz:** Der Produktiv-Pfad bleibt mess-frei und verursacht keinen Mess-Overhead; wählbar sind vier Modi, in der Bau-Semantik verschieden ist allein `measure` — alle vier Registry-Zeilen führen den Release-Bautyp. **Umsetzungsstand:** Der modus-spezifische Replay-Ablauf von `compare` und die aus Messwerten erzeugte optimale Release-Binary sind für spätere Haupt-Wellen vorgesehen, aber nicht implementiert; `compare` ist ein wählbares, validierbares Etikett mit release-gleicher Bau-Semantik. **Beleg:** Abschnitte 4.1, 4.3.2 und 5.1; Abbildungen 3.10 und 4.11.

ADR-10 — `std::map`-Vertrag als gemeinsamer Nenner mit Konformitäts-Gate

Kontext: Strukturell verschiedene Suchalgorithmen sind nur über eine gemeinsame, gut verstandene Außen-Schnittstelle fair vergleichbar. **Entscheidung:** Alle Verfahren werden auf den C++-Vertrag der geordneten Map `std::map<Key, Value>` [ISO24] abgebildet; die Container-Gattungen werden gegen je ein Standard-Orakel gehalten — `Set` gegen `std::set`, `Sequence` gegen `std::deque`, `Adapter` gegen `std::queue`, `std::stack` und `std::priority_queue`, `View` gegen `std::span` —, und `std::vector` dient als Inner-Substrat der Gattung `Adapter`. Ein Konformitäts-Gate prüft diesen Vertrag vor jeder Messung in der bindenden Reihenfolge `Import` → `Gate` → `Messen`: die siebzehn in Anhang F markierten `std::map`-Operationen (zwölf Kern-Prüfblöcke, `std::map` als Orakel), mit Konformitäts-Quoten als Ergebnisform; die fünf Antriebs-Operationen der Modulgrenze (`insert`, `lookup`, `erase`, `clear`, `size`) sind die ABI-Primitive, über denen der host-seitige Adapter diesen Vertrag komponiert, nicht der Prüfumfang. Unter `-debug` fährt das Gate zusätzlich den Standard-Testsatz des jeweiligen Tier-Interfaces: den generischen Gattungs-Teil (gültig für alle Genera) und den Genus-spezifischen Teil (die gegebenenfalls erweiterten Spezial-Interfaces); dieser indirekte Debug-Modus fällt mit dem Release-Bau zusammen und wird nicht separat geführt. **Konsequenz:** Nur Vergleichbares wird verglichen; die Achsen beschreiben ausschließlich das innere Verhalten hinter der festen Schnittstelle. **Umsetzungsstand:** Der Ausbau des Gates auf die volle `std::map`-Schnittstelle und die Gattungs-Iterator-Bibliothek sind nicht implementiert; die Forderung, die Container-Gattungen über eine vollständige `std::vector`-Hülle zu führen, ist nicht erfüllt — das implementierte Sequence-Orakel ist `std::deque`. **Beleg:** Abschnitte 3.4 und 4.4.

ADR-11 — Entwurfsmuster-Register mit belegten Rollen

Command als Bündelungs- und Befehlsmuster.

Kontext: Der Apparat ist bewusst eine Komposition benannter Entwurfsmuster; unklare Muster-Rollen würden die Architektur verwässern. **Entscheidung:** Verwendet werden, je mit benanntem Code-Ort: *Strategy* mit CRTP [Cop95] je Organ-Achse (auch der Stempel-Vertrag der Träger ist ein CRTP-Vertrag, ADR-13), *Abstract Factory* (Hybrid-Docks, Kompositions-Fabrik, Prüflings-Registry), *Director* mit *Builder* (Plan-Emission), *Iterator* (der lazy durchlaufene Experimentbaum), *Adapter* (Fremd-Code hinter der ABI-Grenze), *Observer* (Mess-Snapshots je Achse), *Visitor* (die Mess-Naht am `dlopen`-Übergang, ADR-15), *Chain of Responsibility* (Hardware-Erhebungskette

und Auswahl-Filterkette), *Memento* (Copy-on-Write im ABI-Adapter, Zustand je Tier), *Command* und *Facade* — dieselben Rollen wie das Muster-Register in Abschnitt 4.5.1 (Abbildung 4.12), benannt nach dem Katalog von Gamma et al. [GHJV94] und als Tupel von Achsen-Policies komponiert [Ale01]. Die Muster *Composite* und *Decorator* gehören nicht dazu: Die Mess-Hüllen der Achsen halten keine Komponenten-Instanz und kein Voll-Interface und sind ausdrücklich kein Decorator, und ein Composite im Sinne des Katalogs hat keine Entsprechung im Code. *Nicht* verwendet werden im Hot-Path und in den Organ-Achsen *Singleton*, *Flyweight* und *Bridge*; host-seitige Registries nutzen Meyers-Singletons⁴, die Katalog-Erzeugung Flyweight-Tabellen je Achse, und „Bridge“ ist im Code nur eine umgangssprachliche Bezeichnung für Verbindungsstücke (ABI-Adapter, CI-Bridge-Jobs, Sandy Bridge), kein Muster. Das *Command*-Muster trägt die übersetzungszeitliche Bündelung von Achsen-Rekombinationen (die System-Komplex-Achse als Command-Wrapper, der Hub der externen Werkzeuge als Invoker, die Command-Basis der Achsen im Querschnitt M) und den typisierten Steuer-Befehlskanal; der Befehls-Kanal des Hybrid-Modus ist eine dieser Rollen. **Konsequenz:** Jedes Muster trägt benannte Rollen mit Code-Ort in der Verkabelung des Apparats und wird übersetzungszeitlich statt laufzeit-polymorph ausgeführt. **Beleg:** Abschnitte 3.3.3, 3.6.2 und 4.5.1.

ADR-12 — Mess-Regime-Doppel: immutable Talos und root-Linux

Kontext: Produktions-Treue und Mess-Tiefe stehen im Konflikt: Ein produktionsnahes System verwehrt gerade den Hardware-Zähler-Zugriff, den tiefe Messungen brauchen. **Entscheidung:** Auf den Produktions-Zielmaschinen wird jede Messung unter zwei Betriebssystem-Regimes erhoben: einem immutablen Betriebssystem (Talos⁵) als Produktiv-Spiegel und einem root-Linux mit vollem Hardware-Zähler-Zugriff (`perf` [Mel10, Lin26b]) und die modellspezifischen Register, MSR). **Konsequenz:** Die zeit- und observer-basierten Kategorien laufen unter beiden Regimes identisch und sichern die Vergleichbarkeit; die PMC-Kategorien werden nur unter dem privilegierten Regime erhoben. **Umsetzungsstand:** Die Entscheidung gilt; umgesetzt ist die root-Linux-Hälfte: Die berichteten Messungen stammen sämtlich vom root-Linux-Regime der beiden Produktionsmaschinen — prod1 mit der AMD-Mikroarchitektur Zen 5 und prod2 mit dem Intel Core i9-12900K (Alder Lake) —; das Talos-Regime ist vorgesehen, aber nicht gemessen. Für Intel enthält der PMC-Rohereignis-Katalog keine Belegung, für Zen 5 ist sie durch Messung gegengeprüft. **Beleg:** Abschnitte 5.1 und 1.4.

ADR-13 — Vier Träger-Stufen und ein Stempel-Vertrag

Der Stempel als Identität, Lager-Schlüssel, Wiedererkennung-Kriterium und Einordnung.

Kontext: Die Konfiguration wandert durch vier Träger-Stufen (Planer, CacheEngineBuilder, Tier-Binary und die optional zwischen CEB-Prüf-Dock und Tier-Binary eingeschobene Hybrid-Tier-Binary); jede Stufe braucht eine ABI-stabile, übersetzungszeitliche Identität, in die kein Messwert eingeht. **Entscheidung:** Ein einziger CRTP-Vertrag bindet alle vier Träger: sieben Stempel-Interfaces (Version, Mess-Zeile, System-Zeile, Organ-Zeile, Fingerprint, Gesamt-Stempel, Ange-schlossene) mal vier Träger ergeben 28 dreiwertige Zulassungszellen (Tabelle 4.3); die Absenz der Organ-Zeile beim CEB ist die statische Sperre „der CEB führt keine Organ-Achsen“, und die

⁴Meyers-Singleton: die eine Instanz als funktionslokales statisches Objekt, das beim ersten Aufruf initialisiert wird [Mey05].

⁵Talos Linux: das unveränderliche, API-gesteuerte Betriebssystem für Kubernetes von Sidero Labs [Sid26].

Prüfung läuft als `static_assert` unter jeder Erbin. Der Planer führt einen SHA-256-Fingerprint aus Version, ISA und Betriebssystem, die Tier-Binary einen SHA-512-Fingerprint über elf Glieder (Format 6), darunter die drei Realm-Zeilen, das Werkzeugketten-Glied und den Hash der Overlay-Quellmenge (Konkatenation in fester Ordnung, einmal SHA-512); der CEB führt einen SHA-512-Provenienz-Fingerprint, der kein Wiederverwendungs-Kriterium ist. Die Klammer-Grammatik der Stempel-Einträge ist an einer Stelle definiert (`c{p.e}` ist `c` mit den Untergliedern `p` und `e`, nie flach), und die Klammer-Tiefe ist die Meta-Meta-Ebene des Eintrags. **Konsequenz:** Der Stempel erfüllt fünf Rollen — Identität, Cache-Schlüssel, Lager-Schlüssel, Wiedererkennungskriterium und Einordnung; der Selbst-Stempel des Treiber-Binaries (`-version`) führt nur die Einordnung und ist weder Cache-Schlüssel noch Bau-Eingang. **Umsetzungsstand:** Der Registry-Wrapper-Stempel des SOTA-Zweigs und der Stempel-Export der Hybrid-Stufe sind nicht implementiert; die Stempel-Identität in jedem Diagramm und jeder Tabelle des Anhangs ist ebenfalls nicht implementiert. **Beleg:** Abschnitte 3.4.4, 3.3.2, 3.4.6, 4.4 und 5.2.1; Abbildung 3.9.

ADR-14 — Identitäts-Schnitt: nur Organ-Achsen bilden die `binary_id`

System- und Mess-Achsen sind identitätsneutral.

Kontext: Drei Achsen-Realms (Organ, System, Mess) führen je eine eigene Registry mit generierter XML; gingen Mess- oder System-Werte in die Identität ein, entstünden Doppel-Binaries derselben Permutation und ein Lager, das dieselbe Binary mehrfach hält. **Entscheidung:** Die `binary_id` ist der Pfad aus achtzehn `achse=wert`-Segmenten in kanonischer Slot-Reihenfolge (die Organ-Zahl achtzehn ist an genau einer Stelle definiert); alle Mess-Achsen und alle System-Achsen führen in ihren Registries `binary_id = never`; Unter-Achsen sind Laufzeit-Permutation und stehen in keiner Bau-Job-Legende; der Arbeitsmodus ist identitätsneutral. Die Bau-Welt reist stattdessen als Fingerprint-Glied mit — die System-Zeile und das Werkzeugketten-Glied mit `vendoropt` (ADR-17). **Konsequenz:** Lager-Schlüssel und Regressions-Detektor (Referenzkatalog 2¹⁷ mit CRC-64-Prüfsumme) sind organ-stabil; dieselbe Organ-Permutation gegen zwei Bau-Welten ist ein verschiedenes Artefakt mit verschiedenem Fingerprint. **Beleg:** Abschnitte 3.4.1, 3.4.4 und 4.5; Tabelle 3.2.

ADR-15 — Mess-Visitor-Naht (Fläche 3) statt Sidecar

Konzept- und Gattungs-Interfaces bleiben unverändert.

Kontext: Ein Beobachter, der von außen Felder aus einer Tier-Binary abholt, misst die Nachbarschaft und nicht den Gegenstand: Was gemessen wird, entscheidet dann der Host durch die Wahl der Felder; Mess-Felder in den fachlichen Interfaces würden die Konzept- und Gattungs-Verträge verändern. **Entscheidung:** Jede Träger-Stufe bietet drei Interface-Flächen: Fläche 1 das Gattungs-Interface (Laufzeit, Abstract Factory), Fläche 2 den Stempel (Übersetzungszeit, ABI-stabil), Fläche 3 den Mess-Durchstich; die Messung quert als eigene Naht (`tier_measure_accept` mit `IMessVisitor`), ein Visitor am `dlopen`-Übergang mit doppeltem Dispatch [GHJV94]; je nachdem, ob die Messeinrichtung in CEB und Tier-Binary aktiviert ist, wird am Interface ein Mess-Visitor übergeben oder nicht. **Konsequenz:** Die fachlichen Interfaces enthalten keine Mess-Felder; die Gate-Kombination CEB aus, Tier an ist durch einen Deckungs-Test belegt; die Mess-Gate-Belegung reist als eigenes Fingerprint-Glied mit eigenen Positionen für die Hybrid-Ebene. **Beleg:** Abschnitte 3.3.2, 4.3.2 und 3.4; Abbildung 3.9.

ADR-16 — Lager als Bestand mit Übersetzungszeit-Schlüsseln

Messung immer an, im Lager nur Fehlendes nachmessen.

Kontext: Ein Referenzraum von 2^{17} Kompositionen je System-Permutation im golden-Referenzkatalog (laut Katalogkopf rund 224 GB und 24 h vollständiger Bau; überholte Annahme zweier Auslegungen je Achse, die Obergrenze bestimmt der Planer aus dem XML-Experiment) ist nur mit Wiederverwendung gebauter Binaries und gemessener Werte beherrschbar; die Messung soll immer eingeschaltet sein und im Lager nur nachmessen, was dort nicht zu finden ist. **Entscheidung:** Bestand 1 hält die Binary unter ihrem SHA-512-Fingerprint; Bestand 2 hält die Messwerte unter dem Paar aus Fingerprint der vermessenen Binary und Hardware-Identität der messenden Maschine, die Mess-Zelle steht daneben und wird nie in den Digest fusioniert; eine vorgefundene Binary wird übersprungen, wenn ihr beständenes Prüf-Testat inventarisiert ist, sonst gebaut oder gemessen; ein Versions-Lock über die Overlay-Quellmenge macht unbemerkte Quelländerungen zum CI-Fehler. **Konsequenz:** Die Identität reist unverändert vom Lager über die Mess-Zeile (`permutation_id`, `fingerprint`) in Mappe und Anhang. **Umsetzungsstand:** Bestand 1 und 2 und das Schlüssel-Schema sind implementiert; Bestand 3 und 4 (Synthese-Batches und Lösungen), die Planer-Blattfunktion, die CEB-Vollplatzierung, die Hybrid-Ablage, der Commit-Skip, das Rebuild-Flag je Stufe und die XML-Sektion `publish` bilden den Lager-Vollausbau und sind nicht implementiert. **Beleg:** Abschnitte 3.4.5, 5.1, 3.4.4 und 3.6.2; Abbildungen 3.14 und 5.2.

ADR-17 — Optimierungsstufe als Anwender-Wahl

Haus-weit O2, O3 auf XML-Wunsch; die Bau-Welt ist identitätswirksam.

Kontext: Messreihen über verschieden optimierte Bau-Welten sind nicht vergleichbar; die maximale Optimierung muss dem Anwender wählbar bleiben. **Entscheidung:** Die Release-Flags gelten haus-weit mit O2 — einschließlich Vendor-Bau, FetchContent und Einbettung — per erzwungenem Cache-Wert; O3 wird nur auf XML-Eingangswunsch mitgebaut; die Bau-Welt-Stufe geht als Feld `vendoropt` in das Werkzeugketten-Glied des Fingerprints ein (Übersetzungsfehler statt stillem Default); ein Gate verwirft eigene Release-Optimierungsflags an der Achse vorbei; die Unter-Achse `opt_level` bietet O0 bis Ofast mit dem Determinismus-Flag nach IEEE 754 [IEE19], das allein Ofast verneint. **Konsequenz:** Zwei Baue derselben Permutation gegen eine O2- und eine O3-Welt sind verschiedene Artefakte mit verschiedenem Fingerprint; ein stiller Skip ist ausgeschlossen. **Umsetzungsstand:** O0 und O1 als Anwender-Wahl in der XML und die Durchreichung weiterer Compiler-Flags aus der XML sind vorgesehen, aber nicht implementiert (Default O2, O3 überschreibend wählbar). **Beleg:** Abschnitte 5.1.6, 4.5 und 3.4.2.

ADR-18 — Ein offizieller Bauweg als GNU-Vorbau über CMake

configure.sh, make, make check und make install.

Kontext: Das einzige unter Linux offizielle Verfahren ist `configure.sh`, `make` und `make install` beziehungsweise `make check`, und diese drei müssen im Wurzelordner liegen; eine Konfiguration darf nur eine Quelle haben. **Entscheidung:** `configure.sh` ist ein POSIX-sh-Vorbau ohne Python, gebaut wird weiterhin mit CMake; das Makefile ist statisch eingchecked und reicht nur an CMake weiter, mit den GNU-Standardzielen [Fre26] `all`, `install`, `check`, `installcheck`, `uninstall`, `clean`, `distclean`, `mostlyclean`

und `maintainer-clean` (das Test-Ziel heißt `check`); die Konfiguration ist ausschließlich in `config.status` und im CMake-Cache abgelegt, und das Makefile enthält bewusst keine Vorgabewerte — ein generiertes Makefile wäre eine zweite Konfigurationsquelle ohne Nutzen. **Konsequenz:** Es gibt keinen zweiten Bauweg neben CMake und keine zweite Konfigurationsquelle; der Vorbau ist mit `sh -n` prüfbar. **Umsetzungsstand:** `configure.sh` und `Makefile` liegen im Repositorium; der Framework-Export für Konsumenten (`install(EXPORT)`, `find_package`) und die Nachlauf-Prüfungen sind nicht implementiert. **Beleg:** Anhang B.

ADR-19 — Vier CI-Zellen ohne Skip

gcc und clang je Release und Debug; ein Job schlägt fehl oder besteht.

Kontext: Zwei Übersetzer finden verschiedene Fehler; ein stiller Job-Skip oder ein tolerierter Fehler täuscht einen bestandenen Lauf vor. **Entscheidung:** Die Test-Suite läuft in vier Zellen — gcc und clang jeweils als Release- und Debug-Bau — ohne Änderungs-Skip; die Zelle ist die Warnung, der Job schlägt fehl; `allow_failure` führt nur zwei deklarierte Ausnahmen — den `arm64-Smoke-Bau` ohne verfügbaren Runner und den manuellen Relock-Wartungsjob —, keine Bau- oder Test-Zelle; je Übersetzer und Bautyp gibt es einen Objekt-Cache-Schlüssel mit genau einem Schreiber; der Übersetzer-Mindeststand ist `g++ 15.3`; der golden-Referenzkatalog ²¹⁷ mit CRC-64-Prüfsumme ist der Regressions-Detektor mit festgeschriebener Kardinalität. **Konsequenz:** Akzeptiert wird nur ein Lauf ohne fehlgeschlagene Tests in allen vier Zellen ohne Ausnahmeliste — ein Gate im Sinne der Continuous Delivery [HF10] —, und eine Test-Zahl wird nur mit dem Lauf genannt, der sie erzeugt hat. **Umsetzungsstand:** Die Mini-Pipeline je Träger-Stufe (vier Stufen mal gcc und clang mal Release und Debug) ist vorgesehen, aber nicht implementiert. **Beleg:** Abschnitt 4.4.

ADR-20 — XML als einzige Steuerung

Die XML trägt Was, Wo und Wann, nie das Wie.

Kontext: Der Apparat wird aus einem deklarativen Experiment-Profil betrieben; Begriffe werden nicht übersetzt, sondern müssen konform sein — wäre eine Übersetzung nötig, ist das eine Regression und ein Übersetzungszeit-Fehler. **Entscheidung:** Zwei Schemata beschreiben Profil und Messreihe (`experiment_schema.xsd`, `messreihe_v32_schema.xsd`); die Parse-Naht bricht mit Fehlermeldung ab, die Validierung ist ein eigener Baustein, und die CI prüft die Wohlgeformtheit; die XML beschreibt Was, Wo und Wann, nie das Wie; die Achsen-Registries werden als XML generiert und sind damit maschinenlesbare Abbilder; der Planer ist ein Subkommando-Werkzeug (`validate`, `plan`, `cache-key`, `fingerprint`, `status`, `check-size`, `simulate`, `version`, `help`) und der Shell-Steuerpfad; als zweiter Steuerpfad ist eine API-Kommandozeile mit headless Dienst vorgesehen, der den Planer als Zelle auf Kubernetes oder auf bare metal bedient, sowie ein kommandozeilengeführter Erstellungsmodus für die Anwender-XML, in dem nicht verfügbare Optionen nicht wählbar sind. **Konsequenz:** Es gibt keinen Nebenweg neben der XML; die SOTA-Profile sind als XML ausführbar. **Umsetzungsstand:** Der headless Steuerpfad und der Erstellungsmodus sind nicht implementiert; die Hybrid-Sektion ist geparkt, aber im Schema nicht deklariert, die Organ-Meta-Meta-Sektion deklariert, aber nicht gelesen, und die `publish`-Sektion fehlt. **Beleg:** Abschnitte 3.4.6 und 4.3.2; Abbildung 4.7.

F Vergleichsinterfaces: `std::map` und `std::vector`

Dieser Anhang führt — ergänzend zu Abschnitt 3.4 — die vollständige Auflistung aller Standard-Interface-Funktionen der beiden Vergleichshüllen `std::map` (geordnet-assoziativ) und `std::vector` (Sequenz) nach dem Stand von C++23 mit ihrer offiziell erwarteten Semantik und ihrer Komplexitäts-Garantie gemäß dem ISO-C++-Standard [ISO24]. Er dient zugleich als zentrale Referenz für die Weiterentwicklung der Cache-Engine: Da die Suchalgorithmus-Hülle ihr Innenleben zur Übersetzungszeit austauscht — je Achsen-Wahl entsteht ein eigenes Kompilat —, während zur Laufzeit nur die Parameter der dynamischen Unter-Achsen nachgestellt werden (die registrierten dynamischen Dimensionen des Mess-Realms, Abschnitt 3.4), wird je Standard-Funktion erklärungsbedürftig, welches Verhalten der äußere Vertrag garantiert.

Variadische Abbildung. Die hybride Such-Engine des Prüflings PRT-ART bindet die Parameterzahl an die Hülle: Ein Template-Parameter fordert alle Funktionsinterfaces eines `std::vector`, zwei die eines `std::map` und $N > 2$ die einer `std::map<K, std::tuple<...>` (Abschnitt 4.3.1). Die Cache-Engine leitet aus derselben Regel die Typen ab — bei einem Parameter wird der Schlüssel implizit als monoton gezählter 64-Bit-Wert vergeben, bei $N > 2$ wird der Wert zum Tupel —; ihre beiden Vertrags-Hüllen sind als getrennte Gattungs-Adapter implementiert: die Gattung `SearchAlgorithm` (Konzept Map) über den funktionalen Antrieb der Modulgrenze mit den fünf Operationen `tier_insert`, `tier_lookup`, `tier_erase`, `tier_clear` und `tier_size`, die Sequenz-Gattung über den Sequenz-Antrieb mit `tier_push_back`, `tier_at`, `tier_size` und `tier_clear`; ein Adapter quer zur Gattung ist im Typsystem ausgeschlossen. Der Fall $N > 2$ ist Typ-Ableitung ohne eigenen Adapter; die Modulgrenze führt Schlüssel und Wert durchgängig als 64-Bit-Werte. Die geteilten Funktionen (Iteration, `size/empty/clear`, `swap`, `get_allocator`) besitzen in beiden Hüllen denselben Namen und dieselbe Rolle; über die Modulgrenze reichen beide Hüllen davon `size` und `clear`, `empty` ist host-seitig abgeleitet, Iteration und `swap` queren die Grenze nicht, und `get_allocator` ist auf der Map-Seite ein optionales Proxy-Sub-Interface außerhalb des Orakels (Abschnitt F.3).

Zusammengesetzte Funktionen. Einige Funktionen ergeben sich intern als Komposition primitiverer Operationen — etwa `operator[]` als `lower_bound` mit anschließendem Default-Einfügen bei fehlendem Schlüssel oder `at` als `find` mit Wurf einer `std::out_of_range`-Ausnahme bei Fehlen —; solche Fälle sind in der Spalte „Anmerkung“ gekennzeichnet. Die Spalte „seit“ nennt die einführende Standard-Fassung. Drei Begriffe der Tafeln stammen aus der jüngeren Standard-Entwicklung: der Node-Handle¹, die Deduktions-Leitfäden (CTAD)² und der transparente Schlüssel (Abschnitt F.1). Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*³) dokumentiert das Entwickler-

¹Seit C++17 kann ein Knoten eines assoziativen Containers mit `extract` ohne Kopie oder Move als Node-Handle gelöst und mit `insert` bzw. `merge` in einen anderen Container gespleißt werden [TWH16].

²*Class template argument deduction*, seit C++17: Die Template-Argumente eines Klassentemplates werden aus den Konstruktor-Argumenten abgeleitet, über implizit aus den Konstruktoren gebildete oder explizit angegebene Leitfäden [SVS16].

³Projektinterner Begriff der Cache-Engine für die Folge der internen Funktions-Sprünge (Handles) vom Eintritt an der Modulgrenze über die Achsen-Organen bis zum Ergebnis; das Entwickler-Dokument verzeichnet sie je Standard-Funktion und markiert nicht angebundene Funktionen als solche.

Dokument `docs/ENTWICKLER-FUNCTION-HANDLE-HOPS.md` der Cache-Engine; es verzeichnet je Standard-Funktion den Sprungpfad und den Status an der Modulgrenze.

Konformitäts-Teilmenge. Das Konformitäts-Gate (Abschnitt 3.4) prüft vor jeder Messung je Gattung eine Kern-Teilmenge der Hüllen-Operationen gegen ein Orakel der Standardbibliothek — `std::map` für die Gattung `SearchAlgorithm` (Konzept Map), `std::deque` für die Sequenz-Gattung —, damit nur Vergleichbares verglichen wird; die Teilmenge ist in beiden Tafeln mit † gekennzeichnet. Unter `-debug` fährt das Gate zusätzlich den Standard-Testsatz des Tier-Interfaces (generischer Gattungs-Teil und Genus-spezifischer Teil).

F.1 `std::map` (geordnet-assoziativ)

Tabelle F.1 listet die vollständige Schnittstelle von `std::map<Key, T, Compare, Allocator>` nach C++23. Ein † markiert die Kern-Operationen, die das Konformitäts-Gate (Abschnitt 3.4) vor jeder Messung prüft, damit nur Vergleichbares verglichen wird. Komplexitäten beziehen sich auf $n = \text{size}()$; bei Bereichs-Operationen ist N die Zahl der Eingabe-Elemente; Fassungs-Marker und Komplexitäten folgen dem Normtext [ISO24] und sind an der Online-Referenz [cpp26a] gegengelesen. Die siebzehn markierten Operationen sind genau die Operationsmenge der zwölf Kern-Prüfblöcke des Gates: Keine markierte Operation bleibt ungeprüft, keine geprüfte Operation ist unmarkiert (`size` wird in den Blöcken mitgeprüft, gehört aber zum Zufalls-Kernblock und ist deshalb nicht markiert). Vier der markierten Operationen entsprechen unmittelbar Operationen der Modulgrenze (`insert_or_assign` dem Einfüge-Antrieb, `find` der Punkt-Suche, `erase` und `clear` ihren Antrieben); `at`, `operator[]`, `count`, `contains`, `empty` sowie `insert`, `emplace` und `try_emplace` werden host-seitig komponiert — `empty` aus der Größe, die übrigen aus Punkt-Suche und Einfügen; die drei nicht überschreibenden Formen laufen über einen Pfad und werden gegen drei verschiedene Orakel-Aufrufe gestellt; `begin/end`, `lower_bound`, `upper_bound` und `equal_range` synthetisiert der Host über die additive Scan-Fähigkeit der Modulgrenze geordnet, und fehlt sie einer geladenen Binary, protokolliert das Gate diese Blöcke als übersprungen, nicht als bestanden; `key_comp` wird am Orakel festgestellt (Schlüsselordnung kleiner-als auf 64-Bit-Werten), weil die Hülle keinen Komparator über die Grenze trägt.

Tabelle F.1: Vollständige `std::map`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			
(Konstruktor)	leer; aus <code>Compare/Allocator</code> ; Bereich; Kopie/Move (auch allokator-erweitert); <code>initializer_list</code> ; ab C++23 <code>from_range</code>	$O(1)$ bis $O(N \log N)$	C++98
(Destruktor)	zerstört alle Elemente und Knoten	$O(n)$	C++98
<code>operator=</code>	Kopier-/Move- <code>/initializer_list</code> -Zuweisung mit Allokator-Propagation	$O(n)$	C++98

Funktion	Offizielle Semantik	Komplexität	seit
<code>get_allocator</code>	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
<code>at</code> †	Referenz auf den Mapped-Wert; wirft <code>std::out_of_range</code> bei Fehlen	$O(\log n)$	C++11
<code>operator[]</code> †	Referenz; fügt bei Fehlen wertinitialisiert ein (kein <code>std::out_of_range</code> ; das Einfügen kann <code>std::bad_alloc</code> oder Konstruktor-Ausnahmen werfen)	$O(\log n)$	C++98
Iteratoren			
<code>begin/end</code> †	Iterator auf erstes Element bzw. Ende, in Schlüsselordnung (<code>cbegin/cend</code> ab C++11)	$O(1)$	C++98
<code>rbegin/rend</code>	Rückwärts-Iteratoren (<code>crbegin/crend</code> ab C++11)	$O(1)$	C++98
Kapazität			
<code>empty</code> †	leer-Test (<code>begin() == end()</code>); <code>[[nodiscard]]</code> ab C++20	$O(1)$	C++98
<code>size</code>	Anzahl der Elemente	$O(1)$	C++98
<code>max_size</code>	theoretische Maximalgröße	$O(1)$	C++98
Modifikatoren			
<code>clear</code> †	alle Elemente entfernen	$O(n)$	C++98
<code>insert</code> †	Element(e) ohne Überschreiben (Wert, P&&, Hint, Bereich, <code>initializer_list</code> , Node-Handle ab C++17)	$O(\log n)$; Hint amort. $O(1)$; Bereich $O(N \log(n+N))$	C++98
<code>insert_range</code>	Eingabe-Bereich einfügen, keine Überschreibung	$O(N \log(n+N))$	C++23
<code>insert_or_assign</code> †	einfügen oder vorhandenen Wert zuweisen; <code>bool</code> meldet Einfügung	$O(\log n)$	C++17
<code>emplace</code> †	in-place konstruieren, nur falls Schlüssel fehlt (sonst verworfen)	$O(\log n)$	C++11
<code>emplace_hint</code>	wie <code>emplace</code> , positionsnah zum Hint	$O(\log n)$; amort. $O(1)$	C++11
<code>try_emplace</code> †	wie <code>emplace</code> , rührt die Argumente bei vorhandenem Schlüssel nicht an	$O(\log n)$	C++17
<code>erase</code> †	nach Iterator/Bereich/Schlüssel (Schlüssel: 0 oder 1 entfernt)	Iter. amort. $O(1)$; Key $O(\log n)$	C++98

Funktion	Offizielle Semantik	Komplexität	seit
<code>swap</code>	Inhalt und Comparator tauschen; Iteratoren bleiben gültig	$O(1)$	C++98
<code>extract</code>	Element als Node-Handle entnehmen (ohne Kopie/Move)	Iter. amort. $O(1)$; Key $O(\log n)$	C++17
<code>merge</code>	Knoten aus Quell-Map splicen; vorhandene Schlüssel bleiben dort	$O(N \log(n+N))$	C++17
Suche			
<code>find†</code>	Iterator auf den Schlüssel oder <code>end()</code>	$O(\log n)$	C++98
<code>count†</code>	Anzahl (0 oder 1)	$O(\log n)$	C++98
<code>contains†</code>	Mitgliedschaft (bool)	$O(\log n)$	C++20
<code>lower_bound†</code>	erster Schlüssel $\geq$ key	$O(\log n)$	C++98
<code>upper_bound†</code>	erster Schlüssel $>$ key	$O(\log n)$	C++98
<code>equal_range†</code>	Paar <code>[lower_bound,</code> <code>upper_bound)</code>	$O(\log n)$	C++98
Beobachter			
<code>key_comp†</code>	Kopie des Schlüssel-Comparators	$O(1)$	C++98
<code>value_comp</code>	<code>value_compare</code> (vergleicht Paare über den Schlüssel)	$O(1)$	C++98
Nicht-Member-Funktionen			
<code>operator==</code>	gleiche Größe und paarweise gleiche Elemente	$O(n)$	C++98
<code>operator<=></code>	lexikografischer Drei-Wege-Vergleich; synthetisiert <code><<=>=></code>	$O(n)$	C++20
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions- Leitfäden	CTAD aus Bereich/ <code>initializer_list</code> / <code>from_range</code> (<code>from_range</code> ab C++23)	— (Compile-Zeit)	C++17

Die vor C++20 eigenständigen Vergleichsoperatoren `<<=>=>` werden seit C++20 aus `operator<=>` synthetisiert, `!=` aus `operator==` umgeschrieben. Die heterogenen Überladungen von `operator[]`, `at`, `try_emplace` und `insert_or_assign` (transparenter Schlüssel⁴) gehören erst zu C++26 und bleiben hier ausgeklammert; die C++23-Neuzugänge sind der `from_range`-Konstruktor,

⁴Heterogene Suche seit C++14: Trägt der Komparator den Typ `is_transparent`, nehmen die Suchfunktionen Argumente

`insert_range`, die transparenten Schlüssel-Überladungen von `erase/extract` sowie die zugehörigen `from_range`-Deduktions-Leitfäden [JNC22, BVA21]. Auch die `constexpr`-Fähigkeit von `std::map` gehört erst zu C++26 und bleibt ausgeklammert [cpp26a].

F.2 `std::vector` (Sequenz)

Tabelle F.2 listet die vollständige Schnittstelle von `std::vector<T, Allocator>` nach C++23 ($n = \text{size}()$, bei Bereichs-Operationen N die Zahl der Eingabe-Elemente; Fassungs-Marker und Komplexitäten folgen dem Normtext [ISO24] und sind an der Online-Referenz [cpp26b] gegengelesen). Sämtliche Member sind seit C++20 `constexpr`⁵. Ein † markiert die Kern-Operationen, die das Konformitäts-Gate der Sequenz-Gattung vor jeder Messung gegen ein `std::deque`-Orakel prüft (Anhängen, Index-Zugriff, Größe, Leeren; für diese vier Operationen ist die `deque`-Semantik mit der des `std::vector` deckungsgleich): Die eigene Sequenz-Hülle bietet genau diese vier Funktionen über die Modulgrenze an, der Index-Zugriff ist dort geprüft statt undefiniert (Status-Rückgabe: Treffer-Flag und Ausgabewert an der Modulgrenze, `std::optional` im Organ). Alle übrigen Funktionen der Tafel — Kapazitäts-Steuerung, Positions- und Bereichs-Operationen, `front/back/pop_back`, `data`, Iteratoren, `swap` und die Nicht-Member — queren die Modulgrenze nicht; das Wachstum am vorderen Ende (`push_front`), das das `deque`-Orakel zusätzlich böte, ist als Erweiterung der Modulgrenze vorgesehen, aber nicht implementiert, und wird im Gatter als übersprungen protokolliert, nicht als bestanden.

Tabelle F.2: Vollständige `std::vector`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			
(Konstruktor)	leer/Allocator; $n \times$ Default/Wert; Bereich; Kopie/Move (auch allokator-erweitert); <code>initializer_list</code> ; ab C++23 <code>from_range</code>	$O(1)$ oder $O(n)$	C++98
(Destruktor)	zerstört alle Elemente, gibt Speicher frei	$O(n)$	C++98
<code>operator=</code>	Kopie/Move/ <code>initializer_list</code>	$O(n)$	C++98
<code>assign</code>	Inhalt durch $n \times$ Wert/Bereich/ <code>initializer_list</code> ersetzen	$O(n)$	C++98
<code>assign_range</code>	Inhalt durch Eingabe-Bereich ersetzen	$O(n)$	C++23
<code>get_allocator</code>	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			

eines anderen Typs als `key_type` an, ohne ein temporäres Schlüsselobjekt zu bilden [WLLM13]; C++23 dehnt dies auf `erase/extract` aus [BVA21], C++26 auf `operator[]`, `at`, `try_emplace` und `insert_or_assign` [BVA23].

⁵Seit C++20 sind alle Elementfunktionen von `std::vector` `constexpr`; ein `constexpr`-Objekt selbst scheitert in der Regel am dynamischen Speicher [Dio19].

Funktion	Offizielle Semantik	Komplexität	seit
<code>at†</code>	geprüfter Zugriff; wirft <code>std::out_of_range</code>	$O(1)$	C++98
<code>operator[]</code>	ungeprüfter Zugriff (UB außerhalb des Bereichs)	$O(1)$	C++98
<code>front/back</code>	Referenz auf erstes/letztes Element (UB bei leer)	$O(1)$	C++98
<code>data</code>	Zeiger auf den zusammenhängenden Speicher (nicht bei <code>vector<bool></code>)	$O(1)$	C++11
Iteratoren			
<code>begin/end</code> , <code>rbegin/rend</code>	Vorwärts-/Rückwärts-Iteratoren in Indexordnung (c...-Varianten ab C++11)	$O(1)$	C++98
Kapazität			
<code>empty/size†/max_size</code>	leer-Test, Elementanzahl, Maximalgröße	$O(1)$	C++98
<code>reserve</code>	Kapazität $\geq n$; realloziert und invalidiert; wirft <code>length_error</code> bei $n > \text{max_size}()$	$O(n)$	C++98
<code>capacity</code>	aktuell allozierte Kapazität	$O(1)$	C++98
<code>shrink_to_fit</code>	nicht-bindende Reduktion der Kapazität auf <code>size()</code>	$O(n)$	C++11
Modifikatoren			
<code>clear†</code>	alle Elemente entfernen (Kapazität bleibt)	$O(n)$	C++98
<code>insert</code>	vor Position: Wert/ $n \times$ /Bereich/ <code>initializer_list</code> ; verschiebt den Schwanz	$O(n)$; Bereichsformen $O(n+N)$	C++98
<code>insert_range</code>	Eingabe-Bereich vor Position einfügen	$O(n+N)$	C++23
<code>emplace</code>	Element in-place vor Position konstruieren	$O(n)$	C++11
<code>erase</code>	Position/Bereich entfernen, Schwanz nachrücken	$O(n)$	C++98
<code>push_back†</code>	Element anhängen (Kopie/Move)	amort. $O(1)$	C++98
<code>emplace_back</code>	in-place anhängen; Referenz-Rückgabe ab C++17	amort. $O(1)$	C++11
<code>append_range</code>	Eingabe-Bereich anhängen	amort. $O(N)$	C++23

Funktion	Offizielle Semantik	Komplexität	seit
<code>pop_back</code>	letztes Element entfernen (UB bei leer)	$O(1)$	C++98
<code>resize</code>	auf n kürzen oder mit Default/Wert auffüllen	$O(n)$	C++98
<code>swap</code>	Inhalt und Kapazität mit anderem Vektor tauschen	$O(1)$	C++98
Nicht-Member-Funktionen			
<code>operator==</code>	gleiche Größe und elementweise Gleichheit	$O(n)$	C++98
<code>operator<=></code>	lexikografischer Drei-Wege-Vergleich	$O(n)$	C++20
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase</code>	entfernt alle Elemente gleich dem Wert; gibt die Anzahl zurück	$O(n)$	C++20
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions-Leitfäden	CTAD aus Bereich/ <code>from_range</code> (<code>from_range</code> ab C++23)	— (Compile-Zeit)	C++17

Die Spezialisierung `std::vector<bool>` packt Wahrheitswerte bitweise: Sie besitzt kein `data()` und keine Zusammenhangs-Garantie, liefert über `operator[]/at/front/back` einen Proxy-Typ⁶ `reference` statt `bool&` und bietet zusätzlich `flip()` (auf Vektor- und auf `reference`-Ebene), ein statisches `swap(reference, reference)` sowie eine `std::hash`-Spezialisierung. Die Vergleichsoperatoren verhalten sich wie bei `std::map`.

F.3 Zusammengesetzte Funktionen und Konformitäts-Teilmenge

Mehrere Schnittstellen-Funktionen sind intern Kompositionen primitiverer Operationen derselben Schnittstelle (Tabelle F.3); die Tabelle beschreibt den ISO-Standardvertrag. Die eigene Mess-Hülle weicht an zwei Stellen bewusst davon ab, weil der ABI-Konformitätsvertrag `noexcept` verlangt und eine werfende Hülle als nicht-konform gilt: `at` liefert statt der Ausnahme eine Status-Rückgabe (Treffer-Flag und Ausgabewert an der Modulgrenze, `std::optional` im Organ), und `operator[]` ist an der Modulgrenze kein eigener Aufruf, sondern wird host-seitig aus Punkt-Suche und Default-Einfügen komponiert — mit der ISO-Semantik (Einfügen bei Fehlen, kein `out_of_range`), aber ohne Referenz-Rückgabe über die binäre Grenze; der Schreibpfad läuft über `insert_or_assign`. In der Sequenz-Hülle ist `operator[]` zudem geprüft statt undefiniert (Abschnitt F.2). Für diese Arbeit ist das die zentrale Beobachtung: Sämtli-

⁶Stellvertreter-Objekt: `vector<bool>::reference` verhält sich im Ausdruck wie `bool&`, bezeichnet aber ein einzelnes Bit statt eines adressierbaren Objekts, weshalb `data()` und die Zusammenhangs-Garantie entfallen [ISO24].

che Map-Zugriffe und -Modifikatoren der Hülle ruhen auf einem einzigen Such-Primitiv der Modulgrenze — der Punkt-Suche, flankiert von Einfügen und Löschen —; eine Achsen-Implementierung muss daher nur eine korrekte Punkt-Suche samt Knoten-Verkettung bereitstellen, der Rest ergibt sich als feste Komposition. Die geordneten Operationen — `begin/end`, `lower_bound`, `upper_bound`, `equal_range` — setzen zusätzlich eine Ordnungs-Suche voraus und sind deshalb als eigene, additive Fähigkeit der Modulgrenze geführt; ungeordnete Achsen erfüllen dieselbe Hülle ohne sie und bezahlen die fehlende Ordnung beim geordneten Lesen mit linearem Aufwand. Genau diese Austauschbarkeit ist der Gegenstand der Messung (Abschnitt 3.4). Das Konformitäts-Gate prüft je Gattung die in den Tabellen F.1 und F.2 mit † markierte Kern-Teilmenge.

Tabelle F.3: Zusammengesetzte Funktionen (interne Komposition primitiver Operationen)

Funktion	Zusammensetzung	Anmerkung
<code>map::operator[]</code>	<code>lower_bound</code> + Default-Einfügen bei Fehlen → Referenz	mutiert bei Fehlen; kein <code>out_of_range</code> , das Einfügen kann werfen
<code>map::at</code>	<code>find</code> + Wurf <code>std::out_of_range</code> bei Fehlen	einzigster Zugriff mit Wurf bei Fehlen; mutiert nie
<code>map::insert / emplace</code>	Ordnungs-Suche + Knoten verlinken nur bei fehlendem Schlüssel	kein Überschreiben; <code>emplace</code> kann Argumente bei Treffer dennoch konsumieren
<code>map::try_ emplace</code>	wie <code>emplace</code> , aber Argumente bei Treffer unangetastet	bewahrt move-only- Argumente
<code>map::insert_ or_assign</code>	Ordnungs-Suche + Einfügen oder Zuweisen	überschreibt (anders als <code>insert</code>)
<code>map::count / contains / equal_range</code>	reduzieren auf <code>find</code> bzw. <code>lower_bound+upper_bound</code>	bei Unikat-Map stets 0 oder 1
<code>vector:: push_back / emplace_back</code>	ggf. Wachstum (Realloc + Umzug) + Konstruktion am Ende	versteckte Kosten + Invalidierung beim Realloc
<code>vector::insert / emplace</code>	ggf. Wachstum + Schwanz verschieben + Einsetzen	zusätzlicher $O(n)$ -Shift
<code>vector::resize</code>	anhängen (Default/Wert) oder Schwanz kürzen	bidirektional; verkleinert die Kapazität nicht

Funktion	Zusammensetzung	Anmerkung
<code>vector::assign</code>	<code>clear</code> + Bereich/ $n \times$ einfügen	Kapazität wird ggf. wiederverwendet

An der Modulgrenze ist die Primitivitäts-Ordnung dieser Tafel umgekehrt: Der eine Einfüge-Antrieb der Gattung `SearchAlgorithm` (Konzept `Map`) trägt die Semantik von `insert_or_assign` — Einfügen oder Aktualisieren, die Rückgabe meldet, ob ein neuer Schlüssel entstand —, sodass `insert_or_assign` die Durchreichung ist und die nicht überschreibenden Formen `insert`, `emplace` und `try_emplace` host-seitig als Punkt-Suche mit anschließendem Einfügen nur bei fehlendem Schlüssel komponiert werden: ein Pfad für drei Standard-Funktionen, den das Gate gegen drei Orakel-Aufrufe stellt. Der ISO-Vertrag führt es umgekehrt (`insert` als Grundform, `insert_or_assign` als Zusammensetzung); beide Sichten sind korrekt und beschreiben verschiedene Ebenen — genau der Kontrast, der die Austauschbarkeit hinter einer festen Schnittstelle konkret macht. Funktionen, die beide Hüllen dem Namen nach *teilen* — `size`, `empty`, `clear`, die Iterator-Familien, `swap`, `get_allocator` sowie `insert/emplace/erase` —, besitzen dieselbe Rolle, aber verschiedene Bezugsgrößen: in der `Map` Schlüssel-, im Vektor Positions-/Index-Semantik; `operator[]` etwa fügt in der `Map` ein, ist im Vektor hingegen ein ungeprüfter Indexzugriff — in der eigenen Sequenz-Hülle geprüft (Abschnitt F.2), in der eigenen `Map`-Hülle host-seitig komponiert (siehe oben). Diese Trennung ist in der Cache-Engine strukturell erzwungen: `Map`- und Sequenz-Gattung besitzen getrennte Antriebs-Schnittstellen und getrennte Orakel, ein gattungsfremder Adapter ist im Typsystem ausgeschlossen. Welche dieser Funktionen die Modulgrenze bereitstellt, welche der Host komponiert und welche nicht angebunden sind, verzeichnet das eingangs genannte Entwickler-Dokument je Funktion mit ihrem Sprungpfad.

Danksagung

Mein Dank gilt zuerst apl. Prof. Dr.-Ing. Dirk Habich — einem weisen, freundlichen und disziplinierten Menschen — für die engagierte Betreuung dieser Arbeit: Trotz stets knapper Ressourcen an Zeit hat er sich dennoch stets die Zeit für präzise Rückfragen genommen, um klare Prioritäten und den Freiraum, was diese Arbeit weitgehend eigenständig entwickeln soll, mit punktueller Unterstützung dort, wo sie gebraucht wurde, zu unterstützen. Dr.-Ing. Alexander Krause danke ich für die Begleitung dieser Arbeit als zweiter Betreuer. Schließlich danke ich dem Team von UltiHash: Die lehrreichen Jahre in diesem Startup haben viele der Erfahrungen und Fragestellungen geprägt, die diese Arbeit nun technisch umsetzt.

Erklärungen zum Urheberrecht

Sperrvermerk (eingeschränkt)

Diese Arbeit selbst unterliegt keinem Sperrvermerk. Ein eingeschränkter Sperrvermerk betrifft den Software-Kern der `comdare-cache-engine` und den Prüfling PRT-ART: Beide sind zur Begutachtung einsehbar und testbar; eine geschäftliche Nutzung, private Einzelnutzung oder Weitergabe über die Begutachtung hinaus bedarf jeweils einer gesonderten Lizenz der Rechteinhaberin, der BEP Venture UG (haftungsbeschränkt), der Inhaberin der eingetragenen Marke Comdare. Ausgenommen ist die Forschung: Für Forschungszwecke an einer Einrichtung sind beide frei nutzbar. Die in dieser Arbeit beschriebenen Experimente verwenden die Cache-Engine als Anwender über frei wählbare Testparameter und sind von der Einschränkung nicht berührt.

Lizenzen und Copyright

Der Code des Diplomarbeits-Repositories (Mess-Treiber und Auswertungs-Pipeline) steht unter Apache-2.0; eingebundene Submodule und Fremdkomponenten behalten ihre jeweils eigene Lizenz. Der Software-Kern der `comdare-cache-engine` und der Prüfling PRT-ART stehen unter der Comdare Research License 1.0: Der Quelltext ist für alle einsehbar; Forschung an einer Einrichtung darf frei nutzen; geschäftliche Nutzung und private Einzelnutzung bedürfen jeweils einer gesonderten schriftlichen Lizenz der Rechteinhaberin. Für die `comdare-cache-engine` gilt das Change Date 2031-08-10: Ab diesem Tag steht sie allen unter der Apache License 2.0 zur Verfügung. Früher veröffentlichte Revisionen behalten ihre damalige Lizenz unwiderruflich (`comdare-cache-engine`: Apache-2.0 vor dem 02.08.2026, duales Forschungs-/Kommerz-Modell vom 02.08. bis 09.08.2026; PRT-ART: Apache-2.0 vom 02.08. bis 09.08.2026). Die Rechte an beiden Software-Kernen liegen bei der BEP Venture UG (haftungsbeschränkt), Marke Comdare. Der Text dieser Arbeit ist Copyright Benjamin-Elias Probst und steht unter der Lizenz CC BY 4.0 (Creative Commons Namensnennung 4.0 International).

Drittmaterial

Wörtliche und sinngemäße Übernahmen aus fremden Werken sind als Zitate gekennzeichnet und im Literaturverzeichnis nachgewiesen; sie bleiben Eigentum der jeweiligen Urheber und sind von der Lizenz dieser Arbeit nicht erfasst. Alle Abbildungen und Tabellen dieser Arbeit wurden vom Autor aus den eigenen Messdaten und Bauplänen erzeugt (TikZ/pgfplots bzw. Generator-Ausgaben der Auswertungs-Pipeline); fremde Grafiken sind nicht enthalten. Das Logo auf der Titelseite ist ein Kennzeichen der Technischen Universität Dresden (ZIH), wird nach deren Vorgaben für Abschlussarbeiten verwendet und ist von der CC BY-Lizenz ausgenommen. Benchmark-Datensätze Dritter (etwa die Datensätze des SOSD-Benchmarks) sind nicht Bestandteil dieser Arbeit; sie werden nur zitiert oder als Eingabe über den

Datensatz-Lader verwendet, ihre Nutzung richtet sich nach den Bedingungen ihrer Herausgeber. In der `comdare-cache-engine` vendorierter Fremdcode (Verzeichnis `ext/`) ist nicht Teil der Software im Sinne der Comdare Research License und behält seine jeweilige Originallizenz; die vollständige Zuordnung steht in den Dateien `NOTICE`, `REUSE.toml` und `LICENSE_AUDIT_EXT.md` des Repositoriums. *Comdare* ist eine eingetragene Marke der Rechteinhaberin; die Nennung in dieser Arbeit begründet keine Markenrechte Dritter.